

REPARTO DE PROBLEMAS

Marca qui n ya lo leyo. En Tematica anota de que se trata; en Dificultad, que tan duro se ve y si vale la pena ahora.

Problema	Juan	Felipe	Reyes	Tematica	Dificultad
A	☐	☐	☐		
B	☐	☐	☐		
C	☐	☐	☐		
D	☐	☐	☐		
E	☐	☐	☐		
F	☐	☐	☐		
G	☐	☐	☐		
H	☐	☐	☐		
I	☐	☐	☐		
J	☐	☐	☐		
K	☐	☐	☐		
L	☐	☐	☐		
M	☐	☐	☐		
N	☐	☐	☐		

GATILLOS - del enunciado a la tecnica

Para cuando el problema no te suena a nada. Busca la senal, quedate con el nombre de la tecnica y de ahi usa el indice. El numero de la derecha es la seccion.

Cuanto aguanta cada complejidad (~10^8 operaciones en 1 s)

n <= 11	O(n!) - permutaciones, next_permutation	
n <= 20	O(2^n * n) - bitmask DP	2
n <= 25	O(2^n) - si no cabe, meet in the middle	
n <= 40	meet in the middle, O(2^(n/2))	
n <= 450	O(n^3) - Floyd, DP de intervalos	3
n <= 5.000	O(n^2 log n)	
n <= 10.000	O(n^2) - DP de dos indices	2
n <= 100.000	O(n sqrt n) - Mo, bloques	1
n <= 200.000	O(n log^2 n) - binaria + estructura	
n <= 1.000.000	O(n log n) - sort, Fenwick, Dijkstra	1
n <= 100.000.000	O(n) - dos punteros, criba, prefijos	
n hasta 10^18	formula cerrada, log, o binaria	5

Lo que dice el enunciado -> la tecnica

maximizar el minimo / minimizar el maximo	binaria sobre la respuesta	11
cuantas formas hay, modulo 10^9+7	DP, o combinatoria con inverso modular	7
el k-esimo mas grande / mas pequeno	binaria + contar, o heap de tamano k	11
minimo numero de movimientos o pasos	BFS (todas las aristas pesan 1)	3
los pesos son solo 0 y 1	BFS 0-1 con deque	3
pesos cualesquiera, ninguno negativo	Dijkstra con heap	3
puede haber pesos negativos	Bellman-Ford	3
distancia entre TODOS los pares	Floyd-Warshall (n <= 450)	3
conectar todo al menor costo	MST: Kruskal + DSU	3
quedaron en el mismo grupo?	DSU (union-find)	3
hay ciclo en un grafo dirigido?	Kahn, o DFS de tres colores	3
dependencias, en que orden hacerlo	orden topologico	3
dos nodos se alcanzan mutuamente	SCC (Kosaraju o Tarjan)	3
si quito esta arista se desconecta?	puentes y puntos de articulacion	3
emparejar dos grupos, uno a uno	Kuhn, o flujo maximo	3
separar / cortar al menor costo	min cut = max flow (Dinic)	3
recorrer cada arista exactamente una vez	camino o circuito euleriano	3
ancestro comun, o distancia en un arbol	LCA con binary lifting	3
aplicar el mismo salto k veces	binary lifting en grafo funcional	3
a lo sumo k de algo, en un tramo	ventana deslizante	13
subarreglo de suma maxima	Kadane	2
consultas de rango y el arreglo NO cambia	prefijos, o sparse table para min/max	1
consultas de rango y SI cambia	Fenwick si es suma, segtree si no	1
sumar un valor a todo un rango	segment tree con lazy propagation	1
contar pares desordenados	inversiones: merge sort o Fenwick	1
el primer mayor (o menor) a la derecha	pila monotona	1
existe una asignacion verdadero/falso?	2-SAT	3
cuantos numeros entre A y B cumplen X	digit DP	2
subsecuencia comun pero por pedazos seguidos	LCS con bloques de K: dos tablas	2
palindromos, todos los centros	Manacher	9
donde aparece este patron	KMP, o funcion Z	9
muchas comparaciones de subcadenas	hashing doble	9
prefijos de un diccionario	trie	9
MUCHOS patrones contra un mismo texto	Aho-Corasick (trie + suffix links)	9
preguntas sobre TODAS las subcadenas	suffix automaton	9
la k-esima subcadena en orden alfabetico	bajar por el suffix automaton con un dp	9
valores gigantes pero pocos distintos	compresion de coordenadas	1
fechas dadas como dia, mes y ano	calendario a mano; OJO el ano bisiesto	13
partir un total en partes exactamente iguales	el tamano tiene que DIVIDIR el total	5
subsecuencia creciente mas larga	LIS en O(n log n) con lower_bound	2

Lo que dice el enunciado -> la tecnica (cont.)

mochila, escoger con un tope	DP de mochila	2
dos juegan optimo, quien gana	Grundy / Nim, o DP de estados	8
area, contener, o cruzarse en el plano	primitivas: cross, dot, orientacion	4
el poligono mas chico que cubre los puntos	convex hull	4
cuantos divisores, o factorizar muchos	criba con menor factor primo	6
dividir en modulo	inverso modular (Fermat)	6
congruencias simultaneas	teorema chino del resto	6

Trampas que cuestan el problema entero

dos numeros de 10^9 multiplicados	se sale de int: long long siempre	14
tres numeros de 10^9 multiplicados	se sale de long long: _int128	14
Fenwick con indice 0	es 1-indexado: i & -i con i=0 no avanza	1
el enunciado pide el orden de ENTRADA	no ordenes ni uses set: guarda el indice	
hay empates	ordena por (clave, indice) y define el criterio	
aparecen decimales	nunca ==, usa eps; mejor aun, pasa a enteros	14
el grafo puede venir desconectado	el DFS va en un for por TODOS los nodos	3
n = 0 y n = 1	el borde que siempre se olvida	
la respuesta cabe pero el intermedio no	reordena la formula o divide antes	14
recursion profunda en Python	sys.setrecursionlimit, o hazlo iterativo	

Secciones del notebook

Data Structures	23 entradas	1
Dynamic Programming	17 entradas	2
Graph	36 entradas	3
Geometry	15 entradas	4
Math	24 entradas	5
Number Theory	23 entradas	6
Combinatorics	16 entradas	7
Game Theory	4 entradas	8
String	17 entradas	9
Greedy	22 entradas	10
Binary Search	3 entradas	11
Constructive Algorithms	2 entradas	12
Implementation	28 entradas	13
Formulario	16 entradas	14
Divide and Conquer	1 entrada	15
CSes: Graph Algorithms	21 entradas	16
CSes: Introductory Problems	24 entradas	17
CSes: Range Queries	25 entradas	18
CSes: String Algorithms	21 entradas	19
Hub LeetCode (referencia, del equipo)	188 entradas, secciones 20-41	

XXXIX MARATON NACIONAL DE PROGRAMACION ACIS/REDIS 2026

----- NOTEBOOK -----

Universidad de Ibague

Participantes:

Juan David Reyes Cure
Juan Felipe Rodriguez Barbosa
Juan Esteban Rodriguez Castellanos

October 01, 2026

Entrenador:

Stiven Correa

Contents

1 Data Structures

1.1 Brute Force - Count Majority Subarrays	11
1.2 Compresion de Coordenadas	12
1.3 Conteo de Inversiones con Fenwick - Frosh	13
1.4 Conteo de Inversiones y Redondeo Exacto - Sky Line	13
1.5 Fenwick Tree (Consultas offline) - D Query	14
1.6 Fenwick Tree (Offline Queries) - K Query	14
1.7 Fenwick Tree (Prefix Minimum, Dominancia 3D) - The Day Of The Competitors	15
1.8 Hash Map y Ordenamiento	16
1.9 Hash Set - Dangerous Dive	16
1.10 Heap con Borrado Perezoso - Aquarium Bubble Lights	17
1.11 Mo's Algorithm - Wine Production	17
1.12 Ordered Set (PBDS)	18
1.13 Persistencia (DFS + Rollback) - The Declaration Of Independence	18
1.14 Prefix Sums - Count Majority Subarrays2	19
1.15 Prefix Sums con Primera Aparicion - Accounting Qualifying Con Prefix	19
1.16 Segment Tree - Block Placement Queries	20
1.17 Segment Tree - Monarchy In Vertigo	21
1.18 Segment Tree - Running A Penitentiary	21
1.19 Segment Tree sobre Coordenadas Comprimidas - Emergency Rations	22
1.20 Small-to-Large Merging (DSU on Tree) - Distinct Colors	23
1.21 Sparse Table sobre Bloques Iguales - Frequent	24
1.22 Sqrt Decomposition - Holes	25
1.23 Tree Construction - Binary Tree	26

2 Dynamic Programming

2.1 Binary Search - Patches	26
2.2 Bitmask DP - Music Festival	26
2.3 Bitmask SOS DP - El Juego Del Calamar	27
2.4 Camino en Grilla con Balance de Parentesis - Has Valid Path	27
2.5 Convex Hull Trick	28

2.6 Digit DP	29
2.7 Distancia de Damerau-Levenshtein	29
2.8 Divide and Conquer Optimization	30
2.9 DP con Arreglo de Diferencias - Balanced Balloons	30
2.10 DP sobre Intervalos con Bandera de Inversion - Artic Virus	31
2.11 Interval Scheduling - Yuyuan Market	32
2.12 LCS con Bloques de K Caracteres	32
2.13 Prefix Sums - Zig Zag Array	33
2.14 SOS (Sum over Subsets) + Inclusion-Exclusion - How Many Teams	33
2.15 Steiner Tree - Joining Capitals	34
2.16 Subset Sum (Knapsack) - Karamell	34
2.17 Valor Esperado - Revenge Of The Salary Of At Coder Inc	35

3 Graph

3.1 2-SAT (Kosaraju) - Signal Coverage	35
3.2 Articulation Points (Tarjan) - Submerging Islands	37
3.3 BFS - Ladder Length	38
3.4 BFS - Boss	38
3.5 BFS - Bacon Number	39
3.6 BFS Backtracking - Find Ladders	40
3.7 BFS sobre Grafo Producto (dos fichas a la vez) - Infiltration Route	41
3.8 Biconnected Components (Block-Cut Tree) + LCA - Hedge Mazes	43
3.9 Bipartite Matching - Attacking Rooks	45
3.10 Bipartite Matching - Knights In The Board	45
3.11 Bipartite Matching (Algoritmo de Kuhn) - School Dance	46
3.12 Circuito Euleriano (Algoritmo de Hierholzer) - Mail Delivery	46
3.13 Clique Maximo (Bron-Kerbosch con Pivote) - Holy Network	47
3.14 Clique Maximo con Backtracking Iterativo - Holy Network (clique Iterativo)	48
3.15 Componentes Fuente con Kosaraju - Lumina Juan	49
3.16 Componentes Fuente de la Condensacion (Tarjan) - Lumina	50
3.17 Componentes Fuertemente Conexas (Kosaraju) - Planets And Kingdoms	51

Contents (cont.)

3.18 DFS - Artwork	51	5.7 Divisores de la Suma Total - Puntos Equiespaciados En Un Circulo	86
3.19 DFS - Denouncing Mafia	52	5.8 Eliminacion Gaussiana (mod p)	87
3.20 DFS - A Mazing!	52	5.9 Fibonacci (Fast Doubling) - Fibonacci Sum	88
3.21 Dijkstra - Fantastic Robot	53	5.10 Fibonacci (Zeckendorf-like Greedy) - Getting In Shape	88
3.22 Dijkstra con Nodos Virtuales - Chinese Innovation	54	5.11 Fibonacci Fast Doubling - Pou	89
3.23 DSU (enfoque INCORRECTO para este problema) - Holy Network (DSU, Incorrecta)	55	5.12 Greedy - A Healthy Menu	89
3.24 Existencia de Camino Euleriano con Aristas Ocultas - Apple Pie	56	5.13 Implementation - Hour For A Run	89
3.25 Heavy-Light Decomposition - Los Ratones III	58	5.14 Logaritmos - Gatuno's Descent Into Psychopathy	89
3.26 Kruskal Reconstruction Tree - Trucks	59	5.15 Matrix Exponentiation - Buses	90
3.27 LCA (Binary Lifting) - Subway Lines	60	5.16 Numeros Romanos	90
3.28 Max Flow - Download Speed	61	5.17 Optimization - Satoru Gojo	90
3.29 Max Flow - March Of The Penguins	62	5.18 Paridad - Call	91
3.30 Max Flow (Binary Search + Dinic) - Gasoline	63	5.19 Polynomial Composition - Coach	91
3.31 MST con Prim (Grafo Denso) - Brute	64	5.20 Prefix Sums - Triangles	91
3.32 Quitar el DAG de Rutas Minimias - Almost Shortest Path	65	5.21 Recurrencia (Fibonacci) - Fractions Are Better When Continued	92
3.33 State-Space BFS - Folding Machine	66	5.22 Recurrencias y busqueda de intervalo factible - Investigating Quadradômeda	92
3.34 Strongly Connected Components (Tarjan) - Proving Equivalences	66	5.23 Recursion (Simulacion tipo Josephus) - Metal Detector	93
3.35 Tree Degree Analysis - Colmillos	67	5.24 Simulacion - Cahokia Ruins	93
3.36 Tree Reconstruction (BFS por capas) - Expansion Of The Road Network	68		
4 Geometry		6 Number Theory	
4.1 2D Difference Array (Coordinate Compression) - Office Space	69	6.1 BFS sobre Residuos - Only1s0s	93
4.2 Area Circulo-Poligono	70	6.2 Big Exponentiation - Uj (cpp)	94
4.3 Bounding Box de una Elipse Rotada - Eclipse	71	6.3 Big Exponentiation - Uj (java)	94
4.4 Contar Cruces de Cuerdas con Fenwick - Lonely Creatures	71	6.4 Criba de Eratostenes (Solo Impares)	95
4.5 Convex Hull y Barrido Angular - Guard Deployment	73	6.5 Criba de Factor Primo Minimo + Inclusion-Exclusion - Ingredients That May Harm You	95
4.6 Coordenadas Polares	76	6.6 Diferencia de Cuadrados - Square Diff	96
4.7 Geometria 1D - Kilometric Intersection	76	6.7 Divisibilidad - Archmage's Crystals	96
4.8 K Vecinos Mutuos y Clasificacion de Componentes - Celestial Veins	77	6.8 Divisors - Aesthetics In Poetry	96
4.9 K-Means (Clustering)	78	6.9 Exponenciacion Modular (Torre de Exponentes) - Exponentiation II	97
4.10 Primitivas y Convex Hull	79	6.10 Factorizacion de un Numero Gigante Redondeado - Fuzzy Factorization	97
4.11 Probabilidad de Circulo Contenido - Omens	80	6.11 Factorizacion en Digitos	98
4.12 Punto en Poligono Convexo ($O(\log n)$)	80	6.12 GCD and LCM - Journeygrapes	98
4.13 Rotating Calipers (Par Mas Lejano)	81	6.13 Modular Arithmetic - Broul	98
4.14 Shoelace (Area de Poligono)	82	6.14 Modular Arithmetic - Creating Multiples	99
4.15 Triangle Classification - AROD	83	6.15 Modulo de Entero Gigante	99
		6.16 Modulo de un Binario Gigante - Binary Dozens	99
5 Math		6.17 Modulo de un Binario Gigante (Python) - Binary Dozens	100
5.1 2-Coloreo por Suma Potencia de 2 - Drug Test	84	6.18 Number Theory (Criba y Factorizacion) - Counting Divisors	100
5.2 Ad-hoc - Attention To Meeting	84	6.19 Number Theory (GCD y Divisores) - The Seals Of Shanghai	100
5.3 Algebra - The Wisdom Of Master Wei	84	6.20 Number Theory (LCM via Factorizacion de Ciclos) - Permutation Rounds	101
5.4 Bit Manipulation - Meme Wars	85	6.21 Reconstruccion con Divisores - Remainders	102
5.5 Brute Force - Antiarithmetic	85	6.22 Sieve of Eratosthenes - Prime Sieve	102
5.6 Construction - Ezraft And Array	85	6.23 Ternas Pitagoricas - Pythagorean	103

Contents (cont.)

7 Combinatorics

7.1 Combinatoria (Factoriales Modulares) - Binomial Coefficients	103
7.2 Combinatoria Avanzada (Lucas, Catalan, Stirling)	104
7.3 Construction - Farmpiggy And Subset Sum	104
7.4 Conteo de Distintos con Inventario de Frecuencias - Total Numbers	105
7.5 Conteo de Inversiones (cadena repetida) - Inversions	105
7.6 Euler's Formula - Pizza Cutter	106
7.7 Formula Cerrada con Inverso Modular - Number Of Sets	106
7.8 Paridad de Binomiales (Kummer) - Less Coint Tosses	107
7.9 Permutation Cycles - Lines Of Containers	107
7.10 Probabilidad (Paradoja del cumpleaños) - Mount Rushmore And Birthdays	108
7.11 Probabilidad (Valor esperado con inverso modular) - Frangolino Ali Na Mesa	109
7.12 Ranking - Palindromic Lexi Small	109
7.13 Ranking de Combinaciones - Discrete Catalog	110
7.14 Suma de Subconjuntos (2^n) - Group Work	110
7.15 Tecnica de Contribucion - Bytelandia's Stones	110
7.16 Triangulos en Poligono Convexo	111

8 Game Theory

8.1 Misere Nim y Grundy Acotado	111
8.2 Nim con Segment Tree - Juan Vs Frank	112
8.3 Posiciones Perdedoras (potencias de 2) - Digitsduel	112
8.4 Sprague-Grundy - Marbles	112

9 String

9.1 Brute Force Matching - Enigma	113
9.2 Chequeo de Palindromo	113
9.3 KMP para Contar Ocurrencias Solapadas - Judgmental Crowd	114
9.4 KMP sobre Cadena Recursiva (Duplicacion) - Song Of The Sirens	115
9.5 Maximo Solape Sufijo-Prefijo (KMP) - Sign	116
9.6 Monotonic Stack - Remove Duplicates1	117
9.7 Palindromo Ignorando Simbolos - Just Palindromes	117
9.8 Palindromos No Solapados (Greedy) - Max Palindromes	118
9.9 Parentesis Balanceados (Todas las Variantes)	118
9.10 Partition and Permutation - Policianacional	120
9.11 Remove Duplicate Letters - Remove Duplicates	121
9.12 Rotacion Minima (Dos Punteros) - Fingerprints	121
9.13 Simulation - Process Str	122
9.14 Suffix Array + LCP - Couple Of Bip Bop	122
9.15 Suffix Automaton - Beautiful Words	123
9.16 Trie + DP de Segmentacion (Maximo) - Spacebar Tokenizer	124
9.17 Z-Function - A String Problem	125

10 Greedy

10.1 Binary Search - Maratona Brasileira De Popcorn	126
---	-----

10.2 Bit Manipulation - Lecographically Maximum	126
10.3 Budget Optimization - Posters	127
10.4 Buy Low Sell High (Comprar y Vender Acciones)	127
10.5 Constructive Algorithms - Nomad	128
10.6 DP - Exhibition Of Clownfish	128
10.7 Emparejar Extremos - Axles	128
10.8 Huffman	129
10.9 Huffman sobre Frecuencias Variables en el Tiempo - Efficient Encoding	129
10.10 Interval Scheduling - Interval Scheduling Greedy	130
10.11 Maximo Conjunto a Distancia d en Arbol - Oficinas A Distancia D En Arbol	130
10.12 Prefijos sobre Secuencias Balanceadas - Incompatible Pairs	131
10.13 Prefix Sums - Maximizing Advertising	132
10.14 Puente y Linterna - The Bridge At Night	132
10.15 Puente y Linterna (version con indices) - Bridge Juan	133
10.16 Segment Counting - Cerros	133
10.17 Simulation - Kathmandu	133
10.18 Sliding Window	134
10.19 Sorted Pairing - Balatro	134
10.20 Sorting - Finding Forks	134
10.21 Sorting (Prefix Sums) - Coven Complications	134
10.22 Two Pointers (Container With Most Water)	135

11 Binary Search

11.1 Busqueda sobre la Respuesta con Fracciones Exactas - Clean Streets	135
11.2 Greedy (busqueda binaria sobre la respuesta) - Minas Gerais' Walls	137
11.3 Two Pointers - Black Hills Golden Jewels	137

12 Constructive Algorithms

12.1 Hashing - Handling The Blocks	138
12.2 Secuencias de Langford - Doubled Sequence II	138

13 Implementation

13.1 Ad Hoc - Toilet Seat	139
13.2 Ad Hoc - Rock Paper Scissors Tournament	139
13.3 Ad Hoc - Zero Or One	140
13.4 Ad Hoc - Collatz Polynomial	140
13.5 Ad Hoc - Unraveling Monty Hall	140
13.6 Arreglos - Buffoon	140
13.7 Backtracking (Enumeracion de bits desconocidos) - Harmonics With Interference	141
13.8 Brute Force - Cake Hater	141
13.9 Brute Force - LLMs	141
13.10 Circular Queue - Eeny Meeny	142
13.11 Clasificacion de Cadenas por Prefijo - Joker Or Judger	143
13.12 Conjuntos - João João	143
13.13 Constructive - A Story About Tea	143

Contents (cont.)

13.14	Conteo de Records - Harder Horizons	144	16.8	Game Routes - DP sobre DAG en Orden Topologico	167
13.15	Cycle Detection - Switches	144	16.9	High Score - Bellman-Ford (Camino Maximo)	168
13.16	Date Parsing - Crochet	145	16.10	Investigation - Dijkstra con Cuatro Cantidades a la Vez	169
13.17	Dia Siguiente y Anos Bisiestos	145	16.11	Labyrinth - BFS con Reconstruccion de Camino	169
13.18	Fuerza Bruta $O(n^2)$ sobre Subperiodos - Accounting Qualifying	146	16.12	Longest Flight Route - Camino Mas Largo en DAG (DFS con Memo)	171
13.19	Generacion de Numeros con Digitos Distintos - Division Digitos Distintos	146	16.13	Message Route - BFS de Camino Minimo con Reconstruccion	171
13.20	Greedy - Escalator	147	16.14	Monsters - BFS Multifuente + BFS de Escape	172
13.21	Lectura de Entrada	147	16.15	Planetqueriesi - Binary Lifting sobre Grafo Funcional	173
13.22	Lights Out (Primera Fila Forzada) - Turnswitch	149	16.16	Round Trip - Deteccion de Ciclo con BFS	174
13.23	Maquina de Estados por Objeto - Fraud	150	16.17	Round Trip (Stack) - Deteccion de Ciclo con Pila (DFS Iterativo)	175
13.24	Maximizar Area del Rectangulo Envolvente - Kings Conquest	150	16.18	Round Trip II - Ciclo en Grafo Dirigido (DFS de Tres Colores)	176
13.25	Simulacion con Parsing deCodigo - Impossible Primebox	151	16.19	Shortest Routes I - Dijkstra con Priority Queue	177
13.26	Simulation - Enigma Of The Jewelry Case	152	16.20	Shortest Routes II - Dijkstra desde Cada Nodo	177
13.27	Template Base	152	16.21	Shortest Routes II (Floyd Warshall) - Floyd-Warshall	178
13.28	Umbral de Frecuencia - Lottery	152	17 CSES: Introductory Problems		
14 Formulario			17.1	Apple Divisions - Fuerza Bruta 2^n	179
14.1	Algebra y Ecuaciones	153	17.2	Bit Strings - Exponenciacion Modular Rapida	179
14.2	Areas y Volúmenes	153	17.3	Chessboard And Queens - Backtracking con Poda por Diagonales	180
14.3	Combinatoria	154	17.4	Coin Piles - Invariantes Aritmeticos	180
14.4	Cotas y Limites	155	17.5	Creating Strings - Backtracking sobre Frecuencias	180
14.5	Datos que no se Deducen	155	17.6	Digit Queries - Busqueda por Bloques de Longitud	181
14.6	Geometria	156	17.7	Gray Code - Identidad i XOR (i >> 1)	181
14.7	Probabilidad Geometrica y Areas Raras	156	17.8	Grid Coloring I - Greedy Local con 4 Colores	181
14.8	Rectas y Conicas	157	17.9	Grid Path Description - Backtracking con Poda de Zona Encerrada	181
14.9	Secuencias Notables	158	17.10	Increasing Array - Greedy de Maximo Prefijo	182
14.10	STL Indispensable	158	17.11	Knight Moves Grid - BFS en Grilla	182
14.11	Sumas y Series	159	17.12	Mex Grid Construction - Identidad MEX = XOR	183
14.12	Tabla ASCII	159	17.13	Missing Number - Suma de Gauss	183
14.13	Teoria de Numeros	159	17.14	Number Spiral - Formula por Capas	183
14.14	Trampas de C++	160	17.15	Palindrome Reorder - Conteo de Frecuencias	183
14.15	Trigonometria	161	17.16	Permutations - Construccion Directa	184
14.16	Trucos de Bits	161	17.17	Rab Game I - Construccion por Rotacion de Bloque	184
15 Divide and Conquer			17.18	Repetitions - Barrido Lineal de Rachas	184
15.1	Conteo de Inversiones con Merge Sort - Frosh Merge Sort	162	17.19	String Reorder - Greedy Lexicografico con Cota de Factibilidad	185
16 CSES: Graph Algorithms			17.20	Tower Of Hanoi - Recursion Clasica	185
16.1	Building Roads - DSU (Union-Find)	163	17.21	Trailing Zeros - Formula de Legendre	185
16.2	Building Teams - BFS de Bicoloreo (Bipartito)	163	17.22	Two Knights - Conteo por Complemento	185
16.3	Counting Rooms - Flood Fill en Grilla (BFS)	164	17.23	Two Sets - Construccion por Bloques de 4	186
16.4	Course Schedule - Orden Topologico (Kahn)	164	17.24	Weird Algorithm - Simulacion Directa	186
16.5	Cycle Finding - Bellman-Ford (Ciclo Negativo)	165	18 CSES: Range Queries		
16.6	Flight Discount - Dijkstra sobre Grafo de Capas	166	18.1	Distinct Value Queries II - Segment Tree sobre Ocurrencia Anterior	186
16.7	Flight Routes - K Caminos Mas Cortos	166	18.2	Distinct Values Queries - Mo's Algorithm	188

Contents (cont.)

18.3 Dynamic Range Minimum Queries - Segment Tree (Minimo)	188	19.20 Substring Order II - Descenso por el Suffix Automaton (con Repetidas)	220
18.4 Dynamic Range Sum Queries - Fenwick Tree (BIT)	189	19.21 Word Combinations - Trie + DP de Segmentacion (Conteo)	222
18.5 Forest Queries - Prefix Sums 2D	190		
18.6 Forest Queries II - Fenwick 2D	190	20 Hub LeetCode: Arrays	
18.7 Hotel Queries - Segment Tree con Descenso	191	20.1 3sum Closest - Ordenar y Dos Punteros	222
18.8 Increasing Array Queries - Monotonic Stack + Binary Lifting con Costos	191	20.2 Beautiful Array - Construccion Divide y Conquista	223
18.9 List Removals - Fenwick con Descenso Binario (k-esimo)	192	20.3 Buy Two Chocolates - Los Dos mas Baratos	223
18.10 Missing Coin Sum Queries - Cubetas por Potencia de 2 + Segment Tree	193	20.4 Check If Array Is Sorted And Rotated - Ordenado y Rotado	223
18.11 Movie Festival Queries - Greedy + Binary Lifting sobre el Tiempo	193	20.5 Cinema Seat Allocation - Grupos de Sillas por Fila	223
18.12 Pizzeria Queries - Truco de p[i] mas y menos i	194	20.6 Concatenation Of Array - Concatenacion	224
18.13 Polynomial Queries - Lazy Propagation de Progresiones Aritmeticas	195	20.7 Construct Uniform Parity Array I - Paridad por Fuerza Bruta	224
18.14 Prefix Sum Queries - Segment Tree con Nodo Compuesto	196	20.8 Construct Uniform Parity Array II - Paridad con el Menor Impar	224
18.15 Range Interval Queries - Barrido Offline con Fenwick	197	20.9 Contains Duplicate - Buscar Repetidos	224
18.16 Range Queries And Copies - Segment Tree Persistente	197	20.10 Count Subarrays With Majority Element I - Contar por Bloques Iguales	224
18.17 Range Update Queries - Fenwick de Diferencias	198	20.11 Daily Temperatures - Siguiente Dia mas Caliente	225
18.18 Range Updates And Sums - Lazy Propagation con Suma y Asignacion	199	20.12 Destroying Asteroids - Greedy Creciente	225
18.19 Range XOR Queries - Prefix XOR	200	20.13 Digit Frequency Score - Frecuencia de Digitos	225
18.20 Salary Queries - Fenwick con Compresion Offline	200	20.14 Earliest Finish Time For Land And Water Rides I - Probar Todos los Pares	225
18.21 Static Range Minimum Queries - Sparse Table	201	20.15 Earliest Finish Time For Land And Water Rides II - Fijar el Mejor Primero	226
18.22 Static Range Sum Queries - Prefix Sums	201	20.16 Final Prices With A Special Discount In A Shop - Siguiente Menor o Igual	226
18.23 Sub Array Sum Queries - Segment Tree de Maximo Subarreglo	202	20.17 Find All Numbers Disappeared In An Array - Faltantes de 1 a n	226
18.24 Sub Array Sum Queries II - Maximo Subarreglo en un Rango	203	20.18 Find Missing Elements - Faltantes entre el Minimo y el Maximo	226
18.25 Visible Building Queries - Monotonic Stack + Binary Lifting	203	20.19 Find Right Interval - Buscar el Intervalo de la Derecha	226
19 CSES: String Algorithms		20.20 Find The Highest Altitude - Suma Acumulada	227
19.1 All Palindromes - Manacher (Palindromo Mas Largo que Termina en Cada Posicion)	204	20.21 Find The Prefix Common Array Of Two Arrays - Interseccion de Prefijos	227
19.2 Counting Patterns - Aho-Corasick (Conteo de Ocurrencias)	205	20.22 First Missing Positive - Primer Positivo Faltante	227
19.3 Distinct Subsequences - DP con Resta de la Aparicion Anterior	206	20.23 Hoy Many Numbers Are Smaller Than The Current Number - Fuerza Bruta $O(n^2)$	227
19.4 Distinct Substrings - Suffix Automaton (Contar Distintas)	207	20.24 Keep Multiplying Found Values By Two - Duplicar mientras Exista	227
19.5 Finding Borders - Funcion de Prefijos (Cadena de Bordes)	207	20.25 Kth Largest Element In An Array - Counting Sort Parcial	227
19.6 Finding Patterns - Aho-Corasick (Aparece o No)	208	20.26 Last Stone Weight - Simulacion con los Dos Mayores	228
19.7 Finding Periods - Periodos = n - Bordes	209	20.27 Left And Right Sum Differences - Sumas a Izquierda y Derecha	228
19.8 Inverse Suffix Array - Reconstruir la Cadena desde el Suffix Array	210	20.28 Magical String - Secuencia Autodescriptiva	228
19.9 Longest Palindrome - Manacher	211	20.29 Majority Element - Elemento Mayoritario	228
19.10 Minimal Rotation - Rotacion Minima (Dos Punteros)	211	20.30 Max Consecutive Ones - Racha Maxima	228
19.11 Palindrome Queries - Hashing Doble con Segment Tree	212	20.31 Maximum Ice Cream Bars - Greedy del mas Barato Primero	229
19.12 Pattern Positions - Aho-Corasick (Primera Aparicion)	213	20.32 Maximum Product Of Two Elements In An Array - Los Dos Mayores	229
19.13 Repeating Substring - Suffix Automaton (Ocurrencias por Estado)	214	20.33 Maximum Total Subarray Value I - Observacion que Colapsa el Problema	229
19.14 Required Substring - DP sobre el Automata de KMP	215	20.34 Median Of Two Sorted Arrays - Avanzar hasta la Mitad	229
19.15 String Functions - Funcion Z y Funcion de Prefijos	216	20.35 Merge Intervals - Fusionar Intervalos	229
19.16 String Matching - KMP (Busqueda de Patron)	217	20.36 Minimum Absolute Difference - Diferencia Minima en Ordenado	230
19.17 String Transformation - Transformada de Burrows-Wheeler Inversa	218	20.37 Minimum Cost Of Buying Candies With Discount - Greedy de Tercias	230
19.18 Substring Distribution - Suffix Automaton + Arreglo de Diferencias	218	20.38 Minimum Element After Replacement With Digit Sum - Suma de Digitos	230
19.19 Substring Order I - Descenso por el Suffix Automaton (Distintas)	219	20.39 Missing Number - Numero Faltante	230
		20.40 Pascal's Triangle - Triangulo de Pascal	230

Contents (cont.)

20.41 Plus One - Sumar Uno a un Numero por Digitos	231
20.42 Rank Transform Of An Array - Compresion de Coordenadas	231
20.43 Reduction Operations To Make The Array Elements Equal - Contar por Escalones	231
20.44 Remove Covered Intervals - Intervalos Cubiertos	231
20.45 Remove Duplicates From Sorted Array - Quitar Repetidos en Ordenado	231
20.46 Remove Element - Filtrar en el Sitio	231
20.47 Search Insert Position - Búsqueda Lineal	232
20.48 Set Mismatch - Repetido y Faltante	232
20.49 Shuffle The Array - Intercalar Dos Mitades	232
20.50 Single Number - El que Aparece Una Vez	232
20.51 Smallest Index With Digit Sum Equal To Index - Suma de Digitos	232
20.52 Smallest Missing Integer Greater Than Sequential Prefix Sum - Prefijo Secuencial	232
20.53 Smallest Missing Multiple Of K - Buscar el Primer Multiplo Ausente	232
20.54 Smallest Stable Index I - Maximo por Prefijo y Minimo por Sufijo	233
20.55 Smallest Stable Index II - Maximo por Prefijo y Minimo por Sufijo	233
20.56 Summary Ranges - Agrupar Rangos Consecutivos	233
20.57 Two Sum - Fuerza Bruta $O(n^2)$	233
20.58 Valid Mountain Array - Verificar Sube y Baja	233
21 Hub LeetCode: Binary Search	
21.1 Binary Search - Búsqueda Binaria	234
21.2 Count Elements With At Least K Greater Values - Ordenar y Buscar	234
21.3 Find Minimum In Rotated Sorted Array - Búsqueda en Arreglo Rotado	234
21.4 Peak Index In A Mountain Array - Binaria sobre Monotonía	234
21.5 Search In Rotated Sorted Array - Búsqueda en Arreglo Rotado	234
22 Hub LeetCode: Binary Tree	
22.1 Construct Binary Tree From Inorder And Postorder Traversal - Divide y Conquista sobre...	235
22.2 Counts Nodes Equal To Average Of Subtree - DFS que Sube Informacion	235
22.3 Maximum Depth Of Binary Tree - DFS en Arbol Binario	235
23 Hub LeetCode: Bit Manipulation	
23.1 Minimum Number Of Flips To Reverse Binary String - Comparar un Binario con su Reve...	236
24 Hub LeetCode: Combinatorics	
24.1 Count The Number Of Computer Unlocking Permutations - Factorial con un Filtro	236
24.2 Distribute Candies Among Children I - Fuerza Bruta con Poda	236
24.3 Number Of Sets Of K Non Overlapping Line Segments - Stars and Bars	236
24.4 Unique 3 Digit Event Numbers - Conteo con Frecuencias	236
25 Hub LeetCode: DP	
25.1 Climbing Stairs - Memoización	237
25.2 Distinct Subsequences I - DP en Dos Cadenas	237
25.3 Distinct Subsequences II - DP con Resta de Repetidos	237
25.4 Find X Value Of Array I - DP sobre Residuos	237
25.5 Maximum Score Of Non Overlapping Intervals - DP de Intervalos con Binaria	237
26 Hub LeetCode: Game Theory	
26.1 Divisor Game - Paridad	238
26.2 Find The Winning Player In Coin Game - Simulación de Turnos	238
26.3 Nim Game - Posiciones Perdedoras	238
27 Hub LeetCode: Geometry	
27.1 Angle Between Hands Of A Clock - Angulos de un Reloj	238
27.2 Circle And Rectangle Overlapping - Punto mas Cercano en un Rectangulo	238
27.3 Minimum Cuts To Divide A Circle - Casos por Paridad	238
27.4 Rectangle Overlap - Intersección de Rectangulos	238
28 Hub LeetCode: Graphs	
28.1 All Paths From Source To Target - Backtracking de Caminos	239
28.2 Cheapest Flights Within K Stops - DP con Estado (nodo, saltos)	239
28.3 Check If There Is A Valid Path In A Grid - Compatibilidad de Tubos en Matriz	239
28.4 Clone Graph - Clonar Grafo con Diccionario	240
28.5 Course Schedule - Orden Topologico de Kahn	240
28.6 Course Schedule II - Orden Topologico de Kahn	240
28.7 Evaluate Division - Grafo con Pesos Multiplicativos	241
28.8 Find A Safe Walk Through A Grid - Búsqueda con Vida Restante	241
28.9 Find Eventual Safe States - Nodos que No Llegan a un Ciclo	242
28.10 Holding Hands - DSU sobre Parejas	242
28.11 Is Graph Bipartite - Coloreo a Dos Colores	242
28.12 Jump Game III - DFS con Visitados	242
28.13 Longest Increasing Path In A Matrix - DFS con Memoización en Matriz	243
28.14 Minimum Height Trees - Pelar Hojas (Centro del Arbol)	243
28.15 Minimum Moves To Clean The Classroom - Dijkstra sobre (nodo, bitmask) con Poda d...	244
28.16 Network Delay Time - Dijkstra	246
28.17 Network Recovery Pathways - Binaria sobre la Respuesta en un DAG	246
28.18 Number Of Provinces - Componentes Conexas	246
28.19 Reconstruct Itinerary - Camino Euleriano (Hierholzer)	247
28.20 Redundant Connection - DSU (Union-Find)	247
28.21 Redundant Connection II - Ciclo en Grafo Dirigido por Casos	247
28.22 Remove Methods From Project - Alcanzabilidad y Contaminación	248
28.23 Sum Of Distances In Tree - Rerooting en Arbol	248
29 Hub LeetCode: Greedy	
29.1 Maximum Number Of Coins You Can Get - Greedy sobre Ordenado	249
29.2 Maximum Number Of Non Overlapping Palindrome Substrings - Greedy de Izquierda a...	249
29.3 Stone Game - Greedy que Acierta por Casualidad	249
30 Hub LeetCode: Heap	
30.1 Construct Target Array With Multiple Sums - Deshacer el Proceso al Revés	249
30.2 Find K Pairs With Smallest Sums - Heap Maximo de Tamano k	249
30.3 Maximum Number Of Eaten Apples - Heap por Fecha de Vencimiento	250

Contents (cont.)

31 Hub LeetCode: Linked List

31.1 Add Two Numbers - Suma con Acarreo en Lista Ligada	250
31.2 Copy List With Random Pointer - Atajo de Libreria	251
31.3 Insertion Sort List - Insertion Sort en Lista Ligada	251
31.4 Linked List Cycle - Liebre y Tortuga	251
31.5 Merge Two Sorted Lists - Fusión de Listas Ligadas	251
31.6 Odd Even Linked List - Separar en Dos Listas	252
31.7 Palindrome Linked List - Palindromo en Lista Ligada	252
31.8 Remove Duplicates From Sorted List - Recorrido de Lista Ligada	252
31.9 Reverse Linked List - Invertir Lista Ligada	252

32 Hub LeetCode: Math

32.1 Counts Commas In Range I - Conteo por Magnitud	253
32.2 Counts Commas In Range II - Conteo por Bandas de Magnitud	253
32.3 Palindrome Number - Palindromo con Dos Punteros	253
32.4 Roman To Integer - Numeros Romanos	253
32.5 Smallest Divisible Digit Product I - Fuerza Bruta	253
32.6 Sqrt(x) - Raíz Entera	254
32.7 Sum Of Square Numbers - Dos Punteros sobre Cuadrados	254
32.8 Total Waviness Of Numbers In Range I - Fuerza Bruta sobre Dígitos	254

33 Hub LeetCode: Matrix

33.1 Image Overlap - Conteo de Desplazamientos	254
33.2 Island Perimeter - Recorrido de Matriz	254

34 Hub LeetCode: Prefix Sum

34.1 Continuous Subarray Sum - Residuos de Prefijos	255
34.2 Make Sum Divisible By P - Residuos de Prefijos	255
34.3 Minimum Operations To Reduce X To Zero - Prefijos y Sufijos	255
34.4 Ways To Make A Fair Array - Prefijos Pares e Impares por Separado	255

35 Hub LeetCode: Queue

35.1 Design Circular Queue - Cola Circular con Arreglo Fijo	256
35.2 Implement Queue Using Stacks - Cola	256
35.3 Number Of Students Unable To Eat Lunch - Simulación de Cola	256
35.4 Time Needed To Buy Tickets - Simulación de Cola	257

36 Hub LeetCode: Segment Tree

36.1 Find X Value Of Array II - Segment Tree con Nodo Compuesto	257
---	-----

37 Hub LeetCode: Sliding Window

37.1 Find The Largest Almost Missing Integer - Contar en Cuantas Ventanas Aparece	257
37.2 Find Two Non Overlapping Subarrays Each With Target Sum - Dos Ventanas sin Traslape	258
37.3 Length Of Longest Subarray With At Most K Frequency - Ventana con Frecuencias	258
37.4 Longest Substring Without Repeating Characters - Ventana Deslizante con Set	258
37.5 Maximum Length Substring With Two Occurrences - Ventana con Frecuencias	259

37.6 Shortest And Lexicographically Smallest Beautiful String - Ventana con Conteo Exacto	259
--	-----

38 Hub LeetCode: Sorting

38.1 Merge Sorted Array - Merge Sort	259
38.2 Reverse Pairs - Contar en la Fusión del Merge Sort	260
38.3 Sort An Array - Counting Sort	260

39 Hub LeetCode: Stack

39.1 Build An Array With Stack Operations - Simulación de Pila	260
39.2 Evaluate Reverse Polish Notation - Evaluación en Notación Polaca	260
39.3 Exclusive Time Of Functions - Pila de Llamadas	261
39.4 Largest Rectangle In Histogram - Pila Monotona (versión enredada)	261
39.5 Next Greater Node In Linked List - Siguiendo Mayor con Pila	262
39.6 Remove Duplicate Letters - Pila Monotona Lexicográfica	262
39.7 Valid Parentheses - Pila de Parentesis	262

40 Hub LeetCode: String

40.1 Brace Expansion II - Expansión Recursiva de Llaves	263
40.2 Concatenate Non Zero Digits And Multiply By Sum I - Manejo de Dígitos	263
40.3 Counts The Number Of Special Characters I - Sets de Letras	263
40.4 Counts The Number Of Special Characters II - Orden de Primera y Última Aparición	263
40.5 Detect Capital - Casos de Mayúsculas	263
40.6 Evaluate The Bracket Pairs Of A String - Sustitución de Plantilla	264
40.7 Find The Index Of The First Occurrence In A String - Búsqueda de Subcadena	264
40.8 Find The Length Of The Longest Common Prefix - Prefijos en un Set	264
40.9 Length Of Last Word - Split	264
40.10 License Key Formatting - Agrupar de Atrás hacia Adelante	264
40.11 Longest Common Prefix - Prefijo Común por Columnas	265
40.12 Masking Personal Information - Enmascarar por Formato	265
40.13 Maximum Number Of Ballons - Frecuencias con Letras Repetidas	265
40.14 Maximum Number Of Non Overlapping Substrings - Intervalos Cerrados por Letra	266
40.15 Maximum Repeating Substring - Repeticiones Consecutivas	266
40.16 Maximum Substrings With Distinct Start - Observación que Colapsa el Problema	266
40.17 Minimum Number Of Pushes To Type Word I - Asignación por Bloques de 8	267
40.18 Minimum Number Of Pushes To Type Word II - Greedy por Frecuencia	267
40.19 Number Of Strings That Appear As Substrings In Word - Contar Subcadenas	267
40.20 Number Of Substrings Containing All Three Characters - Contar Subcadenas con las Tr...	267
40.21 Process String With Special Operations I - Simulación de Operaciones	267
40.22 Reformat Date - Formateo de Fecha	267
40.23 Repeated String Match - Repetir hasta Contener	268
40.24 Repeated Substring Pattern - Período de una Cadena	268
40.25 Reverse Degree Of A String - Suma Ponderada	268
40.26 Reverse Substrings Between Each Pair Of Parentheses - Pila de Parentesis con Reverso	268
40.27 Rotate String - Rotación de Cadena	268
40.28 Smallest Palindromic Rearrangement I - Palindromo Lexicográficamente Menor	269

Contents (cont.)

40.29 Valid Anagram - Anagrama por Orden	269
40.30 Valid Palindrome - Palindromo Filtrando Caracteres	269
41 Hub LeetCode: Trie	
41.1 Counts Prefix And Suffix Pairs I - Trie hacia Adelante y al Reves	269
41.2 Implement Trie (prefix Tree) - Trie	270

1 - Data Structures

1.1 Brute Force - Count Majority Subarrays

Cuenta subarreglos donde "target" es mayoría (aparece en más de la mitad de las posiciones). Fuerza bruta $O(n^2)$: para cada inicio i expande el final j acumulando cuantas veces aparece target en el subarreglo actual y compara el doble de esa cuenta contra el tamaño del subarreglo.

C++\LeetCode\countMajoritySubarrays.cpp

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 class Solution {
6 public:
```

1.1 Brute Force - Count Majority Subarrays (cont.)

```

1  int countMajoritySubarrays(vector<int>& nums, int target) {
2      int god = 0;
3      for(int i = 0; i < nums.size(); i++)
4      {
5          vector<int> temp;
6          int tempi = 0;
7          for(int j = i; j < nums.size(); j++)
8          {
9              temp.push_back(nums[j]);
10             if(nums[j]==target)
11             {
12                 tempi++;
13             }
14             if(tempi*2 > temp.size())
15             {
16                 god++;
17             }
18         }
19     }
20     return god;
21 }

```

1.2 Compresion de Coordenadas

O: (n log n) construir, (log n) consultar

Uso: cp.construir(vals); i = cp.comp(x); x = cp.orig(i) // 0-indexado

Cuando los VALORES llegan a $1e9$ pero solo hay $n \leq 2e5$ distintos, se reemplaza cada valor por su POSICION en la lista ordenada de valores distintos. Así un arreglo indexado por valor pasa de $1e9$ casillas a n , y de golpe caben Fenwick, segment tree o arreglos de diferencias. Son tres líneas y siempre las mismas: `sort(v.begin(), v.end()); v.erase(unique(v.begin(), v.end()), v.end()); idx = lower_bound(v.begin(), v.end(), x) - v.begin();` La primera ordena, la segunda deja un solo ejemplar de cada valor (unique NO borra: mueve los repetidos al final y devuelve donde empieza la basura, por eso hace falta el erase), y lower_bound da el índice comprimido en $O(\log n)$. PROPIEDADES QUE HAY QUE TENER CLARAS, porque de ellas depende que se pueda usar: - Es MONOTONA: si $a < b$ entonces $\text{comp}(a) < \text{comp}(b)$. Por eso se puede comprimir y seguir comparando, ordenando o preguntando por rangos de valores. - Los valores IGUALES caen en el mismo índice, y eso casi siempre es lo que uno quiere. - NO preserva las distancias: $\text{comp}(1)$ y $\text{comp}(1000000)$ pueden quedar en 0 y 1. Si el problema suma longitudes, áreas o distancias, comprimir a secas da respuestas mal, y hay que guardar los valores originales para multiplicar por el ancho real de cada tramo (es lo que hace un arreglo de diferencias 2D sobre coordenadas comprimidas). Verificado: ida y vuelta exacta sobre 200000 casos aleatorios, monotonia y los duplicados cayendo en el mismo índice, sin un solo fallo. LOS DOS ERRORES QUE CUESTAN EL PROBLEMA: 1) Olvidar el erase despues del unique. El vector queda con basura al final, el tamaño esta mal y lower_bound puede caer en la zona muerta. 2) Comprimir solo los valores INICIALES cuando las actualizaciones van a introducir valores nuevos. Si una consulta futura escribe un valor que no esta en la tabla, no tiene índice donde ir. La salida es leer TODAS las consultas primero y meter esos valores en la compresion, lo que obliga a que la solución sea OFFLINE. (Así funciona "Salary Queries" de CSES, que esta en este mismo cuaderno.) CUANDO NO HACE FALTA: si los valores ya son chicos, no se comprime. Y si hacen falta los índices pero el problema es ONLINE y no se pueden juntar todos los valores antes, la alternativa es un segment tree dinámico (nodos creados a demanda) o un map, que cuesta un log mas y bastante mas memoria por elemento.

C++\Notebook\Compresion de Coordenadas.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  // ----- la version corta, la que se escribe en contest -----
5  struct Compresor {
6      vector<long long> v;
7  };
8
9  // Se le pasan TODOS los valores que van a aparecer, incluidos los de las
10 // actualizaciones futuras.
11 void construir(vector<long long> vals) {
12     v = vals;
13     sort(v.begin(), v.end());
14     v.erase(unique(v.begin(), v.end()), v.end());
15 }

```

1.2 Compresion de Coordenadas (cont.)

```

5
6  int n() const { return v.size(); }
7
8  // valor -> indice comprimido [0, n). El valor TIENE que existir.
9  int comp(long long x) const {
10     return lower_bound(v.begin(), v.end(), x) - v.begin();
11 }
12
13 // indice -> valor original
14 long long orig(int i) const { return v[i]; }
15
16 // Cuantos valores hay < x. Sirve cuando x puede NO estar en la tabla.
17 int menores(long long x) const {
18     return lower_bound(v.begin(), v.end(), x) - v.begin();
19 }
20
21 // Cuantos son <= x.
22 int menoresIguales(long long x) const {
23     return upper_bound(v.begin(), v.end(), x) - v.begin();
24 }
25 };
26
27 int main() {
28     ios::sync_with_stdio(false);
29     cin.tie(nullptr);
30
31     int n;
32     if (!cin >> n) return 0;
33
34     vector<long long> a(n);
35     for (long long &x : a) cin >> x;
36
37     Compresor c;
38     c.construir(a);
39
40     cout << "valores distintos: " << c.n() << '\n';
41
42     for (long long x : a) {
43         cout << x << " -> " << c.comp(x) << '\n';
44     }
45
46     // Uso típico: el arreglo ya comprimido, listo para cualquier estructura
47     vector<int> comprimido(n);
48     for (int i = 0; i < n; i++) comprimido[i] = c.comp(a[i]);
49
50     cout << "comprimido:";
51     for (int x : comprimido) cout << ' ' << x;
52     cout << '\n';
53     return 0;
54 }

```

1.3 Conteo de Inversiones con Fenwick - Frosh

Resuelve "Frosh Week" (problema F, CCPL): hay n estudiantes en fila y solo se pueden intercambiar PARES CONSECUTIVOS; se pide el minimo de intercambios para dejarlos ordenados. LO QUE HAY QUE VER ES LA EQUIVALENCIA, porque el resto es una plantilla: cada intercambio de vecinos cambia el orden relativo de UNA sola pareja, así que arreglar la fila cuesta exactamente tantos intercambios como PAREJAS DESORDENADAS haya, o sea el numero de INVERSIONES (pares $i < j$ con $a[i] > a[j]$). No es una cota, es una igualdad: cada inversion hay que deshacerla y cada swap deshace una. COMO SE CUENTAN AQUÍ: se recorre de izquierda a derecha metiendo cada elemento en un Fenwick indexado por VALOR. Al llegar a $a[i]$ ya hay i elementos adentro; de esos, $fw.sum(pos)$ son los menores o iguales, así que $i - fw.sum(pos)$ son los MAYORES, y esos son justo los que quedaron a la izquierda siendo mas grandes: las inversiones que aporta $a[i]$. Se acumula y se inserta. La compresion de coordenadas (el `lower_bound` sobre el arreglo ordenado) no es opcional: los numeros de estudiante son arbitrarios y el Fenwick se indexa por posicion, no por valor. El $+1$ es porque un Fenwick es 1-indexado a la fuerza (con $i = 0$, $i \& -i$ vale 0 y el for no avanza nunca). El acumulador va en long long y eso TAMPOCO es opcional: con $n = 10^6$ al revés el total es $n*(n-1)/2$, casi $5*10^{11}$, que revienta un int sin avisar. HAY OTRA FORMA, con merge sort, que esta en "froshMergeSort" en este mismo cuaderno. Medido con $n = 10^6$ en esta maquina, las dos ya arregladas: Fenwick 477 ms aleatorio, 335 invertido, 337 ordenado, ~16 MB merge sort con buffer unico 360 ms aleatorio, 311 invertido, 293 ordenado, ~8 MB O sea que para ESTE problema el merge sort gana, porque el Fenwick paga un sort y ademas guarda tres arreglos (a, el ordenado y el bit de long long) contra dos del merge. La razon para quedarse igual con la version de Fenwick es que se GENERALIZA: el mismo codigo, cambiando que se pregunta, cuenta cuantos hay menores que x , responde por rangos, o soporta actualizaciones. El merge sort solo sabe contar inversiones y se acabo. Verificado contra el sample y contra la version de merge sort sobre $n = 10^6$ en los tres casos (aleatorio, invertido y ya ordenado): mismo resultado, 250188280112 en el aleatorio.

C++/CCPL/frosh.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Fenwick {
5     int n;
6     vector<long long> bit;
7
8     Fenwick(int n) : n(n), bit(n + 1, 0) {}
9
10    void add(int idx, long long val) {
11        for (; idx <= n; idx += idx & -idx)
12            bit[idx] += val;
13    }
14
15    long long sum(int idx) {
16        long long res = 0;
17
18        for (; idx > 0; idx -= idx & -idx)
19            res += bit[idx];
20
21        return res;
22    }
23 };
24
25 int main() {
26     ios::sync_with_stdio(false);
27     cin.tie(nullptr);
28
29     int n;
30
31     while (cin >> n) {
32         vector<int> a(n);
33
34         for (int i = 0; i < n; i++) {
35             cin >> a[i];
36         }
37
38         vector<int> sorted = a;
39         sort(sorted.begin(), sorted.end());
40
41         Fenwick fw(n);
```

1.3 Conteo de Inversiones con Fenwick - Frosh (cont.)

```
48 long long god = 0;
49
50 for (int i = 0; i < n; i++) {
51
52     int pos = lower_bound(
53         sorted.begin(),
54         sorted.end(),
55         a[i]
56     ) - sorted.begin() + 1;
57
58     god += i - fw.sum(pos);
59
60     fw.add(pos, 1);
61 }
62
63 cout << god << '\n';
64 }
65
66 return 0;
67 }
```

1.4 Conteo de Inversiones y Redondeo Exacto - Sky Line

Resuelve "Skyline" (problema K, ICPC 2024): cuenta las inversiones de la secuencia de alturas (pares $i < j$ con $h[i] > h[j]$) y responde la fraccion que son del total de pares, inversiones / $(N(N-1)/2)$, con 3 decimales. El conteo es el de "frosh" (Frosh Week, CCPL, en este cuaderno): recorriendo de izquierda a derecha, los i elementos anteriores menos los que son $\leq h[i]$ dan los mayores que quedaron a la izquierda. DOS TRAMPAS, Y LA PRIMERA VERSION DE ESTE ARCHIVO CAIA EN LAS DOS: 1) El enunciado dice $-20 < h < 300$, pero su propio sample trae una altura de -29 . La primera version indexaba el Fenwick con $h + 21$, y con -29 quedaba en -8 : $fw.add(-8)$ no termina nunca ($i \& -i$ se hace 0 y el indice deja de avanzar). Se colgaba con el sample. Por eso ahora se comprime, que funciona con cualquier valor. Leccion: cuando el sample contradice las cotas, manda el sample. 2) Las reglas generales piden redondear con los empates HACIA ARRIBA. Con double y printf, un empate exacto como 0.0625 ($N = 32$ con 31 inversiones) se redondea hacia el par y sale 0.062. Ahora se calcula con enteros: $(2000*HD + PHD) / (2*PHD)$ da las milésimas con el empate hacia arriba. Verificado en 48 empates exactos y 1500 casos generales. El caso $N = 1$ va aparte porque el total de pares seria 0 y la division, 0/0.

C++/ICPC/2024/K - SkyLine.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Fenwick {
5     int n;
6     vector<long long> bit;
7
8     Fenwick(int n) : n(n), bit(n + 1, 0) {}
9
10    void add(int idx, long long val) {
11        for (; idx <= n; idx += idx & -idx)
12            bit[idx] += val;
13    }
14
15    long long sum(int idx) {
16        long long res = 0;
17
18        for (; idx > 0; idx -= idx & -idx)
19            res += bit[idx];
20
21        return res;
22    }
23 };
24
25 int main() {
26     ios::sync_with_stdio(false);
27     cin.tie(nullptr);
28
29     int N;
30
31     while (cin >> N && N != 0) {
32
```

1.4 Conteo de Inversiones y Redondeo Exacto - Sky Line (cont.)

```
33 vector<int> h(N);
34
35 for (int &x : h)
36     cin >> x;
37
38 if (N == 1) {
39     cout << "0.000\n";
40     continue;
41 }
42
43 // Compresion de coordenadas: no se confia en el rango del enunciado,
44 // porque el propio sample trae una altura de -29.
45 vector<int> v = h;
46 sort(v.begin(), v.end());
47 v.erase(unique(v.begin(), v.end()), v.end());
48
49 Fenwick fw(v.size());
50
51 long long inversions = 0;
52
53 for (int i = 0; i < N; i++) {
54
55     int x = lower_bound(v.begin(), v.end(), h[i]) - v.begin() + 1;
56
57     // Elementos anteriores que son <= h[i]
58     long long leq = fw.sum(x);
59
60     // De los i anteriores, los demas son > h[i]
61     inversions += i - leq;
62
63     fw.add(x, 1);
64 }
65
66 long long totalPairs = 1LL * N * (N - 1) / 2;
67
68 // Redondeo a 3 decimales con enteros: empates hacia arriba, como pide
69 // el enunciado. Con double + printf un empate exacto (0.0625) puede
70 // redondearse hacia el par y dar 0.062 en vez de 0.063.
71 long long mil = (2000 * inversions + totalPairs) / (2 * totalPairs);
72
73 string frac = to_string(mil % 1000);
74 while (frac.size() < 3)
75     frac = "0" + frac;
76
77 cout << mil / 1000 << "." << frac << '\n';
78 }
79
80 return 0;
81 }
```

1.5 Fenwick Tree (Consultas offline) - D Query

Resuelve "D-query" (Day 5, problema H - Contest 05 [Avanzados]): dada una secuencia $a_1..a_n$ y q consultas (i, j) , para cada una hay que responder cuantos elementos distintos hay en la subsecuencia $a_i..a_j$. Ordena las consultas por su extremo derecho j (vector `order`) y las procesa offline avanzando un puntero `curJ` de izquierda a derecha: por cada nueva posición, si el valor ya apareció antes (`lastPos`) le resta 1 en el BIT a su ocurrencia previa y suma 1 en la posición actual, de modo que el BIT siempre cuenta, para cada valor, solo su última ocurrencia hasta `curJ`; la respuesta de cada consulta es `query(j) - query(i-1)`.

C++\Training_Camp_2026\Day 5\H - D - query.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int bit_tree[30005];
5 int n;
6
7 void update(int pos, int val){
8     for(; pos <= n; pos += pos & (-pos))
9         bit_tree[pos] += val;
10 }
11
```

1.5 Fenwick Tree (Consultas offline) - D Query (cont.)

```
12 int query(int pos){
13     int res = 0;
14     for(; pos > 0; pos -= pos & (-pos))
15         res += bit_tree[pos];
16     return res;
17 }
18
19 int main(){
20     ios_base::sync_with_stdio(false);
21     cin.tie(nullptr);
22
23     cin >> n;
24     vector<int> a(n + 1);
25     for(int i = 1; i <= n; i++)
26         cin >> a[i];
27
28     int q;
29     cin >> q;
30
31     vector<int> qi(q), qj(q);
32     vector<int> order(q);
33     for(int k = 0; k < q; k++){
34         cin >> qi[k] >> qj[k];
35         order[k] = k;
36     }
37
38     // sort query indices by right endpoint j
39     sort(order.begin(), order.end(), [&](int x, int y){
40         return qj[x] < qj[y];
41     });
42
43     vector<int> answer(q);
44     unordered_map<int,int> lastPos; // value -> last position it was seen
45     lastPos.reserve(1 << 20);
46
47     int curJ = 0; // how far we've processed (1-indexed)
48     for(int idx : order){
49         int j = qj[idx];
50         // advance pointer up to j, updating BIT
51         while(curJ < j){
52             curJ++;
53             int val = a[curJ];
54             auto it = lastPos.find(val);
55             if(it != lastPos.end()){
56                 update(it->second, -1); // remove old occurrence's contribution
57             }
58             update(curJ, 1);
59             lastPos[val] = curJ;
60         }
61         answer[idx] = query(j) - query(qi[idx] - 1);
62     }
63
64     for(int k = 0; k < q; k++)
65         cout << answer[k] << "\n";
66
67     return 0;
68 }
```

1.6 Fenwick Tree (Offline Queries) - K Query

Resuelve "K-query" (Day 4, problema I - Contest 04 [Avanzados]): dado un arreglo de N números y varias consultas (i, j, k) , hay que responder cuantos elementos del subarreglo $[i, j]$ son mayores que k . Ordena las consultas por k descendente y los elementos del arreglo por valor descendente (`cmpQuery`, `cmpElement`); recorre las consultas insertando en un Fenwick tree (bit) todos los elementos con valor mayor a k acumulados hasta el momento, de forma que una consulta de rango sobre el Fenwick (`rangeQuery`) cuenta exactamente los elementos $> k$ dentro de $[l, r]$.

C++\Training_Camp_2026\Day 4\I - K Query.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
```

1.6 Fenwick Tree (Offline Queries) - K Query (cont.)

```

4 struct Element {
5     int value;
6     int pos;
7 };
8
9 struct Query {
10    int l, r, k;
11    int id;
12 };
13
14 bool cmpElement(const Element &a, const Element &b) {
15     return a.value > b.value;
16 }
17
18 bool cmpQuery(const Query &a, const Query &b) {
19     return a.k > b.k;
20 }
21
22 class Fenwick {
23     vector<int> bit;
24     int n;
25
26 public:
27     Fenwick(int n) {
28         this->n = n;
29         bit.assign(n + 1, 0);
30     }
31
32     void update(int idx, int val) {
33         while (idx <= n) {
34             bit[idx] += val;
35             idx += idx & -idx;
36         }
37     }
38
39     int query(int idx) {
40         int sum = 0;
41         while (idx > 0) {
42             sum += bit[idx];
43             idx -= idx & -idx;
44         }
45         return sum;
46     }
47
48     int rangeQuery(int l, int r) {
49         return query(r) - query(l - 1);
50     }
51 };
52
53 int main() {
54     ios::sync_with_stdio(false);
55     cin.tie(nullptr);
56
57     int n;
58     cin >> n;
59
60     vector<Element> arr(n);
61
62     for (int i = 0; i < n; i++) {
63         cin >> arr[i].value;
64         arr[i].pos = i + 1;
65     }
66
67     sort(arr.begin(), arr.end(), cmpElement);
68
69     int q;
70     cin >> q;
71
72     vector<Query> queries(q);
73
74     for (int i = 0; i < q; i++) {
75         cin >> queries[i].l >> queries[i].r >> queries[i].k;

```

1.6 Fenwick Tree (Offline Queries) - K Query (cont.)

```

76         queries[i].id = i;
77     }
78
79     sort(queries.begin(), queries.end(), cmpQuery);
80
81     Fenwick bit(n);
82     vector<int> ans(q);
83
84     int idx = 0;
85
86     for (auto &qu : queries) {
87         while (idx < n && arr[idx].value > qu.k) {
88             bit.update(arr[idx].pos, 1);
89             idx++;
90         }
91         ans[qu.id] = bit.rangeQuery(qu.l, qu.r);
92     }
93
94     for (int x : ans)
95         cout << x << "\n";
96
97     return 0;
98 }

```

1.7 Fenwick Tree (Prefix Minimum, Dominancia 3D) - The Day Of The Competitors

Resuelve "The day of the competitors" (Day 4, problema J - Contest 04 [Avanzados]): cada uno de N competidores tiene un ranking en 3 competencias distintas, un competidor es "excelente" si ningun otro lo supera en las tres a la vez, y hay que contar cuantos competidores excelentes hay. Ordena a los competidores por el primer ranking (a) y recorre en ese orden manteniendo un Fenwick tree indexado por el segundo ranking (b) que guarda el minimo tercer ranking (c) visto; para cada competidor consulta el minimo de c entre los que tienen b menor (queryPrefixMin(b-1)): si ese minimo es >= c, nadie lo domina en las tres competencias y se cuenta como excelente.

C++\Training_Camp_2026\Day 4U - The day of the competitors.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static char buf_[1 << 25];
5 static int bufLen_, bufPos_;
6 inline int gc() {
7     if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!bufLen_) return -1; }
8     return buf_[bufPos_++];
9 }
10 inline int readInt() {
11     int c = gc();
12     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
13     bool neg = false; if (c == '-') { neg = true; c = gc(); }
14     int x = 0;
15     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
16     return neg ? -x : x;
17 }
18
19 int N;
20 vector<int> bitArr;
21
22 inline void update(int pos, int val) {
23     for (; pos <= N; pos += pos & (-pos)) {
24         if (val < bitArr[pos]) bitArr[pos] = val;
25     }
26 }
27
28 inline int queryPrefixMin(int pos) {
29     int res = INT_MAX;
30     for (; pos > 0; pos -= pos & (-pos)) {
31         if (bitArr[pos] < res) res = bitArr[pos];
32     }
33     return res;
34 }
35

```

1.7 Fenwick Tree (Prefix Minimum, Dominancia 3D) - The Day Of The Competitors (cont.)

```

36 int main() {
37     int t = readInt();
38     string out;
39     while (t--) {
40         N = readInt();
41         vector<array<int,3>> people(N);
42         for (int i = 0; i < N; i++) {
43             int a = readInt(), b = readInt(), c = readInt();
44             people[i] = {a, b, c};
45         }
46         sort(people.begin(), people.end(), [](const array<int,3>& x, const array<int,3>& y) {
47             return x[0] < y[0];
48         });
49         bitArr.assign(N + 1, INT_MAX);
50         int count = 0;
51         for (int i = 0; i < N; i++) {
52             int b = people[i][1];
53             int c = people[i][2];
54             int mn = queryPrefixMin(b - 1);
55             if (mn >= c) count++;
56             update(b, c);
57         }
58         out += to_string(count);
59         out += "\n";
60     }
61     fwrite(out.data(), 1, out.size(), stdout);
62     return 0;
63 }

```

1.8 Hash Map y Ordenamiento

O: (n log n) por el sort del volcado

Uso: mp cuenta; luego vector<pair<string,int>> v(mp.begin(), mp.end())

Diferencia clave: unordered_map es tabla hash, O(1) promedio pero SIN orden; map es arbol rojo-negro, O(log n) pero siempre ordenado por clave. Ninguno de los dos se puede ordenar por el valor (second) directamente, así que el patron obligado es volcarlo a un vector<pair<...>> y ordenar ese vector con un comparador lambda. Aqui estan los comparadores mas usados: por clave ascendente/descendente, por valor ascendente/descendente, y por valor con desempate por clave (el que piden casi siempre en rankings y conteos de frecuencia). Tambien el truco de recorrer con structured bindings para modificar valores in-place y el de invertir el mapa.

C++\Notebook\Hash Map y Ordenamiento.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main()
6 {
7     ios_base::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    // contar frecuencias
11    unordered_map<string, int> mp = {{ "c", 3 }, { "a", 1 }, { "b", 2 }};
12
13    // para ordenar hay que volcar a un vector de pares
14    vector<pair<string,int>> v(mp.begin(), mp.end());
15
16    // por clave ascendente
17    sort(v.begin(), v.end(), [](auto &l, auto &r){ return l.first < r.first; });
18
19    // por valor descendente
20    sort(v.begin(), v.end(), [](auto &l, auto &r){ return l.second > r.second; });
21
22    // por valor descendente con desempate por clave ascendente
23    sort(v.begin(), v.end(), [](auto &l, auto &r){
24        if (l.second == r.second) return l.first < r.first;
25        return l.second > r.second;
26    });
27
28    for (auto &p : v)
29    {

```

1.8 Hash Map y Ordenamiento (cont.)

```

30     cout << p.first << " => " << p.second << "\n";
31 }
32
33 // modificar todos los valores in-place (el & es obligatorio, si no se copia)
34 // con C++17 tambien vale: for (auto &[key, val] : mp) val += 3;
35 for (auto &p : mp)
36 {
37     p.second += 3;
38 }
39
40 // buscar sin crear la clave por accidente (operator[] la crearia)
41 if (mp.count("b"))
42 {
43     cout << "b vale " << mp["b"] << "\n";
44 }
45
46 // invertir el mapa (valor -> clave) para ordenar por valor
47 vector<pair<int,string>> inv;
48 for (auto &p : mp) inv.push_back({p.second, p.first});
49 sort(inv.begin(), inv.end());
50
51 // map ya viene ordenado por clave, no necesita sort
52 map<int, string> m;
53 m[3] = "tres";
54 m[1] = "uno";
55 m[2] = "dos";
56 for (auto &p : m)
57 {
58     cout << p.first << " -> " << p.second << "\n";
59 }
60
61 return 0;
62 }

```

1.9 Hash Set - Dangerous Dive

Resuelve "Dangerous Dive" (Day 2, problema E - Maratona de Programacao da SBC 2013): de N buzos voluntarios que fueron a una mision, se conocen los R que regresaron; hay que reportar, en orden ascendente, los identificadores de los que NO volvieron (o '*' si volvieron todos). Guarda los identificadores que regresaron en un set k para poder consultar pertenencia en O(1), recorre los identificadores de 1 a n y va concatenando a la salida los que no estan en k (es decir, los que faltan).

C++\Training_Camp_2026\Day 2\E - Dangerous Dive.py

```

1 n, m = list(map(int,input().split()))
2 k = set(list(map(int,input().split())))
3 if(n == m):
4     print("")
5 fin = ""
6 for i in range(1,n+1):
7     if(i not in k):
8         fin+=(str(i)+" ")
9 print(fin)

```

1.10 Heap con Borrado Perezoso - Aquarium Bubble Lights

Resuelve "Aquarium Bubble Lights" (Codeforces, problema B): hay n enteros no negativos y q operaciones; en cada una PRIMERO se le resta 1 a todos (sin bajar de cero) y DESPUES, si x no es cero, se hace $a[x] = \max(a[x], v)$. Tras cada operacion hay que decir cuantos quedan positivos, y al final imprimir la secuencia. EL TRUCO ES NO DECREMENTAR NUNCA. Restarle 1 a los n elementos en cada una de las q operaciones cuesta $O(n \cdot q) = 4 \cdot 10^4 \cdot 10$ y no pasa ni de lejos. En vez de guardar el valor, se guarda CUANDO se va a volver cero: $\text{expire}[i] = \text{valor} + \text{el instante en que se fija Con eso, "a[i] sigue positivo en el instante t" es simplemente } \text{expire}[i] > t$, y el valor real en ese momento es $\max(0, \text{expire}[i] - t)$. El decremento global desaparece: no se toca a nadie, solo avanza el reloj. Al inicio el reloj esta en 0, así que $\text{expire}[i]$ arranca valiéndolo $a[i]$, y al final los valores son $\text{expire}[i] - q$. EL BORRADO PEREZOSO ES LA OTRA MITAD. Se mete cada $(\text{expire}, \text{indice})$ en un heap de mínimos para saber quien expira primero. El problema es que cuando $a[x]$ se actualiza, su entrada VIEJA sigue tirada en el heap, y `std::priority_queue` no sabe borrar un elemento del medio. La salida es no borrarla: se deja ahí y, al sacarla, se compara contra el valor vigente `if (expire[id] != e)` continue; y si no coinciden es una entrada muerta y se ignora. Cada operacion mete a lo sumo una entrada, así que el heap nunca pasa de $n + q$ elementos y el costo total sigue siendo $O((n+q) \log n)$. Este patron es el que se usa siempre que haga falta un decrease-key y la cola de prioridad no lo tenga: es el mismo "if (dist > distancia[actual]) continue;" de Dijkstra. EL ORDEN DE LOS DOS PASOS NO ES NEGOCIABLE, y el enunciado lo recalca: primero se vacian los que expiran en el instante t (el while), y solo despues se aplica la asignacion. Si se hace al revés, un elemento que acaba de recibir un valor nuevo se contaria como expirado. El contador de positivos se lleva al vuelo, restando cuando alguien expira y sumando cuando uno pasa de cero a positivo; recorrer el arreglo en cada operacion seria volver al $O(n \cdot q)$. Sobre los tipos: `expire` llega a lo sumo a $10^9 + 2 \cdot 10^5$, que todavia cabe en `int`, pero en `long long` uno no tiene que pararse a comprobarlo. Verificado contra una simulacion directa del enunciado (restar 1 a todos, asignar, contar) en 4000 casos aleatorios con valores chicos para forzar que todo expire seguido: sin una sola diferencia, mas el sample. Medido con $n = q = 2 \cdot 10^5$ y valores hasta 10^9 : 0.27 s contra un limite de 1 s, y 0.16 s en el adversario de valores 0..3 donde el heap trabaja todo el tiempo. OJO: usa structured bindings (`auto [e, id]`), que piden C++17. En Codeforces compila, pero con un g++ viejo hay que volver a `.first` y `.second`.

C++\Codeforces\aquariumBubbleLights.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    int n, q;
11    cin >> n >> q;
12
13    vector<ll> expire(n + 1);
14
15    // (expire, index)
16    priority_queue<pair<ll, int>,
17                vector<pair<ll, int>>,
18                greater<pair<ll, int>>> pq;
19
20    for (int i = 1; i <= n; ++i) {
21        ll a;
22        cin >> a;
23
24        expire[i] = a;
25
26        if (a > 0)
27            pq.push({expire[i], i});
28    }
29
30    int positive = 0;
31
32    // Initially, all a[i] > 0 are positive.
33    for (int i = 1; i <= n; ++i)
34        if (expire[i] > 0)
35            positive++;
36
37    for (int t = 1; t <= q; ++t) {
38        int x;
```

1.10 Heap con Borrado Perezoso - Aquarium Bubble Lights (cont.)

```
39        ll v;
40        cin >> x >> v;
41
42        // After the mandatory decrement at operation t,
43        // an element is zero iff expire[i] <= t.
44        while (!pq.empty() && pq.top().first <= t) {
45            auto [e, id] = pq.top();
46            pq.pop();
47
48            // Old heap entry.
49            if (expire[id] != e)
50                continue;
51
52            expire[id] = 0;
53            positive--;
54        }
55
56        if (x != 0) {
57            // Current value after the decrement.
58            ll cur = max(0LL, expire[x] - t);
59
60            ll nw = max(cur, v);
61
62            if (cur == 0 && nw > 0)
63                positive++;
64
65            if (nw > 0) {
66                expire[x] = t + nw;
67                pq.push({expire[x], x});
68            } else {
69                expire[x] = 0;
70            }
71        }
72
73        cout << positive << '\n';
74    }
75
76    // Final values are expire[i] - q.
77    for (int i = 1; i <= n; ++i) {
78        if (i > 1)
79            cout << ' ';
80
81        cout << max(0LL, expire[i] - q);
82    }
83
84    cout << '\n';
85
86    return 0;
87 }
```

1.11 Mo's Algorithm - Wine Production

Resuelve "Wine Production" (Day 7, problema E - Contest 07 [Avanzados]): dadas las temperaturas de N días y Q consultas de rango $[l, r]$, hay que calcular para cada rango la "calidad" x maxima tal que existan al menos x temperaturas distintas que se repitan al menos x veces dentro de ese rango (un índice tipo h-index sobre las frecuencias de temperatura). Usa el algoritmo de Mo (offline, ordenando las consultas por bloques de tamaño $\sqrt{N}$) para mover los punteros l/r y mantener `freqArr` (frecuencia de cada valor comprimido) junto con `B[k]` (cantidad de valores distintos con frecuencia $\geq k$); la variable h se ajusta incrementalmente en `addElem/removeElem` para responder cada consulta en $O(1)$ amortizado.

C++\Training_Camp_2026\Day 7\E - Wine Production.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int N, Q;
5 vector<int> A; // valores comprimidos, 1-indexado
6 vector<int> freqArr; // frecuencia actual de cada valor comprimido
7 vector<int> B; // B[k] = cantidad de valores distintos con frecuencia >= k
8 int h;
9
10 inline void addElem(int val) {
```

1.11 Mo's Algorithm - Wine Production (cont.)

```

11 int f = freqArr[val];
12 B[f + 1] += 1;
13 freqArr[val] = f + 1;
14 // solo puede crecer como maximo 1 (demostrado matematicamente),
15 // pero dejamos el while por seguridad/robustez.
16 while (h + 1 < (int)B.size() && B[h + 1] >= h + 1) h++;
17 }
18
19 inline void removeElem(int val) {
20     int f = freqArr[val];
21     B[f] -= 1;
22     freqArr[val] = f - 1;
23     while (h > 0 && B[h] < h) h--;
24 }
25
26 int main() {
27     scanf("%d %d", &N, &Q);
28     vector<int> rawA(N + 1);
29     vector<int> sortedVals(N);
30     for (int i = 1; i <= N; i++) {
31         scanf("%d", &rawA[i]);
32         sortedVals[i - 1] = rawA[i];
33     }
34     sort(sortedVals.begin(), sortedVals.end());
35     sortedVals.erase(unique(sortedVals.begin(), sortedVals.end()), sortedVals.end());
36     int M = (int)sortedVals.size();
37
38     A.assign(N + 1, 0);
39     for (int i = 1; i <= N; i++) {
40         int idx = (int)(lower_bound(sortedVals.begin(), sortedVals.end(), rawA[i]) - sortedVals.begin());
41         A[i] = idx + 1;
42     }
43
44     freqArr.assign(M + 2, 0);
45     B.assign(N + 2, 0);
46     h = 0;
47
48     vector<int> ql(Q), qr(Q), qIdx(Q);
49     for (int i = 0; i < Q; i++) {
50         scanf("%d %d", &ql[i], &qr[i]);
51         qIdx[i] = i;
52     }
53
54     int blockSize = max(1, (int)sqrt((double)N));
55     sort(qIdx.begin(), qIdx.end(), [&](int a, int b) {
56         int ba = ql[a] / blockSize, bb = ql[b] / blockSize;
57         if (ba != bb) return ba < bb;
58         if (ba & 1) return qr[a] > qr[b];
59         return qr[a] < qr[b];
60     }));
61
62     vector<int> ans(Q);
63     int curL = 1, curR = 0;
64     for (int qi : qIdx) {
65         int l = ql[qi], r = qr[qi];
66         while (curR < r) { curR++; addElem(A[curR]); }
67         while (curL > l) { curL--; addElem(A[curL]); }
68         while (curR > r) { removeElem(A[curR]); curR--; }
69         while (curL < l) { removeElem(A[curL]); curL++; }
70         ans[qi] = h;
71     }
72
73     string out;
74     out.reserve(Q * 4);
75     for (int i = 0; i < Q; i++) {
76         out += to_string(ans[i]);
77         out += '\n';
78     }
79     fputs(out.c_str(), stdout);
80     return 0;
81 }

```

1.12 Ordered Set (PBDS)

O: (log n) insertar, borrar, find_by_order y order_of_key

Uso: s.find_by_order(k) -> k-esimo (0-indexado); s.order_of_key(x) -> menores

Estructura basada en políticas de GNU: funciona como un set<> normal pero ademas esta indexada internamente, lo que da dos operaciones que un set no tiene, ambas en O(log n): find_by_order(k) devuelve un iterador al k-esimo elemento (0-indexado, o end() si k >= size()), y order_of_key(x) devuelve cuantos elementos hay estrictamente menores que x. Sirve para contar inversiones, rankings dinamicos o "cuantos menores que X llevo hasta ahora" sin montar un Fenwick con compresion de coordenadas. Para permitir duplicados usa less_equal<int> en vez de less<int> (ojo: con eso erase por valor deja de funcionar y hay que borrar via find_by_order). AVISO: es una extension de GNU, solo compila con g++ (no con MSVC ni clang sin libstdc++). La instalacion local de MinGW 6.3 de este equipo tiene los headers de pb_ds incompletos y da error al compilar, pero en Codeforces y en la mayoría de jueces funciona sin problema.

C++\Notebook\Ordered Set (PBDS).cpp

```

1 #include <bits/stdc++.h>
2 #include <ext/pb_ds/assoc_container.hpp>
3 #include <ext/pb_ds/tree_policy.hpp>
4
5 using namespace std;
6 using namespace __gnu_pbds;
7
8 typedef tree<int, null_type, less<int>, rb_tree_tag,
9         tree_order_statistics_node_update> ordered_set;
10
11 int main()
12 {
13     ios_base::sync_with_stdio(false);
14     cin.tie(nullptr);
15
16     ordered_set s;
17     s.insert(10);
18     s.insert(20);
19     s.insert(30);
20     s.insert(40);
21
22     cout << *s.find_by_order(0) << "\n"; // 10, el primero
23     cout << *s.find_by_order(2) << "\n"; // 30, el tercero
24
25     cout << s.order_of_key(30) << "\n"; // 2, hay dos menores que 30
26     cout << s.order_of_key(35) << "\n"; // 3, no hace falta que 35 exista
27
28     if (s.find_by_order(99) == s.end())
29     {
30         cout << "fuera de rango\n";
31     }
32
33     return 0;
34 }

```

1.13 Persistencia (DFS + Rollback) - The Declaration Of Independence

Resuelve "The Declaration of Independence" (Day 5, problema D - Contest 05 [Avanzados]): el documento empieza vacío (version 0) y cada query crea una nueva version a partir de otra version v ya existente: "E v x" copia la version v y agrega la oracion x al final, "D v" copia la version v y elimina la primera oracion; para cada query D hay que imprimir la oracion eliminada. Modela las versiones como un arbol (dicc[v] guarda los hijos que se crearon a partir de v) y recorre ese arbol con un DFS (funcion dfs) manteniendo un unico deque global "arr" que representa el documento actual; al entrar a un nodo aplica su operacion (append o popleft) y al salir la deshace (rollback), simulando asi todas las versiones sin copiar el documento.

C++\Training_Camp_2026\Day 5\D - The Declaration of Independence.py

```

1 import sys
2 from collections import deque
3
4
5 sys.setrecursionlimit(3*10**6)
6 n = int(input())
7 queries = []
8
9 dicc = {0: []}

```

1.13 Persistencia (DFS + Rollback) - The Declaration Of Independence (cont.)

```

1 deletes = []
2 for i in range(1, n+1):
3     q = input().split(" ")
4     kind = q[0]
5
6     if kind == 'E':
7         father = int(q[1])
8         num = int(q[2])
9         if father not in dicc:
10            dicc[father] = [(i, num)]
11        else:
12            dicc[father].append((i, num))
13    else:
14        father = int(q[1])
15        deletes.append(i)
16        if father not in dicc:
17            dicc[father] = [(i, kind)]
18        else:
19            dicc[father].append((i, kind))
20
21 arr = deque([])
22 res = [0] * (n + 1)
23 def dfs(currNode):
24     global arr
25     quitado = 0
26     if currNode[1] == "D":
27         if len(arr) > 0:
28             quitado = arr.popleft()
29     else:
30         if currNode[1] != None:
31             arr.append(currNode[1])
32     res[currNode[0]] = quitado
33
34     if currNode[0] in dicc:
35         for son in dicc[currNode[0]]:
36             dfs(son)
37
38     if currNode[1] == "D":
39         arr.appendleft(quitado)
40     else:
41         if len(arr) > 0:
42             arr.pop()
43
44     dfs((0, None))
45
46 for i in deletes:
47     print(res[i], end=" ")

```

1.14 Prefix Sums - Count Majority Subarrays2

Misma idea que countMajoritySubarrays.cpp pero en O(n): transforma el arreglo a +1 (target) / -1 (cualquier otro valor) y cuenta subarreglos con suma positiva. Como el balance (prefijo acumulado desplazado por n) cambia en +-1 en cada paso, usa un arreglo pre[] indexado por ese balance para llevar, sin necesitar un Fenwick tree, cuantos prefijos anteriores tuvieron un balance menor al actual, actualizando presum incrementalmente en O(1) por posicion.

C++\LeetCode\countMajoritySubarrays2.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 class Solution {
6 public:
7     long long countMajoritySubarrays(vector<int>& nums, int target) {
8         int n = nums.size();
9
10        vector<int> pre(2 * n + 1, 0);
11
12        pre[n] = 1;
13
14        int cnt = n;
15        long long presum = 0;
16        long long ans = 0;

```

1.14 Prefix Sums - Count Majority Subarrays2 (cont.)

```

17
18        for (int x : nums) {
19            if (x == target) {
20                presum += pre[cnt];
21
22                ++cnt;
23                ++pre[cnt];
24            } else {
25                --cnt;
26
27                presum -= pre[cnt];
28                ++pre[cnt];
29            }
30
31            ans += presum;
32        }
33
34        return ans;
35    }
36 };

```

1.15 Prefix Sums con Primera Aparicion - Accounting Qualifying Con Prefix

Resuelve "Account Qualifying" (problema A, ICPC 2025): de una lista de transacciones hay que dar el mayor deposito d, el mayor retiro w (el mas negativo) y r, el largo del subperiodo mas largo con TANTOS depositos como retiros. EL TRUCO DE r: se mapea deposito a +1, retiro a -1 y consulta de saldo a 0. Entonces "tantos depositos como retiros" es "la suma del tramo es 0", y con sumas de prefijo eso es prefijo[j] == prefijo[i]. Guardando la PRIMERA posicion donde aparece cada valor de prefijo, el tramo mas largo que termina en j sale restando esa primera aparicion. Por eso el mapa solo se escribe cuando el valor es nuevo: reescribirlo acortaria los tramos. El first[0] = -1 no es un detalle menor: cubre los tramos que arrancan en la posicion 0, donde el prefijo vale 0 "antes de empezar". OJO CON d Y w: si no hay depositos d vale 0, y si no hay retiros w vale 0. Tomar el maximo y el minimo del arreglo a secas esta MAL, y el propio sample lo pilló: con 100 200 300 el minimo es 100 pero la respuesta es 0. Por eso van acotados con max(0, ...) y min(0, ...). La primera version de este archivo fallaba justo ahi. Los ceros (consultas de saldo) no rompen nada: aportan 0 a la suma, así que un tramo de puros ceros cuenta como valido, que es lo que pide el enunciado. Costo O(n) por caso. Medido: 5 casos de n = 10000 en 20 ms. Verificado contra fuerza bruta en 2900 casos, incluidos los que no tienen depositos o no tienen retiros.

C++\ICPC\2025\A - Accounting Qualifying con prefix.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9
10    while (cin >> n && n != 0) {
11
12        vector<int> a(n);
13
14        for (int &x : a)
15            cin >> x;
16
17        // Si no hay depositos d es 0, y si no hay retiros w es 0: por eso se
18        // acotan con 0 en vez de tomar el maximo y el minimo a secas.
19        int d = max(0, *max_element(a.begin(), a.end()));
20        int w = min(0, *min_element(a.begin(), a.end()));
21
22        // Convertimos:
23        // deposito -> +1
24        // balance -> 0
25        // retiro -> -1
26        for (int &x : a) {
27            if (x > 0) x = 1;
28            else if (x < 0) x = -1;
29        }
30

```

1.15 Prefix Sums con Primera Aparicion - Accounting Qualifying Con Prefix (cont.)

```

31 // first[s] = primera posicion donde aparecio
32 // el prefix sum s
33 unordered_map<int, int> first;
34
35 int sum = 0;
36 int r = 0;
37
38 // prefix sum 0 aparece antes de empezar
39 first[0] = -1;
40
41 for (int i = 0; i < n; i++) {
42     sum += a[i];
43
44     if (first.count(sum)) {
45         // Mismo prefix sum => entre ambas posiciones
46         // la suma es 0
47         r = max(r, i - first[sum]);
48     } else {
49         // Guardamos solamente la primera aparicion,
50         // porque nos da el segmento mas largo
51         first[sum] = i;
52     }
53 }
54
55 cout << d << ' ' << w << ' ' << r << '\n';
56 }
57
58 return 0;
59 }

```

1.16 Segment Tree - Block Placement Queries

Resuelve "Block Placement Queries": usa un segment tree de maximos y un set de obstaculos para determinar si un bloque de tamano sz cabe en algun hueco del rango [0, x].

C++ LeetCode/blockPlacementQueries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef int T; // tipo de dato del segtree
5 struct segtree {
6     vector<T> st; //, lazy;
7     int n;
8     T neutro = 0; // neutro para MAX (los huecos son >= 0)
9
10    segtree(const vector<T> &v) {
11        n = v.size();
12        st.assign(n*4, 0);
13        //lazy.assign(n*4, neutro);
14        build(1, 0, n-1, v);
15    }
16
17    void build(int p, int L, int R, const vector<T> &v) {
18        if (L == R) st[p] = v[L];
19        else {
20            int m=(L+R)/2, l=p*2, r=l+1;
21            build(1,L,m,v); build(r,m+1,R,v);
22            st[p] = max(st[l], st[r]); // <- max
23        }
24    }
25
26    /*
27    void propagate(int p, int L, int R, T val) {
28        if (val == neutro) return;
29        st[p] = val;
30        lazy[p] = neutro;
31        if (L == R) return;
32        int l = p*2, r = l+1;
33        lazy[l] = lazy[r] = val;
34    }
35    */

```

1.16 Segment Tree - Block Placement Queries (cont.)

```

36
37 T query(int i, int j) { return query(1, 0, n-1, i, j); }
38 void upd(int i, int j, T val) { upd(1, 0, n-1, i, j, val); }
39
40 T query(int p, int L, int R, int i, int j) {
41     //propagate(p, L, R, lazy[p]);
42     if (i > R || j < L) return neutro;
43     if (i <= L && R <= j) return st[p];
44     int m=(L+R)/2, l=p*2, r=l+1;
45     T lf = query(1, L, m, i, j);
46     T rg = query(r, m+1, R, i, j);
47     return max(lf, rg); // <- max
48 }
49
50 void upd(int p, int L, int R, int i, int j, T val) {
51     //propagate(p, L, R, lazy[p]);
52     if (i > R || j < L) return;
53     if (i <= L && R <= j) st[p] = val; // cambiar por propagate(p,L,R,val);
54     else {
55         int m=(L+R)/2, l=p*2, r=l+1;
56         upd(1, L, m, i, j, val);
57         upd(r, m+1, R, i, j, val);
58         st[p] = max(st[l], st[r]); // <- max
59     }
60 }
61 };
62
63 class Solution {
64 public:
65
66
67     vector<bool> getResults(vector<vector<int>>& queries) {
68         int MAX = 0;
69         for (auto &q : queries) MAX = max(MAX, q[1]);
70
71         vector<T> base(MAX + 1, 0); // árbol de huecos, todo en 0
72         segtree seg(base);
73
74         set<int> obs = {0}; // 0 es la pared izquierda
75         vector<bool> res;
76
77         for (auto &q : queries) {
78             if (q[0] == 1) { // colocar obstáculo en x
79                 int x = q[1];
80                 auto it = obs.lower_bound(x); // primer elemento > x
81                 int prev = *std::prev(it);
82                 seg.upd(x, x, x - prev); // hueco que termina en x
83                 if (it != obs.end()) {
84                     int next = *it;
85                     seg.upd(next, next - x); // el hueco viejo se acorta
86                 }
87                 obs.insert(x);
88             } else { // consulta [2, x, sz]
89                 int x = q[1], sz = q[2];
90                 auto it = obs.upper_bound(x); // primer elemento > x
91                 int prev = *std::prev(it); // mayor obstáculo <= x
92                 bool ok = (x - prev >= sz) || (seg.query(0, x) >= sz);
93                 res.push_back(ok);
94             }
95         }
96     }
97     return res;
98
99 }
100 };

```

1.17 Segment Tree - Monarchy In Vertigo

Resuelve "Monarchy in Vertigo" (Day 4, problema H - Contest 04 [Avanzados]): se mantiene un arbol genealogico donde nacen hijos (agregados al final de la lista de hijos de su padre) y mueren personas, y tras cada muerte hay que reportar quien es el nuevo monarca recorriendo el arbol con la regla "primero los hijos, de mayor a menor edad". Ese recorrido es exactamente un preorder del arbol final (el orden de nacimiento ya respeta "hijos antes que hermanos, mayor antes que menor"), asi que precalcula el rango preorder de cada persona (rankOf) y usa un segment tree con OR para marcar vivos/muertos y encontrar en $O(\log N)$ la primera posicion viva del preorder (queryFirstAlive), que es el monarca actual.

C++\Training_Camp_2026\Day 4\H - Monarchy in Vertigo.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static int N;
5 static vector<int> seg;
6
7 void update(int node, int lo, int hi, int pos, int val) {
8     if (lo == hi) { seg[node] = val; return; }
9     int mid = (lo + hi) >> 1;
10    if (pos <= mid) update(node<<1, lo, mid, pos, val);
11    else update(node<<1|1, mid+1, hi, pos, val);
12    seg[node] = seg[node<<1] | seg[node<<1|1];
13 }
14
15 int queryFirstAlive(int node, int lo, int hi) {
16     if (seg[node] == 0) return -1;
17     if (lo == hi) return lo;
18     int mid = (lo + hi) >> 1;
19     int r = queryFirstAlive(node<<1, lo, mid);
20     if (r != -1) return r;
21     return queryFirstAlive(node<<1|1, mid+1, hi);
22 }
23
24 int main(){
25     int Q;
26     scanf("%d", &Q);
27     vector<int> type(Q), arg(Q);
28     for (int i = 0; i < Q; i++) scanf("%d %d", &type[i], &arg[i]);
29
30     int births = 0;
31     for (int i = 0; i < Q; i++) if (type[i] == 1) births++;
32     N = births + 1; // total number of people that will ever exist
33
34     vector<vector<int>> children(N + 1);
35     vector<int> childId(Q, -1);
36     int nextId = 2;
37     for (int i = 0; i < Q; i++) {
38         if (type[i] == 1) {
39             int cid = nextId++;
40             children[arg[i]].push_back(cid);
41             childId[i] = cid;
42         }
43     }
44
45     // Pre-order numbering of the FINAL tree (birth order never changes relative order)
46     vector<int> rankOf(N + 1), idAtRank(N + 1);
47     vector<int> st; st.reserve(N + 1);
48     st.push_back(1);
49     int cur = 0;
50     while (!st.empty()) {
51         int u = st.back(); st.pop_back();
52         rankOf[u] = ++cur;
53         idAtRank[cur] = u;
54         for (int i = (int)children[u].size() - 1; i >= 0; i--) st.push_back(children[u][i]);
55     }
56
57     seg.assign(4 * (N + 1), 0);
58     update(1, 1, N, rankOf[1], 1); // Constant starts alive
59
60     string out; out.reserve(Q * 7);
61     for (int i = 0; i < Q; i++) {
```

1.17 Segment Tree - Monarchy In Vertigo (cont.)

```
62     if (type[i] == 1) {
63         update(1, 1, N, rankOf[childId[i]], 1);
64     } else {
65         update(1, 1, N, rankOf[arg[i]], 0);
66         int pos = queryFirstAlive(1, 1, N);
67         out += to_string(idAtRank[pos]);
68         out += '\n';
69     }
70 }
71 fputs(out.c_str(), stdout);
72 return 0;
73 }
```

1.18 Segment Tree - Running A Penitentiary

Resuelve "Running a penitentiary" (Day 7, problema D - Contest 07 [Avanzados]): N guardias vigilan cada uno un intervalo de celdas [L,R], y hay que soportar Q operaciones: cambiar el intervalo de un guardia, o, dado un rango de guardias [a,b], calcular cuantas celdas son vigiladas simultaneamente por TODOS los guardias de ese rango (la interseccion de sus intervalos). Usa un segment tree doble (treeL guarda el maximo de los L en el rango, treeR el minimo de los R) con point update (updatePos) y range query (query); la interseccion resultante tiene longitud $\max(0, \min R - \max L + 1)$.

C++\Training_Camp_2026\Day 7\D - Running a penitentiary.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 int n;
6 vector<ll> treeL, treeR; // treeL: max de L en el rango, treeR: min de R en el rango
7
8 const ll NEG_INF = -2000000000000000000LL;
9 const ll POS_INF = 2000000000000000000LL;
10
11 inline void pull(int i) {
12     treeL[i] = max(treeL[2*i], treeL[2*i+1]);
13     treeR[i] = min(treeR[2*i], treeR[2*i+1]);
14 }
15
16 // actualiza el guarda en posicion 0-indexada 'pos' con nuevo intervalo [lVal, rVal]
17 void updatePos(int pos, ll lVal, ll rVal) {
18     pos += n;
19     treeL[pos] = lVal;
20     treeR[pos] = rVal;
21     for (pos >= 1; pos >= 1; pos >= 1) pull(pos);
22 }
23
24 // consulta semiabierto [l, r) 0-indexada -> devuelve (max L, min R) en ese rango
25 pair<ll,ll> query(int l, int r) {
26     ll resL = NEG_INF, resR = POS_INF;
27     for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
28         if (l & 1) {
29             resL = max(resL, treeL[l]);
30             resR = min(resR, treeR[l]);
31             ++l;
32         }
33         if (r & 1) {
34             --r;
35             resL = max(resL, treeL[r]);
36             resR = min(resR, treeR[r]);
37         }
38     }
39     return make_pair(resL, resR);
40 }
41
42 int main() {
43     ios_base::sync_with_stdio(false);
44     cin.tie(nullptr);
45
46     int N, Q;
47     cin >> N >> Q;
48     n = N;
```

1.18 Segment Tree - Running A Penitentiary (cont.)

```

48 treeL.assign(2 * n, 0);
49 treeR.assign(2 * n, 0);
50
51 for (int i = 0; i < n; i++) {
52     ll l, r;
53     cin >> l >> r;
54     treeL[n + i] = l;
55     treeR[n + i] = r;
56 }
57 for (int i = n - 1; i >= 1; i--) pull(i);
58
59 string output;
60 output.reserve((size_t)0 * 8);
61
62 for (int qq = 0; qq < Q; qq++) {
63     char op;
64     cin >> op;
65     if (op == 'C') {
66         int i; ll l, r;
67         cin >> i >> l >> r;
68         updatePos(i - 1, l, r);
69     } else { // '?'
70         int a, b;
71         cin >> a >> b;
72         pair<ll, ll> res = query(a - 1, b);
73         ll ans = res.second - res.first + 1;
74         if (ans < 0) ans = 0;
75         output += to_string(ans);
76         output += '\n';
77     }
78 }
79
80 cout << output;
81 return 0;
82 }

```

1.19 Segment Tree sobre Coordenadas Comprimidas - Emergency Rations

Resuelve "Emergency Rations" (problema E, Regionals 2025). Las consultas traen posiciones que llegan hasta valores enormes, así que lo primero es COMPRIMIR: se juntan todos los x que aparecen, se ordenan, se quitan repetidos, y el segment tree trabaja sobre esos índices. EL DETALLE QUE SE OLVIDA: hay que incluir el 0 en la lista de coordenadas. Es un tamaño válido (caja vacía) y si no está, las consultas que lo necesitan caen fuera del arreglo o se contestan con el siguiente valor, que es otra respuesta. Se comprime por valor ABSOLUTO porque el problema es simétrico: lo que importa es la distancia al origen, no el lado. Eso además reduce a la mitad las coordenadas distintas. PATRON GENERAL: cada vez que los índices son enormes pero la cantidad de valores distintos es chica, se comprime. La plantilla está en "Compresión de Coordenadas" de este cuaderno; lo único que cambia de problema a problema es QUE valores hay que meter en la lista, y ahí es donde se falla: hay que incluir no solo los datos, sino también los extremos que las consultas puedan necesitar (aquí, el 0).

C++\VCPC\Regionals 2025\E - Emergency Rations.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long long ll;
5
6 #define forn(i,n) for(int i = 0; i < n; i++)
7
8 struct SegmentTree
9 {
10     int n;
11     vector<ll> tree, lazy;
12
13     SegmentTree(vector<ll>& a)
14     {
15         n = a.size();
16         tree.resize(4 * n);
17         lazy.resize(4 * n, 0);
18
19         build(1, 0, n - 1, a);
20     }

```

1.19 Segment Tree sobre Coordenadas Comprimidas - Emergency Rations (cont.)

```

21
22 void build(int node, int l, int r, vector<ll>& a)
23 {
24     if(l == r)
25     {
26         tree[node] = a[l];
27         return;
28     }
29
30     int mid = (l + r) / 2;
31
32     build(node * 2, l, mid, a);
33     build(node * 2 + 1, mid + 1, r, a);
34
35     tree[node] = min(tree[node * 2], tree[node * 2 + 1]);
36 }
37
38 void push(int node)
39 {
40     if(lazy[node] == 0)
41         return;
42
43     ll x = lazy[node];
44
45     tree[node * 2] += x;
46     tree[node * 2 + 1] += x;
47
48     lazy[node * 2] += x;
49     lazy[node * 2 + 1] += x;
50
51     lazy[node] = 0;
52 }
53
54 void update(int node, int l, int r, int ql, int qr, ll val)
55 {
56     if(ql > r || qr < l)
57         return;
58
59     if(ql <= l && r <= qr)
60     {
61         tree[node] += val;
62         lazy[node] += val;
63         return;
64     }
65
66     push(node);
67
68     int mid = (l + r) / 2;
69
70     update(node * 2, l, mid, ql, qr, val);
71     update(node * 2 + 1, mid + 1, r, ql, qr, val);
72
73     tree[node] = min(tree[node * 2], tree[node * 2 + 1]);
74 }
75
76 void update(int l, int r, ll val)
77 {
78     if(l > r)
79         return;
80
81     update(1, 0, n - 1, l, r, val);
82 }
83
84 ll query()
85 {
86     return tree[1];
87 }
88 };
89
90 int main()
91 {
92     ios::sync_with_stdio(false);

```

1.19 Segment Tree sobre Coordenadas Comprimidas - Emergency Rations (cont.)

```

93 cin.tie(nullptr);
94
95 int Q;
96 cin >> Q;
97
98 vector<ll> x(Q);
99 vector<ll> coords;
100
101 forn(i, Q)
102 {
103     cin >> x[i];
104
105     coords.push_back(abs(x[i]));
106 }
107
108 // Coordenadas de los posibles tamanos de las cajas
109 sort(coords.begin(), coords.end());
110 coords.erase(unique(coords.begin(), coords.end()), coords.end());
111
112 /*
113     Incluimos el tamaño 0.
114
115     t = 0 significa:
116     no hacer ninguna operación de "quitar una ración
117     de cada caja", sino vaciar cada caja individualmente.
118 */
119 vector<ll> values;
120
121 // 0 siempre es una posibilidad
122 values.push_back(0);
123
124 for(ll v : coords)
125     values.push_back(v);
126
127 int M = values.size();
128
129 /*
130     freq[i] = cantidad de cajas cuyo tamaño es values[i]
131 */
132 vector<ll> freq(M, 0);
133
134 /*
135     Al principio no hay cajas.
136
137     Para cada posible t:
138     valor[t] = t + cantidad de cajas con tamaño > t
139
140     Como inicialmente no hay cajas:
141     valor[t] = t
142 */
143 vector<ll> initial(M);
144
145 for(int i = 0; i < M; i++)
146     initial[i] = values[i];
147
148 SegmentTree st(initial);
149
150 /*
151     Posición de cada valor en el vector comprimido.
152 */
153 unordered_map<ll,int> pos;
154
155 for(int i = 0; i < M; i++)
156     pos[values[i]] = i;
157
158 /*
159     Agregar una caja de tamaño x.
160
161     Para todo t < x:
162     cantidad de cajas > t aumenta en 1
163
164     Por lo tanto hacemos:

```

1.19 Segment Tree sobre Coordenadas Comprimidas - Emergency Rations (cont.)

```

65     [0, pos[x]-1] += 1
66 */
67 auto addBox = [&](ll x)
68 {
69     int p = pos[x];
70
71     st.update(0, p - 1, 1);
72
73     freq[p]++;
74 };
75
76 /*
77     Quitar una caja de tamaño x.
78
79     Para todo t < x:
80     cantidad de cajas > t disminuye en 1
81 */
82 auto removeBox = [&](ll x)
83 {
84     int p = pos[x];
85
86     st.update(0, p - 1, -1);
87
88     freq[p]--;
89 };
90
91 forn(i, Q)
92 {
93     if(x[i] > 0)
94     {
95         addBox(x[i]);
96     }
97     else
98     {
99         removeBox(-x[i]);
100     }
101
102     cout << st.query() << ' ';
103 }
104
105 cout << '\n';
106
107 return 0;
108 }

```

1.20 Small-to-Large Merging (DSU on Tree) - Distinct Colors

Resuelve "Distinct Colors" (Day 4, problema M - Contest 04 [Avanzados]): dado un árbol con raíz en el nodo 1 donde cada nodo tiene un color, hay que calcular para cada nodo la cantidad de colores distintos en su subárbol. Hace un DFS iterativo para obtener un orden donde cada hijo aparece antes que su padre (orden invertido), y procesa los nodos en ese orden fusionando el set de colores de cada hijo (colorSet) dentro del set más grande entre padre e hijo (swap + insert), técnica small-to-large que garantiza complejidad $O(N \log^2 N)$ en vez de copiar sets completos en cada fusión.

C++\Training_Camp_2026\Day 4\M - Distinct Colors.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main(){
5     ios_base::sync_with_stdio(false);
6     cin.tie(NULL);
7
8     int n;
9     cin >> n;
10
11     vector<int> color(n + 1);
12     for (int i = 1; i <= n; i++) cin >> color[i];
13
14     vector<vector<int>> adj(n + 1);
15     for (int i = 0; i < n - 1; i++) {
16         int a, b;

```

1.20 Small-to-Large Merging (DSU on Tree) - Distinct Colors (cont.)

```

17 cin >> a >> b;
18 adj[a].push_back(b);
19 adj[b].push_back(a);
20 }
21
22 vector<int> parent(n + 1, 0);
23 vector<int> order;
24 order.reserve(n);
25
26 // DFS iterativo (evita overflow de pila en arboles muy profundos, tipo cadena)
27 vector<int> stck;
28 stck.reserve(n);
29 vector<char> visited(n + 1, 0);
30 stck.push_back(1);
31 visited[1] = 1;
32 while (!stck.empty()) {
33     int u = stck.back();
34     stck.pop_back();
35     order.push_back(u);
36     for (int v : adj[u]) {
37         if (!visited[v]) {
38             visited[v] = 1;
39             parent[v] = u;
40             stck.push_back(v);
41         }
42     }
43 }
44
45 // Invertimos el preorder: en este orden, cada hijo aparece antes que su padre,
46 // así que al procesar u, todos sus hijos ya están listos.
47 reverse(order.begin(), order.end());
48
49 vector<set<int>> colorSet(n + 1);
50 vector<int> ans(n + 1, 0);
51
52 for (int u = 1; u <= n; u++) colorSet[u].insert(color[u]);
53
54 // Small-to-large merging: siempre fusionamos el set mas chico dentro del mas grande
55 for (int u : order) {
56     for (int v : adj[u]) {
57         if (v != parent[u]) { // v es hijo de u
58             if (colorSet[u].size() < colorSet[v].size())
59                 colorSet[u].swap(colorSet[v]); // swap de sets es O(1), no copia elementos
60             colorSet[u].insert(colorSet[v].begin(), colorSet[v].end());
61             colorSet[v].clear(); // liberamos memoria que ya no se necesita
62         }
63     }
64     ans[u] = (int)colorSet[u].size();
65 }
66
67 for (int i = 1; i <= n; i++) {
68     cout << ans[i];
69     cout << (i < n ? ' ' : '\n');
70 }
71
72 return 0;
73 }

```

1.21 Sparse Table sobre Bloques Iguales - Frequent

Resuelve "Frequent Values" (problema E, CCPL): un arreglo de n enteros NO DECRECIENTE y q consultas $[i, j]$; para cada una hay que decir cuantas veces aparece el valor mas repetido de ese rango. LA LLAVE DEL PROBLEMA ES QUE EL ARREGLO VIENE ORDENADO, y sin eso no habria solucion facil. Al estar ordenado, los valores iguales quedan pegados en BLOQUES contiguos, así que la pregunta deja de ser sobre valores y pasa a ser sobre longitudes de bloque. Si el arreglo llegara desordenado esto no sirve: ahí tocaria Mo, que está en la ficha de CSES. Un rango $[i, j]$ toca a lo sumo tres cosas: - el PEDAZO del bloque donde cae i , que va de i hasta el final de su bloque - el PEDAZO del bloque donde cae j , desde el inicio de ese bloque hasta j - los bloques COMPLETOS que quedan en el medio. Los dos pedazos se calculan con una resta, y para los bloques completos hay que pedir el maximo de un rango de longitudes, que es exactamente lo que responde una sparse table en $O(1)$. La respuesta es el mayor de los tres. EL CASO QUE SE OLVIDA: si i y j caen en el MISMO bloque no hay medio ni dos pedazos, y la respuesta es $j - i + 1$ directo. Si uno no separa ese caso, el rango de bloques del medio sale invertido ($b_i + 1 > b_j - 1$) y la consulta a la sparse table se va a leer basura. Por eso el `maxRango` devuelve 0 cuando $l > r$, que es el segundo cinturón de seguridad. POR QUE SPARSE TABLE Y NO SEGMENT TREE: el arreglo de longitudes no cambia nunca, y el maximo es idempotente, así que se pueden solapar los dos bloques de la consulta y responder en $O(1)$. Un segment tree daria $O(\log n)$ y mas codigo, y solo hace falta si hubiera actualizaciones. Costo $O(n \log n)$ para construir y $O(1)$ por consulta. Medido con $n = q = 100000$: 0.09 s, incluyendo los adversarios de todos iguales (un solo bloque) y todos distintos (n bloques). Verificado contra fuerza bruta: 19407 consultas sobre 3000 arreglos aleatorios con bloques de repetidos, sin una sola diferencia, mas los tres casos del sample (1, 4, 3). OJO CON LA ENTRADA: los casos vienen uno tras otro y se acaban con una linea con un 0 solo, así que el while lee n y corta cuando es cero, sin leer q en ese ultimo.

C++\CCPL\frequency.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n, q;
9
10    while (cin >> n && n != 0) {
11        cin >> q;
12
13        vector<int> a(n + 1);
14        for (int i = 1; i <= n; i++) cin >> a[i];
15
16        // Como el arreglo viene ordenado, los iguales quedan en BLOQUES
17        // contiguos: se guarda a que bloque pertenece cada posicion, y donde
18        // empieza y termina cada bloque.
19        vector<int> bloque(n + 1), ini(n + 2), fin(n + 2), tam(n + 2);
20
21        int b = 0;
22
23        for (int i = 1; i <= n; i++) {
24            if (i == 1 || a[i] != a[i - 1]) {
25                b++;
26                ini[b] = i;
27            }
28            bloque[i] = b;
29            fin[b] = i;
30        }
31
32        for (int t = 1; t <= b; t++) tam[t] = fin[t] - ini[t] + 1;
33
34        // Sparse table de MAXIMOS sobre los tamanos de bloque.
35        int K = 1;
36        while ((1 << K) <= b) K++;
37
38        vector<vector<int>> st(K, vector<int>(b + 1, 0));
39
40        for (int i = 1; i <= b; i++) st[0][i] = tam[i];
41
42        for (int k = 1; k < K; k++) {
43            int len = 1 << k;
44            for (int i = 1; i + len - 1 <= b; i++)
45                st[k][i] = max(st[k - 1][i], st[k - 1][i + len / 2]);
46        }
47    }
48 }

```

1.21 Sparse Table sobre Bloques Iguales - Frequent (cont.)

```

46     }
47
48     // Maximo de tam[l..r]; los dos bloques se solapan y no pasa nada,
49     // porque el maximo es idempotente.
50     auto maxRango = [&](int l, int r) {
51         if (l > r) return 0;
52         int k = 31 - __builtin_clz(r - l + 1);
53         return max(st[k][l], st[k][r - (1 << k) + 1]);
54     };
55
56     while (q--) {
57         int i, j;
58         cin >> i >> j;
59
60         int bi = bloque[i], bj = bloque[j];
61
62         // Todo el rango cae dentro de un mismo bloque.
63         if (bi == bj) {
64             cout << j - i + 1 << '\n';
65             continue;
66         }
67
68         int izq = fin[bi] - i + 1;    // cola del bloque de i
69         int der = j - ini[bj] + 1;    // cabeza del bloque de j
70         int medio = maxRango(bi + 1, bj - 1);
71
72         cout << max(medio, max(izq, der)) << '\n';
73     }
74 }
75 return 0;
76 }

```

1.22 Sqrt Decomposition - Holes

Resuelve "Holes" (Day 4, problema K - Contest 04 [Avanzados]): hay N agujeros en fila, cada uno con una potencia que hace saltar la bola a la posición $i+a_i$, y hay que soportar M operaciones que cambian la potencia de un agujero o tiran una bola por un agujero, reportando cuantos saltos da y desde que agujero sale de la fila. Divide la fila en bloques de tamaño $\sqrt{N}$ y precalcula por bloque, para cada posición, a donde sale del bloque (jmp_), cuantos saltos (cnt_) y desde que agujero deja la fila si aplica (last_); una actualización solo recalcula su bloque (recomputeBlock) y una consulta encadena esos saltos precomputados bloque a bloque en $O(\sqrt{N})$ por operación.

C++ Training_Camp_2026/Day 4/K - Holes.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  static char buf_[1 << 22];
5  static int bufLen_, bufPos_;
6  inline int gc() {
7      if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!bufLen_) return -1; }
8      return buf_[bufPos_++];
9  }
10 inline int readInt() {
11     int c = gc();
12     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
13     bool neg = false; if (c == '-') { neg = true; c = gc(); }
14     int x = 0;
15     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
16     return neg ? -x : x;
17 }
18
19 int N, M;
20 vector<int> a_;    // power of each hole, 1-indexed
21 vector<int> jmp_;  // first position outside the hole's block reached (or N+1 sentinel = exits the row)
22 vector<int> cnt_;  // number of individual jumps taken to reach jmp_[i]
23 vector<int> last_; // valid only when jmp_[i] == N+1: last hole visited before leaving the row
24 int blockSize;
25
26 void recomputeBlock(int p) {

```

1.22 Sqrt Decomposition - Holes (cont.)

```

27     int blockIdx = (p - 1) / blockSize;
28     int start = blockIdx * blockSize + 1;
29     int end = min(N, start + blockSize - 1);
30     for (int i = end; i >= start; i--) {
31         int nxt = i + a_[i];
32         if (nxt > N) {
33             jmp_[i] = N + 1;
34             cnt_[i] = 1;
35             last_[i] = i;
36         } else if (nxt > end) {
37             // leaves the current block to a later block (nxt > i always since a_[i] >= 1)
38             jmp_[i] = nxt;
39             cnt_[i] = 1;
40             // last_[i] intentionally left as-is; never read while jmp_[i] <= N
41         } else {
42             jmp_[i] = jmp_[nxt];
43             cnt_[i] = cnt_[nxt] + 1;
44             last_[i] = last_[nxt];
45         }
46     }
47 }
48
49 int main() {
50     N = readInt();
51     M = readInt();
52     a_.assign(N + 2, 0);
53     jmp_.assign(N + 2, 0);
54     cnt_.assign(N + 2, 0);
55     last_.assign(N + 2, 0);
56
57     for (int i = 1; i <= N; i++) a_[i] = readInt();
58
59     blockSize = max(1, (int)sqrt((double)N));
60
61     for (int start = 1; start <= N; start += blockSize) {
62         recomputeBlock(start);
63     }
64
65     string out;
66     out.reserve(M * 12);
67     char tmp[32];
68
69     for (int q = 0; q < M; q++) {
70         int type = readInt();
71         if (type == 0) {
72             int pos = readInt(), val = readInt();
73             a_[pos] = val;
74             recomputeBlock(pos);
75         } else {
76             int pos = readInt();
77             long long jumps = 0;
78             int cur = pos;
79             int lastHole = -1;
80             while (cur <= N) {
81                 jumps += cnt_[cur];
82                 if (jmp_[cur] > N) {
83                     lastHole = last_[cur];
84                 }
85                 cur = jmp_[cur];
86             }
87             int len = sprintf(tmp, "%d %lld\n", lastHole, jumps);
88             out.append(tmp, len);
89         }
90     }
91     fwrite(out.data(), 1, out.size(), stdout);
92     return 0;
93 }

```

1.23 Tree Construction - Binary Tree

Resuelve "Create Binary Tree From Descriptions": dado un arreglo de tripletas [parent, child, isLeft], construye el arbol enlazando cada hijo a su padre y devuelve la raiz (el unico nodo que nunca aparece como hijo).

C++\LeetCode\binaryTree.cpp

```
1 #include <bits/stdc++.h>;
2
3 using namespace std;
4
5 class Solution {
6 public:
7     struct TreeNode {
8         int val;
9         TreeNode *left;
10        TreeNode *right;
11        TreeNode() : val(0), left(nullptr), right(nullptr) {}
12        TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
13        TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
14    };
15
16    TreeNode* createBinaryTree(vector<vector<int>>& descriptions) {
17        unordered_map<int, TreeNode*> nodes;
18        unordered_set<int> children;
19
20        for (auto &d : descriptions) {
21            int parent = d[0];
22            int child = d[1];
23            int isLeft = d[2];
24
25            if (!nodes.count(parent))
26                nodes[parent] = new TreeNode(parent);
27
28            if (!nodes.count(child))
29                nodes[child] = new TreeNode(child);
30
31            if (isLeft)
32                nodes[parent]->left = nodes[child];
33            else
34                nodes[parent]->right = nodes[child];
35
36            children.insert(child);
37        }
38
39        for (auto &d : descriptions) {
40            int parent = d[0];
41            if (!children.count(parent))
42                return nodes[parent];
43        }
44
45        return nullptr;
46    }
47};
```

2 - Dynamic Programming

2.1 Binary Search - Patches

Resuelve "Patches" (Day 2, problema I - Maratona de Programacao da SBC 2013): una llanta de circunferencia C tiene N agujeros a distancias F_i desde una marca de referencia; hay dos tipos de parche, de largo T1 y T2, y cada parche cubre de forma continua todos los agujeros que caigan dentro de su longitud. Hay que cubrir todos los agujeros usando el menor largo total de parches. rec(voy) es una recursion memoizada sobre el indice del siguiente agujero sin cubrir: prueba colocar un parche de largo t1 o de largo t2 empezando en l[voy], usa bisect_right para hallar en $O(\log N)$ el primer agujero que ya queda fuera del parche, y se queda con el minimo entre ambas opciones (largo del parche mas lo que falte cubrir despues, rec(i1) o rec(i2)).

C++\Training_Camp_2026\Day 2\I - Patches.py

```
1 from bisect import bisect_right
2
3 memo = {}
4
```

2.1 Binary Search - Patches (cont.)

```
5 def rec(voy):
6     if voy == n:
7         return 0
8
9     if voy in memo:
10        return memo[voy]
11
12    i1 = bisect_right(l, l[voy] + t1)
13    i2 = bisect_right(l, l[voy] + t2)
14
15    ans = min(
16        t1 + rec(i1),
17        t2 + rec(i2)
18    )
19
20    memo[voy] = ans
21    return ans
22
23
24 n, c, t1, t2 = map(int, input().split())
25 l = list(map(int, input().split()))
26
27 print(rec(0))
```

2.2 Bitmask DP - Music Festival

Resuelve "Music Festival" (Day 8, problema F - Contest 08 [Avanzados]): hay $N \leq 10$ escenarios, cada uno con varios shows con horario [inicio,fin) y una cantidad de canciones conocidas del artista; hay que elegir un show por cada escenario (viendolo completo, sin faltar a ningun escenario) tal que los horarios elegidos no se superpongan entre si, maximizando la suma de canciones conocidas, o -1 si no hay forma valida. Ordena todos los shows por inicio, precalcula con bisect_left el siguiente show compatible (next_show) tras terminar cada uno, y usa dp[i][mask] = mejor puntaje considerando shows desde el indice i en adelante con "mask" marcando que escenarios ya quedaron cubiertos, transicionando a next_show[i] al tomar el show i (o saltandolo).

C++\Training_Camp_2026\Day 8\F - Music Festival.py

```
1 from bisect import bisect_left
2
3 n = int(input())
4
5 shows = []
6
7 for stage in range(n):
8     data = list(map(int, input().split()))
9     m = data[0]
10    for i in range(m):
11        s = data[1 + 3 * i]
12        e = data[2 + 3 * i]
13        p = data[3 + 3 * i]
14        shows.append((s, e, stage, p))
15
16 shows.sort()
17
18 m = len(shows)
19
20 starts = [x[0] for x in shows]
21
22 next_show = [0] * m
23 for i in range(m):
24     next_show[i] = bisect_left(starts, shows[i][1])
25
26 FULL = (1 << n) - 1
27 NEG = -10**18
28
29 dp = [[NEG] * (1 << n) for _ in range(m + 1)]
30 dp[m][FULL] = 0
31
32 for i in range(m - 1, -1, -1):
33     for mask in range(1 << n):
34         dp[i][mask] = dp[i + 1][mask]
35
36 _, _, stage, songs = shows[i]
```

2.2 Bitmask DP - Music Festival (cont.)

```

37     bit = 1 << stage
38
39     for mask in range(1 << n):
40         if dp[next_show[i]][mask | bit] != NEG:
41             dp[i][mask] = max(
42                 dp[i][mask],
43                 songs + dp[next_show[i]][mask | bit]
44             )
45
46     ans = dp[0][0]
47
48     print(ans if ans != NEG else -1)

```

2.3 Bitmask SOS DP - El Juego Del Calamar

Resuelve "El Juego del Calamar" (Day 9, problema D - Contest 08 [Inicial & Avanzado]): una torre de n pisos con $k \leq 15$ habitaciones por piso (capacidad $C[i][j]$) y escaleras que solo suben de piso i a $i+1$; en cada turno se descartan jugadores sobrantes por capacidad y luego los sobrevivientes deben subir por una escalera o son eliminados. Para cada turno t (de forma independiente) se busca el maximo de jugadores que pueden seguir vivos. Como k es chico, representa por bitmask que habitaciones del piso actual se usan, y con una transformada tipo SOS (Sum over Subsets, arreglos $r/h/g/Nmask$) calcula en $O(2^k * k)$ el cuello de botella (min-cut) entre el conjunto de habitaciones alcanzadas y las capacidades del siguiente piso, propagando piso a piso hasta el turno t .

C++\Training_Camp_2026\Day 9\D - El Juego del Calamar.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  typedef long long ll;
4  const ll INF = (1ll)4e18;
5
6  int main(){
7      int n, k, m;
8      scanf("%d %d %d", &n, &k, &m);
9
10     // C[i][j]: piso i (1..n), habitacion j (0..k-1)
11     vector<vector<ll>> C(n+1, vector<ll>(k));
12     for(int i = 1; i <= n; i++){
13         for(int j = 0; j < k; j++){
14             scanf("%lld", &C[i][j]);
15         }
16
17         // adjOut[i][j]: mascara de habitaciones del piso i+1 alcanzables desde la habitacion j del piso i
18         vector<vector<int>> adjOut(n+1, vector<int>(k, 0));
19         for(int c = 0; c < m; c++){
20             int f, u, v;
21             scanf("%d %d %d", &f, &u, &v);
22             adjOut[f][u-1] |= (1 << (v-1));
23         }
24
25         int SZ = 1 << k;
26         int full = SZ - 1;
27
28         vector<ll> r(SZ), h(SZ), g(SZ);
29         vector<int> Nmask(SZ);
30
31         // r[mask] para el piso 1: suma de C[1][j] para j en mask (sin cuello de botella, fuente directa)
32         r[0] = 0;
33         for(int mask = 1; mask < SZ; mask++){
34             int lb = mask & (-mask);
35             int j = __builtin_ctz(lb);
36             r[mask] = r[mask ^ lb] + C[1][j];
37         }
38
39         vector<ll> ans(n+1);
40         ans[1] = r[full];
41
42         for(int i = 1; i <= n-1; i++){
43             // h[B] = r[complemento de B] = flujo maximo usando SOLO las habitaciones de B
44             for(int B = 0; B < SZ; B++){

```

2.3 Bitmask SOS DP - El Juego Del Calamar (cont.)

```

45             // Nmask[B] = union de vecinos (piso i+1) de las habitaciones en B
46             Nmask[0] = 0;
47             for(int B = 1; B < SZ; B++){
48                 int lb = B & (-B);
49                 int j = __builtin_ctz(lb);
50                 Nmask[B] = Nmask[B ^ lb] | adjOut[i][j];
51             }
52
53             // g[N] = mejor (min) h[B] entre los B cuya vecindad es exactamente N
54             fill(g.begin(), g.end(), INF);
55             for(int B = 0; B < SZ; B++){
56                 int N = Nmask[B];
57                 if(h[B] < g[N]) g[N] = h[B];
58             }
59
60             // transformada tipo SOS: de g a r_{i+1}, procesando cada bit (habitacion del piso i+1)
61             for(int mask = 0; mask < SZ; mask++){ r[mask] = g[mask];
62             for(int b = 0; b < k; b++){
63                 int bit = 1 << b;
64                 ll cap = C[i+1][b];
65                 for(int mask = 0; mask < SZ; mask++){
66                     if(mask & bit){
67                         int m0 = mask ^ bit;
68                         ll old0 = r[m0], old1 = r[mask];
69                         r[m0] = min(old0, old1);
70                         r[mask] = min(old0, old1 + cap);
71                     }
72                 }
73             }
74
75             ans[i+1] = r[full];
76         }
77
78         for(int t = 1; t <= n; t++){
79             printf("%lld%c", ans[t], t==n?'\\n':' ');
80         }
81
82         return 0;
83     }

```

2.4 Camino en Grilla con Balance de Parentesis - Has Valid Path

Resuelve "Check if There Is a Valid Parentheses String Path" (LeetCode 2267): en una grilla de '(' y ')' hay que decir si existe un camino de la esquina superior izquierda a la inferior derecha, moviendose solo abajo o a la derecha, cuya cadena sea de parentesis valida. LA IDEA: del camino recorrido solo importa el BALANCE (+1 por '(', -1 por ')'), no la cadena entera. Dos caminos que llegan a la misma casilla con el mismo balance tienen exactamente el mismo futuro, asi que el estado es (i, j, balance) y se memoiza. Es la misma idea de fondo de la ficha "Parentesis Balanceados" de este cuaderno, llevada a una grilla. El balance nunca pasa del largo del camino, $m+n-1$, asi que la memo es $m * n * (m+n)$: con la grilla maxima de 100×100 son unos 2 millones de estados, que caben de sobra. Un balance negativo se corta en el acto: un ')' de mas ya no lo arregla nada que venga despues. PODAS QUE NO HACEN FALTA PERO CORTAN TRABAJO: si el camino tiene largo impar ($m+n-1$ impar) la respuesta es false sin mirar nada; si el balance supera las casillas que faltan, ($m-1-i$) + ($n-1-j$), ya no se alcanza a cerrar; y la primera casilla tiene que ser '(' y la ultima ')'. OJO: la funcion valid y la cadena s que se arma en dfs no se usan para la respuesta, la memo sobre el balance ya decide todo. Se pueden borrar sin cambiar nada. Verificado con los dos ejemplos de LeetCode y contra la fuerza bruta de probar TODOS los caminos en 3000 grillas de hasta 6×6 , sin un solo fallo. Una grilla de 99×100 tarda ~20 ms.

C++\LeetCode\hasValidPath.cpp

```

1  class Solution {
2  public:
3
4      bool valid(string &s) {
5          int balance = 0;
6
7          for (char c : s) {
8              if (c == '(')
9                  balance++;
10             else
11                 balance--;

```

2.4 Camino en Grilla con Balance de Parentesis - Has Valid Path (cont.)

```

12
13     if (balance < 0)
14         return false;
15 }
16
17 return balance == 0;
18 }
19
20 vector<vector<vector<int>>> memo;
21
22 bool dfs(vector<vector<char>>& grid,
23         int i, int j,
24         int balance,
25         string &s) {
26
27     int m = grid.size();
28     int n = grid[0].size();
29
30     // Nunca podemos tener mas ')' que '('
31     if (balance < 0)
32         return false;
33
34     // Ya calculamos este estado
35     if (memo[i][j][balance] != -1)
36         return memo[i][j][balance];
37
38     // Llegamos al final
39     if (i == m - 1 && j == n - 1) {
40         return memo[i][j][balance] = (balance == 0);
41     }
42
43     // Abajo
44     if (i + 1 < m) {
45
46         char c = grid[i + 1][j];
47
48         s.push_back(c);
49
50         int newBalance = balance + (c == '(' ? 1 : -1);
51
52         if (dfs(grid, i + 1, j, newBalance, s))
53             return memo[i][j][balance] = true;
54
55         s.pop_back();
56     }
57
58     // Derecha
59     if (j + 1 < n) {
60
61         char c = grid[i][j + 1];
62
63         s.push_back(c);
64
65         int newBalance = balance + (c == '(' ? 1 : -1);
66
67         if (dfs(grid, i, j + 1, newBalance, s))
68             return memo[i][j][balance] = true;
69
70         s.pop_back();
71     }
72
73     return memo[i][j][balance] = false;
74 }
75
76 bool hasValidPath(vector<vector<char>>& grid) {
77
78     int m = grid.size();
79     int n = grid[0].size();
80
81     // La longitud maxima del string es m+n-1
82     int maxBalance = m + n;
83

```

2.4 Camino en Grilla con Balance de Parentesis - Has Valid Path (cont.)

```

84     memo.assign(
85         m,
86         vector<vector<int>>>(
87             n,
88             vector<int>(maxBalance + 1, -1)
89         )
90     );
91
92     string s;
93     s.push_back(grid[0][0]);
94
95     int balance = (grid[0][0] == '(' ? 1 : -1);
96
97     return dfs(grid, 0, 0, balance, s);
98 }
99 };

```

2.5 Convex Hull Trick

O: (n) amortizado, add y query juntos

Uso: cht.add(m,b) con m DECRECIANTE; cht.query(x) con x CRECIENTE

Optimiza DP's de la forma $dp[i] = \min_j (m[j] \cdot x[i] + b[j])$, es decir, cuando cada estado anterior aporta una recta y hay que evaluar la envolvente inferior en $x[i]$. Baja de $O(n^2)$ a $O(n)$ amortizado. Mantiene un deque con solo las rectas que llegan a ser minimas en algun punto: al insertar una recta nueva se descartan por atras las que quedaron "tapadas" (su punto de corte ya no aporta), y al consultar se descartan por adelante las que ya quedaron atras del x pedido. Requiere que las pendientes se inserten monotonas (decrecientes para minimo) y que las x de consulta vengan crecientes; si no se cumple, hay que usar Li Chao Tree o CHT dinamico.

C++\Notebook\Convex Hull Trick.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 struct CHT
6 {
7     struct L { long long m, b; double x; };
8     deque<L> dq;
9
10    double inter(const L &a, const L &c)
11    {
12        return double(c.b - a.b) / double(a.m - c.m);
13    }
14
15    void add(long long m, long long b)
16    {
17        L l = {m, b, 0};
18        while (dq.size() >= 2 && inter(dq[dq.size() - 2], dq.back()) >= inter(dq.back(), l))
19        {
20            dq.pop_back();
21        }
22        if (dq.empty()) l.x = -1e300;
23        else l.x = inter(dq.back(), l);
24        dq.push_back(l);
25    }
26
27    long long query(long long x)
28    {
29        while (dq.size() >= 2 && dq[1].x <= x)
30        {
31            dq.pop_front();
32        }
33        return dq.front().m * x + dq.front().b;
34    }
35 };
36
37 int main()
38 {
39     ios_base::sync_with_stdio(false);
40     cin.tie(nullptr);
41     return 0;
42 }

```

2.6 Digit DP

O: (digitos * estados * 10), aqui 20*200*2*10

Uso: solve_digitdp(B,d) - solve_digitdp(A-1,d) // rango [A,B]

Cuenta cuantos numeros de 0 a X cumplen una propiedad definida sobre sus digitos (aqui: que la suma de digitos sea divisible por D). Se recorre el numero digito por digito llevando dos cosas en el estado: el acumulado de la propiedad (s) y el flag tight, que indica si el prefijo construido sigue pegado al prefijo de X. El flag tight es la clave: si sigue activo, el digito actual solo puede llegar hasta el digito correspondiente de X; si ya se rompio (se puso un digito menor), a partir de ahi se puede poner cualquier cosa de 0 a 9 y el subarbol se memoiza y se reutiliza. Para contar en un rango [A,B] se calcula solve(B) - solve(A-1).

C++\Notebook\Digit DP.cpp

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 long long dp[20][200][2];
6 string S;
7 int D;
8
9 long long go(int i, int s, int tight)
10 {
11     if (i == (int)S.size()) return (s % D == 0);
12
13     long long &r = dp[i][s][tight];
14     if (r != -1) return r;
15     r = 0;
16
17     int lim = 9;
18     if (tight) lim = S[i] - '0';
19
20     for (int d = 0; d <= lim; ++d)
21     {
22         r += go(i + 1, (s + d) % D, tight && d == lim);
23     }
24     return r;
25 }
26
27 long long solve_digitdp(long long X, int d)
28 {
29     S = to_string(X);
30     D = d;
31     memset(dp, -1, sizeof dp);
32     return go(0, 0, 1);
33 }
34
35 int main()
36 {
37     ios_base::sync_with_stdio(false);
38     cin.tie(nullptr);
39
40     long long A, B;
41     int d;
42     while (cin >> A >> B >> d)
43     {
44         cout << solve_digitdp(B, d) - solve_digitdp(A - 1, d) << "\n";
45     }
46
47     return 0;
48 }
```

2.7 Distancia de Damerau-Levenshtein

O: (n*m) tiempo y memoria

Uso: lee dos cadenas por stdin; la DP esta en main, extraer el doble for

Minimo numero de operaciones para convertir a en b, con CUATRO operaciones de costo 1: insertar, borrar, sustituir e intercambiar dos caracteres ADYACENTES. Es la distancia de edicion de toda la vida mas la transposicion, que es justo lo que modela los errores de tipeo. OJO, que aqui esta la trampa: casi todo el mundo escribe la version "facil" (OSA, Optimal String Alignment), que es la tabla de Levenshtein con un if extra para el swap. Esa version prohíbe editar lo que quedo entre los dos caracteres intercambiados, y da respuestas mas grandes. El ejemplo que las separa es "ca" -> "abc": esta rutina responde 2 (intercambiar c y a, insertar la b), mientras que OSA responde 3. Si el problema pide la distancia REAL (la que cumple desigualdad triangular), toca esta version. Verificado contra busqueda exhaustiva. Los dos arreglos auxiliares son lo que permite la version irrestricta: da[c] = ultima FILA i donde aparecio la letra c en a (se actualiza al cerrar la fila i, asi que durante la fila i solo ve filas anteriores) db = ultima COLUMNA j de esta fila donde a[i] == b[j] (se lee antes de actualizarlo) Con k = da[b[j]] y l = db, el par (a[k], b[l]) es el candidato a intercambiar. Su costo es H[k][l] + (i-k-1) + 1 + (j-l-1): lo de antes, borrar los (i-k-1) sobrantes de a, pagar 1 por el intercambio, e insertar los (j-l-1) que faltan de b. Esa es la libertad que OSA no tiene. El desplazamiento en 1 no es capricho: el termino de intercambio necesita una fila y una columna "menos uno" donde caer cuando la letra nunca aparecio (k = 0) o no hubo match en la fila (l = 0). Esas casillas valen INF = n+m, mas que cualquier distancia posible, asi que el min las descarta solas y no hay que escribir ningun if. Costo O(n*m) tiempo y O(n*m) memoria. Si solo hace falta el numero y no reconstruir la edicion, se puede bajar a O(m) guardando tres filas, pero hay que arrastrar tambien la fila del intercambio, asi que rara vez vale la pena. LIMITES de esta implementacion: da[26] con b[j] - 'a' asume MINUSCULAS a-z; con mayusculas o digitos se sale del arreglo. Y H es 1005x1005 usando indices hasta n+1, o sea n, m <= 1003. Entrada: pares de palabras, una por linea, hasta la linea "*" *.

C++\Notebook\Distancia de Damerau-Levenshtein.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static int H[1005][1005];
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    string a, b;
11    while (cin >> a >> b) {
12        if (a == "*" && b == "*") break;
13
14        int n = a.size(), m = b.size();
15        int INF = n + m;
16
17        // H desplazada en 1: H[i+1][j+1] = distancia entre a[0..i] y b[0..j]
18        H[0][0] = INF;
19        for (int i = 0; i <= n; i++) { H[i + 1][0] = INF; H[i + 1][1] = i; }
20        for (int j = 0; j <= m; j++) { H[0][j + 1] = INF; H[1][j + 1] = j; }
21
22        int da[26] = {0}; // ultima fila donde aparecio cada letra en a
23
24        for (int i = 1; i <= n; i++) {
25            int db = 0; // ultima columna en esta fila donde a[i]==b[j]
26            for (int j = 1; j <= m; j++) {
27                int k = da[b[j] - 'a'];
28                int l = db;
29                int cost = 1;
30                if (a[i - 1] == b[j - 1]) { cost = 0; db = j; }
31
32                H[i + 1][j + 1] = min({
33                    H[i][j] + cost, // sustituir/igual
34                    H[i + 1][j] + 1, // insertar
35                    H[i][j + 1] + 1, // borrar
36                    H[k][l] + (i - k - 1) + 1 + (j - l - 1) // intercambiar
37                });
38            }
39            da[a[i] - 'a'] = i;
40        }
41
42        cout << H[n + 1][m + 1] << "\n";
43    }
```

2.7 Distancia de Damerau-Levenshtein (cont.)

```
43     }
44     return 0;
45 }
```

2.8 Divide and Conquer Optimization

O: $(n \log n)$ por capa, en vez de $O(n^2)$

Uso: compute(1,n,1,n,C,prev,cur,opt); exige desigualdad de Monge

Optimiza DPs por capas del tipo $dp[i][m] = \min_k (dp[i-1][k] + C(k,m))$, típicos de "partir un arreglo en i grupos". Baja cada capa de $O(n^2)$ a $O(n \log n)$. Se apoya en que el punto de corte óptimo es monotono: si $opt[m]$ es el mejor k para m , entonces opt es no decreciente en m . Entonces se resuelve el m del medio buscando su óptimo en el rango $[optl, optr]$, y con ese resultado se acota la búsqueda de las dos mitades: la izquierda solo puede tener su óptimo en $[optl, bestk]$ y la derecha en $[bestk, optr]$. Solo es válido si el costo C cumple la desigualdad cuadrangular (condición de Monge).

C++\Notebook\Divide and Conquer Optimization.cpp

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 const long long INF = (1LL << 62);
6
7 void compute(int l, int r, int optl, int optr,
8             const function<long long(int,int)>& C,
9             vector<long long>& prev, vector<long long>& cur, vector<int>& opt)
10 {
11     if (l > r) return;
12
13     int m = (l + r) / 2;
14     int bestk = optl;
15     long long best = INF;
16
17     for (int k = optl; k <= min(m - 1, optr); ++k)
18     {
19         long long v = prev[k] + C(k, m);
20         if (v < best)
21         {
22             best = v;
23             bestk = k;
24         }
25     }
26
27     cur[m] = best;
28     opt[m] = bestk;
29
30     compute(l, m - 1, optl, bestk, C, prev, cur, opt);
31     compute(m + 1, r, bestk, optr, C, prev, cur, opt);
32 }
33
34 int main()
35 {
36     ios_base::sync_with_stdio(false);
37     cin.tie(nullptr);
38     return 0;
39 }
```

2.9 DP con Arreglo de Diferencias - Balanced Balloons

Resuelve "Balanced Balloons" (problema B, Regionals 2025): contar secuencias donde el promedio se mantiene entero en cada paso, modulo 998244353. El estado es $dp[q]$ = cuantas secuencias de i elementos tienen promedio actual q . Al agregar el elemento $i+1$, el nuevo promedio q_2 tiene que cumplir que el aporte quede entre 1 y K : $1 \leq (i+1)*q_2 - i*q \leq K$ que despejando da un INTERVALO de q_2 válidos: de $\lceil (i*q+1)/(i+1) \rceil$ hasta $\lfloor (i*q+K)/(i+1) \rfloor$. EL TRUCO QUE LO HACE VIABLE: cada q manda su $dp[q]$ a un intervalo CONTIGUO de destinos. Sumar uno por uno sería $O(K)$ por origen y $O(K^2)$ por paso. Con un ARREGLO DE DIFERENCIAS se marca $+dp[q]$ en lo y $-dp[q]$ en $hi+1$, y al final una suma de prefijos reconstruye toda la fila de golpe: $O(K)$ por paso en vez de $O(K^2)$. Ese patrón -"la transición va a un rango, no a un punto"- es el que convierte muchas DP cuadráticas en lineales, y vale reconocerlo: si al escribir la transición sale un for sobre un intervalo, casi siempre se puede cambiar por diferencias. El $N = \min(N, K)$ del principio es una poda real: pasadas K personas ya no aparecen promedios nuevos, así que iterar más es trabajo perdido. La resta modular se hace con $(x - y + MOD) \% MOD$ para no quedarse en negativo, que es el error clásico de los arreglos de diferencias con modulo.

C++\CPC\Regionals 2025\B - Balanced Balloons.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long long ll;
5
6 const ll MOD = 998244353;
7
8 int main()
9 {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     ll N, K;
14     cin >> N >> K;
15
16     // Despues de K personas ya no aparecen nuevas posibilidades.
17     N = min(N, K);
18
19     /*
20      dp[q] = cantidad de secuencias cuyo promedio actual es q.
21      */
22
23     vector<ll> dp(K + 2, 0);
24
25     // Con una sola persona:
26     // puede aportar cualquier valor entre 1 y K.
27     for(int q = 1; q <= K; q++)
28         dp[q] = 1;
29
30     for(ll i = 1; i < N; i++)
31     {
32         vector<ll> diff(K + 3, 0);
33
34         for(int q = 1; q <= K; q++)
35         {
36             if(dp[q] == 0)
37                 continue;
38
39             /*
40              Queremos:
41
42              1 <= (i+1)*q2 - i*q <= K
43
44              Por lo tanto:
45
46              ceil((i*q + 1)/(i+1)) <= q2
47              q2 <= floor((i*q + K)/(i+1))
48              */
49
50              ll lo = (i * q + 1 + i) / (i + 1);
51              ll hi = (i * q + K) / (i + 1);
52
53              lo = max(lo, 1LL);
54              hi = min(hi, K);
55
56              diff[lo] += dp[q];
57              diff[hi + 1] -= dp[q];
58          }
59
60         vector<ll> ndp(K + 2, 0);
61         ll cur = 0;
62         for(int q = 1; q <= K; q++)
63             cur = (cur + diff[q]) % MOD;
64         ndp[q] = cur;
65         dp = ndp;
66     }
67
68     ll ans = 0;
69     for(int q = 1; q <= K; q++)
70         ans = (ans + dp[q]) % MOD;
71
72     cout << ans << endl;
73 }
```

2.9 DP con Arreglo de Diferencias - Balanced Balloons (cont.)

```

56         if(lo <= hi)
57         {
58             // Sumar dp[q] a todo [lo, hi]
59             diff[lo] = (diff[lo] + dp[q]) % MOD;
60
61             diff[hi + 1] =
62                 (diff[hi + 1] - dp[q] + MOD) % MOD;
63         }
64     }
65
66     // Reconstruimos ndp usando prefijos.
67     vector<ll> ndp(K + 2, 0);
68
69     ll cur = 0;
70
71     for(int q = 1; q <= K; q++)
72     {
73         cur = (cur + diff[q]) % MOD;
74         ndp[q] = cur;
75     }
76
77     dp = ndp;
78 }
79
80 ll ans = 0;
81
82 for(int q = 1; q <= K; q++)
83 {
84     ans = (ans + dp[q]) % MOD;
85 }
86
87 cout << ans << '\n';
88
89 return 0;
90 }

```

2.10 DP sobre Intervalos con Bandera de Inversion - Artic Virus

Resuelve "Arctic Virus" (problema A, ICPC 2024): decidir si una cadena es un virus segun la gramatica $\phi ::= A \mid T \mid \phi C \mid A \phi \mid A \phi^{-1} \mid C$, donde ϕ^{-1} es ϕ al reves. Responde simple (la cadena es A o T), mutation (es virus y mide 2 o mas) o doomed. OJO CON EL LARGO 1: solo A y T son "simple". Una C o una G sola no la genera ninguna regla y es doomed. La primera version imprimia simple para cualquier letra sola; verificada la corregida contra el lenguaje generado desde la gramatica, en todas las cadenas de largo 1 a 8. Las reglas se deshacen de afuera hacia adentro: mirando la primera y la ultima letra se sabe que regla pudo haber sido la ultima, se quita lo que agrego, y se sigue con lo de adentro. EL TRUCO ES NO INVERTIR NUNCA LA CADENA. Dos de las reglas dejan el interior al reves, y copiar e invertir substrings seria caro y ademas daria estados imposibles de memoizar. En vez de eso el estado es (l, r, rev): el substring $s[l..r]$, leido al derecho o al reves segun la bandera. Invertir es solo cambiar la bandera, y rev decide cual extremo es el "primero" y cual el "ultimo". Asi todos los estados posibles son $O(n^2 * 2)$, cada uno con trabajo $O(1)$. La memo va en un arreglo $memo[l][r][rev]$ con -1 para "sin calcular". La primera version usaba $map<tuple>$, y con $n = 1000$ costaba entre 230 y 440 ms por caso; con varios casos grandes en la misma entrada eso ya arriesga el limite de tiempo. CUANDO USAR ESTE ESTADO: cualquier gramatica o juego sobre una cadena donde las operaciones quitan letras de las puntas; y la bandera de inversion en particular siempre que una operacion "de la vuelta" a la cadena.

C++\ICPC\2024\A - Artic Virus.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string s;
5 int n;
6
7 // memo[l][r][rev]: -1 = sin calcular, 0 = no, 1 = si. Un arreglo en vez de
8 // map<tuple>: con n = 1000 el map costaba cientos de ms por caso.
9 signed char memo[1001][1001][2];
10
11 bool solve(int l, int r, bool rev) {
12     if (l > r) return false;
13 }

```

2.10 DP sobre Intervalos con Bandera de Inversion - Artic Virus (cont.)

```

14     signed char &res = memo[l][r][rev];
15
16     if (res != -1)
17         return res;
18
19     int len = r - l + 1;
20
21     // Llegamos a una configuracion simple
22     if (len == 1) {
23         char c = rev ? s[r] : s[l];
24         res = (c == 'A' || c == 'T');
25         return res;
26     }
27
28     // Obtener extremos de la cadena actual
29     char first = rev ? s[r] : s[l];
30     char last = rev ? s[l] : s[r];
31
32     bool ans = false;
33
34     // phiC
35     // Si termina en C, quitamos el ultimo caracter
36     if (last == 'C') {
37         if (!rev)
38             ans |= solve(l, r - 1, false);
39         else
40             ans |= solve(l + 1, r, true);
41     }
42
43     // Aphi
44     // Si comienza en A, quitamos el primero
45     if (first == 'A') {
46         if (!rev)
47             ans |= solve(l + 1, r, false);
48         else
49             ans |= solve(l, r - 1, true);
50     }
51
52     // Aphi^-1
53     // Quitamos A del inicio y luego invertimos
54     if (first == 'A') {
55         if (!rev)
56             ans |= solve(l + 1, r, true);
57         else
58             ans |= solve(l, r - 1, false);
59     }
60
61     // Gphi^-1C
62     // Debe comenzar en G y terminar en C
63     if (first == 'G' && last == 'C' && len >= 3) {
64         if (!rev)
65             ans |= solve(l + 1, r - 1, true);
66         else
67             ans |= solve(l + 1, r - 1, false);
68     }
69
70     res = ans;
71     return res;
72 }
73
74 int main() {
75     ios::sync_with_stdio(false);
76     cin.tie(nullptr);
77
78     while (cin >> n >> s) {
79
80         for (int l = 0; l < n; l++)
81             for (int r = 0; r < n; r++)
82                 memo[l][r][0] = memo[l][r][1] = -1;
83
84         // Largo 1: solo A y T son la etapa simple; C y G no son virus.
85         if (n == 1) {

```

2.10 DP sobre Intervalos con Bandera de Inversion - Artic Virus (cont.)

```

86         if (s[0] == 'A' || s[0] == 'T')
87             cout << "simple\n";
88         else
89             cout << "doomed\n";
90     }
91     else {
92         bool ok = solve(0, n - 1, false);
93
94         cout << (ok ? "mutation\n" : "doomed\n");
95     }
96 }
97
98 return 0;
99 }

```

2.11 Interval Scheduling - Yuyuan Market

Resuelve "Yuyuan Market" (Day 10, problema A - Contest 09 [Avanzados]): una fila de $2N$ jianzhi con símbolos repetidos (cada símbolo aparece dos veces), donde solo se pueden comprar pares (posición i , posición $j > i$) del mismo símbolo, sin cruzar hacia atrás una vez elegido un par. Hay que maximizar la cantidad de pares comprados y, entre los que logran el máximo, minimizar la suma de distancias $j-i$. Cada símbolo define un intervalo $[L[v], R[v]]$ (primera y segunda aparición); es DP de intervalos sobre la posición p : $cnt[p]/len[p]$ guardan el mejor (cantidad, costo) usando las primeras p posiciones, y $endAtL[p]$ permite extender el DP tomando el intervalo que termina en p desde el estado anterior a su inicio.

C++\Training_Camp_2026\Day 10\A - Yuyuan Market.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  static char buf_[1 << 22];
5  static int bufLen_, bufPos_;
6  inline int gc() {
7      if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!bufLen_) return -1; }
8      return buf_[bufPos_++];
9  }
10 inline int readInt() {
11     int c = gc();
12     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
13     bool neg = false; if (c == '-') { neg = true; c = gc(); }
14     int x = 0;
15     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
16     return neg ? -x : x;
17 }
18
19 int main() {
20     int N = readInt();
21     int M = 2 * N;
22     vector<int> L(N + 1, 0), R(N + 1, 0);
23     vector<bool> seen(N + 1, false);
24     for (int pos = 1; pos <= M; pos++) {
25         int v = readInt();
26         if (!seen[v]) { L[v] = pos; seen[v] = true; }
27         else R[v] = pos;
28     }
29
30     vector<int> endAtL(M + 1, -1); // endAtL[p] = L of the symbol whose R equals p, or -1
31     for (int v = 1; v <= N; v++) endAtL[R[v]] = L[v];
32
33     vector<int> cnt(M + 1, 0);
34     vector<long long> len(M + 1, 0);
35     for (int p = 1; p <= M; p++) {
36         cnt[p] = cnt[p - 1];
37         len[p] = len[p - 1];
38         if (endAtL[p] != -1) {
39             int Lv = endAtL[p];
40             int candCnt = cnt[Lv - 1] + 1;
41             long long candLen = len[Lv - 1] + (long long)(p - Lv);
42             if (candCnt > cnt[p] || (candCnt == cnt[p] && candLen < len[p])) {
43                 cnt[p] = candCnt;
44                 len[p] = candLen;

```

2.11 Interval Scheduling - Yuyuan Market (cont.)

```

45     }
46     }
47 }
48
49 printf("%d %lld\n", cnt[M], len[M]);
50 return 0;
51 }

```

2.12 LCS con Bloques de K Caracteres

O: $(n*m)$ tiempo y memoria**Uso:** lee K y las dos cadenas; imprime el largo máximo. $K = 0$ termina

Subsecuencia común máxima, pero con una condición extra: lo que se toma tiene que venir en BLOQUES de al menos K caracteres SEGUIDOS en las dos cadenas a la vez. Con $K = 1$ es el LCS de toda la vida; con K mayor, tomar una letra suelta ya no vale. Son DOS tablas y entender la segunda es todo el problema: $dp[i][j]$ la mejor respuesta usando $a[1..i]$ y $b[1..j]$, sin exigir que i y j se usen. $bloque[i][j]$ la mejor respuesta que TERMINA con un bloque que cierra justo en i y j . Las dos transiciones de bloque son las dos únicas formas de cerrar un bloque en (i, j) : ESTIRAR uno que ya venía: $bloque[i-1][j-1] + 1$ ABRIR uno nuevo de exactamente K : $dp[i-K][j-K] + K$ y se toma el máximo. Si no se escriben las dos, el caso que se escapa es el de dos bloques pegados que juntos miden más que cualquiera por separado. EL TRUCO SUCIO DEL ARCHIVO: $bloque[i][j]$ hace dos oficios. Mientras la racha es menor que K es un CONTADOR de letras iguales seguidas; apenas llega a K se convierte en VALOR de la DP y de ahí en adelante el $+1$ ya significa "estirar el bloque". Funciona porque se cumple que $bloque[i][j] \geq K$ si y solo si la racha real en (i, j) es $\geq K$ o sea que el mismo if sirve para las dos cosas. Verificado con asserts sobre las 8.001 pruebas: 0 violaciones. POR ESO $dp[i-K][j-K]$ NUNCA se sale: solo se entra ahí cuando la racha es al menos K , y para eso hacen falta K letras antes en las dos cadenas, así que $i \geq K$ y $j \geq K$. También verificado. Si la racha se corta ($a[i-1] \neq b[j-1]$) $bloque[i][j]$ se queda en 0 y el contador arranca de cero solo, sin tener que limpiarlo a mano. OJO CON LA MEMORIA: son dos matrices de $(n+1)*(m+1)$ enteros, o sea $8*n*m$ bytes. Con $n = m = 5000$ son 200 MB y no cabe. bloque solo necesita la diagonal anterior y dp necesita $K+1$ filas, así que si los límites suben se puede bajar a $O(K*m)$. VERIFICADO contra una fuerza bruta escrita desde otra formulación (escoger bloques disjuntos en orden, cada uno de largo $\geq K$, maximizando la suma): exhaustivo sobre alfabeto de 2 letras con cadenas de hasta 5 y K de 1 a 3, mas 4.000 aleatorios, 8.001 casos, 0 diferencias. Además con $K = 1$ da exactamente el LCS clásico en 500 casos. Tiempo: 0.13 s con $n = m = 4000$.

C++\Notebook\LCS con Bloques de K.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      ios::sync_with_stdio(false);
7      cin.tie(nullptr);
8
9      int K;
10
11     while (cin >> K && K)
12     {
13         string a, b;
14         cin >> a >> b;
15
16         int n = a.size();
17         int m = b.size();
18
19         vector<vector<int>> dp(n + 1, vector<int>(m + 1));
20         vector<vector<int>> bloque(n + 1, vector<int>(m + 1));
21
22         for (int i = 1; i <= n; i++)
23         {
24             for (int j = 1; j <= m; j++)
25             {
26                 // sin cerrar bloque en (i, j): lo mejor de los dos vecinos
27                 dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
28
29                 if (a[i-1] == b[j-1])
30                 {
31                     // contador de racha, que pasa a ser valor de DP al llegar a K
32                     bloque[i][j] = bloque[i-1][j-1] + 1;
33

```

2.12 LCS con Bloques de K Caracteres (cont.)

```

34         if(bloque[i][j] >= K)
35         {
36             bloque[i][j] = max(
37                 bloque[i][j],          // estirar el bloque de atras
38                 dp[i-K][j-K] + K      // o abrir uno nuevo de K
39             );
40         }
41         dp[i][j] = max(dp[i][j], bloque[i][j]);
42     }
43 }
44 }
45 }
46 }
47 cout << dp[n][m] << '\n';
48 }
49 return 0;
50 }
51 }

```

2.13 Prefix Sums - Zig Zag Array

Cuenta los arreglos "zigzag" de longitud n con valores en [l, r] (cada elemento alterna entre mayor y menor que su vecino), modulo $1e9+7$. DP sobre la longitud manteniendo, para cada valor, cuantas secuencias terminan subiendo (up) o bajando (down); las transiciones se aceleran con sumas de prefijos y sufijos en $O(n * (r-l))$.

C++\LeetCode\ZigZagArray.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 class Solution {
6 public:
7     static constexpr int MOD = 1'000'000'007;
8
9     int zigZagArrays(int n, int l, int r) {
10         int m = r - l + 1;
11
12         vector<long long> up(m), down(m);
13         vector<long long> pref(m + 1);
14         vector<long long> suff(m + 1);
15         vector<long long> newUp(m);
16         vector<long long> newDown(m);
17
18         for (int i = 0; i < m; i++) {
19             up[i] = i;
20             down[i] = m - 1 - i;
21         }
22
23         for (int len = 3; len <= n; len++) {
24             pref[0] = 0;
25             for (int i = 0; i < m; i++) {
26                 pref[i + 1] = (pref[i] + down[i]) % MOD;
27             }
28
29             suff[m] = 0;
30             for (int i = m - 1; i >= 0; i--) {
31                 suff[i] = (suff[i + 1] + up[i]) % MOD;
32             }
33
34             for (int i = 0; i < m; i++) {
35                 newUp[i] = pref[i];
36                 newDown[i] = suff[i + 1];
37             }
38
39             up.swap(newUp);
40             down.swap(newDown);
41         }
42
43         long long ans = 0;
44         for (int i = 0; i < m; i++) {
45             ans = (ans + up[i] + down[i]) % MOD;
46         }

```

2.13 Prefix Sums - Zig Zag Array (cont.)

```

47
48         return (int)ans;
49     }
50 };

```

2.14 SOS (Sum over Subsets) + Inclusion-Exclusion - How Many Teams

Resuelve "How many teams?" (Day 6, problema H - Maratona SBC de Programacao 2025): N estudiantes tienen cada uno un subconjunto de K habilidades (representado como bitmask), un equipo de 3 estudiantes tiene como conjunto de habilidades la union de las de sus miembros, y se dan M subconjuntos especiales; para cada uno hay que contar cuantos equipos de 3 estudiantes distintos tienen union de habilidades exactamente igual a ese subconjunto. El código cuenta primero $f[mask]$ = cantidad de estudiantes con esa máscara exacta, aplica SOS (sum over subsets) para que $f[mask]$ pase a contar estudiantes cuya máscara es submáscara de mask, calcula $h[mask] = C(f[mask], 3)$ (tripleas cuya union está contenida en mask), y luego aplica Mobius / inclusion-exclusion (SOS inverso) sobre $h[]$ para recuperar el conteo exacto por union, de modo que cada consulta se responde en $O(1)$.

C++\Training_Camp_2026\Day 6\H - How many teams.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static char buf_1[<< 22];
5 static int buflen_, bufPos_;
6 inline int gc() {
7     if (bufPos_ == buflen_) { buflen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!buflen_) return -1; }
8     return buf_[bufPos_++];
9 }
10 inline int readInt() {
11     int c = gc();
12     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
13     bool neg = false; if (c == '-') { neg = true; c = gc(); }
14     int x = 0;
15     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
16     return neg ? -x : x;
17 }
18 // Reads a binary string of known length K, skipping any non-'0'/'1' characters first,
19 // and returns the mask with position 1 (first character) as bit 0 (matches problem's
20 // convention, verified against both worked examples).
21 inline int readMaskFixedLen(int K) {
22     int c = gc();
23     while (c != '0' && c != '1') c = gc();
24     int mask = 0;
25     for (int pos = 0; pos < K; pos++) {
26         if (c == '1') mask |= (1 << pos);
27         if (pos + 1 < K) c = gc();
28     }
29     return mask;
30 }
31
32 int main() {
33     int N = readInt(), K = readInt();
34     int full = 1 << K;
35     vector<long long> f(full, 0); // will hold cnt[] then get turned into SOS sums
36
37     for (int i = 0; i < N; i++) {
38         int mask = readMaskFixedLen(K);
39         f[mask]++;
40     }
41
42     // SOS: f[mask] = number of students whose mask is a submask of `mask`
43     for (int bit = 0; bit < K; bit++) {
44         int b = 1 << bit;
45         for (int mask = 0; mask < full; mask++) {
46             if (mask & b) f[mask] += f[mask ^ b];
47         }
48     }
49
50     // g[mask] = C(f[mask], 3)
51     vector<long long> h(full, 0);
52     for (int mask = 0; mask < full; mask++) {

```

2.14 SOS (Sum over Subsets) + Inclusion-Exclusion - How Many Teams (cont.)

```

53     long long v = f[mask];
54     h[mask] = (v >= 3) ? (v * (v - 1) * (v - 2) / 6) : 0;
55 }
56
57 // Mobius inversion (inclusion-exclusion) to get exact-union counts
58 for (int bit = 0; bit < K; bit++) {
59     int b = 1 << bit;
60     for (int mask = 0; mask < full; mask++) {
61         if (mask & b) h[mask] -= h[mask ^ b];
62     }
63 }
64
65 int M = readInt();
66 string out;
67 out.reserve((size_t)M * 8);
68 char tmp[32];
69 for (int q = 0; q < M; q++) {
70     int mask = readMaskFixedLen(K);
71     int len = sprintf(tmp, "%lld\n", h[mask]);
72     out.append(tmp, len);
73 }
74 fwrite(out.data(), 1, out.size(), stdout);
75 return 0;
76 }

```

2.15 Steiner Tree - Joining Capitals

Resuelve "Joining Capitals" (Day 8, problema J - Contest 08 [Avanzados]): un reino tiene N ciudades, K de ellas capitales; hay que construir líneas de transmisión (costo = distancia euclidiana) de forma que todas las capitales queden conectadas entre sí por un camino, cada capital este conectada directamente a una sola otra ciudad, y el costo total sea mínimo. Como cada capital debe tener grado 1, el árbol de Steiner solo se ramifica en las ciudades no- capitales: $dp[mask][idx]$ (algoritmo de Dreyfus-Wagner) guarda el costo mínimo de conectar el subconjunto "mask" de capitales usando como punto de unión la no-capital " $nonCap[idx]$ ", combinando submáscaras disjuntas (paso MERGE) y extendiendo con un Dijkstra $O(M^2)$ entre no- capitales (paso EXTEND) para propagar el costo por las ciudades intermedias.

C++\Training_Camp_2026\Day 8\J - Joining Capitals.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int N, K;
6     cin >> N >> K;
7     vector<double> X(N), Y(N);
8     for (int i = 0; i < N; i++) cin >> X[i] >> Y[i];
9
10    vector<vector<double>> dist(N, vector<double>(N, 0.0));
11    for (int i = 0; i < N; i++) {
12        for (int j = 0; j < N; j++) {
13            double dx = X[i] - X[j], dy = Y[i] - Y[j];
14            dist[i][j] = sqrt(dx * dx + dy * dy);
15        }
16    }
17
18    vector<int> nonCap;
19    for (int v = K; v < N; v++) nonCap.push_back(v);
20    int M = (int)nonCap.size();
21
22    const double INF = 1e18;
23    int fullMask = (1 << K) - 1;
24    // dp[mask][idx] : idx indexes into nonCap (0..M-1)
25    vector<vector<double>> dp(1 << K, vector<double>(M, INF));
26
27    // base case: each capital connects directly to each non-capital
28    for (int i = 0; i < K; i++) {
29        for (int idx = 0; idx < M; idx++) {
30            dp[1 << i][idx] = dist[i][nonCap[idx]];
31        }
32    }
33
34    for (int mask = 1; mask <= fullMask; mask++) {

```

2.15 Steiner Tree - Joining Capitals (cont.)

```

35     // MERGE step: combine two disjoint submasks meeting at the same non-capital
36     for (int sub = (mask - 1) & mask; sub > 0; sub = (sub - 1) & mask) {
37         int other = mask ^ sub;
38         if (sub < other) {
39             for (int idx = 0; idx < M; idx++) {
40                 double cand = dp[sub][idx] + dp[other][idx];
41                 if (cand < dp[mask][idx]) dp[mask][idx] = cand;
42             }
43         }
44     }
45
46     // EXTEND step: simple  $O(M^2)$  Dijkstra among non-capitals only
47     vector<bool> done(M, false);
48     for (int iter = 0; iter < M; iter++) {
49         int u = -1;
50         double best = INF;
51         for (int idx = 0; idx < M; idx++) {
52             if (!done[idx] && dp[mask][idx] < best) { best = dp[mask][idx]; u = idx; }
53         }
54         if (u == -1) break;
55         done[u] = true;
56         for (int w = 0; w < M; w++) {
57             if (!done[w]) {
58                 double cand = dp[mask][u] + dist[nonCap[u]][nonCap[w]];
59                 if (cand < dp[mask][w]) dp[mask][w] = cand;
60             }
61         }
62     }
63 }
64
65 double answer = INF;
66 for (int idx = 0; idx < M; idx++) answer = min(answer, dp[fullMask][idx]);
67
68 printf("%.5f\n", answer);
69 return 0;
70 }

```

2.16 Subset Sum (Knapsack) - Karamell

Resuelve "Karamell" (Day 3, problema K - Maratona SBC de Programacao 2024): Alice y Bob reciben N bolsas de caramelos que se reparten en el orden dado, entregando cada bolsa a quien tenga menos caramelos en ese momento (en caso de empate, a Alice); hay que encontrar un orden de las bolsas tal que ambos terminen con la misma cantidad de caramelos, o determinar que es imposible. El código primero descarta el caso de suma total impar. Luego usa una DP de subset-sum ($dp[i][s]$, con bytearray por fila para ahorrar memoria) para determinar si existe un subconjunto de bolsas cuya suma sea la mitad del total, y reconstruye ese subconjunto (sub). Finalmente separa las bolsas en dos grupos (alice/bob) según pertenezcan o no a ese subconjunto y las intercala de forma greedy, dándole siempre la siguiente bolsa al que menos lleva acumulado, reproduciendo la regla de reparto del enunciado para garantizar el empate final.

C++\Training_Camp_2026\Day 3\K - Karamell.py

```

1 import sys
2 input = sys.stdin.readline
3
4 n = int(input())
5 ka = sorted(map(int, input().split()))
6 suma = sum(ka)
7
8 if suma % 2 != 0:
9     print(-1)
10 else:
11     mit = suma // 2
12
13 # ---- subset-sum DP: reemplaza el rec exponencial ----
14 # dp[i][s] = con los primeros i elementos se puede formar la suma s
15 dp = [bytearray(mit + 1) for _ in range(n + 1)]
16 dp[0][0] = 1
17 for i in range(1, n + 1):
18     w = ka[i - 1]
19     cur = dp[i]
20     prev = dp[i - 1]
21     for s in range(mit + 1):

```

2.16 Subset Sum (Knapsack) - Karamell (cont.)

```

22     if prev[s] or (s >= w and prev[s - w]):
23         cur[s] = 1
24
25     if not dp[n][mit]:
26         print(-1)
27     else:
28         # reconstruyo el subconjunto que suma mit (queda como sub ordenado, igual que f[0])
29         sub = []
30         s = mit
31         for i in range(n, 0, -1):
32             w = ka[i - 1]
33             if s >= w and dp[i - 1][s - w]: # este elemento entra al subconjunto
34                 sub.append(w)
35                 s -= w
36         sub.reverse() # ascendente, como lo esperaba tu código
37
38         # ---- de aquí en adelante: TU MISMA reconstrucción ----
39         alice = []
40         bob = []
41         pos = 0
42         for j in range(len(ka)):
43             if pos < len(sub) and sub[pos] == ka[j]:
44                 alice.append(j)
45                 pos += 1
46             else:
47                 bob.append(j)
48
49         alicel = 0
50         bobl = 0
51         posA = 0
52         posB = len(bob) - 1
53         imp = []
54         for _ in range(len(ka)):
55             if alicel < bobl:
56                 v = ka[alice[posA]]; posA += 1
57                 alicel += v
58             else:
59                 v = ka[bob[posB]]; posB -= 1
60                 bobl += v
61             imp.append(str(v))
62         print(" ".join(imp))

```

2.17 Valor Esperado - Revenge Of The Salary Of AtCoder Inc

Resuelve "Revenge of The Salary of AtCoder Inc." (Day 10, problema N - Contest 09 [Avanzados]): con un dado de N caras y $x=0$, en cada ronda se tira el dado (resultado y); si $x < y$ se cobra A_y yen y x pasa a valer y, si no el proceso termina. Hay que hallar el valor esperado del salario total, modulo 998244353. Define $E[i]$ = valor esperado de pago futuro estando en $x=i$; por linealidad de la esperanza, $E[i] = (1/N) * \sum_{y>i} (A_y + E[y])$, ya que tirar $y < i$ termina el proceso sin pago adicional. El código calcula esto de atrás hacia adelante (i de N a 1) manteniendo S = suma acumulada de $(A_y + E[y])$ para $y > i$, de modo que $E[i] = S * \text{invN}$ (invN es el inverso modular de N via Fermat); el resultado final es $E[0] = S * \text{invN}$ tras procesar todo el arreglo.

C++\Training_Camp_2026\Day 10\N - Revenge of The Salary of AtCoder Inc.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5  const ll MOD = 998244353;
6
7  ll binpow(ll a, ll e) {
8      ll r = 1;
9      while (e) {
10         if (e & 1) r = r * a % MOD;
11         a = a * a % MOD;
12         e >>= 1;
13     }
14     return r;
15 }
16
17 ll inv(ll x) {
18     return binpow(x, MOD - 2);

```

2.17 Valor Esperado - Revenge Of The Salary Of AtCoder Inc (cont.)

```

19 }
20
21 int main() {
22     ios::sync_with_stdio(false);
23     cin.tie(nullptr);
24
25     int N;
26     cin >> N;
27
28     vector<ll> A(N + 1);
29     for (int i = 1; i <= N; i++)
30         cin >> A[i];
31
32     ll invN = inv(N);
33
34     vector<ll> E(N + 2, 0);
35
36     ll S = 0;
37
38     for (int i = N; i >= 1; i--) {
39         E[i] = S * invN % MOD;
40         S = (S + A[i] + E[i]) % MOD;
41     }
42
43     ll ans = S * invN % MOD;
44
45     cout << ans << '\n';
46 }

```

3 - Graph

3.1 2-SAT (Kosaraju) - Signal Coverage

Resuelve "Signal Coverage" (problema G, ICPC 2024). Cada antena se instala o no; hay parejas de las que AL MENOS UNA debe instalarse, y dos antenas que coinciden en tiempo (sus intervalos se cruzan) y en espacio (sus círculos se tocan) NO pueden instalarse las dos. El código decide si se puede y da una asignación. Todas las restricciones son de dos variables, y eso es 2-SAT: al menos una: $x \vee y$ no las dos: $\neg x \vee \neg y$. Cada cláusula ($a \vee b$) se vuelve dos implicaciones, $\neg a \rightarrow b$ y $\neg b \rightarrow a$. Con la codificación $2i = \text{instalada}$, $2i+1 = \text{no instalada}$, negar es un xor con 1. Se sacan las componentes fuertemente conexas con Kosaraju. Hay solución si y solo si ninguna variable queda en la misma componente que su negación (eso sería x implica $\neg x$). LA TRAMPA DE LA ASIGNACIÓN: Kosaraju numera las componentes en ORDEN TOPOLOGICO, y la regla es x verdadera si $\text{comp}[x] > \text{comp}[\neg x]$. Con Tarjan la numeración sale al revés y la desigualdad se voltea. Mezclar las dos da asignaciones que violan restricciones sin avisar. El choque de círculos va en enteros: $\text{dist}^2 \leq ((r1 + r2)^2)$, sin raíz cuadrada. El borde cuenta (tangentes chocan) y los días también: activa de b a $e+1$, así que chocan si $\max(b) \leq \min(e)$. OJO CON LA LECTURA: cada antena viene como " r x y b e", con el radio PRIMERO. La primera versión leía " x y r b e" y respondía "11" en el primer caso del sample, instalando dos antenas que se cubren. Corregida, se valida contra fuerza bruta en 1800 casos (incluye tangentes y días que se tocan), sin una sola salida inválida. RIESGO DE MEMORIA: se revisan todos los pares, y si muchas antenas se solapan las cláusulas crecen como n^2 . Medido con todas solapadas: 6000 antenas piden 371 MB, y con las 10000 del enunciado sería del orden de 1 GB. Si el juez trae un caso así de denso, la salida es no guardar las aristas de conflicto y recalcularlas al recorrer, como hace "J - Lumina".

C++\ICPC\2024\G - Signal Coverage.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct Antenna {
5      long long x, y, r;
6      int b, e;
7  };
8
9  int main() {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     int T;
14     cin >> T;
15

```

3.1 2-SAT (Kosaraju) - Signal Coverage (cont.)

```

6   while (T--) {
7
8       int n, m;
9       cin >> n >> m;
10
11      vector<Antenna> a(n);
12
13      for (int i = 0; i < n; i++) {
14          // el enunciado da el RADIO primero: r x y b e
15          cin >> a[i].r >> a[i].x >> a[i].y
16              >> a[i].b >> a[i].e;
17      }
18
19      /*
20       2-SAT:
21
22       2*i    -> antena i instalada
23       2*i + 1 -> antena i NO instalada
24
25       negacion:
26       x ^ 1
27
28       */
29
30      int V = 2 * n;
31
32      vector<vector<int>> g(V), gr(V);
33
34      auto addEdge = [&](int u, int v) {
35          g[u].push_back(v);
36          gr[v].push_back(u);
37      };
38
39      // (u OR v)
40      auto addClause = [&](int u, int v) {
41          // !u -> v
42          addEdge(u ^ 1, v);
43
44          // !v -> u
45          addEdge(v ^ 1, u);
46      };
47
48      // -----
49      // Restricciones dadas por las parejas
50      // -----
51
52      for (int i = 0; i < m; i++) {
53
54          int u, v;
55          cin >> u >> v;
56
57          --u;
58          --v;
59
60          // Al menos una debe instalarse
61          // xu OR xv
62          addClause(2 * u, 2 * v);
63      }
64
65      // -----
66      // Restricciones entre antenas
67      // -----
68
69      for (int i = 0; i < n; i++) {
70
71          for (int j = i + 1; j < n; j++) {
72
73              // Sus periodos se intersectan?
74              bool sameTime =
75                  max(a[i].b, a[j].b) <=
76                  min(a[i].e, a[j].e);
77

```

3.1 2-SAT (Kosaraju) - Signal Coverage (cont.)

```

78          if (!sameTime)
79              continue;
80
81          // Sus circulos se intersectan?
82          long long dx = a[i].x - a[j].x;
83          long long dy = a[i].y - a[j].y;
84
85          long long dist2 = dx * dx + dy * dy;
86
87          long long radius = a[i].r + a[j].r;
88
89          bool samePlace =
90              dist2 <= radius * radius;
91
92          if (samePlace) {
93
94              /*
95               No pueden instalarse las dos:
96
97               !(xi AND xj)
98
99               = !xi OR !xj
100              */
101
102              addClause(
103                  2 * i + 1,
104                  2 * j + 1
105              );
106          }
107      }
108
109      // -----
110      // Kosaraju - primera pasada
111      // -----
112
113      vector<bool> visited(V, false);
114      vector<int> order;
115
116      function<void(int)> dfs1 = [&](int u) {
117
118          visited[u] = true;
119
120          for (int v : g[u]) {
121              if (!visited[v])
122                  dfs1(v);
123          }
124
125          order.push_back(u);
126      };
127
128      for (int i = 0; i < V; i++) {
129          if (!visited[i])
130              dfs1(i);
131      }
132
133      // -----
134      // Kosaraju - segunda pasada
135      // -----
136
137      vector<int> comp(V, -1);
138
139      function<void(int, int)> dfs2 =
140          [&](int u, int id) {
141
142          comp[u] = id;
143
144          for (int v : gr[u]) {
145              if (comp[v] == -1)
146                  dfs2(v, id);
147          }
148      };

```

3.1 2-SAT (Kosaraju) - Signal Coverage (cont.)

```

80
81     reverse(order.begin(), order.end());
82
83     int components = 0;
84     for (int u : order) {
85
86         if (comp[u] == -1) {
87             dfs2(u, components++);
88         }
89     }
90
91     // -----
92     // Comprobar si existe solucion
93     // -----
94
95     bool possible = true;
96
97     for (int i = 0; i < n; i++) {
98
99         int on = 2 * i;
100        int off = 2 * i + 1;
101
102        if (comp[on] == comp[off]) {
103            possible = false;
104            break;
105        }
106    }
107
108    if (!possible) {
109        cout << "Impossible\n";
110        continue;
111    }
112
113    // -----
114    // Construir respuesta
115    // -----
116
117    string ans(n, '0');
118
119    for (int i = 0; i < n; i++) {
120
121        int on = 2 * i;
122        int off = 2 * i + 1;
123
124        ans[i] = (comp[on] > comp[off]) ? '1' : '0';
125    }
126
127    cout << ans << '\n';
128 }
129
130 return 0;
131 }

```

3.2 Articulation Points (Tarjan) - Submerging Islands

Resuelve "Submerging Islands" (Day 4, problema L - Contest 04 [Avanzados]): dado un grafo de N islas conectadas por M puentes, hay que contar cuantas islas, al sumergirse, desconectarían partes de la ciudad. Es el clásico problema de puntos de articulación: hace un DFS (función dfs) calculando disc/low de cada nodo, marca como articulación a todo nodo no raíz cuyo hijo v cumpla $low[v] \geq disc[u]$, y a la raíz si tiene más de un hijo en el árbol DFS, contando al final cuantos nodos quedaron marcados (soporta grafos no conexos probando DFS desde cada nodo no visitado).

C++\Training_Camp_2026\Day 4\L - Submerging Islands.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXN = 10005;
5
6 vector<int> adj[MAXN];
7 int disc[MAXN], low[MAXN], timer;

```

3.2 Articulation Points (Tarjan) - Submerging Islands (cont.)

```

8 bool visited[MAXN], articulation[MAXN];
9
10 void dfs(int u, int parent) {
11     visited[u] = true;
12     disc[u] = low[u] = ++timer;
13     int children = 0;
14
15     for (int v : adj[u]) {
16         if (!visited[v]) {
17             children++;
18             dfs(v, u);
19
20             low[u] = min(low[u], low[v]);
21
22             if (parent != -1 && low[v] >= disc[u])
23                 articulation[u] = true;
24         }
25         else if (v != parent) {
26             low[u] = min(low[u], disc[v]);
27         }
28     }
29
30     if (parent == -1 && children > 1)
31         articulation[u] = true;
32 }
33
34 int main() {
35     ios::sync_with_stdio(false);
36     cin.tie(nullptr);
37
38     int N, M;
39
40     while (cin >> N >> M && (N || M)) {
41
42         for (int i = 1; i <= N; i++) {
43             adj[i].clear();
44             visited[i] = false;
45             articulation[i] = false;
46         }
47
48         timer = 0;
49
50         for (int i = 0; i < M; i++) {
51             int u, v;
52             cin >> u >> v;
53             adj[u].push_back(v);
54             adj[v].push_back(u);
55         }
56
57         // Por si el grafo no fuera conexo.
58         for (int i = 1; i <= N; i++)
59             if (!visited[i])
60                 dfs(i, -1);
61
62         int ans = 0;
63         for (int i = 1; i <= N; i++)
64             if (articulation[i])
65                 ans++;
66
67         cout << ans << '\n';
68     }
69
70     return 0;
71 }

```

3.3 BFS - Ladder Length

Resuelve "Word Ladder": halla la longitud de la transformacion mas corta de beginWord a endWord cambiando una letra a la vez, usando solo palabras de wordList. BFS por niveles donde cada palabra es un nodo y dos palabras se conectan si difieren en exactamente una letra.

C++\LeetCode\ladderLength.cpp

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 class Solution {
6 public:
7     int ladderLength(string beginWord, string endWord, vector<string>& wordList) {
8         unordered_set<string> words(wordList.begin(), wordList.end());
9
10        if (!words.count(endWord))
11            return 0;
12
13        queue<pair<string, int>> q;
14        q.push({beginWord, 1});
15
16        while (!q.empty()) {
17            auto [word, dist] = q.front();
18            q.pop();
19
20            if (word == endWord)
21                return dist;
22
23            for (int i = 0; i < word.size(); i++) {
24                char original = word[i];
25
26                for (char c = 'a'; c <= 'z'; c++) {
27                    if (c == original) continue;
28
29                    word[i] = c;
30
31                    if (words.count(word)) {
32                        q.push({word, dist + 1});
33                        words.erase(word); // mark as visited
34                    }
35                }
36
37                word[i] = original;
38            }
39        }
40
41        return 0;
42    }
43 };
```

3.4 BFS - Boss

Resuelve "Boss" (Day 2, problema C - Maratona de Programacao da SBC 2013): en un organigrama de N empleados donde X maneja directamente a Y, la operacion "T A B" intercambia las POSICIONES que ocupan A y B dentro del organigrama (que nunca cambia), y "P E" pregunta la edad del jefe mas joven, directo o indirecto, de E. Guarda el grafo invertido rev[Y] = jefes directos de una posicion, y los arreglos empPos/posEmp para saber que empleado ocupa cada posicion despues de cada swap. Cada consulta hace BFS desde la posicion de E sobre rev, visita todas las posiciones jefe (ancestros) y se queda con la edad minima entre los empleados que hoy las ocupan.

C++\Training_Camp_2026\Day 2\C - Boss.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int N, M, I;
9
10    // El problema puede contener varios casos de prueba
```

3.4 BFS - Boss (cont.)

```
11    while (cin >> N >> M >> I) {
12
13        // age[i] = edad del empleado i
14        vector<int> age(N + 1);
15        for (int i = 1; i <= N; i++)
16            cin >> age[i];
17
18        /*
19         rev representa el grafo invertido.
20
21         Si X maneja directamente a Y:
22             X -> Y
23
24         entonces almacenamos:
25
26             rev[Y].push_back(X)
27
28         porque durante una consulta queremos subir por la cadena
29         de mando (buscar todos los jefes).
30        */
31        vector<vector<int>> rev(N + 1);
32
33        for (int i = 0; i < M; i++) {
34            int X, Y;
35            cin >> X >> Y;
36            rev[Y].push_back(X);
37        }
38
39        /*
40         El grafo nunca cambia.
41
42         Lo único que cambia es qué empleado ocupa cada posición
43         dentro del organigrama.
44
45         empPos[e] = posición donde está actualmente el empleado e.
46
47         posEmp[p] = empleado que ocupa actualmente la posición p.
48        */
49
50        vector<int> empPos(N + 1);
51        vector<int> posEmp(N + 1);
52
53        // Inicialmente cada empleado está en su propia posición.
54        for (int i = 1; i <= N; i++) {
55            empPos[i] = i;
56            posEmp[i] = i;
57        }
58
59        while (I--) {
60
61            char op;
62            cin >> op;
63
64            // Cambio de posiciones
65            if (op == 'T') {
66
67                int A, B;
68                cin >> A >> B;
69
70                // Posiciones actuales de ambos empleados
71                int posA = empPos[A];
72                int posB = empPos[B];
73
74                // Intercambiar las posiciones de los empleados
75                swap(empPos[A], empPos[B]);
76
77                // Actualizar qué empleado queda en cada posición
78                swap(posEmp[posA], posEmp[posB]);
79            }
80
81            // Consulta
82            else {
```

3.4 BFS - Boss (cont.)

```

83
84     int E;
85     cin >> E;
86
87     // Posición donde se encuentra actualmente el empleado E
88     int start = empPos[E];
89
90     vector<int> vis(N + 1, 0);
91     queue<int> q;
92
93     q.push(start);
94     vis[start] = true;
95
96     int ans = INT_MAX;
97
98     /*
99     Hacemos BFS sobre el grafo invertido.
100
101     Desde la posición del empleado vamos visitando
102     todos los jefes (directos e indirectos).
103
104     Cada nodo del BFS representa una POSICIÓN.
105
106     Para conocer la edad del jefe debemos averiguar
107     qué empleado ocupa actualmente esa posición.
108     */
109     while (!q.empty()) {
110
111         int u = q.front();
112         q.pop();
113
114         // Revisar todos los jefes directos
115         for (int bossPos : rev[u]) {
116
117             if (vis[bossPos])
118                 continue;
119
120             vis[bossPos] = true;
121             q.push(bossPos);
122
123             // Empleado que ocupa esa posición
124             int bossEmp = posEmp[bossPos];
125
126             // Actualizar la edad mínima encontrada
127             ans = min(ans, age[bossEmp]);
128         }
129     }
130
131     // Si nunca encontramos un jefe
132     if (ans == INT_MAX)
133         cout << "*\n";
134     else
135         cout << ans << "\n";
136 }
137 }
138 }
139
140 return 0;
141 }

```

3.5 BFS - Bacon Number

Resuelve "Bacon Number" (Day 3, problema B - Maratona SBC de Programacao 2024): dadas N películas con sus actores, hay que responder Q consultas que piden conectar dos actores x e y mediante una secuencia alternada actor-película-actor-...-película-actor, donde cada actor consecutivo comparte la película que los une con el siguiente, o determinar que no es posible. Construye un grafo cuyos nodos son películas y donde hay arista entre dos películas si comparten al menos un actor (agrupando películas por actor en actorMovies), y corre un BFS desde cada película (dist, parent) para precalcular caminos mínimos entre películas; luego, para cada consulta, prueba todas las combinaciones de películas donde aparecen x e y y reconstruye el camino de actores/películas siguiendo el arreglo parent.

C++\Training_Camp_2026\Day 3\B - Bacon Number.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int INF = 1e9;
5
6  int main() {
7      ios::sync_with_stdio(false);
8      cin.tie(nullptr);
9
10     int N, M;
11     cin >> N >> M;
12
13     vector<vector<int>> movieActors(N + 1);
14     unordered_map<int, vector<int>> actorMovies;
15
16     for (int i = 1; i <= N; i++) {
17         int k;
18         cin >> k;
19         movieActors[i].resize(k);
20         for (int j = 0; j < k; j++) {
21             cin >> movieActors[i][j];
22             actorMovies[movieActors[i][j]].push_back(i);
23         }
24     }
25
26     // actor común entre películas
27     vector<vector<int>> common(N + 1, vector<int>(N + 1, 0));
28     vector<vector<int>> adj(N + 1);
29
30     for (auto &it : actorMovies) {
31         auto &v = it.second;
32         for (int i = 0; i < (int)v.size(); i++) {
33             for (int j = i + 1; j < (int)v.size(); j++) {
34                 int a = v[i], b = v[j];
35                 if (!common[a][b]) {
36                     common[a][b] = common[b][a] = it.first;
37                     adj[a].push_back(b);
38                     adj[b].push_back(a);
39                 }
40             }
41         }
42     }
43
44     vector<vector<int>> dist(N + 1, vector<int>(N + 1, INF));
45     vector<vector<int>> parent(N + 1, vector<int>(N + 1, -1));
46
47     for (int s = 1; s <= N; s++) {
48         queue<int> q;
49         dist[s][s] = 0;
50         q.push(s);
51
52         while (!q.empty()) {
53             int u = q.front();
54             q.pop();
55
56             for (int v : adj[u]) {
57                 if (dist[s][v] == INF) {
58                     dist[s][v] = dist[s][u] + 1;
59                     parent[s][v] = u;
60                     q.push(v);

```

3.5 BFS - Bacon Number (cont.)

```

61     }
62     }
63 }
64 }
65
66 int Q;
67 cin >> Q;
68
69 while (Q-- > 0) {
70     int x, y;
71     cin >> x >> y;
72
73     if (!actorMovies.count(x) || !actorMovies.count(y)) {
74         cout << -1 << '\n';
75         continue;
76     }
77
78     int bestA = -1, bestB = -1, best = INF;
79
80     for (int a : actorMovies[x])
81         for (int b : actorMovies[y])
82             if (dist[a][b] < best) {
83                 best = dist[a][b];
84                 bestA = a;
85                 bestB = b;
86             }
87
88     if (best == INF) {
89         cout << -1 << '\n';
90         continue;
91     }
92
93     vector<int> movies;
94     int cur = bestB;
95     while (cur != bestA) {
96         movies.push_back(cur);
97         cur = parent[bestA][cur];
98     }
99     movies.push_back(bestA);
100     reverse(movies.begin(), movies.end());
101
102     vector<int> out;
103
104     out.push_back(x);
105
106     if (x != common[movies[0]][movies[0]])
107         ; // nada
108
109     int currentActor = x;
110
111     if (actorMovies[x][0] != movies[0]) {
112         // x aparece en movies[0] por construcción
113     }
114
115     for (int i = 0; i + 1 < (int)movies.size(); i++) {
116         int actor = common[movies[i]][movies[i + 1]];
117         if (currentActor != actor) {
118             out.push_back(movies[i]);
119             out.push_back(actor);
120             currentActor = actor;
121         } else {
122             out.push_back(movies[i]);
123         }
124     }
125
126     out.push_back(movies.back());
127
128     if (currentActor != y)
129         out.push_back(y);
130
131     int actors = 0;
132     for (int i = 0; i < (int)out.size(); i += 2)

```

3.5 BFS - Bacon Number (cont.)

```

133     actors++;
134
135     cout << actors << '\n';
136
137     for (int i = 0; i < (int)out.size(); i++) {
138         if (i) cout << ' ';
139         cout << out[i];
140     }
141     cout << '\n';
142 }
143 }

```

3.6 BFS Backtracking - Find Ladders

Resuelve "Word Ladder II": encuentra TODAS las secuencias de transformacion mas cortas de beginWord a endWord cambiando una letra a la vez. BFS por niveles guardando los padres de cada palabra, y luego backtracking desde endWord para reconstruir todos los caminos minimos.

C++\LeetCode\findLadders.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 class Solution {
6 public:
7     vector<vector<string>> ans;
8     unordered_map<string, vector<string>> parents;
9
10    void buildPaths(string word,
11                   string beginWord,
12                   vector<string>& path) {
13
14        if (word == beginWord) {
15            vector<string> cur = path;
16            reverse(cur.begin(), cur.end());
17            ans.push_back(cur);
18            return;
19        }
20
21        for (auto& p : parents[word]) {
22            path.push_back(p);
23            buildPaths(p, beginWord, path);
24            path.pop_back();
25        }
26    }
27
28    vector<vector<string>> findLadders(string beginWord,
29                                     string endWord,
30                                     vector<string>& wordList) {
31
32        unordered_set<string> words(wordList.begin(), wordList.end());
33
34        if (!words.count(endWord))
35            return {};
36
37        queue<string> q;
38        q.push(beginWord);
39
40        unordered_set<string> visited;
41        visited.insert(beginWord);
42
43        bool found = false;
44
45        while (!q.empty() && !found) {
46
47            int sz = q.size();
48
49            unordered_set<string> levelVisited;
50
51            while (sz-- > 0) {
52
53                string word = q.front();

```

3.6 BFS Backtracking - Find Ladders (cont.)

```

54     q.pop();
55
56     string original = word;
57
58     for (int i = 0; i < word.size(); i++) {
59
60         char old = word[i];
61
62         for (char c = 'a'; c <= 'z'; c++) {
63
64             if (c == old) continue;
65
66             word[i] = c;
67
68             if (!words.count(word))
69                 continue;
70
71             if (!visited.count(word)) {
72
73                 if (!levelVisited.count(word)) {
74                     q.push(word);
75                     levelVisited.insert(word);
76                 }
77
78                 parents[word].push_back(original);
79
80                 if (word == endWord)
81                     found = true;
82             }
83         }
84
85         word[i] = old;
86     }
87 }
88
89 for (auto& w : levelVisited)
90     visited.insert(w);
91 }
92
93 if (!found)
94     return {};
95
96 vector<string> path = {endWord};
97 buildPaths(endWord, beginWord, path);
98
99 return ans;
100 }
101 };

```

3.7 BFS sobre Grafo Producto (dos fichas a la vez) - Infiltration Route

Resuelve "Infiltration Route" (problema I, Regionals 2025): hay que mover DOS posiciones a la vez por un grafo dirigido y llegar a una configuracion objetivo. LA TECNICA ES EL GRAFO PRODUCTO: el estado no es un nodo sino un PAR (p1, p2), mas alguna bandera extra segun lo que pida el problema. El BFS corre sobre esos estados, y desde cada uno se ramifica moviendo una ficha o la otra. Con n nodos son n^2 estados, lo que acota a unos pocos cientos, y por eso MAXN es 505. CUANDO USARLO: cualquier problema con dos entidades que se mueven y que interactuan (dos robots que no pueden chocar, un perseguidor y un perseguido, dos tokens que deben encontrarse). Si las entidades fueran independientes se resolverian por separado; el grafo producto es justamente para cuando NO lo son. El BFS da el camino minimo porque todos los movimientos cuestan lo mismo, y la reconstruccion se hace guardando de que estado se vino, igual que en un BFS normal pero con el estado compuesto. El toclear sirve para limpiar solo lo que se toco entre casos, en vez de reiniciar los n^2 estados enteros: util cuando hay muchos casos de prueba y el grafo es grande. OJO: usa structured bindings (auto [a, b]), que piden C++17. En el juez compila, pero con un g++ viejo hay que volver a .first y .second.

C++\CP\Regionals 2025V - Infiltration Route.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 505;
5

```

3.7 BFS sobre Grafo Producto (dos fichas a la vez) - Infiltration Route (cont.)

```

6  bool seen[MAXN][MAXN][2];
7  array<int, 3> bck[MAXN][MAXN][2];
8
9  int main() {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     int n, m;
14     cin >> n >> m;
15
16     vector<vector<int>> adj(2 * n);
17
18     for (int i = 0; i < m; i++) {
19         int s, t;
20         cin >> s >> t;
21
22         --s;
23         --t;
24
25         adj[s].push_back(t);
26     }
27
28     bool can = false;
29     vector<int> route;
30
31     queue<array<int, 3>> q;
32     vector<array<int, 3>> toclear;
33
34     /*
35      * cross = unica arista que pasa de:
36      *
37      * [0 ... n-1]
38      *
39      * a:
40      *
41      * [n ... 2n-1]
42      */
43
44     for (int cross = n + 1; cross < 2 * n && !can; cross++) {
45
46         q.push({0, cross - n, 0});
47
48         seen[0][cross - n][0] = true;
49
50         bck[0][cross - n][0] = {
51             -1, -1, 0
52         };
53
54         while (!q.empty()) {
55
56             auto [p1, p2, t] = q.front();
57             q.pop();
58
59             toclear.push_back({p1, p2, t});
60
61             // -----
62             // Mover p1
63             // -----
64
65             if (p1 < p2 && t == 0) {
66
67                 for (auto v : adj[p1]) {
68
69                     if (v < n && v != p2) {
70
71                         if (!seen[v][p2][0]) {
72
73                             seen[v][p2][0] = true;
74
75                             bck[v][p2][0] = {
76                                 p1, p2, t
77                             };
78

```

3.7 BFS sobre Grafo Producto (dos fichas a la vez) - Infiltration Route (cont.)

```

78
79         q.push({v, p2, 0});
80     }
81 }
82 }
83 }
84
85 // -----
86 // Mover p2
87 // -----
88
89 if (t == 1 || (t == 0 && p1 < p2)) {
90     for (auto v : adj[n + p2]) {
91         int np2 = v - n;
92         if (np2 != p1) {
93             if (!seen[p1][np2][1]) {
94                 seen[p1][np2][1] = true;
95                 bck[p1][np2][1] = {
96                     p1, p2, t
97                 };
98                 q.push({p1, np2, 1});
99             }
100         }
101     }
102 }
103
104 // -----
105 // Caso p1 > p2
106 // -----
107
108 else if (t == 0 && p1 > p2) {
109     for (auto v : adj[n + p2]) {
110         int np2 = v - n;
111         if (np2 != p1) {
112             if (!seen[p1][np2][t]) {
113                 seen[p1][np2][t] = true;
114                 bck[p1][np2][t] = {
115                     p1, p2, t
116                 };
117                 q.push({p1, np2, t});
118             }
119         }
120     }
121 }
122
123 // -----
124 // Buscamos:
125 // -----
126 // p1 = bc
127 // p2 = n-1
128 // -----
129 // y la arista bc -> cross.
130 // -----
131
132 for (int bc = 0; bc < n - 1 && !can; bc++) {
133     for (int t = 0; t <= 1 && !can; t++) {

```

3.7 BFS sobre Grafo Producto (dos fichas a la vez) - Infiltration Route (cont.)

```

134         if (!seen[bc][n - 1][t])
135             continue;
136
137         for (auto v : adj[bc]) {
138             if (v != cross)
139                 continue;
140
141             can = true;
142
143             // -----
144             // Reconstruir
145             // -----
146
147             array<int, 3> cur = {
148                 bc,
149                 n - 1,
150                 t
151             };
152
153             while (cur[0] != -1) {
154                 route.push_back(cur[0]);
155                 route.push_back(n + cur[1]);
156
157                 cur =
158                     bck[cur[0]][cur[1]][cur[2]];
159             }
160
161             sort(route.begin(), route.end());
162
163             route.erase(
164                 unique(route.begin(), route.end()),
165                 route.end()
166             );
167
168             break;
169         }
170     }
171 }
172
173 // -----
174 // Limpiar solamente los estados visitados
175 // -----
176
177 for (auto [u, v, t] : toclear) {
178     seen[u][v][t] = false;
179
180     bck[u][v][t] = {
181         0, 0, 0
182     };
183 }
184
185 toclear.clear();
186 }
187
188 if (can) {
189     cout << route.size() << '\n';
190
191     for (int x : route)
192         cout << x + 1 << ' ';
193
194     cout << '\n';
195 } else {
196     cout << "*\n";
197 }
198 }
199
200

```

3.7 BFS sobre Grafo Producto (dos fichas a la vez) - Infiltration Route (cont.)

```

222 return 0;
223 }

```

3.8 Biconnected Components (Block-Cut Tree) + LCA - Hedge Mazes

Resuelve "Hedge Mazes" (Day 4, problema N - Contest 04 [Avanzados]): dado un laberinto de R salas y C corredores, para cada consulta (S, T) hay que decidir si existe exactamente un camino simple entre S y T (si hay un ciclo en el medio, existiría mas de un camino simple). Construye las componentes biconexas (buildBiconnected, via low-link) y arma el arbol de bloques y puntos de corte (block-cut tree), donde cada bloque de tamaño ≥ 3 vertices representa un ciclo; usando LCA con binary lifting (bfsBuildLCA/queryPath) calcula el maximo tamaño de bloque en el camino entre S y T, y responde 'Y' solo si ese maximo es ≤ 2 (solo aristas, sin ciclos).

C++\Training_Camp_2026\Day 4\N - Hedge Mazes.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static char buf_[1 << 22];
5 static int bufLen_, bufPos_;
6 inline int gc() {
7     if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!bufLen_) return -1; }
8     return buf_[bufPos_++];
9 }
10 inline int readInt() {
11     int c = gc();
12     while (c != '-' && (c < '0' || c > '9') && c != '-') c = gc();
13     bool neg = false; if (c == '-') { neg = true; c = gc(); }
14     int x = 0;
15     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
16     return neg ? -x : x;
17 }
18
19 int R, C, Q;
20 vector<pair<int,int>> edges; // edge id -> (u,v)
21 vector<vector<pair<int,int>>> adj; // adj[u] -> list of (v, edgeId)
22 vector<int> disc_, low_;
23 vector<char> visited_, isCut;
24 int timer_;
25
26 vector<int> posOfVertex; // "position" node id in block-cut tree for each original vertex
27 // for a cut vertex v: position = v itself (1..R)
28 // for a non-cut vertex v: position = its unique block's node id (R+1..R+numBlocks)
29 vector<int> blockWeight; // weight[nodeId] : block-size for block nodes, 0 for cut-vertex nodes (irrelevant)
30 vector<vector<int>> treeAdj; // adjacency list over node ids 1..(R+numBlocks)
31 vector<int> treeCompId; // which tree (of the block-cut forest) each node belongs to
32 int numBlocks;
33
34 vector<int> lastMarked; // lastMarked[v] = block index that last touched vertex v (avoids duplicate adds)
35
36 void buildBiconnected() {
37     disc_.assign(R + 1, -1);
38     low_.assign(R + 1, -1);
39     visited_.assign(R + 1, 0);
40     isCut.assign(R + 1, 0);
41     posOfVertex.assign(R + 1, -1);
42     lastMarked.assign(R + 1, -1);
43     timer_ = 0;
44     numBlocks = 0;
45
46     vector<vector<int>> blockVerts; // blockVerts[b] = list of vertices in block b (0-indexed b)
47     vector<int> edgeStack;
48
49     for (int s = 1; s <= R; s++) {
50         if (visited_[s]) continue;
51         // iterative DFS
52         vector<array<int,4>> stk; // {u, nextIdx, parentEdge, parentVertex}
53         visited_[s] = 1;

```

3.8 Biconnected Components (Block-Cut Tree) + LCA - Hedge Mazes (cont.)

```

54     disc_[s] = low_[s] = timer_++;
55     stk.push_back({s, 0, -1, -1});
56     int rootChildren = 0;
57
58     while (!stk.empty()) {
59         auto &fr = stk.back();
60         int u = fr[0];
61         if (fr[1] < (int)adj[u].size()) {
62             int v = adj[u][fr[1]].first;
63             int eid = adj[u][fr[1]].second;
64             fr[1]++;
65             if (eid == fr[2]) continue; // skip edge back to parent
66             if (!visited_[v]) {
67                 visited_[v] = 1;
68                 disc_[v] = low_[v] = timer_++;
69                 edgeStack.push_back(eid);
70                 if (u == s) rootChildren++;
71                 stk.push_back({v, 0, eid, u});
72             } else if (disc_[v] < disc_[u]) {
73                 edgeStack.push_back(eid);
74                 low_[u] = min(low_[u], disc_[v]);
75             }
76         } else {
77             int parentEdge = fr[2];
78             int parentVertex = fr[3];
79             int uLow = low_[u];
80             stk.pop_back();
81             if (!stk.empty()) {
82                 int p = parentVertex;
83                 low_[p] = min(low_[p], uLow);
84                 if (uLow >= disc_[p]) {
85                     if (p != s) isCut[p] = 1;
86                     // pop edges up to and including parentEdge -> forms one block
87                     int b = numBlocks++;
88                     blockVerts.push_back({});
89                     while (!edgeStack.empty()) {
90                         int e = edgeStack.back();
91                         edgeStack.pop_back();
92                         int a1 = edges[e].first, a2 = edges[e].second;
93                         if (lastMarked[a1] != b) { lastMarked[a1] = b; blockVerts[b].push_back(a1); }
94                         if (lastMarked[a2] != b) { lastMarked[a2] = b; blockVerts[b].push_back(a2); }
95                         if (e == parentEdge) break;
96                     }
97                 }
98             }
99         }
100     }
101     if (rootChildren > 1) isCut[s] = 1;
102 }
103
104 // assign block weights and non-cut vertex positions
105 blockWeight.assign(1 + R + numBlocks, 0);
106 for (int b = 0; b < numBlocks; b++) {
107     int nodeId = R + 1 + b;
108     blockWeight[nodeId] = (int)blockVerts[b].size();
109     for (int v : blockVerts[b]) {
110         if (!isCut[v]) {
111             posOfVertex[v] = nodeId;
112         }
113     }
114 }
115 for (int v = 1; v <= R; v++) {
116     if (isCut[v]) posOfVertex[v] = v;
117     else if (posOfVertex[v] == -1) {
118         // isolated vertex (no edges at all) -> give it its own trivial "block" node conceptual ly.
119         // We reuse it as its own position id (acts like a size-1 pseudo node, weight 0, never matches
120         // any other vertex's position unless that vertex IS this same isolated vertex, which c

```

3.8 Biconnected Components (Block-Cut Tree) + LCA - Hedge Mazes (cont.)

```

// happen for a distinct query since S != T is guaranteed... but S or T might indepen
21 ntly
22 // be isolated vertices in different queries).
    posOfVertex[v] = v; // treat as its own node; weight will be 0 (irrelevant) and it will
23 simply
24 // have no tree edges, forming its own singleton tree.
25 }
26 }
27
28 // Build tree adjacency
29 treeAdj.assign(1 + R + numBlocks, {});
30 for (int b = 0; b < numBlocks; b++) {
31     int nodeId = R + 1 + b;
32     for (int v : blockVerts[b]) {
33         if (isCut[v]) {
34             treeAdj[v].push_back(nodeId);
35             treeAdj[nodeId].push_back(v);
36         }
37     }
38 }
39 }
40
41 // LCA with binary lifting, tracking max blockWeight along the path (node weights only matter for b
    lock nodes;
42 // cut-vertex nodes / isolated singleton nodes have weight 0 which never triggers the "bad" conditi
    on).
43 int LOG_;
44 vector<vector<int>> up_;
45 vector<vector<int>> upMax_; // upMax_[k][v] = max weight among the 2^k nodes starting at v going up
    ward (v itself
46 // included at k=0 as weight[v], combined appropriately)
47 vector<int> depth_;
48
49 void bfsBuildLCA(int totalNodes) {
50     LOG_ = 1;
51     while ((1 << LOG_) < totalNodes + 1) LOG_++;
52     up_.assign(LOG_, vector<int>(totalNodes + 1, 0));
53     upMax_.assign(LOG_, vector<int>(totalNodes + 1, 0));
54     depth_.assign(totalNodes + 1, -1);
55     treeCompId.assign(totalNodes + 1, -1);
56
57     int curComp = -1;
58     for (int start = 1; start <= totalNodes; start++) {
59         if (depth_[start] != -1) continue;
60         curComp++;
61         queue<int> bfsq;
62         depth_[start] = 0;
63         up_[0][start] = 0; // root's parent = 0 (sentinel "no parent")
64         upMax_[0][start] = blockWeight[start];
65         treeCompId[start] = curComp;
66         bfsq.push(start);
67         while (!bfsq.empty()) {
68             int u = bfsq.front(); bfsq.pop();
69             for (int w : treeAdj[u]) {
70                 if (depth_[w] == -1) {
71                     depth_[w] = depth_[u] + 1;
72                     up_[0][w] = u;
73                     upMax_[0][w] = max(blockWeight[w], blockWeight[u]);
74                     treeCompId[w] = curComp;
75                     bfsq.push(w);
76                 }
77             }
78         }
79     }
80
81     for (int k = 1; k < LOG_; k++) {
82         for (int v = 1; v <= totalNodes; v++) {
83             int mid = up_[k-1][v];
84             if (mid == 0) { up_[k][v] = 0; upMax_[k][v] = upMax_[k-1][v]; }
85             else {
86                 up_[k][v] = up_[k-1][mid];
87                 upMax_[k][v] = max(upMax_[k-1][v], upMax_[k-1][mid]);

```

3.8 Biconnected Components (Block-Cut Tree) + LCA - Hedge Mazes (cont.)

```

88     }
89 }
90 }
91 }
92
93 // returns {lca, maxWeightOnPath} or {-1, -1} if different components
94 pair<int, int> queryPath(int u, int v) {
95     if (treeCompId[u] != treeCompId[v]) return {-1, -1};
96     int best = max(blockWeight[u], blockWeight[v]);
97     if (depth_[u] < depth_[v]) swap(u, v);
98     int diff = depth_[u] - depth_[v];
99     for (int k = 0; k < LOG_; k++) {
100         if ((diff >> k) & 1) {
101             best = max(best, upMax_[k][u]);
102             u = up_[k][u];
103         }
104     }
105     if (u == v) return {u, best};
106     for (int k = LOG_ - 1; k >= 0; k--) {
107         if (up_[k][u] != up_[k][v]) {
108             best = max({best, upMax_[k][u], upMax_[k][v]});
109             u = up_[k][u];
110             v = up_[k][v];
111         }
112     }
113     best = max({best, blockWeight[up_[0][u]], upMax_[0][u], upMax_[0][v]});
114     return {up_[0][u], best};
115 }
116
117 int main() {
118     while (true) {
119         R = readInt(); C = readInt(); Q = readInt();
120         if (R == 0 && C == 0 && Q == 0) break;
121
122         edges.assign(C, {0, 0});
123         adj.assign(R + 1, {});
124         for (int i = 0; i < C; i++) {
125             int a = readInt(), b = readInt();
126             edges[i] = {a, b};
127             adj[a].push_back({b, i});
128             adj[b].push_back({a, i});
129         }
130
131         buildBiconnected();
132         int totalNodes = R + numBlocks;
133         bfsBuildLCA(totalNodes);
134
135         string out;
136         for (int q = 0; q < Q; q++) {
137             int s = readInt(), t = readInt();
138             int ps = posOfVertex[s], pt = posOfVertex[t];
139             auto {lca, mx} = queryPath(ps, pt);
140             if (lca == -1) {
141                 out += "N\n";
142             } else {
143                 out += (mx <= 2) ? "Y\n" : "N\n";
144             }
145         }
146         out += "-\n";
147         fwrite(out.data(), 1, out.size(), stdout);
148     }
149     return 0;
150 }

```

3.9 Bipartite Matching - Attacking Rooks

Resuelve "Attacking rooks" (Day 7, problema I - Contest 07 [Avanzados]): en un tablero NxN con algunas casillas ocupadas por peones ('X'), dos torres se atacan si comparten fila o columna sin un peon entre medio; hay que colocar el maximo numero de torres en casillas vacias sin que se ataquen entre si. Modela cada tramo horizontal maximal de casillas vacias entre peones como un nodo "fila" (rowId) y cada tramo vertical como un nodo "columna" (colId); cada casilla vacia conecta su tramo de fila con su tramo de columna. El maximo de torres no atacadas es entonces el maximo matching bipartito entre tramos-fila y tramos-columna, calculado con el algoritmo de Hopcroft-Karp (BFS por capas + DFS de caminos aumentantes, struct HopcroftKarp).

C++\Training_Camp_2026\Day 7\I - Attacking rooks.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct HopcroftKarp {
5     int n, m;
6     vector<vector<int>> adj;
7     vector<int> matchL, matchR, dist;
8     const int INF = INT_MAX;
9     HopcroftKarp(int n=0, int m=0): n(n), m(m), adj(n), matchL(n,-1), matchR(m,-1), dist(n) {}
10    void addEdge(int u,int v){ adj[u].push_back(v); }
11    bool bfs(){
12        queue<int> q;
13        for(int u=0;u<n;u++){
14            if(matchL[u]==-1){ dist[u]=0; q.push(u); }
15            else dist[u]=INF;
16        }
17        bool found=false;
18        while(!q.empty()){
19            int u=q.front();q.pop();
20            for(int v: adj[u]){
21                int w = matchR[v];
22                if(w==-1){ found=true; }
23                else if(dist[w]==INF){ dist[w]=dist[u]+1; q.push(w); }
24            }
25        }
26        return found;
27    }
28    bool dfs(int u){
29        for(int v: adj[u]){
30            int w = matchR[v];
31            if(w==-1 || (dist[w]==dist[u]+1 && dfs(w))){
32                matchL[u]=v; matchR[v]=u;
33                return true;
34            }
35        }
36        dist[u]=INF;
37        return false;
38    }
39    int maxMatching(){
40        int result=0;
41        while(bfs()){
42            for(int u=0;u<n;u++){
43                if(matchL[u]==-1){
44                    if(dfs(u)) result++;
45                }
46            }
47        }
48        return result;
49    }
50 };
51
52 int main(){
53     ios::sync_with_stdio(false);
54     cin.tie(nullptr);
55     int N;
56     while(cin >> N){
57         vector<string> grid(N);
58         for(auto &s: grid) cin >> s;
59
60         vector<vector<int>> rowId(N, vector<int>(N,-1));
61         vector<vector<int>> colId(N, vector<int>(N,-1));
```

3.9 Bipartite Matching - Attacking Rooks (cont.)

```
62     int R=0, C=0;
63
64     // etiqueta cada tramo horizontal de casillas vacías
65     for(int i=0;i<N;i++){
66         int j=0;
67         while(j<N){
68             if(grid[i][j]=='X'){ j++; continue; }
69             while(j<N && grid[i][j]=='.'){ rowId[i][j]=R; j++; }
70             R++;
71         }
72     }
73     // etiqueta cada tramo vertical de casillas vacías
74     for(int j=0;j<N;j++){
75         int i=0;
76         while(i<N){
77             if(grid[i][j]=='X'){ i++; continue; }
78             while(i<N && grid[i][j]=='.'){ colId[i][j]=C; i++; }
79             C++;
80         }
81     }
82
83     HopcroftKarp hk(R,C);
84     for(int i=0;i<N;i++){
85         for(int j=0;j<N;j++){
86             if(grid[i][j]=='.')
87                 hk.addEdge(rowId[i][j], colId[i][j]);
88         }
89     }
90     cout << hk.maxMatching() << "\n";
91 }
92 return 0;
93 }
```

3.10 Bipartite Matching - Knights In The Board

Resuelve "Knights In The Board" (Day 7, problema K - Contest 07 [Avanzados]): dado un tablero NxN con K caballos ya colocados, hay que hallar el minimo numero de caballos a remover para que ninguno ataque a otro (movimiento en L de ajedrez). El grafo de ataques entre caballos es bipartito, ya que un caballo en una casilla de un color (segun (fila+columna)%2, ver color_) solo puede atacar caballos del color opuesto. Por el teorema de Konig, el minimo vertex cover (minimo de caballos a remover para romper todos los conflictos) equivale al maximo matching bipartito. Construye adj solo desde los caballos de color 0 hacia sus vecinos alcanzables con movimiento de caballo, y calcula el maximo matching con el algoritmo de Kuhn (tryKuhn), que es directamente la respuesta pedida.

C++\Training_Camp_2026\Day 7\K - Knights In The Board.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int N, K;
5 map<pair<int,int>, int> pos2idx;
6 vector<int> R, C, color_;
7 vector<vector<int>> adj;
8 vector<int> matchRight;
9 vector<bool> visited;
10
11 int dr[8] = {1,1,-1,-1,2,2,-2,-2};
12 int dc[8] = {2,-2,2,-2,1,-1,1,-1};
13
14 bool tryKuhn(int u) {
15     for (int v : adj[u]) {
16         if (!visited[v]) {
17             visited[v] = true;
18             if (matchRight[v] == -1 || tryKuhn(matchRight[v])) {
19                 matchRight[v] = u;
20                 return true;
21             }
22         }
23     }
24     return false;
25 }
26
27 int main(){
```

3.10 Bipartite Matching - Knights In The Board (cont.)

```

28 cin >> N >> K;
29 R.resize(K); C.resize(K); color_.resize(K);
30 for (int i = 0; i < K; i++) {
31     cin >> R[i] >> C[i];
32     pos2idx[{R[i], C[i]}] = i;
33     color_[i] = (R[i] + C[i]) % 2;
34 }
35 adj.assign(K, {});
36 for (int i = 0; i < K; i++) {
37     if (color_[i] != 0) continue;
38     for (int d = 0; d < 8; d++) {
39         auto it = pos2idx.find({R[i]+dr[d], C[i]+dc[d]});
40         if (it != pos2idx.end()) adj[i].push_back(it->second);
41     }
42 }
43 matchRight.assign(K, -1);
44 int matching = 0;
45 for (int i = 0; i < K; i++) {
46     if (color_[i] != 0 || adj[i].empty()) continue;
47     visited.assign(K, false);
48     if (tryKuhn(i)) matching++;
49 }
50 cout << matching << "\n";
51 }

```

3.11 Bipartite Matching (Algoritmo de Kuhn) - School Dance

Resuelve "School Dance" (Day 5, problema G - Contest 05 [Avanzados]): hay N chicos y M chicas, y una lista de K parejas potenciales (chico, chica) dispuestos a bailar juntos; hay que hallar el máximo número de parejas de baile que se pueden formar (cada chico y cada chica en a lo sumo una pareja) y mostrar una asignación que lo logre. Implementa matching bipartito con el algoritmo de Kuhn: para cada chico intenta encontrar, vía la función tryMatch (DFS de caminos aumentantes sobre el arreglo matchGirl), una chica libre o reasignable, marcando visited para no repetir chicos durante la búsqueda de un mismo camino aumentante.

C++\Training_Camp_2026\Day 5\G - School Dance.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int n, m, k;
5 vector<vector<int>> pairs;
6 vector<int> matchGirl;
7 vector<bool> visited;
8
9 bool tryMatch(int boy) {
10     if (visited[boy]) return false;
11     visited[boy] = true;
12
13     for (int girl : pairs[boy]) {
14         if (matchGirl[girl] == 0 || tryMatch(matchGirl[girl])) {
15             matchGirl[girl] = boy;
16             return true;
17         }
18     }
19     return false;
20 }
21
22 int main() {
23     ios::sync_with_stdio(false);
24     cin.tie(nullptr);
25
26     cin >> n >> m >> k;
27
28     pairs.assign(n + 1, {});
29
30     for (int i = 0; i < k; i++) {
31         int boy, girl;
32         cin >> boy >> girl;
33         pairs[boy].push_back(girl);
34     }
35
36     matchGirl.assign(m + 1, 0);

```

3.11 Bipartite Matching (Algoritmo de Kuhn) - School Dance (cont.)

```

37
38     int maximumPairs = 0;
39
40     for (int boy = 1; boy <= n; boy++) {
41         visited.assign(n + 1, false);
42         if (tryMatch(boy)) {
43             maximumPairs++;
44         }
45     }
46
47     cout << maximumPairs << endl;
48
49     for (int girl = 1; girl <= m; girl++) {
50         if (matchGirl[girl] != 0) {
51             cout << matchGirl[girl] << " " << girl << endl;
52         }
53     }
54
55     return 0;
56 }

```

3.12 Circuito Euleriano (Algoritmo de Hierholzer) - Mail Delivery

Resuelve "Mail Delivery" (Day 5, problema J - Contest 05 [Avanzados]): dado un plano de n cruces y m calles bidireccionales, hay que encontrar una ruta que empiece y termine en la oficina postal (cruce 1) y recorra cada calle exactamente una vez, o determinar que es imposible. Primero verifica las condiciones de existencia de un circuito euleriano: todo vértice debe tener grado par, y todos los vértices con aristas deben ser alcanzables desde el vértice 1 (BFS). Luego construye el circuito con el algoritmo de Hierholzer de forma iterativa (pila st y arreglo ptr por vértice, marcando usedEdge), y revierte el resultado para obtener la ruta final.

C++\Training_Camp_2026\Day 5\J - Mail Delivery.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios_base::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n, m;
9     cin >> n >> m;
10
11     vector<vector<pair<int,int>>> adj(n + 1); // (to, edge_id)
12     vector<int> deg(n + 1, 0);
13
14     for (int i = 0; i < m; i++) {
15         int a, b;
16         cin >> a >> b;
17         adj[a].push_back({b, i});
18         adj[b].push_back({a, i});
19         deg[a]++;
20         deg[b]++;
21     }
22
23     // 1) every vertex must have even degree
24     for (int v = 1; v <= n; v++) {
25         if (deg[v] % 2 != 0) {
26             cout << "IMPOSSIBLE\n";
27             return 0;
28         }
29     }
30
31     // 2) all vertices that have edges must be reachable from vertex 1
32     // (this also catches the case where vertex 1 itself has no edges but m > 0)
33     vector<bool> visited(n + 1, false);
34     queue<int> bfsQ;
35     bfsQ.push(1);
36     visited[1] = true;
37     int reached = 0;
38     while (!bfsQ.empty()) {
39         int u = bfsQ.front(); bfsQ.pop();
40         reached++;

```

3.12 Circuito Euleriano (Algoritmo de Hierholzer) - Mail Delivery (cont.)

```

41     for(auto &pr : adj[u]){
42         int v = pr.first;
43         if(!visited[v]){
44             visited[v] = true;
45             bfsQ.push(v);
46         }
47     }
48 }
49
50 int totalWithEdges = 0;
51 for(int v = 1; v <= n; v++){
52     if(deg[v] > 0) totalWithEdges++;
53 }
54
55 if(reached < totalWithEdges){
56     cout << "IMPOSSIBLE\n";
57     return 0;
58 }
59
60 // 3) Hierholzer's algorithm (iterative) to build the Eulerian circuit
61 vector<int> ptr(n + 1, 0);
62 vector<bool> usedEdge(m, false);
63 vector<int> circuit;
64 circuit.reserve(m + 1);
65
66 vector<int> st;
67 st.reserve(m + 1);
68 st.push_back(1);
69
70 while(!st.empty()){
71     int v = st.back();
72     if(ptr[v] < (int)adj[v].size()){
73         auto [to, eid] = adj[v][ptr[v]];
74         ptr[v]++;
75         if(usedEdge[eid]) continue;
76         usedEdge[eid] = true;
77         st.push_back(to);
78     } else {
79         circuit.push_back(v);
80         st.pop_back();
81     }
82 }
83
84 reverse(circuit.begin(), circuit.end());
85
86 // circuit must use exactly m edges -> m+1 vertices, and start/end at 1
87 if((int)circuit.size() != m + 1 || circuit.front() != 1 || circuit.back() != 1){
88     cout << "IMPOSSIBLE\n";
89     return 0;
90 }
91
92 for(int i = 0; i < (int)circuit.size(); i++){
93     cout << circuit[i];
94     if(i + 1 < (int)circuit.size()) cout << ' ';
95 }
96 cout << "\n";
97
98 return 0;
99 }

```

3.13 Clique Maximo (Bron-Kerbosch con Pivote) - Holy Network

Resuelve "Holy Network" (problema H, ICPC 2025): dados N números, hallar el grupo mas grande en el que CADA PAR comparta un factor mayor que 1. LA TRAMPA ES "CADA PAR". Que el grupo este conectado NO alcanza: hay que pedir que todos con todos sean compatibles, o sea un CLIQUE, no una componente conexa. El contraejemplo mas chico es {6, 10, 35}: 6 y 10 comparten el 2, 10 y 35 comparten el 5, pero 6 y 35 son coprimos. La componente conexa mide 3 y el grupo valido mide 2. La primera version de este archivo usaba DSU y devolvía el tamaño de la componente: pasaba los tres casos del sample (donde ambas cosas coinciden) pero fallaba 762 de 1500 casos aleatorios contra fuerza bruta. Esta version, con clique maximo, no falla ninguno. EL CLIQUE MAXIMO ES NP-DIFÍCIL, pero aqui $N \leq 50$ y Bron-Kerbosch con pivote lo resuelve rapido: se mantiene R (el clique en curso), P (candidatos) y X (ya descartados), todo en mascarar de 64 bits porque N cabe en un unsigned long long. El PIVOTE es lo que evita la explosión: se elige el vertice de $P \setminus X$ con mas vecinos en P y solo se ramifica por los candidatos que NO son sus vecinos, porque cualquier clique maximal o contiene al pivote o contiene a alguno de esos. La poda extra (si lo que llevamos mas todos los candidatos no supera al mejor, se corta) ayuda bastante cuando el grafo es denso. POR QUE NO EXPLOTA CON ESTOS DATOS: cada numero es $\leq 10^{12}$, así que tiene a lo sumo 11 primos distintos ($2 \cdot 3 \cdot 5 \cdot \dots \cdot 31$ ya pasa de $2 \cdot 10^{11}$). El grafo es entonces la union de unos pocos cliques, uno por primo, y esa estructura deja pocos cliques maximales. Medido con 200 casos de $N = 50$: 17 ms si todos comparten factor, 20 ms con numeros de 11 primos, y 84 ms en el adversario que mas cliques maximales logre construir. El minimo es 1: un grupo de un solo ingrediente es armonico porque no hay pares que revisar. Ojo con el 1 como entrada: $\gcd(1, x) = 1$, así que un 1 no es compatible con nada. Se usa `_gcd` en vez de `std::gcd` para no depender de C++17 ni de incluir `<numeric>`.

C++\ICPC\2025\H - Holy Network.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  typedef unsigned long long ull;
5
6  int n;
7  ull adj[64];
8  int mejor;
9
10 // Bron-Kerbosch con pivote sobre mascarar de 64 bits.
11 // R = clique que llevamos, P = candidatos, X = ya descartados
12 void bron(ull R, ull P, ull X) {
13     if (P == 0 && X == 0) {
14         mejor = max(mejor, __builtin_popcountll(R));
15         return;
16     }
17
18     // Cota: ni tomando todos los candidatos se mejora lo que ya hay.
19     if (__builtin_popcountll(R) + __builtin_popcountll(P) <= mejor)
20         return;
21
22     // Pivote: el de P \ X con mas vecinos en P. Solo hace falta ramificar por
23     // los candidatos que NO son vecinos suyos.
24     ull PX = P | X;
25     int piv = __builtin_ctzll(PX);
26     int mejorGrado = -1;
27
28     for (ull t = PX; t; t &= t - 1) {
29         int u = __builtin_ctzll(t);
30         int g = __builtin_popcountll(adj[u] & P);
31         if (g > mejorGrado) {
32             mejorGrado = g;
33             piv = u;
34         }
35     }
36
37     for (ull t = P & ~adj[piv]; t; t &= t - 1) {
38         int v = __builtin_ctzll(t);
39         ull bit = 1ULL << v;
40
41         bron(R | bit, P & adj[v], X & adj[v]);
42
43         P &= ~bit;
44         X |= bit;
45     }
46 }

```

3.13 Clique Maximo (Bron-Kerbosch con Pivote) - Holy Network (cont.)

```

47
48 int main() {
49     ios::sync_with_stdio(false);
50     cin.tie(nullptr);
51
52     while (cin >> n && n != 0) {
53
54         vector<long long> a(n);
55
56         for (auto &x : a)
57             cin >> x;
58
59         for (int i = 0; i < n; i++)
60             adj[i] = 0;
61
62         // Dos ingredientes son compatibles si comparten un factor mayor que 1.
63         for (int i = 0; i < n; i++)
64             for (int j = i + 1; j < n; j++)
65                 if (__gcd(a[i], a[j]) > 1) {
66                     adj[i] |= 1ULL << j;
67                     adj[j] |= 1ULL << i;
68                 }
69
70         // Un grupo de UN solo ingrediente siempre es armonico: no hay pares
71         // que revisar. Por eso la respuesta nunca baja de 1.
72         mejor = 1;
73
74         ull todos = (n == 64) ? ~0ULL : ((1ULL << n) - 1);
75         bron(0, todos, 0);
76
77         cout << mejor << '\n';
78     }
79
80     return 0;
81 }

```

3.14 Clique Maximo con Backtracking Iterativo - Holy Network (clique Iterativo)

La otra solución correcta del problema H de ICPC 2025 ("Holy Network"): el grupo más grande donde CADA PAR comparte un factor mayor que 1, o sea un clique máximo. A diferencia de la versión con Bron-Kerbosch (el archivo "H - Holy Network" de esta carpeta), esta hace un backtracking directo y además ITERATIVO, con una pila explícita de estados (candidatos que quedan, tamaño del grupo). Vale tenerla porque el patrón sirve para cualquier búsqueda que se quiera sin recursión. LA PODA ES LO QUE LA HACE VIABLE: los candidatos se recorren en orden y, si lo que ya se llevo más los que faltan por mirar no supera al mejor encontrado, se corta el ciclo de una (actual.tamGrupo + quedan <= mejorGrupo). Sin esa línea sería 2^n . Cada vez que se elige un candidato, los nuevos candidatos son SOLO los que vienen después y además son compatibles con el elegido: así todo lo que quede en la pila ya es compatible con todo el grupo, y no hay que revalidar nada. El gcd se calcula a mano con el algoritmo de Euclides en vez de llamar a `__gcd`, lo que evita depender de extensiones del compilador o de `<numeric>` de C++17. Verificado contra fuerza bruta 2^n en 1500 casos, sin fallos, y pasa el sample (4, 5, 1) y el contraejemplo (6, 10, 35) que tumba la versión con DSU.

C++/ICPC/2025/H - Holy Network (clique iterativo).cpp

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 #include <stack>
5
6 using namespace std;
7
8 struct Estado
9 {
10     vector<long long> candidatos;
11     long long tamGrupo;
12 };
13
14 int main()
15 {
16     ios::sync_with_stdio(false);
17     cin.tie(nullptr);
18

```

3.14 Clique Maximo con Backtracking Iterativo - Holy Network (clique Iterativo) (cont.)

```

18     long long n;
19
20     while (cin >> n && n != 0)
21     {
22         vector<long long> resonancia(n);
23
24         for (long long i = 0; i < n; i++)
25         {
26             cin >> resonancia[i];
27         }
28
29         vector<vector<bool>> compatibles(n, vector<bool>(n, false));
30
31         for (long long a = 0; a < n; a++)
32         {
33             for (long long b = a + 1; b < n; b++)
34             {
35                 long long x = resonancia[a];
36                 long long y = resonancia[b];
37
38                 while (y != 0)
39                 {
40                     long long resto = x % y;
41                     x = y;
42                     y = resto;
43                 }
44
45                 if (x > 1)
46                 {
47                     compatibles[a][b] = true;
48                     compatibles[b][a] = true;
49                 }
50             }
51         }
52
53         vector<long long> todos(n);
54
55         for (long long i = 0; i < n; i++)
56         {
57             todos[i] = i;
58         }
59
60         stack<Estado> pila;
61         pila.push({todos, 0});
62
63         long long mejorGrupo = 0;
64
65         while (!pila.empty())
66         {
67             Estado actual = pila.top();
68             pila.pop();
69
70             mejorGrupo = max(mejorGrupo, actual.tamGrupo);
71
72             for (long long i = 0; i < (long long)actual.candidatos.size(); i++)
73             {
74                 long long quedan = actual.candidatos.size() - i;
75
76                 if (actual.tamGrupo + quedan <= mejorGrupo)
77                 {
78                     break;
79                 }
80
81                 long long elegido = actual.candidatos[i];
82
83                 vector<long long> nuevosCandidatos;
84
85                 for (long long j = i + 1; j < (long long)actual.candidatos.size(); j++)
86                 {
87                     if (compatibles[elegido][actual.candidatos[j]])
88                     {
89                         nuevosCandidatos.push_back(actual.candidatos[j]);
90                     }
91                 }
92             }
93         }
94     }
95 }

```

3.14 Clique Maximo con Backtracking Iterativo - Holy Network (clique Iterativo) (cont.)

```

91         }
92     }
93
94     pila.push({
95         nuevosCandidatos,
96         actual.tamGrupo + 1
97     });
98 }
99
100 cout << mejorGrupo << '\n';
101 }
102
103 return 0;
104 }

```

3.15 Componentes Fuente con Kosaraju - Lumina Juan

La otra soluci n del problema J de ICPC 2024 ("Lumina"): el m nimo de gemas a encender a mano para que la reacci n en cadena las encienda todas. La respuesta es la cantidad de componentes fuertemente conexas SIN aristas entrantes desde otra componente. A diferencia de "J - Lumina" de esta carpeta, que usa Tarjan, esta usa KOSARAJU: un DFS sobre el grafo normal que apila los nodos por orden de terminaci n, y despu s un DFS sobre el grafo INVERSO tomando los nodos en orden inverso de esa pila. Cada  rbol del segundo recorrido es una componente. CUAL CONVIENE: Kosaraju es m s f cil de recordar (dos DFS y un reverse) pero necesita construir el grafo inverso, o sea el doble de memoria. Tarjan hace una sola pasada y no lo necesita, pero la l gica de low[] y la pila es m s f cil de escribir mal. Para un cuaderno vale tener las dos. OJO CON EL ORDEN DE NUMERACION: Kosaraju numera las componentes en orden topol gico y Tarjan al rev s. Aqu  solo se cuentan las fuentes, as  que da igual, pero en 2-SAT esa diferencia volte a la regla de asignaci n (ver "G - Signal Coverage" de esta carpeta). El grafo es DIRIGIDO e IMPL CITO: u alcanza a v si v cae dentro del radio de u, y como cada gema tiene su propio radio eso no es sim trico. Las aristas no se guardan, se recalculan con $dist^2 \leq r^2$ en enteros. Verificado con los tres casos del sample (1, 2, 2).

C++\ICPC\2024\Lumina Juan.cpp

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <stack>
5
6 using namespace std;
7
8 struct Circulo
9 {
10     long long x;
11     long long y;
12     long long r;
13 };
14
15 int main()
16 {
17     ios::sync_with_stdio(false);
18     cin.tie(nullptr);
19
20     long long n;
21
22     while(cin >> n && n != 0)
23     {
24         vector<Circulo> circulos(n);
25
26         for(long long i = 0; i < n; i++)
27         {
28             cin >> circulos[i].x >> circulos[i].y >> circulos[i].r;
29         }
30
31         vector<vector<int>> conexion(n);
32
33         for(long long a = 0; a < n; a++)
34         {
35             for(long long b = 0; b < n; b++)
36             {
37                 if(a == b)

```

3.15 Componentes Fuente con Kosaraju - Lumina Juan (cont.)

```

38         {
39             continue;
40         }
41
42         long long dx = circulos[a].x - circulos[b].x;
43         long long dy = circulos[a].y - circulos[b].y;
44
45         long long distancia = dx * dx + dy * dy;
46
47         if(distancia <= circulos[a].r * circulos[a].r)
48         {
49             conexion[a].push_back(b);
50         }
51     }
52 }
53
54 vector<bool> visitado(n, false);
55 vector<long long> ordenFinal;
56
57 for(long long i = 0; i < n; i++)
58 {
59     if(visitado[i])
60     {
61         continue;
62     }
63
64     stack<pair<long long, long long>> pila;
65     visitado[i] = true;
66     pila.push({i, 0});
67
68     while(!pila.empty())
69     {
70         long long actual = pila.top().first;
71         long long posicion = pila.top().second;
72
73         if(posicion == (long long)conexion[actual].size())
74         {
75             ordenFinal.push_back(actual);
76             pila.pop();
77             continue;
78         }
79
80         long long siguiente = conexion[actual][posicion];
81         pila.top().second++;
82
83         if(!visitado[siguiente])
84         {
85             visitado[siguiente] = true;
86             pila.push({siguiente, 0});
87         }
88     }
89 }
90
91 vector<bool> encontrado(n, false);
92
93 long long contador = 0;
94
95 for(long long k = n - 1; k >= 0; k--)
96 {
97     long long inicio = ordenFinal[k];
98
99     if(encontrado[inicio])
100     {
101         continue;
102     }
103
104     queue<long long> cola;
105     cola.push(inicio);
106     encontrado[inicio] = true;
107     contador++;
108
109     while(!cola.empty())

```

3.15 Componentes Fuente con Kosaraju - Lumina Juan (cont.)

```

10         {
11             long long actual = cola.front();
12             cola.pop();
13
14             for(long long siguiente : conexion[actual])
15             {
16                 if(!encontrado[siguiente])
17                 {
18                     encontrado[siguiente] = true;
19                     cola.push(siguiente);
20                 }
21             }
22         }
23     }
24
25     cout << contador << '\n';
26 }
27
28 return 0;
29 }

```

3.16 Componentes Fuente de la Condensacion (Tarjan) - Lumina

Resuelve "Lumina" (problema J, ICPC 2024). Cada gema ilumina a las que caen dentro de SU radio, y el código cuenta el mínimo de gemas que hay que encender a mano para que al final todas queden encendidas. EL GRAFO ES DIRIGIDO aunque venga de círculos: u alcanza a v si v esta dentro del radio de u, y como cada gema tiene su propio radio, que u ilumine a v no implica lo contrario. Además es IMPLICITO: las aristas no se guardan, se recalculan con $\text{dist}^2 \leq r^2$ (en enteros, sin raíz) cada vez que hacen falta. Son $O(n^2)$ aristas. LA IDEA: dentro de una componente fuertemente conexa, encender una gema enciende a todas. Así que se comprime cada SCC a un nodo y queda un DAG (la condensación). En ese DAG, una componente a la que no le llega ninguna arista desde otra (grado de entrada 0) no la puede encender nadie de afuera: hay que encender una de sus gemas a mano. Y basta con eso, porque toda componente del DAG es alcanzable desde alguna fuente. La respuesta es la cantidad de componentes FUENTE. Las SCC salen con Tarjan: `disc[]` es el orden de descubrimiento, `low[]` el menor `disc` alcanzable sin salir de la pila, y cuando `low[u] == disc[u]`, u es la raíz de una componente y se saca de la pila hasta llegar a él. Las aristas que cruzan entre componentes distintas son las que marcan `hasIncoming`. CUANDO USAR: "mínimo de puntos de partida para alcanzarlo todo", "a cuantos hay que avisarle para que el rumor llegue a todos". Si además piden el mínimo de aristas a AGREGAR para que el grafo quede fuertemente conexo, la respuesta es $\max(\# \text{fuentes}, \# \text{sumideros})$, salvo que ya sea una sola componente. OJO: Tarjan recursivo llega a profundidad n. Con aristas $O(n^2)$, n no puede ser muy grande y no hay problema, pero en un grafo con millones de nodos habría que hacerlo iterativo. AMBIGÜEDAD DEL ENUNCIADO: una frase define "cerca" de forma SIMÉTRICA ("si una gema esta dentro del radio de la otra"), y con esa lectura la respuesta sería solo el número de componentes conexas. Pero el ejemplo y la figura hablan del aura de CADA gema alcanzando a otras, que es la lectura dirigida de este código. Los tres samples dan igual con las dos. El caso que las separa: gemas en $(-10^9, 0)$ radio 10^9 , $(0, 0)$ radio 0 y $(10^9, 0)$ radio 10^9 da 2 dirigido y 1 simétrico. Verificado bajo la lectura dirigida contra fuerza bruta en 700 casos; N = 5000 en ~200 ms.

C++\ICPC\2024\J - Lumina.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Gem {
5     long long x, y, r;
6 };
7
8 int n;
9 vector<Gem> gems;
10
11 vector<int> disc, low, st, component;
12 vector<bool> inStack;
13
14 int timer;
15 int sccCount;
16
17 void tarjan(int u) {
18     disc[u] = low[u] = timer++;

```

3.16 Componentes Fuente de la Condensacion (Tarjan) - Lumina (cont.)

```

21     st.push_back(u);
22     inStack[u] = true;
23
24     for (int v = 0; v < n; v++) {
25
26         if (u == v)
27             continue;
28
29         long long dx = gems[u].x - gems[v].x;
30         long long dy = gems[u].y - gems[v].y;
31
32         // u puede iluminar v
33         if (dx * dx + dy * dy > gems[u].r * gems[u].r)
34             continue;
35
36         if (disc[v] == -1) {
37             tarjan(v);
38
39             low[u] = min(low[u], low[v]);
40
41         }
42         else if (inStack[v]) {
43
44             low[u] = min(low[u], disc[v]);
45         }
46     }
47
48     if (low[u] == disc[u]) {
49         while (true) {
50             int v = st.back();
51             st.pop_back();
52
53             inStack[v] = false;
54             component[v] = sccCount;
55
56             if (v == u)
57                 break;
58         }
59         sccCount++;
60     }
61 }
62
63 int main() {
64     ios::sync_with_stdio(false);
65     cin.tie(nullptr);
66
67     while (cin >> n && n != 0) {
68         gems.resize(n);
69
70         for (auto &gem : gems)
71             cin >> gem.x >> gem.y >> gem.r;
72
73         disc.assign(n, -1);
74         low.assign(n, 0);
75         component.assign(n, -1);
76         inStack.assign(n, false);
77
78         st.clear();
79
80         timer = 0;
81         sccCount = 0;
82
83         // Encontrar SCC
84         for (int i = 0; i < n; i++) {
85             if (disc[i] == -1)

```

3.16 Componentes Fuente de la Condensacion (Tarjan) - Lumina (cont.)

```

93     tarjan(i);
94 }
95
96 // inDegree de cada SCC
97 vector<bool> hasIncoming(sccCount, false);
98
99 // Revisar todas las aristas
100 for (int u = 0; u < n; u++) {
101
102     for (int v = 0; v < n; v++) {
103
104         if (u == v)
105             continue;
106
107         long long dx = gems[u].x - gems[v].x;
108         long long dy = gems[u].y - gems[v].y;
109
110         // u -> v
111         if (dx * dx + dy * dy <= gems[u].r * gems[u].r) {
112
113             if (component[u] != component[v]) {
114                 hasIncoming[component[v]] = true;
115             }
116         }
117     }
118 }
119
120 int answer = 0;
121
122 for (int c = 0; c < sccCount; c++) {
123     if (!hasIncoming[c])
124         answer++;
125 }
126
127 cout << answer << '\n';
128 }
129
130 return 0;
131 }

```

3.17 Componentes Fuertemente Conexas (Kosaraju) - Planets And Kingdoms

Resuelve "Planets and Kingdoms" (Day 5, problema I - Contest 05 [Avanzados]): dados n planetas y m teleportadores dirigidos, dos planetas pertenecen al mismo reino si y solo si existe una ruta de a a b y de b a a ; hay que hallar todos los reinos (componentes fuertemente conexas) y asignarle a cada planeta la etiqueta de su reino. Implementa Kosaraju con DFS iterativos: primero un DFS sobre el grafo original (matrix) que llena la pila "cam" en orden de finalizacion, luego recorre "cam" en reversa haciendo DFS sobre el grafo transpuesto (matrix2, funcion dfs_gay) para etiquetar cada componente conexa como un reino distinto en "respuesta".

C++\Training_Camp_2026\Day 5\I - Planets and Kingdoms.py

```

1 import sys
2 sys.setrecursionlimit(300000)
3
4 n, m = map(int, input().split())
5
6 matrix = [[] for _ in range(n)]
7 matrix2 = [[] for _ in range(n)]
8
9 for _ in range(m):
10     p, s = map(int, input().split())
11     p -= 1
12     s -= 1
13
14     matrix[p].append(s)
15     matrix2[s].append(p)
16
17
18 visited = [False] * n
19 cam = []
20
21 def dfs(start):

```

3.17 Componentes Fuertemente Conexas (Kosaraju) - Planets And Kingdoms (cont.)

```

22     stack = [(start, 0)]
23     visited[start] = True
24
25     while stack:
26         nodo, estado = stack[-1]
27
28         if estado < len(matrix[nodo]):
29             vecino = matrix[nodo][estado]
30             stack[-1] = (nodo, estado + 1)
31
32             if not visited[vecino]:
33                 visited[vecino] = True
34                 stack.append((vecino, 0))
35
36         else:
37             cam.append(nodo)
38             stack.pop()
39
40     for i in range(n):
41         if not visited[i]:
42             dfs(i)
43
44
45 visited = [False] * n
46 respuesta = [0] * n
47
48 def dfs_gay(start, reino):
49     stack = [start]
50     visited[start] = True
51
52     while stack:
53         nodo = stack.pop()
54         respuesta[nodo] = reino
55
56         for vecino in matrix2[nodo]:
57             if not visited[vecino]:
58                 visited[vecino] = True
59                 stack.append(vecino)
60
61
62 voy = 0
63
64 for i in reversed(cam):
65     if not visited[i]:
66         voy += 1
67         dfs_gay(i, voy)
68
69
70 print(voy)
71 print(*respuesta)

```

3.18 DFS - Artwork

Resuelve "Artwork" (Day 1, problema A - Maratona de Programacao da SBC 2019): un ladrón debe cruzar una sala rectangular $M \times N$, desde la esquina de entrada (0,0) hasta la esquina opuesta donde esta la obra, sin acercarse a menos de S metros de ningún sensor de movimiento i . Modela los sensores que se tocan entre si (distancia entre centros $\leq$ suma de radios) como componentes conexas; si una componente que toca un borde de la sala llega a tocar el borde opuesto o perpendicular, bloquea el paso del ladrón. DFS desde cada sensor que toca un borde, propagandose por los sensores superpuestos y marcando visitados, para decidir si queda un camino libre ('S') o no ('N').

C++\Training_Camp_2026\Day 1\A - Artwork.py

```

1 m, n, s = list(map(int, input().split()))
2 sensors = []
3
4 for i in range(s):
5     sensors.append(list(map(int, input().split())))
6 ans = "S"
7 visited = set()
8 def rec(x,y,r, start):

```

3.18 DFS - Artwork (cont.)

```

9     global ans, m, n
10
11     if ans == "N":
12         return
13     for xi,yi,ri in sensors:
14         if (xi, yi, ri) in visited:
15             continue
16         dist = ((x-xi)**2+(y-yi)**2)**(1/2)
17         if dist <= (r+ri):
18             if (start == "l" and (xi+ri >= m or yi-ri <= 0)) or (start == "d" and (xi-ri <= 0 or yi
+ri >= n)) or (start == "u" and (xi-ri >= m or yi-ri <= 0)) or (start == "r" and (yi+ri >= n or
x-ri <= 0)):
19                 ans = "N"
20                 break
21             visited.add((xi,yi,ri))
22             rec(xi,yi, ri, start)
23
24
25     for i, sensor in enumerate(sensors):
26         r = sensor[2]
27         x = sensor[0]
28         y = sensor[1]
29         if (x + r >= m and x - r <= 0) or (y + r >= n and y - r <= 0) or (x - r <= 0 and y - r <= 0) or
(x + r >= m and y + r >= n):
30             ans = "N"
31             break
32         if x-r<=0 or y-r<=0 or y+r>=n or x+r>=m:
33             dir = ""
34             if x-r<=0:
35                 dir = "l"
36             elif y-r<=0:
37                 dir = "d"
38             elif y+r>=n:
39                 dir = "u"
40             elif x+r>=m:
41                 dir = "r"
42             if dir != "":
43                 visited = set()
44                 rec(x,y,r, dir)
45     print(ans)

```

3.19 DFS - Denouncing Mafia

Resuelve "Denouncing Mafia" (Day 1, problema D - Maratona de Programacao da SBC 2019): un arbol enraizado de N miembros de mafia (el 1 es el jefe, cada quien tiene un superior directo) donde interrogar a alguien obliga a subir denunciando a su superior hasta llegar al jefe; con un vidente que solo puede senalar a K miembros iniciales, se busca el maximo numero de mafiosos que se pueden arrestar en total. DFS que para cada nodo se queda con la rama de mayor profundidad para "seguir gratis" hacia la raiz, y registra el costo extra (diferencia de profundidad) de las demas ramas como candidatas; al final toma las K ramas de mayor costo y las suma.

C++\Training_Camp_2026\Day 1\D - Denouncing_Mafia.py

```

1 import copy
2 import sys
3
4 sys.setrecursionlimit(10**6)
5 m, k = list(map(int, input().split()))
6 entry = list(map(int, input().split()))
7
8 currMemb = 1
9 dicc = {1: []}
10 for node in entry:
11     currMemb += 1
12     if node in dicc:
13         dicc[node] = dicc[node] + [currMemb]
14     else:
15         dicc[node] = [currMemb]
16
17 visited = set()
18 ans = []
19
20 def rec(curr, depth, prev):

```

3.19 DFS - Denouncing Mafia (cont.)

```

21     global visited, ans
22     if curr in dicc:
23         children = dicc[curr]
24         first = True
25         leafsDepth = []
26         for son in children:
27             newDep = rec(son, depth + 1, curr)
28             leafsDepth.append(newDep)
29         leafsDepth.sort(reverse=True)
30         max = leafsDepth.pop(0)
31         for i in leafsDepth:
32             ans.append(i - depth)
33         return max
34     else:
35         return depth
36     ans.append(rec(1, 1, None))
37     ans.sort(reverse=True)
38     sum = 0
39
40     curr = 0
41     while(curr < k and curr < len(ans)):
42         sum+=ans[curr]
43         curr+=1
44
45     print(sum)

```

3.20 DFS - A Mazing!

Resuelve "A Mazing!" (Day 11, problema L - CCPL (Kattis)): un problema interactivo donde hay que explorar un laberinto de celdas cuadradas moviendose (up/down/left/right) segun las respuestas del juez ("wall", "ok", "solved"), sin conocer el mapa de antemano, hasta encontrar una salida o determinar que no existe ninguna (reportando "no way out"). Implementa un DFS iterativo sobre el grafo implicito de celdas: `todo` guarda por celda las direcciones aun no probadas (en un deque, excluyendo la direccion opuesta a la usada para entrar a esa celda) y `parent` guarda la celda predecesora; al agotar las direcciones de la celda actual, retrocede (backtrack) hacia su padre repitiendo el movimiento inverso, como una pila explicita de DFS.

C++\Training_Camp_2026\Day 11 - CCPL\L - A Mazing!.py

```

1 import sys
2 from collections import deque
3
4 dirs = {
5     "up": (0, 1),
6     "down": (0, -1),
7     "left": (-1, 0),
8     "right": (1, 0),
9 }
10
11 opposite = {
12     "up": "down",
13     "down": "up",
14     "left": "right",
15     "right": "left",
16 }
17
18 curr = (0, 0)
19
20 todo = {
21     curr: deque(["down", "right", "left", "up"])
22 }
23
24 parent = {}
25
26 while True:
27
28     if todo[curr]:
29         move = todo[curr].popleft()
30
31         print(move, flush=True)
32         ans = input()
33
34         if ans == "solved" or ans == "wrong":

```

3.20 DFS - A Mazing! (cont.)

```

35     break
36
37     if ans == "wall":
38         continue
39
40     dx, dy = dirs[move]
41     nxt = (curr[0] + dx, curr[1] + dy)
42
43     if nxt not in todo:
44         todo[nxt] = deque(["down", "right", "left", "up"])
45         # Ya usamos la dirección opuesta para llegar aquí
46         todo[nxt].remove(opposite[move])
47         parent[nxt] = curr
48
49     curr = nxt
50
51 else:
52
53     if curr == (0, 0):
54         print("no way out", flush=True)
55         ans = input()
56         break
57
58     par = parent[curr]
59
60     dx = par[0] - curr[0]
61     dy = par[1] - curr[1]
62
63     if dx == 1:
64         move = "right"
65     elif dx == -1:
66         move = "left"
67     elif dy == 1:
68         move = "up"
69     else:
70         move = "down"
71
72     print(move, flush=True)
73     ans = input()
74
75     if ans == "solved" or ans == "wrong":
76         break
77
78     curr = par

```

3.21 Dijkstra - Fantastic Robot

Resuelve "Fantastic Robot" (Day 9, problema E - Contest 08 [Inicial & Avanzado]): HERBIE debe llegar desde una celda a otra en una matriz binaria (0 = libre, 1 = pared) pero no puede moverse instrucción por instrucción, sino que debe ejecutar completa una de sus "órdenes" preprogramadas (cadenas de movimientos U/D/L/R), donde cada movimiento bloqueado por una pared o el borde simplemente no cuenta como paso. Modela cada orden como una arista de costo variable (los "pasos" realmente efectivos) entre la celda inicial y la celda final de aplicar esa orden completa (funcion generar_vecinos), y corre Dijkstra con priority_queue sobre ese grafo implícito para hallar el mínimo total de movimientos hasta la celda objetivo.

C++\Training_Camp_2026\Day 9\E - Fantastic Robot.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5
6  const ll INF = 1e18;
7
8
9  int n,m;
10
11 vector<vector<int>> mat;
12 vector<string> comandos;
13
14
15 vector<tuple<int,int,int>> generar_vecinos(int x, int y){

```

3.21 Dijkstra - Fantastic Robot (cont.)

```

6
7     vector<tuple<int,int,int>> vecinos;
8
9
10    for(string &cmd : comandos){
11
12        int xi=x;
13        int yi=y;
14        int pasos=0;
15
16
17        for(char c: cmd){
18
19            if(c=='U'){
20                pasos++;
21                xi--;
22
23                if(xi<0 || mat[xi][yi]==1){
24                    pasos--;
25                    xi++;
26                }
27
28            }else if(c=='D'){
29
30                pasos++;
31                xi++;
32
33                if(xi>=n || mat[xi][yi]==1){
34                    pasos--;
35                    xi--;
36                }
37
38            }else if(c=='L'){
39
40                pasos++;
41                yi--;
42
43                if(yi<0 || mat[xi][yi]==1){
44                    pasos--;
45                    yi++;
46                }
47
48            }else{
49
50                pasos++;
51                yi++;
52
53                if(yi>=m || mat[xi][yi]==1){
54                    pasos--;
55                    yi--;
56                }
57
58            }
59
60            vecinos.push_back({xi,yi,pasos});
61        }
62
63        return vecinos;
64    }
65
66    int main(){
67
68        ios::sync_with_stdio(false);
69        cin.tie(nullptr);
70
71
72        cin>>n>>m;
73

```

3.21 Dijkstra - Fantastic Robot (cont.)

```

88
89     mat.assign(n,vector<int>(m));
90
91
92     for(int i=0;i<n;i++){
93
94         string s;
95         cin>>s;
96
97         for(int j=0;j<m;j++){
98             mat[i][j]=s[j]-'0';
99         }
100     }
101
102
103     int sx,sy;
104     cin>>sx>>sy;
105
106
107     int tx,ty;
108     cin>>tx>>ty;
109
110
111     int k;
112     cin>>k;
113
114
115     comandos.resize(k);
116
117     for(auto &x:comandos)
118         cin>>x;
119
120
121
122     vector<vector<ll>> dist(n, vector<ll>(m,INF));
123
124
125     priority_queue<
126         tuple<ll,int,int>,
127         vector<tuple<ll,int,int>>,
128         greater<tuple<ll,int,int>>
129     > pq;
130
131
132     dist[sx][sy]=0;
133
134     pq.push({0,sx,sy});
135
136
137     while(!pq.empty()){
138
139
140         auto [d,x,y]=pq.top();
141         pq.pop();
142
143
144         if(d!=dist[x][y])
145             continue;
146
147
148         if(x==tx && y==ty)
149             break;
150
151
152         for(auto [nx,ny,costo] : generar_vecinos(x,y)){
153
154
155             if(dist[nx][ny] > d+costo){
156
157                 dist[nx][ny]=d+costo;
158
159

```

3.21 Dijkstra - Fantastic Robot (cont.)

```

60         pq.push({
61             dist[nx][ny],
62             nx,
63             ny
64         });
65     }
66 }
67
68 }
69
70
71 if(dist[tx][ty]==INF)
72     cout<<-1<<"\n";
73 else
74     cout<<dist[tx][ty]<<"\n";
75
76 }
```

3.22 Dijkstra con Nodos Virtuales - Chinese Innovation

Resuelve "Chinese Innovation" (Day 10, problema C - Contest 09 [Avanzados]): un imperio de n ciudades con m caminos bidireccionales con costo, mas k tipos de teletransportadores; dos ciudades con teletransportador del mismo tipo se conectan instantaneamente, pero el costo de usarlo depende de la ciudad de origen (cada mayor cobra su propia tarifa de salida). Hay que hallar el costo minimo de viajar de la ciudad 1 a la ciudad n . Modela cada tipo de teletransportador t como un nodo virtual ($\text{virtual_node} = n + t$): cada ciudad con ese tipo agrega una arista dirigida hacia el nodo virtual con peso igual a su tarifa de salida, y el nodo virtual agrega aristas de costo 0 hacia todas esas ciudades; asi, saltar entre ciudades del mismo tipo cuesta solo la tarifa de la ciudad de origen. Corre Dijkstra estandar (con `priority_queue` y distancias en un `unordered_map`) desde 1 hasta n sobre este grafo ampliado.

C++\Training_Camp_2026\Day 10\C - Chinese Innovation.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using pii = pair<long long, int>;
5
6 const long long INF = 1e18;
7
8 long long dijkstra(unordered_map<int, vector<pair<int, int>>> &graph, int start, int target) {
9     unordered_map<int, long long> dist;
10    unordered_set<int> visited;
11
12    priority_queue<pii, vector<pii>, greater<pii>> pq;
13
14    dist[start] = 0;
15    pq.push({0, start});
16
17    while (!pq.empty()) {
18        auto [d, u] = pq.top();
19        pq.pop();
20
21        if (visited.count(u))
22            continue;
23        visited.insert(u);
24
25        if (u == target)
26            return d;
27
28        for (auto [v, w] : graph[u]) {
29
30            long long nd = d + w;
31
32            if (!dist.count(v) || nd < dist[v]) {
33                dist[v] = nd;
34                pq.push({nd, v});
35            }
36        }
37    }
38
39    if (dist.count(target))
40        return dist[target];

```

3.22 Dijkstra con Nodos Virtuales - Chinese Innovation (cont.)

```

41     return INF;
42 }
43
44 int main() {
45     ios::sync_with_stdio(false);
46     cin.tie(nullptr);
47
48     int n, m, k;
49     cin >> n >> m >> k;
50
51     unordered_map<int, vector<pair<int, int>>> graph;
52
53     for (int i = 0; i < m; i++) {
54         int a, b, w;
55         cin >> a >> b >> w;
56         graph[a].push_back({b, w});
57         graph[b].push_back({a, w});
58     }
59
60     for (int i = 1; i <= n; i++) {
61         int tel;
62         cin >> tel;
63
64         for (int j = 0; j < tel; j++) {
65             int t, w;
66             cin >> t >> w;
67
68             int virtual_node = n + t;
69
70             graph[i].push_back({virtual_node, w});
71             graph[virtual_node].push_back({i, 0});
72         }
73     }
74
75     long long costo = dijkstra(graph, 1, n);
76     cout << costo << "\n";
77
78     return 0;
79 }

```

3.23 DSU (enfoque INCORRECTO para este problema) - Holy Network (DSU, Incorrecta)

Intento del problema H de ICPC 2025 ("Holy Network") con DSU: une dos numeros cuando comparten un factor, y responde el tamaño de la componente mas grande. ESTA VERSION ESTA MAL Y SE GUARDA JUSTAMENTE POR ESO. El enunciado pide que CADA PAR del grupo comparta un factor, no que el grupo este conectado. El contraejemplo mínimo es {6, 10, 35}: 6-10 comparten el 2 y 10-35 comparten el 5, así que la componente mide 3, pero 6 y 35 son coprimos y el grupo valido mide 2. Medido: falla 872 de 1500 casos aleatorios contra fuerza bruta, aunque pasa los tres casos del sample, donde ambas cosas coinciden. La lección que vale guardar: "todos conectados" y "todos compatibles entre si" son cosas distintas. Lo primero es una componente conexa (facil, DSU o BFS); lo segundo es un CLIQUE (NP-difícil en general). Si un enunciado dice "cada par", "todos con todos" o "mutuamente", casi seguro es clique y DSU no sirve. La version correcta esta en "H - Holy Network" (Bron-Kerbosch con pivot) y en "H - Holy Network (clique iterativo)", ambas en esta carpeta. Lo unico reutilizable de aqui es como se arman las aristas: en vez de comparar todos los pares con gcd, se factoriza cada numero probando divisores hasta la raiz y se une cada numero con sus factores, usando un map para los primos que aparecen. Eso da $O(n \cdot \sqrt{V})$ en vez de $O(n^2 \log V)$, que ayuda cuando n es grande, aunque aqui $n \leq 50$ y no hace falta.

C++\ICPC\2025\H - Holy Network (DSU, incorrecta).cpp

```

1  #include <iostream>
2  #include <vector>
3  #include <map>
4
5  using namespace std;
6
7  long long buscar(map<long long, long long>& padre, long long x)
8  {
9      if(padre[x] == x)
10     {
11         return x;

```

3.23 DSU (enfoque INCORRECTO para este problema) - Holy Network (DSU, Incorrecta) (cont.)

```

12     }
13
14     return padre[x] = buscar(padre, padre[x]);
15 }
16
17 void unir(map<long long, long long>& padre, map<long long, long long>& tam, long long a, long long
18 b)
19 {
20     a = buscar(padre, a);
21     b = buscar(padre, b);
22
23     if(a == b)
24     {
25         return;
26     }
27
28     if(tam[a] < tam[b])
29     {
30         swap(a, b);
31     }
32
33     padre[b] = a;
34     tam[a] += tam[b];
35 }
36
37 int main()
38 {
39     ios::sync_with_stdio(false);
40     cin.tie(nullptr);
41
42     long long n;
43
44     while(cin >> n && n != 0)
45     {
46         vector<long long> numeros(n);
47
48         map<long long, bool> esInput;
49         map<long long, long long> padre;
50         map<long long, long long> tam;
51
52         for(long long i = 0; i < n; i++)
53         {
54             cin >> numeros[i];
55
56             esInput[numeros[i]] = true;
57             padre[numeros[i]] = numeros[i];
58             tam[numeros[i]] = 1;
59         }
60
61         for(long long x : numeros)
62         {
63             for(long long i = 2; i * i <= x; i++)
64             {
65                 if(x % i == 0)
66                 {
67                     long long z = x / i;
68
69                     if(!padre.count(i))
70                     {
71                         padre[i] = i;
72                         tam[i] = 1;
73                     }
74
75                     if(!padre.count(z))
76                     {
77                         padre[z] = z;
78                         tam[z] = 1;
79                     }
80
81                     unir(padre, tam, x, i);
82                     unir(padre, tam, x, z);
83                 }
84             }
85         }
86     }
87 }

```

3.23 DSU (enfoque INCORRECTO para este problema) - Holy Network (DSU, Incorrecta) (cont.)

```

83     }
84 }
85
86 map<long long, long long> familias;
87 long long maxValor = 0;
88
89 for(long long x : numeros)
90 {
91     long long raiz = buscar(padre, x);
92
93     familias[raiz]++;
94
95     maxValor = max(maxValor, familias[raiz]);
96 }
97
98 cout << maxValor << '\n';
99 }
100
101 return 0;
102 }

```

3.24 Existencia de Camino Euleriano con Aristas Ocultas - Apple Pie

Resuelve "Apple Pie" (problema A, Regionals 2025). Se conoce un prefijo L y un sufijo R de una secuencia, y la parte del medio quedo tapada por la torta: hay que decidir si existe alguna forma de completarla. EL MODELO: cada elemento consecutivo de la secuencia es una ARISTA entre dos valores, así que la secuencia entera es un camino en un multigrafo. Lo conocido (L y R) fija unas aristas; lo tapado tiene que ser un camino que use EXACTAMENTE las aristas restantes. O sea, la pregunta es si las aristas que sobran forman un CAMINO EULERIANO entre dos extremos dados. LAS CONDICIONES DE EULER, que es lo que hay que tener a mano: - camino cerrado (circuito): todos los grados son PARES y el grafo es conexo sobre las aristas - camino abierto: exactamente DOS vertices de grado impar, y son los extremos La funcion canEuler separa cuatro casos segun cuantos extremos se conocen (los dos, solo el inicio, solo el final, ninguno), porque en cada uno la paridad que hay que exigir cambia. EL CHEQUEO DE CONEXIDAD NO ES OPCIONAL: los grados pueden cuadrar perfectamente y aun así el grafo estar partido en dos componentes con aristas, y entonces no hay camino. Se revisa solo sobre los vertices que tienen alguna arista; los aislados no estorban. El caso de 0 elementos tapados va aparte: ahí basta pegar L y R y ver si la secuencia resultante es valida, sin nada de Euler.

C++\ICPC\Regionals 2025\A - Apple Pie.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXN = 1005;
5
6 bool used[MAXN][MAXN];
7 int usedDegree[MAXN];
8
9 bool addPart(const vector<int>& a)
10 {
11     for(int i = 1; i < (int)a.size(); i++)
12     {
13         int u = a[i - 1];
14         int v = a[i];
15
16         if(u == v)
17             return false;
18
19         if(used[u][v])
20             return false;
21
22         used[u][v] = used[v][u] = true;
23
24         usedDegree[u]++;
25         usedDegree[v]++;
26     }
27
28     return true;
29 }
30
31 bool connected(int n, const vector<int>& degree)
32 {

```

3.24 Existencia de Camino Euleriano con Aristas Ocultas - Apple Pie (cont.)

```

33     int s = -1;
34
35     for(int i = 1; i <= n; i++)
36     {
37         if(degree[i] > 0)
38         {
39             s = i;
40             break;
41         }
42     }
43
44     if(s == -1)
45         return true;
46
47     vector<bool> vis(n + 1);
48
49     queue<int> q;
50     q.push(s);
51     vis[s] = true;
52
53     while(!q.empty())
54     {
55         int u = q.front();
56         q.pop();
57
58         for(int v = 1; v <= n; v++)
59         {
60             if(u == v)
61                 continue;
62
63             if(!used[u][v] && !vis[v])
64             {
65                 vis[v] = true;
66                 q.push(v);
67             }
68         }
69     }
70
71     for(int i = 1; i <= n; i++)
72     {
73         if(degree[i] > 0 && !vis[i])
74             return false;
75     }
76
77     return true;
78 }
79
80 bool canEuler(int n, int start, int finish, int remainingEdges)
81 {
82     if(remainingEdges == 0)
83         return false;
84
85     vector<int> degree(n + 1);
86
87     for(int i = 1; i <= n; i++)
88         degree[i] = n - 1 - usedDegree[i];
89
90     // Si conocemos un extremo, debe poder salir/entrar
91     // por alguna arista restante.
92     if(start != -1 && degree[start] == 0)
93         return false;
94
95     if(finish != -1 && degree[finish] == 0)
96         return false;
97
98     vector<int> odd;
99
100    for(int i = 1; i <= n; i++)
101    {
102        if(degree[i] & 1)
103            odd.push_back(i);
104    }

```

3.24 Existencia de Camino Euleriano con Aristas Ocultas - Apple Pie (cont.)

```

05
06 // Ambos extremos conocidos.
07 if(start != -1 && finish != -1)
08 {
09     if(start == finish)
10     {
11         // Euler cerrado.
12         if(!odd.empty())
13             return false;
14     }
15     else
16     {
17         // Euler abierto.
18         if(odd.size() != 2)
19             return false;
20
21         if(!((odd[0] == start && odd[1] == finish) ||
22             (odd[0] == finish && odd[1] == start)))
23             return false;
24     }
25 }
26
27 // Solo conocemos el inicio.
28 else if(start != -1)
29 {
30     if(odd.size() == 0)
31     {
32         // Puede terminar donde empezo.
33     }
34     else if(odd.size() == 2)
35     {
36         if(odd[0] != start && odd[1] != start)
37             return false;
38     }
39     else
40     {
41         return false;
42     }
43 }
44
45 // Solo conocemos el final.
46 else if(finish != -1)
47 {
48     if(odd.size() == 0)
49     {
50         // Puede empezar donde termina.
51     }
52     else if(odd.size() == 2)
53     {
54         if(odd[0] != finish && odd[1] != finish)
55             return false;
56     }
57     else
58     {
59         return false;
60     }
61 }
62
63 // Ningun extremo conocido.
64 else
65 {
66     if(odd.size() != 0 && odd.size() != 2)
67         return false;
68 }
69
70 return connected(n, degree);
71 }
72
73 // Caso en el que la torta cubrio 0 elementos.
74 // Entonces simplemente tenemos L + R.
75 bool zeroCovered(int n,

```

3.24 Existencia de Camino Euleriano con Aristas Ocultas - Apple Pie (cont.)

```

77     const vector<int>& L,
78     const vector<int>& R)
79 {
80     int E = n * (n - 1) / 2;
81
82     if((int)L.size() + (int)R.size() != E + 1)
83         return false;
84
85     vector<int> a;
86
87     for(int x : L)
88         a.push_back(x);
89
90     for(int x : R)
91         a.push_back(x);
92
93     bool seen[MAXN][MAXN] = {};
94
95     for(int i = 1; i < (int)a.size(); i++)
96     {
97         int u = a[i - 1];
98         int v = a[i];
99
100         if(u == v)
101             return false;
102
103         if(seen[u][v])
104             return false;
105
106         seen[u][v] = seen[v][u] = true;
107     }
108
109     return true;
110 }
111
112
113 int main()
114 {
115     ios::sync_with_stdio(false);
116     cin.tie(nullptr);
117
118     int n;
119     cin >> n;
120
121     int p;
122     cin >> p;
123
124     vector<int> L(p);
125
126     for(int &x : L)
127         cin >> x;
128
129     int q;
130     cin >> q;
131
132     vector<int> R(q);
133
134     for(int &x : R)
135         cin >> x;
136
137     // -----
138     // Caso: no hubo elementos cubiertos
139     // -----
140
141     if(zeroCovered(n, L, R))
142     {
143         cout << "Y\n";
144         return 0;
145     }
146
147
148

```

3.24 Existencia de Camino Euleriano con Aristas Ocultas - Apple Pie (cont.)

```

248 // -----
250 // Marcar las aristas conocidas
251 // -----
252
253 if(!addPart(L))
254 {
255     cout << "N\n";
256     return 0;
257 }
258
259 if(!addPart(R))
260 {
261     cout << "N\n";
262     return 0;
263 }
264
265
266 int totalEdges = n * (n - 1) / 2;
267
268 int usedEdges = 0;
269
270 for(int i = 1; i <= n; i++)
271     usedEdges += usedDegree[i];
272
273 usedEdges /= 2;
274
275 int remainingEdges = totalEdges - usedEdges;
276
277
278 // -----
279 // Determinar extremos de la parte cubierta
280 // -----
281
282 int start = -1;
283 int finish = -1;
284
285 if(!L.empty())
286     start = L.back();
287
288 if(!R.empty())
289     finish = R.front();
290
291
292 // -----
293 // Las aristas restantes pueden formar
294 // el camino que falta?
295 // -----
296
297 if(canEuler(n, start, finish, remainingEdges))
298     cout << "Y\n";
299 else
300     cout << "N\n";
301
302 return 0;
303 }

```

3.25 Heavy-Light Decomposition - Los Ratones III

Resuelve "Los Ratones III" (Day 9, problema K - Contest 08 [Inicial & Avanzado]): un arbol de n casas conectadas por calles con valor, donde Q cartas piden la suma de los valores de las aristas en el camino entre u y v despues de multiplicar TODO ese camino por x, y las modificaciones son persistentes (afectan a las cartas siguientes). Usa Heavy-Light Decomposition (dfs para pesos/hijo pesado, decompose para aplanar el arbol en posiciones de arreglo) sobre un segment tree con lazy propagation MULTIPLICATIVO (push, seg[node].lazy) para poder multiplicar y sumar rangos de aristas en el camino en $O(\log^2 N)$ por consulta (path_update/path_query), todo modulo $1e9+7$.

C++\Training_Camp_2026\Day 9\K - Los Ratones III.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;

```

3.25 Heavy-Light Decomposition - Los Ratones III (cont.)

```

5
6 const ll MOD = 1000000007LL;
7 const int MAXN = 100005;
8
9 struct Edge{
10     int to;
11     ll w;
12 };
13
14 int N,Q;
15
16 vector<Edge> g[MAXN];
17
18 int parent[MAXN], depth[MAXN], heavy[MAXN], sz[MAXN];
19 int head[MAXN], pos[MAXN];
20 ll val[MAXN];
21 int curPos;
22
23 struct Node{
24     ll sum,lazy;
25 };
26
27 Node seg[4*MAXN];
28
29 int dfs(int v,int p){
30     parent[v]=p;
31     sz[v]=1;
32     heavy[v]=-1;
33     int mx=0;
34     for(auto e:g[v]){
35         if(e.to==p) continue;
36         depth[e.to]=depth[v]+1;
37         val[e.to]=e.w;
38         dfs(e.to,v);
39         sz[v]+=sz[e.to];
40         if(sz[e.to]>mx){
41             mx=sz[e.to];
42             heavy[v]=e.to;
43         }
44     }
45     return sz[v];
46 }
47
48 ll base[MAXN];
49
50 void decompose(int v,int h){
51     head[v]=h;
52     pos[v]=++curPos;
53     base[curPos]=val[v];
54
55     if(heavy[v]!=-1)
56         decompose(heavy[v],h);
57
58     for(auto e:g[v]){
59         if(e.to==parent[v] || e.to==heavy[v]) continue;
60         decompose(e.to,e.to);
61     }
62 }
63
64 void build(int node,int l,int r){
65     seg[node].lazy=1;
66     if(l==r){
67         seg[node].sum=base[l]%MOD;
68         return;
69     }
70     int mid=(l+r)>>1;
71     build(node<<1,l,mid);
72     build(node<<1|1,mid+1,r);
73     seg[node].sum=(seg[node<<1].sum+seg[node<<1|1].sum)%MOD;
74 }
75
76 void push(int node,int l,int r){

```

3.25 Heavy-Light Decomposition - Los Ratones III (cont.)

```

77 if(seg[node].lazy==1) return;
78 ll x=seg[node].lazy;
79 seg[node].sum=seg[node].sum*x%MOD;
80 if(l!=r){
81     seg[node<<1].lazy=seg[node<<1].lazy*x%MOD;
82     seg[node<<1|1].lazy=seg[node<<1|1].lazy*x%MOD;
83 }
84 seg[node].lazy=1;
85 }
86
87 void update(int node,int l,int r,int ql,int qr,ll x){
88     push(node,l,r);
89     if(r<ql||qr<l) return;
90     if(ql<=l&&r<=qr){
91         seg[node].lazy=x%MOD;
92         push(node,l,r);
93         return;
94     }
95     int mid=(l+r)>>1;
96     update(node<<1,l,mid,ql,qr,x);
97     update(node<<1|1,mid+1,r,ql,qr,x);
98     seg[node].sum=(seg[node<<1].sum+seg[node<<1|1].sum)%MOD;
99 }
100
101 ll query(int node,int l,int r,int ql,int qr){
102     push(node,l,r);
103     if(r<ql||qr<l) return 0;
104     if(ql<=l&&r<=qr) return seg[node].sum;
105     int mid=(l+r)>>1;
106     return (query(node<<1,l,mid,ql,qr)+
107             query(node<<1|1,mid+1,r,ql,qr))%MOD;
108 }
109
110 void path_update(int a,int b,ll x){
111     while(head[a]!=head[b]){
112         if(depth[head[a]]<depth[head[b]]) swap(a,b);
113         update(1,1,N,pos[head[a]],pos[a],x);
114         a=parent[head[a]];
115     }
116     if(depth[a]>depth[b]) swap(a,b);
117     if(pos[a]+1<pos[b])
118         update(1,1,N,pos[a]+1,pos[b],x);
119 }
120
121 ll path_query(int a,int b){
122     ll ans=0;
123     while(head[a]!=head[b]){
124         if(depth[head[a]]<depth[head[b]]) swap(a,b);
125         ans=(ans+query(1,1,N,pos[head[a]],pos[a]))%MOD;
126         a=parent[head[a]];
127     }
128     if(depth[a]>depth[b]) swap(a,b);
129     if(pos[a]+1<pos[b])
130         ans=(ans+query(1,1,N,pos[a]+1,pos[b]))%MOD;
131     return ans;
132 }
133
134 int main(){
135     ios::sync_with_stdio(false);
136     cin.tie(nullptr);
137
138     cin>>N>>Q;
139
140     for(int i=1;i<N;i++){
141         int u,v;
142         ll c;
143         cin>>u>>v>>c;
144         g[u].push_back({v,c});
145         g[v].push_back({u,c});
146     }
147
148     depth[1]=0;

```

3.25 Heavy-Light Decomposition - Los Ratones III (cont.)

```

149     val[1]=0;
150
151     dfs(1,0);
152     decompose(1,1);
153
154     build(1,1,N);
155
156     while(Q--){
157         int u,v;
158         ll x;
159         cin>>u>>v>>x;
160
161         path_update(u,v,x);
162         cout<<path_query(u,v)%MOD<<"\n";
163     }
164
165     return 0;
166 }

```

3.26 Kruskal Reconstruction Tree - Trucks

Resuelve "Trucks" (Day 2, problema J - Maratona de Programacao da SBC 2013): Nlogonia es un archipelago de N islas unidas por M puentes, cada uno con un limite maximo de peso W; para S sitios de competencia hay que hallar el mayor peso que puede transportarse por camion entre el almacen H y el sitio L, es decir, el maximo sobre todos los caminos posibles del minimo W en ese camino (cuello de botella maximo). Ordena los puentes por peso decreciente y los va uniendo con un DSU al estilo Kruskal, pero en vez de solo unir arma un arbol binario de reconstruccion (nodeRepr/leftChild/rightChild) donde cada fusion crea un nodo interno con el peso del puente. El ancestro comun mas bajo (LCA, calculado con binary lifting) de dos hojas en ese arbol es justamente el nodo cuyo peso es la respuesta a la consulta.

C++\Training_Camp_2026\Day 2\J - Trucks.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static char buf_[1 << 22];
5 static int buflen_, bufPos_;
6 inline int gc() {
7     if (bufPos_ == buflen_) { buflen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!buflen_) return -1; }
8     return buf_[bufPos_++];
9 }
10 inline int readInt() {
11     int c = gc();
12     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
13     bool neg = false; if (c == '-') { neg = true; c = gc(); }
14     int x = 0;
15     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
16     return neg ? -x : x;
17 }
18
19 int N, M, S;
20 vector<int> dsuParent;
21 int find(int x) {
22     while (dsuParent[x] != x) { dsuParent[x] = dsuParent[dsuParent[x]]; x = dsuParent[x]; }
23     return x;
24 }
25
26 int main() {
27     N = readInt(); M = readInt(); S = readInt();
28
29     vector<array<int,3>> edgeList(M); // {W, A, B} for easy sort by weight
30     for (int i = 0; i < M; i++) {
31         int a = readInt(), b = readInt(), w = readInt();
32         edgeList[i] = {w, a, b};
33     }
34     sort(edgeList.begin(), edgeList.end(), [](const array<int,3>&x, const array<int,3>&y){
35         return x[0] > y[0]; // decreasing weight
36     });
37
38     int totalNodes = 2 * N; // leaves 1..N, internal nodes N+1..2N-1 (at most N-1 of them)
39     dsuParent.assign(totalNodes + 1, 0);

```

3.26 Kruskal Reconstruction Tree - Trucks (cont.)

```

40 iota(dsuParent.begin(), dsuParent.end(), 0);
41
42 vector<int> nodeRepr(totalNodes + 1, 0);
43 for (int i = 1; i <= N; i++) nodeRepr[i] = i;
44
45 vector<int> leftChild(totalNodes + 1, 0), rightChild(totalNodes + 1, 0), weightOf(totalNodes + 1, 0);
46
47 int nextInternal = N + 1;
48 int merges = 0;
49 for (auto &e : edgeList) {
50     if (merges == N - 1) break;
51     int w = e[0], a = e[1], b = e[2];
52     int ra = find(a), rb = find(b);
53     if (ra == rb) continue;
54     int newNode = nextInternal++;
55     weightOf[newNode] = w;
56     leftChild[newNode] = nodeRepr[ra];
57     rightChild[newNode] = nodeRepr[rb];
58     dsuParent[ra] = rb; // union
59     nodeRepr[rb] = newNode; // rb is now the representative root (arbitrary choice), pointing to
60     o new node
61     merges++;
62 }
63
64 int root = nodeRepr[find(1)];
65
66 int totalUsed = nextInternal - 1; // highest node id actually used
67 int LOG = 1;
68 while ((1 << LOG) < totalUsed + 1) LOG++;
69 vector<vector<int>> up(LOG, vector<int>(totalUsed + 1, 0));
70 vector<int> depth(totalUsed + 1, 0);
71
72 // iterative pre-order traversal to set depth & up[0]
73 {
74     vector<int> stk;
75     stk.push_back(root);
76     depth[root] = 0;
77     up[0][root] = 0;
78     while (!stk.empty()) {
79         int u = stk.back(); stk.pop_back();
80         if (u <= N) continue; // leaf, no children
81         int lc = leftChild[u], rc = rightChild[u];
82         depth[lc] = depth[u] + 1; up[0][lc] = u; stk.push_back(lc);
83         depth[rc] = depth[u] + 1; up[0][rc] = u; stk.push_back(rc);
84     }
85 }
86
87 for (int k = 1; k < LOG; k++) {
88     for (int v = 1; v <= totalUsed; v++) {
89         up[k][v] = up[k-1][up[k-1][v]];
90     }
91 }
92
93 auto lca = [&](int u, int v) -> int {
94     if (depth[u] < depth[v]) swap(u, v);
95     int diff = depth[u] - depth[v];
96     for (int k = 0; k < LOG; k++) if ((diff >> k) & 1) u = up[k][u];
97     if (u == v) return u;
98     for (int k = LOG - 1; k >= 0; k--) {
99         if (up[k][u] != up[k][v]) { u = up[k][u]; v = up[k][v]; }
100     }
101     return up[0][u];
102 };
103
104 string out;
105 out.reserve(S * 8);
106 for (int i = 0; i < S; i++) {
107     int L = readInt(), H = readInt();
108     int anc = lca(L, H);
109     out += to_string(weightOf[anc]);
110     out += "\n";
111 }
112 fwrite(out.data(), 1, out.size(), stdout);

```

3.26 Kruskal Reconstruction Tree - Trucks (cont.)

```

10 return 0;
11 }

```

3.27 LCA (Binary Lifting) - Subway Lines

Resuelve "Subway Lines" (Day 8, problema L - Contest 08 [Avanzados]): en un sistema de subte con forma de arbol, las lineas de tren van y vuelven entre pares de estaciones terminales (hojas), pasando por el unico camino entre ellas; dadas Q consultas con dos pares de terminales (A,B) y (C,D), hay que calcular cuantas estaciones tienen en comun los caminos que definen esas dos lineas. Precalcula profundidad y ancestros con binary lifting (bfsSetup/up_) para responder lca(u,v) y dist(u,v) en $O(\log N)$, y solveQuery halla la interseccion de los dos caminos comparando las profundidades de lca(a,b) y lca(c,d), usando lca/isAncestor para ubicar los extremos de la interseccion y devolver su longitud (dist+1) o 0 si los caminos no se cruzan.

C++\Training_Camp_2026\Day 8\L - Subway Lines.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static char buf_[1 << 22];
5 static int buflen_, bufPos_;
6 inline int gc() {
7     if (bufPos_ == buflen_) { buflen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!buflen_) return -1; }
8     return buf_[bufPos_++];
9 }
10 inline int readInt() {
11     int c = gc();
12     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
13     bool neg = false; if (c == '-') { neg = true; c = gc(); }
14     int x = 0;
15     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
16     return neg ? -x : x;
17 }
18
19 int N, Q;
20 vector<vector<int>> adj;
21 vector<int> depth_;
22 int LOG_;
23 vector<vector<int>> up_;
24
25 void bfsSetup(int root) {
26     depth_.assign(N + 1, -1);
27     up_.assign(LOG_, vector<int>(N + 1, root));
28     depth_[root] = 0;
29     up_[0][root] = root;
30     queue<int> q;
31     q.push(root);
32     while (!q.empty()) {
33         int u = q.front(); q.pop();
34         for (int v : adj[u]) {
35             if (depth_[v] == -1) {
36                 depth_[v] = depth_[u] + 1;
37                 up_[0][v] = u;
38                 q.push(v);
39             }
40         }
41     }
42     for (int k = 1; k < LOG_; k++) {
43         for (int v = 1; v <= N; v++) {
44             up_[k][v] = up_[k-1][up_[k-1][v]];
45         }
46     }
47 }
48
49 int lca(int u, int v) {
50     if (depth_[u] < depth_[v]) swap(u, v);
51     int diff = depth_[u] - depth_[v];
52     for (int k = 0; k < LOG_; k++) if ((diff >> k) & 1) u = up_[k][u];
53     if (u == v) return u;
54     for (int k = LOG_ - 1; k >= 0; k--) {
55         if (up_[k][u] != up_[k][v]) { u = up_[k][u]; v = up_[k][v]; }
56     }

```

3.27 LCA (Binary Lifting) - Subway Lines (cont.)

```

57     return up[0][u];
58 }
59
60 inline int dist(int u, int v) {
61     return depth[u] + depth[v] - 2 * depth[lca(u, v)];
62 }
63
64 inline bool isAncestor(int u, int v) {
65     return lca(u, v) == u;
66 }
67
68 int solveQuery(int a, int b, int c, int d) {
69     int l1 = lca(a, b), l2 = lca(c, d);
70     if (depth[l1] > depth[l2]) {
71         swap(a, c); swap(b, d); swap(l1, l2);
72     }
73     // is l2 on path(a,b)?
74     if (dist(a, l2) + dist(l2, b) != dist(a, b)) return 0;
75
76     if (l1 == l2) {
77         int lac = lca(a, c), lad = lca(a, d);
78         int deepA = (depth[lac] >= depth[lad]) ? lac : lad;
79         int lbc = lca(b, c), lbd = lca(b, d);
80         int deepB = (depth[lbc] >= depth[lbd]) ? lbc : lbd;
81         return dist(deepA, deepB) + 1;
82     } else {
83         if (isAncestor(l2, a)) {
84             int lac = lca(a, c), lad = lca(a, d);
85             int deepA = (depth[lac] >= depth[lad]) ? lac : lad;
86             return dist(l2, deepA) + 1;
87         } else {
88             int lbc = lca(b, c), lbd = lca(b, d);
89             int deepB = (depth[lbc] >= depth[lbd]) ? lbc : lbd;
90             return dist(l2, deepB) + 1;
91         }
92     }
93 }
94
95 int main() {
96     N = readInt(); Q = readInt();
97     adj.assign(N + 1, {});
98     for (int i = 0; i < N - 1; i++) {
99         int u = readInt(), v = readInt();
100         adj[u].push_back(v);
101         adj[v].push_back(u);
102     }
103
104     LOG = 1;
105     while ((1 << LOG) < N + 1) LOG++;
106     bfsSetup(1);
107
108     string out;
109     out.reserve(Q * 7);
110     char tmp[16];
111     for (int q = 0; q < Q; q++) {
112         int a = readInt(), b = readInt(), c = readInt(), d = readInt();
113         int res = solveQuery(a, b, c, d);
114         int len = sprintf(tmp, "%d\n", res);
115         out.append(tmp, len);
116     }
117     fwrite(out.data(), 1, out.size(), stdout);
118     return 0;
119 }

```

3.28 Max Flow - Download Speed

Resuelve "Download Speed" (Day 7, problema H - Contest 07 [Avanzados]): dada una red de n computadoras y m conexiones dirigidas con capacidad c , hay que hallar la velocidad maxima a la que Kotivalo (computadora n) puede descargar datos del servidor (computadora 1) usando la red. Es el problema clasico de flujo maximo: la struct network implementa Dinic (BFS por niveles para construir el grafo por capas + DFS de flujo bloqueante con aristas residuales), y `max_flow(1, a)` calcula el flujo maximo entre el servidor y la computadora de Kotivalo, que es exactamente la velocidad de descarga pedida.

C++\Training_Camp_2026\Day 7\H - Download Speed.cpp

```

1 #include <bits/stdc++.h>
2
3 typedef long long ll;
4
5 using namespace std;
6
7 struct edge { ll v, cap, inv, flow; };
8
9 struct network
10 {
11     ll n, s, t;
12     vector<ll> lvl;
13     vector<vector<edge>> g;
14     network(ll n) : n(n), lvl(n), g(n) {}
15     void add_edge(ll u, ll v, ll c)
16     {
17         g[u].push_back({v, c, (ll)g[v].size(), 0});
18         g[v].push_back({u, 0, (ll)g[u].size()-1, c});
19     }
20
21     bool bfs()
22     {
23         fill(lvl.begin(), lvl.end(), -1);
24         queue<ll> q;
25         lvl[s] = 0;
26         for(q.push(s); q.size(); q.pop())
27         {
28             ll u = q.front();
29             for(auto &e : g[u]) {
30                 if(e.cap > 0 && lvl[e.v] == -1)
31                 {
32                     lvl[e.v] = lvl[u]+1;
33                     q.push(e.v);
34                 }
35             }
36         }
37         return lvl[t] != -1;
38     }
39
40     ll dfs(ll u, ll nf)
41     {
42         if(u == t) return nf;
43         ll res = 0;
44         for(auto &e : g[u])
45         {
46             if(e.cap > 0 && lvl[e.v] == lvl[u]+1)
47             {
48                 ll tf = dfs(e.v, min(nf, e.cap));
49                 res += tf; nf -= tf; e.cap -= tf;
50                 g[e.v][e.inv].cap += tf;
51                 g[e.v][e.inv].flow -= tf;
52                 e.flow += tf;
53                 if(nf == 0) return res;
54             }
55         }
56     }
57     if(!res) lvl[u] = -1;
58     return res;
59 }
60
61 ll max_flow(ll so, ll si, ll res = 0) {
62     s = so; t = si;

```

3.28 Max Flow - Download Speed (cont.)

```

63 while(bfs()) res += dfs(s, 1000000000000);
64 return res;
65 }
66 };
67
68 int main()
69 {
70
71     ll a,b; cin >> a >> b;
72     network pcs(a+1);
73     for(int i = 0; i < b;i++)
74     {
75         ll x,y,z;cin >> x >> y >> z;
76         pcs.add_edge(x,y,z);
77     }
78     cout << pcs.max_flow(1,a)<< endl;
79
80     return 0;
81 }

```

3.29 Max Flow - March Of The Penguins

Resuelve "March of the Penguins" (Day 7, problema J - Contest 07 [Avanzados]): hay N témpanos de hielo con coordenadas (x,y), cada uno con n_i pingüinos y un límite m_i de saltos que se pueden dar desde el (un pingüino puede saltar de un témpano a otro si la distancia entre ellos es $\leq D$); hay que hallar todos los témpanos donde TODOS los pingüinos puedan reunidos sin que ningún témpano se quede sin saltos. Para cada témpano candidato t, arma una red de flujo con nodos entrada/salida por témpano ($in=i$, $out=N+i$) donde la arista $in \rightarrow out$ tiene capacidad m_i (presupuesto de saltos), una fuente S conectada a cada témpano con capacidad n_i (pingüinos que parten de ahí), y aristas de capacidad infinita entre témpanos alcanzables saltando; corre Dinic (maxflow) de S a t y, si el flujo máximo iguala al total de pingüinos, t es una respuesta válida.

C++\Training_Camp_2026\Day 7J - March of the Penguins.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Dinic {
5     struct Edge { int to; int cap; int flow; };
6     int n;
7     vector<Edge> edges;
8     vector<vector<int>> g;
9     vector<int> level, it;
10    Dinic(int n=0): n(n), g(n) {}
11    int addEdge(int from,int to,int cap){
12        g[from].push_back((int)edges.size());
13        edges.push_back({to,cap,0});
14        g[to].push_back((int)edges.size());
15        edges.push_back({from,0,0});
16        return (int)edges.size()-2;
17    }
18    void resetFlow(){ for(auto &e: edges) e.flow = 0; }
19    bool bfs(int s,int t){
20        level.assign(n,-1);
21        queue<int> q; level[s]=0;q.push(s);
22        while(!q.empty()){
23            int u=q.front();q.pop();
24            for(int id: g[u]){
25                auto &e=edges[id];
26                if(e.cap - e.flow>0 && level[e.to]<0){
27                    level[e.to]=level[u]+1;
28                    q.push(e.to);
29                }
30            }
31        }
32        return level[t]>=0;
33    }
34    int dfs(int u,int t,int f){
35        if(u==t) return f;
36        for(int &i=it[u]; i<(int)g[u].size(); i++){
37            int id=g[u][i];
38            auto &e=edges[id];

```

3.29 Max Flow - March Of The Penguins (cont.)

```

39         if(e.cap-e.flow>0 && level[e.to]==level[u]+1){
40             int d=dfs(e.to,t,min(f,e.cap-e.flow));
41             if(d>0){ e.flow+=d; edges[id+1].flow-=d; return d; }
42         }
43     }
44     return 0;
45 }
46 int maxflow(int s,int t){
47     int flow=0;
48     while(bfs(s,t)){
49         it.assign(n,0);
50         int f;
51         while((f=dfs(s,t,INT_MAX))>0) flow+=f;
52     }
53     return flow;
54 }
55 };
56
57 int main(){
58     ios::sync_with_stdio(false);
59     cin.tie(nullptr);
60
61     int T;
62     if(!(cin >> T)) return 0;
63     while(T--){
64         int N; double D;
65         cin >> N >> D;
66         vector<double> x(N), y(N);
67         vector<int> n(N), m(N);
68         for(int i=0;i<N;i++) cin >> x[i] >> y[i] >> n[i] >> m[i];
69
70         int totalPenguins = 0;
71         for(int i=0;i<N;i++) totalPenguins += n[i];
72
73         vector<vector<bool>> canJump(N, vector<bool>(N,false));
74         const double eps = 1e-6;
75         for(int i=0;i<N;i++){
76             for(int j=i+1;j<N;j++){
77                 double dx = x[i]-x[j], dy = y[i]-y[j];
78                 double dist = sqrt(dx*dx+dy*dy);
79                 if(dist <= D + eps) canJump[i][j] = canJump[j][i] = true;
80             }
81         }
82         // in_node(i)=i, out_node(i)=N+i, S=2N
83         int S = 2*N;
84         int numNodes = 2*N+1;
85         Dinic dinic(numNodes);
86
87         for(int i=0;i<N;i++) dinic.addEdge(i, N+i, m[i]); // presupuesto de saltos
88         for(int i=0;i<N;i++) dinic.addEdge(S, i, n[i]); // pingüinos que parten de i
89
90         const int INF = 1000000;
91         for(int i=0;i<N;i++){
92             for(int j=i+1;j<N;j++){
93                 if(canJump[i][j]){
94                     dinic.addEdge(N+i, j, INF);
95                     dinic.addEdge(N+j, i, INF);
96                 }
97             }
98         }
99         vector<int> answer;
100        for(int t=0;t<N;t++){
101            dinic.resetFlow();
102            int f = dinic.maxflow(S, t); // sink = in_node(t)
103            if(f == totalPenguins) answer.push_back(t);
104        }
105        if(answer.empty()) cout << -1 << "\n";
106        else {
107            for(size_t i=0;i<answer.size();i++){
108                if(i) cout << ' ';
109                cout << answer[i];
110            }

```

3.29 Max Flow - March Of The Penguins (cont.)

```
111         cout << "\n";
112     }
113 }
114 return 0;
115 }
```

3.30 Max Flow (Binary Search + Dinic) - Gasoline

Resuelve "Gasoline" (Day 8, problema G - Contest 08 [Avanzados]): hay P gasolineras con demanda D_i y R refineras con stock E_j , con solo algunos pares (refineria, gasolinera) habilitados por contrato y un tiempo de viaje T conocido para esos pares; hay que hallar el menor tiempo t tal que, usando solo rutas con $T \leq t$, se pueda abastecer completamente a todas las gasolineras (repartiendo entre varias refineras si hace falta), o -1 si nunca alcanza. Hace busqueda binaria sobre los tiempos distintos y, para cada candidato t, feasible(t) arma una red fuente->refineria->gasolinera->sumidero (capacidades = stock, infinito solo si tiempo $\leq t$, y demanda) y corre Dinic (network::max_flow) comparando contra sumD.

C++ Training_Camp_2026/Day 8/G - Gasoline.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct edge {
5     int v, cap, inv, flow;
6 };
7
8 struct network {
9     int n, s, t;
10    vector<int> lvl;
11    vector<vector<edge>> g;
12    network(int n) : n(n), lvl(n), g(n) {}
13    void add_edge(int u, int v, int c) {
14        g[u].push_back({v, c, (int)g[v].size(), 0});
15        g[v].push_back({u, 0, (int)g[u].size()-1, c});
16    }
17    bool bfs() {
18        fill(lvl.begin(), lvl.end(), -1);
19        queue<int> q;
20        lvl[s] = 0;
21        for(q.push(s); q.size(); q.pop()) {
22            int u = q.front();
23            for (auto &e : g[u]) {
24                if(e.cap > 0 && lvl[e.v] == -1) {
25                    lvl[e.v] = lvl[u]+1;
26                    q.push(e.v);
27                }
28            }
29        }
30        return lvl[t] != -1;
31    }
32    int dfs(int u, int nf) {
33        if(u == t) return nf;
34        int res = 0;
35        for (auto &e : g[u]) {
36            if(e.cap > 0 && lvl[e.v] == lvl[u]+1) {
37                int tf = dfs(e.v, min(nf, e.cap));
38                res += tf;
39                nf -= tf;
40                e.cap -= tf;
41                g[e.v][e.inv].cap += tf;
42                g[e.v][e.inv].flow -= tf;
43                e.flow += tf;
44                if(nf == 0) return res;
45            }
46        }
47        if(!res) lvl[u] = -1;
48        return res;
49    }
50    int max_flow(int so, int si, int res = 0) {
51        s = so; t = si;
52        while(bfs()) res += dfs(s, INT_MAX);
53        return res;
54    }
```

3.30 Max Flow (Binary Search + Dinic) - Gasoline (cont.)

```
55 };
56
57 int P, R, C;
58 vector<int> D, E; // D[1..P], E[1..R]
59 vector<array<int,3>> edges; // I (station), J (refinery), T
60 long long sumD;
61
62 // nodos: 0 = source
63 // 1..R = refineras
64 // R+1..R+P = gasolineras
65 // R+P+1 = sumidero
66 bool feasible(int t) {
67     int n = R + P + 2;
68     int S = 0, T = R + P + 1;
69     network net(n);
70
71     // source -> refineria, capacidad = stock
72     for (int j = 1; j <= R; j++) {
73         net.add_edge(S, j, E[j]);
74     }
75     // gasolinera -> sumidero, capacidad = demanda
76     for (int i = 1; i <= P; i++) {
77         net.add_edge(R + i, T, D[i]);
78     }
79     // refineria -> gasolinera, solo si T_edge <= t
80     int INF = 1000000000;
81     for (auto &e : edges) {
82         int i = e[0], j = e[1], time = e[2];
83         if (time <= t) {
84             net.add_edge(j, R + i, INF);
85         }
86     }
87
88     long long f = net.max_flow(S, T);
89     return f >= sumD;
90 }
91
92 int main(){
93     ios::sync_with_stdio(false);
94     cin.tie(nullptr);
95
96     cin >> P >> R >> C;
97     D.assign(P+1, 0);
98     E.assign(R+1, 0);
99     for (int i = 1; i <= P; i++) cin >> D[i];
100    for (int j = 1; j <= R; j++) cin >> E[j];
101
102    edges.resize(C);
103    vector<int> times(C);
104    for (int k = 0; k < C; k++) {
105        int i, j, t;
106        cin >> i >> j >> t;
107        edges[k] = {i, j, t};
108        times[k] = t;
109    }
110
111    sumD = 0;
112    for (int i = 1; i <= P; i++) sumD += D[i];
113
114    // valores distintos de tiempo, ordenados
115    sort(times.begin(), times.end());
116    times.erase(unique(times.begin(), times.end()), times.end());
117
118    // primero chequeamos si es factible usando TODOS los arcos (t = maximo)
119    if (times.empty() || !feasible(times.back())) {
120        cout << -1 << "\n";
121        return 0;
122    }
123
124    // binary search: encontrar el menor t factible
125    int lo = 0, hi = (int)times.size() - 1;
126    while (lo < hi) {
```

3.30 Max Flow (Binary Search + Dinic) - Gasoline (cont.)

```

27     int mid = (lo + hi) / 2;
28     if (feasible(times[mid])) {
29         hi = mid;
30     } else {
31         lo = mid + 1;
32     }
33 }
34
35 cout << times[lo] << "\n";
36 return 0;
37 }

```

3.31 MST con Prim (Grafo Denso) - Brute

Resuelve "Anti-brute Force Lock" (problema B, CCPL 2026): una caja fuerte con N llaves de 4 dígitos que hay que desbloquear todas. Los diales arrancan en 0000, cada rueda gira de 0 a 9 dando la vuelta (de 9 se pasa a 0), y hay un boton JUMP que lleva los diales GRATIS a cualquier llave YA desbloqueada. Se pide el minimo total de giros. El costo entre dos combinaciones es la suma, rueda por rueda, de $\min(|x-y|, 10-|x-y|)$: girar por el lado corto del circulo. Esa es toda la funcion distancia. EL JUMP ES LO QUE CONVIERTE ESTO EN UN MST. Sin el, habria que recorrer las llaves en orden y el problema seria un TSP, o sea NP-difícil. Con el JUMP gratis, para abrir una llave nueva uno se para en la llave ya abierta que mas le convenga y paga solo ese tramo. Entonces el costo total es la suma de las aristas de un arbol que conecta todas las llaves: el minimo es el arbol de expansion minima. EL DETALLE FINO, Y ES LO QUE HACE A ESTE PROBLEMA MAS QUE UN MST DE LIBRO: el 0000 NO es una llave desbloqueada, asi que el JUMP nunca puede devolverte ahi. El 0000 sirve una sola vez, para la primerisima llave, y despues deja de existir. Por eso NO entra como nodo del grafo: se paga una vez $\min d(0000, \text{llave})$ y aparte se calcula el MST de las llaves SOLAS. Y se puede separar asi porque el MST de las llaves no depende de cual se abra primero: el peso del arbol es el mismo empiece Prim donde empiece. Entonces el total es $\min_k [d(0000, k) + \text{MST}(\text{llaves})]$ y minimizar cada parte por separado da el optimo global. SI EL 0000 SI FUERA SALTABLE la respuesta seria otra y MENOR: seria el MST del grafo con el 0000 adentro como un nodo mas, que puede gastar VARIAS aristas desde el 0000. Con las llaves {1000, 9000}: $d(0000, 1000)=1$, $d(0000, 9000)=1$, $d(1000, 9000)=2$. Como el 0000 no es saltable la respuesta es $1+2=3$; si lo fuera, seria $1+1=2$. Vale la pena tener claro el ejemplo, porque meter el origen al grafo por costumbre es el error natural aqui. POR QUE PRIM Y NO KRUSKAL: el grafo es COMPLETO, las aristas son N^2 . Prim con arreglo simple es $O(N^2)$, que con $N \leq 500$ son 250000 pasos y ni se siente. Kruskal tendria que ordenar las N^2 aristas, $O(N^2 \log N)$, y ademas construirlas todas en memoria. La regla: grafo denso, Prim con arreglo; grafo disperso, Kruskal con DSU o Prim con priority_queue. Verificado: los 4 casos del sample dan 16, 20, 26 y 17, y ademas 100000 casos aleatorios (incluidos adversarios con llaves pegadas al 0000) contra una DP de mascara de bits que simula las reglas del enunciado al pie de la letra, sin una sola diferencia.

C++\CCPL\brute.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 // Giros para pasar de la combinacion a a la b, por el lado corto de cada rueda.
6 int distancia(const string& a, const string& b)
7 {
8     int dis = 0;
9
10    for (int i = 0; i < 4; i++)
11    {
12        int x = a[i] - '0';
13        int y = b[i] - '0';
14
15        dis += min(abs(x - y), 10 - abs(x - y));
16    }
17
18    return dis;
19 }
20
21 int main()
22 {
23     ios::sync_with_stdio(false);
24     cin.tie(nullptr);
25
26     int T;
27     cin >> T;

```

3.31 MST con Prim (Grafo Denso) - Brute (cont.)

```

28
29     while (T--)
30     {
31         int N;
32         cin >> N;
33
34         vector<string> locks(N);
35
36         for (int i = 0; i < N; i++)
37             cin >> locks[i];
38
39         // Costo de llegar inicialmente desde 0000
40         // a cada una de las llaves.
41         int respuestaInicial = INT_MAX;
42
43         for (int i = 0; i < N; i++)
44         {
45             respuestaInicial = min(
46                 respuestaInicial,
47                 distancia("0000", locks[i])
48             );
49         }
50
51         // Prim solamente entre las llaves.
52         // 0000 NO forma parte del grafo.
53         const int INF = INT_MAX;
54
55         vector<int> mejor(N, INF);
56         vector<bool> visitado(N, false);
57
58         // Podemos comenzar el MST desde cualquier llave,
59         // porque el costo inicial desde 0000 ya lo escogimos
60         // como el minimo posible.
61         mejor[0] = 0;
62
63         int total = respuestaInicial;
64
65         for (int k = 0; k < N; k++)
66         {
67             int u = -1;
68
69             for (int i = 0; i < N; i++)
70             {
71                 if (!visitado[i] &&
72                     (u == -1 || mejor[i] < mejor[u]))
73                 {
74                     u = i;
75                 }
76             }
77
78             visitado[u] = true;
79             total += mejor[u];
80
81             for (int v = 0; v < N; v++)
82             {
83                 if (!visitado[v])
84                 {
85                     int peso = distancia(locks[u], locks[v]);
86
87                     mejor[v] = min(mejor[v], peso);
88                 }
89             }
90         }
91
92         cout << total << '\n';
93     }
94
95     return 0;
96 }

```

3.32 Quitar el DAG de Rutas Mínimas - Almost Shortest Path

O: (m log n), dos Dijkstra mas un barrido del DAG

Uso: leer el grafo dirigido, s y d; imprime la ruta mas corta que NO usa ninguna arista minima. Resuelve "Almost Shortest Path" (UVa 12144): dado un grafo dirigido con pesos, hallar la ruta mas corta de s a d que no use NINGUNA arista que aparezca en ALGUNA ruta minima de s a d. Son tres fases y la del medio es la que hay que entender: 1. Dijkstra desde s, guardando ademas los PREDECESORES de cada nodo sobre rutas minimas. 2. Desde d, ir hacia atras por esos predecesores marcando cada arista como prohibida. Eso recorre exactamente el DAG de rutas minimas de s a d, ni una arista mas. 3. Dijkstra otra vez saltando las prohibidas. Si d queda inalcanzable, la respuesta es -1. LA FASE 2 ES LA CLAVE: no basta con borrar las aristas tensas (dist[u] + w == dist[v]), porque hay aristas tensas que no llevan a d y esas SI se pueden usar. Por eso se entra al DAG desde d y no desde s. Ese es el error tipico del problema. Como se llenan los predecesores: si la distancia MEJORA en sentido estricto, la lista se REINICIA a ese unico predecesor; si EMPATA, se agrega. Ese par reiniciar/agregar es lo unico que hay que copiar bien, y sirve igual para contar rutas minimas o para el k-esimo camino. El caminos[s] = {-1} es el centinela que corta el barrido hacia atras al llegar al origen. OJO CON EL VISTO DEL BFS (la linea marcada abajo): sin el, un nodo se vuelve a encolar una vez por cada ruta que lo cruza, no una vez en total, y eso es EXPONENCIAL. Medido: con una escalera de capas de 2 nodos donde toda ruta es minima, sin el visto tarda 0.06 s con 46 nodos, 0.24 s con 50, 0.96 s con 54, y con 82 nodos y 160 aristas se queda sin memoria y sale sin imprimir nada. Con el visto, los mismos casos van en 0.01 s. Y 82 nodos caben de sobra en el limite del problema (n <= 500), asi que no es un caso de laboratorio. OJO CON LA MATRIZ paso: es n x n, o sea que con n grande no cabe, y colapsa las aristas paralelas (si hubiera dos a->b con pesos distintos, prohibir una prohibe las dos). Con n <= 500 va bien; si el limite sube, hay que marcar por INDICE de arista y no por par (a, b). VERIFICADO: el sample oficial (5 / -1 / 6) y 10.000 grafos aleatorios contra una implementacion independiente, 0 diferencias.

C++\Notebook\Almost Shortest Path.cpp

```
1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <utility>
5 #include <limits>
6 #include <algorithm>
7
8 using namespace std;
9
10 int main()
11 {
12     ios::sync_with_stdio(false);
13     cin.tie(nullptr);
14
15     long long n, m;
16     long long s, d;
17
18     while (cin >> n >> m && n != 0 && m != 0)
19     {
20         cin >> s >> d;
21
22         vector<vector<pair<long long, long long>>> conexiones(n);
23         vector<vector<bool>> paso(n, vector<bool>(n, false));
24
25         for (long long i = 0; i < m; i++)
26         {
27             long long a, b, c;
28             cin >> a >> b >> c;
29
30             conexiones[a].push_back({b, c});
31             paso[a][b] = true;
32         }
33
34         vector<vector<long long>> caminos(n);
35         vector<long long> distancia(n, LLONG_MAX);
36
37         priority_queue<
38             pair<long long, long long>,
39             vector<pair<long long, long long>>,
40             greater<pair<long long, long long>>
41             > cola;
42
```

3.32 Quitar el DAG de Rutas Mínimas - Almost Shortest Path (cont.)

```
43     caminos[s] = {-1};
44     distancia[s] = 0;
45     cola.push({0, s});
46
47     while (!cola.empty())
48     {
49         long long peso = cola.top().first;
50         long long actual = cola.top().second;
51         cola.pop();
52
53         if (peso != distancia[actual])
54         {
55             continue;
56         }
57
58         for (pair<long long, long long> siguiente : conexiones[actual])
59         {
60             long long hacia = siguiente.first;
61             long long cuanto = siguiente.second;
62
63             long long nuevaDistancia = peso + cuanto;
64
65             if (distancia[hacia] > nuevaDistancia)
66             {
67                 distancia[hacia] = nuevaDistancia;
68
69                 caminos[hacia] = {actual}; // mejora: se REINICIA la lista
70
71                 cola.push({nuevaDistancia, hacia});
72             }
73             else if (distancia[hacia] == nuevaDistancia)
74             {
75                 caminos[hacia].push_back(actual); // empate: se AGREGA
76             }
77         }
78     }
79
80     queue<long long> colita;
81     vector<bool> visto(n, false);
82     colita.push(d);
83     visto[d] = true;
84
85     while (!colita.empty())
86     {
87         long long ahora = colita.front();
88         colita.pop();
89
90         for (long long poraca : caminos[ahora])
91         {
92             if (poraca == -1) continue;
93             paso[poraca][ahora] = false;
94             if (visto[poraca]) continue; // sin esta linea el BFS es exponencial
95             visto[poraca] = true;
96             colita.push(poraca);
97         }
98     }
99
100     distancia.assign(n, LLONG_MAX);
101
102     distancia[s] = 0;
103     cola.push({0, s});
104
105     while (!cola.empty())
106     {
107         long long peso = cola.top().first;
108         long long actual = cola.top().second;
109         cola.pop();
110
111         if (peso != distancia[actual])
112         {
113             continue;
114         }
115
```

3.32 Quitar el DAG de Rutas Mínimas - Almost Shortest Path (cont.)

```

15     }
16
17     for(pair<long long, long long> siguiente : conexiones[actual])
18     {
19         long long hacia = siguiente.first;
20         long long cuanto = siguiente.second;
21
22         if(!paso[actual][hacia]) continue;
23
24         long long nuevaDistancia = peso + cuanto;
25
26         if(distancia[hacia] > nuevaDistancia)
27         {
28             distancia[hacia] = nuevaDistancia;
29
30             cola.push({nuevaDistancia, hacia});
31         }
32     }
33 }
34
35 cout << (distancia[d] == LLONG_MAX ? -1 : distancia[d]) << '\n';
36 }
37
38 return 0;
39 }

```

3.33 State-Space BFS - Folding Machine

Resuelve "Folding Machine" (Day 2, problema D - Maratona de Programacao da SBC 2013): una cinta de N celdas se puede doblar por cualquier punto entre celdas adyacentes (incluso en los extremos, lo que invierte la cinta), sumando los valores de las celdas que quedan superpuestas. Dado un tape de entrada y uno de salida, determina si existe una secuencia de dobleces que transforme el primero en el segundo. Como cada doblez real reduce estrictamente el largo de la cinta, genera todos los estados alcanzables agrupandolos por longitud (states[*len*]), probando cada pliegue posible de *h* celdas desde la izquierda o la derecha; *normalized()* reduce cada estado a su forma canonica (el minimo entre la cinta y su reverso) para no explorar duplicados. Al final solo compara si el output normalizado aparece entre los estados generados de longitud *M*.

C++\Training_Camp_2026\Day 2\D - Folding Machine.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 // A fold at either end may reverse the tape, so both directions
7 // represent the same state.
8 vector<ll> normalized(vector<ll> tape) {
9     vector<ll> reversed = tape;
10    reverse(reversed.begin(), reversed.end());
11    return min(tape, reversed);
12 }
13
14 int main() {
15     ios::sync_with_stdio(false);
16     cin.tie(nullptr);
17
18     int n;
19     cin >> n;
20     vector<ll> input(n);
21     for (ll &x : input) cin >> x;
22
23     int m;
24     cin >> m;
25     vector<ll> output(m);
26     for (ll &x : output) cin >> x;
27
28     // The sum of all cells never changes after a fold.
29     if (accumulate(input.begin(), input.end(), 0LL) !=
30         accumulate(output.begin(), output.end(), 0LL)) {
31         cout << "N\n";
32         return 0;
33     }

```

3.33 State-Space BFS - Folding Machine (cont.)

```

34
35     vector<set<vector<ll>>> states(n + 1);
36     states[n].insert(normalized(input));
37
38     // Every real fold lowers the tape length. Thus a state only needs
39     // to be generated once, in the set indexed by its length.
40     for (int len = n; len > m; --len) {
41         for (const vector<ll> &tape : states[len]) {
42             for (int h = 1; h <= len / 2; ++h) {
43                 // Fold the h cells at the left end over the rest.
44                 vector<ll> next;
45                 next.reserve(len - h);
46                 for (int j = 0; j < h; ++j)
47                     next.push_back(tape[h + j] + tape[h - 1 - j]);
48                 for (int j = 2 * h; j < len; ++j)
49                     next.push_back(tape[j]);
50                 if (len - h >= m)
51                     states[len - h].insert(normalized(next));
52
53                 // Fold the h cells at the right end over the rest.
54                 next.clear();
55                 const int leftSize = len - h;
56                 for (int j = 0; j < leftSize - h; ++j)
57                     next.push_back(tape[j]);
58                 for (int j = 0; j < h; ++j)
59                     next.push_back(tape[leftSize - h + j] + tape[len - 1 - j]);
60                 if (len - h >= m)
61                     states[len - h].insert(normalized(next));
62             }
63         }
64     }
65
66     cout << (states[m].count(normalized(output)) ? "S\n" : "N\n");
67 }

```

3.34 Strongly Connected Components (Tarjan) - Proving Equivalences

Resuelve "Proving Equivalences" (Day 4, problema O - Contest 04 [Avanzados]): dadas *n* proposiciones y *m* implicaciones ya probadas entre ellas, hay que calcular el minimo numero de implicaciones adicionales necesarias para probar que todas son equivalentes entre si. Calcula las componentes fuertemente conexas con el algoritmo de Tarjan (tarjanSCC), condensa el grafo en un DAG de SCCs y cuenta cuantas componentes no tienen aristas entrantes (hasIn) ni salientes (hasOut) en ese DAG; la respuesta es el maximo entre la cantidad de fuentes y sumideros (0 si ya es una unica SCC, que es la minima cantidad de aristas para volverlo fuerte).

C++\Training_Camp_2026\Day 4\O - Proving Equivalences.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static char buf_[1 << 20];
5 static int buflen_, bufPos_;
6 inline int gc() {
7     if (bufPos_ == buflen_) { buflen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!buflen_) return -1; }
8     return buf_[bufPos_++];
9 }
10 inline int readInt() {
11     int c = gc();
12     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
13     bool neg = false; if (c == '-') { neg = true; c = gc(); }
14     int x = 0;
15     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
16     return neg ? -x : x;
17 }
18
19 int n, m;
20 vector<vector<int>> adj;
21 vector<int> disc_, low_, comp_;
22 vector<char> onStack_;
23 vector<int> tarjanStack;
24 int timer_, sccCount;
25

```

3.34 Strongly Connected Components (Tarjan) - Proving Equivalences (cont.)

```

26 // Iterative Tarjan's SCC algorithm.
27 void tarjanSCC() {
28     disc_.assign(n + 1, -1);
29     low_.assign(n + 1, -1);
30     comp_.assign(n + 1, -1);
31     onStack_.assign(n + 1, 0);
32     tarjanStack.clear();
33     timer_ = 0;
34     sccCount = 0;
35
36     for (int s = 1; s <= n; s++) {
37         if (disc_[s] != -1) continue;
38         vector<array<int, 2>> stk; // {vertex, nextIdx}
39         stk.push_back({s, 0});
40         disc_[s] = low_[s] = timer_++;
41         tarjanStack.push_back(s);
42         onStack_[s] = 1;
43
44         while (!stk.empty()) {
45             auto &fr = stk.back();
46             int u = fr[0];
47             if (fr[1] < (int)adj[u].size()) {
48                 int v = adj[u][fr[1]];
49                 fr[1]++;
50                 if (disc_[v] == -1) {
51                     disc_[v] = low_[v] = timer_++;
52                     tarjanStack.push_back(v);
53                     onStack_[v] = 1;
54                     stk.push_back({v, 0});
55                 } else if (onStack_[v]) {
56                     low_[u] = min(low_[u], disc_[v]);
57                 }
58             } else {
59                 stk.pop_back();
60                 if (!stk.empty()) {
61                     int p = stk.back()[0];
62                     low_[p] = min(low_[p], low_[u]);
63                 }
64                 if (low_[u] == disc_[u]) {
65                     // pop the SCC
66                     while (true) {
67                         int w = tarjanStack.back();
68                         tarjanStack.pop_back();
69                         onStack_[w] = 0;
70                         comp_[w] = sccCount;
71                         if (w == u) break;
72                     }
73                     sccCount++;
74                 }
75             }
76         }
77     }
78 }
79
80 int main() {
81     int T = readInt();
82     string out;
83     while (T--) {
84         n = readInt(); m = readInt();
85         adj.assign(n + 1, {});
86         vector<pair<int, int>> rawEdges(m);
87         for (int i = 0; i < m; i++) {
88             int a = readInt(), b = readInt();
89             rawEdges[i] = {a, b};
90             adj[a].push_back(b);
91         }
92
93         tarjanSCC();
94
95         vector<char> hasIn(sccCount, 0), hasOut(sccCount, 0);
96         for (auto &e : rawEdges) {
97             int ca = comp_[e.first], cb = comp_[e.second];

```

3.34 Strongly Connected Components (Tarjan) - Proving Equivalences (cont.)

```

98             if (ca != cb) {
99                 hasOut[ca] = 1;
100                hasIn[cb] = 1;
101            }
102        }
103
104        int ans;
105        if (sccCount <= 1) {
106            ans = 0;
107        } else {
108            int sources = 0, sinks = 0;
109            for (int i = 0; i < sccCount; i++) {
110                if (!hasIn[i]) sources++;
111                if (!hasOut[i]) sinks++;
112            }
113            ans = max(sources, sinks);
114        }
115        out += to_string(ans);
116        out += "\n";
117    }
118    fwrite(out.data(), 1, out.size(), stdout);
119    return 0;
120 }

```

3.35 Tree Degree Analysis - Colmillos

Calcula cuántos pares de nodos pueden removerse de un árbol para desconectarlo. Es n menos la suma de los dos mayores grados de los nodos.

C++(Codeforces)colmillos.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9     cin >> n;
10
11     vector<int> deg(n + 1, 0);
12
13     for (int i = 0; i < n - 1; i++) {
14         int a, b;
15         cin >> a >> b;
16
17         deg[a]++;
18         deg[b]++;
19     }
20
21     int mx1 = 0, mx2 = 0;
22
23     for (int i = 1; i <= n; i++) {
24         if (deg[i] >= mx1) {
25             mx2 = mx1;
26             mx1 = deg[i];
27         } else if (deg[i] > mx2) {
28             mx2 = deg[i];
29         }
30     }
31
32     cout << n - mx1 - mx2 << '\n';
33
34     return 0;
35 }

```

3.36 Tree Reconstruction (BFS por capas) - Expansion Of The Road Network

Resuelve "Expansion of the road network" (Day 6, problema E - Maratona SBC de Programacao 2025): el mapa actual de N ciudades y M caminos pudo haber surgido de un arbol original (un camino unico entre cada par de ciudades) al que se le agrego una carretera directa entre cada par de ciudades cuyo camino original pasaba por exactamente una ciudad intermedia (distancia 2 en el arbol). Hay que decidir si el mapa actual es consistente con ese proceso y, de serlo, reconstruir un arbol original valido (o imprimir "*" si es imposible). El codigo hace BFS desde el nodo 1 para obtener una hoja candidata del arbol original (la de menor grado en la ultima capa), luego hace BFS desde esa hoja para obtener las capas "layer[]", identifica los posibles vertices "soporte" intersectando pares de padres de los nodos de profundidad ≥ 2 , construye el arbol candidato asignando "parent[]" capa por capa segun la paridad de profundidad, y por ultimo "verify" comprueba que los caminos de distancia 1 y 2 de ese arbol coincidan exactamente con las M aristas dadas antes de imprimirlo.

C++\Training_Camp_2026\Day 6\E - Expansion of the road network.py

```
1 import sys
2 from collections import deque
3
4 def solve(data):
5     n = int(data[0]); m = int(data[1])
6     B = n + 1
7     adj = [[] for _ in range(B)]
8     eset = set()
9     ptr = 2
10    for _ in range(m):
11        a = int(data[ptr]); b = int(data[ptr + 1]); ptr += 2
12        adj[a].append(b); adj[b].append(a)
13        eset.add(a * B + b if a < b else b * B + a)
14
15    # Grafo completo <=> el arbol original era una estrella
16    if m == n * (n - 1) // 2:
17        return '\n'.join('1 %d' % i for i in range(2, n + 1))
18
19    def bfs(src):
20        dist = [-1] * B
21        dist[src] = 0
22        q = deque([src])
23        while q:
24            u = q.popleft()
25            du = dist[u] + 1
26            for v in adj[u]:
27                if dist[v] < 0:
28                    dist[v] = du
29                    q.append(v)
30        return dist
31
32    # 1) Hoja garantizada del arbol original:
33    # el de grado minimo dentro de la ultima capa BFS
34    d1 = bfs(1)
35    maxd = max(d1[1:])
36    leaf = -1; best = float('inf')
37    for v in range(1, n + 1):
38        if d1[v] == maxd and len(adj[v]) < best:
39            best = len(adj[v]); leaf = v
40
41    # 2) Capas BFS desde la hoja: la capa k contiene las profundidades
42    # 2k-2 y 2k-1 del arbol enraizado en el soporte de la hoja
43    layer = bfs(leaf)
44    inK = bytearray(B)
45    for v in adj[leaf]:
46        inK[v] = 1
47
48    # 3) Candidatos a soporte: todo vertice externo con exactamente 2 vecinos
49    # dentro de N(hoja) es "nieto" del soporte y esos 2 vecinos son
50    # {soporte, su padre}. Interseccion de esos pares => <= 2 candidatos.
51    cand = None
52    for z in range(1, n + 1):
53        if layer[z] >= 2:
54            c = 0; p1 = 0; p2 = 0
55            for y in adj[z]:
56                if inK[y]:
57                    c += 1
```

3.36 Tree Reconstruction (BFS por capas) - Expansion Of The Road Network (cont.)

```
58     if c == 1: p1 = y
59     elif c == 2: p2 = y
60     else: break
61     if c == 2:
62         if cand is None:
63             cand = {p1, p2}
64         else:
65             cand.intersection_update({p1, p2})
66             if not cand:
67                 return '*'
68     if not cand:
69         return '*'
70
71    maxl = max(layer[1:])
72    layers = [[] for _ in range(maxl + 1)]
73    for v in range(1, n + 1):
74        layers[layer[v]].append(v)
75
76    def build(w):
77        parent = [0] * B
78        role = bytearray(B) # 1 = prof. par ("arriba"), 2 = prof. impar ("abajo")
79        role[w] = 1
80        parent[leaf] = w
81        for v in layers[1]:
82            if v != w:
83                role[v] = 2
84                parent[v] = w
85        for k in range(2, maxl + 1):
86            lows = []
87            km1 = k - 1
88            for x in layers[k]:
89                c = 0; a = 0; b = 0
90                for y in adj[x]:
91                    if layer[y] == km1:
92                        c += 1
93                    if c == 1: a = y
94                    elif c == 2: b = y
95                    else: return None
96            if c == 2:
97                ra = role[a]
98                if ra == role[b]: return None
99                if ra == 1: a, b = b, a
100               if parent[a] != b: return None # el "arriba" debe ser el abuelo
101               role[x] = 1
102               parent[x] = a
103            elif c == 1:
104                if role[a] != 2: return None # unico vecino previo = abuelo
105                role[x] = 2
106                lows.append((x, a))
107            else:
108                return None
109            for x, g in lows:
110                p = 0
111                for y in adj[x]:
112                    if layer[y] == k and role[y] == 1 and parent[y] == g:
113                        p = y; break
114                if not p: return None
115                parent[x] = p
116            return parent
117
118    def verify(parent, root):
119        degT = [0] * B
120        for v in range(1, n + 1):
121            if v != root:
122                degT[v] += 1
123                degT[parent[v]] += 1
124        extra = m - (n - 1)
125        s = 0
126        for v in range(1, n + 1):
127            d = degT[v]
128            s += d * (d - 1) // 2
129            if s > extra: return False
```

3.36 Tree Reconstruction (BFS por capas) - Expansion Of The Road Network (cont.)

```

80     if s != extra: return False
81     ch = [[] for _ in range(8)]
82     for v in range(1, n + 1):
83         if v != root:
84             p = parent[v]
85             ch[p].append(v)
86             key = v * B + p if v < p else p * B + v
87             if key not in eset: return False
88     for v in range(1, n + 1):
89         nb = ch[v]
90         if v != root:
91             nb = nb + [parent[v]]
92         ln = len(nb)
93         for i in range(ln - 1):
94             x = nb[i]
95             for j in range(i + 1, ln):
96                 y = nb[j]
97                 key = x * B + y if x < y else y * B + x
98                 if key not in eset: return False
99     return True
100
101     for w in cand:
102         parent = build(w)
103         if parent is not None and verify(parent, w):
104             out = []
105             ap = out.append
106             for v in range(1, n + 1):
107                 if v != w:
108                     ap('%d %d' % (v, parent[v]))
109             return '\n'.join(out)
110     return '*'
111
112 def main():
113     data = sys.stdin.buffer.read().split()
114     sys.stdout.write(solve(data) + '\n')
115
116 if __name__ == '__main__':
117     main()

```

4 - Geometry

4.1 2D Difference Array (Coordinate Compression) - Office Space

Resuelve "Office Space" (Day 11, problema K - CCPL (Kattis)): en una oficina rectangular de $w \times h$ pies, hasta 20 empleados piden un cubiculo rectangular (esquinas suroeste y noreste); pide reportar el area total, el area sin reclamar, el area en conflicto (solicitada por 2 o mas empleados) y, para cada empleado, el area que tiene garantizada (lo que pidio menos lo que se solapa con otros). Comprime las coordenadas x/y de las esquinas de todos los rectangulos en xs'/ys' , y usa un arreglo de diferencias 2D ('grid') para contar cuantas solicitudes cubren cada celda comprimida, junto con un arreglo paralelo $sumIdx$ que acumula (indice+1) del empleado para identificar sin ambigüedad al unico dueño cuando una celda queda cubierta exactamente una vez; una suma prefija 2D final obtiene el conteo y dueño real de cada celda.

C++/Training_Camp_2026/Day 11 - CCPL/K - Office Space.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 static char buf_[1 << 22];
6 static int bufLen_, bufPos_;
7 inline int gc() {
8     if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!bufLen_) return -1; }
9     return buf_[bufPos_++];
10 }
11 inline bool readInt(int &out) {
12     int c = gc();
13     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
14     if (c == -1) return false;
15     bool neg = false; if (c == '-') { neg = true; c = gc(); }
16     int x = 0;

```

4.1 2D Difference Array (Coordinate Compression) - Office Space (cont.)

```

17     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
18     out = neg ? -x : x;
19     return true;
20 }
21 inline bool isSpaceChar(int c) { return c == ' ' || c == '\n' || c == '\r' || c == '\t'; }
22 inline bool readWord(string &s) {
23     s.clear();
24     int c = gc();
25     while (c != -1 && isSpaceChar(c)) c = gc();
26     if (c == -1) return false;
27     while (c != -1 && !isSpaceChar(c)) { s.push_back((char)c); c = gc(); }
28     return true;
29 }
30
31 int main() {
32     int W, H;
33     string out;
34     while (readInt(W)) {
35         readInt(H);
36         int n;
37         readInt(n);
38
39         vector<string> names(n);
40         vector<int> X1(n), Y1(n), X2(n), Y2(n);
41         vector<int> xs, ys;
42         xs.reserve(2 * n + 2);
43         ys.reserve(2 * n + 2);
44         xs.push_back(0); xs.push_back(W);
45         ys.push_back(0); ys.push_back(H);
46
47         for (int i = 0; i < n; i++) {
48             readWord(names[i]);
49             readInt(X1[i]); readInt(Y1[i]); readInt(X2[i]); readInt(Y2[i]);
50             xs.push_back(X1[i]); xs.push_back(X2[i]);
51             ys.push_back(Y1[i]); ys.push_back(Y2[i]);
52         }
53
54         sort(xs.begin(), xs.end());
55         xs.erase(unique(xs.begin(), xs.end()), xs.end());
56         sort(ys.begin(), ys.end());
57         ys.erase(unique(ys.begin(), ys.end()), ys.end());
58
59         int M = (int)xs.size() - 1; // number of cell columns
60         int K = (int)ys.size() - 1; // number of cell rows
61         int W2 = K + 1; // row width for flat indexing over the (M+1) x (K+1) corner gr
62
63         auto xIdx = [&](int v) { return (int)(lower_bound(xs.begin(), xs.end(), v) - xs.begin()); };
64         auto yIdx = [&](int v) { return (int)(lower_bound(ys.begin(), ys.end(), v) - ys.begin()); };
65
66         auto at = [&](int i, int j) { return i * W2 + j; };
67
68         // Flat 2D difference array (row-major) over the (M+1) x (K+1) grid of "corner"
69         // points, computed in place into the count grid (avoids a second full grid, and
70         // flat 1D storage avoids the per-row bookkeeping overhead of nested vectors).
71         vector<int> grid((size_t)(M + 1) * (K + 1), 0);
72         // Parallel difference array summing (employee_index + 1) instead of 1. When a
73         // cell's final count is exactly 1, this sum unambiguously equals that single
74         // covering employee's (index + 1), letting us identify the owner without a
75         // separate large prefix-sum array.
76         vector<ll> sumIdx((size_t)(M + 1) * (K + 1), 0);
77
78         for (int i = 0; i < n; i++) {
79             int a = xIdx(X1[i]), b = xIdx(X2[i]);
80             int c = yIdx(Y1[i]), d = yIdx(Y2[i]);
81             grid[at(a, c)] += 1;
82             grid[at(b, c)] -= 1;
83             grid[at(a, d)] -= 1;
84             grid[at(b, d)] += 1;
85
86             ll idPlus1 = (ll)i + 1;

```

4.1 2D Difference Array (Coordinate Compression) - Office Space (cont.)

```

86     sumIdx[at(a, c)] += idPlus1;
87     sumIdx[at(b, c)] -= idPlus1;
88     sumIdx[at(a, d)] -= idPlus1;
89     sumIdx[at(b, d)] += idPlus1;
90 }
91
92 // 2D prefix sum in place -> grid[.] becomes the count of rectangles covering each
93 // cell, and sumIdx[.] becomes the sum of (index+1) of covering rectangles.
94 for (int i = 0; i <= M; i++) {
95     int rowBase = i * W2;
96     int prevRowBase = rowBase - W2;
97     for (int j = 0; j <= K; j++) {
98         int idx = rowBase + j;
99         if (i > 0) { grid[idx] += grid[prevRowBase + j]; sumIdx[idx] += sumIdx[prevRowBase
100 + j]; }
101         if (j > 0) { grid[idx] += grid[idx - 1]; sumIdx[idx] += sumIdx[idx - 1]; }
102         if (i > 0 && j > 0) { grid[idx] -= grid[prevRowBase + j - 1]; sumIdx[idx] -= sumIdx
103 [prevRowBase + j - 1]; }
104     }
105 }
106
107 ll total = (ll)W * H;
108 ll unalloc = 0, contested = 0;
109 vector<ll> guaranteed(n, 0);
110
111 for (int i = 0; i < M; i++) {
112     ll colWidth = (ll)(xs[i + 1] - xs[i]);
113     int rowBase = i * W2;
114     for (int j = 0; j < K; j++) {
115         ll cellArea = colWidth * (ll)(ys[j + 1] - ys[j]);
116         int c = grid[rowBase + j];
117         if (c == 0) {
118             unalloc += cellArea;
119         } else if (c >= 2) {
120             contested += cellArea;
121         } else { // c == 1
122             int owner = (int)(sumIdx[rowBase + j] - 1);
123             guaranteed[owner] += cellArea;
124         }
125     }
126 }
127
128 out += "Total " + to_string(total) + "\n";
129 out += "Unallocated " + to_string(unalloc) + "\n";
130 out += "Contested " + to_string(contested) + "\n";
131
132 for (int i = 0; i < n; i++) {
133     out += names[i] + " " + to_string(guaranteed[i]) + "\n";
134 }
135 out += "\n";
136 }
137
138 fwrite(out.data(), 1, out.size(), stdout);
139 return 0;
140 }

```

4.2 Area Circulo-Poligono

O: (n) por círculo, con n = lados del poligono

Uso: circleRectangleArea(cx,cy,r,w,h); triangleCircleArea(p,q,r)

Area de la interseccion entre un círculo y un poligono, usando el truco de sumar areas con signo arista por arista: se traslada todo para dejar el círculo en el origen y se recorre el poligono sumando triangleCircleArea de cada arista, de modo que las contribuciones de fuera se cancelan solas y al final basta el valor absoluto.

Funciona con el poligono en cualquier orientacion y sin importar si el círculo queda dentro, fuera o a medias.

Para cada arista p-q se resuelve $|p + t*d|^2 = r^2$, una cuadratica en t, para hallar donde corta la circunferencia; esos cortes parten el segmento en tramos. Cada tramo se clasifica mirando su punto medio: si cae dentro del círculo aporta el area del triangulo (cross/2), y si cae fuera aporta la del sector circular ($r^2 * \text{angulo} / 2$, con atan2 para quedarse con el angulo con signo correcto). Aquí se usa con un rectangulo [0,w]x[0,h] para calcular la probabilidad de que un punto uniforme caiga dentro del círculo, y con eso el valor esperado de la suma de los v.

C++(Notebook)Area Circulo-Poligono.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const double EPS = 1e-12;
5
6 struct Pt { double x, y; };
7
8 double cross(Pt a, Pt b) { return a.x * b.y - a.y * b.x; }
9 double dot(Pt a, Pt b) { return a.x * b.x + a.y * b.y; }
10 Pt sub(Pt a, Pt b) { return {a.x - b.x, a.y - b.y}; }
11 Pt add(Pt a, Pt b) { return {a.x + b.x, a.y + b.y}; }
12 Pt mul(Pt a, double t) { return {a.x * t, a.y * t}; }
13 double norm2(Pt a) { return dot(a, a); }
14
15 // Area con signo de la interseccion entre el círculo centrado en (0,0) de radio r
16 // y el triangulo (0,0)-p-q.
17 double triangleCircleArea(Pt p, Pt q, double r) {
18     Pt d = sub(q, p);
19
20     // |p + t*d|^2 = r^2 -> A t^2 + B t + C = 0
21     double A = norm2(d);
22     double B = 2.0 * dot(p, d);
23     double C = norm2(p) - r * r;
24
25     vector<double> cuts = {0.0, 1.0};
26
27     if (A > EPS) {
28         double disc = B * B - 4.0 * A * C;
29         if (disc > EPS) {
30             double s = sqrt(disc);
31             double t1 = (-B - s) / (2.0 * A);
32             double t2 = (-B + s) / (2.0 * A);
33             if (t1 > EPS && t1 < 1.0 - EPS) cuts.push_back(t1);
34             if (t2 > EPS && t2 < 1.0 - EPS) cuts.push_back(t2);
35         }
36     }
37
38     sort(cuts.begin(), cuts.end());
39
40     double ans = 0.0;
41     for (size_t i = 0; i + 1 < cuts.size(); i++) {
42         double l = cuts[i], rr = cuts[i + 1];
43         double mid = (l + rr) / 2.0;
44
45         Pt a = add(p, mul(d, l));
46         Pt b = add(p, mul(d, rr));
47         Pt m = add(p, mul(d, mid));
48
49         if (norm2(m) <= r * r) {
50             ans += cross(a, b) / 2.0; // tramo dentro: triangulo
51         } else {
52             ans += r * r * atan2(cross(a, b), dot(a, b)) / 2.0; // tramo fuera: sector
53         }
54     }
55     return ans;

```

4.2 Area Circulo-Poligono (cont.)

```

56 }
57
58 double circleRectangleArea(double cx, double cy, double r, double w, double h) {
59     // Trasladar para que el circulo quede en el origen.
60     Pt rect[4] = {{-cx, -cy}, {w - cx, -cy}, {w - cx, h - cy}, {-cx, h - cy}};
61
62     double area = 0.0;
63     for (int i = 0; i < 4; i++) area += triangleCircleArea(rect[i], rect[(i + 1) % 4], r);
64     return fabs(area);
65 }
66
67 int main() {
68     ios::sync_with_stdio(false);
69     cin.tie(nullptr);
70
71     int n;
72     double r, w, h;
73     cin >> n >> r >> w >> h;
74
75     double ans = 0.0;
76     for (int i = 0; i < n; i++) {
77         double x, y, v;
78         cin >> x >> y >> v;
79         ans += v * circleRectangleArea(x, y, r, w, h) / (w * h);
80     }
81
82     cout << fixed << setprecision(15) << ans << "\n";
83     return 0;
84 }

```

4.3 Bounding Box de una Elipse Rotada - Eclipse

NO ES MIO: código de Stiven Correa, del repo del equipo (carpeta RPC/). Resuelve "Eclipse" (RPC 2026-07, problema E): dados los dos focos de una elipse y el largo a de la cuerda, dar el rectangulo mas pequeno (alineado a los ejes) que la contiene. Tecnica: de la definicion de elipse, el semieje mayor es $A = a/2$ y la distancia del centro a cada foco es $c = d/2$, así que el semieje menor sale de $B = \text{raiz}(A^2 - c^2)$. El centro es el punto medio de los focos. La parte que vale: la elipse esta ROTADA (el eje mayor va en la direccion de un foco al otro), y la extension en X de una elipse rotada es $\text{raiz}((A*ux)^2 + (B*vx)^2)$, donde u es el vector unitario del eje mayor y v el perpendicular. Igual para Y. Es la formula del bounding box de una elipse en cualquier orientacion, y no es obvia si uno no la ha visto.

RPC/2026-07/E - Eclipse.py

```

1 import math
2
3 x1, y1, x2, y2, a = map(int, input().split())
4
5 # Distancia entre los focos
6 d = ((x2 - x1)**2 + (y2 - y1)**2)**(1/2)
7
8 # Semieje mayor
9 A = a / 2
10
11 # Distancia del centro a cada foco
12 c = d / 2
13
14 # Semieje menor
15 B = math.sqrt(A * A - c * c)
16
17 # Centro
18 cx = (x1 + x2) / 2
19 cy = (y1 + y2) / 2
20
21 # Direccion del eje mayor
22 ux = (x2 - x1) / d
23 uy = (y2 - y1) / d
24
25 # Direccion perpendicular
26 vx = -uy
27 vy = ux
28
29 # Extension en X

```

4.3 Bounding Box de una Elipse Rotada - Eclipse (cont.)

```

30 rx = math.sqrt((A * ux) ** 2 + (B * vx) ** 2)
31
32 # Extension en Y
33 ry = math.sqrt((A * uy) ** 2 + (B * vy) ** 2)
34
35 print(cx - rx, cy - ry, cx + rx, cy + ry)

```

4.4 Contar Cruces de Cuerdas con Fenwick - Lonely Creatures

Resuelve "Lonely Creatures" (problema L, Regionals 2025): cada criatura define una recta que corta una parabola en dos puntos, o sea una CUERDA de la region, y hay que contar cuantos pares de cuerdas se cruzan por dentro. DOS PASOS, Y EL PRIMERO ES EL QUE SE PIENSA: 1) Sacar los extremos de cada cuerda resolviendo la ecuacion cuadratica $Ax^2 - Mx + (B-C) = 0$. El discriminante $D = M^2 - 4A(B-C)$ decide: si es negativo o cero la recta no entra al interior y esa criatura se descarta. 2) Dos cuerdas $(l1, r1)$ y $(l2, r2)$ se cruzan por dentro si y solo si sus extremos se INTERCALAN: $l1 < l2 < r1 < r2$. Ordenando por el extremo izquierdo, contar los cruces se vuelve contar INVERSIONES sobre los extremos derechos, que es lo que hace el Fenwick. El desempate importa: si dos cuerdas comparten el extremo izquierdo no se cruzan por dentro (se tocan en la frontera), y por eso el orden y las comparaciones usan $<$ y no $<=$. Ese detalle cambia la respuesta y no lo avisa ningun caso chico. Los extremos son reales, así que se comprimen con lowerBound y upperBound escritos a mano sobre long double, y con EPS para los empates. Cuando se puede, es preferible comparar en enteros (como en "C - Clean Streets"); aqui las raices son irracionales y no queda opcion. Es el mismo conteo de inversiones de "frosh" (Frosh Week, CCPL) de este cuaderno, aplicado a geometria.

C++\VCPC\Regionals 2025\L - Lonely Creatures.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5 using ld = long double;
6
7 const ld EPS = 1e-12L;
8
9 struct Chord {
10     ld L, R;
11 };
12
13 struct Fenwick {
14     int n;
15     vector<int> bit;
16
17     Fenwick(int n) : n(n), bit(n + 1, 0) {}
18
19     void add(int pos, int val) {
20         for (; pos <= n; pos += pos & -pos)
21             bit[pos] += val;
22     }
23
24     int sum(int pos) {
25         int ans = 0;
26
27         for (; pos > 0; pos -= pos & -pos)
28             ans += bit[pos];
29
30         return ans;
31     }
32 };
33
34 // -----
35 // Primer indice cuyo valor es > x
36 // -----
37 int upperBound(const vector<ld>& a, ld x) {
38     int l = 0;
39     int r = a.size();
40
41     while (l < r) {
42         int m = (l + r) / 2;
43
44         if (a[m] <= x + EPS)
45             l = m + 1;
46         else

```

4.4 Contar Cruces de Cuerdas con Fenwick - Lonely Creatures (cont.)

```

47     r = m;
48 }
49
50 return l;
51 }
52
53 // -----
54 // Primer índice cuyo valor es >= x
55 // -----
56 int lowerBound(const vector<ld>& a, ld x) {
57     int l = 0;
58     int r = a.size();
59
60     while (l < r) {
61         int m = (l + r) / 2;
62
63         if (a[m] < x - EPS)
64             l = m + 1;
65         else
66             r = m;
67     }
68
69     return l;
70 }
71
72 int main() {
73     ios::sync_with_stdio(false);
74     cin.tie(nullptr);
75
76     int N;
77     ll A, B;
78
79     cin >> N >> A >> B;
80
81     vector<Chord> chords;
82
83     for (int i = 0; i < N; i++) {
84
85         ll M, C;
86         cin >> M >> C;
87
88         /*
89           $Mx + C = A \cdot x^2 + B$ 
90           $A \cdot x^2 - M \cdot x + (B - C) = 0$ 
91           $D = M^2 - 4A(B - C)$ 
92          */
93         ll D = M * M - 4LL * A * (B - C);
94
95         // No entra al interior.
96         if (D <= 0)
97             continue;
98
99         ld sq = sqrtl((ld)D);
100
101         ld L = ((ld)M - sq) / (2.0L * A);
102         ld R = ((ld)M + sq) / (2.0L * A);
103
104         chords.push_back({L, R});
105     }
106
107     if (chords.size() < 2) {
108         cout << 0 << '\n';
109         return 0;
110     }
111
112 // -----
113 // Ordenar por L.
114 //
115 // Si L es igual, NO pueden cruzarse dentro:

```

4.4 Contar Cruces de Cuerdas con Fenwick - Lonely Creatures (cont.)

```

116 // se encuentran sobre la frontera.
117 // -----
118 sort(chords.begin(), chords.end(), [](const Chord& a,
119                                       const Chord& b) {
120
121     if (fabsl(a.L - b.L) > EPS)
122         return a.L < b.L;
123
124     return a.R < b.R;
125 });
126
127 // -----
128 // Compresion de R
129 // -----
130
131 vector<ld> vals;
132
133 for (auto &c : chords)
134     vals.push_back(c.R);
135
136 sort(vals.begin(), vals.end());
137
138 vals.erase(
139     unique(vals.begin(), vals.end(),
140           [](ld a, ld b) {
141               return fabsl(a - b) <= EPS;
142           }),
143     vals.end()
144 );
145
146 Fenwick fw(vals.size());
147
148 ll ans = 0;
149
150 int i = 0;
151 int processed = 0;
152
153 while (i < (int)chords.size()) {
154
155     int j = i + 1;
156
157     while (j < (int)chords.size() &&
158           fabsl(chords[j].L - chords[i].L) <= EPS) {
159         j++;
160     }
161
162     /*
163      Todas las cuerdas anteriores tienen:
164
165      L_anterior < L_actual
166
167      porque estamos procesando un grupo completo
168      de L iguales.
169
170      Queremos:
171
172      L_actual < R_anterior < R_actual
173
174      Entonces:
175
176      count(R <= L_actual)
177      -
178      count(R >= R_actual)
179
180      equivalente a:
181
182      # R en (L_actual, R_actual)
183      */
184
185     for (int k = i; k < j; k++) {

```

4.4 Contar Cruces de Cuerdas con Fenwick - Lonely Creatures (cont.)

```

81         ld L = chords[k].L;
82         ld R = chords[k].R;
83
84         // cantidad de R <= L
85         int p1 = upperBound(vals, L);
86         int left = fw.sum(p1);
87
88         // cantidad de R < R
89         int p2 = lowerBound(vals, R);
90         int beforeR = fw.sum(p2);
91
92         /*
93          Queremos estrictamente:
94
95          L < R_previous < R
96
97          */
98         ans += beforeR - left;
99     }
100
101     // Ahora agregamos las cuerdas de este grupo.
102     for (int k = i; k < j; k++) {
103
104         int pos = lowerBound(vals, chords[k].R) + 1;
105
106         fw.add(pos, 1);
107         processed++;
108     }
109
110     i = j;
111 }
112
113 cout << ans << '\n';
114
115 return 0;
116 }

```

4.5 Convex Hull y Barrido Angular - Guard Deployment

Resuelve "Guard Deployment" (problema G, ICPC 2025): hay edificios rectangulares con altura, y hay que dar el perimetro de la cerca convexa mas corta que los encierra y la altura minima comun de las torres para que todas se vean entre si. Son dos problemas sueltos pegados: 1) EL PERIMETRO es el del convex hull de las cuatro esquinas de todos los edificios. La cerca mas corta que encierra un conjunto de puntos es siempre su envolvente convexa. 2) LA ALTURA depende de que edificios se atraviesan. Una linea de vista entre dos torres pasa por encima de un edificio si el segmento que las une (visto desde arriba) toca ese rectangulo; y si pasa, hay que ir al menos un metro mas alto que el. La respuesta es el maximo de $h+1$ sobre los edificios atravesados, y nunca menos de 3. LO INTERESANTE ES COMO DECIDE SI UN EDIFICIO SE ATRAVIESA, sin probar los $O(H^2)$ pares de torres: desde un vertice del hull, todas las direcciones que dan al rectangulo forman un INTERVALO ANGULAR, delimitado por sus cuatro esquinas. Si otro vertice del hull cae dentro de ese intervalo, el segmento entre ambos pasa por el edificio. Teniendo los angulos hacia los demas vertices ordenados, cada consulta es una busqueda binaria. El intervalo se obtiene al reves de lo que uno esperaria: se ordenan los angulos a las cuatro esquinas, se busca el HUECO mas grande entre angulos consecutivos (en circulo), y el intervalo bueno es el complemento de ese hueco. Es la forma de manejar el caso en que el intervalo cruza el 0 sin escribir casos especiales. Verificado contra una referencia que prueba cada par de torres contra cada edificio con aritmetica entera exacta: 1600 casos con rectangulos alineados y rotados 45 grados, perimetro y altura correctos en todos, ademas del sample. Medido: 10 casos de $N = 1000$ edificios en 200 ms.

C++\ICPC\2025\G - Guard Deployment.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ld = long double;
5
6 const ld PI = acos(-1.0L);
7 const ld EPS = 1e-12L;
8
9 struct Point {
10     ld x, y;
11

```

4.5 Convex Hull y Barrido Angular - Guard Deployment (cont.)

```

12     bool operator<(const Point& p) const {
13         if (fabsl(x - p.x) > EPS)
14             return x < p.x;
15         return y < p.y;
16     }
17
18     bool operator==(const Point& p) const {
19         return fabsl(x - p.x) < EPS &&
20             fabsl(y - p.y) < EPS;
21     }
22 };
23
24 struct Building {
25     Point p[4];
26     int h;
27 };
28
29 ld cross(Point a, Point b, Point c) {
30     return (b.x - a.x) * (c.y - a.y)
31         - (b.y - a.y) * (c.x - a.x);
32 }
33
34 // -----
35 // CONVEX HULL
36 // -----
37
38 vector<Point> convexHull(vector<Point> p) {
39
40     sort(p.begin(), p.end());
41
42     p.erase(unique(p.begin(), p.end(), p.end()));
43
44     int n = p.size();
45
46     if (n <= 1)
47         return p;
48
49     vector<Point> h(2 * n);
50     int k = 0;
51
52     // Lower
53     for (int i = 0; i < n; i++) {
54
55         while (k >= 2 &&
56             cross(h[k - 2], h[k - 1], p[i]) <= EPS)
57             k--;
58
59         h[k++] = p[i];
60     }
61
62     // Upper
63     for (int i = n - 2, t = k + 1; i >= 0; i--) {
64
65         while (k >= t &&
66             cross(h[k - 2], h[k - 1], p[i]) <= EPS)
67             k--;
68
69         h[k++] = p[i];
70     }
71
72     h.resize(k - 1);
73
74     return h;
75 }
76
77 // -----
78 // DISTANCIA
79 // -----
80
81 ld dist(Point a, Point b) {

```

4.5 Convex Hull y Barrido Angular - Guard Deployment (cont.)

```

84     ld dx = a.x - b.x;
85     ld dy = a.y - b.y;
86
87     return sqrtl(dx * dx + dy * dy);
88 }
89
90
91 // -----
92 // PUNTO SOBRE RECTANGULO
93 // Como los edificios son convexos, basta revisar sus 4 lados.
94 // -----
95
96 bool pointOnSegment(Point a, Point b, Point p) {
97
98     if (fabsl(cross(a, b, p)) > EPS)
99         return false;
100
101     return p.x >= min(a.x, b.x) - EPS &&
102            p.x <= max(a.x, b.x) + EPS &&
103            p.y >= min(a.y, b.y) - EPS &&
104            p.y <= max(a.y, b.y) + EPS;
105 }
106
107 bool pointInBuilding(const Building& b, Point p) {
108     for (int i = 0; i < 4; i++) {
109
110         Point a = b.p[i];
111         Point c = b.p[(i + 1) % 4];
112
113         if (pointOnSegment(a, c, p))
114             return true;
115     }
116
117     bool pos = false;
118     bool neg = false;
119
120     for (int i = 0; i < 4; i++) {
121
122         ld cr = cross(
123             b.p[i],
124             b.p[(i + 1) % 4],
125             p
126         );
127
128         if (cr > EPS) pos = true;
129         if (cr < -EPS) neg = true;
130     }
131
132     return !(pos && neg);
133 }
134
135 // -----
136 // ANGULO
137 // -----
138
139 ld angle(Point a, Point b) {
140
141     ld x = b.x - a.x;
142     ld y = b.y - a.y;
143
144     ld ang = atan2l(y, x);
145
146     if (ang < 0)
147         ang += 2 * PI;
148
149     return ang;
150 }
151
152 // -----

```

4.5 Convex Hull y Barrido Angular - Guard Deployment (cont.)

```

153 // EL EDIFICIO ES ATRAVESADO POR ALGUN SEGMENTO
154 // ENTRE DOS VERTICES DEL HULL?
155 //
156 // Para un vertice v del hull:
157 //
158 // todas las direcciones que desde v alcanzan el rectangulo
159 // forman un intervalo angular.
160 //
161 // Los cuatro corners del rectangulo determinan ese intervalo.
162 // Si existe otro vertice del hull dentro de ese intervalo,
163 // el segmento entre ambos atraviesa/toca el rectangulo.
164 // -----
165
166 bool buildingBlocks(
167     const vector<Point>& hull,
168     const Building& b,
169     const vector<vector<ld>>& hullAngles
170 ) {
171
172     int H = hull.size();
173
174     for (int v = 0; v < H; v++) {
175
176         // Si el vertice del hull pertenece al edificio,
177         // claramente este edificio participa.
178         if (pointInBuilding(b, hull[v]))
179             return true;
180
181         // Angulos hacia las 4 esquinas del edificio
182         vector<ld> a(4);
183
184         for (int k = 0; k < 4; k++)
185             a[k] = angle(hull[v], b.p[k]);
186
187         sort(a.begin(), a.end());
188
189         // Buscamos el mayor hueco circular.
190         // El intervalo complementario es el menor intervalo
191         // que contiene las 4 esquinas.
192         int bestGap = 0;
193         ld maxGap = -1;
194
195         for (int k = 0; k < 4; k++) {
196
197             ld next = (k == 3 ? a[0] + 2 * PI : a[k + 1]);
198
199             ld gap = next - a[k];
200
201             if (gap > maxGap) {
202                 maxGap = gap;
203                 bestGap = k;
204             }
205         }
206
207         ld L = a[(bestGap + 1) % 4];
208         ld R = a[bestGap];
209
210         if (R < L)
211             R += 2 * PI;
212
213         const vector<ld>& angles = hullAngles[v];
214
215         // Tenemos los angulos de los demas vertices.
216         // Probamos tambien una copia desplazada 2PI para
217         // resolver el caso en que el intervalo cruza 0.
218         auto check = [&](ld x) {
219             return x + EPS >= L && x <= R + EPS;
220         };
221
222         auto it = lower_bound(
223             angles.begin(),
224             angles.end(),

```

4.5 Convex Hull y Barrido Angular - Guard Deployment (cont.)

```

288     L = EPS
289 );
290
291 if (it != angles.end() && check(*it))
292     return true;
293
294 // Si L > 2PI, usamos los angulos originales + 2PI.
295 if (L > 2 * PI - EPS) {
296
297     ld L2 = L - 2 * PI;
298     ld R2 = R - 2 * PI;
299
300     auto it2 = lower_bound(
301         angles.begin(),
302         angles.end(),
303         L2 - EPS
304     );
305
306     if (it2 != angles.end() &&
307         *it2 <= R2 + EPS)
308         return true;
309     }
310 }
311
312 return false;
313 }
314
315 // -----
316 // MAIN
317 // -----
318
319 int main() {
320
321     ios::sync_with_stdio(false);
322     cin.tie(nullptr);
323
324     int T;
325     cin >> T;
326
327     while (T--) {
328
329         int N;
330         cin >> N;
331
332         vector<Building> buildings(N);
333
334         vector<Point> allPoints;
335
336         for (int i = 0; i < N; i++) {
337             for (int j = 0; j < 4; j++) {
338                 cin >> buildings[i].p[j].x
339                     >> buildings[i].p[j].y;
340             }
341             allPoints.push_back(buildings[i].p[j]);
342         }
343
344         cin >> buildings[i].h;
345     }
346
347 // -----
348 // 1. Convex Hull
349 // -----
350
351 vector<Point> hull = convexHull(allPoints);
352
353 int H = hull.size();
354
355 // -----
356 // 2. Perimetro

```

4.5 Convex Hull y Barrido Angular - Guard Deployment (cont.)

```

357 // -----
358
359 ld perimeter = 0;
360
361 for (int i = 0; i < H; i++)
362     perimeter += dist(
363         hull[i],
364         hull[(i + 1) % H]
365     );
366
367 // -----
368 // 3. Angulos de los vertices del hull
369 //
370 // Para cada vertice guardamos los angulos hacia
371 // todos los demas vertices.
372 // -----
373
374 vector<vector<ld>> hullAngles(H);
375
376 for (int i = 0; i < H; i++) {
377     for (int j = 0; j < H; j++) {
378         if (i == j)
379             continue;
380
381         hullAngles[i].push_back(
382             angle(hull[i], hull[j])
383         );
384     }
385
386     sort(
387         hullAngles[i].begin(),
388         hullAngles[i].end()
389     );
390 }
391
392 // -----
393 // 4. Altura minima
394 // -----
395
396 int minHeight = 3;
397
398 for (int i = 0; i < N; i++) {
399     if (buildingBlocks(
400         hull,
401         buildings[i],
402         hullAngles
403     )) {
404         minHeight = max(
405             minHeight,
406             buildings[i].h + 1
407         );
408     }
409 }
410
411 cout << fixed << setprecision(6)
412     << (double)perimeter << ' '
413     << minHeight << '\n';
414 }
415
416 return 0;
417 }

```

4.6 Coordenadas Polares

O: (1) cada conversion, (n log n) ordenar por angulo

Uso: aPolar(x,y) -> {r,th}; normalizar(th) deja th en [0,2pi)

Pasar de (x, y) a (r, theta) y al contrario. Las formulas son de colegio, pero en competitiva lo que tumba la solucion son los detalles de implementacion, y por eso vale la ficha. 1) EL ANGULO SE SACA CON atan2(y, x), NUNCA CON atan(y/x). atan(y/x) pierde el cuadrante (manda (-1,-1) al mismo angulo que (1,1)) y explota cuando $x = 0$. atan2 mira los signos de los dos argumentos por separado y acierta en los cuatro cuadrantes. OJO CON EL ORDEN DE LOS ARGUMENTOS: es atan2(y, x), la y va PRIMERO. Invertirlos da el angulo respecto al eje y, y el error pasa desapercibido porque para 45 grados da lo mismo. 2) EL RADIO CON hypot(x, y), no con $\sqrt{x^2 + y^2}$. hypot escala internamente y no pierde el paso intermedio. Medido en esta maquina: con $x = y = 1e-200$, $\sqrt{x^2 + y^2}$ da 0 porque el cuadrado se va a cero por UNDERFLOW, mientras hypot da 1.41e-200. Por arriba, con 1e200, aqui las dos dieron 1.41e200 (el x87 calcula con 80 bits y salva el caso), asi que el problema real de sqrt es el underflow, no el overflow. Igual hypot sale gratis. 3) atan2 DEVUELVE EL RANGO (-pi, pi], no [0, 2pi). Los angulos de abajo del eje x salen NEGATIVOS. Si el problema necesita [0, 2pi) hay que sumar 2pi a los negativos, y si se olvida, el barrido angular arranca en el eje x NEGATIVO en vez del positivo. LO QUE DE VERDAD IMPORTA: PARA ORDENAR POR ANGULO, NO USES atan2. El uso numero uno de las polares en competitiva es el barrido angular. Con atan2 funciona, pero es la version fragil, y conviene tener claro en que falla exactamente: - El orden ANGULAR sale bien con los dos metodos. Medido con 200000 puntos enteros aleatorios: las dos ordenaciones son monotonas en angulo y NO hay ni un solo par cuyo angulo verdadero quede en el orden equivocado. - Donde atan2 se rompe es en los COLINEALES. De las 15156 posiciones en que las dos listas no coinciden, las 15156 son entre puntos del MISMO angulo exacto: atan2 les da doubles que difieren en el ultimo bit, el desempate por distancia nunca se ejecuta y quedan revueltos. Con 200 puntos colineales barajados que deben salir por distancia, el comparador exacto los ordena bien y el de atan2 los saca MAL, tanto con coordenadas chicas como de 2e8. - Y hay algo peor que el orden: comparar angulos double con epsilon rompe la transitividad, y un comparador no transitivo en std::sort es comportamiento indefinido. La version exacta usa solo ENTEROS: se parte el plano en dos mitades (arriba o abajo del eje x) y dentro de la misma mitad se compara con el producto cruz. Cero punto flotante. SU LIMITE: cross y la norma multiplican coordenadas, y 2^*c^2 tiene que caber en long long, o sea [coord] hasta $\sim 2.1e9$ (medido: con 2e9 cabe, con 3e9 la norma ya sale negativa). Para las coordenadas de 1e9 tipicas queda un factor 4 de margen. CUANDO SI CONVIENEN LAS POLARES: rotar (sumar al angulo), simetria circular, sectores y arcos, y cualquier enunciado que ya venga en (r, theta). Para SUMAR vectores hay que volver a cartesianas: en polares la suma no es componente a componente. Y para rotar un punto exacto conviene la matriz de rotacion sobre las cartesianas, que no pasa por atan2 ni acumula el error de ida y vuelta (medido: el viaje cartesiano -> polar -> cartesiano sobre 300000 puntos de hasta 1e6 pierde a lo sumo 4.7e-10, suficiente para un eps de 1e-9 pero no gratis).

C++\Notebook\Coordenadas Polares.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const double PI = acos(-1.0);
5
6 // ----- conversion -----
7 struct Polar { double r, th; };
8
9 Polar aPolar(double x, double y) {
10     return {hypot(x, y), atan2(y, x)}; // th queda en (-pi, pi]
11 }
12
13 pair<double, double> aCartesiano(double r, double th) {
14     return {r * cos(th), r * sin(th)};
15 }
16
17 // Lleva cualquier angulo al rango [0, 2pi).
18 double normalizar(double th) {
19     th = fmod(th, 2 * PI);
20     if (th < 0) th += 2 * PI;
21     return th;
22 }
23
24 double grados(double rad) { return rad * 180.0 / PI; }
25 double radianes(double deg) { return deg * PI / 180.0; }
26
27 // ----- barrido angular exacto, sin atan2 -----
28 struct P {
```

4.6 Coordenadas Polares (cont.)

```
29     long long x, y;
30 };
31
32 long long cross(const P &a, const P &b) { return a.x * b.y - a.y * b.x; }
33
34 // semiplano: 0 = angulo en [0, pi), 1 = en [pi, 2pi). El (0,0) cae en 0.
35 int mitad(const P &p) {
36     if (p.y > 0) return 0;
37     if (p.y < 0) return 1;
38     if (p.x >= 0) return 0; // sobre el eje x: el +x va en la 1a
39     return 1; // mitad y el -x en la 2a
40 }
41
42 // Ordena por angulo en [0, 2pi) arrancando en el +x. Los colineales quedan
43 // juntos y entre ellos ordenados por distancia al origen.
44 bool menorAngulo(const P &a, const P &b) {
45     int ha = mitad(a), hb = mitad(b);
46     if (ha != hb) return ha < hb;
47
48     long long c = cross(a, b);
49     if (c != 0) return c > 0; // a va antes si b queda a su izq
50
51     return a.x * a.x + a.y * a.y < b.x * b.x + b.y * b.y;
52 }
53
54 int main() {
55     ios::sync_with_stdio(false);
56     cin.tie(nullptr);
57     cout << fixed << setprecision(6);
58
59     double x, y;
60     while (cin >> x >> y) {
61         Polar p = aPolar(x, y);
62         pair<double, double> q = aCartesiano(p.r, p.th);
63
64         cout << "r = " << p.r
65              << " theta = " << p.th
66              << " rad = " << grados(p.th) << " grados"
67              << " [0,2pi) = " << normalizar(p.th)
68              << " vuelta = (" << q.first << ", " << q.second << ")\n";
69     }
70     return 0;
71 }
```

4.7 Geometria 1D - Kilometric Intersection

Resuelve "Kilometric Intersection" (Day 9, problema J - Contest 08 [Inicial & Avanzado]): dados dos intervalos cerrados [a,b] y [c,d] sobre una recta (tramos de carretera en kilometros), se pide la longitud de su interseccion, siendo 0 si no se superponen o solo se tocan en un punto. Calcula el intervalo de interseccion tomando $l = \max(a,c)$ y $r = \min(b,d)$, e imprime $\max(0, r - l)$, que es exactamente la longitud del solapamiento (o 0 si $l > r$).

C++\Training_Camp_2026\Day 9J - Kilometric Intersection.py

```
1 import sys
2
3 n = int(input())
4
5 for _ in range(n):
6     a, b, c, d = map(int, input().split())
7
8     l = max(a, c)
9     r = min(b, d)
10
11     print(max(0, r - l))
```

4.8 K Vecinos Mutuos y Clasificación de Componentes - Celestial Veins

Resuelve "Celestial Veins" (problema C, ICPC 2025): cada estrella mira a sus K vecinas mas cercanas, dos estrellas quedan unidas solo si CADA UNA esta entre las K de la otra (vinculo mutuo), y hay que clasificar cada componente conexa en una de cinco formas y contarlas. TRES COSAS QUE HAY QUE HACER BIEN Y SON FACILES DE ARRUIINAR: 1) EL DESEMPATE. Si dos candidatas quedan a la misma distancia, gana la que aparece antes en la entrada. Ordenando pares (distancia al cuadrado, indice) eso sale solo, porque el indice desempata. Con coordenadas hasta 10^9 , $dx * dx + dy * dy$ llega a $8 * 10^{18}$ y cabe JUSTO en long long: en int o en double se rompe. 2) EL VINCULO ES MUTUO. Que A tenga a B entre sus K no basta; hay que comprobar tambien al reves. Como cada estrella tiene a lo sumo 5 vecinas, revisarlo es $O(K)$. 3) EL ORDEN DE LA CLASIFICACION, que el enunciado fija: Cluster, Ouroboros, Claw, Path, Nebula. Importa porque las formas se solapan. Un triangulo es a la vez completo y un ciclo, y se cuenta como Cluster; un camino de 3 estrellas es a la vez garra y camino, y se cuenta como Claw. Evaluarlas en otro orden cambia las respuestas. Cada forma se reconoce contando aristas y grados, sin recorrer nada mas: completo: $E = V * (V - 1) / 2$ ciclo: $E = V$ y todos los grados valen 2 garra: $E = V - 1$, un grado $V - 1$ y los demas 1 camino: $E = V - 1$ y ningun grado pasa de 2 Una estrella sola ($V = 1$) es Nebula, y cualquier otra cosa tambien. El if de $V == 1$ va primero porque con $V = 1$ la formula del completo tambien se cumple ($E = 0$) y la clasificaria mal como Cluster. Costo $O(n^2)$ por mapa para hallar las vecinas, que con $n = 1500$ son unos 2 millones de pares. Medido: 5 mapas de $n = 1500$ con $K = 5$ en 93 ms, tanto con puntos al azar como en rejilla (que es el caso con mas empates de distancia). Verificado contra una implementacion independiente del enunciado en 1800 mapas, incluidos puntos alineados y en rejilla para forzar empates, sin una sola diferencia, ademas de los dos casos del sample.

C++\ICPC\2025\C - Celestial Veins.cpp

```
1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <algorithm>
5
6 using namespace std;
7
8 int main()
9 {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     long long n, k;
14
15     while(cin >> n >> k && !(n == 0 && k == 0))
16     {
17         vector<long long> coordX(n), coordY(n);
18
19         for(long long i = 0; i < n; i++)
20         {
21             cin >> coordX[i] >> coordY[i];
22         }
23
24         vector<vector<long long>> cercanas(n);
25
26         long long cuantasVecinas = min(k, n - 1);
27
28         for(long long estrella = 0; estrella < n; estrella++)
29         {
30             vector<pair<long long, long long>> candidatas;
31
32             for(long long otra = 0; otra < n; otra++)
33             {
34                 if(otra == estrella)
35                 {
36                     continue;
37                 }
38
39                 long long dx = coordX[estrella] - coordX[otra];
40                 long long dy = coordY[estrella] - coordY[otra];
41
42                 long long distancia2 = dx * dx + dy * dy;
43
44                 candidatas.push_back({distancia2, otra});
45             }
46         }
47     }
```

4.8 K Vecinos Mutuos y Clasificación de Componentes - Celestial Veins (cont.)

```
47     partial_sort(candidatas.begin(),
48                 candidatas.begin() + cuantasVecinas,
49                 candidatas.end());
50
51     for(long long p = 0; p < cuantasVecinas; p++)
52     {
53         cercanas[estrella].push_back(candidatas[p].second);
54     }
55 }
56
57 vector<vector<long long>> vinculos(n);
58
59 for(long long a = 0; a < n; a++)
60 {
61     for(long long b : cercanas[a])
62     {
63         if(b < a)
64         {
65             continue;
66         }
67
68         bool bTieneA = false;
69
70         for(long long c : cercanas[b])
71         {
72             if(c == a)
73             {
74                 bTieneA = true;
75             }
76         }
77
78         if(bTieneA)
79         {
80             vinculos[a].push_back(b);
81             vinculos[b].push_back(a);
82         }
83     }
84 }
85
86 long long travelersPath = 0;
87 long long cosmicOuroboros = 0;
88 long long stellarCluster = 0;
89 long long dragonsClaw = 0;
90 long long amorphousNebula = 0;
91
92 vector<bool> visitada(n, false);
93
94 for(long long inicio = 0; inicio < n; inicio++)
95 {
96     if(visitada[inicio])
97     {
98         continue;
99     }
100
101     vector<long long> grupo;
102     queue<long long> cola;
103
104     cola.push(inicio);
105     visitada[inicio] = true;
106
107     while(!cola.empty())
108     {
109         long long actual = cola.front();
110         cola.pop();
111
112         grupo.push_back(actual);
113
114         for(long long vecina : vinculos[actual])
115         {
116             if(!visitada[vecina])
117             {
118                 visitada[vecina] = true;
```

4.8 K Vecinos Mutuos y Clasificación de Componentes - Celestial Veins (cont.)

```

18         cola.push(vecina);
19     }
20 }
21 }
22 }
23
24 long long estrellas = grupo.size();
25 long long sumaGrados = 0;
26 long long conGrado1 = 0;
27 long long conGrado2 = 0;
28 long long gradoMaximo = 0;
29
30 for(long long estrella : grupo)
31 {
32     long long grado = vinculos[estrella].size();
33
34     sumaGrados += grado;
35     gradoMaximo = max(gradoMaximo, grado);
36
37     if(grado == 1) conGrado1++;
38     if(grado == 2) conGrado2++;
39 }
40
41 long long enlaces = sumaGrados / 2;
42
43 if(estrellas == 1)
44 {
45     amorphousNebula++;
46 }
47 else if(enlaces == estrellas * (estrellas - 1) / 2)
48 {
49     stellarCluster++;
50 }
51 else if(enlaces == estrellas && conGrado2 == estrellas)
52 {
53     cosmicOuroboros++;
54 }
55 else if(enlaces == estrellas - 1 &&
56         gradoMaximo == estrellas - 1 &&
57         conGrado1 == estrellas - 1)
58 {
59     dragonsClaw++;
60 }
61 else if(enlaces == estrellas - 1 && gradoMaximo <= 2)
62 {
63     travelersPath++;
64 }
65 else
66 {
67     amorphousNebula++;
68 }
69 }
70
71 cout << travelersPath << ' '
72 << cosmicOuroboros << ' '
73 << stellarCluster << ' '
74 << dragonsClaw << ' '
75 << amorphousNebula << '\n';
76 }
77
78 return 0;
79 }

```

4.9 K-Means (Clustering)

O: (iteraciones * n * k * d)

Uso: kmeansOnce(p,k,rng,maxIter) -> centroides y etiquetas; correr varias veces

Agrupar n puntos (de cualquier dimension d) en k clusters minimizando el SSE: la suma de las distancias al cuadrado de cada punto al centroide de su cluster. Algoritmo de Lloyd: (1) asignar cada punto al centroide mas cercano, (2) mover cada centroide al promedio de sus puntos, y repetir hasta que ninguna asignacion cambie. Cada paso solo puede bajar el SSE, así que siempre converge, pero a un optimo LOCAL que depende de los centroides iniciales. Por eso se siembra con k-means++ (cada centroide nuevo sale con probabilidad proporcional a su distancia² al centroide mas cercano, lo que los reparte) y se corre con varias semillas quedandose con el menor SSE. Lo que rompe la version ingenua: (a) un cluster puede quedar vacio y su promedio no existe; aqui se re-siembrar en el punto que peor encaja; (b) sin tope de iteraciones no hay cota practica de tiempo. Costo: O(reinicios * iteraciones * n * k * d). OJO: el optimo exacto es NP-hard. Si el problema es 1D y pide el optimo, ordenar y hacer DP de particion en segmentos contiguos (O(k n²), o menos con Divide and Conquer Optimization).

C++\Notebook\K-Means.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 typedef vector<double> Vec;
6
7 double dist2(const Vec &a, const Vec &b)
8 {
9     double s = 0;
10    for (size_t t = 0; t < a.size(); t++) s += (a[t] - b[t]) * (a[t] - b[t]);
11    return s;
12 }
13
14 struct KMeans
15 {
16     vector<Vec> c; // centroides
17     vector<int> label; // cluster de cada punto
18     double sse; // suma de distancias^2 de cada punto a su centroide
19 };
20
21 // Centroide mas cercano a x (en empate, el de menor indice)
22 int nearest(const Vec &x, const vector<Vec> &c)
23 {
24     int best = 0;
25     double bd = dist2(x, c[0]);
26     for (int j = 1; j < (int)c.size(); j++)
27     {
28         double d = dist2(x, c[j]);
29         if (d < bd)
30         {
31             bd = d;
32             best = j;
33         }
34     }
35     return best;
36 }
37
38 KMeans kmeansOnce(const vector<Vec> &p, int k, mt19937 &rng, int maxIter)
39 {
40     int n = p.size(), d = p[0].size();
41
42     // Semilla k-means++
43     vector<Vec> c(1, p[uniform_int_distribution<int>(0, n - 1)(rng)]);
44     vector<double> md(n, 1e300); // distancia^2 al centroide elegido mas cercano
45     while ((int)c.size() < k)
46     {
47         double total = 0;
48         for (int i = 0; i < n; i++)
49         {
50             md[i] = min(md[i], dist2(p[i], c.back()));
51             total += md[i];
52         }
53         int pick = n - 1;

```

4.9 K-Means (Clustering) (cont.)

```

54     if (total > 0)
55     {
56         double r = uniform_real_distribution<double>(0, total)(rng);
57         for (int i = 0; i < n; i++)
58         {
59             r -= md[i];
60             if (r <= 0)
61             {
62                 pick = i;
63                 break;
64             }
65         }
66     }
67     else pick = uniform_int_distribution<int>(0, n - 1)(rng); // todos repetidos
68     c.push_back(p[pick]);
69 }
70
71 // Lloyd
72 vector<int> label(n, -1);
73 for (int it = 0; it < maxIter; it++)
74 {
75     bool changed = false;
76     for (int i = 0; i < n; i++)
77     {
78         int j = nearest(p[i], c);
79         if (j != label[i])
80         {
81             label[i] = j;
82             changed = true;
83         }
84     }
85     if (!changed) break;
86
87     vector<Vec> sum(k, Vec(d, 0.0));
88     vector<int> cnt(k, 0);
89     for (int i = 0; i < n; i++)
90     {
91         cnt[label[i]]++;
92         for (int t = 0; t < d; t++) sum[label[i]][t] += p[i][t];
93     }
94
95     // Para re-sembrar un cluster vacio: el punto mas lejano a su centroide actual
96     vector<double> err(n);
97     for (int i = 0; i < n; i++) err[i] = dist2(p[i], c[label[i]]);
98
99     for (int j = 0; j < k; j++)
100     {
101         if (cnt[j] > 0)
102         {
103             for (int t = 0; t < d; t++) c[j][t] = sum[j][t] / cnt[j];
104         }
105         else
106         {
107             int far = max_element(err.begin(), err.end()) - err.begin();
108             c[j] = p[far];
109             err[far] = -1; // que otro cluster vacio no tome el mismo punto
110         }
111     }
112 }
113
114 KMeans res;
115 res.c = c;
116 res.label.assign(n, 0);
117 res.sse = 0;
118 for (int i = 0; i < n; i++)
119 {
120     res.label[i] = nearest(p[i], c);
121     res.sse += dist2(p[i], c[res.label[i]]);
122 }
123 return res;
124 }
125

```

4.9 K-Means (Clustering) (cont.)

```

86 // Mejor de `restarts` corridas. Semilla fija: resultado reproducible
87 KMeans kmeans(const vector<Vec> &p, int k, int restarts = 10, int maxIter = 300,
88               unsigned seed = 7)
89 {
90     k = max(1, min(k, (int)p.size()));
91     mt19937 rng(seed);
92     KMeans best;
93     best.sse = numeric_limits<double>::infinity();
94     for (int r = 0; r < restarts; r++)
95     {
96         KMeans cur = kmeansOnce(p, k, rng, maxIter);
97         if (cur.sse < best.sse) best = cur;
98     }
99     return best;
100 }
101
102 int main()
103 {
104     ios_base::sync_with_stdio(false);
105     cin.tie(nullptr);
106
107     int n, k, d;
108     cin >> n >> k >> d;
109     vector<Vec> p(n, Vec(d));
110     for (auto &x : p)
111         for (auto &v : x) cin >> v;
112
113     KMeans r = kmeans(p, k);
114
115     cout << fixed << setprecision(6) << r.sse << "\n";
116     for (auto &cc : r.c)
117     {
118         for (int t = 0; t < d; t++) cout << cc[t] << (t + 1 < d ? ' ' : '\n');
119     }
120     for (int i = 0; i < n; i++) cout << r.label[i] << (i + 1 < n ? ' ' : '\n');
121     return 0;
122 }

```

4.10 Primitivas y Convex Hull

O: (n log n) el hull, (1) las primitivas

Uso: cross(a,b,c)>0 izquierda; convexHull(p) antihorario; polygonArea(P)

Base de todo problema geometrico: struct Pt con operadores, producto punto y producto cruz. El cruz es la herramienta central: su signo dice si tres puntos giran a la izquierda, a la derecha o son colineales, y con eso se resuelve casi todo. Nunca comparar doubles con == : usar sgn() con EPS. Incluye interseccion de segmentos (dos pruebas de orientacion opuesta, mas los casos degenerados colineales via onSeg), convex hull por monotone chain en O(n log n) (ordena por coordenada y arma la cadena inferior y la superior descartando giros no convexos), y area de poligono simple por la formula del cordón (shoelace), que vale para cualquier poligono cerrado sin auto-intersecciones y devuelve el doble del area en valor absoluto entre dos.

C++\Notebook\Primitivas y Convex Hull.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 const double EPS = 1e-9;
6
7 int sgn(double x)
8 {
9     return (x > EPS) - (x < -EPS);
10 }
11
12 struct Pt
13 {
14     double x, y;
15     Pt() {}
16     Pt(double x, double y) : x(x), y(y) {}
17     Pt operator+(const Pt &o) const { return Pt(x + o.x, y + o.y); }
18     Pt operator-(const Pt &o) const { return Pt(x - o.x, y - o.y); }
19     Pt operator*(double k) const { return Pt(x * k, y * k); }
20 };

```

4.10 Primitivas y Convex Hull (cont.)

```

21
22 double dot(Pt a, Pt b) { return a.x * b.x + a.y * b.y; }
23 double cross(Pt a, Pt b) { return a.x * b.y - a.y * b.x; }
24 double cross(Pt a, Pt b, Pt c) { return cross(b - a, c - a); }
25
26 // p esta sobre el segmento a-b (asumiendo colinealidad)
27 bool onSeg(Pt a, Pt b, Pt p)
28 {
29     return sgn(cross(a, b, p)) == 0 && sgn(dot(p - a, p - b)) <= 0;
30 }
31
32 bool segIntersect(Pt a, Pt b, Pt c, Pt d)
33 {
34     int o1 = sgn(cross(a, b, c)), o2 = sgn(cross(a, b, d));
35     int o3 = sgn(cross(c, d, a)), o4 = sgn(cross(c, d, b));
36     if (o1 * o2 < 0 && o3 * o4 < 0) return true;
37     return onSeg(a, b, c) || onSeg(a, b, d) || onSeg(c, d, a) || onSeg(c, d, b);
38 }
39
40 // Monotone chain: devuelve el casco convexo en sentido antihorario
41 vector<Pt> convexHull(vector<Pt> p)
42 {
43     sort(p.begin(), p.end(), [](const Pt &a, const Pt &b){
44         if (a.x == b.x) return a.y < b.y;
45         return a.x < b.x;
46     });
47
48     vector<Pt> lo, up;
49     for (auto &pt : p)
50     {
51         while (lo.size() >= 2 && sgn(cross(lo[lo.size() - 2], lo.back(), pt)) <= 0)
52             lo.pop_back();
53         lo.push_back(pt);
54     }
55
56     for (int i = (int)p.size() - 1; i >= 0; --i)
57     {
58         Pt pt = p[i];
59         while (up.size() >= 2 && sgn(cross(up[up.size() - 2], up.back(), pt)) <= 0)
60             up.pop_back();
61         up.push_back(pt);
62     }
63
64     lo.pop_back();
65     up.pop_back();
66     lo.insert(lo.end(), up.begin(), up.end());
67     return lo;
68 }
69
70 // Formula del cordón (shoelace) para polígono simple
71 double polygonArea(const vector<Pt> &P)
72 {
73     long double s = 0;
74     int n = P.size();
75     for (int i = 0; i < n; i++)
76     {
77         s += (long double)P[i].x * P[(i + 1) % n].y - (long double)P[i].y * P[(i + 1) % n].x;
78     }
79     return fabs((double)s) * 0.5;
80 }
81
82 int main()
83 {
84     ios_base::sync_with_stdio(false);
85     cin.tie(nullptr);
86     return 0;
87 }

```

4.11 Probabilidad de Circulo Contenido - Omens

Resuelve "Omens" (problema I, ICPC 2024): caen dos piedras uniformemente al azar dentro de un rectángulo $L \times W$ y se dibuja el círculo que las tiene como DIAMETRO (centro en el punto medio, radio la mitad de la distancia). Se pide la probabilidad de que ese círculo quede completamente dentro del rectángulo, a 4 decimales. De entrada parece una integral en CUATRO dimensiones y da miedo. Lo que la desarma es una caracterización de geometría, no de programación: el círculo de diámetro AB cabe dentro de un convexo K si y solo si TODO punto de la frontera de K ve al segmento AB con ángulo de a lo sumo 90 grados porque el círculo de diámetro AB es justo el lugar donde ese ángulo vale exactamente 90 (arco capaz), y por fuera del círculo el ángulo es menor. Aplicándolo al lado $x = 0$ con un punto $Q = (0, q)$, "ángulo $\leq 90^\circ$ " es $(A-Q) \cdot (B-Q) \geq 0$, o sea $x_1^2 x_2 + (y_1 - q)(y_2 - q) \geq 0$ y como hay que pedirlo para TODO q , se minimiza: el mínimo cae en $q = (y_1 + y_2)/2$ y vale $x_1^2 x_2 - (y_1 - y_2)^2/4$. Repitiendo en los cuatro lados quedan cuatro desigualdades limpias, sin una sola raíz cuadrada: $4^4 x_1^2 x_2 \geq (y_1 - y_2)^2 4^4 (W - x_1)(W - x_2) \geq (y_1 - y_2)^2 4^4 y_1 y_2 \geq (x_1 - x_2)^2 4^4 (L - y_1)(L - y_2) \geq (x_1 - x_2)^2 4^4$. Verificado: esas cuatro equivalen a la condición directa (centro y radio) en 3000000 de pruebas aleatorias, sin una sola diferencia. DE AHI SALE UNA FORMULA CERRADA. Las desigualdades son homogéneas de grado 2, así que escalar el rectángulo no cambia la probabilidad: solo depende de la PROPORCIÓN entre los lados. Con $t = \min(L, W) / \max(L, W)$, la respuesta es $P = (\pi/6) * t * (2 - t)$. Con el cuadrado ($t = 1$) queda $\pi/6 = 0.5236$, y con proporción 1:2 queda $\pi/8 = 0.3927$, que son justo los dos primeros casos del enunciado. HONESTIDAD SOBRE ESTA FORMULA: salio de conjeturar desde los ejemplos y esta VERIFICADA NUMERICAMENTE, no demostrada. Se contrasto en 8 proporciones distintas (de 0.001 hasta 1) integrando exacto la última variable para bajar la varianza, y todas las diferencias quedaron en $2 \cdot 10^{-5}$, dentro del error y sin sesgo. Si en un problema parecido la fórmula no cuadra, lo que si esta demostrado y se puede usar siempre son las cuatro desigualdades de arriba: con ellas se integra numericamente y se sale igual. El lado útil para llevarse: cuando un enunciado pida "probabilidad de que una figura quepa", conviene traducir el "cabe" a desigualdades sobre las coordenadas antes de pensar en integrar. Casi siempre se simplifican, y a veces colapsan a una fórmula de una línea como aquí.

C++\ICPC\2024\I - Omens.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7     cout << fixed << setprecision(4);
8
9     double L, W;
10
11     // La entrada termina con la línea "0 0", que no se procesa.
12     while (cin >> L >> W && (L != 0 || W != 0)) {
13         double t = min(L, W) / max(L, W);
14
15         cout << acos(-1.0) / 6.0 * t * (2 - t) << '\n';
16     }
17
18     return 0;
19 }

```

4.12 Punto en Polígono Convexo ($O(\log n)$)

O: $(n \log n)$ el hull, $(\log n)$ cada consulta

Uso: $h = \text{convexHull}(p)$; $\text{inside}(h, q)$ con h antihorario y sin colineales

Arma el casco convexo de un conjunto de puntos y luego responde muchas consultas del tipo "este punto esta dentro?" en $O(\log n)$ cada una, en vez de $O(n)$ revisando arista por arista. Todo se hace con enteros (el cross devuelve long long), así que no hay error de precisión. El casco usa monotone chain de Andrew; descartar con $\text{cross} \leq 0$ elimina los puntos colineales y deja solo los vértices reales, devolviendo el hull en sentido antihorario. La consulta triangula mentalmente el polígono en abanico desde $h[0]$: primero descarta el punto si cae fuera de las dos aristas extremas ($h[0]-h[1]$ y $h[0]-h[n-1]$), y si no, busca binariamente el sector ($h[0]$, $h[lo]$, $h[lo+1]$) donde cae y comprueba de que lado esta de esa arista. Los cross con ≥ 0 hacen que el borde cuente como dentro. Contempla los casos degenerados en que el casco queda reducido a un punto o a un segmento.

C++\Notebook\Punto en Polígono Convexo.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5

```

4.12 Punto en Poligono Convexo ($O(\log n)$) (cont.)

```

5 struct P { ll x, y; };
6
7 ll cross(const P&O, const P&A, const P&B){
8     return (A.x-O.x)*(B.y-O.y) - (A.y-O.y)*(B.x-O.x);
9 }
10
11 vector<P> convexHull(vector<P> p){
12     int n = p.size(), k = 0;
13     sort(p.begin(), p.end(), [](const P&a, const P&b){
14         return a.x != b.x ? a.x < b.x : a.y < b.y;
15     });
16     p.erase(unique(p.begin(), p.end(), [](const P&a, const P&b){
17         return a.x==b.x && a.y==b.y;
18     }), p.end());
19     n = p.size();
20     if (n < 3) return p;
21
22     vector<P> h(2*n);
23     for (int i = 0; i < n; i++){ // cadena inferior
24         while (k >= 2 && cross(h[k-2], h[k-1], p[i]) <= 0) k--;
25         h[k++] = p[i];
26     }
27     for (int i = n-2, t = k+1; i >= 0; i--){ // cadena superior
28         while (k >= t && cross(h[k-2], h[k-1], p[i]) <= 0) k--;
29         h[k++] = p[i];
30     }
31     h.resize(k-1); // quita el repetido final
32     return h; // CCW, sin colineales
33 }
34
35 bool inside(const vector<P>& h, const P& q){
36     int n = h.size();
37     if (n == 1) return h[0].x==q.x && h[0].y==q.y;
38     if (n == 2){ // segmento degenerado
39         if (cross(h[0], h[1], q) != 0) return false;
40         return min(h[0].x, h[1].x) <= q.x && q.x <= max(h[0].x, h[1].x)
41             && min(h[0].y, h[1].y) <= q.y && q.y <= max(h[0].y, h[1].y);
42     }
43     if (cross(h[0], h[1], q) < 0) return false;
44     if (cross(h[0], h[n-1], q) > 0) return false;
45
46     int lo = 1, hi = n-1; // busca el sector
47     while (hi - lo > 1){
48         int mid = (lo + hi) / 2;
49         if (cross(h[0], h[mid], q) >= 0) lo = mid;
50         else hi = mid;
51     }
52     return cross(h[lo], h[lo+1], q) >= 0;
53 }
54
55 int main(){
56     ios::sync_with_stdio(false);
57     cin.tie(nullptr);
58     int L;
59     while (cin >> L){
60         vector<P> big(L);
61         for (auto &b : big) cin >> b.x >> b.y;
62         vector<P> h = convexHull(big);
63
64         int S; cin >> S;
65         int cnt = 0;
66         for (int i = 0; i < S; i++){
67             P q; cin >> q.x >> q.y;
68             if (inside(h, q)) cnt++;
69         }
70         cout << cnt << "\n";
71     }
72     return 0;
73 }

```

4.13 Rotating Calipers (Par Mas Lejano)

O: $(n \log n)$ el hull, (n) el diametro

Uso: $h = \text{convexHull}(p)$; $\text{diameter}(h) \rightarrow \{\text{dist}^2, \{P, Q\}\}$ // dist^2 , no dist

Par de puntos mas lejano (diametro del conjunto) en $O(n \log n)$. El par optimo siempre son dos vertices del casco convexo, asi que se arma el casco y se recorre con dos punteros: para cada arista $(i, i+1)$ se avanza j mientras el triangulo $(i, i+1, j)$ crezca en area. Donde deja de crecer, j es el vertice mas lejano a la recta de esa arista (su antipodal). Como j solo avanza y da a lo sumo una vuelta, el recorrido cuesta $O(n)$. Detalle que la version ingenua olvida: el antipodal se mide contra LOS DOS extremos de la arista $(i, i+1)$; medir solo contra i puede perder el par optimo. Todo en enteros con distancia AL CUADRADO: exacto, sin sqrt ni EPS (coordenadas hasta $\sim 1e9$ para que no desborde long long). El casco sale sin repetidos ni colineales, que es lo que necesita el while (con colineales el area deja de ser estrictamente unimodal). Casos borde: un solo punto distinto $\rightarrow 0$; todos colineales $\rightarrow$ el casco tiene 2 puntos. El mismo recorrido de antipodales resuelve: ancho minimo del conjunto (minimo, sobre las aristas, de la distancia a su antipodal), rectangulo de area minima que lo cubre y distancia entre dos poligonos convexos.

C++\Notebook\Rotating Calipers.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 typedef long long ll;
6
7 struct P
8 {
9     ll x, y;
10     bool operator<(const P &o) const { return x != o.x ? x < o.x : y < o.y; }
11     bool operator==(const P &o) const { return x == o.x && y == o.y; }
12 };
13
14 P operator-(P a, P b) { return {a.x - b.x, a.y - b.y}; }
15 ll cross(P a, P b) { return a.x * b.y - a.y * b.x; }
16 ll cross(P o, P a, P b) { return cross(a - o, b - o); }
17 ll dist2(P a, P b)
18 {
19     P d = a - b;
20     return d.x * d.x + d.y * d.y;
21 }
22
23 // Monotone chain: casco en sentido antihorario, sin repetidos ni colineales
24 vector<P> convexHull(vector<P> p)
25 {
26     sort(p.begin(), p.end());
27     p.erase(unique(p.begin(), p.end(), p.end()));
28     int n = p.size();
29     if (n <= 2) return p;
30
31     vector<P> h(2 * n);
32     int k = 0;
33     for (int i = 0; i < n; i++)
34     {
35         while (k >= 2 && cross(h[k-2], h[k-1], p[i]) <= 0) k--;
36         h[k++] = p[i];
37     }
38     for (int i = n-2, lo = k+1; i >= 0; i--)
39     {
40         while (k >= lo && cross(h[k-2], h[k-1], p[i]) <= 0) k--;
41         h[k++] = p[i];
42     }
43     h.resize(k-1);
44     return h;
45 }
46
47 // {distancia^2 maxima, par de puntos}. h = casco antihorario (el de convexHull)
48 pair<ll, pair<P, P>> diameter(const vector<P> &h)
49 {
50     int n = h.size();
51     if (n == 1) return {0, {h[0], h[0]}};
52     if (n == 2) return {dist2(h[0], h[1]), {h[0], h[1]}};
53 }

```

4.13 Rotating Calipers (Par Mas Lejano) (cont.)

```

54 ll best = -1;
55 P a = h[0], b = h[0];
56 for (int i = 0, j = 1; i < n; i++)
57 {
58     int ni = (i + 1) % n;
59     // avanzar j mientras el area del triangulo (i, ni, j) siga creciendo
60     while (cross(h[i], h[ni], h[(j + 1) % n]) > cross(h[i], h[ni], h[j])) j = (j + 1) % n;
61
62     if (dist2(h[i], h[j]) > best)
63     {
64         best = dist2(h[i], h[j]);
65         a = h[i];
66         b = h[j];
67     }
68     if (dist2(h[ni], h[j]) > best)
69     {
70         best = dist2(h[ni], h[j]);
71         a = h[ni];
72         b = h[j];
73     }
74 }
75 return {best, {a, b}};
76 }
77
78 int main()
79 {
80     ios_base::sync_with_stdio(false);
81     cin.tie(nullptr);
82
83     int n;
84     cin >> n;
85     vector<P> p(n);
86     for (auto &q : p) cin >> q.x >> q.y;
87
88     auto r = diameter(convexHull(p));
89
90     cout << r.first << "\n"; // distancia al cuadrado (exacta)
91     cout << fixed << setprecision(6) << sqrt((double)r.first) << "\n";
92     cout << r.second.first.x << " " << r.second.first.y << " "
93     << r.second.second.x << " " << r.second.second.y << "\n";
94     return 0;
95 }

```

4.14 Shoelace (Area de Poligono)

O: (n)

Uso: area2(p) = 2*area con signo; puntosInteriores(p) por Pick

Area de CUALQUIER poligono simple en $O(n)$, convexo o concavo, dando solo sus vertices en orden. Suma cruzada de cada arista con la siguiente: $\sum(x_i \cdot y_{i+1} - x_{i+1} \cdot y_i)$, y el area es la mitad del valor absoluto. El truco que importa en competencia es no dividir entre 2: se devuelve el DOBLE del area como long long, que con coordenadas enteras es exacto y evita por completo el punto flotante. Si el problema pide el area y puede ser .5, imprime a2/2 y a2%2 por separado, o multiplica todo por 2 desde el principio. El signo tambien sirve: positivo significa que los vertices vienen en sentido antihorario, negativo en sentido horario, asi que de paso detecta la orientacion gratis. Requisitos: los vertices deben ir en orden alrededor del poligono y el borde no puede cruzarse consigo mismo. No hace falta repetir el primer vertice al final.

C++\Notebook\Shoelace.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 struct P { ll x, y; };
6
7 // Doble del area CON SIGNO. Exacto con coordenadas enteras.
8 // > 0 antihorario, < 0 horario, = 0 degenerado (todos colineales).
9 ll area2(const vector<P>& p) {
10     ll s = 0;
11     int n = p.size();
12     for (int i = 0; i < n; i++) {
13         int j = (i + 1) % n;

```

4.14 Shoelace (Area de Poligono) (cont.)

```

14     s += p[i].x * p[j].y - p[j].x * p[i].y;
15 }
16 return s;
17 }
18
19 // Area de verdad. Solo aqui aparece el punto flotante.
20 double area(const vector<P>& p) {
21     return llabs(area2(p)) / 2.0;
22 }
23
24 // Puntos de coordenadas enteras SOBRE el borde.
25 // Cada arista aporta gcd(|dx|, |dy|) puntos sin contar dos veces los vertices.
26 ll puntosBorde(const vector<P>& p) {
27     ll b = 0;
28     int n = p.size();
29     for (int i = 0; i < n; i++) {
30         int j = (i + 1) % n;
31         b += __gcd(llabs(p[i].x - p[j].x), llabs(p[i].y - p[j].y));
32     }
33     return b;
34 }
35
36 // Puntos enteros Estrictamente interiores, despejando Pick: A = I + B/2 - 1.
37 // Con 2A entero la cuenta sale exacta y sin dividir.
38 ll puntosInteriores(const vector<P>& p) {
39     return (llabs(area2(p)) - puntosBorde(p) + 2) / 2;
40 }
41
42 // Centroide (centro de masa) del poligono. No es el promedio de los vertices.
43 pair<double, double> centroide(const vector<P>& p) {
44     int n = p.size();
45     double a = 0, cx = 0, cy = 0;
46     for (int i = 0; i < n; i++) {
47         int j = (i + 1) % n;
48         double cr = (double)p[i].x * p[j].y - (double)p[j].x * p[i].y;
49         a += cr;
50         cx += (p[i].x + p[j].x) * cr;
51         cy += (p[i].y + p[j].y) * cr;
52     }
53     a /= 2;
54     return {cx / (6 * a), cy / (6 * a)};
55 }
56
57 int main() {
58     ios::sync_with_stdio(false);
59     cin.tie(nullptr);
60
61     int n;
62     if (!(cin >> n)) return 0;
63
64     vector<P> p(n);
65     for (auto &v : p) cin >> v.x >> v.y;
66
67     ll a2 = area2(p);
68
69     // Imprimir sin perder el medio punto: parte entera y el .5 aparte.
70     cout << llabs(a2) / 2;
71     if (llabs(a2) % 2) cout << ".5";
72     cout << "\n";
73
74     return 0;
75 }
76
77 // OJO con el overflow: si las coordenadas llegan a 1e9, cada producto
78 // llega a 1e18 y la suma de varios se pasa de long long. En ese caso
79 // acumula en __int128, o traslada todos los puntos restando el primer
80 // vertice antes de sumar (las coordenadas se vuelven chicas y el area
81 // no cambia).

```

4.15 Triangle Classification - AROD

Clasifica triángulos en una grilla de $p \times q$ puntos como agudos, rectos, obtusos o degenerados. Utiliza análisis de productos para determinar el tipo de ángulo en cada triángulo.

C++\RPC\AROD.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using int64 = long long;
5
6 static inline int64 comb3(int64 n) {
7     if (n < 3) return 0;
8     return n * (n - 1) * (n - 2) / 6;
9 }
10
11 int main() {
12     ios::sync_with_stdio(false);
13     cin.tie(nullptr);
14
15     int p, q;
16     cin >> p >> q;
17
18     if (p > q) swap(p, q); // p = lado pequeño, q = lado grande
19
20     const int W = p + 1; // cantidad de columnas de puntos
21     const int H = q + 1; // cantidad de filas de puntos
22
23     // gtProd[s][k] = cantidad de alpha en [0..s] con alpha*(s-alpha) > k
24     // eqProd[s][k] = cantidad de alpha en [0..s] con alpha*(s-alpha) = k
25     int maxS = p;
26     int maxK = maxS * maxS / 4;
27
28     vector<vector<int>> gtProd(maxS + 1, vector<int>(maxK + 1, 0));
29     vector<vector<int>> eqProd(maxS + 1, vector<int>(maxK + 1, 0));
30
31     for (int s = 0; s <= maxS; s++) {
32         int localMax = s * s / 4;
33         vector<int> freq(localMax + 1, 0);
34
35         for (int alpha = 0; alpha <= s; alpha++) {
36             int val = alpha * (s - alpha);
37             freq[val]++;
38         }
39
40         int suf = 0;
41         for (int k = localMax; k >= 0; k--) {
42             gtProd[s][k] = suf;
43             suf += freq[k];
44             eqProd[s][k] = freq[k];
45         }
46     }
47
48     auto getGt = [&](int s, int64 k) -> int64 {
49         int64 localMax = 1LL * s * s / 4;
50         if (k > localMax) return 0;
51         return gtProd[s][(int)k];
52     };
53
54     auto getEq = [&](int s, int64 k) -> int64 {
55         int64 localMax = 1LL * s * s / 4;
56         if (k > localMax) return 0;
57         return eqProd[s][(int)k];
58     };
59
60     int64 acute = 0, right = 0, obtuse = 0, degenerate = 0;
61
62     // -----
63     // 1) Todos en la misma fila -> degenerados
64     // -----
65     degenerate += 1LL * H * comb3(W);
66
67     // -----
```

4.15 Triangle Classification - AROD (cont.)

```
68 // 2) Triángulos usando exactamente 2 filas
69 //
70 // Si la distancia vertical es h:
71 // - caso "punto solo entre la base"
72 // - caso "punto solo afuera de la base"
73 // -----
74 for (int h = 1; h <= q; h++) {
75     int64 verticalWays = 2LL * (H - h); // escoger las dos filas y cuál tiene 2 puntos
76
77     for (int s = 1; s <= p; s++) {
78         int64 waysPlaceX = W - s;
79         int64 weight = verticalWays * waysPlaceX;
80
81         // Punto solo entre los dos de la base
82         int64 rightBetween = 2 + getEq(s, 1LL * h * h); // extremos + posibles internos
83         int64 obtuseBetween = getGt(s, 1LL * h * h);
84         int64 acuteBetween = (s + 1) - rightBetween - obtuseBetween;
85
86         // Punto solo fuera del intervalo horizontal de la base
87         // todos esos son obtusos, salvo el caso alineado con un extremo,
88         // que ya quedó contado en "between" con alpha=0 o beta=0
89         int64 obtuseOutside = 2LL * (s - 1);
90
91         right += weight * rightBetween;
92         obtuse += weight * (obtuseBetween + obtuseOutside);
93         acute += weight * acuteBetween;
94     }
95 }
96
97 // -----
98 // 3) Triángulos usando exactamente 3 filas distintas
99 //
100 // Distancias verticales:
101 // fila1 -> fila2 = a
102 // fila2 -> fila3 = b
103 //
104 // Se cuenta por span horizontal total s.
105 // -----
106 for (int a = 1; a < q; a++) {
107     for (int b = 1; a + b <= q; b++) {
108         int64 verticalWays = H - (a + b);
109         int64 ab = 1LL * a * b;
110         int64 A = 1LL * a * (a + b);
111         int64 B = 1LL * b * (a + b);
112
113         for (int s = 0; s <= p; s++) {
114             int64 waysPlaceX = W - s;
115             int64 weight = verticalWays * waysPlaceX;
116
117             // Caso monótono:
118             // x1<=x2<=x3 o x1>=x2>=x3
119             // Todo no degenerado aquí es obtuso.
120             int64 orient = (s == 0 ? 1 : 2);
121
122             int64 degHit = 0;
123             if ((1LL * a * s) % (a + b) == 0) {
124                 degHit = 1;
125             }
126
127             degenerate += weight * orient * degHit;
128             obtuse += weight * orient * ((s + 1) - degHit);
129
130             if (s == 0) continue;
131
132             // -----
133             // Caso donde x1 queda estrictamente "en medio"
134             // u < 0 y u+v > 0
135             // alpha en [1 .. s-1]
136             // -----
137             {
138                 int64 total = s - 1;
139                 if (total > 0) {
```

4.15 Triangle Classification - AROD (cont.)

```

10:         int64 low = min<int64>(total, (ab - 1) / s);
11:         int64 rightLin = ((ab % s == 0) && (1 <= ab / s) && (ab / s <= s - 1)) ? 1
12:         : 0;
13:
14:         int64 obt1 = low + getGt(s, A);
15:         int64 rig1 = rightLin + getEq(s, A);
16:         int64 acu1 = total - obt1 - rig1;
17:
18:         obtuse += weight * 2 * obt1;
19:         right += weight * 2 * rig1;
20:         acute += weight * 2 * acu1;
21:     }
22:
23:     // -----
24:     // Caso donde x3 queda en medio o empata con x1
25:     // v < 0 y u+v >= 0
26:     // alpha en [0 .. s-1]
27:     // -----
28:     {
29:         int64 total = s;
30:         int64 low = min<int64>(total, (ab - 1) / s);
31:         int64 rightLin = ((ab % s == 0) && (1 <= ab / s) && (ab / s <= s)) ? 1 : 0;
32:
33:         int64 obt3 = low + getGt(s, B);
34:         int64 rig3 = rightLin + getEq(s, B);
35:         int64 acu3 = total - obt3 - rig3;
36:
37:         obtuse += weight * 2 * obt3;
38:         right += weight * 2 * rig3;
39:         acute += weight * 2 * acu3;
40:     }
41: }
42:
43: }
44:
45: cout << acute << '\n';
46: cout << right << '\n';
47: cout << obtuse << '\n';
48: cout << degenerate << '\n';
49:
50: return 0;
51: }

```

5 - Math

5.1 2-Coloreo por Suma Potencia de 2 - Drug Test

Resuelve "Drug Test" (problema D, ICPC 2024). El código le asigna a cada x de 1 a N un tipo, A o P, con el 0 fijo en P, de forma que x y $p - x$ queden con tipos OPUESTOS, siendo p la menor potencia de 2 que es mayor o igual a x (o sea, x y su complemento suman potencia de 2). LA OBSERVACION QUE LO VUELVE UN FOR: el complemento $p - x$ siempre es MENOR que x . Como p es la menor potencia de 2 que alcanza a x , $p/2 < x$, o sea $p < 2x$, y entonces $p - x < x$ (verificado para x hasta $2 \cdot 10^5$). Así que cada x depende de un número más chico, y recorriendo en orden creciente su complemento ya tiene tipo cuando se le necesita. Visto como grafo: cada x tiene exactamente un "padre" menor que él, así que las restricciones forman un ARBOL con raíz en 0. Un árbol siempre se puede 2-colorear y la coloración queda determinada por la raíz, por eso no hay que probar nada ni hacer backtracking. El caso " x es potencia de 2" es redundante: ahí el complemento es 0, que es P, y la regla general ya le da A. COMO LEER BIEN EL ENUNCIADO: al pie de la letra solo pide que cada tamaño tenga AL MENOS UN compañero de tipo opuesto, y así hay muchas asignaciones válidas (con $N = 12$ salen 66), lo que choca con que el enunciado diga que el diseño es único. La lectura que lo hace único es la del ejemplo, que lista TODOS los pares de tamaños distintos que suman potencia de 2: ninguno puede quedar con el mismo tipo. El código solo mira un compañero por número (el del árbol), pero esa coloración resulta respetar también todos los demás pares: comprobado para N hasta 9999, sin un conflicto, y enumerando todas las asignaciones para N hasta 15 es la única.

C++\ICPC\2024\D - Drug Test.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;

```

5.1 2-Coloreo por Suma Potencia de 2 - Drug Test (cont.)

```

3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int N, Q;
9
10    while (cin >> N >> Q && (N || Q)) {
11
12        vector<char> tipo(N + 1);
13
14        // 0 siempre es placebo
15        tipo[0] = 'P';
16
17        for (int x = 1; x <= N; x++) {
18
19            int p = 1;
20
21            while (p < x)
22                p *= 2;
23
24            if (p == x) {
25                // x + 0 = potencia de 2
26                tipo[x] = 'A';
27            }
28            else {
29                int complement = p - x;
30
31                // x y complement tienen tipos opuestos
32                tipo[x] = (tipo[complement] == 'A' ? 'P' : 'A');
33            }
34        }
35
36        while (Q--) {
37            int x;
38            cin >> x;
39            cout << tipo[x];
40        }
41
42        cout << '\n';
43    }
44
45    return 0;
46 }

```

5.2 Ad-hoc - Attention To Meeting

Resuelve "Attention to the Meeting" (Day 3, problema A - Maratona SBC de Programacao 2024): Vinicius quiere saber la duración máxima que puede tener cada uno de los N discursos de una reunión, sabiendo que todos duran lo mismo, que hay una pausa de 1 minuto entre discursos consecutivos, y que la reunión completa no debe superar K minutos. El problema se reduce a una simple desigualdad lineal: $N \cdot T + (N-1) \leq K$, así que el código despeja T directamente con división entera, $T = (K - (N-1)) // N$, sin búsqueda ni iteración.

C++\Training_Camp_2026\Day 3\A - Attention to Meeting.py

```

1 n = int(input())
2 t = int(input())
3
4 print((t - (n-1))//n)

```

5.3 Algebra - The Wisdom Of Master Wei

Resuelve "The Wisdom of Master Wei" (Day 10, problema E - Contest 09 [Avanzados]): el maestro Wei empezó a programar en el año W y el discípulo Kai en el año K ($W < K$); hay que hallar el año exacto Y en el que la experiencia del maestro ($Y - W$ años) sea el doble de la de Kai ($Y - K$ años). Despeja la ecuación $Y - W = 2 \cdot (Y - K)$, que da $Y = 2K - W$, y la imprime directamente como $m + (m - n)$ usando $n=W$ y $m=K$, sin necesidad de búsqueda ni iteración.

C++\Training_Camp_2026\Day 10\E - The Wisdom of Master Wei.py

```

1 n,m = list(map(int,input().split()))
2 print(m+(m-n))

```

5.4 Bit Manipulation - Meme Wars

Resuelve "Meme Wars" (Day 7, problema G - Contest 07 [Avanzados]): la palabra $S(x)$ se define recursivamente como $S(x) = S(p(x)) + x + S(p(x))$ (con $S('a')='a'$, y $p(x)$ la letra anterior), y hay que hallar la letra en la posición n de la enorme palabra $S('z')$. Esta es la clásica "ruler sequence": la letra en la posición n depende únicamente de la mayor potencia de 2 que divide exactamente a n (su valoración 2-adica). El diccionario `dic` mapea cada potencia de 2 a su letra correspondiente, y el bucle recorre las potencias en orden creciente guardando en 'cual' la última potencia que divide a n , hasta encontrar la primera que no lo divide; en ese momento imprime la letra asociada a esa última potencia (equivalente a contar los ceros finales de n en binario).

C++\Training_Camp_2026\Day 7\G - Meme Wars.py

```
1  dic = {
2      1: "a",
3      2: "b",
4      4: "c",
5      8: "d",
6      16: "e",
7      32: "f",
8      64: "g",
9      128: "h",
10     256: "i",
11     512: "j",
12     1024: "k",
13     2048: "l",
14     4096: "m",
15     8192: "n",
16     16384: "o",
17     32768: "p",
18     65536: "q",
19     131072: "r",
20     262144: "s",
21     524288: "t",
22     1048576: "u",
23     2097152: "v",
24     4194304: "w",
25     8388608: "x",
26     16777216: "y",
27     33554432: "z"
28 }
29 n = int(input())
30 cual = 1
31 for clave, valor in dic.items():
32     if (n % clave == 0):
33         cual = clave
34         continue
35     else:
36         print(dic[cual])
37         break
```

5.5 Brute Force - Antiarithmetic

Resuelve "Antiarithmetic?" (Day 11, problema H - CCPL (Kattis)): dada una permutación de $0..n-1$, determina si es antiaritmética, es decir si no existen tres índices $i < j < k$ tales que los valores p_i, p_j, p_k formen una progresión aritmética (ninguna subsecuencia de longitud mayor a 2 puede ser una progresión aritmética). Para cada par de índices (i, k) calcula el valor promedio $v = (p_i + p_k) / 2$ (cuando la suma es par) y usa el arreglo inverso 'pos' (posición de cada valor dentro de la permutación) para ubicar en $O(1)$ donde está v ; si esa posición j queda entre i y k , existe una progresión aritmética y la permutación no es antiaritmética.

C++\Training_Camp_2026\Day 11 - CCPL\H - Antiarithmetic.cpp

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  static char buf_[1 << 22];
5  static int bufLen_, bufPos_;
6  inline int gc() {
7      if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!bufLen_) return -1; }
8      return buf_[bufPos_++];
9  }
10 inline bool readInt(int &out) {
11     int c = gc();
```

5.5 Brute Force - Antiarithmetic (cont.)

```
12 while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
13 if (c == -1) return false;
14 bool neg = false; if (c == '-') { neg = true; c = gc(); }
15 int x = 0;
16 while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
17 out = neg ? -x : x;
18 return true;
19 }
20 // Reads up to (and past) the colon before the first number is read normally, since ':' is
21 // simply skipped as a non-digit character by readInt's own skip loop.
22
23 bool isAntiarithmetic(int n, const vector<int> &p) {
24     vector<int> pos(n);
25     for (int idx = 0; idx < n; idx++) pos[p[idx]] = idx;
26     for (int i = 0; i < n; i++) {
27         for (int k = i + 1; k < n; k++) {
28             int s = p[i] + p[k];
29             if (s % 2 == 0) {
30                 int v = s / 2;
31                 int j = pos[v];
32                 if (i < j && j < k) return false;
33             }
34         }
35     }
36     return true;
37 }
38
39 int main() {
40     int n;
41     string out;
42     while (readInt(n)) {
43         if (n == 0) break;
44         vector<int> p(n);
45         for (int i = 0; i < n; i++) readInt(p[i]);
46         out += isAntiarithmetic(n, p) ? "yes\n" : "no\n";
47     }
48     fwrite(out.data(), 1, out.size(), stdout);
49     return 0;
50 }
```

5.6 Construction - Ezraft And Array

Problema de construcción estilo Codeforces: para cada n , imprime un arreglo de n enteros positivos (o -1 si $n=2$, caso imposible). Para $n=1$ usa $[1]$; para $n \geq 3$ arranca con $[1, 2, 3]$ y cada elemento siguiente se hace igual a la suma acumulada de todos los anteriores (duplicándola en cada paso), construyendo una secuencia donde cada nuevo término "rompe" el balance de las sumas de prefijos previas.

C++\Codeforces\ezraft and Array.cpp

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios_base::sync_with_stdio(false);
6      cin.tie(NULL);
7
8      int t;
9      cin >> t;
10     while (t--) {
11         int n;
12         cin >> n;
13
14         if (n == 2) {
15             cout << -1 << "\n";
16             continue;
17         }
18
19         vector<long long> a;
20         if (n == 1) {
21             a = {1};
22         } else {
23             a = {1, 2, 3};
24             long long S = 6;
```

5.6 Construction - Ezraft And Array (cont.)

```

25     for (int i = 4; i <= n; i++) {
26         a.push_back(S);
27         S *= 2;
28     }
29 }
30
31     for (size_t i = 0; i < a.size(); i++) {
32         cout << a[i];
33         if (i + 1 != a.size()) cout << ' ';
34     }
35     cout << "\n";
36 }
37 return 0;
38 }

```

5.7 Divisores de la Suma Total - Puntos Equiespaciados En Un Circulo

O: $(\sqrt{S}) + d(S) \cdot n^2$ en el peor caso, pero los dos filtros lo dejan casi lineal

Uso: lee n y los n huecos del circulo; imprime cuantos puntos quitar, o -1

Hay n puntos en un circulo y se dan los n HUECOS entre puntos consecutivos (dist[i] va del punto i al i+1, y el ultimo cierra el circulo). Hay que quitar la menor cantidad de puntos para que los que queden esten igualmente espaciados, dejando al menos 3. -1 si no se puede. LA IDEA QUE RESUELVE TODO: si al final quedan k puntos equiespaciados, cada arco mide S/k con S la suma de todos los huecos. O sea que el arco TIENE que ser un DIVISOR de S. Entonces no hay que buscar entre 2^n subconjuntos: basta probar los divisores de S, que son $O(\sqrt{S})$ de encontrar y pocos de recorrer. Los dos filtros son los que hacen que esto vuele, y cada uno sale de una observacion distinta: $i \geq \text{maxi}$ cada arco es una suma de huecos CONSECUTIVOS, asi que no puede medir menos que el hueco mas grande, porque ese hueco vive adentro de algun arco. $i \leq S/3$ quedan $k = S/i$ puntos y el enunciado pide al menos 3. Y como los divisores se recorren de MENOR A MAYOR, el primero que funciona es el que deja mas puntos ($k = S/i$) y por lo tanto el que quita menos. Por eso se corta con el primer exito y no hace falta comparar candidatos. Dentro de un divisor se prueban los n puntos de arranque porque el recorrido es un greedy: va acumulando y cada vez que llega EXACTO al arco corta. Si arranca en un punto que no es un corte real, se pasa ($\text{anti} > i$) y se descarta ese arranque. Como cualquier particion valida tiene al menos 3 cortes, alguno de los n arranques cae en uno. quitados sale solo: cada arco consume cont huecos y conserva 1 punto, asi que suma cont - 1. Al final quitados = n - k. RAMA MUERTA: el "else cout << -1" de adentro del exito nunca se ejecuta, porque n - quitados es exactamente $k = S/i$ y el filtro $i \leq S/3$ ya garantiza $k \geq 3$. Verificado instrumentando la rama sobre las 2.451 pruebas: 0 ejecuciones. Se deja como estaba para no tocar codigo que sirve. OJO: sum es int. Con n y huecos grandes se desborda; si los limites suben, a long long. OJO: el atajo de "todos iguales -> 0" responde antes de mirar si hay al menos 3 puntos, asi que con $n = 1$ o $n = 2$ contesta 0. Solo importa si el problema permite $n < 3$. VERIFICADO contra fuerza bruta sobre subconjuntos: exhaustivo para $n = 3, 4$ y 5 con huecos de 1 a 3 (todas las combinaciones) mas 2.100 casos aleatorios, 2.451 en total, 0 diferencias. Tiempo: 0.01 s hasta $n = 8000$ en los peores casos que arme (suma muy divisible, huecos casi iguales con uno roto, todos unos menos el ultimo).

C++\Notebook\Puntos Equiespaciados en un Circulo.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      int n;
7      while(cin >> n)
8      {
9          if(n == 0) return 0;
10
11          vector<int> dist(n);
12          int maxi = -1;
13          bool iguales = true;
14          int anti = 0;
15          int sum = 0;
16
17          for(int i = 0; i < n; i++)
18          {
19              cin >> dist[i];
20
21              if(i == 0)
22                  anti = dist[i];
23

```

5.7 Divisores de la Suma Total - Puntos Equiespaciados En Un Circulo (cont.)

```

24         if(dist[i] > maxi)
25         {
26             maxi = dist[i];
27         }
28
29         if(anti != dist[i])
30         {
31             iguales = false;
32         }
33
34         sum += dist[i];
35     }
36
37     if(iguales)
38     {
39         cout << 0 << endl;
40     }
41     else
42     {
43         bool entro = false;
44
45         vector<int> divisores;
46
47         for(int i = 1; i * i <= sum; i++)
48         {
49             if(sum % i == 0)
50             {
51                 divisores.push_back(i);
52
53                 if(i != sum / i)
54                 {
55                     divisores.push_back(sum / i);
56                 }
57             }
58         }
59
60         sort(divisores.begin(), divisores.end());
61
62         for(int i : divisores)
63         {
64             // i es el largo del arco: no puede ser menor que el hueco mas
65             // grande, ni tan grande que queden menos de 3 puntos.
66             if(i < maxi || i > sum / 3)
67                 continue;
68
69             for(int inicio = 0; inicio < n; inicio++)
70             {
71                 int anti = 0;
72                 int quitados = 0;
73                 int cont = 0;
74                 bool ok = true;
75
76                 for(int j = 0; j < n; j++)
77                 {
78                     int pos = (inicio + j) % n;
79
80                     anti += dist[pos];
81                     cont++;
82
83                     if(anti == i)
84                     {
85                         anti = 0;
86                         quitados += cont - 1; // el arco conserva 1 punto
87                         cont = 0;
88                     }
89                     else if(anti > i)
90                     {
91                         ok = false; // este arranque no es corte
92                         break;
93                     }
94                 }
95

```

5.7 Divisores de la Suma Total - Puntos Equiespaciados En Un Circulo (cont.)

```

96         if(ok && anti == 0)
97         {
98             entro = true;
99
100             if(n - quitados >= 3)
101             {
102                 cout << quitados << endl;
103             }
104             else
105             {
106                 cout << -1 << endl;
107             }
108             break;
109         }
110     }
111 }
112
113 if(entro)
114 {
115     break;
116 }
117 }
118
119 if(!entro)
120 {
121     cout << -1 << endl;
122 }
123 }
124 }
125
126 return 0;
127 }

```

5.8 Eliminacion Gaussiana (mod p)

O: $(n^2 * m)$ en modulo p primo

Uso: gauss(a,m,sol,incompatible,p) -> grados de libertad; determinante(a,p)

Resuelve sistemas lineales $Ax = b$ sobre $\mathbb{Z}_p$ con p PRIMO, que es lo que aparece cuando el problema pide contar soluciones de un sistema, resolver ecuaciones modulares, o sacar el determinante y el rango de una matriz modulo un primo. La unica diferencia con el Gauss de toda la vida es que no se puede dividir: para normalizar el pivote se multiplica por su inverso modular (Fermat, $a^{(p-2)} \bmod p$). Como p es primo, TODO elemento distinto de cero tiene inverso, así que no hace falta pivoteo parcial por estabilidad numerica como en double; basta tomar el primer pivote no nulo. Devuelve el rango. Con m incognitas, si el sistema es compatible hay exactamente $p^{(m - \text{rango})}$ soluciones: rango == m significa solucion unica, y rango < m deja variables libres (aqui se fijan en 0 para entregar una solucion cualquiera). Costo $O(n * m * \min(n,m))$. Si p no es primo la normalizacion falla; y si el sistema es sobre GF(2), conviene la version con bitset, que va 64 veces mas rapido.

C++\Notebook\Eliminacion Gaussiana.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 ll mpow(ll a, ll e, ll p) {
6     ll r = 1; a %= p;
7     while (e) { if (e & 1) r = r * a % p; a = a * a % p; e >>= 1; }
8     return r;
9 }
10
11 ll inverso(ll a, ll p) { return mpow((a % p + p) % p, p - 2, p); }
12
13 // Matriz AUMENTADA de n filas por (m+1) columnas: las m incognitas y el termino
14 // independiente al final. Devuelve el rango; deja una solucion en sol.
15 int gauss(vector<vector<ll>> a, int m, vector<ll>& sol, bool& incompatible, ll p) {
16     int n = a.size(), fila = 0;
17     vector<int> donde(m, -1); // donde[col] = fila que tiene el pivote de esa columna
18
19     for (int col = 0; col < m && fila < n; col++) {
20         int piv = -1;
21         for (int i = fila; i < n; i++)
22             if (a[i][col] % p) { piv = i; break; }

```

5.8 Eliminacion Gaussiana (mod p) (cont.)

```

23         if (piv < 0) continue; // columna sin pivote: variable libre
24
25         swap(a[piv], a[fila]);
26
27         ll iv = inverso(a[fila][col], p);
28         for (int j = col; j <= m; j++)
29             a[fila][j] = a[fila][j] * iv % p;
30
31         for (int i = 0; i < n; i++)
32             if (i != fila && a[i][col] % p) {
33                 ll f = a[i][col];
34                 for (int j = col; j <= m; j++)
35                     a[i][j] = ((a[i][j] - f * a[fila][j]) % p + p) % p;
36             }
37
38         donde[col] = fila++;
39     }
40
41     sol.assign(m, 0); // las variables libres quedan en 0
42     for (int j = 0; j < m; j++)
43         if (donde[j] >= 0) sol[j] = a[donde[j]][m] % p;
44
45     // Una fila 0 = c con c != 0 significa que no hay solucion.
46     incompatible = false;
47     for (int i = fila; i < n; i++) {
48         bool cero = true;
49         for (int j = 0; j < m; j++)
50             if (a[i][j] % p) { cero = false; break; }
51         if (cero && a[i][m] % p) { incompatible = true; break; }
52     }
53     return fila;
54 }
55
56 // Determinante mod p de una matriz cuadrada. 0 significa que es singular.
57 ll determinante(vector<vector<ll>> a, ll p) {
58     int n = a.size();
59     ll det = 1;
60     for (int c = 0; c < n; c++) {
61         int piv = -1;
62         for (int i = c; i < n; i++)
63             if (a[i][c] % p) { piv = i; break; }
64         if (piv < 0) return 0;
65         if (piv != c) { swap(a[piv], a[c]); det = (p - det) % p; } // cada swap cambia el signo
66
67         det = det * a[c][c] % p;
68         ll iv = inverso(a[c][c], p);
69         for (int i = c + 1; i < n; i++) {
70             ll f = a[i][c] * iv % p;
71             for (int j = c; j < n; j++)
72                 a[i][j] = ((a[i][j] - f * a[c][j]) % p + p) % p;
73         }
74     }
75     return det;
76 }
77
78 int main() {
79     ios::sync_with_stdio(false);
80     cin.tie(nullptr);
81
82     const ll P = 1000000007;
83     int n, m;
84     if (!(cin >> n >> m)) return 0;
85
86     vector<vector<ll>> a(n, vector<ll>(m + 1));
87     for (int i = 0; i < n; i++)
88         for (int j = 0; j <= m; j++) { cin >> a[i][j]; a[i][j] = ((a[i][j] % P) + P) % P; }
89
90     vector<ll> sol;
91     bool incompatible;
92     int rango = gauss(a, m, sol, incompatible, P);
93
94     if (incompatible) {

```

5.8 Eliminación Gaussiana (mod p) (cont.)

```

95     cout << "sin solucion\n";
96 } else {
97     cout << "rango " << rango << ", soluciones = P^n" << (m - rango) << "\n";
98     for (int j = 0; j < m; j++) cout << sol[j] << " \n"[j == m - 1];
99 }
100 return 0;
101 }

```

5.9 Fibonacci (Fast Doubling) - Fibonacci Sum

Resuelve "Fibonacci Sum" (Day 10, problema M - Contest 09 [Avanzados]): para T casos con N y M (hasta $1e9$), calcula la suma $F(N)+F(N+1)+\dots+F(M)$ modulo $1e9+7$. Usa la identidad telescópica de que la suma de Fibonacci $\sum_{i=0}^k F(i) = F(k+2) - 1$, por lo que la suma pedida es $F(M+2) - F(N+1)$ mod MOD (visible en la línea $\text{fibmod}(b+2) - \text{fibmod}(a+1)$). Cada $F(n)$ se calcula en $O(\log n)$ con el algoritmo de duplicación rápida (fast doubling), que recorre los bits de n de mas a menos significativo aplicando las formulas $F(2k)=F(k)*(2*F(k+1)-F(k))$ y $F(2k+1)=F(k)^2+F(k+1)^2$ en aritmética modular.

C++\Training_Camp_2026\Day 10\M - Fibonacci Sum.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long long ll;
5 const ll MOD = 1000000007;
6
7 ll fibmod(ll n, ll mod) {
8     ll a = 0, b = 1, c;
9     for (int i = 63 - __builtin_clzll(n); i >= 0; i--) {
10         c = a;
11         a = (c * (2 * b - c + mod)) % mod;
12         b = (c * c + b * b) % mod;
13
14         if ((n >> i) & 1) {
15             c = (a + b) % mod;
16             a = b;
17             b = c;
18         }
19     }
20     return a;
21 }
22
23 int main() {
24     ios::sync_with_stdio(false);
25     cin.tie(nullptr);
26
27     int q;
28     cin >> q;
29
30     while (q--) {
31         ll a, b;
32         cin >> a >> b;
33
34         ll ans = (fibmod(b + 2, MOD) - fibmod(a + 1, MOD) + MOD) % MOD;
35         cout << ans << '\n';
36     }
37 }

```

5.10 Fibonacci (Zeckendorf-like Greedy) - Getting In Shape

Resuelve "Getting in Shape" (Day 4, problema D - Contest 04 [Avanzados]): dado N, hay que construir una secuencia de ejercicios tipo A/B (terminada en B) tal que la cantidad de formas de completarla saltándose opcionalmente el ejercicio siguiente a cada A sea exactamente N, y devolver la lexicográficamente menor, o "IMPOSSIBLE" si no existe. El número de formas de un bloque $A^k(k-1)B$ resulta ser $Fib(k+1)$, y las formas de sufijos se suman; solveSum busca en forma greedy el mayor $Fib(k+2)$ que divida el resto S para minimizar el largo del bloque de A's (dando el prefijo lexicográfico mas chico), y solve arma la respuesta de forma recursiva y memoizada (memo) para evitar recomputo exponencial entre llamadas repetidas.

C++\Training_Camp_2026\Day 4\D - Getting in Shape.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // fib[i] = Fib(i+1), i.e. fib[0]=Fib(1)=1, fib[1]=Fib(2)=1, fib[2]=Fib(3)=2, ...

```

5.10 Fibonacci (Zeckendorf-like Greedy) - Getting In Shape (cont.)

```

5 vector<long long> fib;
6
7 void buildFib() {
8     fib.push_back(1);
9     fib.push_back(1);
10    while (true) {
11        long long a = fib[fib.size() - 1];
12        long long b = fib[fib.size() - 2];
13        if (a > (long long)4e18 - b) break; // stop well before overflow
14        fib.push_back(a + b);
15        if (fib.back() > (long long)4e15) break; // more than enough headroom past 1e15
16    }
17 }
18
19 bool solve(long long N, string &result);
20
21 unordered_map<long long, pair<bool, string>> memo;
22
23 // Find lex-smallest V (ending in B) with h_V(1) + h_V(2) = S.
24 bool solveSum(long long S, string &result) {
25     // Try the largest k such that Fib(k+2) divides S exactly; Fib(k+2) = fib[k+1].
26     for (int k = (int)fib.size() - 2; k >= 1; k--) {
27         long long fibK2 = fib[k + 1]; // Fib(k+2)
28         if (fibK2 > S) continue;
29         if (S % fibK2 == 0) {
30             long long m = S / fibK2;
31             if (m == 1) {
32                 result = string(k - 1, 'A') + "B";
33                 return true;
34             } else {
35                 string sub;
36                 if (solve(m, sub)) {
37                     result = string(k - 1, 'A') + "B" + sub;
38                     return true;
39                 }
40             }
41         }
42     }
43     return false;
44 }
45
46 // Find lex-smallest string (ending in B) with h(1) = N.
47 // Memoized: the same sub-value N can be reached via many different divisor
48 // orderings when the original input has several Fibonacci-related factors,
49 // so without caching this can blow up (verified: >5,000,000 redundant calls
50 // on some inputs without memoization, vs ~1 with it).
51 bool solve(long long N, string &result) {
52     if (N == 1) {
53         result = "B";
54         return true;
55     }
56     auto it = memo.find(N);
57     if (it != memo.end()) {
58         if (it->second.first) result = it->second.second;
59         return it->second.first;
60     }
61     string sub;
62     bool ok = solveSum(N, sub);
63     string ans = ok ? ("A" + sub) : "";
64     memo[N] = {ok, ans};
65     if (ok) result = ans;
66     return ok;
67 }
68
69 int main() {
70     ios_base::sync_with_stdio(false);
71     cin.tie(nullptr);
72     buildFib();
73
74     long long N;
75     cin >> N;
76

```

5.10 Fibonacci (Zeckendorf-like Greedy) - Getting In Shape (cont.)

```

77     string result;
78     if (solve(N, result)) {
79         cout << result << "\n";
80     } else {
81         cout << "IMPOSSIBLE\n";
82     }
83     return 0;
84 }

```

5.11 Fibonacci Fast Doubling - Pou

Calcula el n-ésimo número de Fibonacci modulo 2026 usando exponenciación rápida de matriz.

C++\Codeforces\pou.cpp

```

1  #include <bits/stdc++.h>
2  typedef long long ll;
3  using namespace std;
4
5  int fibmod(ll n, int m)
6  {
7      int a = 0, b = 1, c;
8      for (int i = 63-__builtin_clzll(n); i >= 0; i--)
9      {
10         c = a;
11         a = (11l*c*(21l*b-c+m)) % m;
12         b = (11l*c*c + 11l*b*b) % m;
13         if ((n>>i) & 1)
14         {
15             c = (a+b) % m; a = b; b = c;
16         }
17     }
18     return a;
19 }
20
21 int main() {
22
23     int n;
24     cin >> n;
25
26     cout << fibmod(n+1,2026) << endl;
27
28     return 0;
29 }

```

5.12 Greedy - A Healthy Menu

Resuelve "A healthy menu" (Day 6, problema A - Maratona SBC de Programacao 2025): un colegio tiene M clases y N frutas, y para cada fruta i y clase j se conoce $G[i][j]$, la cantidad de estudiantes de esa clase a quienes les gusta esa fruta (un estudiante puede gustarle varias frutas). Hay que hallar el menor numero total de estudiantes posible en todo el colegio, consistente con esos conteos. Para cada clase, la cota minima de estudiantes es el mayor $G[i][j]$ entre todas las frutas i (y siempre es alcanzable dejando que un mismo estudiante le guste mas de una fruta). El codigo recorre cada clase i, calcula "maxxx" como el maximo de $clases[j][i]$ sobre todas las frutas j, y acumula esos maximos en "maxi", que es la respuesta final.

C++\Training_Camp_2026\Day 6\A - A healthy menu.cpp

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main()
6  {
7      int n; cin >> n;
8      int m; cin >> m;
9
10     vector<vector<int>> clases;
11     long long maxi = 0;
12
13     for(int i = 0 ; i < n; i++)
14     {
15         vector<int> pp;
16         for(int j = 0 ; j < m; j++)
17         {

```

5.12 Greedy - A Healthy Menu (cont.)

```

18             int a; cin>>a;
19             pp.push_back(a);
20         }
21         clases.push_back(pp);
22     }
23     for(int i = 0 ; i < m; i++)
24     {
25         long long maxxx = 0;
26         for(int j = 0 ; j < n; j++)
27         {
28             if(clases[j][i] >= maxxx)
29             {
30                 maxxx = clases[j][i];
31             }
32         }
33         maxi += maxxx;
34     }
35
36     cout << maxi<< endl;
37
38     return 0;
39 }

```

5.13 Implementation - Hour For A Run

Resuelve "Hour for a Run" (Day 1, problema H - Maratona de Programacao da SBC 2019): Vinicius corre V vueltas a una pista con N carteles igualmente espaciados y quiere saber, para cada decil (10%, 20%, ..., 90% del entrenamiento), cuantos carteles debe contar como minimo para garantizar que ya completo esa fraccion. Calcula $\text{ceil}(V*N*i/10)$ para i de 1 a 9 e imprime los 9 valores.

C++\Training_Camp_2026\Day 1\H - Hour_for_a_run.py

```

1  import math
2
3  n,c = list(map(int, input().split()))
4
5  mult = c*n
6  res = ""
7  for i in range(1, 10):
8      res += str(math.ceil((mult*i)/10))+ " "
9  print(res)

```

5.14 Logaritmos - Gatuno's Descent Into Psychopathy

Resuelve "Gatuno's Descent into Psychopathy" (Day 9, problema F - Contest 08 [Inicial & Avanzado]): el corazon de Gatuno de tamano H_1 se reduce con cada mordida segun $H_n = H_1 * ((B-1)/B)^n$, donde B es el "factor de brutalidad"; se pide el minimo numero de mordidas n para que H_n sea menor o igual a H_2 . Despeja n algebraicamente tomando logaritmo en base r = $(B-1)/B$ de la razon H_2/H_1 (o/h en el codigo) con $\text{math.log}(o/h, r)$, y redondea hacia arriba con math.ceil para obtener el minimo entero de mordidas que cumple la condicion.

C++\Training_Camp_2026\Day 9\F - Gatuno's Descent into Psychopathy.py

```

1  import math
2  n = int(input())
3
4  for _ in range(n):
5      h, o, r = list(map(int, input().split()))
6
7      r = (r-1)/r
8
9      print(math.ceil(math.log(o/h, r)))

```

5.15 Matrix Exponentiation - Buses

Resuelve "Buses" (Day 2, problema H - Maratona de Programacao da SBC 2013): dada una fila de longitud N metros formada por buses (10 m) y minibuses (5 m), con K colores posibles para minibus y L colores para bus, cuenta de cuantas formas distintas se puede armar la fila, entregando los ultimos 6 digitos del resultado. El numero de formas para una fila de largo $5*m$ sigue la recurrencia $f(m) = k*f(m-1) + l*f(m-2)$ (agregar al final un minibus de cualquier color o un bus de cualquier color), como un Fibonacci ponderado. Arma la matriz de transicion $M = \begin{bmatrix} k & l \\ 1 & 0 \end{bmatrix}$ y usa exponenciacion rapida de matrices (mat_pow) para obtener $f(m)$ en $O(\log m)$ modulo 10^6 , ya que N puede ser hasta 10^{15} .

C++\Training_Camp_2026\Day 2\H - Buses.py

```
1 def mat_mult(A, B, MOD):
2     return [
3         [(A[0][0]*B[0][0] + A[0][1]*B[1][0]) % MOD,
4          (A[0][0]*B[0][1] + A[0][1]*B[1][1]) % MOD],
5         [(A[1][0]*B[0][0] + A[1][1]*B[1][0]) % MOD,
6          (A[1][0]*B[0][1] + A[1][1]*B[1][1]) % MOD],
7     ]
8
9 def mat_pow(M, p, MOD):
10    R = [[1, 0], [0, 1]] # identidad
11    while p > 0:
12        if p & 1:
13            R = mat_mult(R, M, MOD)
14            M = mat_mult(M, M, MOD)
15            p >>= 1
16    return R
17
18 n, k, l = map(int, input().split())
19 MOD = 1000000
20 m = n // 5
21
22 M = [[k % MOD, l % MOD], [1, 0]]
23 P = mat_pow(M, m, MOD)
24 # f(m) = P[1][0]*f(1) + P[1][1]*f(0) = P[1][0]*k + P[1][1]
25 res = (P[1][0] * (k % MOD) + P[1][1]) % MOD
26 print(f"{res:06d}")
```

5.16 Numeros Romanos

O: (1), tabla fija de 13 valores

Uso: toRoman(n) con $1 \leq n \leq 3999$

Convierte un entero a numeral romano con un greedy sobre una tabla de valores ordenada de mayor a menor. El truco esta en incluir en la tabla los seis casos sustractivos (CM, CD, XC, XL, IX, IV) como si fueran simbolos propios: asi el greedy de "restar el mayor valor que quepa" nunca se equivoca y no hace falta ningun caso especial. Funciona para 1..3999.

C++\Notebook\Numeros Romanos.cpp

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 string toRoman(int num)
6 {
7     vector<pair<int, string>> roman = {
8         {1000, "M"}, {900, "CM"}, {500, "D"}, {400, "CD"},
9         {100, "C"}, {90, "XC"}, {50, "L"}, {40, "XL"},
10        {10, "X"}, {9, "IX"}, {5, "V"}, {4, "IV"},
11        {1, "I"}
12    };
13    string res;
14    for (const auto &p : roman)
15    {
16        while (num >= p.first)
17        {
18            res += p.second;
19            num -= p.first;
20        }
21    }
22    return res;
23 }
```

5.16 Numeros Romanos (cont.)

```
24
25 int main()
26 {
27     ios_base::sync_with_stdio(false);
28     cin.tie(nullptr);
29
30     int n;
31     while (cin >> n)
32     {
33         cout << toRoman(n) << "\n";
34     }
35
36     return 0;
37 }
```

5.17 Optimization - Satoru Gojo

Problema estilo Codeforces: dado S y M , halla el costo minimo de escoger k enteros positivos cuyo producto sea al menos S , donde usar mas de un factor ($k > 1$) cuesta M por cada factor extra ($(k-1)*M$). Para cada k candidato calcula la suma minima de k factores con producto $\geq S$ usando la raiz k -esima entera de S (los factores optimos son r y $r+1$, ya que por AM-GM la suma se minimiza haciendolos lo mas parejos posible) y prueba todos los k de 1 a 62 para quedarse con el mejor total.

C++\Codeforces\SatoruGojo.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 long long S;
5
6 // Multiply acc by base, but cap at S+1 (i.e. "definitely enough") to avoid overflow;
7 // once acc exceeds S we no longer care about the exact value, only that it's >= S.
8 inline long long capMul(long long acc, long long base) {
9     if (acc > S) return acc; // already known to be enough
10    // check for overflow before multiplying: if acc * base would exceed a safe bound, cap
11    if (base != 0 && acc > (S + 5) / base + 1) {
12        return S + 5; // definitely exceeds S, safe sentinel
13    }
14    long long r = acc * base;
15    if (r > S) r = S + 5; // cap
16    return r;
17 }
18
19 // largest r (>=1) such that r^k <= S
20 long long integerKthRoot(int k) {
21     long long lo = 1, hi = S;
22     while (lo < hi) {
23         long long mid = lo + (hi - lo + 1) / 2;
24         long long val = 1;
25         bool exceeds = false;
26         for (int i = 0; i < k; i++) {
27             val = capMul(val, mid);
28             if (val > S) { exceeds = true; break; }
29         }
30         if (!exceeds) lo = mid;
31         else hi = mid - 1;
32     }
33     return lo;
34 }
35
36 // minimal sum of k positive integers with product >= S
37 long long minSumForK(int k) {
38     long long r = integerKthRoot(k);
39     // try j = 0..k bumps from r to r+1, find smallest j with r^(k-j)*(r+1)^j >= S
40     for (int j = 0; j <= k; j++) {
41         long long val = 1;
42         for (int t = 0; t < k - j; t++) {
43             val = capMul(val, r);
44         }
45         for (int t = 0; t < j; t++) {
46             val = capMul(val, r + 1);
47         }
48         if (val >= S) {
```

5.17 Optimization - Satoru Gojo (cont.)

```

48     return (long long)k * r + j;
49 }
50 }
51 }
52 // fallback (shouldn't happen): all r+1
53 return (long long)k * (r + 1);
54 }
55
56 int main() {
57     long long M;
58     cin >> S >> M;
59
60     long long best = LLONG_MAX;
61     // k up to ~62 is more than enough: 2^62 far exceeds 1e12, and the natural optimum
62     // (ignoring M) is around ln(S) <= ~28 for S<=1e12; checking generously up to 62
63     // safely covers the region where increasing k could still help when M is small.
64     int maxK = 62;
65     for (int k = 1; k <= maxK; k++) {
66         long long sum = minSumForK(k);
67         long long total = sum + (long long)(k - 1) * M;
68         if (total < best) best = total;
69     }
70
71     cout << best << "\n";
72     return 0;
73 }

```

5.18 Paridad - Call

NO ES MIO: código de Stiven Correa, del repo del equipo (carpeta RPC/). Resuelve "Call" (RPC 2026-07, problema C): contar cuantos de los n valores son impares. Técnica: $r \% 2$ devuelve 1 si es impar y 0 si es par, así que se suma directo sin ningún if.

RPC\2026-07\C - Call.py

```

1 n = int(input())
2
3 res = 0
4 for _ in range(n):
5     r = int(input())
6     res += r%2
7 print(res)

```

5.19 Polynomial Composition - Coach

Calcula la composición de polinomios $P(Q(x))$ usando multiplicación polinómica modular. El resultado es modular 998244353 y tiene grado máximo $n*m$.

C++\Codeforces\coach.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const long long MOD = 998244353;
5
6 // Multiplica dos polinomios A y B
7 // A[i] representa el coeficiente de x^i
8 // B[j] representa el coeficiente de x^j
9 vector<long long> multiply(const vector<long long>& A, const vector<long long>& B) {
10     vector<long long> result(A.size() + B.size() - 1, 0);
11
12     for (int i = 0; i < (int)A.size(); i++) {
13         for (int j = 0; j < (int)B.size(); j++) {
14             // A[i] * x^i multiplicado por B[j] * x^j
15             // queda en el grado i + j
16             result[i + j] += A[i] * B[j];
17             result[i + j] %= MOD;
18         }
19     }
20
21     return result;
22 }
23
24 int main() {
25     ios::sync_with_stdio(false);
26     cin.tie(nullptr);

```

5.19 Polynomial Composition - Coach (cont.)

```

27
28     int n, m;
29     cin >> n >> m;
30
31     // P tiene grado n, por eso tiene n + 1 coeficientes
32     vector<long long> P(n + 1);
33
34     // Q tiene grado m, por eso tiene m + 1 coeficientes
35     vector<long long> Q(m + 1);
36
37     for (int i = 0; i <= n; i++) {
38         cin >> P[i];
39     }
40
41     for (int i = 0; i <= m; i++) {
42         cin >> Q[i];
43     }
44
45     // El grado máximo de P(Q(x)) es n * m
46     // Por eso la respuesta tiene n*m + 1 coeficientes
47     vector<long long> ans(n * m + 1, 0);
48
49     // power representa Q^i
50     // Al inicio i = 0, entonces Q^0 = 1
51     vector<long long> power = {1};
52
53     // Calculamos:
54     // P(Q(x)) = p0*Q^0 + p1*Q^1 + p2*Q^2 + ... + pn*Q^n
55     for (int i = 0; i <= n; i++) {
56
57         // Sumamos P[i] * Q^i a la respuesta
58         for (int j = 0; j < (int)power.size(); j++) {
59             ans[j] += P[i] * power[j];
60             ans[j] %= MOD;
61         }
62
63         // Preparamos la siguiente potencia:
64         // si ahora power = Q^i,
65         // después de multiplicar por Q queda power = Q^(i+1)
66         if (i < n) {
67             power = multiply(power, Q);
68         }
69     }
70
71     // Imprimimos los coeficientes r0, r1, ..., r(n*m)
72     for (int i = 0; i <= n * m; i++) {
73         cout << ans[i] << " ";
74     }
75
76     cout << "\n";
77
78     return 0;
79 }

```

5.20 Prefix Sums - Triangles

Resuelve "Triangles" (Day 2, problema F - Maratona de Programacao da SBC 2013): dados N puntos sobre un círculo, descritos por las longitudes de arco entre puntos consecutivos, cuenta cuantos triángulos equiláteros distintos se pueden formar usando esos puntos como vértices. Convierte los arcos en posiciones absolutas sobre el círculo con una suma prefija (tot), marcando cada posición ocupada en el arreglo booleano `nums[]`. Un triángulo equilátero solo puede existir si el perímetro total es múltiplo de 3 ($tri = tot/3$); para cada posición marcada i , verifica si $i+tri$ e $i+2*tri$ también están marcadas, y de ser así cuenta un triángulo y desmarca las tres posiciones para no reutilizarlas.

C++\Training_Camp_2026\Day 2\F - Triangles.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     long long n; cin >> n;

```

5.20 Prefix Sums - Triangles (cont.)

```

1   long long tot = 0;
2   long long god = 0;
3   vector<long long> nums1;
4   for(long long i = 0 ; i < n ; i++)
5   {
6       long long a; cin >>a;
7       tot+= a;
8       nums1.push_back(tot);
9   }
10  vector<long long> nums(1000000,0);
11  for(long long i = 0 ; i < nums1.size() ; i++)
12  {
13      nums[nums1[i]]=1;
14  }
15
16  if(tot%3!=0)
17  {
18      cout<< 0 <<endl;
19  }
20  else
21  {
22      long long tri = tot/3;
23      for(long long i = 0 ; i < (1000000) ; i++)
24      {
25          if(nums[i]==1 && nums[i]==nums[i+tri]&&nums[i]==nums[i+tri+tri]&&i<1000000&&i+tri<1000000&&i+tri+tri<1000000)
26          {
27              god++;
28              nums[i]=0;
29              nums[i+tri] = 0;
30              nums[i+tri+tri] = 0;
31          }
32      }
33
34      cout << god << endl;
35  }
36  return 0;
37 }

```

5.21 Recurrencia (Fibonacci) - Fractions Are Better When Continued

Resuelve "Fractions are better when continued" (Day 3, problema F - Maratona SBC de Programacao 2024): dado un entero N, hay que hallar el numerador (en forma irreducible) de la fraccion continua p_N definida recursivamente como $p_0 = 1$, $p_1 = 1/(1+1)$, $p_2 = 1/(1+1/(1+1))$, y así sucesivamente, agregando un nivel mas de fraccion en cada paso. Al expandir la recurrencia, los numeradores de p_N resultan ser exactamente los numeros de Fibonacci (1, 1, 2, 3, 5, 8, ...). El código aprovecha esto y calcula el resultado con un simple bucle iterativo que va sumando e intercambiando num y dem en cada paso (equivalente a $\text{num} = \text{dem}$, $\text{num} += \text{dem}$), evitando aritmetica de fracciones o recursion explicita.

C++\Training_Camp_2026\Day 3\F - Fractions are better when continued.cpp

```

1   #include <bits/stdc++.h>
2
3   using namespace std;
4
5   int main()
6   {
7       long long n ; cin >> n;
8
9       if(n==1)
10      {
11          cout<<1<<endl;
12      }
13      else if(n == 2)
14      {
15          cout<<2<<endl;
16      }
17      else
18      {
19          long long num = 1;

```

5.21 Recurrencia (Fibonacci) - Fractions Are Better When Continued (cont.)

```

20      long long dem = 2;
21      long long temp = 0;
22      long long temp2 = 0;
23      while(n>1)
24      {
25          temp2 = num + dem;
26          num = temp2;
27
28          temp = num;
29          num = dem;
30          dem = temp;
31
32          n--;
33      }
34
35      cout<<num<<endl;
36  }
37
38  return 0;
39 }

```

5.22 Recurrencias y búsqueda de intervalo factible - Investigating Quadradômeda

Resuelve "Investigating Quadradômeda" (Day 6, problema I - Maratona SBC de Programacao 2025): N estrellas alineadas consecutivamente en horizontal o vertical deben orbitarse con radios enteros $R_i \geq 1$, cada una estrictamente menor que la distancia d_i a la siguiente estrella, y la condicion de tangencia al pasar de una orbita a la siguiente obliga a que $R_{i+1} = d_i - R_i$ (recurrencia alternante). Hay que hallar el mayor R_1 entero para el cual existe una asignacion valida de todos los R_i (o -1 si no existe ninguna). El código acumula C_i via $C_{i+1} = d_i - C_i$ (con $R_i = \pm R_1 + C_i$ segun la paridad de i), y va combinando, para cada indice, una cota inferior L (de los i impares, $R_1 \geq 1 - C_i$) o superior U (de los i pares, $R_1 \leq C_i - 1$); si $L \leq U$ al final, la respuesta es U, si no, -1.

C++\Training_Camp_2026\Day 6\I - Investigating Quadradômeda.cpp

```

1   #include <bits/stdc++.h>
2   using namespace std;
3
4   int main(){
5       ios_base::sync_with_stdio(false);
6       cin.tie(nullptr);
7
8       int n;
9       cin >> n;
10
11      vector<long long> x(n), y(n);
12      for(int i = 0; i < n; i++) cin >> x[i] >> y[i];
13
14      // R_i = (-1)^(i-1) * R_1 + C_i, with C_1 = 0 and C_{i+1} = d_i - C_i
15      // Need R_i >= 1 for all i:
16      // i odd -> R_1 >= 1 - C_i (lower bound)
17      // i even -> R_1 <= C_i - 1 (upper bound)
18      // Also R_i < d_i for i < n is automatically implied by R_{i+1} >= 1.
19
20      long long C = 0; // C_i, starts at C_1 = 0
21      long long L = LLONG_MIN;
22      long long U = LLONG_MAX;
23
24      for(int i = 1; i <= n; i++){
25          if(i % 2 == 1){
26              // odd index: lower bound constraint
27              L = max(L, 1 - C);
28          } else {
29              // even index: upper bound constraint
30              U = min(U, C - 1);
31          }
32          if(i < n){
33              long long d;
34              if(x[i - 1] == x[i]){
35                  d = llabs(y[i] - y[i - 1]);
36              } else {
37                  d = llabs(x[i] - x[i - 1]);
38              }

```

5.22 Recurrencias y búsqueda de intervalo factible - Investigating Quadradômêda (cont.)

```

38     C = d - C;
39 }
40 }
41 }
42 }
43 if(L > U){
44     cout << -1 << "\n";
45 } else {
46     cout << U << "\n";
47 }
48 return 0;
49 }
50 }

```

5.23 Recursion (Simulacion tipo Josephus) - Metal Detector

Resuelve "Metal detector" (Day 5, problema C - Contest 05 [Avanzados]): en una fila de N personas, un detector defectuoso se activa en forma alternada (la persona 1 no lo activa, la 2 si, la 3 no, y así sucesivamente), cada persona que lo activa se manda al final de la fila, y para la persona que esta en la posición i hay que hallar en que turno j finalmente pasara sin activarlo. Simula el proceso por rondas sin construir la fila explicita: la variable "cual" indica si en esta ronda pasan sin activar las posiciones pares o impares, acumula en "llevo" cuantas personas ya pasaron libres, y reduce n y p a la mitad (floor) para continuar con la sub- fila restante hasta que la posición p caiga del lado que no activa el detector.

C++\Training_Camp_2026\Day 5\C - Metal detector.py

```

1 import math
2 t = int(input())
3 for i in range(t):
4     n, p = list(map(int, input().split()))
5     if(p%2 != 0):
6         print(math.ceil(p/2))
7         continue
8     llevo = 0
9     cual = 1
10    q = n
11    while True:
12        if(cual == 1):
13            if(p%2!=0):
14                print(llevo + math.ceil(p/2))
15                break
16            a = math.ceil(n/2)
17            llevo += a
18            n = math.floor(n/2)
19            p = math.floor(p/2)
20        else:
21            if(p%2!=1):
22                print(llevo + math.ceil(p/2))
23                break
24            a = math.ceil(n/2)
25            llevo += a
26            n = math.floor(n/2)
27            p = math.floor(p/2)
28
29    if(q%2==0):
30        cual = 1
31    else:
32        cual = 2
33
34    q = n

```

5.24 Simulacion - Cahokia Ruins

Resuelve "Cahokia ruins" (Day 5, problema A - Contest 05 [Avanzados]): al entrar a una sala rectangular H x W, las paredes este y oeste empiezan a moverse una hacia la otra a 1 m/s cada una; cada fila i tiene un ancho de pared (arreglos left y right en el código) medido desde su borde respectivo, y hay que calcular en cuanto tiempo chocan las paredes por primera vez. Para cada fila calcula el hueco libre restante, w - right[i] - left[i], y se queda con el mínimo (variable mins), ya que esa fila es la que se cierra primero; como ambas paredes se acercan a 1 m/s, el tiempo de colisión es ese hueco mínimo dividido entre 2 (velocidad de cierre combinada).

C++\Training_Camp_2026\Day 5\A - Cahokia ruins.py

```

1 h, w = list(map(int, input().split()))
2

```

5.24 Simulacion - Cahokia Ruins (cont.)

```

3 left = list(map(int, input().split()))
4 right = list(map(int, input().split()))
5
6 mins = float("inf")
7 for i in range(len(right)):
8     x = w - right[i]
9     mins = min(mins, x-left[i])
10
11 if mins // 2 == mins / 2:
12     print(int(mins/2))
13 else:
14     print(mins/2)

```

6 - Number Theory

6.1 BFS sobre Residuos - Only1s0s

Resuelve "Only1s0s" (problema H, ICPC 2024). El código busca el menor múltiplo M de N que se escribe solo con unos y ceros, y responde M / N. M puede tener muchísimos dígitos, así que no se busca entre números sino entre RESIDUOS modulo N: dos números con el mismo residuo se comportan igual de ahí en adelante (agregarles un dígito lleva al mismo residuo), así que basta quedarse con el primero que llega a cada residuo. Hay a lo sumo N estados, y el BFS termina en O(N). Siempre existe: entre 1, 11, 111, ... (N+1 números) dos tienen el mismo residuo, y su resta es un múltiplo de N hecho de unos y ceros. POR QUE SALE EL MENOR: el BFS encuentra primero los más cortos, y dentro del mismo largo, como se agrega el 0 antes que el 1 y la cola ya viene ordenada, los números salen en orden creciente. Así que la primera vez que se alcanza el residuo 0 es con el menor M. Se reconstruye con parent[] y digit[]. Al final M se divide entre N como en la escuela, dígito por dígito con el resto, porque no cabe en ningún tipo entero. El 10LL en la transición evita el desborde de cur * 10.

C++\ICPC\2024\H - Only1s0s.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string solve(int N) {
5
6     vector<int> parent(N, -1);
7     vector<char> digit(N);
8     vector<bool> visited(N, false);
9
10    queue<int> q;
11
12    // Empezamos con el número 1
13    int r = 1 % N;
14
15    visited[r] = true;
16    digit[r] = '1';
17
18    q.push(r);
19
20    while (!q.empty()) {
21
22        int cur = q.front();
23        q.pop();
24
25        // Encontramos un múltiplo de N
26        if (cur == 0)
27            break;
28
29        // Agregar 0
30        int r0 = (cur * 10LL) % N;
31
32        if (!visited[r0]) {
33
34            visited[r0] = true;
35            parent[r0] = cur;
36            digit[r0] = '0';
37
38            q.push(r0);
39        }
40
41        // Agregar 1

```

6.1 BFS sobre Residuos - Only1s0s (cont.)

```

42     int r1 = (cur * 10LL + 1) % N;
43
44     if (!visited[r1]) {
45
46         visited[r1] = true;
47         parent[r1] = cur;
48         digit[r1] = '1';
49
50         q.push(r1);
51     }
52 }
53
54 // Reconstruir M
55 string M;
56
57 int cur = 0;
58
59 while (cur != -1) {
60     M.push_back(digit[cur]);
61     cur = parent[cur];
62 }
63
64 reverse(M.begin(), M.end());
65
66 return M;
67 }
68
69
70 // Divide un numero gigante representado como string
71 // entre N.
72 string divide(string M, int N) {
73
74     string ans;
75     long long rem = 0;
76
77     for (char c : M) {
78
79         rem = rem * 10 + (c - '0');
80
81         ans += char('0' + rem / N);
82
83         rem %= N;
84     }
85
86     // Quitar ceros iniciales
87     int pos = 0;
88
89     while (pos + 1 < (int)ans.size() &&
90            ans[pos] == '0') {
91         pos++;
92     }
93
94     return ans.substr(pos);
95 }
96
97
98 int main() {
99
100     ios::sync_with_stdio(false);
101     cin.tie(nullptr);
102
103     int N;
104
105     while (cin >> N && N != 0) {
106
107         string M = solve(N);
108
109         string D = divide(M, N);
110
111         cout << D << '\n';
112     }
113 }

```

6.1 BFS sobre Residuos - Only1s0s (cont.)

```

14     return 0;
15 }

```

6.2 Big Exponentiation - Uj (cpp)

Calcula p^q usando aritmética de precisión arbitraria (boost::multiprecision) para manejar resultados con exponentes muy grandes.

C++\CCPL\uj.cpp

```

1 #include <bits/stdc++.h>
2 #include <boost/multiprecision/cpp_int.hpp>
3
4 using namespace std;
5 using namespace boost::multiprecision;
6
7 int main() {
8     while (true) {
9         long long p, q;
10         cin >> p >> q;
11
12         if (p == 0 && q == 0) return 0;
13
14         cpp_int b = 1;
15
16         for (long long i = 0; i < q; i++) {
17             b *= p;
18         }
19
20         cout << b << "\n";
21     }
22 }

```

6.3 Big Exponentiation - Uj (java)

Calcula N^D usando BigInteger para manejar resultados con exponentes muy grandes.

JAVA\CCPL\uj.java

```

1 import java.util.*;
2 import java.math.*;
3
4 class Main {
5     public static void main (String[] args) throws Exception {
6         Scanner sc = new Scanner(System.in);
7
8         while (sc.hasNext()) {
9             int N = sc.nextInt();
10            int D = sc.nextInt();
11
12            if (N == 0 && D == 0) break;
13
14            System.out.println(BigInteger.valueOf(N).pow(D));
15        }
16    }
17 }

```

6.4 Criba de Eratostenes (Solo Impares)

O: (n log log n), memoria n/2 bits

Uso: vector<int> p = criba(n); // devuelve los primos hasta n

La criba que uno debería escribir por defecto: como el 2 es el unico primo par, no vale la pena reservar ni recorrer las posiciones pares. El arreglo solo guarda los impares, con el indice k representando al numero $2k+1$, asi que ocupa la MITAD de memoria y hace la mitad del trabajo. Medido en esta maquina, mediana de 3 corridas, contra la criba normal de n bits: $n=10^7$ $n=3*10^7$ $n=10^8$ Criba clasica (n bits) 30 ms 98 ms 427 ms Esta (solo impares) 15 ms 46 ms 170 ms Dos veces y media mas rapida en 10^8 , y con la mitad de RAM. La ganancia no es solo por hacer menos operaciones: al ocupar la mitad, el arreglo cabe mejor en cache, y una criba es un recorrido de memoria casi puro. LAS DOS CONVERSIONES QUE HAY QUE TENER CLARAS, que es donde se equivoca uno: valor $2k+1 \rightarrow$ indice $k = (valor - 1) / 2$ se empieza a marcar en p^* , que es impar, y su indice es $(p^* - 1) / 2$ se avanza de p en p SOBRE LOS INDICES, no de $2p$ en $2p$ sobre los valores. Es lo mismo: subir un indice p equivale a subir $2p$ en el valor, que es justo saltarse el multiplo par de p. Ejemplo para $p = 3$: se arranca en el indice $(9-1)/2 = 4$, que es el valor 9. El siguiente indice es $4+3 = 7$, o sea el valor 15. Se salta el 12 solito, porque es par y no esta en el arreglo. Empezar en p^* y no en $2p$ es lo de siempre: todo multiplo de p menor que p^* ya lo marco un primo mas chico. Y el bucle interno se corta por si mismo cuando $(p^*-1)/2 > m$, asi que no hace falta ningun if extra ni una raiz cuadrada. Costo $O(n \log \log n)$ en tiempo y n/2 bits de memoria. Con $n = 10^8$ son unos 6 MB. Verificado contra la criba clasica: lista de primos identica para todo n hasta 200000, y el mismo conteo en 10^6 , 10^7 , $3*10^7$ y 10^8 (664579 primos hasta 10^7 , 5761455 hasta 10^8). CUANDO USAR ESTA Y CUANDO OTRA COSA: - Hace falta la lista de primos, o saber si un numero es primo, hasta 10^7 o 10^8 : esta. - Hace falta el MENOR FACTOR PRIMO de cada numero para factorizar rapido: criba de spf, que es el mismo costo pero guarda un int por posicion en vez de un bit (y ahi si conviene el arreglo completo, porque hay que indexar por el numero). - Un solo numero hasta 10^{12} y solo saber si es primo: no se criba, se prueba dividir hasta la raiz. Mas grande que eso, Miller-Rabin. - El rango es [a,b] con b hasta 10^{12} pero b-a chico: criba segmentada, cribando primero hasta $\sqrt{b}$ con esta misma y despues marcando el segmento. OJO: vector<bool> es un bitset empaquetado, y eso es lo que da los n/2 bits. Si se cambia por vector<char> el consumo se multiplica por 8, aunque a veces corra mas rapido por no desempacar bits. Con $n = 10^8$ la diferencia es 6 MB contra 50 MB, asi que suele no haber opcion.

C++\Notebook\Criba de Eratostenes (Solo Impares).cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // Todos los primos <= n, en orden.
5 vector<int> criba(int n) {
6     if (n < 2) return {};
7
8     int m = (n - 1) / 2; // el indice k es el impar 2k+1
9     vector<bool> compuesto(m + 1, false);
10
11     vector<int> primos;
12     primos.push_back(2); // el unico par, va aparte
13
14     for (int k = 1; k <= m; k++) {
15         if (compuesto[k]) continue;
16
17         long long p = 2LL * k + 1; // long long: p*p se pasa de int
18         primos.push_back((int)p);
19
20         for (long long j = (p * p - 1) / 2; j <= m; j += p)
21             compuesto[j] = true;
22     }
23
24     return primos;
25 }
26
27 // Version que deja la tabla para consultar despues si x es primo en O(1).
28 // esPrimo(x) = x == 2, o x impar con !compuesto[(x-1)/2].
29
30 int main() {
31     ios::sync_with_stdio(false);
32     cin.tie(nullptr);
33
34     int n;
35     if (!(cin >> n)) return 0;
36 }
```

6.4 Criba de Eratostenes (Solo Impares) (cont.)

```
37 vector<int> primos = criba(n);
38
39 cout << primos.size() << " primos hasta " << n << '\n';
40 for (int p : primos) cout << p << ' ';
41 cout << '\n';
42 return 0;
43 }
```

6.5 Criba de Factor Primo Mínimo + Inclusion-Exclusion - Ingredients That May Harm You

Resuelve "Ingredients that may Harm You" (Day 3, problema I - Maratona SBC de Programacao 2024): en un restaurante self-service con N platillos identificados por numeros (donde cada primo que los divide representa un ingrediente basico, con multiplicidad), cada uno de Q comensales tiene un numero X que identifica sus alergias (los primos que dividen a X); para cada comensal hay que contar cuantos subconjuntos de platillos se pueden combinar sin que ninguno de ellos contenga un ingrediente al que sea alergico. El codigo precalcula el menor factor primo (spf) hasta $1e6$ con una criba, para factorizar rapido cualquier numero, y para cada platillo enumera por mascara de bits (mask) todos los divisores libres de cuadrados formados por sus primos, acumulando cnt[d]. Para cada consulta factoriza X y aplica inclusion-exclusion sobre los divisores de X (sumando o restando cnt[d] segun la paridad de bits del mask) para contar cuantos platillos son coprimos con X; la respuesta final es $2^{(\text{coprimos})} \bmod 1e9+7$, ya que cada platillo compatible puede incluirse o no libremente en el plato.

C++\Training_Camp_2026\Day 3\ - Ingredients that may Harm You.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXV = 1000000;
5 const int MOD = 1000000007;
6
7 int spf[MAXV + 1];
8 int cnt[MAXV + 1];
9 int pw[100005];
10
11 int main() {
12     ios::sync_with_stdio(false);
13     cin.tie(nullptr);
14
15     // SPF
16     for (int i = 0; i <= MAXV; i++) spf[i] = i;
17     for (int i = 2; i * i <= MAXV; i++) {
18         if (spf[i] == i) {
19             for (int j = i * i; j <= MAXV; j += i)
20                 if (spf[j] == j) spf[j] = i;
21         }
22     }
23
24     int N;
25     cin >> N;
26
27     pw[0] = 1;
28     for (int i = 1; i <= N; i++)
29         pw[i] = (2LL * pw[i - 1]) % MOD;
30
31     while (N--) {
32         int x;
33         cin >> x;
34
35         vector<int> p;
36         while (x > 1) {
37             int pr = spf[x];
38             p.push_back(pr);
39             while (x % pr == 0) x /= pr;
40         }
41
42         int m = p.size();
43
44         for (int mask = 0; mask < (1 << m); mask++) {
45             int d = 1;
46             for (int i = 0; i < m; i++)
47                 if (mask & (1 << i))
```

6.5 Criba de Factor Primo Mínimo + Inclusion-Exclusion - Ingredients That May Harm You (cont.)

```

48         d *= p[i];
49         cnt[d]++;
50     }
51 }
52
53 int Q;
54 cin >> Q;
55
56 while (Q--) {
57     int x;
58     cin >> x;
59
60     vector<int> p;
61     while (x > 1) {
62         int pr = spf[x];
63         p.push_back(pr);
64         while (x % pr == 0) x /= pr;
65     }
66
67     int m = p.size();
68
69     int coprime = 0;
70
71     for (int mask = 0; mask < (1 << m); mask++) {
72         int d = 1;
73         int bits = 0;
74
75         for (int i = 0; i < m; i++) {
76             if (mask & (1 << i)) {
77                 d *= p[i];
78                 bits++;
79             }
80         }
81
82         if (bits & 1)
83             coprime -= cnt[d];
84         else
85             coprime += cnt[d];
86     }
87
88     cout << pw[coprime] << '\n';
89 }
90
91 return 0;
92 }

```

6.6 Diferencia de Cuadrados - Square Diff

Resuelve "SquareDiff" (problema C, ICPC 2024). El código responde si n se puede escribir como diferencia de dos cuadrados, y toda la solución es una condición: $n = a^2 - b^2$ tiene solución $\Leftrightarrow n \bmod 4 \neq 2$. Por que: $a^2 - b^2 = (a - b)(a + b)$, y esos dos factores tienen la MISMA paridad (se diferencian en 2b). Dos impares dan un producto impar, y dos pares dan un múltiplo de 4; nunca sale algo que sea par pero no múltiplo de 4. Y al revés siempre se puede: un impar $2k+1 = (k+1)^2 - k^2$, y un múltiplo de 4, $4m = (m+1)^2 - (m-1)^2$. Verificado por fuerza bruta para n de 1 a 3000. El reflejo que deja este problema: ante "se puede escribir como...", factorizar la expresión y mirar la PARIDAD de los factores suele matarlo en una línea. OJO si n pudiera ser negativo: en C++ $n \% 4$ conserva el signo, así que $-2 \% 4$ da -2 y no 2.

C++\ICPC\2024\C - SquareDiff.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9
10    while (cin >> n && n != 0) {
11        cout << (n % 4 == 2 ? 'N' : 'Y') << '\n';
12    }
13 }

```

6.6 Diferencia de Cuadrados - Square Diff (cont.)

```

14 return 0;
15 }

```

6.7 Divisibilidad - Archmage's Crystals

Resuelve "Archmage's Crystals" (Day 9, problema A - Contest 08 [Inicial & Avanzado]): un archimago tiene n cristales con potencias a_i ; cada hechizo de fusión toma dos cristales x, y y los reemplaza por uno de poder $x+y$, reduciendo la cantidad de cristales en 1. Se busca el mínimo número de hechizos para que el promedio de los cristales restantes sea un entero. La suma total nunca cambia con las fusiones, así que solo hace falta hallar el mayor divisor i de la suma con $i \leq n$ (recorriendo i desde n hacia 1 con la variable c contando los intentos fallidos); la respuesta es $n - i$, es decir, la cantidad de fusiones necesarias para llegar a ese divisor.

C++\Training_Camp_2026\Day 9\A - Archmage's Crystals.py

```

1 n = int(input())
2 l = list(map(int, input().split()))
3
4 suma = sum(l)
5 c = 0
6 for i in range(n, 0, -1):
7     if (suma % i == 0):
8         break
9     c += 1
10 print(c)

```

6.8 Divisors - Aesthetics In Poetry

Resuelve "Aesthetics in poetry" (Day 7, problema A - Contest 07 [Avanzados]): dado un poema de N versos con longitudes $l_1, l_2, \dots, l_N$, hay que hallar el menor entero $K > 1$ (K -elegante) tal que N sea múltiplo de K y, para cada resto $i = 0..K-1$ al dividir las longitudes por K , haya exactamente N/K versos con ese resto (o -1 si no existe tal K). Genera todos los divisores de N (función divisores), los ordena ascendentemente y para cada candidato $k \geq 2$ cuenta en $\text{cont}[v \% k]$ cuantos versos caen en cada resto, verificando que todos los conteos sean iguales a n/k ; el primer k que cumple es la respuesta mínima.

C++\Training_Camp_2026\Day 7\A - Aesthetics in poetry.cpp

```

1 #include <bits/stdc++.h>
2 typedef long long ll;
3 using namespace std;
4
5 vector<ll> divisores(ll n)
6 {
7     vector<ll> div;
8     for (ll i = 1; i * i <= n; i++)
9     {
10         if (n % i == 0)
11         {
12             div.push_back(i);
13             if (i != n/i) div.push_back(n/i);
14         }
15     }
16     return div;
17 }
18
19 int main()
20 {
21     ll n; cin >> n;
22     vector<ll> xd(n);
23     for (int i = 0; i < n; i++)
24     {
25         cin >> xd[i];
26     }
27
28     vector<ll> juan = divisores(n);
29     sort(juan.begin(), juan.end());
30
31     for (ll k : juan)
32     {
33         if (k >= 2)
34         {
35
36             vector<ll> cont(k, 0);
37             for (ll v : xd)

```

6.8 Divisors - Aesthetics In Poetry (cont.)

```

38         {
39             cont[v % k]++;
40         }
41
42         bool ok = true;
43         for(ll c : cont)
44         {
45             if(c != n/k) { ok = false; break; }
46         }
47
48         if(ok)
49         {
50             cout << k << endl;
51             return 0;
52         }
53     }
54 }
55
56 cout << -1 << endl;
57 return 0;
58 }

```

6.9 Exponenciacion Modular (Torre de Exponentes) - Exponentiation II

Resuelve "Exponentiation II" (Day 10, problema K - Contest 09 [Avanzados]): calcula a^{b^c} modulo $1e9+7$ para n consultas, con a, b, c hasta $1e9$, donde el exponente b^c es demasiado grande para calcularlo directamente. Usa el Pequeño Teorema de Fermat: como MOD es primo, $a^{(MOD-1)} = 1$ (para a no multiplo de MOD), así que el exponente de la torre puede reducirse modulo MOD-1 antes de aplicarlo; el codigo llama `expmod(b, c, MOD-1)` para obtener ese exponente reducido y luego `expmod(a, ese_valor, MOD)` para el resultado final, reutilizando la misma exponenciacion binaria (fast power) para ambos niveles.

C++\Training_Camp_2026\Day 10\K - Exponentiation II.cpp

```

1 #include <iostream>
2 using namespace std;
3 typedef long long ll;
4 ll MOD = 1000000007;
5
6 ll expmod(ll b, ll e, ll m)
7 {
8     ll ans = 1;
9     while (e) {
10         if (e&1) ans = (1ll*ans*b) % m;
11         b = (1ll*b*b) % m;
12         e /= 2;
13     }
14     return ans;
15 }
16
17
18 int main()
19 {
20     ll n; cin >> n;
21
22     while(n--)
23     {
24         ll a , b , c; cin >> a >> b >> c;
25
26         cout<< expmod(a,expmod(b,c,MOD-1),MOD)<< endl;
27     }
28
29     return 0;
30 }

```

6.10 Factorizacion de un Numero Gigante Redondeado - Fuzzy Factorization

Resuelve "Fuzzy Factorization" (problema F, Regionals 2025): factorizar un numero que llega como cadena y que esta redondeado, o sea que todo lo que sigue despues de los primeros digitos significativos son ceros. LA OBSERVACION QUE LO RESUELVE: si el numero tiene D digitos y solo los primeros 10 son significativos, entonces vale (primeros 10 digitos) $\times 10^{(D-10)}$. Y 10^k aporta exactamente k factores 2 y k factores 5, sin tener que tocar el numero grande. Por eso el primer for solo suma +1 al 2 y al 5 por cada digito de mas. Lo que queda es factorizar Y , un numero de a lo sumo 10 digitos, con division por tentativa hasta la raiz: $O(\sqrt{Y}) = \text{unas } 10^5 \text{ operaciones}$. El `if(Y > 1)` del final es OBLIGATORIO: si despues de dividir por todo hasta la raiz queda algo mayor que 1, ese resto es un primo y hay que contarlo. Olvidarlo es el error mas comun de esta plantilla, y solo se nota cuando el numero tiene un factor primo grande. El map deja los primos ordenados automaticamente, que es lo que pide la salida, y ademas fusiona los 2 y los 5 que vienen de las dos fuentes (los ceros del final y la factorizacion de Y).

C++\ICPC\Regionals 2025\F - Fuzzy Factorization.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4 #define SIZE(c) int((c).size())
5 #define forn(i,n) for(int i = 0 ; i < n ; i++ )
6
7 const int DIGITS = 10;
8
9 int main() {
10
11     ios::sync_with_stdio(false);
12     cin.tie(nullptr);
13
14     string s;
15
16     cin >> s;
17
18     map<ll,int> f;
19     forn(t,SIZE(s)-DIGITS)
20     {
21         f[2]++;
22         f[5]++;
23     }
24     ll Y = stoll(s.substr(0,min(10, SIZE(s))));
25
26     for(ll p = 2; p*p <= Y;p++)
27     {
28         while(Y % p == 0)
29         {
30             Y /= p;
31             f[p]++;
32         }
33     }
34
35     if(Y>1)
36         f[Y]++;
37
38
39     cout << SIZE(f) << "\n";
40     for(auto it : f)
41         cout << it.first << " " << it.second << "\n";
42
43     return 0;
44 }

```

6.11 Factorizacion en Digos

O: ($\log n * 8$) probando divisores de 9 a 2

Uso: recursive(num) -> string con los digitos, o vacio si no se puede

Halla el menor numero cuyos digitos multiplicados dan N. La idea greedy: dividir siempre por el mayor factor posible entre 9 y 2, porque usar factores grandes minimiza la cantidad de digitos, y al final ordenar los digitos de menor a mayor da el numero mas chico posible con esa multiset de digitos. Si al terminar el producto de los digitos no coincide con N, es que N tiene un factor primo mayor que 9 (11, 13, ...) y la respuesta es -1. Casos borde: $N < 2$ se responde con el propio N.

C++\Notebook\Factorizacion en Digos.cpp

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 string recursive(long long num, string sol = "")
6 {
7     for (int i = 9; i > 1; --i)
8     {
9         if (num % i == 0)
10         {
11             return recursive(num / i, sol + to_string(i));
12         }
13     }
14     return sol;
15 }
16
17 int main()
18 {
19     ios::sync_with_stdio(false);
20     cin.tie(nullptr);
21
22     int t;
23     cin >> t;
24     while (t--)
25     {
26         long long n;
27         cin >> n;
28
29         if (n < 2)
30         {
31             cout << n << "\n";
32             continue;
33         }
34
35         string factors = recursive(n);
36
37         long long mult = 1;
38         for (char ch : factors)
39         {
40             mult *= (ch - '0');
41         }
42
43         if (mult == n)
44         {
45             sort(factors.begin(), factors.end());
46             cout << factors << "\n";
47         }
48         else
49         {
50             cout << -1 << "\n";
51         }
52     }
53
54     return 0;
55 }
```

6.12 GCD and LCM - Journeygrapes

Calcula a dividido entre mcd(a,b), que es el multiplicador para obtener lcm(a,b) desde b.

C++\Codeforces\journeygrapes.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     long long a, b;
6     cin >> a >> b;
7     //c++17
8     cout << a / gcd(a, b) << endl;
9
10    return 0;
11 }
```

6.13 Modular Arithmetic - Broul

Calcula el número mínimo de saltos de tamaño $3*b$ necesarios para cubrir la distancia vertical $abs(a-c)$, o determina si no es posible porque la distancia no es múltiplo de b.

C++\CCPL\broul.cpp

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main() {
6     while(true){
7         int a,b,c;
8         cin >> a >> b >> c;
9         if(a==0 && b ==0 && c ==0)
10         {
11             return 0;
12         }
13
14         int god = abs(a-c);
15         if(god%b!=0)
16         {
17             cout << "No accounting tablet" << "\n";
18         }
19         else
20         {
21             if((god/b) % 3 ==0)
22             {
23                 cout << (god/b)/3 << "\n";
24             }
25             else
26             {
27                 cout << ((god/b)/3)+1 << "\n";
28             }
29         }
30     }
31
32     return 0;
33 }
```

6.14 Modular Arithmetic - Creating Multiples

Resuelve "Creating Multiples" (Day 4, problema B - Contest 04 [Avanzados]): dado un numero N de L digitos en base B, hay que reducir (nunca aumentar) como mucho un digito para que el numero resultante M sea multiplo de B+1, eligiendo la solucion que minimice M. Usa que B es congruente con -1 modulo B+1, por lo que el resto de N modulo B+1 es la suma alternada de sus digitos (variable rem); recorre los digitos de izquierda a derecha calculando cuanto hay que restarle a cada uno (delta) para anular ese resto, y toma la primera posicion valida, que por ser la mas significativa da el M mas chico posible.

C++\Training_Camp_2026\Day 4\B - Creating Multiples.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int B, L;
9     cin >> B >> L;
10
11     vector<int> D(L);
12     for (int &x : D) cin >> x;
13
14     int mod = B + 1;
15
16     // Calculate N modulo (B + 1).
17     // B ≡ -1 (mod B+1), so powers alternate.
18     int rem = 0;
19     for (int i = 0; i < L; i++) {
20         if ((L - 1 - i) % 2 == 0)
21             rem += D[i];
22         else
23             rem -= D[i];
24         rem %= mod;
25     }
26     if (rem < 0) rem += mod;
27
28     if (rem == 0) {
29         cout << "0 0\n";
30         return 0;
31     }
32
33     // Try positions from left to right. The first possible position
34     // gives the smallest resulting number.
35     for (int i = 0; i < L; i++) {
36         int sign = ((L - 1 - i) % 2 == 0) ? 1 : -1;
37
38         // Need a decrease delta such that:
39         // rem - delta * sign ≡ 0 (mod B+1)
40         int delta;
41         if (sign == 1)
42             delta = rem;
43         else
44             delta = (mod - rem) % mod;
45
46         // Digits are smaller than B+1, so this is the only possible delta.
47         if (delta <= D[i] && delta > 0) {
48             cout << i + 1 << ' ' << D[i] - delta << '\n';
49             return 0;
50         }
51     }
52
53     cout << "-1 -1\n";
54     return 0;
55 }
```

6.15 Modulo de Entero Gigante

O: (largo de la cadena)

Uso: modString(P,N) con P el numero como string y N el modulo

Cuando el numero de entrada tiene miles de digitos no cabe en long long, asi que se lee como string y se calcula el resto digito por digito con la regla de Horner: se arrastra el resto parcial y en cada paso se hace $rem = (rem * 10 + \text{digito}) \% N$. Como rem siempre es menor que N, el producto $rem * 10$ nunca desborda mientras N quepa comodo en long long. Sirve para problemas de divisibilidad con numeros gigantes, o como paso previo antes de aplicar exponenciacion modular sobre un exponente enorme.

C++\Notebook\Modulo de Entero Gigante.cpp

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 long long modString(const string& P, long long N)
6 {
7     long long rem = 0;
8     for (char c : P)
9     {
10         int d = c - '0';
11         rem = (rem * 10 + d) % N;
12     }
13     return rem;
14 }
15
16 int main()
17 {
18     ios_base::sync_with_stdio(false);
19     cin.tie(nullptr);
20
21     string P;
22     long long N;
23     while (cin >> P >> N)
24     {
25         cout << modString(P, N) << "\n";
26     }
27
28     return 0;
29 }
```

6.16 Modulo de un Binario Gigante - Binary Dozens

Resuelve "Binary Dozens" (problema B, ICPC 2025): dado un numero en binario de hasta 500 bits, dar su resto al dividir por 12. 500 bits son numeros de hasta 10^{150} , asi que no hay tipo entero que los guarde. Pero el resto si se puede calcular leyendo los digitos de izquierda a derecha con el metodo de Horner: si el numero visto hasta ahora vale v, agregarle un bit b lo convierte en $2v + b$, y tomando modulo en cada paso el acumulador nunca pasa de 24. $rem = (rem * 2 + \text{bit}) \% 12$ Funciona porque el modulo es compatible con sumas y productos: se puede reducir en cada paso en vez de al final. La misma idea sirve para cualquier base y cualquier modulo, no solo binario y 12, y es la forma estandar de sacar el resto de un numero que llega como cadena. Que 12 no sea primo da igual: esto no usa inversos ni nada por el estilo. Verificado contra los enteros exactos de Python en 3004 cadenas, incluidas las de 500 bits todos en 1 y todos en 0. El archivo B - Binary Dozens.py de esta carpeta hace lo mismo aprovechando que Python tiene enteros de precision arbitraria: $\text{int}(b, 2) \% 12$. Vale tenerlo presente cuando el problema sea justamente de numeros gigantes y el lenguaje este permitido.

C++\ICPC\2025\B - Binary Dozens.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     string b;
6
7     while (cin >> b && b != "") {
8
9         int rem = 0;
10
11         for (char c : b) {
12             rem = (rem * 2 + (c - '0')) % 12;
13         }
14 }
```

6.16 Modulo de un Binario Gigante - Binary Dozens (cont.)

```

5         cout << rem << '\n';
6     }
7
8     return 0;
9 }

```

6.17 Modulo de un Binario Gigante (Python) - Binary Dozens

La version en Python del problema B de ICPC 2025 ("Binary Dozens"): el resto entre 12 de un numero binario de hasta 500 bits. En Python son dos lineas porque los enteros son de precision arbitraria: `int(b, 2)` convierte la cadena binaria completa (aunque mida 10^{150}) y despues se toma el modulo. No hay desborde posible. CUANDO ELEGIR PYTHON EN UN CONTEST: justo en esto, cuando el problema pide aritmetica con numeros enormes. Lo que en C++ obliga a Horner con modulo, a `_int128` o a una clase de bignum, en Python es el operador de siempre. Tambien sirve para factoriales grandes, potencias exactas y combinatoria sin modulo. El precio es la velocidad: si ademas hay que hacer muchas operaciones, C++ con el modulo aplicado paso a paso gana por lejos. La version en C++ esta en "B - Binary Dozens.cpp", en esta misma carpeta, y usa `rem = (rem * 2 + bit) % 12`. Verificado contra los enteros exactos en 3004 cadenas, incluidas las de 500 bits.

C++\ICPC\2025\B - Binary Dozens.py

```

1 while True:
2     b = input()
3
4     if b == "":
5         break
6
7     n = int(b, 2)
8     print(n % 12)

```

6.18 Number Theory (Criba y Factorizacion) - Counting Divisors

Resuelve "Counting Divisors" (Day 10, problema I - Contest 09 [Avanzados]): dados n enteros, imprime la cantidad de divisores de cada uno. Precomputa con una criba de Eratosthenes (sieve) la lista de primos hasta $MX=1e6$; para cada consulta k , la funcion `divisors` factoriza k probando solo los primos precomputados (hasta $\sqrt{k}$) y acumula el conteo de divisores con la formula producto de $(exponente_i + 1)$ para cada primo, multiplicando por 2 al final si queda un factor primo residual mayor a 1 (el propio n si es primo grande).

C++\Training_Camp_2026\Day 10\I - Counting Divisors.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const int MX = 1000000;
7
8 bool marked[MX + 1];
9 vector<int> primes;
10
11 void sieve() {
12     marked[0] = marked[1] = true;
13
14     for (int i = 2; i <= MX; i++) {
15         if (marked[i]) continue;
16
17         primes.push_back(i);
18
19         if (1LL * i * i > MX) continue;
20         for (ll j = 1LL * i * i; j <= MX; j += i)
21             marked[j] = true;
22     }
23 }
24
25 ll divisors(ll n) {
26     ll ans = 1;
27
28     for (ll p : primes) {
29         if (p * p > n) break;
30
31         if (n % p == 0) {
32             int cnt = 0;
33             while (n % p == 0) {

```

6.18 Number Theory (Criba y Factorizacion) - Counting Divisors (cont.)

```

34         n /= p;
35         cnt++;
36     }
37     ans *= (cnt + 1);
38 }
39
40 if (n > 1) ans *= 2;
41
42 return ans;
43 }
44
45 int main() {
46     ios::sync_with_stdio(false);
47     cin.tie(nullptr);
48
49     sieve();
50
51     int t;
52     cin >> t;
53
54     while (t--) {
55         ll k;
56         cin >> k;
57         cout << divisors(k) << '\n';
58     }
59
60     return 0;
61 }

```

6.19 Number Theory (GCD y Divisores) - The Seals Of Shanghai

Resuelve "The Seals of Shanghai" (Day 10, problema B - Contest 09 [Avanzados]): eligiendo un modulo fijo $M > 1$, cada sello con numero A_i se desbloquea en la ronda de su residuo $A_i \bmod M$; hay que hallar el minimo numero de residuos distintos (rondas) necesarias y cuantos M distintos logran ese minimo. Usa que dos elementos caen en el mismo residuo sii M divide su diferencia: calcula $D = \gcd$ de todas las diferencias contra $A[0]$; si $D=0$ el costo minimo es 1 para todo M ($MAXM-1$ valores); si $D \geq 2$ el costo minimo es 1 y la cuenta es el numero de divisores de D mayores a 1 (`countDivisorsGt1`); si $D=1$ el minimo es 2, y por pigeonhole sobre un triple de valores distintos, todo M valido debe dividir alguna de sus 3 diferencias, por lo que basta probar los divisores de esas 3 diferencias (candidates) y, para cada uno, contar residuos distintos con una tabla hash reusable.

C++\Training_Camp_2026\Day 10\B - The Seals of Shanghai.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4 typedef unsigned long long ull;
5
6 static char buf_[1 << 22];
7 static int buflen_, bufPos_;
8 inline int gc() {
9     if (bufPos_ == buflen_) { buflen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!buflen_) return -1; }
10    return buf_[bufPos_++];
11 }
12 inline ll readLL() {
13     int c = gc();
14     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
15     bool neg = false; if (c == '-') { neg = true; c = gc(); }
16     ll x = 0;
17     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
18     return neg ? -x : x;
19 }
20
21 const ll MAXM = 1000000000LL;
22
23 ll gcdll(ll a, ll b) { while (b) { a %= b; swap(a, b); } return a; }
24
25 // Counts divisors of d that are > 1.
26 ll countDivisorsGt1(ll d) {
27     if (d <= 1) return 0;

```

6.19 Number Theory (GCD y Divisores) - The Seals Of Shanghai (cont.)

```

28     ll cnt = 0;
29     for (ll i = 1; i * i <= d; i++) {
30         if (d % i == 0) {
31             if (i > 1) cnt++;
32             ll other = d / i;
33             if (other != i && other > 1) cnt++;
34         }
35     }
36     return cnt;
37 }
38
39 void collectDivisorsGt1(ll d, vector<ll> &out) {
40     if (d <= 1) return;
41     for (ll i = 1; i * i <= d; i++) {
42         if (d % i == 0) {
43             if (i > 1) out.push_back(i);
44             ll other = d / i;
45             if (other != i && other > 1) out.push_back(other);
46         }
47     }
48 }
49
50 int main() {
51     int N = (int)readLL();
52     vector<ll> A(N);
53     for (auto &x : A) x = readLL();
54
55     if (N == 1) {
56         printf("1 %lld\n", MAXM - 1);
57         return 0;
58     }
59
60     ll D = 0;
61     for (int i = 1; i < N; i++) D = gcdll(D, llabs(A[i] - A[0]));
62
63     if (D == 0) {
64         printf("1 %lld\n", MAXM - 1);
65         return 0;
66     }
67     if (D >= 2) {
68         printf("1 %lld\n", countDivisorsGt1(D));
69         return 0;
70     }
71
72     // D == 1: minimum is provably exactly 2 (M=2 always splits into exactly the two parity
73     // classes, and D=1 rules out cost=1 for any M>1). We still need to COUNT every M that
74     // achieves cost=2. Key fact used for counting: for ANY 3 elements, since there are only
75     // 2 residue classes under an optimal M, by pigeonhole at least 2 of the 3 must share a
76     // class, so M must divide at least one of their 3 pairwise differences. This holds for
77     // every valid M and every triple, so scanning divisors of one arbitrary triple's 3
78     // differences is guaranteed to contain the full set of valid M's.
79     vector<ll> distinctVals;
80     {
81         vector<ll> tmp = A;
82         sort(tmp.begin(), tmp.end());
83         tmp.erase(unique(tmp.begin(), tmp.end(), tmp.end()), tmp.end());
84         distinctVals = tmp;
85     }
86
87     if ((int)distinctVals.size() < 3) {
88         // Only 2 distinct values overall, differing by exactly 1 (since D=1): every M>1
89         // fails to merge them, so cost=2 for every single M from 2..MAXM.
90         printf("2 %lld\n", MAXM - 1);
91         return 0;
92     }
93
94     ll a = distinctVals[0], b = distinctVals[1], c = distinctVals[2];
95     ll d1 = llabs(a - b), d2 = llabs(b - c), d3 = llabs(a - c);
96
97     vector<ll> candidates;
98     collectDivisorsGt1(d1, candidates);
99     collectDivisorsGt1(d2, candidates);

```

6.19 Number Theory (GCD y Divisores) - The Seals Of Shanghai (cont.)

```

00     collectDivisorsGt1(d3, candidates);
01     sort(candidates.begin(), candidates.end());
02     candidates.erase(unique(candidates.begin(), candidates.end(), candidates.end()));
03
04     // Lightweight open-addressing hash set, sized by N (not by M), reused across all
05     // candidates via a version stamp. This keeps memory at O(N) regardless of how large
06     // M gets (unlike indexing directly by residue, which would need an array of size M,
07     // and M can be up to just under 1e9 here).
08     int tableSize = 1;
09     while (tableSize < 4 * N + 16) tableSize <= 1;
10     int mask = tableSize - 1;
11     vector<ll> tableKey(tableSize);
12     vector<int> tableStamp(tableSize, -1);
13
14     int bestCost = INT_MAX;
15     ll bestCount = 0;
16
17     for (int idx = 0; idx < (int)candidates.size(); idx++) {
18         ll M = candidates[idx];
19         int distinctCount = 0;
20         for (ll x : A) {
21             ll r = x % M;
22             ull h = (ull)r * 0x9E3779B97F4A7C15ULL;
23             int pos = (int)(h >> 40) & mask;
24             while (tableStamp[pos] == idx && tableKey[pos] != r) {
25                 pos = (pos + 1) & mask;
26             }
27             if (tableStamp[pos] != idx) {
28                 tableStamp[pos] = idx;
29                 tableKey[pos] = r;
30                 distinctCount++;
31             }
32         }
33         if (distinctCount < bestCost) {
34             bestCost = distinctCount;
35             bestCount = 1;
36         } else if (distinctCount == bestCost) {
37             bestCount++;
38         }
39     }
40
41     printf("%d %lld\n", bestCost, bestCount);
42     return 0;
43 }

```

6.20 Number Theory (LCM via Factorizacion de Ciclos) - Permutation Rounds

Resuelve "Permutation Rounds" (Day 10, problema L - Contest 09 [Avanzados]): partiendo del arreglo ordenado [1..n], en cada ronda todo elemento en la posición i se mueve a la posición p_i según una permutación fija; hay que hallar tras cuántas rondas el arreglo vuelve a estar ordenado por primera vez, módulo $1e9+7$. La respuesta es el LCM de las longitudes de todos los ciclos de la permutación vista como grafo funcional. El código recorre cada posición no visitada siguiendo $p[i]$ para medir la longitud de su ciclo (length), factoriza esa longitud en primos y actualiza $\text{maxPower}[\text{primo}]$ con el mayor exponente visto entre todos los ciclos (equivalente a construir el LCM por factorización en vez de multiplicar longitudes directamente, evitando overflow); al final multiplica $\text{primo}^{\text{maxPower mod MOD}}$ para cada primo usando exponenciación modular (power_mod).

C++\Training_Camp_2026\Day 10\L - Permutation Rounds.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 static char buf_[1 << 22];
6 static int bufLen_, bufPos_;
7 inline int gc() {
8     if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!bufLen_) return -1; }
9     return buf_[bufPos_++];
10 }
11 inline int readInt() {
12     int c = gc();
13     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();

```

6.20 Number Theory (LCM via Factorizacion de Ciclos) - Permutation Rounds (cont.)

```

14 bool neg = false; if (c == '-') { neg = true; c = gc(); }
15 int x = 0;
16 while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
17 return neg ? -x : x;
18 }
19
20 const ll MOD = 1000000007LL;
21
22 ll power_mod(ll base, ll exp, ll mod) {
23     base %= mod;
24     ll result = 1;
25     while (exp > 0) {
26         if (exp & 1) result = (result * base) % mod;
27         base = (base * base) % mod;
28         exp >>= 1;
29     }
30     return result;
31 }
32
33 int main() {
34     int n = readInt();
35     vector<int> p(n + 1);
36     for (int i = 1; i <= n; i++) p[i] = readInt();
37
38     vector<char> visited(n + 1, 0);
39     unordered_map<int, int> maxPower; // prime -> max exponent seen across all cycle lengths
40
41     for (int i = 1; i <= n; i++) {
42         if (visited[i]) continue;
43         int length = 0;
44         int j = i;
45         while (!visited[j]) {
46             visited[j] = 1;
47             j = p[j];
48             length++;
49         }
50         // factorize `length`, updating maxPower
51         int x = length;
52         for (int d = 2; (ll)d * d <= x; d++) {
53             if (x % d == 0) {
54                 int exp = 0;
55                 while (x % d == 0) { x /= d; exp++; }
56                 auto it = maxPower.find(d);
57                 if (it == maxPower.end() || it->second < exp) maxPower[d] = exp;
58             }
59         }
60         if (x > 1) {
61             auto it = maxPower.find(x);
62             if (it == maxPower.end() || it->second < 1) maxPower[x] = 1;
63         }
64     }
65
66     ll result = 1;
67     for (auto &pr : maxPower) {
68         result = (result * power_mod(pr.first, pr.second, MOD)) % MOD;
69     }
70
71     printf("%lld\n", result);
72     return 0;
73 }

```

6.21 Reconstruccion con Divisores - Remainders

Reconstruye el multiconjunto oculto K a partir de la sucesion $S(n)$ = suma de $(n \bmod k)$ para todo k en K , dados $S(1)..S(N)$. Se apoya en dos observaciones: como todo k es al menos 2, $1 \bmod k = 1$ y por tanto $S(1)$ es directamente el tamaño $|K|$; y al mirar la diferencia $S(n) - S(n-1)$, cada k que NO divide a n aporta +1 mientras que cada k que sí divide a n aporta $-(k-1)$, de modo que $D = |K| - (S(n) - S(n-1))$ termina siendo exactamente la suma de los elementos de K que dividen a n . Con eso se procesa n de menor a mayor: se le restan a D los aportes $k * \text{cnt}[k]$ de los divisores propios ya conocidos, y lo que sobra es $\text{cnt}[n] * n$, de donde sale la multiplicidad de n en K . Al final se expande el conteo en la lista ordenada de elementos.

C++\RPC\Remainders.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int N;
6     if (!(cin >> N)) return 0;
7
8     vector<long long> S(N + 1, 0); // S[0] = 0
9     for (int i = 1; i <= N; i++) cin >> S[i];
10
11     long long m = S[1]; // |K| = S(1)
12     vector<long long> cnt(N + 1, 0);
13
14     for (int n = 2; n <= N; n++) {
15         long long D = m - (S[n] - S[n - 1]); // suma de los k de K que dividen a n
16         for (int k = 2; k < n; k++)
17             if (n % k == 0) D -= (long long)k * cnt[k];
18         if (D > 0) cnt[n] = D / n; // lo que queda es cnt[n]*n
19     }
20
21     vector<int> K;
22     for (int k = 2; k <= N; k++)
23         for (int i = 0; i < cnt[k]; i++) K.push_back(k);
24
25     cout << K.size();
26     for (int x : K) cout << ' ' << x;
27     cout << '\n';
28     return 0;
29 }

```

6.22 Sieve of Eratosthenes - Prime Sieve

Resuelve "Prime Sieve" (Day 11, problema E - CCPL (Kattis)): dado un límite n (hasta 10^8) y q consultas x , pide primero cuantos números primos hay menores o iguales a n , y luego para cada consulta si x es primo (1) o compuesto (0). Implementa una criba de Eratosthenes clásica sobre el vector `marked` (marca compuestos empezando en i^2 para evitar overflow con $1LL * i^2$), contando los primos menores o iguales al límite pedido en la variable `cont` mientras criba, y responde cada consulta en $O(1)$ revisando `marked[aa]`.

C++\Training_Camp_2026\Day 11 - CCPL\E - Prime Sieve.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6
7
8 const int MX = 100000000;
9
10
11
12 vector<bool> marked(MX + 1, false);
13 int cont = 0;
14
15
16 void sieve(int uwu) {
17     marked[0] = marked[1] = true;
18
19     for (int i = 2; i <= MX; i++) {
20         if (marked[i]) continue;
21

```

6.22 Sieve of Eratosthenes - Prime Sieve (cont.)

```

22     if(i <= uwu)
23     {
24         cont++;
25     }
26     if (1LL * i * i > MX) continue;
27     for (ll j = 1LL * i * i; j <= MX; j += i)
28         marked[j] = true;
29 }
30 }
31
32 int main() {
33     ios::sync_with_stdio(false);
34     cin.tie(nullptr);
35
36     int t;
37     cin >> t;
38
39     sieve(t);
40
41     int n; cin >> n;
42     cout << cont << endl;
43     while (n--) {
44         int aa; cin >> aa;
45
46         if(marked[aa]==false)
47         {
48             cout << 1 << endl;
49         }
50         else
51         {
52             cout << 0 << endl;
53         }
54     }
55
56     return 0;
57 }

```

6.23 Ternas Pitagoricas - Pythagorean

Dado n , cuenta en cuantas ternas pitagoricas participa, separando cuatro casos: n como hipotenusa (primitivas y no primitivas) y n como cateto (primitivas y no primitivas). Para n como hipotenusa recorre cada cateto a y verifica si $n^2 - a^2$ es cuadrado perfecto, ajustando la raiz con `sqrtl` mas dos while para corregir el error de punto flotante (nunca confiar en `sqrt` directo con enteros grandes); exige $b > a$ para no contar la misma terna dos veces, y `gcd(a,b) == 1` la marca como primitiva. Para n como cateto usa la factorizacion $(c-b)(c+b) = n^2$: cada divisor $d < n$ de n^2 da $e = n^2/d$, y si d y e tienen la misma paridad se recupera $b = (e-d)/2$ y $c = (e+d)/2$.

C++\RPC\Pythagorean.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  typedef long long ll;
4
5  int main() {
6      ll n;
7      if (!(cin >> n)) return 0;
8
9      ll hypPrim = 0, hypNon = 0, legPrim = 0, legNon = 0;
10     ll n2 = n * n;
11
12     // --- n como hipotenusa ---
13     for (ll a = 1; a < n; a++) {
14         ll t = n2 - a * a;
15         ll b = (ll)sqrtl((long double)t);
16         while (b * b < t) b++;
17         while (b * b > t) b--;
18         if (b * b == t && b > a) { // b > a evita contar dos veces
19             if (__gcd(a, b) == 1) hypPrim++;
20             else hypNon++;
21         }
22     }
23
24     // --- n como cateto: (c-b)(c+b) = n^2 ---

```

6.23 Ternas Pitagoricas - Pythagorean (cont.)

```

25     for (ll d = 1; d < n; d++) { // d = c-b, d < e obliga d < n
26         if (n2 % d) continue;
27         ll e = n2 / d; // e = c+b
28         if (((e - d) & 1LL)) continue; // misma paridad
29         ll b = (e - d) / 2;
30         ll c = (e + d) / 2;
31         if (b <= 0) continue;
32         if (__gcd(n, b) == 1) legPrim++;
33         else legNon++;
34     }
35
36     cout << hypPrim << ' ' << hypNon << ' '
37         << legPrim << ' ' << legNon << '\n';
38     return 0;
39 }

```

7 - Combinatorics

7.1 Combinatoria (Factoriales Modulares) - Binomial Coefficients

Resuelve "Binomial Coefficients" (Day 10, problema J - Contest 09 [Avanzados]): calcula n coeficientes binomiales $C(a,b)$ modulo $1e9+7$. Precomputa $fact[i] = i! \bmod MOD$ para todo i hasta $MAXN=1e6$, y su inverso modular $invFact[i]$ de atras hacia adelante en $O(MAXN)$ (calculando solo una exponenciacion modular $invFact[MAXN] = fact[MAXN]^{(MOD-2)}$ via Fermat, y derivando el resto con $invFact[i-1] = invFact[i]*i$). Cada consulta responde en $O(1)$ con la formula $fact[a] * invFact[b] * invFact[a-b] \bmod MOD$.

C++\Training_Camp_2026\Day 10J - Binomial Coefficients.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5
6  const int MOD = 1000000007;
7  const int MAXN = 1000000;
8
9  vector<ll> fact(MAXN + 1), invFact(MAXN + 1);
10
11 ll modPow(ll a, ll e) {
12     ll res = 1;
13     while (e > 0) {
14         if (e & 1) res = res * a % MOD;
15         a = a * a % MOD;
16         e >>= 1;
17     }
18     return res;
19 }
20
21 int main() {
22     ios::sync_with_stdio(false);
23     cin.tie(nullptr);
24
25     fact[0] = 1;
26     for (int i = 1; i <= MAXN; i++)
27         fact[i] = fact[i - 1] * i % MOD;
28
29     invFact[MAXN] = modPow(fact[MAXN], MOD - 2);
30
31     for (int i = MAXN; i >= 1; i--)
32         invFact[i - 1] = invFact[i] * i % MOD;
33
34     int n;
35     cin >> n;
36
37     while (n--) {
38         int a, b;
39         cin >> a >> b;
40
41         ll ans = fact[a];
42         ans = ans * invFact[b] % MOD;
43         ans = ans * invFact[a - b] % MOD;
44     }

```

7.1 Combinatoria (Factoriales Modulares) - Binomial Coefficients (cont.)

```

45     cout << ans << '\n';
46 }
47
48 return 0;
49 }

```

7.2 Combinatoria Avanzada (Lucas, Catalan, Stirling)

O: (1) por C tras precomputar(); Lucas $O(\log_p n)$

Uso: precomputar() una vez; luego C(n,k), multinomial(v), catalan(n)

Formulas combinatorias que van mas alla del C(n,k) basico con factoriales precomputados (ese caso ya esta resuelto en "Binomial Coefficients"). Aqui van cuatro herramientas: Lucas, para cuando n y k son gigantes (hasta $1e18$) pero el modulo es un primo pequeno, descomponiendo n y k en base p y multiplicando los binomiales de cada dígito; multinomial, para repartir n objetos en grupos de tamanos distintos; Catalan, que cuenta estructuras balanceadas (parentesis validos, arboles binarios, caminos de Dyck); y Stirling de segunda especie, que cuenta en cuantas formas se parten n elementos en k grupos no vacios. Las tres primeras necesitan que el modulo sea primo (usan el inverso via Fermat).

C++\Notebook\Combinatoria Avanzada.cpp

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  using ll = long long;
6
7  ll modpow(ll a, ll e, ll m)
8  {
9      ll r = 1 % m;
10     a %= m;
11     while (e)
12     {
13         if (e & 1) r = r * a % m;
14         a = a * a % m;
15         e >>= 1;
16     }
17     return r;
18 }
19
20 // C(n,k) mod p directo en  $O(k)$ , para n y k chicos (uso interno de Lucas)
21 ll Csmall(ll n, ll k, ll p)
22 {
23     if (k < 0 || k > n) return 0;
24     ll num = 1, den = 1;
25     for (ll i = 1; i <= k; i++)
26     {
27         num = num * ((n - k + i) % p) % p;
28         den = den * i % p;
29     }
30     return num * modpow(den, p - 2, p) % p;
31 }
32
33 // Teorema de Lucas: C(n,k) mod p con n, k enormes y p primo pequeno
34 ll C_lucas(ll n, ll k, ll p)
35 {
36     ll res = 1;
37     while (n || k)
38     {
39         ll ni = n % p, ki = k % p;
40         res = res * Csmall(ni, ki, p) % p;
41         n /= p;
42         k /= p;
43         if (!res) return 0;
44     }
45     return res;
46 }
47
48 const int MOD = 1e9 + 7;
49 const int MAXN = 1000000;
50 vector<ll> fact(MAXN + 1), invFact(MAXN + 1);
51
52 void precomputar()

```

7.2 Combinatoria Avanzada (Lucas, Catalan, Stirling) (cont.)

```

53 {
54     fact[0] = 1;
55     for (int i = 1; i <= MAXN; i++) fact[i] = fact[i - 1] * i % MOD;
56     invFact[MAXN] = modpow(fact[MAXN], MOD - 2, MOD);
57     for (int i = MAXN; i >= 1; i--) invFact[i - 1] = invFact[i] * i % MOD;
58 }
59
60 ll C(int n, int k)
61 {
62     if (k < 0 || k > n) return 0;
63     return fact[n] * invFact[k] % MOD * invFact[n - k] % MOD;
64 }
65
66 // n! / (a1! * a2! * ... * ak!) : repartir n objetos en grupos de tamanos dados
67 ll multinomial(const vector<int>& a)
68 {
69     ll n = 0;
70     for (int x : a) n += x;
71     ll r = fact[n];
72     for (int x : a) r = r * invFact[x] % MOD;
73     return r;
74 }
75
76 // Catalan(n) = C(2n,n) / (n+1) : parentesis balanceados, arboles binarios, caminos de Dyck
77 ll catalan(int n)
78 {
79     return C(2 * n, n) * modpow(n + 1, MOD - 2, MOD) % MOD;
80 }
81
82 // Stirling de segunda especie: partir n elementos en k grupos no vacios
83 vector<vector<ll>> stirling2(int n, int k)
84 {
85     vector<vector<ll>> S(n + 1, vector<ll>(k + 1, 0));
86     S[0][0] = 1;
87     for (int i = 1; i <= n; i++)
88     {
89         for (int j = 1; j <= k; j++)
90         {
91             S[i][j] = (S[i - 1][j - 1] + (ll)j * S[i - 1][j]) % MOD;
92         }
93     }
94     return S;
95 }
96
97 int main()
98 {
99     ios_base::sync_with_stdio(false);
100    cin.tie(nullptr);
101    return 0;
102 }

```

7.3 Construction - Farmpiggy And Subset Sum

Problema de construccion (el nombre sugiere relacion con subconjuntos de suma prohibida): arma una permutacion de 1..n intercambiando cada pareja de consecutivos ($p[1]=2, p[2]=1, p[3]=4, p[4]=3, \dots$), de forma que $p[i] \neq i$ para todo i (derangement simple) y cada indice queda emparejado con su vecino inmediato.

C++\Codeforces\Farmpiggy and Subset Sum.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios_base::sync_with_stdio(false);
6      cin.tie(NULL);
7
8      int t;
9      cin >> t;
10     while (t--) {
11         int n;
12         cin >> n;
13
14         vector<int> p(n + 1);

```

7.3 Construction - Farmppiggie And Subset Sum (cont.)

```

5      for (int i = 1; i <= n; i += 2) {
6          p[i] = i + 1;
7          p[i + 1] = i;
8      }
9
10     for (int i = 1; i <= n; i++) {
11         cout << p[i];
12         if (i != n) cout << ' ';
13     }
14     cout << "\n";
15 }
16
17 return 0;
18 }

```

7.4 Conteo de Distintos con Inventario de Frecuencias - Total Numbers

Resuelve "Unique 3-Digit Even Numbers" (LeetCode 3483): dado un arreglo de dígitos, contar cuantos numeros DISTINTOS de tres cifras se pueden armar usando tres de ellos, sin cero inicial y que el resultado sea par. La trampa es "distintos". Si uno cuenta permutaciones de posiciones, [6,6,6] daría 6 en vez de 1, porque los tres seises son intercambiables. El arreglo cnt da la vuelta al problema: en vez de recorrer POSICIONES del arreglo, recorre VALORES. Cada terna (first, second, last) se visita exactamente una vez, así que cada numero se cuenta una sola vez por construcción, sin sets ni deduplicación posterior. Lo que hace que sea correcto y no solo "distinto" es el préstamo: cnt[last]-- antes de abrir el ciclo de first, y cnt[first]-- antes del de second. Eso reserva la copia física del dígito que ya se uso, para que cnt[second] > 0 signifique "queda todavía un ejemplar libre". Sin ese préstamo, [2,0,0] contaría 220 aunque solo haya un dos. Cada decremento se devuelve al salir del ciclo, así que cnt vuelve intacto en cada vuelta. El ultimo dígito manda la paridad, por eso se fija primero y solo con [0,2,4,6,8]; first va de 1 a 9 porque no puede haber cero inicial; second es libre de 0 a 9. Costo $O(n + 5 \cdot 9 \cdot 10) = O(n)$ con 450 iteraciones fijas, sin memoria extra mas alla de cnt[10]. El patron sirve para cualquier "cuantos resultados distintos salen de un multiconjunto": iterar sobre los valores posibles y usar el arreglo de frecuencias como inventario que se presta y se devuelve.

C++\LeetCode\totalNumbers.cpp

```

1  class Solution {
2  public:
3      int totalNumbers(vector<int>& digits) {
4          int cnt[10] = {};
5
6          for (int x : digits) {
7              cnt[x]++;
8          }
9
10         int ans = 0;
11
12         for (int last : {0, 2, 4, 6, 8}) {
13             if (cnt[last] == 0) continue;
14
15             cnt[last]--;
16
17             for (int first = 1; first <= 9; first++) {
18                 if (cnt[first] == 0) continue;
19
20                 cnt[first]--;
21
22                 for (int second = 0; second <= 9; second++) {
23                     if (cnt[second] > 0) ans++;
24                 }
25
26                 cnt[first]++;
27             }
28
29             cnt[last]++;
30         }
31
32         return ans;
33     }
34 };

```

7.5 Conteo de Inversiones (cadena repetida) - Inversions

Cuenta las inversiones de S repetida N veces (N hasta $1e12$) modulo $1e9+7$, es decir pares $i < j$ donde la letra de i va despues en el alfabeto que la de j. No se puede construir la cadena, así que se cuenta la contribucion de cada posición por separado. Una posición i con letra c aporta dos cosas: dentro de su propia copia aporta suf = cuantas letras menores que c hay de i en adelante, y eso ocurre igual en las N copias ($N \cdot \text{suf}$); contra las copias posteriores le sirven TODAS las letras menores que c de la cadena completa ($\text{freqGod}[c]$), una vez por cada par ordenado de copias, o sea $N(N-1)/2$. $\text{freqAtras}[i]$ guarda cuantas letras menores que cada letra quedan desde i hasta el final, y freqGod acumula las frecuencias globales. La division entre 2 va con inverso modular (Fermat, $\text{modpow}(b, \text{MOD}-2)$) porque $N(N-1)/2$ ya se calcula en modulo. Nota: el if de $\text{freqGod} == \text{freqAtras}$ es redundante, las dos ramas dan el mismo valor (cuando $\text{suf} == \text{freqGod}[c]$, $N(N+1)/2 \cdot \text{freqGod} == N(N-1)/2 \cdot \text{freqGod} + N \cdot \text{suf}$).

C++\Codeforces\Inversions.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  long long MOD = 1000000007;
5
6  long long modpow(long long a, long long e) {
7      long long ans = 1;
8
9      while (e) {
10         if (e & 1)
11             ans = ans * a % MOD;
12
13         a = a * a % MOD;
14         e >>= 1;
15     }
16
17     return ans;
18 }
19
20 long long division_mod(long long a, long long b) {
21     return a % MOD * modpow(b, MOD - 2) % MOD;
22 }
23
24
25
26 int main()
27 {
28     string s; cin >> s;
29     long long n; cin >> n;
30
31     vector<vector<long long>> freqAtras;
32     vector<long long> freq(26,0);
33     vector<long long> vectorsito(26,0);
34
35     for(long long i = s.size()-1; i >=0; i--)
36     {
37         char letra = s[i];
38         freq[letra-'0'-49]++;
39         for(long long j = 0; j < 26; j++)
40         {
41             if(letra-'0'-49 < j)
42             {
43                 vectorsito[j]++;
44             }
45         }
46         freqAtras.push_back(vectorsito);
47     }
48
49     vector<long long> freqGod(26,0);
50     long long sum = 0;
51     for(long long i =1; i <26; i++)
52     {
53         sum += freq[i-1];
54         freqGod[i] = sum;
55     }
56
57

```

7.5 Cuento de Inversiones (cadena repetida) - Inversions (cont.)

```

58     long long GOD = 0;
59     for(long long i = 0 ; i < s.size();i++)
60     {
61         if(freqGod[s[i]-'0'-49] == freqAtras[s.size()-i-1][s[i]-'0'-49])
62         {
63             GOD = (GOD + division_mod((n % MOD) * ((n + 1) % MOD) % MOD , 2) * freqGod[s[i]-'0'-49
64             ] % MOD) % MOD;
65         }
66         else
67         {
68             GOD = (GOD + division_mod(((n-1) % MOD) * ((n) % MOD) % MOD , 2) * freqGod[s[i]-'0'-49
69             ] % MOD) % MOD;
70             GOD = (GOD + (freqAtras[s.size()-i-1][s[i]-'0'-49] * n) % MOD ) % MOD;
71         }
72     }
73     cout << GOD << endl;
74
75     return 0;
76 }

```

7.6 Euler's Formula - Pizza Cutter

Resuelve "Pizza Cutter" (Day 8, problema C - Contest 08 [Avanzados]): el abuelo Giuseppe corta una pizza rectangular con H cortes que entran por el lado izquierdo y salen por el derecho, y V cortes que entran por abajo y salen por arriba (cada corte es una curva continua, sin tres cortes concurrentes ni cortes tangentes); dado donde entra y sale cada corte, hay que contar en cuantas piezas queda dividida la pizza. Usa la formula de Euler para arreglos de curvas en el plano (piezas = $1 + \# \text{cortes} + \# \text{cruces}$): cada par de cortes de tipos opuestos siempre se cruza una vez (aporta $H \cdot V$ cruces), y los cruces entre cortes del mismo tipo se cuentan como inversiones (hh, vv) del orden de sus extremos derecho/superior via countInversions (merge sort), sumando todo en "answer".

C++\Training_Camp_2026\Day 8\C - Pizza Cutter.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  static char buf_[1 << 22];
5  static int bufLen_, bufPos_;
6  inline int gc() {
7      if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!bufL
8          en_) return -1; }
9      return buf_[bufPos_++];
10 }
11 inline long long readLL() {
12     int c = gc();
13     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
14     bool neg = false; if (c == '-') { neg = true; c = gc(); }
15     long long x = 0;
16     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
17     return neg ? -x : x;
18 }
19
20 // Counts inversions in `a` (pairs i<j with a[i] > a[j]) via merge sort. O(n log n).
21 long long countInversions(vector<long long>& a) {
22     int n = (int)a.size();
23     if (n <= 1) return 0;
24     vector<long long> buf(n);
25     long long inv = 0;
26     for (int width = 1; width < n; width *= 2) {
27         for (int i = 0; i < n; i += 2 * width) {
28             int mid = min(i + width, n);
29             int end = min(i + 2 * width, n);
30             int p = i, q = mid, k = i;
31             while (p < mid && q < end) {
32                 if (a[p] <= a[q]) buf[k++] = a[p++];
33                 else { buf[k++] = a[q++]; inv += (mid - p); }
34             }
35             while (p < mid) buf[k++] = a[p++];
36             while (q < end) buf[k++] = a[q++];

```

7.6 Euler's Formula - Pizza Cutter (cont.)

```

36         for (int t = i; t < end; t++) a[t] = buf[t];
37     }
38 }
39 return inv;
40 }
41
42 int main() {
43     // X, Y (pizza dimensions) are read for input-format purposes only; the piece
44     // count depends solely on H, V, and the relative order of cut endpoints.
45     (void)readLL(); (void)readLL();
46     long long H = readLL(), V = readLL();
47
48     vector<pair<long long,long long>> hcuts(H), vcuts(V);
49     for (auto& p : hcuts) { p.first = readLL(); p.second = readLL(); }
50     for (auto& p : vcuts) { p.first = readLL(); p.second = readLL(); }
51
52     sort(hcuts.begin(), hcuts.end());
53     vector<long long> y2seq;
54     y2seq.reserve(H);
55     for (auto& p : hcuts) y2seq.push_back(p.second);
56     long long hh = countInversions(y2seq);
57
58     sort(vcuts.begin(), vcuts.end());
59     vector<long long> x2seq;
60     x2seq.reserve(V);
61     for (auto& p : vcuts) x2seq.push_back(p.second);
62     long long vv = countInversions(x2seq);
63
64     long long answer = 1 + H + V + hh + vv + H * V;
65     printf("%lld\n", answer);
66     return 0;
67 }

```

7.7 Formula Cerrada con Inverso Modular - Number Of Sets

Resuelve "Number of Sets of K Non-Overlapping Line Segments" (LeetCode 1621): con n puntos alineados hay que contar de cuantas formas se dibujan k segmentos que no se solapan (si pueden compartir un extremo), modulo $1e9+7$. Toda la DP se colapsa en un solo binomial: la respuesta es $C(n+k-1, 2k)$. La idea es que elegir k segmentos equivale a escoger 2k extremos de entre n+k-1 posiciones, porque cada extremo compartido se "desdobra" agregando k-1 posiciones ficticias. Como hay que dividir en modular, se precomputan factoriales y sus inversos: una sola exponenciacion modular para inverso[limite] = factorial[limite]^(MOD-2) por Fermat, y el resto baja con inverso[i-1] = inverso[i] * i. Asi cada binomial sale en O(1) en vez de pagar un modpow por consulta. Costo total O(n + k + log MOD). OJO con el indice: si $2k > \text{limite}$, limite - 2k se sale del arreglo. No pasa dentro de las restricciones del problema ($k \leq n-1$ obliga a $2k \leq n+k-1$), pero si reusas este patron en otro problema, valida el rango antes.

C++\LeetCode\NumberOfSets.cpp

```

1  class Solution {
2  public:
3      long long MOD = 1000000007;
4
5      long long modpow(long long a, long long e) {
6          long long ans = 1;
7
8          while (e) {
9              if (e & 1)
10                 ans = ans * a % MOD;
11
12                 a = a * a % MOD;
13                 e >>= 1;
14             }
15
16             return ans;
17         }
18
19         long long division_mod(long long a, long long b) {
20             return a % MOD * modpow(b, MOD - 2) % MOD;
21         }
22
23         int numberOfSets(int n, int k) {

```

7.7 Formula Cerrada con Inverso Modular - Number Of Sets (cont.)

```

24 long long limite = n + k - 1;
25
26 vector<long long> factorial(limite + 1);
27 vector<long long> inverso(limite + 1);
28
29 factorial[0] = 1;
30
31 for (long long i = 1; i <= limite; i++) {
32     factorial[i] = factorial[i - 1] * i % MOD;
33 }
34
35 inverso[limite] = modpow(factorial[limite], MOD - 2);
36
37 for (long long i = limite; i >= 1; i--) {
38     inverso[i - 1] = inverso[i] * i % MOD;
39 }
40
41 long long GOD = factorial[limite];
42
43 GOD = GOD * inverso[2 * k] % MOD;
44 GOD = GOD * inverso[limite - 2 * k] % MOD;
45
46 return GOD;
47
48 // Lo mismo sin la variable limite, escribiendo C(n+k-1, 2k) completo:
49 //
50 // long long GOD = factorial[n + k - 1];
51 //
52 // GOD = GOD * inverso[2 * k] % MOD;
53 // GOD = GOD * inverso[n - k - 1] % MOD;
54 //
55 // return GOD;
56 //
57 // Es identico porque limite - 2k = (n + k - 1) - 2k = n - k - 1.
58 // Ayuda a ver que esto es literalmente C(arriba, abajo) con la forma
59 // factorial[arriba] * inverso[abajo] * inverso[arriba - abajo].
60 }
61 };

```

7.8 Paridad de Binomiales (Kummer) - Less Coin Tosses

Resuelve "Less Coin Tosses" (Day 1, problema L - Maratona de Programacao da SBC 2019): hay que repartir las 2^N cadenas binarias de longitud N entre Carla y Daniel de forma que la probabilidad de ganar sea igual sin importar el sesgo de la moneda, lo cual solo se logra emparejando equitativamente las cadenas dentro de cada nivel de peso de Hamming k (ya que P(s) depende solo de k), y pide el minimo numero de cadenas que quedan sin asignar. Por el teorema de Kummer, $C(N, k)$ es impar solo cuando k es submascara de los bits de N, y hay exactamente $2^{\text{popcount}(N)}$ de esos k; por eso basta calcular 2 elevado a la cantidad de bits en 1 de N.

C++\Training_Camp_2026\Day 1\L - Less_coin_tosses.py

```

1 import math
2 n = int(input())
3 m = bin(n)[2:]
4 j = m.count("1")
5 print(2**j)

```

7.9 Permutation Cycles - Lines Of Containers

Resuelve "Lines of Containers" (Day 2, problema G - Maratona de Programacao da SBC 2013): un cargamento de L filas por C columnas de contenedores, numerados 1..LC en orden, fue desordenado moviendo filas y columnas completas con una grua; hay que determinar si la grua puede devolverlos a su posicion original y, de ser posible, el minimo numero de swaps de filas y columnas necesario. Para cada fila (y cada columna) actual deduce a que fila/columna ORIGINAL deberia pertenecer segun los ids, usando (id-1)/C y (id-1)%C, y valida que sea consistente y forme una permutacion valida. Con eso arma rowPerm y colPerm; minSwaps() descompone cada permutacion en ciclos y suma (tamano_del_ciclo - 1) por ciclo, que es el minimo de swaps para ordenarla.

C++\Training_Camp_2026\Day 2\G - Lines of Containers.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 /*
5 Calcula el mínimo número de swaps necesarios para convertir

```

7.9 Permutation Cycles - Lines Of Containers (cont.)

```

6 una permutación en la identidad.
7
8 Si una permutación tiene un ciclo de tamaño k,
9 se necesitan k-1 intercambios para corregirlo.
10 */
11 int minSwaps(vector<int> &perm) {
12
13     int n = perm.size();
14
15     vector<bool> vis(n, false);
16
17     int ans = 0;
18
19     for (int i = 0; i < n; i++) {
20
21         // Si ya fue visitado o ya está en su lugar
22         if (vis[i] || perm[i] == i)
23             continue;
24
25         int cur = i;
26         int cycleSize = 0;
27
28         // Recorremos el ciclo completo
29         while (!vis[cur]) {
30             vis[cur] = true;
31             cur = perm[cur];
32             cycleSize++;
33         }
34
35         // Un ciclo de tamaño k necesita k-1 swaps
36         ans += cycleSize - 1;
37     }
38
39     return ans;
40 }
41
42 int main() {
43
44     ios::sync_with_stdio(false);
45     cin.tie(nullptr);
46
47     int L, C;
48     cin >> L >> C;
49
50     // Matriz con la configuración actual
51     vector<vector<int>> a(L, vector<int>(C));
52
53     for (int i = 0; i < L; i++)
54         for (int j = 0; j < C; j++)
55             cin >> a[i][j];
56
57     /*
58     rowPerm[i] = fila ORIGINAL a la que realmente pertenece
59     la fila actual i.
60
61     Ejemplo:
62     rowPerm = [3,1,2,0]
63
64     significa:
65     la fila actual 0 debería ser la fila 3,
66     la fila actual 1 debería ser la fila 1,
67     etc.
68 */
69     vector<int> rowPerm(L, -1);
70
71     // Revisamos fila por fila
72     for (int i = 0; i < L; i++) {
73
74         /*
75         La fila original de un contenedor se obtiene como
76
77         (id-1)/C

```

7.9 Permutation Cycles - Lines Of Containers (cont.)

```

78
79     porque cada fila contiene exactamente C elementos.
80 */
81     int expectedRow = (a[i][0] - 1) / C;
82
83     // Todos los elementos de la fila deben pertenecer
84     // a la misma fila original.
85     for (int j = 1; j < C; j++) {
86
87         if ((a[i][j] - 1) / C != expectedRow) {
88             cout << "*" << "\n";
89             return 0;
90         }
91     }
92
93     rowPerm[i] = expectedRow;
94 }
95
96 /*
97     Además debe ser una permutación válida.
98
99     No pueden existir dos filas actuales que correspondan
100     a la misma fila original.
101 */
102 vector<bool> used(L, false);
103
104 for (int x : rowPerm) {
105
106     if (used[x]) {
107         cout << "*" << "\n";
108         return 0;
109     }
110
111     used[x] = true;
112 }
113
114 /*
115     colPerm[j] = columna ORIGINAL a la que corresponde
116     la columna actual j.
117 */
118 vector<int> colPerm(C, -1);
119
120 // Revisamos columna por columna
121 for (int j = 0; j < C; j++) {
122
123     /*
124         La columna correcta de un contenedor es
125
126         (id-1)%C
127     */
128     int expectedCol = (a[0][j] - 1) % C;
129
130     // Todos los elementos de la columna deben pertenecer
131     // a la misma columna original.
132     for (int i = 1; i < L; i++) {
133
134         if ((a[i][j] - 1) % C != expectedCol) {
135             cout << "*" << "\n";
136             return 0;
137         }
138     }
139
140     colPerm[j] = expectedCol;
141 }
142
143 // Verificamos nuevamente que sea una permutación válida
144 used.assign(C, false);
145
146 for (int x : colPerm) {
147
148     if (used[x]) {
149         cout << "*" << "\n";

```

7.9 Permutation Cycles - Lines Of Containers (cont.)

```

150     return 0;
151 }
152
153     used[x] = true;
154 }
155
156 /*
157     Si llegamos aquí, existe una secuencia de intercambios.
158
159     El número mínimo es:
160
161     swaps para ordenar las filas
162     +
163     swaps para ordenar las columnas
164 */
165     cout << minSwaps(rowPerm) + minSwaps(colPerm) << "\n";
166
167     return 0;
168 }

```

7.10 Probabilidad (Paradoja del cumpleaños) - Mount Rushmore And Birthdays

Resuelve "Mount Rushmore and Birthdays" (Day 5, problema F - Contest 05 [Avanzados]): dado N, el número de días que tiene un año, hay que hallar la mínima cantidad de personas necesarias en un cuarto para que la probabilidad de que dos compartan cumpleaños sea mayor a 50%. Calcula iterativamente, persona por persona, la probabilidad "prob" de que todos los cumpleaños sigan siendo distintos (multiplicando $(n - \text{personas})/n$ en cada paso, la generalización de la fórmula clásica de la paradoja del cumpleaños), y se detiene apenas $1 - \text{prob}$ supera 0.5, devolviendo esa cantidad de personas.

C++\Training_Camp_2026\Day 5\F - Mount Rushmore and Birthdays.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4
5 int main() {
6     ios::sync_with_stdio(false);
7     cin.tie(nullptr);
8     double n; cin >> n;
9     double prob = 1;
10    double persons = 2;
11    prob *= (n-1)/n;
12
13    if(n==1)
14    {
15        cout << 2 << endl;
16        return 0;
17    }
18
19
20    while(true)
21    {
22        prob *= (n-persons)/n;
23
24        persons++;
25        if(1-prob>0.5)
26        {
27            cout<< persons << endl;
28            return 0;
29        }
30    }
31
32
33
34
35
36
37    return 0;
38 }

```

7.11 Probabilidad (Valor esperado con inverso modular) - Frangolino Ali Na Mesa

Resuelve "Frangolino ali na mesa" (Day 6, problema F - Maratona SBC de Programacao 2025): un robot mesero recibe Q comandos, cada uno con un argumento X_i , pero cada comando es en realidad "ir a la mesa X_i " o "pedir X_i milanesas en la mesa actual" con 50% de probabilidad cada uno (empezando en la mesa 1); hay que calcular, para cada mesa, el valor esperado de milanesas servidas, modulo $1e9+7$. El codigo usa linealidad de la esperanza: arma $\text{coeff}[i] = X_i \cdot \text{inv}2^i$, calcula sumas por sufijo "suffix[i]" de esos coeficientes, inicializa $\text{ans}[1]$ con $\text{suffix}[1]$ (caso en que el robot nunca se movio) y le suma a $\text{ans}[X_i]$ el termino $\text{pow}2[i-1] \cdot \text{suffix}[i+1]$ (la contribucion esperada de que el comando i sea el ultimo movimiento hacia esa mesa), todo con aritmetica modular precomputada de potencias e inversos de 2.

C++\Training_Camp_2026\Day 6\F - Frangolino ali na mesa.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static const int MOD = 1000000007;
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    int N, Q;
11    cin >> N >> Q;
12
13    vector<int> X(Q + 1);
14
15    long long inv2 = (MOD + 1) / 2;
16
17    vector<long long> invPow(Q + 1), pow2(Q + 1);
18    invPow[0] = 1;
19    pow2[0] = 1;
20    for (int i = 1; i <= Q; i++) {
21        invPow[i] = invPow[i - 1] * inv2 % MOD;
22        pow2[i] = pow2[i - 1] * 2 % MOD;
23    }
24
25    vector<long long> coeff(Q + 1);
26    for (int i = 1; i <= Q; i++) {
27        cin >> X[i];
28        coeff[i] = 1LL * X[i] * invPow[i] % MOD;
29    }
30
31    vector<long long> suffix(Q + 2, 0);
32    for (int i = Q; i >= 1; i--) {
33        suffix[i] = (suffix[i + 1] + coeff[i]) % MOD;
34    }
35
36    vector<long long> ans(N + 1, 0);
37
38    // initial value at table 1
39    ans[1] = suffix[1];
40
41    // increments
42    for (int i = 1; i <= Q; i++) {
43        long long add = pow2[i - 1] * suffix[i + 1] % MOD;
44        ans[X[i]] += add;
45        if (ans[X[i]] >= MOD) ans[X[i]] -= MOD;
46    }
47
48    for (int i = 1; i <= N; i++)
49        cout << ans[i] % MOD << "\n";
50
51    return 0;
52 }
```

7.12 Ranking - Palindromic Lexi Small

Construye el k-esimo palindromo mas pequeno lexicograficamente que se puede formar reordenando el string s (o "" si no existen k reordenamientos posibles). Solo arma la mitad izquierda del palindromo (mas el caracter central si el largo es impar) y la refleja: para cada posicion prueba las letras disponibles de menor a mayor y usa $\text{comb}()$ / $\text{perms}()$ (conteo de permutaciones del multiset restante, saturado a $1e6$ para evitar overflow) para saber, sin generar todas las combinaciones, cuantos reordenamientos empiezan con esa letra y saltarselos si k cae mas adelante.

C++\LeetCode\palindromicLexiSmall.cpp

```
1 class Solution {
2 public:
3     const long long LIM = 1000000;
4
5     // C(n, r) saturado a LIM+1 si el valor real lo supera.
6     // O(r), con división exacta garantizada en cada paso (identidad de binomiales),
7     // en vez de armar vectores de tamaño r y simplificar con gcd (eso era O(r^2) y el cuello de bo
8     // tella real).
9     long long comb(long long n, long long r) {
10        if (r < 0 || r > n) return 0;
11        r = min(r, n - r);
12        long long result = 1;
13        for (long long i = 1; i <= r; i++) {
14            result = result * (n - i + 1) / i;
15            if (result > LIM) return LIM + 1;
16        }
17        return result;
18    }
19
20    // Permutaciones distintas del multiset cnt[0..25] (n = total restante), saturado a LIM+1.
21    long long perms(long long n, const array<int, 26>& cnt) {
22        long long ans = 1;
23        long long restantes = n;
24        for (int c = 0; c < 26; c++) {
25            if (cnt[c] == 0) continue;
26            long long ways = comb(restantes, cnt[c]);
27            if (ways > LIM) return LIM + 1;
28            if (ans > LIM / ways) return LIM + 1;
29            ans *= ways;
30            restantes -= cnt[c];
31            if (ans > LIM) return LIM + 1;
32        }
33        return ans;
34    }
35
36    string smallestPalindrome(string s, int k) {
37        int n = s.size();
38        bool mid = (n % 2 != 0);
39        char midi = mid ? s[n / 2] : '\0';
40        string half = s.substr(0, n / 2);
41
42        array<int, 26> cnt{};
43        long long tot = 0;
44        for (char c : half) { cnt[c - 'a']++; tot++; }
45
46        if ((long long)k > perms(tot, cnt)) return "";
47
48        string god;
49        god.reserve(half.size());
50        long long cont = 0;
51        long long curTot = tot;
52
53        for (size_t i = 0; i < half.size(); i++) {
54            for (int c = 0; c < 26; c++) {
55                if (cnt[c] == 0) continue;
56                cnt[c]--;
57                curTot--;
58                long long permi = perms(curTot, cnt);
59                if (cont + permi < k) {
60                    cont += permi;
61                    cnt[c]++;
62                    curTot++;
63                }
64            }
65        }
```

7.12 Ranking - Palindromic Lexi Small (cont.)

```

62         } else {
63             god += char('a' + c);
64             break;
65         }
66     }
67 }
68
69 string sol = god;
70 if (mid) sol += midi;
71 reverse(god.begin(), god.end());
72 sol += god;
73 return sol;
74 }
75 };

```

7.13 Ranking de Combinaciones - Discrete Catalog

Resuelve "Discrete Catalog" (problema D, ICPC 2025): dado un subconjunto de tamaño k escrito en orden creciente, decir que posición ocupa en la lista de TODOS los subconjuntos de tamaño k ordenados lexicográficamente, contando desde 0. LA IDEA ES CONTAR LO QUE VA ANTES, sin generar nada. Recorriendo el subconjunto de izquierda a derecha, cuando en la posición i está el valor $a[i]$, todos los subconjuntos que en esa posición tienen un valor MENOR (y coinciden en lo anterior) van antes. Para cada candidato j menor que $a[i]$, lo que queda por elegir son $k-i-1$ valores de entre los $n-j$ que siguen, o sea $C(n-j, k-i-1)$. Se suman todos esos y se pasa a la siguiente posición. Ese patrón de "contar los que van antes prefijo por prefijo" es el ranking, y sirve igual para permutaciones o para cualquier orden lexicográfico: cambia solo la fórmula de cuantos completan. El nCr usado multiplica y divide alternando ($r = r * (n-i+1) / i$), que mantiene los números chicos porque en cada paso r es exactamente un binomial. Con $n \leq 60$ no se desborda: el mayor ID posible es $C(60,30) - 1 = 118264581564861423$, que cabe holgado en long long. Ojo que esa misma función SI se desborda a partir de $n = 62$ aunque el resultado quepa. Verificado de dos formas: contra la lista completa ordenada para n hasta 12 (6556 escenarios) y contra el ranking exacto calculado aparte para n hasta 60 (4005 escenarios). Sin fallos, y los nueve casos del sample dan exacto.

C++\ICPC\2025\D - Discrete Catalog.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 ll nCr(int n, int k) {
7     if (k < 0 || k > n)
8         return 0;
9
10    k = min(k, n - k);
11
12    ll r = 1;
13
14    for (int i = 1; i <= k; i++) {
15        r = r * (n - i + 1) / i;
16    }
17
18    return r;
19 }
20
21 int main() {
22     ios::sync_with_stdio(false);
23     cin.tie(nullptr);
24
25     int n, k;
26
27     while (cin >> n >> k) {
28         vector<int> a(k);
29
30         for (int &x : a)
31             cin >> x;
32
33         ll god = 0;
34
35         int anterior = 0;

```

7.13 Ranking de Combinaciones - Discrete Catalog (cont.)

```

37         for (int i = 0; i < k; i++)
38         {
39             for (int j = anterior + 1; j < a[i]; j++)
40             {
41                 god += nCr(n - j, k - i - 1);
42             }
43             anterior = a[i];
44         }
45
46         cout << god << endl;
47     }
48
49     return 0;
50 }

```

7.14 Suma de Subconjuntos (2^n) - Group Work

Resuelve "Group work" (Day 7, problema C - Contest 07 [Avanzados]): dada una clase de N estudiantes, cuenta cuantos grupos distintos de al menos 2 estudiantes se pueden formar (dos grupos son distintos si difieren en al menos un estudiante). La respuesta es la suma de combinaciones $C(n,m)$ para $m=2..n$ (equivalente a 2^n menos los subconjuntos de tamaño 0 y 1); el código usa `math.comb` para sumar directamente esas combinaciones.

C++\Training_Camp_2026\Day 7\C - Group work.py

```

1 import math
2 n = int(input())
3
4 suma = 0
5 for m in range(2,n+1):
6     suma+=math.comb(n,m)
7 print(suma)

```

7.15 Técnica de Contribucion - Bytelandia's Stones

Resuelve "Bytelandia's stones" (Day 9, problema B - Contest 08 [Inicial & Avanzado]): dado un arreglo $a_1..a_n$, se pide la suma de $S(L,R)$ = suma de a_i en $[L,R]$ sobre TODOS los pares (L,R) posibles. En vez de calcular cada subarreglo, usa la técnica de contribucion: cada elemento a_i aparece en exactamente $i*(n-i+1)$ subarreglos $[L,R]$ (eligiendo L en $[1,i]$ y R en $[i,n]$), así que la respuesta es la suma de $x * i * (n-i+1)$ para cada valor x leído en la posición i , acumulada en $O(n)$.

C++\Training_Camp_2026\Day 9\B - Bytelandia's stones.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9     cin >> n;
10
11     long long ans = 0;
12
13     for (int i = 1; i <= n; i++)
14     {
15         long long x;
16         cin >> x;
17         ans += x * 1LL * i * (n - i + 1);
18     }
19
20     cout << ans << endl;
21     return 0;
22 }

```

7.16 Triangulos en Poligono Convexo

O: (1), es $C(n,3)$

Uso: $c3(n) = n*(n-1)*(n-2)/6$ // todo trio de un convexo es triangulo

Dado n, cuantos triangulos distintos salen escogiendo 3 vertices de un poligono regular convexo de n lados. La respuesta es simplemente $C(n,3) = n*(n-1)*(n-2)/6$. Con n = 5 da 10, que son exactamente los diez del dibujo del enunciado. TODO EL PROBLEMA ESTA EN VER POR QUE NO HAY QUE RESTAR NADA. Contar triangulos con vertices de un conjunto de puntos NO es $C(n,3)$ en general: hay que quitar las ternas COLINEALES, que dan un triangulo degenerado de area cero. Lo que salva este problema es la palabra CONVEXO: en un poligono convexo ningun vertice cae sobre el segmento que une a otros dos, así que no existe una sola terna colineal y las $C(n,3)$ ternas sirven todas. Si el enunciado dijera "puntos en el plano" en vez de "poligono convexo", la solución sería otra: contar las ternas colineales agrupando por direccion y restarlas. Y se cuentan CONJUNTOS de vertices, no formas. En el pentagono los 10 incluyen repetidos por congruencia (5 con dos lados del poligono y 5 con un lado y dos diagonales), y el dibujo los muestra como distintos. Si pidieran formas distintas salvo rotacion y reflexion, sería otro problema completamente, de conteo de Burnside. La sucesion son los numeros tetraedricos: n = 3, 4, 5, ... da 1, 4, 10, 20, 35, 56, 84, 120, 165, 220. Sirve para reconocerla si aparece en una salida sin saber de donde viene. Verificado contra fuerza bruta (contar las ternas una por una) para n de 0 a 300, y contra enteros exactos para n hasta 2000: sin un solo fallo. LO UNICO QUE PUEDE FALLAR ES EL DESBORDE, y vale saber donde exactamente (medido): - Escribiendo $n*(n-1)*(n-2)/6$ tal cual, el PRODUCTO intermedio se pasa de long long con n = 2097154. O sea que la formula directa aguanta hasta n = 2097153. - Dividiendo ANTES de multiplicar (que es lo que hace c3 aqui abajo) se llega hasta n = 3810779, y ahi el que ya no cabe es el RESULTADO, no un paso intermedio. - Conviene tener claro que dividir temprano solo gana un factor 1.82 en n: el techo duro lo pone el resultado, y contra eso no hay orden de operaciones que ayude. Si n es mas grande toca __int128, unsigned, o responder modulo algo. POR QUE LA DIVISION TEMPRANA ES EXACTA: entre tres enteros consecutivos siempre hay uno multiplo de 3 y al menos uno par, así que se puede dividir cada factor por separado sin perder nada. Lo que NO se puede es dividir a lo bruto: hay que buscar CUAL de los tres es el multiplo de 3, porque no siempre es el mismo. Si el problema pide la respuesta modulo un primo p, no se divide: se multiplica por el inverso de 6, o sea $C(n,3) = n*(n-1)*(n-2) * \text{inv6} \% p$, con $\text{inv6} = \text{modpow}(6, p-2, p)$.

C++\Notebook\Triangulos en Poligono Convexo.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 // C(n,3) sin formar el producto completo.
6 ll c3(ll n) {
7     if (n < 3) return 0;
8
9     ll a = n, b = n - 1, c = n - 2;
10
11 // Entre tres consecutivos hay exactamente un multiplo de 3.
12 if (a % 3 == 0) {
13     a /= 3;
14 } else if (b % 3 == 0) {
15     b /= 3;
16 } else {
17     c /= 3;
18 }
19
20 // Y al menos uno par (puede que el de arriba ya haya cambiado la paridad).
21 if (a % 2 == 0) {
22     a /= 2;
23 } else if (b % 2 == 0) {
24     b /= 2;
25 } else {
26     c /= 2;
27 }
28
29 return a * b * c;
30 }
31
32 int main() {
33     ios::sync_with_stdio(false);
34     cin.tie(nullptr);
35
36     ll n;
37     while (cin >> n) {
```

7.16 Triangulos en Poligono Convexo (cont.)

```
38     cout << c3(n) << '\n';
39 }
40 return 0;
41 }
```

8 - Game Theory

8.1 Misere Nim y Grundy Acotado

O: (n) misere_nim, (1) Grundy_bounded

Uso: misere_nim(pilas) -> gana el primero?; Grundy_bounded(n,k) = n % (k+1)

Dos variantes del Nim que NO se resuelven con el XOR normal (ese caso ya esta cubierto en "Marbles" con Sprague-Grundy). Misere Nim invierte la condicion de victoria: pierde quien hace el ultimo movimiento, y la regla cambia solo en el caso degenerado en que todos los montones valen 1, donde gana el primero si la cantidad de montones es par; en cualquier otro caso se aplica el XOR de siempre. Grundy acotado es el atajo para cuando se pueden quitar entre 1 y k piedras: el numero de Grundy resulta ser exactamente $n \% (k+1)$, así que no hace falta calcular el mex, y se pierde justo cuando n es multiplo de k+1.

C++\Notebook\Misere Nim y Grundy Acotado.cpp

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 // devuelve true si gana el primer jugador (pierde quien mueve al final)
6 bool misere_nim(vector<int> a)
7 {
8     bool all_ones = all_of(a.begin(), a.end(), [](int x){ return x == 1; });
9     int xorsum = 0;
10    for (int v : a) xorsum ^= v;
11
12    if (all_ones)
13    {
14        return (a.size() % 2 == 0);
15    }
16    return (xorsum != 0);
17 }
18
19 // se pueden quitar entre 1 y k piedras: Grundy(n) = n % (k+1)
20 int Grundy_bounded(int n, int k)
21 {
22     return n % (k + 1);
23 }
24
25 int main()
26 {
27     ios_base::sync_with_stdio(false);
28     cin.tie(nullptr);
29
30     int n, k;
31     cin >> n >> k;
32
33     vector<int> pilas(n);
34     for (int i = 0; i < n; i++) cin >> pilas[i];
35
36     cout << (misere_nim(pilas) ? "First" : "Second") << "\n";
37
38 // en la version acotada, la suma de subjuegos sigue siendo el XOR de los Grundy
39 int total = 0;
40 for (int x : pilas) total ^= Grundy_bounded(x, k);
41 cout << (total ? "First" : "Second") << "\n";
42
43     return 0;
44 }
```

8.2 Nim con Segment Tree - Juan Vs Frank

Resuelve "Juan vs Frank" (Day 9, problema I - Contest 08 [Inicial & Avanzado]): Frank y Juan juegan duelos de Nim sobre un rango de columnas [l,r], donde cada columna es un monton de fragmentos y en cada turno se retiran fragmentos de una sola columna; pierde quien no puede mover. Por la teoria de Nim, el primer jugador (Frank) gana si y solo si el XOR de los valores del rango es distinto de cero. Mantiene un segment tree que guarda el XOR de rangos (arreglo seg) con soporte de update puntual (rituales que suman x a una columna) y query de rango (profecias), respondiendo "FRANK" si el XOR del rango es != 0 y "JUAN" si es 0.

C++\Training_Camp_2026\Day 9\I - Juan vs Frank.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    int N, Q;
11    cin >> N >> Q;
12
13    vector<ll> a(N);
14
15    int SZ = N;
16    vector<ll> seg(2 * SZ);
17
18    for (int i = 0; i < N; i++) {
19        cin >> a[i];
20        seg[SZ + i] = a[i];
21    }
22
23    for (int i = SZ - 1; i > 0; i--)
24        seg[i] = seg[i << 1] ^ seg[i << 1 | 1];
25
26    auto update = [&](int pos, ll val) {
27        pos--;
28        a[pos] += val;
29        int p = pos + SZ;
30        seg[p] = a[pos];
31        while (p >= 1)
32            seg[p] = seg[p << 1] ^ seg[p << 1 | 1];
33    };
34
35    auto query = [&](int l, int r) {
36        ll ans = 0;
37        l = l - 1 + SZ;
38        r = r + SZ;
39        while (l < r) {
40            if (l & 1) ans ^= seg[l++];
41            if (r & 1) ans ^= seg[--r];
42            l >>= 1;
43            r >>= 1;
44        }
45        return ans;
46    };
47
48    while (Q--) {
49        char op;
50        cin >> op;
51
52        if (op == 'P') {
53            int l, r;
54            cin >> l >> r;
55            if (query(l, r) == 0)
56                cout << "JUAN\n";
57            else
58                cout << "FRANK\n";
59        } else {
60            int k;
61            ll x;
62            cin >> k >> x;
```

8.2 Nim con Segment Tree - Juan Vs Frank (cont.)

```
63         update(k, x);
64     }
65 }
66
67 return 0;
68 }
```

8.3 Posiciones Perdedoras (potencias de 2) - Digitsduel

Determina el ganador de un juego analizando si $N+2$ es potencia de 2. Si lo es, el segundo jugador gana ("mastermei"), si no, el primero ("the greatest").

C++\Codeforces\digitsduel.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool isPowerOfTwo(long long x) {
5     return x > 0 && (x & (x - 1)) == 0;
6 }
7
8 int main() {
9     ios::sync_with_stdio(false);
10    cin.tie(nullptr);
11
12    int T;
13    cin >> T;
14
15    while (T--) {
16        long long N;
17        cin >> N;
18
19        if (isPowerOfTwo(N + 2)) {
20            cout << "the greatest\n";
21        } else {
22            cout << "mastermei\n";
23        }
24    }
25
26    return 0;
27 }
```

8.4 Sprague-Grundy - Marbles

Resuelve "Marbles" (Day 8, problema B - Contest 08 [Avanzados]): en un tablero indexado por filas y columnas, un jugador elige una canica en (l,c) y un entero $u>0$ y la mueve a (l-u,c), (l,c-u) o (l-u,c-u) sin salir del tablero; el primero en llevar una canica a (0,0) gana y el Emperador siempre juega primero. Dado el reparto inicial de N canicas, hay que decidir si el Emperador puede ganar jugando optimo. Modela el juego como la suma de N subjuegos independientes (teoria de Sprague-Grundy): precomputeGrundy calcula el numero de Grundy de cada posicion (l,c) via mex de posiciones alcanzables (marcando INSTA cuando $l==0$, $c==0$ o $l==c$, movimiento ganador inmediato), y el XOR de los Grundy de todas las canicas (mas el chequeo instaWin) decide si hay victoria.

C++\Training_Camp_2026\Day 8\B - Marbles.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 static char buf_[1 << 16];
5 static int bufLen_, bufPos_;
6 inline int gc() {
7     if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!bufLen_) return -1; }
8     return buf_[bufPos_++];
9 }
10 inline int readInt() {
11     int c = gc();
12     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
13     bool neg = false; if (c == '-') { neg = true; c = gc(); }
14     int x = 0;
15     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
16     return neg ? -x : x;
17 }
```

8.4 Sprague-Grundy - Marbles (cont.)

```

8
9 const int MAXV = 101;           // coordinates go from 0 to 100
20 const int INSTA = 1000000;     // sentinel for "instant win" positions: (i,0), (0,i), (i,i)
21 int grundy_[MAXV][MAXV];
22
23 void precomputeGrundy() {
24     // process states in order of increasing max(l,c): every move strictly decreases
25     // max(l,c) or keeps l the same while decreasing c (or vice versa), so processing
26     // by increasing (1+c) is a safe topological order (all moves reduce 1+c).
27     vector<int> seen(3 * MAXV + 10, -1); // seen[v] == stamp means v currently marked reachable
28     int stamp = 0;
29
30     vector<pair<int,int>> order;
31     for (int l = 0; l < MAXV; l++)
32         for (int c = 0; c < MAXV; c++)
33             order.push_back({l, c});
34     sort(order.begin(), order.end(), [](const pair<int,int>&a, const pair<int,int>&b){
35         return (a.first + a.second) < (b.first + b.second);
36     });
37
38     for (auto &pr : order) {
39         int l = pr.first, c = pr.second;
40         if (l == 0 || c == 0 || l == c) {
41             grundy_[l][c] = INSTA;
42             continue;
43         }
44         stamp++;
45         for (int u = 1; u <= max(l, c); u++) {
46             int g;
47             if (l - u >= 0) {
48                 g = grundy_[l - u][c];
49                 if (g < (int)seen.size()) seen[g] = stamp;
50             }
51             if (c - u >= 0) {
52                 g = grundy_[l][c - u];
53                 if (g < (int)seen.size()) seen[g] = stamp;
54             }
55             if (l - u >= 0 && c - u >= 0) {
56                 g = grundy_[l - u][c - u];
57                 if (g < (int)seen.size()) seen[g] = stamp;
58             }
59         }
60         int mex = 0;
61         while (mex < (int)seen.size() && seen[mex] == stamp) mex++;
62         grundy_[l][c] = mex;
63     }
64 }
65
66 int main() {
67     precomputeGrundy();
68
69     int N = readInt();
70     vector<pair<int,int>> marbles(N);
71     for (auto &m : marbles) { m.first = readInt(); m.second = readInt(); }
72
73     bool instaWin = false;
74     for (auto &m : marbles) {
75         if (m.first == m.second) { instaWin = true; break; }
76     }
77
78     if (instaWin) {
79         printf("Y\n");
80         return 0;
81     }
82
83     int x = 0;
84     for (auto &m : marbles) {
85         x ^= grundy_[m.first][m.second];
86     }
87     printf(x != 0 ? "Y\n" : "N\n");
88     return 0;
89 }

```

9 - String

9.1 Brute Force Matching - Enigma

Resuelve "Enigma" (Day 8, problema E - Contest 08 [Avanzados]): dado un mensaje cifrado y un crib (palabra que se asume esta en el mensaje original), hay que contar en cuantas posiciones puede empezar el crib dentro del mensaje cifrado, sabiendo que la maquina Enigma nunca reemplaza una letra por si misma (o sea, en una posicion valida ninguna letra del crib puede coincidir con la letra del mensaje en esa misma posicion). Para cada posicion i de arranque, compara letra a letra el crib contra el substring del mensaje de ese largo (variable "tempi"), y descarta la posicion apenas encuentra una letra igual (bandera "ok"); cuenta en "god" las posiciones donde ninguna letra coincidio, en fuerza bruta $O(n*m)$.

C++\Training_Camp_2026\Day 8\E - Enigma.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 int main()
4 {
5     string s1,s2;
6     cin >> s1 >> s2;
7     int god=0;
8
9     for(int i = 0 ; i <= s1.size()-s2.size();i++)
10    {
11        string tempi = s1.substr(i,s2.size());
12        bool ok = false;
13        for(int j = 0 ; j < s2.size() && ok == false;j++)
14        {
15            if(tempi[j]==s2[j])
16            {
17                ok = true;
18            }
19        }
20        if(ok == false)
21        {
22            god++;
23        }
24    }
25
26    cout << god << endl;
27
28    return 0;
29 }

```

9.2 Chequeo de Palindromo

O: (1) por consulta tras $O(n^2)$ de tabla; manacher es $O(n)$

Uso: tablaPalindromos(s)[l][r]; manacher(s) -> {d1,d2} impar/par

Las tres formas de preguntar "esto es palindromo", segun cuantas veces lo vayas a preguntar. Para una consulta suelta basta el chequeo de dos punteros en $O(n)$ y sin memoria extra. Si el problema pregunta por muchos rangos [l,r] distintos, conviene precalcular la tabla pal[i][j] en $O(n^2)$ y responder cada consulta en $O(1)$. Y si n es grande y hace falta el palindromo mas largo, ahi ya toca Manacher en $O(n)$. La regla para elegir: cuenta cuantas consultas vas a hacer. Una sola, dos punteros; muchas sobre $n \leq 2000$, la tabla; n de 10^4 o mas, Manacher.

C++\Notebook\Chequeo de Palindromo.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // 1) Consulta suelta: O(n) tiempo, O(1) memoria.
5 // Se pasa por referencia para no copiar el string en cada llamada.
6 bool esPalindromo(string &s, int l, int r) {
7     while (l < r) {
8         if (s[l] != s[r]) return false;
9         l++;
10        r--;
11    }
12    return true;
13 }
14
15 // El string completo, con la STL:
16 // equal(s.begin(), s.begin() + s.size() / 2, s.rbegin())

```

9.2 Chequeo de Palindromo (cont.)

```

1 //
2 // 2) Muchas consultas de rango: tabla  $O(n^2)$ , cada consulta  $O(1)$ .
3 // pal[i][j] depende del interior [i+1][j-1], así que i va de DERECHA a
4 // IZQUIERDA para que ese interior ya este resuelto.
5 vector<vector<bool>> tablaPalindromos(const string &s) {
6     int n = s.size();
7     vector<vector<bool>> pal(n, vector<bool>(n, false));
8     for (int i = n - 1; i >= 0; i--)
9         for (int j = i; j < n; j++)
10             if (s[i] == s[j] && (j - i < 2 || pal[i + 1][j - 1]))
11                 pal[i][j] = true;
12     return pal;
13 }
14 // 3) Palindromo mas largo con n grande: Manacher,  $O(n)$ .
15 // d1[i] = cuantos palindromos IMPARES centrados en i
16 // d2[i] = cuantos palindromos PARES centrados entre i-1 e i
17 // El impar mas largo centrado en i mide  $2*d1[i]-1$ , el par mide  $2*d2[i]$ .
18 pair<vector<int>, vector<int>> manacher(const string &s) {
19     int n = s.size();
20     vector<int> d1(n), d2(n);
21     for (int i = 0, l = 0, r = -1; i < n; i++) {
22         int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
23         while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) k++;
24         d1[i] = k--;
25         if (i + k > r) { l = i - k; r = i + k; }
26     }
27     for (int i = 0, l = 0, r = -1; i < n; i++) {
28         int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
29         while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) k++;
30         d2[i] = k--;
31         if (i + k > r) { l = i - k - 1; r = i + k; }
32     }
33     return {d1, d2};
34 }
35
36 int main() {
37     ios::sync_with_stdio(false);
38     cin.tie(nullptr);
39
40     string s;
41     while (cin >> s) {
42         int n = s.size();
43         cout << (esPalindromo(s, 0, n - 1) ? "SI" : "NO") << "\n";
44     }
45     return 0;
46 }
47
48 // OJO con los casos borde del chequeo de dos punteros: con l == r (un solo
49 // caracter) el while no entra y devuelve true, que es lo correcto; con un
50 // rango vacio (l > r) tambien devuelve true. Si en tu problema el palindromo
51 // vacio NO cuenta, filtra el largo ANTES de llamar a la funcion.

```

9.3 KMP para Contar Ocurrencias Solapadas - Judgmental Crowd

Resuelve "Judgmental Crowd" (problema J, Regionals 2025): la reaccion del publico se calcula contando cuantas veces aparecen tres palabras en la cadena y combinandolas con pesos, ha - boooo + 3 * bravo EL DETALLE QUE DECIDE EL PROBLEMA es contar ocurrencias SOLAPADAS. Tras un match no se reinicia $j = 0$ sino $j = pi[j-1]$, que deja el automata en el sufijo mas largo que todavia sirve. Con "haha" hay DOS ocurrencias de "ha" y no una; reiniciando en 0 se perderia la segunda cuando los patrones se pisan (con "aa" dentro de "aaa" pasa lo mismo). Se llama a la misma funcion KMP tres veces, una por palabra, porque los patrones son fijos y cortos. Si fueran muchos patrones convendria Aho-Corasick, que los busca todos en una sola pasada. La funcion de prefijos se calcula sobre el PATRON, no sobre el texto: $pi[i]$ es el borde mas largo de $p[0..i]$, y es lo que permite retroceder sin volver atras en el texto. Por eso todo el conteo es $O(|s| + |p|)$. Comprobado: con "haha" da 2, que son las dos ocurrencias de "ha".

C++\ICPC\Regionals 2025J - Judgmental Crowd.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3

```

9.3 KMP para Contar Ocurrencias Solapadas - Judgmental Crowd (cont.)

```

4 int KMP(const string& s, const string& p) {
5
6     int n = s.size();
7     int m = p.size();
8
9     // Prefix function del patron
10    vector<int> pi(m);
11
12    for (int i = 1; i < m; i++) {
13
14        int j = pi[i - 1];
15
16        while (j > 0 && p[i] != p[j])
17            j = pi[j - 1];
18
19        if (p[i] == p[j])
20            j++;
21
22        pi[i] = j;
23    }
24
25    // Buscar p dentro de s
26    int ans = 0;
27    int j = 0;
28
29    for (int i = 0; i < n; i++) {
30
31        while (j > 0 && s[i] != p[j])
32            j = pi[j - 1];
33
34        if (s[i] == p[j])
35            j++;
36
37        if (j == m) {
38            ans++;
39
40            // Importante para permitir solapamientos
41            j = pi[j - 1];
42        }
43    }
44
45    return ans;
46 }
47
48 int main() {
49
50     ios::sync_with_stdio(false);
51     cin.tie(nullptr);
52
53     string s;
54
55     cin >> s;
56
57     cout << KMP(s, "ha") - KMP(s, "boooo") + 3*KMP(s, "bravo") << "\n";
58
59     return 0;
60 }

```

9.4 KMP sobre Cadena Recursiva (Duplicacion) - Song Of The Sirens

Resuelve "Song of the Sirens" (Codeforces, problema G): hay $n+1$ canciones definidas por $s_{i+1} = s_i + t[i] + s_i$, y cada consulta pide cuantas veces aparece un nombre w dentro de s_k , modulo $1e9+7$. La cadena crece al doble en cada paso, así que s_n puede medir 2^{100000} y construirla esta descartado de entrada. LA RECURRENCIA ES TODO: $f(i+1) = 2*f(i) + g(t[i])$. Las apariciones de w en s_{i+1} son las de la copia izquierda, las de la derecha (de ahí el $2*f(i)$) y las que CRUZAN el caracter del medio. Esas ultimas son g . POR QUE g SOLO DEPENDE DE LA LETRA Y NO DE LA POSICION, que es el paso que hay que ver: una aparicion que cruza el centro tiene largo m , así que no puede meterse mas de $m-1$ caracteres hacia cada lado. O sea que solo depende del SUFIJO de largo $m-1$ de s_i , de la letra del medio, y del PREFIJO de largo $m-1$ de s_i . Y como s_{i+1} empieza y termina con s_i , esos prefijos y sufijos YA NO CAMBIAN en cuanto $|s_i| \geq m-1$. Entonces g se calcula una sola vez para cada una de las 26 letras, contando apariciones de w en (sufijo + letra + prefijo). Por eso primero se hace crecer $cur = s_k$ hasta que mida al menos m . Como se duplica cada vez, eso son a lo sumo unos 20 pasos con $|w|$ hasta 10^6 , no n pasos. DESEROLLANDO la recurrencia desde ese punto k : $f(n) = f(k) * 2^{(n-k)} + \text{suma desde } i=k+1 \text{ hasta } n \text{ de } g(t[i-1]) * 2^{(n-i)}$. Esa suma tiene hasta n terminos, y con q consultas seria $O(n*q) = 10^6 * 10$. El truco para bajarla: como $2^{(n-j-1)} = 2^{(n-1)} * 2^{(-j)}$, se saca el $2^{(n-1)}$ afuera y lo que queda, suma de $2^{(-j)}$ sobre las posiciones j donde $t[j]$ es la letra c , se precalcu la UNA vez en prefijos por letra. Así cada consulta cuesta $O(26)$ en esa parte en vez de $O(n)$. INV2 = 500000004 es el inverso de 2 modulo $1e9+7$, que sale de $(MOD+1)/2$ y evita tener que llamar a una exponenciacion modular. Los $2^{(-j)}$ son potencias de ese inverso. DETALLES DEL KMP: despues de un match se hace $j = pi[j-1]$ en vez de $j = 0$, que es lo que permite contar apariciones SOLAPADAS (con $w = "aa"$ en "aaa" hay dos, no una). Y el mismo arreglo pi se reusa para el conteo en cur y para los 26 strings de g , que por eso se pasa como parametro en vez de recalcularlo. Los limites cuadran porque la suma de todos los $|w|$ no pasa de 10^6 : cur mide $\sim 2m$ y cada uno de los 26 strings de g mide $2m-1$, así que el trabajo por consulta es $O(m)$ veces una constante. Si al terminar de crecer todavia $|cur| < m$, es que ni s_n alcanza el largo de w y la respuesta es 0. Este archivo entro al cuaderno sin la verificacion habitual, a pedido: ya venia aceptado.

C++(Codeforces)songOfTheSirens.cpp

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

const ll MOD = 1000000007;
const ll INV2 = 500000004;

vector<int> build_pi(const string& p) {
    int m = p.size();
    vector<int> pi(m);

    for (int i = 1; i < m; i++) {
        int j = pi[i - 1];

        while (j > 0 && p[i] != p[j])
            j = pi[j - 1];

        if (p[i] == p[j])
            j++;

        pi[i] = j;
    }

    return pi;
}

ll count_occ(const string& s, const string& p, const vector<int>& pi) {
    int j = 0;
    ll ans = 0;

    for (char c : s) {
        while (j > 0 && c != p[j])
            j = pi[j - 1];

        if (c == p[j])
            j++;

        if (j == (int)p.size()) {
            ans++;
        }
    }
}
```

9.4 KMP sobre Cadena Recursiva (Duplicacion) - Song Of The Sirens (cont.)

```
j = pi[j - 1];
}
}

return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N, Q;
    cin >> N >> Q;

    string s0, t;
    cin >> s0 >> t;

    // pow2[i] = 2^i
    // invpow2[i] = 2^(-i)
    vector<ll> pow2(N + 1), invpow2(N + 1);

    pow2[0] = 1;
    invpow2[0] = 1;

    for (int i = 1; i <= N; i++) {
        pow2[i] = pow2[i - 1] * 2 % MOD;
        invpow2[i] = invpow2[i - 1] * INV2 % MOD;
    }

    /*
    pref[c][i] =
    suma de 2^(-j) para j < i donde t[j] == c
    */

    vector<vector<ll>> pref(26, vector<ll>(N + 1));

    for (int c = 0; c < 26; c++) {
        for (int i = 0; i < N; i++) {
            pref[c][i + 1] = pref[c][i];

            if (t[i] - 'a' == c)
                pref[c][i + 1] =
                    (pref[c][i + 1] + invpow2[i]) % MOD;
        }
    }

    while (Q--) {
        int n;
        string w;

        cin >> n >> w;

        int m = w.size();

        vector<int> pi = build_pi(w);

        /*
        Construimos s_k hasta que:

        |s_k| >= |w|

        o hasta llegar a s_n si n es menor.
        */

        string cur = s0;

        int k = 0;

        while (k < n && (int)cur.size() < m) {
            cur = cur + t[k] + cur;
            k++;
        }
    }
}
```

9.4 KMP sobre Cadena Recursiva (Duplicacion) - Song Of The Sirens (cont.)

```

118
119 /*
120     Si todavia |cur| < |w|, entonces estamos en s_n
121     y simplemente hacemos KMP directamente.
122 */
123
124 if ((int)cur.size() < m) {
125     cout << 0 << '\n';
126     continue;
127 }
128
129 /*
130     cur = s_k
131 */
132
133 ll base = count_occ(cur, w, pi);
134
135 /*
136     Si k == n, ya tenemos exactamente s_n.
137 */
138
139 if (k == n) {
140     cout << base % MOD << '\n';
141     continue;
142 }
143
144 /*
145     Para calcular g(c):
146     suffix(|w|-1) + c + prefix(|w|-1)
147
148     Todas las apariciones de w en este string
149     contienen obligatoriamente el caracter central c.
150 */
151
152 string prefix = cur.substr(0, m - 1);
153 string suffix = cur.substr(cur.size() - (m - 1), m - 1);
154
155 vector<ll> g(26);
156
157 for (int c = 0; c < 26; c++) {
158     string x;
159     x.reserve(2 * m - 1);
160
161     x += suffix;
162     x += char('a' + c);
163     x += prefix;
164
165     g[c] = count_occ(x, w, pi);
166 }
167
168 /*
169     f(n,w) =
170     f(k,w) * 2^(n-k)
171     +
172     sum_{i=k+1}^n g(i,w) * 2^(n-i)
173
174     Como g(i,w) depende unicamente de t[i-1]:
175
176     g(i,w) = g[t[i-1]]
177 */
178
179 ll ans = base * pow2[n - k] % MOD;
180
181 /*
182     Queremos:
183
184     sum_{j=k}^{n-1}
185     g[t[j]] * 2^(n-j-1)
186
187     = 2^(n-1) *
188     sum_{j=k}^{n-1}

```

9.4 KMP sobre Cadena Recursiva (Duplicacion) - Song Of The Sirens (cont.)

```

85     g[t[j]] * 2^(-j)
86 */
87
88 ll sum = 0;
89
90 for (int c = 0; c < 26; c++) {
91     ll x = (pref[c][n] - pref[c][k] + MOD) % MOD;
92     sum = (sum + g[c] * x) % MOD;
93 }
94
95 ans = (ans + sum * pow2[n - 1]) % MOD;
96
97 cout << ans << '\n';
98 }
99
200 return 0;
201 }

```

9.5 Maximo Solape Sufijo-Prefijo (KMP) - Sign

Resuelve "Scrolling Sign" (problema H, CCPL): un letrero muestra k caracteres; en cada paso todo se corre una posicion a la izquierda y entra una letra nueva por la derecha. Hay que mostrar w palabras de k letras en orden, metiendo la menor cantidad de letras posible. TODA LA REDUCCION ESTA EN UNA OBSERVACION: si estando en la palabra A se meten t letras, el letrero queda mostrando $A[t.k-1]$ seguido de las t nuevas. Para que eso sea B hace falta que $A[t.k-1]$ sea igual a $B[0.k-t-1]$, o sea que un SUFIJO de A de largo $k-t$ coincida con el PREFIJO de B de ese mismo largo. Entonces el costo minimo de pasar de A a B es $k -$ (mayor L tal que el sufijo de A de largo L es prefijo de B) y el total es k por la primera palabra (el letrero arranca vacio) mas esos costos. Que el enunciado permita mostrar palabras intermedias que no son del mensaje no cambia nada: esas son justamente los estados por los que se pasa mientras entran las t letras. Y lo de "si una palabra se repite seguida, se muestra una sola vez" sale gratis: si $A == B$ el solape es k y el costo es 0, sin escribir ningun caso especial. COMO SE SACA EL SOLAPE SIN PAGAR $O(k^2)$: se arma la cadena $B + \# + A$ y se calcula la funcion de fallo de KMP; su ultimo valor es el mayor prefijo de B que ademas es sufijo de A , que es exactamente L . El separador tiene que ser un caracter que NO aparezca en las palabras (aquí son mayusculas, así que '#' sirve), porque si no el fallo podria cruzar de una mitad a la otra y devolver un valor mayor que k , que no corresponde a ningun solape real. Probar los L de k hacia abajo comparando substrings seria $O(k^2)$ por par y $O(w*k^2)$ en total; con $k = w = 1500$ eso son $3*10^9$ y no pasa. Con KMP es $O(k)$ por par, $O(w*k) = 2.25*10^6$. Medido en el peor caso del enunciado ($k = 1500$, $w = 1500$, palabras casi identicas para que el fallo retroceda lo mas posible), 20 casos completos: 512 ms. Verificado contra una simulacion REAL del letrero, con BFS sobre los estados posibles, en 1500 casos con k de 1 a 4; y el solape de KMP contra comparar substrings a lo bruto en 20000 pares. Cero diferencias en ambas. Los dos samples dan 5 y 5, y el ejemplo del enunciado (CAT ATE TED con $k = 3$, que se logra metiendo CATED) da 5.

C++\CCPL\sign.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // Mayor L tal que los ultimos L caracteres de a son los primeros L de b.
5 int solape(const string &a, const string &b) {
6     string s = b + "#" + a;
7     int m = s.size();
8
9     vector<int> f(m, 0);
10
11     for (int i = 1; i < m; i++) {
12         int j = f[i - 1];
13
14         while (j > 0 && s[i] != s[j])
15             j = f[j - 1];
16
17         if (s[i] == s[j])
18             j++;
19
20         f[i] = j;
21     }
22
23     return f[m - 1];
24 }
25

```

9.5 Maximo Solape Sufijo-Prefijo (KMP) - Sign (cont.)

```

26 int main() {
27     ios::sync_with_stdio(false);
28     cin.tie(nullptr);
29
30     int casos;
31     cin >> casos;
32
33     while (casos--) {
34         int k, w;
35         cin >> k >> w;
36
37         vector<string> pal(w);
38         for (int i = 0; i < w; i++) cin >> pal[i];
39
40         // El letrero arranca vacio, asi que la primera palabra cuesta k.
41         long long total = k;
42
43         for (int i = 1; i < w; i++) {
44             total += k - solape(pal[i - 1], pal[i]);
45         }
46
47         cout << total << '\n';
48     }
49     return 0;
50 }

```

9.6 Monotonic Stack - Remove Duplicates1

Resuelve "Remove Duplicate Letters": devuelve el menor string lexicografico que contiene cada letra de s exactamente una vez. Usa una pila monotona, sacando letras mayores mientras todavia aparezcan mas adelante (cnt > 0) y no esten ya en el resultado.

C++\LeetCode\removeDuplicates1.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 class Solution {
6 public:
7     string removeDuplicateLetters(string s) {
8         // cnt for storing frequency of characters
9         // vis for marking visited characters
10        vector<int> cnt(26,0) , vis(26,0);
11
12        string res = "";
13        int n = s.size();
14
15        for(int i = 0; i < n; ++i)
16            cnt[s[i] - 'a']++;
17
18        for(int i = 0; i < n; ++i)
19        {
20            // decrease cnt of current character
21            cnt[s[i] - 'a']--;
22
23            // If character is not already
24            // in answer
25            if(!vis[s[i] - 'a'])
26            {
27                // Last character > s[i]
28                // and its count > 0
29                while(res.size() > 0 && res.back() > s[i] && cnt[res.back() - 'a'] > 0)
30                {
31                    // marking letter visited
32                    vis[res.back() - 'a'] = 0;
33                    res.pop_back();
34                }
35
36                // Add s[i] in res and
37                // mark it visited
38                res += s[i];
39                vis[s[i] - 'a'] = 1;

```

9.6 Monotonic Stack - Remove Duplicates1 (cont.)

```

40     }
41 }
42 // return resultant string
43 return res;
44 }
45 };

```

9.7 Palindromo Ignorando Simbolos - Just Palindromes

Resuelve "Just Palindromes!" (problema J, ICPC 2025): decir si una frase es palindroma ignorando espacios, puntuacion y mayusculas. Se filtra dejando solo letras y pasandolas a minuscula, y despues es el chequeo de dos punteros de siempre. Lo unico que hay que cuidar es la lectura: la frase trae ESPACIOS, asi que va con getline y no con >>, que cortaria en el primer espacio. DOS BORDES QUE EL ENUNCIADO PERMITE Y QUE CONVIENE PROBAR: - La linea VACIA (dice 0 <= N). Al filtrar queda la cadena vacia, que es palindroma: Y. - Una linea de puros simbolos, como "...." del sample. Tambien queda vacia: Y. Con la cadena vacia, r = t.size() - 1 vale -1 y el while no entra, asi que responde Y sin tocar memoria. Sale bien por como esta escrito el ciclo, pero vale saber que t.size() es SIN SIGNO: si en vez del while hubiera un for con <= y restas, ese -1 se volveria un numero enorme. El enunciado acota los caracteres a ASCII imprimible, asi que isalpha y tolower no dan problemas de localizacion. Verificado contra fuerza bruta en 5008 frases, incluidas la vacia, las de solo simbolos y palindromos armados a proposito con basura intercalada y mayusculas mezcladas.

C++\ICPC\2025J - Just Palindromes.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     string s;
9
10    while (getline(cin, s) && s != "") {
11
12        string t;
13
14        // Dejamos unicamente letras y pasamos a minusculas
15        for (char c : s) {
16            if (isalpha(c))
17                t += tolower(c);
18        }
19
20        bool ok = true;
21
22        int l = 0;
23        int r = t.size() - 1;
24
25        while (l < r) {
26            if (t[l] != t[r]) {
27                ok = false;
28                break;
29            }
30
31            l++;
32            r--;
33        }
34
35        cout << (ok ? 'Y' : 'N') << '\n';
36    }
37
38    return 0;
39 }

```

9.8 Palindromos No Solapados (Greedy) - Max Palindromes

Resuelve "Maximum Number of Non-overlapping Palindrome Substrings" (LeetCode 2472): dado s y un k , hay que escoger la mayor cantidad posible de subcadenas palindromas que no se solapen y que midan AL MENOS k . Barre r de izquierda a derecha y, apenas encuentra un palindromo valido que termine en r y empiece en o despues de "inicio", lo toma y salta con inicio = $r+1$. El greedy es optimo porque cerrar lo antes posible nunca quita opciones: deja el sufijo mas largo disponible para lo que venga. Cual l se escoja dentro del mismo r da igual, porque el corte queda en r . Detalle: r arranca en $k-1$, que es la primera posicion donde cabe algo de largo k . Sin tabla DP a proposito. Parece $O(n^3)$ pero en la practica es lo mas rapido: el chequeo de dos punteros aborta al primer par que no coincide (medido: ~2 pasos por llamada en promedio), y una coincidencia profunda implicaria un palindromo largo, que el greedy se llevaria de una, avanzando inicio. Medido con $n = 2000$: el peor adversario encontrado son 7.5 ms, contra 1-4 ms constantes de la version con tabla $pal[i][j]$, que ademas reserva n^2 bits de memoria. Si necesitas consultar MUCHOS rangos sueltos en vez de barrer una vez, ahi si conviene la tabla: esta en la ficha "Chequeo de Palindromo".

C++\LeetCode\maxPalindromes.cpp

```
1 class Solution {
2 public:
3     bool esPalindromo(string &s, int l, int r) {
4         while (l < r) {
5             if (s[l] != s[r]) return false;
6             l++;
7             r--;
8         }
9         return true;
10    }
11
12    int maxPalindromes(string s, int k) {
13        int n = s.size();
14        int ans = 0;
15        int inicio = 0;
16
17        for (int r = k - 1; r < n; r++) {
18            for (int l = inicio; l <= r - k + 1; l++) {
19                if (esPalindromo(s, l, r)) {
20                    ans++;
21                    inicio = r + 1;
22                    break;
23                }
24            }
25        }
26        return ans;
27    }
28 };
29 };
```

9.9 Parentesis Balanceados (Todas las Variantes)

O: (n) todas las variantes

Uso: valido(s); validoTipos(s); parejas(s)[i] = pareja de i, o -1

Todo problema de parentesis sale de UNA idea: el BALANCE, +1 por cada '(' y -1 por cada ')'. Una cadena es valida si y solo si el balance nunca baja de 0 y termina en 0. Las variantes de abajo son esa misma idea mirada desde distintos angulos. Todas verificadas contra fuerza bruta. CUAL USAR SEGUN LO QUE PIDAN: 1 valido un solo tipo: basta un contador, sin pila, O(n) y O(1) de memoria. 2 validoTipos (i){}: ahi si hace falta pila, porque "(i)" tiene balance bien en cada tipo por separado y aun asi es invalida. 3 parejas/profundidad la pareja de cada parentesis (para recorrer la estructura como un arbol) y el anidamiento maximo, que es el balance maximo. 4 minInserciones los ')' que llegan sin '(' abierto piden uno, y los '(' que quedan abiertos al final piden un ')'. Suma de sueltos. 5 minVolteos cambiar '(' por ')' o al reves. Tras cancelar las parejas queda "))))": con c cerrados y a abiertos sueltos son $\text{ceil}(c/2) + \text{ceil}(a/2)$. Con largo impar es imposible. 6 borrarMinimo se marcan los sueltos (los ')' sin pareja y los '(' que quedan en la pila) y se borran. Sirve aunque la cadena tenga letras mezcladas. 7 masLarga la pila guarda el indice ANTES de donde empieza el tramo valido actual; arranca en -1 y cada ')' suelto pasa a ser el nuevo tope. 8 contarSubcadenas si el ')' de i casa con el '(' de j, las validas que terminan en i son "(...)" mas cada valida que terminaba justo en j-1: $dp[i+1] = dp[j]+1$. 9 validoComodines '*' vale '(', ')' o nada: se lleva el balance MINIMO y el MAXIMO posibles. Si el maximo baja de 0 no hay forma; el minimo se sube a 0 porque nunca conviene pasar por negativo. Al final el minimo debe ser 0. 10 generar backtracking: se pone '(' mientras queden, y ')' si hay mas abiertos que cerrados. Sale en orden lexicografico y son Catalan(n). 11 kEsima sin generarlas todas: formas[i][b] = cuantas maneras hay de completar i caracteres desde balance b. Si k cabe en las que empiezan con '(', se pone '('; si no, se descuentan y se pone ')'. Hasta n de unos 30. 12 SegTree "subsecuencia balanceada mas larga en s[l..r]" con muchas consultas. Cada nodo guarda (parejas, '(' sobrantes, ')' sobrantes); al unir, los '(' sobrantes de la izquierda casan con los ')' sobrantes de la derecha. 13 maxSubsecuenciaTipos con varios tipos y hay que BORRAR o INSERTAR, el greedy de pila ya no es optimo: DP de intervalos $O(n^3)$. El minimo de inserciones para balancear es n menos ese largo. Contar las secuencias validas de n pares es Catalan(n); ver la ficha de Combinatoria.

C++\Notebook\Parentesis Balanceados.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 // 1) Un solo tipo.
6 bool valido(const string &s) {
7     int bal = 0;
8     for (char c : s) {
9         if (c == '(') bal++;
10        else bal--;
11        if (bal < 0) return false;
12    }
13    return bal == 0;
14 }
15
16 // 2) Varios tipos: la pila guarda los que abren.
17 bool validoTipos(const string &s) {
18     string pila;
19     for (char c : s) {
20         if (c == '(' || c == '[' || c == '{') {
21             pila.push_back(c);
22             continue;
23         }
24         char abre = '(';
25         if (c == ']') abre = '[';
26         if (c == '}') abre = '{';
27         if (pila.empty() || pila.back() != abre) return false;
28         pila.pop_back();
29     }
30     return pila.empty();
31 }
32
33 // 3) Pareja de cada parentesis (-1 si queda suelto) y anidamiento maximo.
34 vector<int> parejas(const string &s) {
35     vector<int> match(s.size(), -1), pila;
36     for (int i = 0; i < (int)s.size(); i++) {
37         if (s[i] == '(') {
38             pila.push_back(i);
```

9.9 Parentesis Balanceados (Todas las Variantes) (cont.)

```

38     } else if (!pila.empty()) {
39         match[i] = pila.back();
40         match[pila.back()] = i;
41         pila.pop_back();
42     }
43 }
44 }
45 return match;
46 }
47
48 int profundidad(const string &s) { // para cadenas validas
49     int bal = 0, mx = 0;
50     for (char c : s) {
51         if (c == '(') bal++;
52         else bal--;
53         mx = max(mx, bal);
54     }
55     return mx;
56 }
57
58 // 4) Minimo de inserciones para balancear.
59 int minInserciones(const string &s) {
60     int abiertos = 0, faltan = 0;
61     for (char c : s) {
62         if (c == '(') abiertos++;
63         else if (abiertos > 0) abiertos--;
64         else faltan++; // ')' sin pareja: pide un '('
65     }
66     return abiertos + faltan; // cada '(' abierto pide un ')'
67 }
68
69 // 5) Minimo de volteos '(' <-> ')' para balancear; -1 si es imposible.
70 int minVolteos(const string &s) {
71     if (s.size() % 2) return -1;
72     int abiertos = 0, cerrados = 0;
73     for (char c : s) {
74         if (c == '(') abiertos++;
75         else if (abiertos > 0) abiertos--;
76         else cerrados++;
77     }
78     return (cerrados + 1) / 2 + (abiertos + 1) / 2;
79 }
80
81 // 6) Borrar el minimo para que quede valida (devuelve una de las optimas).
82 string borrarMinimo(const string &s) {
83     vector<int> pila;
84     vector<bool> borrar(s.size(), false);
85     for (int i = 0; i < (int)s.size(); i++) {
86         if (s[i] == '(') {
87             pila.push_back(i);
88         } else if (s[i] == ')') {
89             if (!pila.empty()) pila.pop_back();
90             else borrar[i] = true;
91         }
92     }
93     for (int i : pila) borrar[i] = true;
94     string res;
95     for (int i = 0; i < (int)s.size(); i++)
96         if (!borrar[i]) res += s[i];
97     return res;
98 }
99
100 // 7) Largo de la subcadena valida mas larga.
101 int masLarga(const string &s) {
102     vector<int> pila = {-1}; // indice antes del tramo valido
103     int mejor = 0;
104     for (int i = 0; i < (int)s.size(); i++) {
105         if (s[i] == '(') {
106             pila.push_back(i);
107             continue;
108         }
109         pila.pop_back();
110         if (pila.empty()) pila.push_back(i); // ')' suelto: nuevo tope

```

9.9 Parentesis Balanceados (Todas las Variantes) (cont.)

```

111         else mejor = max(mejor, i - pila.back());
112     }
113     return mejor;
114 }
115
116 // 8) Cantidad de subcadenas validas (distintas por posicion).
117 int contarSubcadenas(const string &s) {
118     int n = s.size();
119     vector<ll> dp(n + 1, 0); // dp[i+1] = validas que terminan en i
120     vector<int> pila;
121     ll total = 0;
122     for (int i = 0; i < n; i++) {
123         if (s[i] == '(') {
124             pila.push_back(i);
125             continue;
126         }
127         if (pila.empty()) continue;
128         int j = pila.back();
129         pila.pop_back();
130         dp[i + 1] = dp[j] + 1;
131         total += dp[i + 1];
132     }
133     return total;
134 }
135
136 // 9) Con comodines '*' que valen '(', ')' o nada.
137 bool validoComodines(const string &s) {
138     int lo = 0, hi = 0; // balance minimo y maximo posibles
139     for (char c : s) {
140         if (c == '(') {
141             lo++;
142             hi++;
143         } else if (c == ')') {
144             lo--;
145             hi--;
146         } else {
147             lo--;
148             hi++;
149         }
150         if (hi < 0) return false;
151         lo = max(lo, 0);
152     }
153     return lo == 0;
154 }
155
156 // 10) Todas las validas de n pares, en orden lexicografico.
157 void generar(int n, int abiertos, int cerrados, string &cur, vector<string> &res) {
158     if ((int)cur.size() == 2 * n) {
159         res.push_back(cur);
160         return;
161     }
162     if (abiertos < n) {
163         cur.push_back('(');
164         generar(n, abiertos + 1, cerrados, cur, res);
165         cur.pop_back();
166     }
167     if (cerrados < abiertos) {
168         cur.push_back(')');
169         generar(n, abiertos, cerrados + 1, cur, res);
170         cur.pop_back();
171     }
172 }
173
174 // 11) La k-esima valida de n pares (k desde 1), en orden lexicografico.
175 string kEsimas(int n, ll k) {
176     vector<vector<ll>> formas(2 * n + 1, vector<ll>(2 * n + 2, 0));
177     formas[0][0] = 1;
178     for (int i = 1; i <= 2 * n; i++)
179         for (int b = 0; b <= 2 * n; b++) {
180             formas[i][b] = formas[i - 1][b + 1]; // poner '('
181             if (b > 0) formas[i][b] += formas[i - 1][b - 1]; // poner ')'
182         }

```

9.9 Parentesis Balanceados (Todas las Variantes) (cont.)

```

83 string res;
84 int b = 0;
85 for (int i = 2 * n; i > 0; i--) {
86     ll conAbre = formas[i - 1][b + 1];
87     if (k <= conAbre) {
88         res += '(';
89         b++;
90     } else {
91         k -= conAbre;
92         res += ')';
93         b--;
94     }
95 }
96 return res;
97 }
98
99 // 12) Subsecuencia balanceada mas larga en s[l..r], con muchas consultas.
100 struct Nodo {
101     int par, abre, cierra;
102 };
103
104 Nodo unir(Nodo a, Nodo b) {
105     int nuevos = min(a.abre, b.cierra);
106     return {a.par + b.par + nuevos, a.abre + b.abre - nuevos,
107            a.cierra + b.cierra - nuevos};
108 }
109
110 struct SegTree {
111     int n;
112     vector<Nodo> t;
113
114     SegTree(const string &s) : n(s.size()), t(4 * s.size()) {
115         build(1, 0, n - 1, s);
116     }
117
118     void build(int p, int l, int r, const string &s) {
119         if (l == r) {
120             if (s[l] == '(') t[p] = {0, 1, 0};
121             else t[p] = {0, 0, 1};
122             return;
123         }
124         int m = (l + r) / 2;
125         build(2 * p, l, m, s);
126         build(2 * p + 1, m + 1, r, s);
127         t[p] = unir(t[2 * p], t[2 * p + 1]);
128     }
129
130     Nodo query(int p, int l, int r, int ql, int qr) {
131         if (ql <= l && r <= qr) return t[p];
132         int m = (l + r) / 2;
133         if (qr <= m) return query(2 * p, l, m, ql, qr);
134         if (ql > m) return query(2 * p + 1, m + 1, r, ql, qr);
135         return unir(query(2 * p, l, m, ql, qr), query(2 * p + 1, m + 1, r, ql, qr));
136     }
137
138     int maxBalanceada(int l, int r) { // indices desde 0
139         return 2 * query(1, 0, n - 1, l, r).par;
140     }
141 };
142
143 // 13) Varios tipos: subsecuencia balanceada mas larga (DP de intervalos).
144 bool casan(char a, char b) {
145     return (a == '(' && b == ')') || (a == '[' && b == ']') || (a == '{' && b == '}');
146 }
147
148 int maxSubsecuenciaTipos(const string &s) {
149     int n = s.size();
150     if (n == 0) return 0;
151     vector<vector<int>> dp(n, vector<int>(n, 0));
152     for (int len = 2; len <= n; len++)
153         for (int i = 0; i + len - 1 < n; i++) {
154             int j = i + len - 1, mejor = 0;

```

9.9 Parentesis Balanceados (Todas las Variantes) (cont.)

```

255         if (casan(s[i], s[j])) mejor = dp[i + 1][j - 1] + 2;
256         for (int k = i; k < j; k++) mejor = max(mejor, dp[i][k] + dp[k + 1][j]);
257         dp[i][j] = mejor;
258     }
259     return dp[0][n - 1];
260 }
261
262 int main() {
263     ios::sync_with_stdio(false);
264     cin.tie(nullptr);
265
266     string s;
267     while (cin >> s) {
268         cout << s << ": valida=" << valido(s)
269              << " inserciones=" << minInserciones(s)
270              << " masLarga=" << masLarga(s)
271              << " subcadenas=" << contarSubcadenas(s)
272              << " queda=" << borrarMinimo(s) << '\n';
273     }
274     return 0;
275 }

```

9.10 Partition and Permutation - Policianacional

Divide una cadena en 3 partes, las reorganiza y verifica si forma otra cadena objetivo. Si es posible, imprime ambas descomposiciones ordenadas.

C++\Codeforces\policianacional.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9     cin >> n;
10
11     string s1, s2;
12     cin >> s1 >> s2;
13
14     // Probar todos los cortes posibles:
15     // A = [0,i)
16     // B = [i,j)
17     // C = [j,n)
18
19     for(int i = 1; i < n-1; i++)
20     {
21         for(int j = i+1; j < n; j++)
22         {
23             string A = s1.substr(0, i);
24             string B = s1.substr(i, j-i);
25             string C = s1.substr(j);
26
27             vector<string> p = {A,B,C};
28
29             sort(p.begin(), p.end());
30
31             do
32             {
33                 string formed = p[0] + p[1] + p[2];
34
35                 if(formed == s2)
36                 {
37                     cout << "YES\n";
38
39                     // Descomposición original del pasaporte
40                     cout << A << " " << B << " " << C << "\n";
41
42                     // Descomposición de la visa
43                     cout << p[0] << " " << p[1] << " " << p[2] << "\n";
44

```

9.10 Partition and Permutation - Policianacional (cont.)

```

45         return 0;
46     }
47 } while(next_permutation(p.begin(), p.end()));
48 }
49 }
50 }
51
52 cout << "NO\n";
53
54 return 0;
55 }

```

9.11 Remove Duplicate Letters - Remove Duplicates

Intento manual (greedy) de "Remove Duplicate Letters": busca construir el menor string lexicografico que contiene cada letra una sola vez, escogiendo la siguiente letra mas pequena posible que aun permita completar las letras restantes. Version explicativa de la idea greedy; la implementacion limpia con pila monotona esta en removeDuplicates1.cpp.

C++\LeetCode\removeDuplicates.cpp

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  class Solution {
6  public:
7      string removeDuplicateLetters(string s) {
8          vector<int> nums(s.size());
9          bool letras[26] = {false};
10         int cont[26] = {0};
11         for(char c : s)
12         {
13             nums.push_back(c - '0');
14             cont[c-'a']++;
15             letras[c - 'a'] = true;
16         }
17
18         int letmax = 0;
19         for (int i = 0; i < 26; i++)
20         {
21             letmax += letras[i];
22         }
23
24         char ini;
25         for (int i = 0; i < 26; i++)
26         {
27             if(cont[i] > 0)
28             {
29                 ini = i + 'a';
30                 break;
31             }
32         }
33
34
35         string god="";
36         int idx = 0;
37         bool contGod[26] = {false};
38         for(int i = 0 ; i < s.size() || god.size() < letmax; i++)
39         {
40             if(s[i] == ini && contGod[s[i]-'a'] == false)
41             {
42                 idx = i + 1;
43                 for (int i = ini - 'a' + 1; i < 26; i++)
44                 {
45                     if(cont[i] > 0 && contGod[i] == false)
46                     {
47                         ini = i + 'a';
48                         break;
49                     }
50                 }
51                 god+=s[i];
52                 contGod[s[i]-'a'] = true;

```

9.11 Remove Duplicate Letters - Remove Duplicates (cont.)

```

53     }
54     else if(cont[s[i]-'a']==1 && contGod[s[i]-'a'] == false)
55     {
56         char mini = s[i];
57         int idxMini = i;
58         while(idx < i)
59         {
60             for(int j = idx; j < i; j++)
61             {
62                 if(s[j] < mini && contGod[s[j]-'a'] == false)
63                 {
64                     mini = s[j];
65                     idxMini = j;
66                 }
67             }
68             if(mini != s[i])
69             {
70                 god+= s[idxMini];
71                 contGod[s[idxMini]-'a'] = true;
72
73                 idx = idxMini + 1;
74                 mini = s[i];
75                 idxMini = i;
76             }
77             else
78             {
79                 break;
80             }
81         }
82
83         for(int j = idx; j <= i; j++)
84         {
85             if(s[j] == s[i])
86             {
87                 god+=s[j];
88                 idx = j + 1;
89                 contGod[s[j]-'a'] = true;
90                 break;
91             }
92         }
93     }
94     cont[s[i]-'a']--;
95 }
96
97 return god;
98 }
99
100 };

```

9.12 Rotacion Minima (Dos Punteros) - Fingerprints

Resuelve "Fingerprints" (problema F, ICPC 2025): las huellas se guardan de forma circular, así que ABCD, BCDA, CDAB y DABC son la misma. Hay que quedarse con una sola copia de cada huella distinta, representada por su rotacion lexicograficamente menor. La rotacion minima sale en $O(n)$ con dos candidatos i y j sobre la cadena duplicada: se comparan caracter por caracter y, al diferir en la posicion k , el perdedor salta $k+1$ posiciones de una. Ese salto es lo que la hace lineal, y esta explicada con detalle en la ficha "Minimal Rotation" de CSES, en este mismo cuaderno. EL ERROR QUE TENIA ESTE ARCHIVO ERA DE ORDEN, NO DE ALGORITMO: guardaba las canonicas en un `set<string>` y las imprimia recorriendolo, o sea en orden ALFABETICO. El enunciado pide el orden en que aparecen en la ENTRADA, y el sample lo delata: con ABCD, BCDA, XYZ, FFFF, ZXY la respuesta es ABCD XYZ FFFF y el set devolvía ABCD FFFF XYZ. La solucion es la de siempre para "sin repetidos pero en orden de llegada": un set aparte solo para saber si ya se vio, y un vector que conserva el orden. El `insert(...).second` dice en una sola operacion si era nueva. Verificado contra fuerza bruta (rotacion minima comparando todas las rotaciones) en 500 casos. Medido: 3 casos de 100 huellas de 50000 caracteres en 33 ms.

C++\ICPC\2025\F - Fingerprints.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  string minimalRotation(string s) {

```

9.12 Rotacion Minima (Dos Punteros) - Fingerprints (cont.)

```

5   int n = s.size();
6
7   s += s;
8
9   int i = 0, j = 1;
10
11  while (i < n && j < n) {
12      int k = 0;
13
14      while (k < n && s[i + k] == s[j + k])
15          k++;
16
17      if (k == n)
18          break;
19
20      if (s[i + k] > s[j + k])
21          i = i + k + 1;
22      else
23          j = j + k + 1;
24
25      if (i == j)
26          j++;
27  }
28
29  int pos = min(i, j);
30
31  return s.substr(pos, n);
32 }
33
34 int main() {
35     ios::sync_with_stdio(false);
36     cin.tie(nullptr);
37
38     int n;
39
40     while (cin >> n && n != 0) {
41
42         // El set solo sirve para detectar repetidas. El ORDEN de salida es el
43         // de aparicion en la entrada, asi que se guarda aparte en un vector:
44         // imprimir el set directamente las sacaria en orden alfabetico.
45         set<string> vistas;
46         vector<string> ans;
47
48         for (int i = 0; i < n; i++) {
49             int m;
50             string s;
51
52             cin >> m >> s;
53
54             string canon = minimalRotation(s);
55
56             if (vistas.insert(canon).second)
57                 ans.push_back(canon);
58         }
59
60         for (const string &s : ans)
61             cout << s << '\n';
62     }
63
64     return 0;
65 }

```

9.13 Simulation - Process Str

Procesa una cadena aplicando operaciones sobre un string acumulador: '*' borra el ultimo caracter, '#' duplica el contenido actual, '%' lo invierte, y cualquier otro caracter se agrega al final. Devuelve el string resultante.

C++\LeetCode\processStr.cpp

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4

```

9.13 Simulation - Process Str (cont.)

```

5  class Solution {
6  public:
7      string processStr(string s) {
8          string god = "";
9          for(char a : s)
10             {
11                 if(a == '*')
12                     {
13                         if(god.size() > 0)
14                             {
15                                 god.erase(god.size() - 1);
16                             }
17                     }
18                 else if(a == '#')
19                     {
20                         god += god;
21                     }
22                 else if(a == '%')
23                     {
24                         reverse(god.begin(), god.end());
25                     }
26                 else
27                     {
28                         god += a;
29                     }
30             }
31         return god;
32     }
33 };

```

9.14 Suffix Array + LCP - Couple Of Bip Bop

Resuelve "Couple of BipBop" (Day 3, problema C - Maratona SBC de Programacao 2024): Bob y Charlie empiezan a bailar cada uno en un verso aleatorio de una cancion de N versos y siguen la coreografia hasta desincronizarse o llegar al final de la cancion; hay que calcular el valor esperado de versos que bailaran en sincronia, expresado como fraccion irreducible P/Q, si ambos eligen su verso inicial de forma uniforme e independiente. El valor esperado equivale a sumar, sobre todos los pares de posiciones iniciales (i,j), el largo del prefijo comun de los sufijos que empiezan en i y en j. El codigo construye el suffix array (suffix_array) y el arreglo de LCP (kasai) de la secuencia de movimientos, y luego usa una pila monotona para sumar en O(N) la suma de minimos de LCP sobre todos los subarreglos (sumPairs), equivalente a la suma de LCP entre todos los pares de sufijos.

C++\Training_Camp_2026\Day 3\C - Couple of BipBop.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5
6  ll gcdll(ll a, ll b) {
7      while (b) {
8          a %= b;
9          swap(a, b);
10     }
11     return a;
12 }
13
14 vector<int> suffix_array(vector<int> s) {
15     int n = s.size();
16     vector<int> sa(n), rnk(n), tmp(n);
17
18     for (int i = 0; i < n; i++) {
19         sa[i] = i;
20         rnk[i] = s[i];
21     }
22
23     for (int k = 1; k <= 1) {
24         auto cmp = [&](int a, int b) {
25             if (rnk[a] != rnk[b])
26                 return rnk[a] < rnk[b];
27
28             int ra = a + k < n ? rnk[a + k] : -1;

```

9.14 Suffix Array + LCP - Couple Of Bip Bop (cont.)

```

29     int rb = b + k < n ? rnk[b + k] : -1;
30     return ra < rb;
31 };
32
33 sort(sa.begin(), sa.end(), cmp);
34
35 tmp[sa[0]] = 0;
36
37 for (int i = 1; i < n; i++)
38     tmp[sa[i]] = tmp[sa[i-1]] + cmp(sa[i-1], sa[i]);
39
40 rnk = tmp;
41
42 if (rnk[sa[n-1]] == n - 1)
43     break;
44 }
45
46 return sa;
47 }
48
49 vector<int> kasai(vector<int> &s, vector<int> &sa) {
50     int n = s.size();
51
52     vector<int> rank(n);
53     for (int i = 0; i < n; i++)
54         rank[sa[i]] = i;
55
56     vector<int> lcp(n - 1);
57
58     int h = 0;
59
60     for (int i = 0; i < n; i++) {
61         int r = rank[i];
62
63         if (r == n - 1)
64             continue;
65
66         int j = sa[r + 1];
67
68         while (i + h < n && j + h < n && s[i+h] == s[j+h])
69             h++;
70
71         lcp[r] = h;
72
73         if (h)
74             h--;
75     }
76
77     return lcp;
78 }
79
80 int main() {
81     ios::sync_with_stdio(false);
82     cin.tie(nullptr);
83
84     int N;
85     cin >> N;
86
87     vector<int> a(N);
88
89     for (int i = 0; i < N; i++)
90         cin >> a[i];
91
92     auto sa = suffix_array(a);
93     auto lcp = kasai(a, sa);
94
95
96
97     // suma de LCP entre pares distintos
98     ll sumPairs = 0;
99
100    vector<pair<ll,ll>> st;

```

9.14 Suffix Array + LCP - Couple Of Bip Bop (cont.)

```

01    ll accumulated = 0;
02
03    for (int i = 0; i < (int)lcp.size(); i++) {
04
05        ll cnt = 1;
06        ll val = lcp[i];
07
08        while (!st.empty() && st.back().first >= val) {
09            accumulated -= st.back().first * st.back().second;
10            cnt += st.back().second;
11            st.pop_back();
12        }
13
14        st.push_back({val, cnt});
15
16        accumulated += val * cnt;
17
18        sumPairs += accumulated;
19    }
20
21
22    ll numerator = 1LL * N * (N + 1) / 2 + 2 * sumPairs;
23    ll denominator = 1LL * N * N;
24
25    ll g = gcdll(numerator, denominator);
26
27    cout << numerator / g << '/' << denominator / g << '\n';
28
29    return 0;
30 }

```

9.15 Suffix Automaton - Beautiful Words

Resuelve "Beautiful Words" (Day 4, problema A - Contest 04 [Avanzados]): dado un string A de N caracteres y un conjunto S de M strings, el score de una permutacion ciclica B_i de A es el largo del substring mas largo de B_i que tambien aparece como substring de algun string de S; hay que hallar el minimo score sobre todas las N permutaciones ciclicas posibles. Construye un suffix automaton (struct SAM) con las cadenas de S invertidas y separadores entre ellas, recorre el string A duplicado e invertido para obtener en G[i] el largo del match mas largo terminado en cada posicion, y usa busqueda binaria sobre la respuesta (funcion feasible) para encontrar la minima cota que deje una ventana de N caracteres sin matches mas largos.

C++\Training_Camp_2026\Day 4\A - Beautiful Words.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // Suffix automaton over an alphabet of 26 lowercase letters + 1 separator symbol.
5 struct SAM {
6     static const int ALPHA = 27;
7     vector<array<int, ALPHA>> trans;
8     vector<int> len, link_;
9     int last;
10
11     void init(int maxStates) {
12         trans.clear(); len.clear(); link_.clear();
13         trans.reserve(maxStates);
14         len.reserve(maxStates);
15         link_.reserve(maxStates);
16         newNode();
17         len[0] = 0; link_[0] = -1;
18         last = 0;
19     }
20     int newNode() {
21         array<int, ALPHA> a; a.fill(-1);
22         trans.push_back(a);
23         len.push_back(0);
24         link_.push_back(-1);
25         return (int)trans.size() - 1;
26     }
27     void extend(int c) {
28         int cur = newNode();
29         len[cur] = len[last] + 1;

```

9.15 Suffix Automaton - Beautiful Words (cont.)

```

30     int p = last;
31     while (p != -1 && trans[p][c] == -1) {
32         trans[p][c] = cur;
33         p = link_[p];
34     }
35     if (p == -1) {
36         link_[cur] = 0;
37     } else {
38         int q = trans[p][c];
39         if (len[p] + 1 == len[q]) {
40             link_[cur] = q;
41         } else {
42             int clone = newNode();
43             len[clone] = len[p] + 1;
44             link_[clone] = link_[q];
45             trans[clone] = trans[q];
46             while (p != -1 && trans[p][c] == q) {
47                 trans[p][c] = clone;
48                 p = link_[p];
49             }
50             link_[q] = clone;
51             link_[cur] = clone;
52         }
53     }
54     last = cur;
55 }
56 };
57
58 inline int charIdx(char c) {
59     if (c >= 'a' && c <= 'z') return c - 'a';
60     return 26; // separator
61 }
62
63 int main() {
64     ios_base::sync_with_stdio(false);
65     cin.tie(nullptr);
66
67     int N, M;
68     cin >> N >> M;
69     string A;
70     cin >> A;
71     vector<string> S(M);
72     long long totalLen = 0;
73     for (auto& s : S) { cin >> s; totalLen += (long long)s.size(); }
74
75     int maxStates = 2 * (int)(totalLen + M + 5) + 10;
76     SAM sam;
77     sam.init(maxStates);
78
79     for (int i = 0; i < M; i++) {
80         string rev = S[i];
81         reverse(rev.begin(), rev.end());
82         for (char c : rev) sam.extend(charIdx(c));
83         if (i + 1 < M) sam.extend(26); // separator between strings
84     }
85
86     int n2 = 2 * N;
87     string doubledA = A + A;
88     string revDoubledA(doubledA.rbegin(), doubledA.rend());
89
90     vector<int> G(n2);
91     {
92         int v = 0, l = 0;
93         for (int i = 0; i < n2; i++) {
94             int c = charIdx(revDoubledA[(size_t)i]);
95             while (v != 0 && sam.trans[v][c] == -1) {
96                 v = sam.link_[v];
97                 l = sam.len[v];
98             }
99             if (sam.trans[v][c] != -1) {
100                 v = sam.trans[v][c];
101                 l++;

```

9.15 Suffix Automaton - Beautiful Words (cont.)

```

102     } else {
103         l = 0;
104     }
105     G[i] = l;
106 }
107 }
108
109 vector<int> C(n2);
110 for (int k = 0; k < n2; k++) C[k] = G[n2 - 1 - k];
111
112 auto feasible = [&](int ans) -> bool {
113     int W = N - ans;
114     if (W <= 0) return true;
115     int runStart = -1;
116     for (int k = 0; k <= n2; k++) {
117         bool bad = (k == n2) || (C[k] > ans);
118         if (!bad) {
119             if (runStart == -1) runStart = k;
120         } else {
121             if (runStart != -1) {
122                 int runEnd = k - 1;
123                 int runLen = runEnd - runStart + 1;
124                 if (runLen >= W && runStart <= N - 1) return true;
125             }
126             runStart = -1;
127         }
128     }
129     return false;
130 };
131
132 int lo = 0, hi = N;
133 while (lo < hi) {
134     int mid = (lo + hi) / 2;
135     if (feasible(mid)) hi = mid; else lo = mid + 1;
136 }
137 cout << lo << "\n";
138 return 0;
139 }

```

9.16 Trie + DP de Segmentación (Maximo) - Spacebar Tokenizer

Resuelve "Spacebar Tokenizer" (problema E, ICPC 2024): partir cada cadena en tokens para que la suma de puntajes sea MAXIMA. Cada token del diccionario vale su puntaje, y el enunciado dice que CUALQUIER otro token vale 0: se puede cortar donde sea. $dp[i]$ = mejor puntaje para el prefijo de largo i . Hay dos transiciones: avanzar una letra sumando 0 (esa letra queda dentro de un token no reconocido) y, por cada palabra del diccionario que empiece en i y termine en j , proponer $dp[i] + \text{puntaje para } dp[j+1]$. LA TRAMPA DEL PROBLEMA ES ESA TRANSICION DE 0. Sin ella se exige que TODA la cadena se parta en palabras del diccionario, y el segundo caso del sample ("theonlytokenizeripo", donde "theonly" no esta) sale imposible en vez de 13. La primera version de este archivo tenia ese error: daba -1000000000 en el sample. Verificada la corregida contra todas las formas de cortar en 1015 frases chicas, sin fallos. Las palabras van en un trie para que desde cada posición se camine una sola vez siguiendo la cadena: cada nodo con puntaje en el camino es una palabra que encaja, y el recorrido se corta en cuanto el trie no tiene la letra. Es exactamente el esqueleto de "Word Combinations" de CSES (que esta en este cuaderno) con max en vez de suma. Vale reconocer que "segmentar en palabras de un diccionario" es siempre esto, cambie o no lo que se optimiza. Con la transición de 0 toda posición es alcanzable, así que ya no hace falta un valor de "imposible". Y dp va en long long porque el enunciado no acota los puntajes. El trie se reconstruye por caso de prueba, ya que el diccionario cambia.

C++\ICPC2024\E - Spacebar Tokenizer.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Node {
5     int next[26];
6     int score;
7
8     Node() {
9         fill(next, next + 26, -1);
10        score = -1;
11    }

```

9.16 Trie + DP de Segmentacion (Maximo) - Spacebar Tokenizer (cont.)

```

2   };
3
4   int main() {
5       ios::sync_with_stdio(false);
6       cin.tie(nullptr);
7
8       int m, s;
9
10      while (cin >> m >> s && (m || s)) {
11
12          vector<Node> trie(1);
13
14          // Insertar palabras
15          for (int i = 0; i < m; i++) {
16
17              string word;
18              int value;
19
20              cin >> word >> value;
21
22              int node = 0;
23
24              for (char c : word) {
25
26                  int x = c - 'a';
27
28                  if (trie[node].next[x] == -1) {
29                      trie[node].next[x] = trie.size();
30                      trie.emplace_back();
31                  }
32
33                  node = trie[node].next[x];
34              }
35
36              trie[node].score = value;
37          }
38
39          while (s--) {
40
41              string a;
42              cin >> a;
43
44              int n = a.size();
45
46              vector<long long> dp(n + 1, 0);
47
48              for (int i = 0; i < n; i++) {
49
50                  // a[i] como parte de un token NO reconocido: suma 0
51                  dp[i + 1] = max(dp[i + 1], dp[i]);
52
53                  int node = 0;
54
55                  for (int j = i; j < n; j++) {
56
57                      int x = a[j] - 'a';
58
59                      if (trie[node].next[x] == -1)
60                          break;
61
62                      node = trie[node].next[x];
63
64                      if (trie[node].score != -1) {
65
66                          dp[j + 1] = max(
67                              dp[j + 1],
68                              dp[i] + trie[node].score
69                          );
70                      }
71                  }
72              }
73          }
74      }
75  }

```

9.16 Trie + DP de Segmentacion (Maximo) - Spacebar Tokenizer (cont.)

```

84      cout << dp[n] << '\n';
85  }
86  }
87
88      return 0;
89  }

```

9.17 Z-Function - A String Problem

Resuelve "a String Problem" (Codeforces): cuenta cuantos $t \neq "a"$ permiten partir s en trozos que sean t o $"a"$, usando al menos un t . Si s es todo 'a', cualquier bloque de 2 a n letras sirve, o sea $n-1$. Si no, todo t valido tiene que cubrir el primer caracter distinto de 'a', asi que se recorta el prefijo de 'a' (largo p) y se prueba como candidato cada prefijo $x[0..len-1]$ del resto. La Z-function decide en $O(1)$ si el candidato reaparece en una posicion $(z[pos] \geq len)$, y $next[]$ salta de un bloque de 'a' al siguiente caracter distinto de 'a' tambien en $O(1)$. Como el recorrido avanza de len en len , cada longitud cuesta $O(m/len)$ y el total queda $O(m \log m)$. mn guarda el bloque de 'a' mas corto que antecede a alguna ocurrencia (contando el prefijo p): el candidato puede llevar hasta mn letras 'a' pegadas al frente, y por eso cada len que funciona aporta $mn+1$ en vez de 1.

C++\Codeforces\AStringProblem.cpp

```

1   #include <bits/stdc++.h>
2   using namespace std;
3
4   vector<int> z_function(string s) {
5       int n = s.size();
6       vector<int> z(n);
7
8       for (int i = 1, l = 0, r = 0; i < n; i++) {
9           if (i <= r)
10              z[i] = min(r - i + 1, z[i - l]);
11
12           while (i + z[i] < n && s[z[i]] == s[i + z[i]])
13              z[i]++;
14
15           if (i + z[i] - 1 > r) {
16               l = i;
17               r = i + z[i] - 1;
18           }
19       }
20
21       return z;
22  }
23
24  int main() {
25      ios::sync_with_stdio(false);
26      cin.tie(nullptr);
27
28      int T;
29      cin >> T;
30
31      while (T--) {
32          string s;
33          cin >> s;
34
35          int n = s.size();
36          int p = 0;
37
38          while (p < n && s[p] == 'a')
39              p++;
40
41          if (p == n) {
42              cout << n - 1 << '\n';
43              continue;
44          }
45
46          string x = s.substr(p);
47          int m = x.size();
48
49          vector<int> z = z_function(x);
50
51          vector<int> nxt(m + 1, m);
52      }

```

9.17 Z-Function - A String Problem (cont.)

```

53     for (int i = m - 1; i >= 0; i--) {
54         if (x[i] != 'a')
55             nxt[i] = i;
56         else
57             nxt[i] = nxt[i + 1];
58     }
59
60     long long ans = 0;
61
62     for (int len = 1; len <= m; len++) {
63         int pos = 0;
64         int mn = p;
65         bool ok = true;
66
67         while (true) {
68             if (pos + len > m) {
69                 ok = false;
70                 break;
71             }
72
73             if (pos != 0 && z[pos] < len) {
74                 ok = false;
75                 break;
76             }
77
78             pos += len;
79
80             int next = nxt[pos];
81
82             if (next == m)
83                 break;
84
85             mn = min(mn, next - pos);
86             pos = next;
87         }
88
89         if (ok)
90             ans += mn + 1;
91     }
92
93     cout << ans << '\n';
94 }
95
96 return 0;
97 }

```

10 - Greedy

10.1 Binary Search - Maratona Brasileira De Popcorn

Resuelve "Maratona Brasileira de Popcorn" (Day 1, problema M - Maratona de Programacao da SBC 2019): N bolsas de palomitas en fila deben repartirse en C tramos contiguos (uno por competidor), cada competidor come a lo sumo T unidades por segundo, y se quiere minimizar el tiempo total (el maximo de todas las bolsas que come cada competidor, dividido T). Búsqueda binaria sobre la cota maxima S que puede cargar un competidor: la funcion gruposNecesarios hace un particionado greedy (como el problema clasico de asignacion de libros/pintores) contando cuantos tramos contiguos hacen falta para que ninguno supere S, y se busca el menor S factible con a lo sumo C tramos. La respuesta final es $\text{ceil}(S / T)$.

C++\Training_Camp_2026\Day 1\M - Maratona Brasileira de Popcorn.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main(){
5      ios::sync_with_stdio(false);
6      cin.tie(nullptr);
7
8      int n, c;
9      long long t;
10     cin >> n >> c >> t;
11

```

10.1 Binary Search - Maratona Brasileira De Popcorn (cont.)

```

12     vector<long long> p(n);
13     long long lo = 0, hi = 0;
14     for (int i = 0; i < n; i++) {
15         cin >> p[i];
16         lo = max(lo, p[i]); // la bolsa más grande fija una cota inferior
17         hi += p[i]; // un solo competidor comiendo todo fija la cota superior
18     }
19
20     // Dada una cota S, calcula cuántos grupos contiguos (competidores)
21     // se necesitan para que ningún grupo tenga suma mayor que S.
22     auto gruposNecesarios = [&](long long S) -> long long {
23         long long grupos = 1;
24         long long acumulado = 0;
25         for (int i = 0; i < n; i++) {
26             if (acumulado + p[i] > S) {
27                 grupos++;
28                 acumulado = p[i];
29             } else {
30                 acumulado += p[i];
31             }
32         }
33         return grupos;
34     };
35
36     // Búsqueda binaria de la mínima "suma máxima" S alcanzable
37     // usando a lo sumo C grupos (competidores).
38     while (lo < hi) {
39         long long mid = lo + (hi - lo) / 2;
40         if (gruposNecesarios(mid) <= c) {
41             hi = mid;
42         } else {
43             lo = mid + 1;
44         }
45     }
46
47     long long S = lo; // suma máxima óptima que come un competidor
48     long long respuesta = (S + t - 1) / t; // techo(S / T)
49
50     cout << respuesta << "\n";
51     return 0;
52 }

```

10.2 Bit Manipulation - Lecographically Maximum

Resuelve "Lecographically Maximum" (Day 3, problema L - Maratona SBC de Programacao 2024): dada una lista de N enteros y una operacion que intercambia el k-esimo bit entre dos elementos cualquiera de la lista, hay que encontrar la secuencia lexicograficamente maxima alcanzable aplicando la operacion tantas veces como se quiera. Como la operacion permite mover libremente cualquier bit entre cualquier par de posiciones, lo unico que se conserva es, para cada bit b, cuantos elementos originalmente lo tenian encendido (cnt[b]). El codigo aplica una estrategia greedy: para cada bit b, enciende ese bit en los primeros cnt[b] elementos del resultado (ans[i] |= (1 << b)), lo que maximiza lexicograficamente la secuencia al concentrar todos los bits posibles en las posiciones mas a la izquierda.

C++\Training_Camp_2026\Day 3\L - Lecographically Maximum.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios::sync_with_stdio(false);
6      cin.tie(nullptr);
7
8      int N;
9      cin >> N;
10
11     vector<int> cnt(31, 0);
12
13     for (int i = 0; i < N; i++) {
14         int x;
15         cin >> x;
16         for (int b = 0; b <= 30; b++)
17             if (x & (1 << b))

```

10.2 Bit Manipulation - Lecographically Maximum (cont.)

```

8         cnt[b]++;
9     }
20
21     vector<int> ans(N, 0);
22
23     for (int b = 0; b <= 30; b++)
24         for (int i = 0; i < cnt[b]; i++)
25             ans[i] |= (1 << b);
26
27     for (int i = 0; i < N; i++) {
28         if (i) cout << ' ';
29         cout << ans[i];
30     }
31     cout << '\n';
32 }

```

10.3 Budget Optimization - Posters

Dado un presupuesto c y costos a, b, calcula el maximo de posters que se pueden comprar comprando al menos uno de cada tipo.

C++\RPC\Posters.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using int64 = long long;
5
6 int main() {
7     long long a,b,c;
8
9     cin >> a >> b >> c;
10
11     cout << (min((c-a) / b, (c-b) / a) < 0 ? 0 : min((c-a) / b, (c-b) / a)) << endl;
12
13
14
15     return 0;
16 }

```

10.4 Buy Low Sell High (Comprar y Vender Acciones)

O: (n) las tres primeras, (n*K) la de K transacciones

Uso: maxProfit(p) -> {ganancia, {compra, venta}}; maxProfitK(p, K)

Precio de una accion por dia: comprar un dia y vender un dia POSTERIOR (una transaccion) maximizando la ganancia. Un solo recorrido O(n). O(1) de memoria: se guarda el dia mas barato visto hasta ahora (el mejor momento para haber comprado) y cada dia se prueba vender hoy contra ese minimo. Si el precio solo baja, la respuesta es 0 (no operar). Es Kadane disfrazado: la ganancia es la suma maxima de un subarreglo de d[i] = p[i]-p[i-1]. Variantes tipicas (en todas no se puede tener dos acciones a la vez): - Ilimitadas: sumar cada subida max(0, d[i]); toda subida se puede cobrar. - Con comision por transaccion: DP de dos estados, cash (sin accion) y hold (con accion). - A lo sumo K transacciones: los mismos dos estados por cada transaccion, O(n*K). Con K >= n/2 ya no limita y equivale a ilimitadas (O(n)).

C++\Notebook\Buy Low Sell High.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 typedef long long ll;
6
7 const ll NEG = LLONG_MIN / 4;
8
9 // Una transaccion: {ganancia, {dia de compra, dia de venta}} (dias -1 si no conviene operar)
10 pair<ll, pair<int, int>> maxProfit(const vector<ll> &p)
11 {
12     ll best = 0;
13     int low = 0, buy = -1, sell = -1;
14     for (int i = 1; i < (int)p.size(); i++)
15     {
16         if (p[i] < p[low]) low = i;
17         else if (p[i] - p[low] > best)
18         {

```

10.4 Buy Low Sell High (Comprar y Vender Acciones) (cont.)

```

8         best = p[i] - p[low];
9         buy = low;
10        sell = i;
11    }
12    }
13    return {best, {buy, sell}};
14 }
15
16 // Transacciones ilimitadas
17 ll maxProfitUnlimited(const vector<ll> &p)
18 {
19     ll s = 0;
20     for (int i = 1; i < (int)p.size(); i++) s += max(0LL, p[i] - p[i - 1]);
21     return s;
22 }
23
24 // Ilimitadas, pagando `fee` por cada transaccion completa
25 ll maxProfitFee(const vector<ll> &p, ll fee)
26 {
27     ll cash = 0, hold = NEG;
28     for (ll x : p)
29     {
30         hold = max(hold, cash - x);
31         cash = max(cash, hold + x - fee);
32     }
33     return cash;
34 }
35
36 // A lo sumo K transacciones
37 ll maxProfitK(const vector<ll> &p, int K)
38 {
39     if (K <= 0) return 0;
40     if (2 * K >= (int)p.size()) return maxProfitUnlimited(p);
41     vector<ll> hold(K + 1, NEG), cash(K + 1, 0);
42     for (ll x : p)
43     {
44         for (int t = 1; t <= K; t++)
45         {
46             hold[t] = max(hold[t], cash[t - 1] - x);
47             cash[t] = max(cash[t], hold[t] + x);
48         }
49     }
50     return cash[K];
51 }
52
53 int main()
54 {
55     ios_base::sync_with_stdio(false);
56     cin.tie(nullptr);
57
58     int n;
59     cin >> n;
60     vector<ll> p(n);
61     for (auto &x : p) cin >> x;
62
63     auto r = maxProfit(p);
64     cout << r.first << "\n";
65     cout << r.second.first << " " << r.second.second << "\n"; // -1 -1 si no se opera
66     return 0;
67 }

```

10.5 Constructive Algorithms - Nomad

Resuelve "Nomad" (Day 10, problema H - Contest 09 [Avanzados]): Kai debe dormir exactamente D noches alternando entre N lugares, cada uno con un límite d_i de días CONSECUTIVOS que tolera antes de expulsarlo; hay que construir un plan de D días que nunca exceda ningún límite, o reportar -1 si es imposible. Busca los primeros dos índices d1, d2 con tolerancia distinta de cero: si hay al menos dos lugares utilizables, alterna entre ellos (d1+1, d2+1, d1+1, ...) durante los D días, lo que garantiza como máximo 1 día consecutivo en cada uno y por lo tanto siempre respeta cualquier límite >= 1. Si solo hay un lugar utilizable, la única opción es quedarse fijo ahí todos los D días, lo cual solo es válido si su tolerancia [d1] alcanza para cubrir los D días consecutivos; en cualquier otro caso (ningún lugar utilizable, o el único disponible no aguanta los D días) imprime -1.

C++\Training_Camp_2026\Day 10\H - Nomad.py

```
1 n,m = list(map(int,input().split()))
2 l = list(map(int,input().split()))
3 d1 = -1
4 d2 = -1
5 for i in range(len(l)):
6     if(l[i]!= 0 and d1 == -1):
7         d1 = i
8     elif(l[i]!= 0 and d1 != -1):
9         d2 = i
10        break
11 if(d2 == -1):
12     if(d1!= -1 and l[d1]>=m):
13         for i in range(m):
14             print(d1+1, end=" ")
15     else:
16         print(-1)
17 else:
18     for i in range(m):
19         if(i%2 == 0):
20             print(d1+1, end=" ")
21     else:
22         print(d2+1, end=" ")
```

10.6 DP - Exhibition Of Clownfish

Resuelve "Exhibition of Clownfish" (Day 1, problema E - Maratona de Programacao da SBC 2019): N peceras con M machos y F hembras cada una; si al final de la noche una pecera queda con machos pero sin hembras, uno de ellos muta a hembra. Zelius puede mover un pez por noche entre peceras y quiere el número mínimo de movimientos para que todos los peces terminen siendo hembra. Para cada pecera define v = 1 + M - F (cuanto "aporta" dejarla vacía de machos vs. usarla como colector) y separa peceras vacías, solo-hembras y mixtas (con v positivo o negativo). Arma greedily una "familia" alrededor de un colector maximizando la suma de v de las mixtas positivas mas las vacías disponibles, y cuando faltan colectores vacíos usa una DP tipo knapsack (dp[j] = costo mínimo en v para "cubrir" un déficit de peso j) para decidir que peceras mixtas conviene sacrificar. El total de movimientos es 2*S menos el mejor ahorro encontrado (S = total de machos).

C++\Training_Camp_2026\Day 1\E - Exhibition of Clownfish.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main(){
5     ios_base::sync_with_stdio(false);
6     cin.tie(NULL);
7
8     int n;
9     if(!(cin >> n)) return 0;
10
11     vector<long long> M(n), F(n);
12     long long S = 0; // total males
13     for(int i = 0; i < n; i++){
14         cin >> M[i] >> F[i];
15         S += M[i];
16     }
17
18     if(S == 0){ cout << 0 << "\n"; return 0; } // ya todas hembras
19
20     long long E = 0, femonly = 0, negmixed = 0;
21     long long PosV = 0, PosM = 0; // suma de v y de M sobre "mixtos positivos"
22     long long bestSingle = LLONG_MIN; // mejor v de una sola cámara
```

10.6 DP - Exhibition Of Clownfish (cont.)

```
23     vector<pair<long long,long long>> pos; // (peso = M, valor = v) de mixtos con v>0
24
25     for(int i = 0; i < n; i++){
26         long long v = 1 + M[i] - F[i];
27         bestSingle = max(bestSingle, v);
28         if(M[i] == 0 && F[i] == 0) E++; // vacío
29         else if(M[i] == 0) femonly++; // solo hembras
30         else { // mixto (M>0 => F>0 por restricción)
31             if(v > 0){ PosV += v; PosM += M[i]; pos.push_back({M[i], v}); }
32             else negmixed++;
33         }
34     }
35
36     long long FM0 = S - PosM; // machos libres provenientes de mixtos con v<=0
37
38     // Familia principal: max sobre w in [0,E] de bestPv(S-w) + w
39     long long best_family = PosV + min(E, FM0); // w <= FM0 : incluir todos los positivos
40
41     if(E > FM0){
42         long long T = E - FM0; // T <= E <= n <= 3000
43         const long long INF = LLONG_MAX / 4;
44         vector<long long> dp(T + 1, INF);
45         dp[0] = 0;
46         for(auto &pr : pos){ // cover DP: min sum(v) para quitar >= j de peso
47             long long w = pr.first, v = pr.second;
48             for(long long j = T; j >= 0; j--){
49                 if(dp[j] < INF){
50                     long long nj = (j + w >= T) ? T : (j + w);
51                     if(dp[j] + v < dp[nj]) dp[nj] = dp[j] + v;
52                 }
53             }
54         }
55         for(long long j = 1; j <= T; j++){
56             if(dp[j] < INF)
57                 best_family = max(best_family, PosV + FM0 + (j - dp[j]));
58         }
59     }
60     bool family_valid = true;
61     if(PosV == 0 && E == 0) family_valid = false; // daría 0 cámaras
62
63     // Caso frontera: TODOS los tanques son mixtos positivos -> hay que dejar 1 como drenaje
64     if(E == 0 && femonly == 0 && negmixed == 0){
65         long long minv = LLONG_MAX;
66         for(auto &pr : pos) minv = min(minv, pr.second);
67         best_family = PosV - minv;
68         family_valid = true;
69     }
70
71     long long maxB = bestSingle; // cámara única (siempre válida, n>=2)
72     if(family_valid) maxB = max(maxB, best_family);
73
74     cout << (2 * S - maxB) << "\n";
75     return 0;
76 }
```

10.7 Emparejar Extremos - Axles

NO ES MIO: código de Stiven Correa, del repo del equipo (carpeta RPC/). Resuelve "Axles" (RPC 2026-07, problema A): las ruedas se agrupan por eje y en cada eje se emparejan de dos en dos; minimizar la suma de los desbalances. Técnica: por cada eje se ordenan los diámetros en un deque y se emparejan el MAS PEQUENO con el MAS GRANDE (popleft con pop), se sigue hacia el centro y se acumula log(mayor/menor). El logaritmo convierte el producto de razones en una suma, que es lo que se puede acumular en un solo acumulador sin desbordar ni perder precisión.

RPC\2026-07\A - Axles.py

```
1 from collections import defaultdict, deque
2 import math
3
4 dicc= defaultdict(list)
5
6 n = int(input())
```

10.7 Emparejar Extremos - Axles (cont.)

```

1
2 for _ in range(n):
3     s, c = map(int, input().split())
4     dicc[s].append(c)
5
6
7 diccS = dict()
8 for key, val in dicc.items():
9     diccS[key] = deque(sorted(dicc[key]))
10
11
12 res = 0
13 for key, val in diccS.items():
14     while(len(val) > 1):
15         a, b = val.popleft(), val.pop()
16         res += math.log(max(a,b)/min(a,b))
17
18 print(res)

```

10.8 Huffman

O: (n log n) con priority_queue

Uso: lee frecuencias por stdin; el costo es la suma de las fusiones

Costo mínimo para combinar N elementos de a dos, donde cada combinacion cuesta la suma de los dos elementos y el resultado vuelve a la mesa. Aparece disfrazado como "unir cuerdas", "mezclar archivos ordenados" o "juntar montones de piedras". El greedy correcto es siempre combinar los dos MAS PEQUEÑOS disponibles, porque cada elemento paga su valor una vez por cada combinacion en la que participa, y los mas grandes deben participar en la menor cantidad posible. Un priority_queue de mínimo lo resuelve en O(n log n). Cuidado: el acumulador debe ser long long, la suma crece rapido.

C++\Notebook\Huffman.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int main()
6 {
7     ios_base::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    int n;
11    cin >> n;
12
13    priority_queue<long long, vector<long long>, greater<long long>> pq;
14    for (int i = 0; i < n; i++)
15    {
16        long long x;
17        cin >> x;
18        pq.push(x);
19    }
20
21    long long cost = 0;
22    while (pq.size() > 1)
23    {
24        long long x = pq.top(); pq.pop();
25        long long y = pq.top(); pq.pop();
26        cost += x + y;
27        pq.push(x + y);
28    }
29
30    cout << cost << "\n";
31    return 0;
32 }

```

10.9 Huffman sobre Frecuencias Variables en el Tiempo - Efficient Encoding

Resuelve "Efficient Encoding" (problema E, ICPC 2025): cada símbolo tiene una frecuencia que cambia con el tiempo, $f_i(t)$, lineal a trozos con un quiebre en m_i . Para cada t se arma el mejor código prefijo (Huffman) y se paga $C(t)$ = suma de $f_i(t)$ por el largo de su código. Hay que dar el MENOR t entero de $[0, T]$ que minimiza $C(t)$, y ese costo con 3 decimales. LA PREGUNTA CLAVE ES CUALES t HAY QUE PROBAR. T llega a 10^6 y hay hasta 1000 casos, así que evaluar todos es imposible. El argumento que lo reduce a unos pocos: - Con las longitudes FIJAS, el costo es suma de $f_i(t) * l_i$, lineal a trozos con quiebres solo en los m_i . - El costo OPTIMO es el mínimo sobre todas las asignaciones validas de longitudes, o sea el mínimo de un monton de funciones lineales: eso es CONCAVO en cada tramo sin quiebres. - Una funcion concava en un intervalo alcanza su mínimo en un EXTREMO, nunca por dentro. Entonces el mínimo global solo puede estar en 0, en T , o en alguno de los m_i : a lo sumo $n+2$ candidatos. Y como los m_i son enteros, todos los candidatos ya son enteros. LA PRIMERA VERSION DE ESTE ARCHIVO SOLO PROBABA $t = 0$ y $t = T$, y fallaba el propio sample: en el segundo caso (un símbolo con $a=2, m=1, b=1, c=2$ y $T=2$) el mínimo esta en $t=1$, donde la frecuencia baja a 1. Daba "0.2.000" en vez de "1.000". Medido contra probar TODOS los t enteros en 1200 casos: la version vieja fallaba 443 y esta no falla ninguno. EL OTRO ERROR ERA EL TIPO: en un t intermedio las frecuencias NO son enteras, así que el Huffman tiene que trabajar en double. Se recorren los candidatos ordenados y solo se cambia el mejor con una mejora Estricta, para quedarse con el t mas chico en caso de empate, que es lo que pide el enunciado. Con un solo símbolo el costo es su frecuencia: igual hace falta 1 bit por ocurrencia, y esa es la excepcion que Huffman no cubre solo (el arbol de un nodo tendria largo 0).

C++\ICPC\2025\E - Efficient Encoding.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct Symbol {
7     ll a, m, b, c;
8 };
9
10 // Costo optimo de un codigo prefijo (Huffman). Las frecuencias son REALES,
11 // porque en un t intermedio f_i(t) no es entera.
12 double huffman(vector<double> freq) {
13
14     priority_queue<double, vector<double>, greater<double>> pq;
15
16     for (double x : freq)
17         pq.push(x);
18
19     // Con un solo simbolo igual hace falta 1 bit por ocurrencia.
20     if (pq.size() == 1)
21         return pq.top();
22
23     double ans = 0;
24
25     while (pq.size() > 1) {
26         double x = pq.top();
27         pq.pop();
28
29         double y = pq.top();
30         pq.pop();
31
32         ans += x + y;
33         pq.push(x + y);
34     }
35
36     return ans;
37 }
38
39
40 // f_i(t), lineal a trozos con el quiebre en m_i.
41 double frecuencia(const Symbol &s, double t, double T) {
42     if (t <= s.m)
43         return s.a + (double)(s.b - s.a) / s.m * t;
44     return s.b + (double)(s.c - s.b) / (T - s.m) * (t - s.m);
45 }
46
47 int main() {

```

10.9 Huffman sobre Frecuencias Variables en el Tiempo - Efficient Encoding (cont.)

```

48 ios::sync_with_stdio(false);
49 cin.tie(nullptr);
50
51 int K;
52 cin >> K;
53
54 while (K--) {
55
56     int n;
57     ll T;
58
59     cin >> n >> T;
60
61     vector<Symbol> s(n);
62
63     for (int i = 0; i < n; i++)
64         cin >> s[i].a >> s[i].m >> s[i].b >> s[i].c;
65
66     // Con las longitudes FIJAS, el costo es lineal a trozos con quiebres en
67     // los m_i. El optimo es el minimo de esas lineales, o sea CONCAVO entre
68     // quiebres, y una concava alcanza su minimo en un extremo del intervalo.
69     // Asi que basta mirar 0, T y cada m_i.
70     vector<ll> cand;
71     cand.push_back(0);
72     cand.push_back(T);
73     for (int i = 0; i < n; i++)
74         cand.push_back(s[i].m);
75
76     sort(cand.begin(), cand.end());
77     cand.erase(unique(cand.begin(), cand.end()), cand.end());
78
79     ll mejorT = 0;
80     double mejorC = 1e18;
81
82     for (ll t : cand) {
83
84         vector<double> f(n);
85
86         for (int i = 0; i < n; i++)
87             f[i] = frecuencia[s[i], t, T);
88
89         double c = huffman(f);
90
91         // Se queda el t mas chico: por eso la comparacion es estricta y los
92         // candidatos vienen ordenados.
93         if (c < mejorC - 1e-9) {
94             mejorC = c;
95             mejorT = t;
96         }
97     }
98
99     cout << mejorT << ' ' << fixed << setprecision(3) << mejorC << '\n';
100 }
101
102 return 0;
103 }

```

10.10 Interval Scheduling - Interval Scheduling Greedy

O: ($n \log n$), ordenar por FIN y tomar el que no choque

Uso: ordenar por $v[i].second$ y avanzar mientras inicio $\geq$ ultimo_fin

Maximo numero de intervalos que se pueden elegir sin que se solapen (tareas con hora de inicio y fin, salones, reservas). Es el greedy clasico y no debe confundirse con la DP de intervalos de "Yuyuan Market", que optimiza otra cosa (cantidad y luego costo). La clave es ordenar por hora de FIN, no por inicio ni por duracion: al quedarse siempre con el intervalo que termina antes, se deja la mayor cantidad de tiempo libre para los que siguen, y se puede probar por intercambio que ninguna otra eleccion mejora el resultado. Ordenar por inicio o por duracion da respuestas incorrectas.

C++\Notebook\Interval Scheduling Greedy.cpp

```

1 #include <bits/stdc++.h>
2

```

10.10 Interval Scheduling - Interval Scheduling Greedy (cont.)

```

3 using namespace std;
4
5 int main()
6 {
7     ios_base::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    int n;
11    cin >> n;
12
13    vector<pair<int,int>> v(n);
14    for (int i = 0; i < n; i++)
15    {
16        cin >> v[i].first >> v[i].second;
17    }
18
19    // ordenar por hora de fin
20    sort(v.begin(), v.end(), [](const pair<int,int> &a, const pair<int,int> &b){
21        return a.second < b.second;
22    });
23
24    int cnt = 0;
25    int last = INT_MIN;
26
27    for (auto &it : v)
28    {
29        if (it.first >= last)
30        {
31            cnt++;
32            last = it.second;
33        }
34    }
35
36    cout << cnt << "\n";
37    return 0;
38 }

```

10.11 Maximo Conjunto a Distancia d en Arbol - Oficinas A Distancia D En Arbol

O: (n), un BFS para ordenar y un barrido de hojas a raiz

Uso: lee el arbol por stdin; el greedy va de las hojas hacia la raiz

Hay n ciudades en forma de arbol, cada una con una oficina, y hay que cerrar la MENOR cantidad posible de oficinas para que cualesquiera dos de las que queden esten a distancia al menos d . Cerrar lo minimo es lo mismo que dejar lo maximo, así que esto es el maximo conjunto de vertices a distancia mutua $\geq d$ en un arbol. En un grafo cualquiera sería NP-difícil, pero en arbol el greedy de hojas a raiz lo resuelve exacto en $O(n)$. La idea que hace que funcione: procesando de abajo hacia arriba, de todo el subarbol de v lo UNICO que el resto del arbol necesita saber es a que distancia esta la oficina superviviente mas cercana a v , porque cualquier oficina de afuera tiene que pasar por v para llegar. Eso es $dist[v]$, y $rep[v]$ guarda cual es esa oficina para poder marcarla si toca cerrarla. Al pegar un hijo c a su padre p , la oficina mas cercana del lado de c queda a $dc = dist[c] + 1$. Si $dist[p] + dc < d$, esas dos se pisan y hay que cerrar una. Se cierra SIEMPRE la mas cercana a p : quedarse con la mas lejana deja $dist[p]$ lo mas grande posible, y $dist[p]$ grande es pura ganancia porque lo unico que falta por decidir esta hacia arriba. Es un intercambio clasico, ninguna solucion optima empeora al cambiarle la cercana por la lejana. Y basta cerrar UNA sola, no hace falta revisar el resto del subarbol: las demas oficinas de p ya cumplen $\geq d$ contra $rep[p]$, y eso mismo las deja lo bastante lejos de $rep[c]$. Si no hay choque, solo se actualiza el minimo: $dist[p] = \min(dist[p], dc)$. Verificado contra fuerza bruta sobre 2^n y contra un DP exacto de arbol, ademas de la salida en si (que todo par sobreviviente quede a $\geq d$). Detalles que importan: $dist[v]$ arranca en 0 y $rep[v] = v$ porque TODA ciudad empieza con oficina, esa es la condicion inicial del problema. El recorrido es BFS iterativo, no DFS recursivo, así que un camino de 200 mil nodos no revienta la pila. Medido: $n = 200000$ en menos de 200 ms. OJO con el formato de salida: deja un espacio sobrante antes del salto de linea. La mayoría de jueces lo ignora, pero si uno sale estricto, ahí esta el problema. Entrada: n y d , despues las $n-1$ aristas. Salida: cuantas oficinas quedan y cuales.

C++\Notebook\Oficinas a Distancia d en Arbol.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {

```

10.11 Maximo Conjunto a Distancia d en Arbol - Oficinas A Distancia D En Arbol (cont.)

```

5   ios::sync_with_stdio(false);
6   cin.tie(nullptr);
7
8   int n, d;
9   cin >> n >> d;
10  vector<vector<int>> adj(n + 1);
11  for (int i = 0; i < n - 1; i++) {
12      int a, b;
13      cin >> a >> b;
14      adj[a].push_back(b);
15      adj[b].push_back(a);
16  }
17
18  // Orden BFS desde la raíz 1
19  vector<int> order, par(n + 1, 0);
20  order.reserve(n);
21  vector<char> vis(n + 1, 0);
22  order.push_back(1);
23  vis[1] = 1;
24  for (int i = 0; i < (int)order.size(); i++) {
25      int v = order[i];
26      for (int u : adj[v]) {
27          if (!vis[u]) {
28              vis[u] = 1;
29              par[u] = v;
30              order.push_back(u);
31          }
32      }
33  }
34
35  vector<int> dist(n + 1, 0), rep(n + 1);
36  vector<char> removed(n + 1, 0);
37  // todas las ciudades empiezan con oficina
38  for (int v = 1; v <= n; v++) rep[v] = v;
39
40  // Procesar de hojas a raíz, uniendo cada nodo con su padre
41  for (int i = n - 1; i >= 1; i--) {
42      int c = order[i], p = par[c];
43      int dc = dist[c] + 1;
44      if (dist[p] + dc < d) {
45          // conflicto: cerrar la oficina mas cercana a p
46          if (dist[p] < dc) {
47              removed[rep[p]] = 1;
48              dist[p] = dc;
49              rep[p] = rep[c];
50          } else {
51              removed[rep[c]] = 1;
52          }
53      } else if (dc < dist[p]) {
54          dist[p] = dc;
55          rep[p] = rep[c];
56      }
57  }
58
59  vector<int> ans;
60  for (int v = 1; v <= n; v++)
61      if (!removed[v]) ans.push_back(v);
62
63  string out = to_string(ans.size()) + "\n";
64  for (int x : ans) out += to_string(x) + ' ';
65  out += '\n';
66  cout << out;
67  return 0;
68 }

```

10.12 Prefijos sobre Secuencias Balanceadas - Incompatible Pairs

Resuelve "Incompatible Pairs" (Day 10, problema D - Contest 09 [Avanzados]): una secuencia de N bailarines Yang ('Y') y Yin ('Y') donde cada Yang debe emparejarse con un Yin posterior; un par (i, j) es incompatible si al formarlos los bailarines restantes ya no pueden emparejarse todos entre sí (rompe el balance tipo parentesis). Usa el balance de prefijos $prefix[k]$ (+1 por '(', -1 por ')'); para cada Yang en la posición i, $f[i]$ es la primera posición $k \geq i$ donde el balance vuelve a 0 (fin del segmento balanceado que arranca en i): cualquier Yin ubicado después de $f[i]$ forma un par incompatible con ese Yang, porque deja un '(' suelto sin ')' extra donde absorberlo. Suma, para cada 'Y' en la posición i, la cantidad de 'Y' que hay después de $f[i]$ (precomputada en $suffixClose$) para obtener el total de pares incompatibles.

C++\Training_Camp_2026\Day 10\D - Incompatible Pairs.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  typedef long long ll;
4
5  static char buf_1[< 22];
6  static int bufLen_, bufPos_;
7  inline int gc() {
8      if (bufPos_ == bufLen_) { bufLen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!bufLen_) return -1; }
9      return buf_[bufPos_++];
10 }
11 inline int readInt() {
12     int c = gc();
13     while (c != '-' && (c < '0' || c > '9') && c != '-') c = gc();
14     bool neg = false; if (c == '-') { neg = true; c = gc(); }
15     int x = 0;
16     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
17     return neg ? -x : x;
18 }
19 inline void readString(string &s, int expectedLen) {
20     s.clear();
21     s.reserve(expectedLen);
22     int c = gc();
23     while (c != '(' && c != ')') c = gc();
24     while (c == '(' || c == ')') {
25         s.push_back((char)c);
26         c = gc();
27     }
28 }
29
30 int main() {
31     int N = readInt();
32     string s;
33     readString(s, N);
34     // in case the read stopped early due to buffering edge cases, N is authoritative
35     // for sizing arrays; s.size() should equal N given well-formed input.
36
37     vector<int> prefix(N + 1, 0);
38     for (int i = 1; i <= N; i++) {
39         prefix[i] = prefix[i - 1] + (s[i - 1] == '(' ? 1 : -1);
40     }
41
42     // f[i] = smallest k >= i with prefix[k] == 0, for i = 1..N (computed right to left).
43     vector<int> f(N + 2, 0);
44     f[N] = (prefix[N] == 0) ? N : N + 1;
45     for (int i = N - 1; i >= 1; i--) {
46         f[i] = (prefix[i] == 0) ? i : f[i + 1];
47     }
48
49     // suffixClose[k] = number of ')' at 1-indexed positions > k, for k = 0..N.
50     vector<ll> suffixClose(N + 2, 0);
51     for (int k = N - 1; k >= 0; k--) {
52         suffixClose[k] = suffixClose[k + 1] + (s[k] == ') ? 1 : 0);
53     }
54
55     ll total = 0;
56     for (int i = 1; i <= N; i++) {
57         if (s[i - 1] == '(') {
58             total += suffixClose[f[i]];
59         }
60     }

```

10.12 Prefijos sobre Secuencias Balanceadas - Incompatible Pairs (cont.)

```

60     }
61
62     printf("%lld\n", total);
63     return 0;
64 }

```

10.13 Prefix Sums - Maximizing Advertising

Resuelve "Maximizing Advertising" (Day 7, problema B - Contest 07 [Avanzados]): dados N votantes con coordenadas (x,y) y preferencia 'b' o 'w', hay que elegir dos rectangulos disjuntos de lados paralelos a los ejes (uno para PSD, otro para PS) que maximicen la suma de votantes 'b' cubiertos por el primero mas los votantes 'w' cubiertos por el segundo. Como la particion optima siempre puede lograrse separando el plano con una linea vertical u horizontal, bestSplit ordena los puntos por una coordenada (x o y) y usa sumas prefijas (prefB, prefW) para probar, en cada corte posible, poner los 'b' de un lado y los 'w' del otro (o viceversa); el main toma el mejor resultado entre cortar por X o por Y.

C++\Training_Camp_2026\Day 7\B - Maximizing Advertising.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  typedef long long ll;
4
5  static char buf[1 << 25];
6  static int bufLen = 0, bufPos = 0;
7
8  inline int gc() {
9      if (bufPos == bufLen) {
10         bufLen = (int)fread(buf, 1, sizeof(buf), stdin);
11         bufPos = 0;
12         if (bufLen <= 0) return -1;
13     }
14     return buf[bufPos++];
15 }
16
17 inline int readInt() {
18     int c = gc();
19     while (c == ' ' || c == '\n' || c == '\r' || c == '\t') c = gc();
20     bool neg = false;
21     if (c == '-') { neg = true; c = gc(); }
22     int x = 0;
23     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
24     return neg ? -x : x;
25 }
26
27 inline char readChar() {
28     int c = gc();
29     while (c == ' ' || c == '\n' || c == '\r' || c == '\t') c = gc();
30     return (char)c;
31 }
32
33 int N;
34 vector<int> Xs, Ys;
35 vector<char> Cs;
36
37 // dado un vector de coordenadas (alineado con Cs), calcula el mejor
38 // resultado separando en dos grupos (izq/der) por esa coordenada
39 ll bestSplit(vector<int>& coord) {
40     int n = N;
41     vector<pair<int, char>> arr(n);
42     for (int i = 0; i < n; i++) arr[i] = {coord[i], Cs[i]};
43     sort(arr.begin(), arr.end(), [](const pair<int, char>& a, const pair<int, char>& b) {
44         return a.first < b.first;
45     });
46
47     ll totalB = 0, totalW = 0;
48     for (auto& pr : arr) { if (pr.second == 'b') totalB++; else totalW++; }
49
50     ll best = max(totalB, totalW); // caso: todo a un lado, el otro rectangulo vacio
51
52     ll prefB = 0, prefW = 0;
53     int i = 0;
54     while (i < n) {

```

10.13 Prefix Sums - Maximizing Advertising (cont.)

```

55     int j = i;
56     int cur = arr[i].first;
57     ll gB = 0, gW = 0;
58     while (j < n && arr[j].first == cur) {
59         if (arr[j].second == 'b') gB++; else gW++;
60         j++;
61     }
62     prefB += gB; prefW += gW;
63     // corte justo despues de este grupo de coordenada 'cur'
64     ll optA = prefB + (totalW - prefW); // b a la izquierda, w a la derecha
65     ll optB = prefW + (totalB - prefB); // w a la izquierda, b a la derecha
66     best = max(best, max(optA, optB));
67     i = j;
68 }
69 return best;
70 }
71
72 int main() {
73     N = readInt();
74     Xs.resize(N); Ys.resize(N); Cs.resize(N);
75     for (int i = 0; i < N; i++) {
76         Xs[i] = readInt();
77         Ys[i] = readInt();
78         Cs[i] = readChar();
79     }
80
81     ll ans = max(bestSplit(Xs), bestSplit(Ys));
82     printf("%lld\n", ans);
83     return 0;
84 }

```

10.14 Puente y Linterna - The Bridge At Night

Resuelve "The Bridge At Night" (problema B, ICPC 2024), el clasico del puente con una sola linterna: cruzan a lo sumo dos a la vez, al paso del mas lento, y alguien tiene que devolver la linterna. El codigo calcula el tiempo minimo para pasar a todos. Se ordenan los tiempos y, mientras queden mas de 3, se mandan al otro lado LOS DOS MAS LENTOS (y, z) con la mejor de dos estrategias, siendo a y b los dos mas rapidos: a + 2b + z cruzan a y b, vuelve a, cruzan y y z juntos, vuelve b 2a + y + z a acompaña a z, vuelve; a acompaña a y, vuelve. La primera gana cuando los lentos son MUY lentos, porque paga z una sola vez para los dos; la segunda cuando no, porque solo usa al mas rapido de ida y vuelta. Cada ronda deja el problema igual pero con dos personas menos. Casos base: con 3 quedan a + b + c (a acompaña a c, vuelve, cruza con b); con 2, el mas lento; con 1, el unico. Con 1, 2, 5 y 10, que es el acertijo famoso, da 17 (comprobado corriendo este codigo): la estrategia ingenua de que el rapido acompaña a todos da 19.

C++\ICPC\2024\B - The Bridge At Night.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios::sync_with_stdio(false);
6      cin.tie(nullptr);
7
8      int n;
9
10     while (cin >> n && n != 0) {
11
12         vector<int> t(n);
13
14         for (int &x : t)
15             cin >> x;
16
17         sort(t.begin(), t.end());
18
19         int ans = 0;
20
21         while (n > 3) {
22
23             int a = t[0];
24             int b = t[1];
25             int y = t[n - 2];

```

10.14 Puente y Linterna - The Bridge At Night (cont.)

```

26     int z = t[n - 1];
27
28     int option1 = a + 2 * b + z;
29     int option2 = 2 * a + y + z;
30
31     ans += min(option1, option2);
32
33     n -= 2;
34 }
35
36 // Quedan 1, 2 o 3 personas
37 if (n == 3) {
38     ans += t[0] + t[1] + t[2];
39 }
40 else if (n == 2) {
41     ans += t[1];
42 }
43 else if (n == 1) {
44     ans += t[0];
45 }
46
47 cout << ans << '\n';
48 }
49
50 return 0;
51 }

```

10.15 Puente y Linterna (version con indices) - Bridge Juan

La otra solución del problema B de ICPC 2024 ("The Bridge at Night"), el clásico del puente con una sola linterna. Misma estrategia que "B - The Bridge At Night" de esta carpeta, escrita con un contador de cuantos quedan en vez de ir recortando el arreglo. Con los tiempos ordenados y mientras queden mas de 3, se cruzan los DOS MAS LENTOS con la mejor de dos jugadas, donde $t[0]$ y $t[1]$ son los dos mas rapidos: $escolta = 2*t[0] + t[quedan-2] + t[quedan-1]$ el mas rapido acompaña a cada lento $parejaLenta = t[0] + 2*t[1] + t[quedan-1]$ los dos lentos cruzan juntos La segunda gana cuando los lentos son MUY lentos, porque paga el mas lento una sola vez para los dos; la primera cuando la diferencia es chica. Cada ronda saca a dos del problema. Casos base: con 3 quedan $t[0]+t[1]+t[2]$; con 2, el mas lento; con 1, el unico. Verificado con el sample (10, 20, 8) y con el acertijo famoso de 1, 2, 5 y 10, que da 17 (la estrategia ingenua de que el rapido acompañe a todos daría 19). La otra version esta verificada además contra un Dijkstra exacto sobre todos los estados en 1500 casos; esta comparte la formula, así que vale lo mismo.

C++\ICPC\2024\bridge Juan.cpp

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  using namespace std;
6
7  int main()
8  {
9      long long n;
10
11      while(cin >> n && n != 0)
12      {
13          vector<long long> tiempo(n);
14
15          for(long long i = 0; i < n; i++)
16          {
17              cin >> tiempo[i];
18          }
19
20          sort(tiempo.begin(), tiempo.end());
21
22          long long total = 0;
23          long long quedan = n;
24
25          while(quedan > 3)
26          {
27              long long escolta = 2 * tiempo[0] + tiempo[quedan - 2] + tiempo[quedan - 1];
28              long long parejaLenta = tiempo[0] + 2 * tiempo[1] + tiempo[quedan - 1];

```

10.15 Puente y Linterna (version con indices) - Bridge Juan (cont.)

```

29
30          total += min(escolta, parejaLenta);
31          quedan -= 2;
32      }
33
34      if(quedan == 3)
35      {
36          total += tiempo[0] + tiempo[1] + tiempo[2];
37      }
38      else if(quedan == 2)
39      {
40          total += tiempo[1];
41      }
42      else
43      {
44          total += tiempo[0];
45      }
46
47      cout << total << '\n';
48  }
49
50  return 0;
51 }

```

10.16 Segment Counting - Cerros

Cuenta la cantidad de segmentos contiguos con elementos $\geq k$ en un arreglo. Cada vez que encuentra un elemento $\geq k$ después de uno $< k$, incrementa el contador.

C++\Codeforces\cerros.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios::sync_with_stdio(false);
6      cin.tie(nullptr);
7
8      long long n,k;
9      cin >> n >> k;
10     long long god=0;
11     bool abierto = false;
12     for(int i = 0; i < n;i++)
13     {
14         int a; cin >> a;
15
16         if(a>=k&&!abierto)
17         {
18             god++;
19             abierto = true;
20         }
21         else if(a<k)
22         {
23             abierto = false;
24         }
25     }
26
27     cout << god << "\n";
28
29     return 0;
30 }

```

10.17 Simulation - Kathmandu

Resuelve "Kathmandu" (Day 4, problema F - Contest 04 [Avanzados]): durante un vuelo de duracion D hay M comidas servidas en instantes y_i (hay que estar despierto para comer), y hay que decidir si se puede dormir T minutos seguidos sin interrupcion sin perderse ninguna comida. Arma la lista c de instantes relevantes (0, cada y_i , y D) y busca si existe un hueco entre dos instantes consecutivos de largo mayor o igual a T ($abs(c[i]-c[i+1]) \geq t$), ya que ese hueco es la unica ventana disponible para dormir sin faltar a ninguna comida.

C++\Training_Camp_2026\Day 4\F - Kathmandu.py

```

1  t, v, n = list(map(int,input().split()))
2  c = [0]

```

10.17 Simulation - Kathmandu (cont.)

```

3  for i in range(n):
4      c.append(int(input()))
5      c.append(v)
6
7  f = False
8  for i in range(len(c)-1):
9      if(t <= abs(c[i]-c[i+1]]):
10         f = True
11         break
12
13 if(f):
14     print("Y")
15 else:
16     print("N")

```

10.18 Sliding Window

O(n) amortizado, cada elemento entra y sale una vez

Uso: exige condicion monotona; con negativos NO sirve, usa prefijos + map

Ventana de tamaño variable para hallar el subarreglo mas largo que cumple una condicion monotona (suma <= K, a lo sumo K distintos, etc). Cada elemento entra y sale de la ventana a lo sumo una vez, así que el costo total es O(n) aunque haya dos ciclos anidados. La condicion tiene que ser monotona: si un rango cumple, cualquier subrango tambien debe cumplir. Por eso funciona con sumas de numeros no negativos, pero NO si el arreglo tiene negativos (ahí encoger la ventana no garantiza reducir la suma y hay que usar prefijos con un map o un deque monotono).

C++\Notebook\Sliding Window.cpp

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main()
6  {
7      ios_base::sync_with_stdio(false);
8      cin.tie(nullptr);
9
10     int n;
11     long long K;
12     cin >> n >> K;
13
14     vector<long long> a(n);
15     for (int i = 0; i < n; i++) cin >> a[i];
16
17     int l = 0;
18     long long cur = 0, ans = 0;
19
20     for (int r = 0; r < n; r++)
21     {
22         cur += a[r];
23         while (cur > K)
24         {
25             cur -= a[l];
26             l++;
27         }
28         ans = max(ans, (long long)(r - l + 1));
29     }
30
31     cout << ans << "\n";
32     return 0;
33 }

```

10.19 Sorted Pairing - Balatro

Calcula la suma de diferencias absolutas entre pares ordenados de dos arreglos. Ordena ambos y suma las diferencias elemento a elemento.

C++\Codeforces\balatro.cpp

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  int main() {

```

10.19 Sorted Pairing - Balatro (cont.)

```

6
7     long long n;
8     cin >> n;
9     vector<long long > yo(n);
10    vector<long long > atk(n);
11
12    for(int i = 0; i<n ; i++)
13    {
14        long long a;
15        cin >> a;
16        yo[i] = a;
17    }
18
19    for(int i = 0; i<n ; i++)
20    {
21        long long a;
22        cin >> a;
23        atk[i] = a;
24    }
25
26    long long ans = 0;
27
28    sort(yo.begin(), yo.end());
29    sort(atk.begin(), atk.end());
30
31    for(int i = 0; i<n ; i++)
32    {
33        ans+= abs(atk[i] - yo[i]);
34    }
35
36    cout << ans << endl;
37
38    return 0;
39 }

```

10.20 Sorting - Finding Forks

Resuelve "Finding Forks" (Day 11, problema A - CCPL (Kattis)): en un cajon con n tipos de tenedores, cada tipo con a_i unidades, al menos dos lugares del cajon quedaron vacios porque no se recuerda que tipo va en cada lugar tras lavarlos. Pide el minimo numero de tenedores que debieron estar en el lavaplatos para causar esa confusion. Ordena las cantidades con l.sort() y suma las dos mas pequenas (l[0]+l[1]), pues la ambigüedad minima se logra siempre entre los dos tipos con menos tenedores.

C++\Training_Camp_2026\Day 11 - CCPL\A - Finding Forks.py

```

1  n = int(input())
2  l = list(map(int, input().split()))
3
4  l.sort()
5
6  print(l[0]+l[1])

```

10.21 Sorting (Prefix Sums) - Coven Complications

Resuelve "Coven Complications" (Day 11, problema F - CCPL (Kattis)): un grafo de n comunidades brujas unidas por m portales, donde las comunidades con f_i=0 son propias y las que tienen f_i>0 ferrets son rivales. Cada día se sella una comunidad propia sin sellar, y ese día todas las demas comunidades propias aun sin sellar pierden brujas iguales a la suma de ferrets de sus comunidades rivales vecinas directas; pide el minimo total de brujas perdidas hasta sellar todas las comunidades propias. Calcula para cada comunidad propia su perdida diaria fija ('citiesW', suma de ferrets de los vecinos rivales via las aristas), y aplica la estrategia greedy clasica de minimizar sumas de tiempos de finalizacion: ordena esas perdidas de forma ascendente y suma las sumas prefijas ('pre') salvo la ultima, sellando primero las comunidades con mayor perdida diaria.

C++\Training_Camp_2026\Day 11 - CCPL\F - Coven Complications.py

```

1  n, c = list(map(int, input().split()))
2
3  hurones = list(map(int, input().split()))
4
5  citiesW = [0]*(n+1)
6  for i in range(c):
7      a, b = list(map(int, input().split()))
8      a, b = (hurones[a-1], a), (hurones[b-1], b)

```

10.21 Sorting (Prefix Sums) - Coven Complications (cont.)

```

9     if a[0] == 0 or b[0] == 0:
10         obj = min(a,b)
11         citiesW[obj[1]] += max(a[0],b[0])
12
13 pre = []
14 citiesW.sort()
15 prev = 0
16 for i in citiesW:
17     prev += i
18     if i != 0:
19         pre.append(prev)
20
21 sol = 0
22 for i in range(len(pre)-1):
23     sol += pre[i]
24
25 print(sol)

```

10.22 Two Pointers (Container With Most Water)

O: (n), dos punteros desde los extremos

Uso: maxArea(h) -> {area, {i, j}}

Paredes verticales de altura $h[i]$ en la posición i : elegir dos que, junto con el eje x , formen el recipiente de mayor área ($j - i) * \min(h[i], h[j])$). Fuerza bruta es $O(n^2)$; con dos punteros desde los extremos es $O(n)$. Por que se mueve la pared MAS BAJA: el agua queda topada por ella. Cualquier recipiente entre esa pared y una interior es mas angosto y sigue topado por la misma altura (o menos), así que nunca supera al actual: esa pared ya dio su mejor recipiente y se puede descartar. Si miden lo mismo, ninguna de las dos mejora con una pared interior: mover cualquiera. El área puede pasar de 2^{31} ($1e9 * 1e5$): usar long long. No confundir con Trapping Rain Water (el agua que retienen TODAS las barras): tambien es de dos punteros, pero en cada posición suma $\min(\maxLzq, \maxDer) - h[i]$.

C++\Notebook\Container With Most Water.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 typedef long long ll;
6
7 // {area maxima, {i, j}} (i = j = -1 si hay menos de dos paredes)
8 pair<ll, pair<int, int>> maxArea(const vector<ll> &h)
9 {
10     int l = 0, r = (int)h.size() - 1, bi = -1, bj = -1;
11     ll best = 0;
12     while (l < r)
13     {
14         ll a = (ll)(r - l) * min(h[l], h[r]);
15         if (a > best || bi == -1)
16         {
17             best = a;
18             bi = l;
19             bj = r;
20         }
21         if (h[l] < h[r]) l++;
22         else r--;
23     }
24     return {best, {bi, bj}};
25 }
26
27 int main()
28 {
29     ios_base::sync_with_stdio(false);
30     cin.tie(nullptr);
31
32     int n;
33     cin >> n;
34     vector<ll> h(n);
35     for (auto &x : h) cin >> x;
36
37     auto r = maxArea(h);
38     cout << r.first << "\n";
39     cout << r.second.first << " " << r.second.second << "\n";

```

10.22 Two Pointers (Container With Most Water) (cont.)

```

40     return 0;
41 }

```

11 - Binary Search

11.1 Búsqueda sobre la Respuesta con Fracciones Exactas - Clean Streets

Resuelve "Clean Streets" (problema C, Regionals 2025): elegir trabajadores con tarifas y ventanas de disponibilidad para limpiar S calles al menor costo. Se busca sobre la RAZON $r = \text{costo por hora}$. Para un r fijo, un trabajador sirve si su intervalo $[L/H, U/H]$ contiene a r , y entre los que sirven se toman los mas baratos hasta juntar S calles: un segment tree responde "cual es el costo minimo de tomar exactamente need calles entre los H mas chicos" con un descenso por el arbol. LOS CANDIDATOS SON FINITOS: la respuesta optima siempre cae en algun extremo de intervalo, o sea en algun L_i/H_i . No hace falta búsqueda binaria real sobre reales: se prueban esos $O(N)$ candidatos en orden, agregando trabajadores cuyo intervalo ya empezo y sacando los que ya terminaron, como un barrido. LO QUE HAY QUE COPIAR DE AQUI ES fracLess Y fracLE: comparar a/b contra c/d se hace con la multiplicacion cruzada $a*d < c*b$, en ENTEROS. Usar double para ordenar fracciones es la forma mas comun de fallar estos problemas, porque dos fracciones distintas pueden dar el mismo double y el orden queda al azar. Ojo con el signo de los denominadores: la multiplicacion cruzada solo conserva la desigualdad si ambos son positivos. El costo final se reporta como fraccion (num * horas / den) en vez de convertirlo a decimal antes de tiempo, para no arrastrar error. OJO: usa structured bindings (auto [a, b]), que piden C++17. En el juez compila, pero con un g++ viejo hay que volver a .first y .second.

C++\VCPC\Regionals 2025\C - Clean Streets.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct Cleaner {
7     ll H, L, U;
8 };
9
10 struct SegTree {
11     int n;
12     vector<ll> cnt;
13     vector<ll> cost;
14
15     SegTree(int n) : n(n) {
16         cnt.assign(4 * n + 5, 0);
17         cost.assign(4 * n + 5, 0);
18     }
19
20     void update(int node, int l, int r, int pos, ll dc, ll dcost) {
21         if (l == r) {
22             cnt[node] += dc;
23             cost[node] += dcost;
24             return;
25         }
26
27         int mid = (l + r) / 2;
28
29         if (pos <= mid)
30             update(node * 2, l, mid, pos, dc, dcost);
31         else
32             update(node * 2 + 1, mid + 1, r, pos, dc, dcost);
33
34         cnt[node] = cnt[node * 2] + cnt[node * 2 + 1];
35         cost[node] = cost[node * 2] + cost[node * 2 + 1];
36     }
37
38     void update(int pos, ll dc) {
39         update(1, 1, n, pos, dc, dc * pos);
40     }
41
42     // Minimo costo para tomar exactamente need calles
43     // usando los H mas pequenos.
44     ll query(int node, int l, int r, ll need) {
45         if (l == r) {
46             return need * l;

```

11.1 Búsqueda sobre la Respuesta con Fracciones Exactas - Clean Streets (cont.)

```

47     }
48
49     int mid = (l + r) / 2;
50
51     if (cnt[node * 2] >= need) {
52         return query(node * 2, l, mid, need);
53     }
54
55     ll leftCost = cost[node * 2];
56     ll leftCnt = cnt[node * 2];
57
58     return leftCost +
59         query(node * 2 + 1, mid + 1, r, need - leftCnt);
60 }
61
62 ll query(ll need) {
63     if (cnt[1] < need)
64         return -1;
65
66     return query(1, 1, n, need);
67 }
68 };
69
70 // a/b < c/d
71 bool fracLess(ll a, ll b, ll c, ll d) {
72     return a * d < c * b;
73 }
74
75 // a/b <= c/d
76 bool fracLE(ll a, ll b, ll c, ll d) {
77     return a * d <= c * b;
78 }
79
80 int main() {
81     ios::sync_with_stdio(false);
82     cin.tie(nullptr);
83
84     int N;
85     ll S, K;
86
87     cin >> N >> S >> K;
88
89     vector<Cleaner> a(N);
90
91     for (auto &[H, L, U] : a) {
92         cin >> H >> L >> U;
93     }
94
95     // Orden por L/H
96     vector<int> byL(N);
97     iota(byL.begin(), byL.end(), 0);
98
99     sort(byL.begin(), byL.end(), [&](int i, int j) {
100         return fracLess(
101             a[i].L, a[i].H,
102             a[j].L, a[j].H
103         );
104     });
105
106     // Orden por U/H
107     vector<int> byU(N);
108     iota(byU.begin(), byU.end(), 0);
109
110     sort(byU.begin(), byU.end(), [&](int i, int j) {
111         return fracLess(
112             a[i].U, a[i].H,
113             a[j].U, a[j].H
114         );
115     });
116
117     // Candidatos r = L_i / H_i
118     vector<int> candidates = byL;

```

11.1 Búsqueda sobre la Respuesta con Fracciones Exactas - Clean Streets (cont.)

```

119
120     SegTree st(100000);
121
122     int pL = 0;
123     int pU = 0;
124
125     bool found = false;
126
127     ll bestNum = 0;
128     ll bestDen = 1;
129
130     for (int id : candidates) {
131
132         ll num = a[id].L;
133         ll den = a[id].H;
134
135         // Agregar todos los trabajadores cuyo
136         // intervalo ya comenzo.
137         while (pL < N) {
138             int j = byL[pL];
139
140             if (!fracLE(
141                 a[j].L, a[j].H,
142                 num, den
143             ))
144                 break;
145
146             ll capacity = K / a[j].H;
147
148             st.update(a[j].H, capacity);
149
150             pL++;
151         }
152
153         // Eliminar trabajadores cuyo intervalo
154         // ya termino antes de r.
155         while (pU < N) {
156             int j = byU[pU];
157
158             if (!fracLess(
159                 a[j].U, a[j].H,
160                 num, den
161             ))
162                 break;
163
164             ll capacity = K / a[j].H;
165
166             st.update(a[j].H, -capacity);
167
168             pU++;
169         }
170
171         // Podemos limpiar S calles?
172         ll totalHours = st.query(S);
173
174         if (totalHours == -1)
175             continue;
176
177         // costo = r * totalHours
178         //
179         // r = num / den
180         //
181         // costo = num * totalHours / den
182
183         ll curNum = num * totalHours;
184         ll curDen = den;
185
186         ll g = gcd(curNum, curDen);
187
188         curNum /= g;
189         curDen /= g;
190

```

11.1 Búsqueda sobre la Respuesta con Fracciones Exactas - Clean Streets (cont.)

```

81         if (!found ||
82             (__int128)curNum * bestDen <
83             (__int128)bestNum * curDen) {
84
85             found = true;
86             bestNum = curNum;
87             bestDen = curDen;
88         }
89     }
90
91     if (!found) {
92         cout << "*" << "\n";
93         return 0;
94     }
95
96     cout << bestNum << " " << bestDen << "\n";
97 }

```

11.2 Greedy (búsqueda binaria sobre la respuesta) - Minas Gerais' Walls

Resuelve "Minas Gerais' walls" (Day 6, problema M - Maratona SBC de Programacao 2025): una muralla de N segmentos consecutivos con alturas x_i se refuerza una unica vez eligiendo un segmento, que recibe K bloques extra, el segmento a su izquierda $K-1$, y así en forma de escalera hasta sumar solo 1 bloque o quedarse sin segmentos a la izquierda; la fortaleza de la muralla es su altura minima, y hay que maximizar esa altura minima tras el unico refuerzo. El codigo hace búsqueda binaria sobre la altura candidata H usando "feasible(H)": para cada H calcula el rango de posiciones p de refuerzo que alcanzarían a subir todo segmento deficitario ($x_i < H$) hasta al menos H , a partir de la formula del escalon $p \leq i + K - H + x_i$, y verifica que ese rango $[lo, hi]$ sea no vacio y quede dentro de $[1, N]$.

C++\Training_Camp_2026\Day 6\M - Minas Gerais' walls.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  typedef long long ll;
4
5  int N;
6  ll K;
7  vector<ll> x; // 0-indexed internamente, x[i] = altura del segmento (i+1)
8
9  // Dado un candidato H, ¿existe algun p tal que reforzando en p,
10 // TODOS los segmentos queden con altura >= H?
11 bool feasible(ll H) {
12     ll L = 0; // mayor indice (1-indexado) con x[i] < H, o 0 si no hay ninguno
13     ll U = LLONG_MAX; // cota superior para p, viene de la condicion de "alcanzar" cada defic
14     it
15
16     for (int i = 1; i <= N; i++) {
17         if (x[i-1] < H) {
18             L = i; // como i crece, la ultima asignacion es el maximo indice con deficit
19             // Necesitamos: K - (p - i) >= H - x[i-1] => p <= i + K - H + x[i-1]
20             ll candidate = (ll)i + K - H + x[i-1];
21             U = min(U, candidate);
22         }
23     }
24
25     ll lo = max((ll)1, L);
26     ll hi = min((ll)N, U);
27     return lo <= hi;
28 }
29
30 int main(){
31     ios_base::sync_with_stdio(false);
32     cin.tie(NULL);
33
34     cin >> N >> K;
35     x.resize(N);
36     for (int i = 0; i < N; i++) cin >> x[i];
37
38     ll lo = *min_element(x.begin(), x.end()); // siempre alcanzable (bonus >= 0 siempre)
39     ll hi = *max_element(x.begin(), x.end()) + K; // cota superior segura
40
41     ll ans = lo;

```

11.2 Greedy (búsqueda binaria sobre la respuesta) - Minas Gerais' Walls (cont.)

```

41     while (lo <= hi) {
42         ll mid = lo + (hi - lo) / 2;
43         if (feasible(mid)) {
44             ans = mid;
45             lo = mid + 1;
46         } else {
47             hi = mid - 1;
48         }
49     }
50
51     cout << ans << "\n";
52     return 0;
53 }

```

11.3 Two Pointers - Black Hills Golden Jewels

Resuelve "Black Hills golden jewels" (Day 5, problema B - Contest 05 [Avanzados]): dado el precio a_i de cada una de N categorías de joyas, un cliente debe elegir dos categorías distintas, y ningún cliente puede repetir una combinacion ya elegida por otro; hay que hallar el monto minimo que garantiza que el K -esimo cliente en la fila pueda comprar, es decir, la K -esima suma de pares $(a_i + a_j)$ mas pequena entre todos los pares posibles. Hace búsqueda binaria sobre la respuesta (variable mid entre lo y hi) y para cada candidato cuenta, con dos punteros (i creciente, j decreciente) sobre el arreglo ordenado, cuantos pares tienen suma $\leq mid$, ajustando el rango hasta converger al K -esimo valor.

C++\Training_Camp_2026\Day 5\B - Black Hills golden jewels.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5
6  int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    int N;
11    ll K;
12    cin >> N >> K;
13
14    vector<ll> a(N);
15
16    for (int i = 0; i < N; i++)
17        cin >> a[i];
18
19    sort(a.begin(), a.end());
20
21    // La respuesta está entre la suma mínima y máxima posibles.
22    ll lo = a[0] + a[1];
23    ll hi = a[N - 1] + a[N - 2];
24
25    while (lo < hi) {
26
27        ll mid = (lo + hi) / 2;
28
29        /*
30         Contaremos cuántos pares tienen suma <= mid.
31
32         Usamos dos punteros.
33
34         Para cada i, buscamos el mayor j tal que
35
36             a[i] + a[j] <= mid
37
38         Entonces todos los índices entre
39         (i+1 ... j) también sirven.
40         */
41        ll cnt = 0;
42
43        int j = N - 1;
44
45        for (int i = 0; i < N; i++) {
46

```

11.3 Two Pointers - Black Hills Golden Jewels (cont.)

```

47 while (j > i && a[i] + a[j] > mid)
48     j--;
49
50 if (j <= i)
51     break;
52
53 cnt += (j - i);
54 }
55
56 if (cnt >= K)
57     hi = mid;
58 else
59     lo = mid + 1;
60 }
61
62 cout << lo << '\n';
63
64 return 0;
65 }

```

12 - Constructive Algorithms

12.1 Hashing - Handling The Blocks

Resuelve "Handling the Blocks" (Day 4, problema E - Contest 04 [Avanzados]): N bloques numerados 1..N tienen cada uno un color, y en cada turno se pueden intercambiar dos bloques que compartan color; hay que decidir si es posible ordenar la secuencia de forma ascendente por número. Como los intercambios solo mueven números entre bloques del mismo color, la posición i solo puede terminar con el número (i+1) si ese número pertenece al mismo color que el bloque originalmente ubicado en la posición i; el código arma por color (lista l de sets) los números que comparten cada color y verifica esa condición posición por posición.

C++\Training_Camp_2026\Day 4\E - Handling the Blocks.py

```

1 n, k = list(map(int, input().split()))
2 l = []
3 for i in range(k):
4     l.append(set())
5 val = []
6 for i in range(n):
7     c, v = list(map(int, input().split()))
8     l[v-1].add(c)
9     val.append([c, v])
10
11
12 f = True
13 for i in range(n):
14     #h = val[i][0]
15     if (i+1 in l[val[i][1]-1]):
16         continue
17     else:
18         f = False
19         break
20
21 if(f):
22     print("Y")
23 else:
24     print("N")

```

12.2 Secuencias de Langford - Doubled Sequence II

Resuelve "Doubled Sequence II" (Day 9, problema C - Contest 08 [Inicial & Avanzado]): para un n dado, construir una secuencia de longitud 2n donde cada valor de 1 a n aparece exactamente dos veces y, para cada i, la distancia entre sus dos ocurrencias es exactamente i+1 (esto es una variante de las secuencias de Langford/Skolem). Solo existe solución cuando n % 4 es 0 o 3; en otro caso imprime -1. Cuando existe, usa la construcción explícita de Davies (1959) (funciones build4m y build4mMinus1) que arma la secuencia por bloques crecientes/decrecientes de paridad fija (appendRange) según n sea 4m o 4m-1, con casos base para n=3 y n=4.

C++\Training_Camp_2026\Day 9\C - Doubled Sequence II.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;

```

12.2 Secuencias de Langford - Doubled Sequence II (cont.)

```

3
4 static char buf_[1 << 22];
5 static int buflen_, bufPos_;
6 inline int gc() {
7     if (bufPos_ == buflen_) { buflen_ = fread(buf_, 1, sizeof(buf_), stdin); bufPos_ = 0; if (!buflen_) return -1; }
8     return buf_[bufPos_++];
9 }
10 inline int readInt() {
11     int c = gc();
12     while (c != -1 && (c < '0' || c > '9') && c != '-') c = gc();
13     bool neg = false; if (c == '-') { neg = true; c = gc(); }
14     int x = 0;
15     while (c >= '0' && c <= '9') { x = x * 10 + (c - '0'); c = gc(); }
16     return neg ? -x : x;
17 }
18
19 string out;
20
21 inline void appendInt(int v) {
22     char tmp[12];
23     int len = 0;
24     if (v == 0) { tmp[len++] = '0'; }
25     else {
26         int x = v;
27         while (x > 0) { tmp[len++] = char('0' + x % 10); x /= 10; }
28     }
29     for (int i = len - 1; i >= 0; i--) out += tmp[i];
30     out += ' ';
31 }
32
33 // Appends consecutive even/odd terms from a to b inclusive (a,b same parity),
34 // increasing if a<=b, decreasing if a>b.
35 inline void appendRange(int a, int b) {
36     if (a <= b) {
37         for (int v = a; v <= b; v += 2) appendInt(v);
38     } else {
39         for (int v = a; v >= b; v -= 2) appendInt(v);
40     }
41 }
42
43 // Davies (1959) construction for n = 4m, m >= 2.
44 void build4m(int m) {
45     appendRange(4*m-4, 2*m);
46     appendInt(4*m-2);
47     appendRange(2*m-3, 1);
48     appendInt(4*m-1);
49     appendRange(1, 2*m-3);
50     appendRange(2*m, 4*m-4);
51     appendInt(4*m);
52     appendRange(4*m-3, 2*m+1);
53     appendInt(4*m-2);
54     appendRange(2*m-2, 2);
55     appendInt(2*m-1);
56     appendInt(4*m-1);
57     appendRange(2, 2*m-2);
58     appendRange(2*m+1, 4*m-3);
59     appendInt(2*m-1);
60     appendInt(4*m);
61 }
62
63 // Davies (1959) construction for n = 4m-1, m >= 2.
64 void build4mMinus1(int m) {
65     appendRange(4*m-4, 2*m);
66     appendInt(4*m-2);
67     appendRange(2*m-3, 1);
68     appendInt(4*m-1);
69     appendRange(1, 2*m-3);
70     appendRange(2*m, 4*m-4);
71     appendInt(2*m-1);
72     appendRange(4*m-3, 2*m+1);
73     appendInt(4*m-2);

```

12.2 Secuencias de Langford - Doubled Sequence II (cont.)

```

74     appendRange(2*m-2, 2);
75     appendInt(2*m-1);
76     appendInt(4*m-1);
77     appendRange(2, 2*m-2);
78     appendRange(2*m+1, 4*m-3);
79 }
80
81 int main() {
82     int t = readInt();
83     out.reserve(1 << 22);
84     while (t--) {
85         int n = readInt();
86         int r = n % 4;
87         if (r != 0 && r != 3) {
88             out += "-1\n";
89             continue;
90         }
91         if (n == 3) {
92             out += "2 3 1 2 1 3\n";
93         } else if (n == 4) {
94             out += "2 3 4 2 1 3 1 4\n";
95         } else if (r == 0) {
96             build4m(n / 4);
97             out.back() = '\n';
98         } else { // r == 3, n = 4m-1 with m = (n+1)/4
99             build4mMinus1((n + 1) / 4);
100             out.back() = '\n';
101         }
102     }
103     fwrite(out.data(), 1, out.size(), stdout);
104     return 0;
105 }

```

13 - Implementation

13.1 Ad Hoc - Toilet Seat

Resuelve "Toilet Seat" (Day 11, problema C - CCPL (Kattis)): dada una secuencia de preferencias 'U' (arriba) o 'D' (abajo) de personas que usan el baño una tras otra, donde el primer carácter es la posición inicial del asiento, pide calcular cuantos ajustes de asiento se necesitan bajo tres políticas: (1) siempre dejarlo arriba, (2) siempre dejarlo abajo, (3) dejarlo como a uno le gustaria encontrarlo. Recorre la cadena una sola vez calculando en paralelo ans1 y ans2 (contando cuantas veces el asiento debe moverse segun cada política fija) y ans3 (contando los cambios de preferencia entre personas consecutivas, que es el costo bajo la política de dejarlo "como te gustaria encontrarlo").

C++\Training_Camp_2026\Day 11 - CCPL\C - Toilet Seat.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    string s;
11    cin >> s;
12    ll ans1=0, ans2=0, ans3=0;
13
14    //1
15    if(s[0]== 'U' && s[1] == 'U') ans1+=0;
16    else if(s[0]== 'U' && s[1] == 'D') ans1+=2;
17    else if(s[0]== 'D' && s[1] == 'D') ans1+=1;
18    else ans1+=1;
19    for(int i = 2; i < s.size(); i++)
20    {
21        if(s[i] == 'D') ans1+=2;
22    }
23
24    //2

```

13.1 Ad Hoc - Toilet Seat (cont.)

```

25    if(s[0]== 'U' && s[1] == 'U') ans2+=1;
26    else if(s[0]== 'U' && s[1] == 'D') ans2+=1;
27    else if(s[0]== 'D' && s[1] == 'D') ans2+=0;
28    else ans2+=2;
29    for(int i = 2; i < s.size(); i++)
30    {
31        if(s[i] == 'U') ans2+=2;
32    }
33
34    //3
35    char act = s[0];
36    for(int i = 1; i < s.size(); i++)
37    {
38        if(s[i] != act)
39        {
40            ans3++;
41            act = s[i];
42        }
43    }
44
45    cout << ans1 << endl;
46    cout << ans2 << endl;
47    cout << ans3 << endl;
48
49    return 0;
50 }

```

13.2 Ad Hoc - Rock Paper Scissors Tournament

Resuelve "Rock-Paper-Scissors Tournament" (Day 11, problema D - CCPL (Kattis)): en un torneo de piedra-papel-tijera con n jugadores, cada par de jugadores disputa k partidas ($k(n-1)/2$ partidas en total); dado el movimiento de cada jugador en cada partida, pide el promedio de victorias $w/(w+l)$ de cada jugador, redondeado a tres decimales, o "-" si nunca gana ni perdio. Simula partida por partida, aplicando las reglas de piedra-papel-tijera para acumular victorias en 'w' y derrotas en 'l' por jugador, y al final calcula e imprime el promedio de cada uno.

C++\Training_Camp_2026\Day 11 - CCPL\D - Rock-Paper-Scissors Tournament.py

```

1 n = list(map(int, input().split()))
2 if len(n) != 1:
3     n, k = n
4     while(n != 0):
5         if n == 1:
6             print("-")
7             n = list(map(int, input().split()))
8             if len(n) == 1:
9                 break
10            else:
11                n, k = n
12            w = [0]*(n+1)
13            l = [0]*(n+1)
14
15            for i in range((k*n*(n-1))/2):
16                num1, res1, num2, res2 = input().split()
17                num1, num2 = int(num1), int(num2)
18                if res1 == res2:
19                    continue
20                if res1 == "rock" and res2 == "paper":
21                    w[num2] += 1
22                    l[num1] += 1
23                elif res1 == "rock" and res2 == "scissors":
24                    w[num1] += 1
25                    l[num2] += 1
26                elif res1 == "paper" and res2 == "rock":
27                    w[num1] += 1
28                    l[num2] += 1
29                elif res1 == "paper" and res2 == "scissors":
30                    w[num2] += 1
31                    l[num1] += 1
32                elif res1 == "scissors" and res2 == "rock":
33                    w[num2] += 1

```

13.2 Ad Hoc - Rock Paper Scissors Tournament (cont.)

```

34         l[num1] += 1
35         elif res1 == "scissors" and res2 == "paper":
36             w[num1] += 1
37             l[num2] += 1
38
39         for i in range(1, n+1):
40             wins = w[i]
41             loses = l[i]
42
43             if wins+loses == 0:
44                 print("-")
45             else:
46                 print(f"{{(wins/(wins+loses)):.3f}}")
47         print("")
48         n = list(map(int, input().split()))
49         if len(n) == 1:
50             break
51         else:
52             n, k = n

```

13.3 Ad Hoc - Zero Or One

Resuelve "Zero or One" (Day 2, problema A - Maratona de Programacao da SBC 2013): Alice, Beto y Clara juegan "cero o uno" mostrando cada uno la mano con un valor 0 o 1; gana quien elige un valor distinto al de los otros dos, y si nadie queda en esa situacion (todos iguales, o empate 2-1 sin un unico distinto) no hay ganador. Usa la suma A+B+C para detectarlo sin comparar par por par: si la suma es 1, el unico que voto 1 es el ganador; si la suma es 2, el unico que voto 0 es el ganador; en cualquier otro caso (suma 0 o 3) no hay ganador y se imprime '*'.

C++\Training_Camp_2026\Day 2\A - Zero or One.py

```

1  a, b, z = list(map(int, input().split()))
2
3  if 0<(a+b+z)< 3:
4      if (a+b+z == 2 and a == 0) or (a+b+z == 1 and a == 1):
5          print("A")
6      if (a+b+z == 2 and b == 0) or (a+b+z == 1 and b == 1):
7          print("B")
8      if (a+b+z == 2 and z == 0) or (a+b+z == 1 and z == 1):
9          print("C")
10 else:
11     print('*')

```

13.4 Ad Hoc - Collatz Polynomial

Resuelve "Collatz polynomial" (Day 6, problema C - Maratona SBC de Programacao 2025): Aline crea una variante de la Conjetura de Collatz para polinomios con coeficientes 0 o 1. Si el polinomio tiene termino independiente, se multiplica por $(x+1)$ y se le suma 1, descartando cualquier termino cuyo coeficiente resulte 2; si no tiene termino independiente, se divide entre x . Hay que contar cuantas operaciones hacen falta para llegar a $P(x) = 1$. El codigo simula el proceso literalmente sobre la lista "val" (coeficiente de x^i en la posicion i): hace $val.pop(0)$ para dividir entre x , o construye una nueva lista "n" desplazando y sumando (multiplicacion por $x+1$, mas 1 en el termino independiente) y poniendo en 0 los coeficientes que dan 2, contando los pasos en "c" hasta que el polinomio quede reducido a 1.

C++\Training_Camp_2026\Day 6\C - Collatz polynomial.py

```

1  grado = int(input())
2  val = list(reversed(list(map(int, input().split()))))
3
4  m = [1, 1]
5  c = 0
6  while val.count(1) > 1 or val[0] != 1:
7      #print(val)
8      if val[0] == 1:
9          n = [0] * len(val)
10         for i in range(len(val)):
11             if val[i]==0:
12                 continue
13             for j in range(len(m)):
14                 try:
15                     n[i+j]+=1
16             except:

```

13.4 Ad Hoc - Collatz Polynomial (cont.)

```

17         n.append(1)
18         n[0]+=1
19         for i in range(len(val)):
20             if(n[i]>=2):
21                 n[i]=0
22         val = n
23         else:
24             val.pop(0)
25         #if(c==5):
26             # break
27         c+= 1
28     print(c)

```

13.5 Ad Hoc - Unraveling Monty Hall

Resuelve "Unraveling Monty Hall" (Day 8, problema D - Contest 08 [Avanzados]): se simulan N juegos del problema de Monty Hall donde el jugador siempre elige la puerta 1 y luego siempre cambia a la otra puerta cerrada tras que el presentador revela una cabra; dado en que puerta esta el auto en cada juego, hay que contar en cuantos de ellos el jugador gana el auto. Como el jugador parte de la puerta 1 y el presentador solo puede abrir una puerta con cabra entre las dos no elegidas, cambiar de puerta gana el auto exactamente cuando el auto esta en la puerta 2 o en la puerta 3; el codigo simplemente cuenta cuantas de las N entradas valen 2 o 3 (variable "sum").

C++\Training_Camp_2026\Day 8\D - Unraveling Monty Hall.cpp

```

1  #include <iostream>
2  using namespace std;
3  int main()
4  {
5      int n; cin >> n;
6      int sum = 0;
7      while(n--)
8      {
9          int a; cin >>a;
10         if(a==3 or a==2)
11             {
12                 sum++;
13             }
14     }
15     cout << sum << endl;
16     //a
17     return 0;
18 }

```

13.6 Arreglos - Buffoon

Resuelve "Buffoon" (Day 1, problema B - Maratona de Programacao da SBC 2019): en la eleccion del bufon de la corte, N candidatos reciben votos y gana el mas votado, rompiendo empates a favor de quien se registro primero. Carlos se asegura de ser el primero en registrarse, asi que basta comparar si sus votos (primer valor de la lista) son iguales al maximo del arreglo para saber si gano ("S") o no ("N").

C++\Training_Camp_2026\Day 1\B - Buffoon.py

```

1  n = int(input())
2  l = []
3  for i in range(n):
4      l.append(int(input()))
5
6  ma = max(l)
7  if(ma == l[0]):
8      print("S")
9  else:
10     print("N")

```

13.7 Backtracking (Enumeracion de bits desconocidos) - Harmonics With Interference

Resuelve "Harmonics with Interference" (Day 3, problema H - Maratona SBC de Programacao 2024): Arthur envia un mensaje binario M junto con una secuencia de control N tal que el entero representado por M es divisible por el entero representado por N; Bruna recibe M' y N' donde algunos bits se perdieron y aparecen como '*' (a lo sumo 16 simbolos '*' en total entre ambos), y hay que reconstruir un M valido compatible con lo recibido, o determinar que no existe. El codigo genera por backtracking (generateBins) todos los valores posibles de N' rellenando sus '*' con 0 o 1, y luego, tambien por backtracking (generateAndCheck), prueba todas las combinaciones de los '*' restantes en M' (peso guarda el valor posicional de cada '*'), devolviendo el primer M cuyo valor entero sea divisible por algun N generado; como el total de incognitas nunca supera 16, la enumeracion completa (2^{16} en el peor caso) es factible.

C++\Training_Camp_2026\Day 3\H - Harmonics with Interference.py

```
1 import copy
2 import sys
3
4 sys.setrecursionlimit(3*10**6)
5 m = input().strip()
6 n = input().strip()
7
8 ns = []
9 def generateBins(s, i, objective):
10     if i < 0:
11         objective.append(int("".join(s), 2))
12         return
13     elif s[i] == "*":
14         way = copy.copy(s)
15         way[i] = "1"
16         generateBins(way, i-1, objective)
17         way = copy.copy(s)
18         way[i] = "0"
19         generateBins(way, i-1, objective)
20     else:
21         generateBins(s, i-1, objective)
22
23 generateBins(list(n), len(n)-1, ns)
24
25 L = len(m)
26 estrellas = [idx for idx, c in enumerate(m) if c == "*"]
27 base = int("".join("0" if c == "*" else c for c in m), 2) # M con todos los * = 0
28 peso = [1 << (L - 1 - idx) for idx in estrellas] # lo que suma poner un 1 en ese *
29
30 def generateAndCheck(t, val):
31     if t == len(estrellas):
32         for k in ns:
33             if k != 0 and val % k == 0:
34                 print(bin(val)[2:].zfill(L)) # una sola vez, ya con la longitud correcta
35                 sys.exit(0)
36         return
37     generateAndCheck(t + 1, val + peso[t]) # ese * = 1
38     generateAndCheck(t + 1, val) # ese * = 0
39
40 generateAndCheck(0, base)
41 print(-1)
```

13.8 Brute Force - Cake Hater

Resuelve "Cake Hater" (Day 10, problema G - Contest 09 [Avanzados]): una receta de pastel con N pasos, cada uno con su propia lista de ingredientes; Antonio omite un conjunto fijo de B ingredientes que no le gustan (donts) de un total de A. El pastel se arruina en el primer paso donde faltan MAS de un tercio de sus ingredientes. Simula paso por paso: para cada paso cuenta cuantos de sus ingredientes estan en el set donts (counter) y compara contra ingreds[0]/3 (el tamaño del paso); en cuanto counter supera ese umbral, imprime el indice del paso (i+1) y detiene la busqueda; si ningun paso lo supera, imprime -1.

C++\Training_Camp_2026\Day 10\G - Cake Hater.py

```
1 n = int(input())
2
3 a, b = list(map(int, input().split()))
4
5 donts = set(map(int, input().split()))
```

13.8 Brute Force - Cake Hater (cont.)

```
6
7 sol = False
8 for i in range(n):
9     ingreds = list(map(int, input().split()))
10
11     counter = 0
12     if not sol:
13         for j in range(1, ingreds[0]+1):
14             if ingreds[j] in donts:
15                 counter += 1
16             if counter > ingreds[0]/3:
17                 print(i+1)
18                 sol = True
19                 break
20 if not sol:
21     print(-1)
```

13.9 Brute Force - LLMs

Resuelve "LLMs" (Day 6, problema L - Maratona SBC de Programacao 2025): Bruno arma un predictor de palabras "SBC" a partir de un diccionario de palabras con vectores 2D, un texto base de conocimiento y consultas; para cada consulta, se buscan en el texto base todas las ocurrencias consecutivas de las ultimas K palabras (ventana de contexto), se toman las palabras siguientes como "candidatas" (si no hay ninguna, se baja K en 1 y se repite hasta K=0), y se elige la palabra del diccionario que maximiza el producto punto de su vector contra la suma de los vectores candidatos (empate: la mas comun, primera en el diccionario). El codigo hace esto de forma directa: busqueda por fuerza bruta de "ctx" dentro de "kb" con bucles anidados para armar "candidates", suma sus vectores en (sumX, sumY), y recorre linealmente las N palabras del diccionario para hallar el maximo "bestS" de dictVec[i].x*sumX + dictVec[i].y*sumY.

C++\Training_Camp_2026\Day 6\L - LLMs .cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main(){
5     ios_base::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int N;
9     cin >> N;
10
11     vector<string> dictWord(N);
12     vector<pair<long long, long long>> dictVec(N);
13     unordered_map<string, int> firstIndex; // first occurrence of a word in the dictionary
14
15     for(int i = 0; i < N; i++){
16         string w; long long x, y;
17         cin >> w >> x >> y;
18         dictWord[i] = w;
19         dictVec[i] = {x, y};
20         if(firstIndex.find(w) == firstIndex.end())
21             firstIndex[w] = i;
22     }
23
24     int M;
25     cin >> M;
26     vector<string> kb(M);
27     for(int i = 0; i < M; i++)
28         cin >> kb[i];
29
30     int Q, K;
31     cin >> Q >> K;
32
33     for(int q = 0; q < Q; q++){
34         int F;
35         cin >> F;
36         vector<string> query(F);
37         for(int i = 0; i < F; i++)
38             cin >> query[i];
39
40         int curK = K;
41         vector<string> candidates;
```

13.9 Brute Force - LLMs (cont.)

```

42
43 while(curK >= 1){
44     // last curK words of the ORIGINAL query
45     vector<string> ctx(query.end() - curK, query.end());
46     candidates.clear();
47
48     // search knowledge base for consecutive occurrences of ctx
49     for(int j = 0; j + curK - 1 < M; j++){
50         bool match = true;
51         for(int t = 0; t < curK; t++){
52             if(kb[j + t] != ctx[t]){ match = false; break; }
53         }
54         if(match){
55             int nextPos = j + curK;
56             if(nextPos < M){
57                 candidates.push_back(kb[nextPos]);
58             }
59         }
60     }
61
62     if(!candidates.empty()) break;
63     curK--;
64 }
65
66 // print the query words
67 for(int i = 0; i < F; i++){
68     cout << query[i];
69     cout << " ";
70 }
71
72 if(candidates.empty()){
73     cout << "*\n";
74     continue;
75 }
76
77 // Precompute candidate vectors (0,0) if word not in dictionary
78 vector<pair<long long,long long>> candVecs;
79 candVecs.reserve(candidates.size());
80 for(auto &c : candidates){
81     auto it = firstIndex.find(c);
82     if(it != firstIndex.end()){
83         candVecs.push_back(dictVec[it->second]);
84     }
85     else
86         candVecs.push_back({0,0});
87 }
88
89 // sum of candidate vectors, so S(d) = d . sum(candidates)
90 long long sumX = 0, sumY = 0;
91 for(auto &cv : candVecs){
92     sumX += cv.first;
93     sumY += cv.second;
94 }
95
96 long long bestS = LLONG_MIN;
97 int bestIdx = -1;
98
99 for(int i = 0; i < N; i++){
100     long long s = dictVec[i].first * sumX + dictVec[i].second * sumY;
101     if(s > bestS){
102         bestS = s;
103         bestIdx = i;
104     }
105 }
106
107 cout << dictWord[bestIdx] << "\n";
108 }
109
110 return 0;
111 }

```

13.10 Circular Queue - Eeny Meeny

Resuelve "Eeny Meeny" (Day 11, problema B - CCPL (Kattis)): un grupo de niños en círculo es dividido en dos equipos aplicando repetidamente una rima (como "eeny meeny miny moe"): se cuenta en sentido horario, saltando a los niños ya elegidos, hasta la última palabra de la rima, que selecciona al siguiente niño para el equipo actual, alternando de equipo en cada ronda. Pide reconstruir los dos equipos en el orden en que fueron elegidos. Simula la ronda con un índice `curr` que recorre circularmente la lista `people`, el conjunto `fuera` para saltar a los ya seleccionados, y un contador `currN` que cuenta palabras hasta llegar a `n` (longitud de la rima), alternando el equipo destino `group` cada vez que se elige a alguien.

C++\Training_Camp_2026\Day 11 - CCPL\B - Eeny Meeny.py

```

1 n = input().split()
2
3 n = len(n)
4
5 k = int(input())
6
7 people = []
8
9 for i in range(k):
10     nombre = input()
11     people.append(nombre)
12
13 curr = 0
14 fuera = set()
15 group = 0
16 groups = [[], []]
17 curr = 0
18 currN = 0
19 while(len(fuera) != len(people)):
20     if curr == k:
21         curr = 0
22     while curr in fuera:
23         curr+=1
24     if curr == k:
25         curr = 0
26     currN += 1
27     if currN == n:
28         groups[group].append(people[curr])
29         fuera.add(curr)
30         currN = 0
31     if group == 1:
32         group = 0
33     else:
34         group = 1
35     curr += 1
36
37 for i in groups:
38     print(len(i))
39     for j in i:
40         print(j)

```

13.11 Clasificaci n de Cadenas por Prefijo - Joker Or Judger

Cada reporte que llega es una cadena de una lista fija y vale cierto puntaje: UnreasonableProblemArrangement vale 10, WrongProblemX vale 100, SameProblemX vale 30, UnreasonableLimitForProblemX vale 5, WeakTestsForProblemX vale 3 y BadProblemX vale 1, donde X es una letra de la A a la L (los 12 problemas del contest). Cualquier otra cosa vale 0. Se suman los n reportes del caso y se responde Joker si el total pasa de P, y Judger si no. De algoritmo no tiene nada: es lectura cuidadosa. Lo  nico que decide si pasa o no es no equivocarse contando caracteres, y por eso vale la pena tener la plantilla lista. LA CLAVE: validar s.size() ANTES de comparar el prefijo, y ese size() no es decorativo. s.substr(0, 12) sobre una cadena mas corta NO lanza excepci n, simplemente devuelve lo que alcance, as  que comparar solo el prefijo dejar a pasar "BadProblemAA" o "BadProblemABC". Fijar el largo exacto es lo que obliga a que haya EXACTAMENTE una letra desp es del prefijo, y por eso s.back() de una vez es la letra del problema. Las seis ramas son mutuamente excluyentes porque cada una fija un largo distinto: 11, 12, 13, 20, 28 y 30. O sea que el orden de los if da igual, no hay una que tape a otra. Vale la pena revisarlo cuando se escribe algo as , porque si dos plantillas compartieran largo Y prefijo, el orden s  mandaria y el bug ser a invisible. Verificado que los seis largos cuadran con sus prefijos (len(prefijo) + 1 == size), que es justo el off-by-one que se cuela solo en este tipo de problema. Alternativas para comparar prefijos, por si el largo no se puede fijar: s.find(prefijo, 0) == 0 // busca el prefijo solo en la posici n 0 s.compare(0, k, prefijo) == 0 // sin crear el substring temporal Y ojo con el comparador final: es total > P estricto, empatar con P es Judger.

C++\Codeforces\JokerOrJudger.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int score(string s) {
5     if (s == "UnreasonableProblemArrangement")
6         return 10;
7
8     if (s.size() == 13 && s.substr(0, 12) == "WrongProblem" &&
9         s.back() >= 'A' && s.back() <= 'L')
10        return 100;
11
12     if (s.size() == 12 && s.substr(0, 11) == "SameProblem" &&
13         s.back() >= 'A' && s.back() <= 'L')
14        return 30;
15
16     if (s.size() == 28 && s.substr(0, 27) == "UnreasonableLimitForProblem" &&
17         s.back() >= 'A' && s.back() <= 'L')
18        return 5;
19
20     if (s.size() == 20 && s.substr(0, 19) == "WeakTestsForProblem" &&
21         s.back() >= 'A' && s.back() <= 'L')
22        return 3;
23
24     if (s.size() == 11 && s.substr(0, 10) == "BadProblem" &&
25         s.back() >= 'A' && s.back() <= 'L')
26        return 1;
27
28     return 0;
29 }
30
31 int main() {
32     ios::sync_with_stdio(false);
33     cin.tie(nullptr);
34
35     int T;
36     cin >> T;
37
38     while (T--) {
39         int n, P;
40         cin >> n >> P;
41
42         int total = 0;
43
44         while (n--) {
45             string s;
46             cin >> s;
47             total += score(s);
48         }
49     }
```

13.11 Clasificaci n de Cadenas por Prefijo - Joker Or Judger (cont.)

```
48
49 cout << (total > P ? "Joker" : "Judger") << '\n';
50 }
51 }
52 }
```

13.12 Conjuntos - Jo o Jo o

Resuelve "Jo o Jo o" (Day 6, problema J - Maratona SBC de Programacao 2025): ya se crearon 10 tareas para un examen, cada una con un nivel de dificultad entre 1 y 4, y se necesita saber cuantas tareas nuevas hacen falta para poder armar un examen con exactamente 4 tareas, una de cada nivel de dificultad distinto. El c digo mete las 10 dificultades en un set "n" (que colapsa los duplicados y deja solo los niveles distintos ya cubiertos) e imprime 4 menos el tama o de ese set, es decir, cuantos niveles de dificultad todav a faltan por cubrir.

C++\Training_Camp_2026\Day 6\J - Jo o Jo o.py

```
1 n = set(map(int,input().split()))
2 print(4-len(n))
```

13.13 Constructive - A Story About Tea

Resuelve "A story about tea" (Day 7, problema F - Contest 07 [Avanzados]): N botes deben viajar de Portugal a Inglaterra usando China como puerto auxiliar, siguiendo las reglas de las Torres de Hanoi (cada viaje mueve el bote mas chico entre las dos pilas involucradas, y nunca hay dos viajes simult neos), en exactamente K viajes; hay que decidir si es posible y, de serlo, imprimir la secuencia de K viajes. El m nimo de viajes es $2^N - 1$ (funci n h, la recursi n cl sica de Hanoi); si K es mayor, completa la diferencia con pares de movimientos "de relleno" A-B-A que no cambian el estado (delta/2 veces) y, si sobra un viaje impar, usa la variante g que gasta un movimiento extra en el  ltimo bote; si $K < 2^N - 1$ no hay soluci n posible ("N").

C++\Training_Camp_2026\Day 7\F - A story about tea.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string out;
5
6 inline void addMove(char from, char to) {
7     out += from;
8     out += ' ';
9     out += to;
10    out += '\n';
11 }
12
13 // mueve n botes de X a Y usando Z como auxiliar, minimal ( $2^n - 1$  movimientos)
14 void h(int n, char X, char Y, char Z) {
15     if (n == 0) return;
16     h(n - 1, X, Z, Y);
17     addMove(X, Y);
18     h(n - 1, Z, Y, X);
19 }
20
21 // mueve n botes de X a Y usando Z como auxiliar, con exactamente 1 movimiento extra ( $2^n$  movimient
22 os)
23 void g(int n, char X, char Y, char Z) {
24     if (n == 1) {
25         addMove(X, Z);
26         addMove(Z, Y);
27         return;
28     }
29     h(n - 1, X, Z, Y);
30     addMove(X, Y);
31     g(n - 1, Z, Y, X);
32 }
33
34 int main() {
35     long long N, K;
36     cin >> N >> K;
37
38     long long minK = (1LL << N) - 1;
39
40     if (K < minK) {
41         cout << "N\n";
42         return 0;
43     }
```

13.13 Constructive - A Story About Tea (cont.)

```

48
49 long long delta = K - minK;
50 bool useG = (delta % 2 == 1);
51 long long padCount = useG ? (delta - 1) / 2 : delta / 2;
52
53 out.reserve((size_t)K * 4 + 16);
54
55 for (long long i = 0; i < padCount; i++) {
56     addMove('A', 'B');
57     addMove('B', 'A');
58 }
59
60 if (useG) {
61     g((int)N, 'A', 'C', 'B');
62 } else {
63     h((int)N, 'A', 'C', 'B');
64 }
65
66 cout << "Y\n";
67 cout << out;
68 return 0;
69 }

```

13.14 Conteo de Records - Harder Horizons

Resuelve "Harder Horizons" (problema H, Regionals 2025): cuenta cuantos elementos son mayores que TODOS los anteriores, o sea los "records" o maximos de prefijo. Una sola pasada llevando el maximo visto: si el actual lo supera, se cuenta y se actualiza. $O(n)$ y memoria $O(1)$, sin guardar el arreglo. El maxi arranca en -1 en vez de 0, que importa si los valores pueden ser 0: con maxi = 0 un primer elemento igual a 0 no se contaria. Si los valores pudieran ser negativos habria que arrancar en `LLONG_MIN`, o tratar el primer elemento aparte. Ojo con el Estricto: se cuenta solo si $x > \text{maxi}$. Si el enunciado considerara record a un empate habria que usar $\geq$, y es el tipo de detalle que cambia la respuesta sin que el codigo se vea distinto. Comprobado a mano: con 1 3 2 5 4 los records son 1, 3 y 5, o sea 3. El mismo barrido sirve para "cuantas veces cambia el maximo", "desde cuantas posiciones se ve el horizonte" y los problemas de edificios que tapan a otros.

C++\ICPC\Regionals 2025\H - Harder Horizons.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4
5 int main() {
6     ios::sync_with_stdio(false);
7     cin.tie(nullptr);
8     ll n;
9     cin >> n;
10    ll maxi = -1;
11    ll res = 0;
12    while(n--)
13    {
14        ll x; cin >> x;
15        if(x>maxi)
16        {
17            res++;
18            maxi = x;
19        }
20    }
21
22    cout << res << endl;
23
24    return 0;
25 }

```

13.15 Cycle Detection - Switches

Resuelve "Switches" (Day 8, problema I - Contest 08 [Avanzados]): hay N interruptores que alternan (encienden/apagan) subconjuntos de M focos; el conserje los presiona en el orden fijo 1,2,...,N,1,2,... y se detiene apenas todos los focos quedan apagados a la vez; dado el estado inicial de los focos y los focos asociados a cada interruptor, hay que contar cuantas veces presiona interruptores hasta lograrlo, o -1 si con esa estrategia nunca se apagan todos a la vez. Simula presionando los interruptores en orden ciclico, y guarda en el set "pos" cada estado visto como (configuracion de focos + indice del interruptor); si ese mismo par (estado,interruptor) se repite antes de apagar todo, esta en un ciclo sin solucion y responde -1.

C++\Training_Camp_2026\Day 8\I - Switches.py

```

1 n,m = list(map(int,input().split()))
2 bomb = ["0"]*m
3 k = list(map(int,input().split()))
4 for i in range(1,len(k)):
5     bomb[k[i]-1]="1"
6
7 q = bomb.count("0")
8 c = 0
9 if(q == m):
10    print(0)
11 else:
12    entrada = []
13    pos = set()
14    c = 0
15    for j in range(n):
16        #print(bomb)
17        w = input().split()
18        s = list(map(int,w))
19        entrada.append(s)
20        for i in range(1,len(s)):
21            if(bomb[s[i]-1] == "0"):
22                bomb[s[i]-1] = "1"
23            else:
24                bomb[s[i]-1] = "0"
25        q = bomb.count("0")
26        c += 1
27        if(q == m):
28            break
29        pos.add(str("".join(bomb))+str(j))
30
31    if(q == m):
32        print(c)
33    else:
34        i = 0
35        while True:
36            #print(bomb)
37            s = entrada[i]
38            for j in range(1,len(s)):
39                if(bomb[s[j]-1] == "0"):
40                    bomb[s[j]-1] = "1"
41                else:
42                    bomb[s[j]-1] = "0"
43            q = bomb.count("0")
44            c += 1
45            if(q == m):
46                print(c)
47                break
48            inte = str("".join(bomb))+str(i)
49            if(inte in pos):
50                print(-1)
51                break
52            pos.add(str("".join(bomb))+str(i))
53
54        i+=1
55        if(i >= len(entrada)):
56            i = 0

```

13.16 Date Parsing - Crochet

Calcula la diferencia de tiempo (hacia adelante) entre dos momentos dados como día de la semana + hora. Usa un arreglo de 14 días (2 semanas) para manejar el wrap-around cuando el segundo momento cae en la siguiente semana.

C++\RPC\Crochet.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using int64 = long long;
5
6 int main() {
7     string d1, d2;
8     string h1, h2;
9     cin >> d1 >> h1 >> d2 >> h2;
10
11     string dias[] = {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun",
12                     "Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"};
13
14     // Parsear horas HH:MM
15     int hh1, mm1, hh2, mm2;
16     sscanf(h1.c_str(), "%d:%d", &hh1, &mm1);
17     sscanf(h2.c_str(), "%d:%d", &hh2, &mm2);
18
19     int t1 = hh1 * 60 + mm1;
20     int t2 = hh2 * 60 + mm2;
21
22     // Encontrar el índice de d1 en la primera semana
23     int idx1 = 0;
24     for (int i = 0; i < 7; i++) {
25         if (dias[i] == d1) { idx1 = i; break; }
26     }
27
28     // Buscar d2 desde idx1 en adelante (en el arreglo de 14 días)
29     // Si es el mismo día y la hora de fin es menor o igual, avanzar a la siguiente semana
30     int idx2 = -1;
31     for (int i = idx1; i < 14; i++) {
32         if (dias[i] == d2) {
33             if (i == idx1 && t2 <= t1) continue;
34             idx2 = i;
35             break;
36         }
37     }
38
39     int64 diff = (int64)(idx2 - idx1) * 24 * 60 + (t2 - t1);
40
41     int64 days = diff / (24 * 60);
42     int64 hours = (diff % (24 * 60)) / 60;
43     int64 mins = diff % 60;
44
45     vector<string> parts;
46     if (days) parts.push_back(to_string(days) + (days == 1 ? " day" : " days"));
47     if (hours) parts.push_back(to_string(hours) + (hours == 1 ? " hour" : " hours"));
48     if (mins) parts.push_back(to_string(mins) + (mins == 1 ? " minute" : " minutes"));
49
50     if (parts.size() == 3)
51         cout << parts[0] << ", " << parts[1] << ", " << parts[2] << "\n";
52     else if (parts.size() == 2)
53         cout << parts[0] << " and " << parts[1] << "\n";
54     else
55         cout << parts[0] << "\n";
56
57     return 0;
58 }
```

13.17 Día Siguiente y Anos Bisiestos

O: (1) siguienteDia, (n) el barrido de registros

Uso: siguienteDia(D1,M1,Y1, D2,M2,Y2) -> la segunda fecha es el día de despues?

Aritmetica de calendario sin usar <ctime>, que es lo que uno necesita cuando el enunciado da fechas como tres enteros. El arreglo dias[] esta 1-INDEXADO a proposito (dias[0] = 0), asi que dias[M] es directamente lo que mide el mes M y no hay que restar 1 en ningun lado. Avanzar un día son tres casos encadenados, y en ese orden: si el día se pasa del largo del mes se vuelve 1 y sube el mes; si el mes llega a 13 se vuelve 1 y sube el año. Escrito así cubre solo el 31 de diciembre con el mismo if de siempre, sin caso especial. EL AÑO BISIESTO NO ES "divisible por 4". La regla completa son tres condiciones: divisible por 400 -> bisiesto (2000 si) divisible por 4 pero NO por 100 -> bisiesto (2024 si) el resto -> no (1900 NO, 2023 no) Los años de siglo son la trampa: 1900 y 2100 no son bisiestos aunque sean divisibles por 4. Si el rango de años del problema no toca un cambio de siglo, Y % 4 == 0 alcanza, pero no cuesta nada escribir la regla buena. OJO: dias[] se declara ADENTRO de la función, así que el dias[2] = 29 se deshace en cada llamada. Si se saca a variable global hay que acordarse de devolverlo a 28. El main de abajo es el uso típico: se leen registros (fecha + un contador) y se acumula la diferencia del contador solo entre registros de días CONSECUTIVOS. El patrón de comparar cada registro con el anterior guardando D0/M0/Y0/C0 sirve para cualquier serie por fechas. VERIFICADO contra datetime de Python: los 87.372 días de 1896 a 2104 y 40.000 pares al azar, 0 fallos, incluyendo 1900 (no bisiesto) y 2000 (si).

C++\Notebook\Dia Siguiente y Anos Bisiestos.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool siguienteDia(long long D1, long long M1, long long Y1,
5                  long long D2, long long M2, long long Y2)
6 {
7     int dias[] = {0,31,28,31,30,31,30,31,31,30,31,30,31};
8
9     if(Y1 % 400 == 0 || (Y1 % 4 == 0 && Y1 % 100 != 0))
10         dias[2] = 29;
11
12     D1++;
13
14     if(D1 > dias[M1])
15     {
16         D1 = 1;
17         M1++;
18
19         if(M1 == 13)
20         {
21             M1 = 1;
22             Y1++;
23         }
24     }
25
26     return D1 == D2 && M1 == M2 && Y1 == Y2;
27 }
28
29 int main()
30 {
31     ios::sync_with_stdio(false);
32     cin.tie(nullptr);
33
34     int n;
35
36     while(cin >> n && n)
37     {
38         long long ans = 0, cnt = 0;
39
40         long long D0, M0, Y0, C0;
41
42         for(int i = 0; i < n; i++)
43         {
44             long long D, M, Y, C;
45             cin >> D >> M >> Y >> C;
46
47             if(i > 0 && siguienteDia(D0, M0, Y0, D, M, Y))
48             {
49                 cnt++;
50                 ans += C - C0;
51             }
52         }
53     }
```

13.17 Dia Siguiente y Anos Bisiestos (cont.)

```

51     }
52
53     D0 = D;
54     M0 = M;
55     Y0 = Y;
56     C0 = C;
57     }
58
59     cout << cnt << ' ' << ans << '\n';
60 }
61 }
```

13.18 Fuerza Bruta $O(n^2)$ sobre Subperiodos - Accounting Qualifying

La otra forma de resolver "Account Qualifying" (problema A, ICPC 2025): probar TODOS los subperiodos. Se mapea deposito a +1, retiro a -1 y saldo a 0, y cada tramo con suma 0 tiene tantos depositos como retiros. Es $O(n^2)$, que con $n = 10000$ son 10^8 sumas. Medido: 5 casos del tamaño máximo en 94 ms, o sea que pasa, pero la version con sumas de prefijo (el otro archivo A de esta carpeta) hace lo mismo en $O(n)$ y tarda 20 ms. POR QUE NO TIENE PODAS: la primera version de este archivo intentaba cortar el ciclo interno contando cuantos positivos y negativos quedaban disponibles, y estaba mal: el break se evaluaba ANTES de comprobar si la suma era 0, así que descartaba tramos validos. Con el primer caso del sample daba $r = 3$ en vez de 4. Una fuerza bruta sin podas es corta y no se equivoca; si hace falta velocidad, lo que corresponde es cambiar de algoritmo, no parchear el ciclo. Lo de d y w es igual que en la otra version: si no hay depositos d es 0 y si no hay retiros w es 0, de ahí el $\max(0, \dots)$ y el $\min(0, \dots)$. Verificado contra fuerza bruta en 2900 casos, sin fallos.

C++\ICPC\2025A - Accounting Qualifying.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      ios::sync_with_stdio(false);
6      cin.tie(nullptr);
7
8      int n;
9
10     while (cin >> n && n != 0) {
11
12         vector<int> a(n);
13
14         for (int &x : a)
15             cin >> x;
16
17         // Si no hay depositos d es 0, y si no hay retiros w es 0: por eso se
18         // acotan con 0 en vez de tomar el maximo y el minimo a secas.
19         int d = max(0, *max_element(a.begin(), a.end()));
20         int w = min(0, *min_element(a.begin(), a.end()));
21
22         // +1 = deposito
23         // 0 = balance
24         // -1 = retiro
25         for (int &x : a) {
26             if (x > 0) x = 1;
27             else if (x < 0) x = -1;
28         }
29
30         int r = 0;
31
32         // Todos los subperiodos, sin podas: si la suma de signos es 0, hay
33         // tantos depositos como retiros.
34         for (int i = 0; i < n; i++) {
35
36             int sum = 0;
37
38             for (int j = i; j < n; j++) {
39
40                 sum += a[j];
41
42                 if (sum == 0)
43                     r = max(r, j - i + 1);
44             }
45         }
```

13.18 Fuerza Bruta $O(n^2)$ sobre Subperiodos - Accounting Qualifying (cont.)

```

45     }
46
47     cout << d << ' ' << w << ' ' << r << '\n';
48 }
49
50 return 0;
51 }
```

13.19 Generacion de Numeros con Digitos Distintos - Division Digitos Distintos

Para cada N del caso de prueba, busca todos los pares $(s1, s2)$ con $s1 / s2 = N$ donde $s1$ y $s2$ tienen, cada uno por su lado, TODOS sus digitos distintos, y los imprime como " $s1 / s2 = N$ ". La idea es precalcular UNA SOLA VEZ la lista de todos los numeros con digitos distintos, y despues cada consulta solo recorre esa lista probando $s1 = s2 * N$. CUANTOS SON: el primer dígito tiene 9 opciones (1-9, el 0 no abre), el segundo vuelve a tener 9 (los diez menos el ya usado, y ahí el 0 si vale), el tercero 8, y así bajando. Sumando los largos de 1 a 10 dígitos: $9 + 81 + 648 + 4536 + 27216 + 136080 + 544320 + 1632960 + 3265920 + 3265920 = 8877690$ Los de 9 y 10 dígitos son la misma cantidad porque al decimo ya solo queda una opcion, y entre esos dos largos esta el 73% del total. Son ~68 MB de vector, así que el reserve no es adorno: sin el, el vector duplica capacidad hasta 128 MB y puede costar el límite de memoria. LA GENERACION es backtracking puro con el esqueleto de siempre: marcar el dígito, recursar, desmarcar. Se usa un bool usado[10] en vez de una mascara de bits a proposito, porque usado[d] se lee de una y mask & (1 << d) hay que decodificarlo. El desmarcado explicito es lo que deja ver que la rama del 2 puede volver a usar el 1. No hace falta comparar contra MAX_NUM al generar: el mayor numero con 10 dígitos distintos ES 9876543210, así que la recursion nunca se pasa sola. EL CORTE DE LA CONSULTA: como la lista esta ordenada, apenas $s2 > MAX_NUM / N$ el producto ya se sale y todos los siguientes tambien, así que se rompe el ciclo. Con N grande eso deja el recorrido en nada; con $N = 2$ todavia visita 7 millones, que es el peor caso. OJO CON DOS COSAS QUE CUESTAN TIEMPO Y NO SE VEN: - Imprimir con endl hace flush en CADA linea. Con $N = 2$ salen 680425 lineas y ahí se va la mayor parte del tiempo. Con '\n' la salida es identica y el caso completo baja de 2084 ms a 1390 ms (medido). - Si a la funcion de generar se le pone un tercer parametro y se llama generate(0, 0, 0), choca con std::generate de <algorithm> por el using namespace std y NO compila. Por eso aqui se llama armar. Es el mismo problema que tiene llamar remove a una funcion propia. Verificado: la salida es identica, linea por linea, a la version original que paso el juez, sobre 709320 lineas incluyendo el peor caso $N = 2$.

C++\CCPL\divisionDigitosDistintos.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5
6  const ll MAX_NUM = 9876543210LL;
7
8  vector<ll> nums;
9
10 // usado[d] = true si el dígito d ya esta puesto en el numero que armamos.
11 bool usado[10];
12
13 // Arma numeros dígito por dígito, probando todas las opciones.
14 // numero = lo que llevamos armado hasta ahora
15 // cuantos = cuantos digitos tiene ese numero
16 // Cada llamada decide CUAL es el siguiente dígito que se pega al final.
17 void armar(ll numero, int cuantos) {
18     // Si ya pusimos por lo menos un dígito, lo que llevamos YA es un numero
19     // valido (todos sus digitos son distintos), así que se guarda.
20     if (cuantos > 0) {
21         nums.push_back(numero);
22     }
23
24     // Con 10 digitos distintos ya se usaron todos, no hay mas que pegar.
25     if (cuantos == 10) {
26         return;
27     }
28
29     // Se prueba cada dígito que todavia no hayamos usado.
30     for (int d = 0; d <= 9; d++) {
31         if (usado[d]) {
32             continue;
33         }
34     }
```

13.19 Generacion de Numeros con Digos Distintos - Division Digos Distintos (cont.)

```

34
35 // El 0 no puede ir de primero (021 no es un numero de 3 digitos).
36 if (cuantos == 0 && d == 0) {
37     continue;
38 }
39
40 usado[d] = true; // se marca: ya lo usamos
41
42 armar(numero * 10 + d, cuantos + 1); // se pega al final y se sigue
43
44 usado[d] = false; // se libera para la otra rama
45 }
46 }
47
48 // Revisa si x tiene todos sus digitos distintos, sacandolos uno por uno.
49 bool distinctDigits(ll x) {
50     int mask = 0;
51
52     while (x > 0) {
53         int d = x % 10;
54         x /= 10;
55
56         if (mask & (1 << d))
57             return false;
58
59         mask |= (1 << d);
60     }
61
62     return true;
63 }
64
65 int main() {
66     ios::sync_with_stdio(false);
67     cin.tie(nullptr);
68
69     nums.reserve(9000000); // el total es 8877690, sin realocar
70
71     armar(0, 0);
72
73     sort(nums.begin(), nums.end());
74
75     int T;
76     cin >> T;
77
78     while (T--) {
79         ll N;
80         cin >> N;
81
82         for (ll s2 : nums) {
83             // Si s2 * N se pasa del maximo, los siguientes tambien: corta.
84             if (s2 > MAX_NUM / N)
85                 break;
86
87             ll s1 = s2 * N;
88
89             if (!distinctDigits(s1))
90                 continue;
91
92             cout << s1 << " / " << s2 << " = " << N << '\n';
93         }
94
95         if (T)
96             cout << '\n';
97     }
98
99     return 0;
100 }

```

13.20 Greedy - Escalator

Resuelve "Escalator" (Day 4, problema C - Contest 04 [Avanzados]): una escalera doble tarda 10 segundos en cruzar a cualquier persona, arranca en la direccion del primero que llega estando detenida, y si alguien llega en direccion contraria a la que se mueve debe esperar a que pare; hay que hallar el momento en que la escalera se detiene por ultima vez. Simula el estado de la escalera con variables `busy_until` (cuando quedaria libre), `dir` (direccion actual) y `waiting` (si hay alguien esperando un cambio de sentido), resolviendo en $O(N)$ los cambios de direccion pendientes antes de procesar la llegada de cada persona.

C++\Training_Camp_2026\Day 4\C - Escalator.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main(){
5     int n;
6     cin >> n;
7
8     long long busy_until = 0;
9     int dir = -1;
10    bool waiting = false;
11
12    for (int i = 0; i < n; i++){
13        long long t; int d;
14        cin >> t >> d;
15
16        while (waiting && t >= busy_until){
17            dir = 1 - dir;
18            busy_until = busy_until + 10;
19            waiting = false;
20        }
21
22        if (busy_until <= t){
23            dir = d;
24            busy_until = t + 10;
25        } else {
26            if (d == dir){
27                busy_until = max(busy_until, t + 10);
28            } else {
29                waiting = true;
30            }
31        }
32    }
33
34    if (waiting){
35        busy_until = busy_until + 10;
36    }
37
38    cout << busy_until << "\n";
39    return 0;
40 }

```

13.21 Lectura de Entrada

O: (1) por token, es E/S

Uso: los seis patrones de lectura: EOF, centinela, T casos, bloques, línea, n
 Catalogo de patrones para leer entradas "raras": hasta EOF, con centinela, con T casos, bloques separados por líneas en blanco, grids de caracteres, tokens con separadores mixtos y enteros gigantes como string. La regla practica: si el problema NO da un T inicial, piensa en EOF o centinela; si la entrada trae blancos o bloques, usa `getline` mas `istream` en vez de `cin >>`; y si alternas `cin >>` con `getline`, consume siempre el salto de línea pendiente con un `getline` extra o el parseo se descuadra. Cada patron esta en su propia funcion para poder copiar solo el que se necesita.

C++\Notebook\Lectura de Entrada.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 // 1) Hasta EOF: no hay T y el archivo termina sin aviso
6 void leerHastaEOF()
7 {
8     int a, b;

```

13.21 Lectura de Entrada (cont.)

```

9   while (cin >> a >> b)
10  {
11      // procesar (a, b)
12  }
13  }
14
15  // 2) Hasta centinela (clasico "0 0" para terminar)
16  void leerHastaCentinela()
17  {
18      int a, b;
19      while (cin >> a >> b)
20      {
21          if (a == 0 && b == 0) break;
22          // procesar (a, b)
23      }
24  }
25
26  // 3) T al inicio y luego T casos
27  void leerTCasos()
28  {
29      int T;
30      if (cin >> T)
31      {
32          while (T-->0)
33          {
34              int n, m;
35              cin >> n >> m;
36              // procesar
37          }
38      }
39  }
40
41  // 4) Cabecera por caso: cada caso empieza con "n m" y trae m lineas
42  void leerCabeceraPorCaso()
43  {
44      int n, m;
45      while (cin >> n >> m)
46      {
47          vector<pair<int,int>> edges;
48          edges.reserve(m);
49          for (int i = 0; i < m; i++)
50          {
51              int u, v;
52              cin >> u >> v;
53              edges.push_back({u, v});
54          }
55          // procesar caso
56      }
57  }
58
59  // 5) Bloques separados por lineas en blanco, sin saber cuantos hay
60  void leerBloques()
61  {
62      string line;
63      while (true)
64      {
65          // saltar lineas vacias entre bloques
66          while (getline(cin, line) && line.empty()) {}
67          if (!cin) break;
68
69          vector<string> bloque;
70          do
71          {
72              if (!line.empty()) bloque.push_back(line);
73          } while (getline(cin, line) && !line.empty());
74
75          // parsear cada linea del bloque con istringstream
76          for (auto &L : bloque)
77          {
78              istringstream iss(L);
79              int x;
80              while (iss >> x)

```

13.21 Lectura de Entrada (cont.)

```

81      {
82          // procesar x
83      }
84  }
85  }
86
87  // 6) Mezclar cin >> con getline: hay que consumir el '\n' colgante
88  void leerLineaConEspacios()
89  {
90      int n;
91      cin >> n;
92      string line;
93      getline(cin, line); // consume el salto de linea pendiente
94      getline(cin, line); // esta si es la linea util (puede traer espacios)
95  }
96
97  // 7) Leer N numeros aunque vengan repartidos en varias lineas
98  void leerNNumeros()
99  {
100     int n;
101     cin >> n;
102     vector<long long> a;
103     a.reserve(n);
104     while ((int)a.size() < n)
105     {
106         long long x;
107         if (!(cin >> x)) break;
108         a.push_back(x);
109     }
110 }
111
112 // 8) Grid de caracteres R x C (getline es mas robusto que >> por fila)
113 void leerGrid()
114 {
115     int R, C;
116     cin >> R >> C;
117     string dummy;
118     getline(cin, dummy); // consume el '\n'
119     vector<string> g(R);
120     for (int i = 0; i < R; i++)
121     {
122         getline(cin, g[i]);
123     }
124 }
125
126 // 9) Leer linea por linea y tokenizar, saltando vacias y comentarios
127 void leerYTokenizar()
128 {
129     string line;
130     while (getline(cin, line))
131     {
132         if (line.empty() || line[0] == '#') continue;
133         istringstream iss(line);
134         int x;
135         while (iss >> x)
136         {
137             // procesar x
138         }
139     }
140 }
141
142 // 10) Tokens con separadores mixtos (comas, tabs, espacios)
143 void leerSeparadoresMixtos()
144 {
145     string line;
146     getline(cin, line);
147     for (char &c : line)
148     {
149         if (c == ',' || c == '\t' || c == ' ');
150     }
151     istringstream iss(line);

```

13.21 Lectura de Entrada (cont.)

```

53     vector<string> tok;
54     for (string t; iss >> t; ) tok.push_back(t);
55 }
56
57 // 11) Enteros gigantes: leerlos como string para no desbordar
58 void leerBigInt()
59 {
60     string P;
61     while (cin >> P)
62     {
63         // procesar P digito por digito (ver modString)
64     }
65 }
66
67 // 12) Consumir TODO el archivo como un solo string (parser propio)
68 void leerTodo()
69 {
70     string all((istreambuf_iterator<char>(cin)), istreambuf_iterator<char>());
71 }
72
73 // 13) Normalizar fin de linea de Windows (CRLF) y espacios finales
74 void rtrim(string &s)
75 {
76     while (!s.empty() && (s.back() == '\r' || s.back() == ' ' || s.back() == '\t'))
77     {
78         s.pop_back();
79     }
80 }
81
82 int main()
83 {
84     ios_base::sync_with_stdio(false);
85     cin.tie(nullptr);
86     return 0;
87 }

```

13.22 Lights Out (Primera Fila Forzada) - Turnswitch

Resuelve "Turnswitch" (problema F, ICPC 2024). Una grilla $n \times n$ de interruptores que estan en '|' o en '-'; girar uno lo cambia a el y a sus cuatro vecinos. El codigo busca el minimo de giros para dejar todo igual, probando las dos opciones (todo '|' o todo '-'). Es un "Lights Out", y tiene dos datos que lo desarmen. Primero, el orden de los giros no importa y girar dos veces el mismo es no girarlo, asi que cada interruptor se gira 0 o 1 vez. Segundo, y es la clave: SI SE DECIDE LA PRIMERA FILA, TODO LO DEMAS QUEDA FORZADO. Una vez resuelta la fila $r-1$, la unica forma de arreglar la casilla $(r-1, c)$ que siga mal es girar (r, c) , porque es el ultimo interruptor que la toca. Entonces la fila siguiente no se elige: se deduce. Asi la busqueda baja de $2^{(n \times n)}$ a 2^n combinaciones de la primera fila, cada una completada en $O(n^2)$, y al final solo hay que ver si la ultima fila quedo bien. CUANDO USAR: cualquier puzzle de "presionar cambia a los vecinos" en una grilla. Si n es grande para 2^n , el mismo sistema se resuelve con eliminacion gaussiana sobre GF(2).

C++ \ICPC\2024\F - Turnswitch.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9
10    while (cin >> n && n != 0) {
11
12        vector<string> a(n);
13
14        for (auto &row : a)
15            cin >> row;
16
17        int ans = 1e9;
18
19        // Probamos los dos objetivos:
20        // 1. Todas verticales '|'

```

13.22 Lights Out (Primera Fila Forzada) - Turnswitch (cont.)

```

21        // 2. Todas horizontales '-'
22        for (int target = 0; target < 2; target++) {
23
24            char wanted = (target == 0 ? '|' : '-');
25
26            // Probamos todas las formas de girar la primera fila
27            for (int mask = 0; mask < (1 << n); mask++) {
28
29                vector<string> b = a;
30                int cnt = 0;
31
32                // Funcion para girar una casilla
33                auto turn = [&](int r, int c) {
34                    cnt++;
35
36                    // Propia
37                    b[r][c] = (b[r][c] == '|') ? '-' : '|';
38
39                    // Arriba
40                    if (r > 0)
41                        b[r - 1][c] =
42                            (b[r - 1][c] == '|') ? '-' : '|';
43
44                    // Abajo
45                    if (r + 1 < n)
46                        b[r + 1][c] =
47                            (b[r + 1][c] == '|') ? '-' : '|';
48
49                    // Izquierda
50                    if (c > 0)
51                        b[r][c - 1] =
52                            (b[r][c - 1] == '|') ? '-' : '|';
53
54                    // Derecha
55                    if (c + 1 < n)
56                        b[r][c + 1] =
57                            (b[r][c + 1] == '|') ? '-' : '|';
58                };
59
60                // Decidimos que switches girar en la primera fila
61                for (int c = 0; c < n; c++) {
62                    if (mask & (1 << c))
63                        turn(0, c);
64                }
65
66                // Desde la segunda fila, los movimientos son obligatorios
67                for (int r = 1; r < n; r++) {
68                    for (int c = 0; c < n; c++) {
69
70                        // Si la casilla de arriba esta mal,
71                        // debemos girar esta.
72                        if (b[r - 1][c] != wanted)
73                            turn(r, c);
74                    }
75                }
76
77                // Comprobamos si la ultima fila quedo correcta
78                bool ok = true;
79
80                for (int c = 0; c < n; c++) {
81                    if (b[n - 1][c] != wanted) {
82                        ok = false;
83                        break;
84                    }
85                }
86
87                if (ok)
88                    ans = min(ans, cnt);
89            }
90        }
91
92        cout << ans << '\n';

```

13.22 Lights Out (Primera Fila Forzada) - Turnswitch (cont.)

```
93     }
94
95     return 0;
96 }
```

13.23 Maquina de Estados por Objeto - Fraud

NO ES MIO: código de Stiven Correa, del repo del equipo (carpeta RPC/). Resuelve "Fraud" (RPC 2026-07, problema F): una lista de pickup y putdown; decir si la secuencia es consistente, o sea que cada objeto se recoge una vez y se suelta una vez. Técnica: un diccionario con el estado de cada objeto, 0 sin tocar, 1 recogido y 2 soltado. Un pickup solo vale desde 0 y un putdown solo desde 1. Al final se exige que TODOS hayan quedado en 2, que es el chequeo que se olvida: la secuencia puede ser valida paso a paso y dejar un objeto en la mano.

RPC\2026-07\F - Fraud.py

```
1 from collections import defaultdict
2
3 n = int(input())
4
5 state = defaultdict(int)
6 ans = 'yes'
7
8 for _ in range(n):
9     inst, item = input().split()
10
11     if inst == 'pickup':
12         if state[item] != 0:
13             ans = 'no'
14             break
15         state[item] = 1
16
17     else:
18         if state[item] != 1:
19             ans = 'no'
20             break
21         state[item] = 2
22
23 if any(x != 2 for x in state.values()):
24     ans = 'no'
25
26 print(ans)
```

13.24 Maximizar Area del Rectangulo Envolvente - Kings Conquest

Resuelve "Kings Conquest" (problema K, Regionals 2025): dados unos reyes en el plano y k movimientos, maximizar el area del rectangulo que los encierra a todos. Solo importa el BOUNDING BOX, o sea minX, maxX, minY, maxY: el area es (ancho)*(alto) y los reyes de adentro no cambian nada. Con esa reduccion hay dos formas de gastar los k movimientos: - bestTwoKing: repartir los k entre ancho y alto, probando las k+1 divisiones posibles. Sale de estirar la caja por lados opuestos. - bestOneKing: mover UN rey hacia una de las cuatro diagonales (+-k, +-k) y recalculer la caja. Cubre el caso en que conviene sacar un solo rey lejos en vez de repartir. Se toma el maximo de las dos. El caso n == 1 va aparte: con un solo rey el area es 1 y las formulas no aplican. OJO CON EL DESBORDE: el area es ancho por alto y las coordenadas pueden ser grandes, así que todo va en long long. Es el tipo de problema donde la respuesta cabe pero el producto intermedio no, si se descuida el tipo. El +1 en w y h es porque se cuentan CASILLAS y no distancias: de la coordenada 3 a la 7 hay 5 casillas, no 4. OJO: usa structured bindings (auto [a, b]), que piden C++17. En el juez compila, pero con un g++ viejo hay que volver a .first y .second.

C++\ICPC\Regionals 2025\K - Kings Conquest.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4 #define SIZE(c) int((c).size())
5 #define forn(i,n) for(int i = 0 ; i < n ; i++)
6
7 const int DIGITS = 10;
8
9 ll bestTwoKing(ll w, ll h, ll k)
10 {
11     ll maxi = -1;
12
13     forn(a, k + 1)
```

13.24 Maximizar Area del Rectangulo Envolvente - Kings Conquest (cont.)

```
14 {
15     maxi = max(maxi, (w + a) * (h + k - a));
16 }
17
18 return maxi;
19 }
20
21 ll bestOneKing(vector<pair<ll,ll>>& reyes,
22               ll minX, ll maxX,
23               ll minY, ll maxY,
24               ll w, ll h, ll k)
25 {
26     ll best = w * h;
27
28     vector<pair<ll,ll>> movimientos = {
29         {-k, -k},
30         {-k, k},
31         {k, -k},
32         {k, k}
33     };
34
35     for(auto [x, y] : reyes)
36     {
37         for(auto [dx, dy] : movimientos)
38         {
39             ll nx = x + dx;
40             ll ny = y + dy;
41
42             ll wc = max(maxX, nx) - min(minX, nx) + 1;
43             ll hc = max(maxY, ny) - min(minY, ny) + 1;
44
45             ll opcion = wc * hc;
46
47             best = max(best, opcion);
48         }
49     }
50
51     return best;
52 }
53
54 int main() {
55
56     ios::sync_with_stdio(false);
57     cin.tie(nullptr);
58
59     ll n,k;
60     cin >> n >> k;
61
62     vector<pair<ll,ll>> reyes(n);
63
64     forn(i,n)
65     {
66         cin >> reyes[i].first >> reyes[i].second;
67     }
68
69     if(n==1)
70     {
71         cout << 1 << endl;
72         return 0;
73     }
74
75     ll maxX = LLONG_MIN;
76     ll maxY = LLONG_MIN;
77     ll minX = LLONG_MAX;
78     ll minY = LLONG_MAX;
79
80     for (auto [x, y] : reyes) {
81         maxX = max(maxX, x);
82         maxY = max(maxY, y);
83         minX = min(minX, x);
84         minY = min(minY, y);
85     }
```

13.24 Maximizar Area del Rectangulo Envolvente - Kings Conquest (cont.)

```

86
87     ll w = maxX - minX + 1;
88     ll h = maxY - minY + 1;
89
90     ll one = bestOneKing(reyes, minX, maxX, minY, maxY, w, h, k);
91     ll two = bestTwoKing(w, h, k);
92
93     cout << max(one, two) << endl;
94
95
96     return 0;
97 }

```

13.25 Simulacion con Parsing deCodigo - Impossible Primebox

Resuelve "Impossible Primebox" (problema I, ICPC 2025): hay L "primelocks", cada uno con la forma $\text{prlck } i: x = x \text{ [} * \text{ Ai]} \text{ [} + \text{ Bi]} \text{ if Pi div x jump to } \{\text{prlck } j \mid \text{end}\} \text{ else jump to } \{\text{prlck } m \mid \text{end}\}$. Se arranca en el candado 0 con un x inicial y se sigue saltando hasta caer en un end. Hay que hallar el MENOR x inicial con el que el candado i se ejecuta exactamente K_i veces, para todos. El enunciado promete que la clave existe y es menor que 50000, así que la solución es probar $x = 0, 1, 2, \dots$ y quedarse con el primero que cumpla: no hay que invertir nada. LO QUE DE VERDAD CUESTA ES LEER LA ENTRADA, porque el formato tiene partes OPCIONALES: el " $* \text{ Ai}$ " puede no estar, el " $+ \text{ Bi}$ " puede no estar, y los destinos son " $\text{prlck } j$ " o " end ", que ocupan distinta cantidad de palabras. Leer con $>>$ palabra por palabra y decidir según lo que aparece es mucho más seguro que intentar un patrón fijo con `getline`. línea 2: " $x = x$ " y después, mientras siga habiendo operador, se lee " $* \text{ A}$ " o " $+ \text{ B}$ ". línea 3: " $\text{if P div x jump to } \langle \text{destino} \rangle \text{ else jump to } \langle \text{destino} \rangle$ ", donde $\langle \text{destino} \rangle$ es " end " (una palabra) o " prlck " seguido del número (dos palabras). Para saber donde termina la línea 2 sin depender de saltos de línea, se mira si la siguiente palabra es " if ": ahí empieza la línea 3. " Pi div x " significa " $\text{Pi divide a } x$ ", o sea $x \% \text{Pi} == 0$. Leerlo al revés es el error fácil. EL CORTE POR CICLO INFINITO ES OBLIGATORIO: con un x malo el proceso puede no terminar nunca. Como cada K_i es a lo sumo 1000 y hay a lo sumo 10 candados, un recorrido válido no puede dar más de 10000 pasos; si se pasa de ahí, ese x no sirve y se corta. Además, apenas un contador supera su K_i ya se puede abandonar, y eso es lo que hace que la mayoría de los x se descarten en pocos pasos. SOBRE EL TIEMPO, PARA QUE NO SORPRENDA: el peor caso teórico es 50000 candidatos por 10000 pasos. Medido con 5 casos armados a propósito para que ninguna x sirva y el recorrido siempre llegue a los 10000 pasos: 15.5 s en total, unos 3 s por caso. El enunciado promete que la clave EXISTE y es menor que 50000, así que en cualquier entrada legítima se encuentra antes y no se paga ese costo; el sample completo corre en 12 ms. Si aun así diera TLE, lo siguiente sería detectar ciclos guardando los estados (candado, residuos) ya vistos. EL VALOR DE x NO SE PUEDE GUARDAR: con A hasta 1000 por paso y hasta 10000 pasos, x llega a tener miles de dígitos. Pero de x solo se pregunta si P lo divide, así que se lleva x MODULO el primo de cada candado. La transformación $x = x * A + B$ se traduce a los residuos sin cambiar nada, porque $(x * A + B) \bmod P$ depende solo de $x \bmod P$. Con $L \leq 10$ son 10 residuos por paso. Verificado con los dos casos del sample (10 y 255) y con una fuerza bruta independiente escrita aparte que simula lo mismo: 600 primeboxes aleatorios, sin diferencias.

C++\ICPC\2025\ - Impossible Primebox.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  typedef long long ll;
5
6  struct Lock {
7      ll A, B, P;
8      int siTrue, siFalse;    // -1 significa "end"
9  };
10
11 int main() {
12     ios::sync_with_stdio(false);
13     cin.tie(nullptr);
14
15     int L;
16
17     while (cin >> L && L != 0) {
18         vector<Lock> lk(L);
19
20         for (int i = 0; i < L; i++) {
21             string s;

```

13.25 Simulacion con Parsing deCodigo - Impossible Primebox (cont.)

```

24
25         // "prlck i:" (dos palabras)
26         cin >> s >> s;
27
28         // "x = x" y después los opcionales
29         cin >> s >> s >> s;
30
31         lk[i].A = 1;
32         lk[i].B = 0;
33
34         // Se leen operadores hasta toparse con el "if" de la línea 3.
35         while (cin >> s && s != "if") {
36             ll v;
37             cin >> v;
38
39             if (s == "**") lk[i].A = v;
40             else lk[i].B = v;
41         }
42
43         // "P div x jump to <destino> else jump to <destino>"
44         cin >> lk[i].P;
45         cin >> s >> s >> s;    // div x jump to
46
47         cin >> s;
48         if (s == "end") {
49             lk[i].siTrue = -1;
50         } else {
51             cin >> lk[i].siTrue;    // venia "prlck j"
52         }
53
54         cin >> s >> s;    // else jump to
55
56         cin >> s;
57         if (s == "end") {
58             lk[i].siFalse = -1;
59         } else {
60             cin >> lk[i].siFalse;
61         }
62     }
63
64     vector<int> K(L);
65     for (int i = 0; i < L; i++) cin >> K[i];
66
67     int pasosMax = 0;
68     for (int i = 0; i < L; i++) pasosMax += K[i];
69
70     // Con los primos DISTINTOS (a lo sumo 10 y cada uno <= 1000) se arma
71     // M = su producto. Si M cabe con margen para multiplicar por A, basta
72     // llevar UN residuo x mod M: como cada P divide a M, preguntar por
73     // r % P sigue valiendo, y el paso cuesta una multiplicación en vez de
74     // una por candado. Si M no cabe, se cae al vector de residuos.
75     vector<ll> primos;
76     for (int i = 0; i < L; i++) primos.push_back(lk[i].P);
77     sort(primos.begin(), primos.end());
78     primos.erase(unique(primos.begin(), primos.end()), primos.end());
79
80     const ll LIMITE = (11)9e18 / 1001;    // deja espacio para r*A + B
81     ll M = 1;
82     bool unico = true;
83
84     for (ll p : primos) {
85         if (M > LIMITE / p) { unico = false; break; }
86         M *= p;
87     }
88
89     ll clave = -1;
90
91     vector<int> veces(L), r(L);
92
93     for (ll x0 = 0; x0 < 50000 && clave < 0; x0++) {
94
95         // x se dispara (se multiplica por A hasta 1000 en cada paso) y no

```

13.25 Simulacion con Parsing deCodigo - Impossible Primebox (cont.)

```

96 // cabe en ningun entero. Pero de x solo se pregunta si es divisible
97 // por P, asi que basta llevar x modulo esos primos:
98 // (x*A + B) mod P se calcula con (x mod P), sin conocer x.
99 for (int i = 0; i < L; i++) veces[i] = 0;
100
101 ll ru = x0 % M;
102
103 if (!unico)
104     for (int i = 0; i < L; i++) r[i] = (int)(x0 % lk[i].P);
105
106 int cur = 0;
107 int pasos = 0;
108 bool sirve = true;
109
110 while (cur != -1) {
111     if (++pasos > pasosMax + 1) { // ciclo o demasiadas vueltas
112         sirve = false;
113         break;
114     }
115     veces[cur]++;
116
117     if (veces[cur] > K[cur]) { // ya se paso de su cuota
118         sirve = false;
119         break;
120     }
121
122     bool divide;
123
124     if (unico) {
125         ru = (ru * lk[cur].A + lk[cur].B) % M;
126         divide = (ru % lk[cur].P == 0);
127     } else {
128         for (int i = 0; i < L; i++)
129             r[i] = (int)((r[i] * lk[cur].A + lk[cur].B) % lk[i].P);
130         divide = (r[cur] == 0);
131     }
132
133     if (divide) cur = lk[cur].siTrue;
134     else cur = lk[cur].siFalse;
135
136     if (sirve && veces == K) clave = x0;
137 }
138
139 cout << clave << '\n';
140 }
141
142 return 0;
143 }

```

13.26 Simulation - Enigma Of The Jewelry Case

Resuelve "Enigma of the Jewelry Case" (Day 3, problema E - Maratona SBC de Programacao 2024): la princesa de Nlogonia tenia una caja NxN donde los valores crecen estrictamente por fila y por columna (de izquierda a derecha y de arriba a abajo); su hermana roto la caja 90 grados en sentido horario un numero desconocido de veces (0 a 3), y hay que hallar el minimo numero de rotaciones antihorarias necesarias para devolverla a su estado original. Como en el arreglo original el valor minimo siempre esta en la esquina superior izquierda, el codigo solo necesita mirar las 4 esquinas de la matriz recibida (pos), encontrar cual de ellas tiene el valor minimo (me = min(pos)) y su indice dentro de esa lista indica directamente, por la simetria de la rotacion, cuantas rotaciones antihorarias hacen falta.

C++\Training_Camp_2026\Day 3\E - Enigma of the Jewelry Case.py

```

1 n = int(input())
2 matrix = []
3 for i in range(n):
4     matrix.append(list(map(int,input().split())))
5 pos = [matrix[0][0], matrix[0][n-1], matrix[n-1][n-1], matrix[n-1][0]]
6 me = min(pos)
7 print(pos.index(me))

```

13.27 Template Base

O: (1), es el esqueleto

Uso: punto de partida: fast IO, lectura de matriz, gcd/lcm, setprecision

Plantilla de arranque para cualquier problema: includes, alias de tipos, constantes (MOD, INF, LLINF, EPS, PI), macros de recorrido (all, forn, forr, ford) y la desincronizacion de cin/cout que hace la lectura casi tan rapida como scanf. Incluye ademas los utilitarios que mas se olvidan en competencia: crear una matriz, imprimir con decimales fijos, borrar un rango de un vector, y obtener el indice (no el valor) del maximo o minimo. Ojo: __gcd / std::gcd y std::lcm ya vienen en la STL, no hace falta escribirlos a mano.

C++\Notebook\Template Base.cpp

```

1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 using ll = long long;
6
7 const int MOD = 1e9 + 7;
8 const ll LLINF = 0x3f3f3f3f3f3f3f3fLL;
9 const int INF = 0x3f3f3f3f;
10 const double EPS = 1e-9;
11 const double PI = acos(-1.0);
12
13 #define all(x) (x).begin(), (x).end()
14 #define forn(i, n) for(int i = 0; i < int(n); i++)
15 #define forr(i, a, b) for(int i = a; i <= b; i++)
16 #define ford(i, n) for(int i = int(n) - 1; i >= 0; i--)
17 #define endl '\n'
18
19 int main()
20 {
21     ios_base::sync_with_stdio(false);
22     cin.tie(nullptr);
23     cout.tie(nullptr);
24
25     int filas = 3, columnas = 4;
26     vector<vector<int>> matriz(filas, vector<int>(columnas, 0));
27
28     vector<int> v = {5, 1, 9, 3, 7};
29
30     // indice del maximo y del minimo (no el valor)
31     int idx_max = max_element(all(v)) - v.begin();
32     int idx_min = min_element(all(v)) - v.begin();
33     cout << idx_max << " " << idx_min << endl;
34
35     // borrar desde el indice 1 hasta el 3 (sin incluir el 3)
36     v.erase(v.begin() + 1, v.begin() + 3);
37
38     // imprimir con decimales fijos
39     double value = 1.0 / 3.0;
40     cout << fixed << setprecision(6) << value << endl;
41
42     // gcd ya viene en la STL (__gcd, o std::gcd/std::lcm si el juez tiene C++17).
43     // Para el lcm, dividir ANTES de multiplicar evita desbordar.
44     int a = 4, b = 6;
45     cout << __gcd(12, 18) << " " << (a / __gcd(a, b)) * b << endl;
46
47     return 0;
48 }

```

13.28 Umbral de Frecuencia - Lottery

NO ES MIO: codigo de Stiven Correa, del repo del equipo (carpeta RPC/). Resuelve "Lottery" (RPC 2026-07, problema L): de todos los numeros jugados, los que aparecen mas de 2n veces, en orden; -1 si no hay ninguno. Tecnica: diccionario de frecuencias y se agrega al resultado en el momento en que uno pasa el umbral, no al final. El set evita repetirlo si sigue apareciendo. El print(*res) desempaca la lista en argumentos y la imprime separada por espacios, que es la forma corta de sacar una lista en una sola linea en Python.

RPC\2026-07\L - Lottery.py

```

1 from collections import defaultdict
2 n = int(input())
3

```

13.28 Umbral de Frecuencia - Lottery (cont.)

```
4 dicc = defaultdict(int)
5 res = set()
6 total = 5*10*n
7 for _ in range(10*n):
8     nums = list(map(int, input().split()))
9     for i in nums:
10         dicc[i] += 1
11         if dicc[i] > 2*n:
12             res.add(i)
13 res = sorted(list(res))
14 if len(res) == 0:
15     print(-1)
16 else:
17     print(*res)
```

14 - Formulario

14.1 Algebra y Ecuaciones

Resolver e igualar ecuaciones: cuadratica con discriminante, Vieta para sacar suma y producto de raices sin calcularlas, sistemas 2x2 por Cramer, productos notables, leyes de exponentes y logaritmos, y proporciones y promedios. Incluye la version numericamente estable de la cuadratica, que importa cuando b^2 domina a $4ac$ y la resta directa pierde todos los digitos. Vieta y Cramer estan verificados.

C++\Formulario\Algebra y Ecuaciones.cpp

Ecuacion cuadratica $ax^2 + bx + c = 0$

```
x = (-b +/- sqrt(b^2 - 4ac)) / (2a)
discriminante D = b^2 - 4ac
D > 0 : dos raices reales distintas
D = 0 : una raiz doble, x = -b/(2a)
D < 0 : ninguna raiz real
```

Vieta (verificado) : $x_1 + x_2 = -b/a$ $x_1 * x_2 = c/a$
Sirve para responder sobre las raices sin calcularlas.

Version estable, cuando b^2 es mucho mayor que $4ac$:

```
q = -(b + sign(b)*sqrt(D)) / 2
x1 = q / a      x2 = c / q
```

Si $a = 0$ no es cuadratica: es $bx + c = 0$, y si $b = 0$ tambien hay que tratarlo aparte. Ese caso borde tumba muchas soluciones.

Sistema de dos ecuaciones (Cramer, verificado)

```
a1 x + b1 y = c1
a2 x + b2 y = c2
det = a1*b2 - a2*b1
det != 0 : solucion unica
det == 0 : o no hay solucion, o hay infinitas (misma recta)
x = (c1*b2 - c2*b1) / det      y = (a1*c2 - a2*c1) / det
Igualar dos funciones y = f(x), y = g(x) es resolver f(x)-g(x)=0.
```

Productos notables

```
(a + b)^2 = a^2 + 2ab + b^2
(a - b)^2 = a^2 - 2ab + b^2
(a + b)(a - b) = a^2 - b^2
(a + b)^3 = a^3 + 3a^2 b + 3a b^2 + b^3
a^3 + b^3 = (a + b)(a^2 - ab + b^2)
a^3 - b^3 = (a - b)(a^2 + ab + b^2)
a^n - b^n siempre es divisible por (a - b)
binomio de Newton : (a+b)^n = sum_k C(n,k) a^(n-k) b^k
```

Exponentes y logaritmos

```
a^m * a^n = a^(m+n)      a^m / a^n = a^(m-n)
(a^m)^n = a^(m*n)      (ab)^n = a^n b^n
a^0 = 1                  a^(-n) = 1 / a^n
a^(1/n) = raiz n-esima de a
log(x*y) = log x + log y  log(x/y) = log x - log y
log(x^n) = n * log x      log_b(x) = log(x) / log(b)
En C++ log() es natural; existen log2() y log10() aparte.
OJO: log2(n) puede dar 2.9999 por flotante. Si necesitas el
exponente exacto, usa 31 - __builtin_clz(n) o un bucle.
```

Proporciones y porcentajes

regla de tres: si a corresponde a b, entonces c da $b*c/a$
aumentar p% : $x * (1 + p/100)$ disminuir : $x * (1 - p/100)$
variacion porcentual = $(nuevo - viejo) / viejo * 100$

Subir p% y luego bajar p% NO devuelve al valor original.

promedio simple = $suma / n$
promedio ponderado = $sum(w_i * x_i) / sum(w_i)$
media geometrica = $(producto de los x_i)^{1/n}$
se usa para tasas de crecimiento encadenadas
media armonica = $n / sum(1/x_i)$
se usa para promediar velocidades sobre la misma distancia
Siempre: armonica <= geometrica <= aritmetica

Valor absoluto e inecuaciones

$|x| < a$ equivale a $-a < x < a$
 $|x| > a$ equivale a $x < -a$ o $x > a$
 $|a + b| <= |a| + |b|$ desigualdad triangular
Al multiplicar o dividir una inecuacion por un NEGATIVO,
el sentido se voltea. Es el error clasico al despejar.

En C++

Cuadratica numericamente estable. Devuelve cuantas raices reales hay.

```
int cuadratica(double a, double b, double c, double& x1, double& x2) {
    double D = b*b - 4*a*c;
    if (D < 0) return 0;
    double q = -(b + copysign(sqrt(D), b)) / 2;
    x1 = q / a;
    x2 = (fabs(q) > 1e-300) ? c / q : x1;
    if (x1 > x2) swap(x1, x2);
    return D > 0 ? 2 : 1;
}
El copysign evita restar dos numeros casi iguales. Con a=1, b=1e8,
c=1 la formula directa da 0 en una raiz; esta da -1e-8.
Sistema 2x2 por Cramer. false = no hay solucion unica.
bool cramer(double a1, double b1, double c1,
             double a2, double b2, double c2, double& x, double& y) {
    double d = a1*b2 - a2*b1;
    if (fabs(d) < 1e-12) return false;
    x = (c1*b2 - c2*b1) / d;
    y = (a1*c2 - a2*c1) / d;
    return true;
}
```

14.2 Areas y Volúmenes

Areas de figuras planas y volúmenes de cuerpos, incluidas las que uno cree recordar y termina equivocando: el rombo es $d1*d2/2$ (no lado por lado), el cono y la piramide llevan el tercio, y el perimetro de la elipse NO tiene formula cerrada elemental, solo la aproximacion de Ramanujan. Todas las formulas de aqui se comprobaron contra integracion numerica o contra el area por shoelace del poligono correspondiente.

C++\Formulario\Areas y Volúmenes.cpp

Figuras planas (A = area, P = perimetro)

```
cuadrado   A = L^2      P = 4L      diagonal = L*sqrt(2)
rectangulo A = b*h      P = 2(b+h) diagonal = sqrt(b^2+h^2)
rombo      A = d1*d2 / 2 P = 4L
las diagonales se cortan perpendicularmente en el centro
el lado sale de ellas: L = sqrt((d1/2)^2 + (d2/2)^2)
tambien A = L^2 * sin(angulo)
romboide   A = b*h      P = 2(a+b)
trapecio   A = (B + b) * h / 2
triangulo   A = b*h/2    (Heron esta en la ficha Geometria)
circulo     A = pi r^2    P = 2 pi r
corona circular A = pi (R^2 - r^2)
sector de angulo t en radianes
A = r^2 * t / 2      arco = r * t
elipse      A = pi a b
El perimetro no tiene formula cerrada elemental.
Ramanujan, APROXIMADO (error ~1e-5 con a=5, b=2, medido):
P ~ pi (3(a+b) - sqrt((3a+b)(a+3b)))
poligono regular de n lados de largo L (verificado)
```

14.2 Areas y Volúmenes (cont.)

$A = n * L^2 / (4 * \tan(\pi/n))$
 $\text{apotema} = L / (2 * \tan(\pi/n))$
 $\text{angulo interior} = (n-2)*180/n$
 $\text{angulo central} = 360/n$
 $\text{suma total} = (n-2)*180$

Cuerpos (V = volumen, S = area total)

cubo $V = a^3$ $S = 6a^2$
 $\text{diagonal} = a*\sqrt{3}$
ortopedro $V = a*b*c$ $S = 2(ab + ac + bc)$
 $\text{diagonal} = \sqrt{a^2 + b^2 + c^2}$
prisma $V = (\text{area base}) * h$
cilindro $V = \pi r^2 h$ $\text{lateral} = 2 \pi r h$
 $S = 2 \pi r (r + h)$
cono $V = \pi r^2 h / 3$ $\text{lateral} = \pi r g$
 $g = \sqrt{r^2 + h^2}$ $S = \pi r (r + g)$
esfera $V = 4/3 \pi r^3$ $S = 4 \pi r^2$
casquete esferico de altura h (verificado)
 $V = \pi h^2 (3r - h) / 3$
piramide $V = (\text{area base}) * h / 3$
tetraedro regular de arista a (verificado)
 $V = a^3 / (6*\sqrt{2})$ $S = \sqrt{3} * a^2$
toro, R al centro del tubo y r el radio del tubo (verificado)
 $V = 2 \pi^2 R r^2$ $S = 4 \pi^2 R r$

Reglas que se olvidan

Al escalar una figura por k: longitudes x k, areas x k², volúmenes x k³. Sirve para problemas de semejanza.
Pappus: al girar una figura plana alrededor de un eje externo,
 $V = \text{area de la figura} * \text{recorrido de su centroide.}$
El cono y la piramide son un tercio del prisma de igual base y altura; la esfera es dos tercios del cilindro que la encierra.

14.3 Combinatoria

Conteos clásicos con sus primeros valores para identificar la sucesión en la salida de ejemplo: 1, 2, 5, 14, 42 son Catalan; 1, 1, 2, 5, 15, 52 son Bell; 1, 0, 1, 2, 9, 44 son desarrreglos. Incluye el triángulo de Pascal hasta n=10 y las identidades que permiten cerrar una sumatoria antes de escribir una sola línea de código.

C++\Formulario\Combinatoria.cpp

Los cuatro conteos base

$P(n,k) = n!/(n-k)!$ ordenado, sin repetir
 $C(n,k) = n!/(k!(n-k)!)$ sin orden, sin repetir
 n^k ordenado, con repetición
 $C(n+k-1, k)$ sin orden, con repetición
 $n!/(a! b! c! \dots)$ permutaciones de un multiconjunto

Identidades de binomiales (sirven para colapsar sumatorias)

$C(n,k) = C(n-1,k-1) + C(n-1,k)$ Pascal
 $C(n,k) = C(n,n-k)$ simetría
 $\sum_k C(n,k) = 2^n$ fila completa
 $\sum_k (-1)^k C(n,k) = 0$ para $n \geq 1$
 $\sum_{i=k..n} C(i,k) = C(n+1,k+1)$ hockey stick
 $\sum_i C(m,i)C(n,k-i) = C(m+n,k)$ Vandermonde
 $kC(n,k) = nC(n-1,k-1)$ absorción

Triángulo de Pascal

n=0	1										
n=1	1	1									
n=2	1	2	1								
n=3	1	3	3	1							
n=4	1	4	6	4	1						
n=5	1	5	10	10	5	1					
n=6	1	6	15	20	15	6	1				
n=7	1	7	21	35	35	21	7	1			
n=8	1	8	28	56	70	56	28	8	1		
n=9	1	9	36	84	126	126	84	36	9	1	
n=10	1	10	45	120	210	252	210	120	45	10	1

Stars and bars: repartir n objetos iguales en k grupos

permitiendo grupos vacíos : $C(n+k-1, k-1)$
 todos con al menos uno : $C(n-1, k-1)$

Catalan $\text{Cat}(n) = C(2n,n)/(n+1) = \sum_i \text{Cat}(i)*\text{Cat}(n-1-i)$

Parentesis balanceados, arboles binarios de n nodos, triangulaciones de un polígono de n+2 lados, caminos de Dyck.

n	0	1	2	3	4	5	6	7	8	9
Cat(n)	1	1	2	5	14	42	132	429	1430	4862
Cat(12)	208012									
Cat(13)	742900									
Cat(14)	2674440									

Desarrreglos: permutaciones sin ningún punto fijo

$!n = (n-1)(!(n-1) + !(n-2)) \sim n!/e$

n	0	1	2	3	4	5	6
!n	1	0	1	2	9	44	265

Bell: formas de particionar un conjunto de n elementos

n	0	1	2	3	4	5	6	7	8
B(n)	1	1	2	5	15	52	203	877	4140

Inclusion-exclusion

$|A \cup B| = |A| + |B| - |A \cap B|$
 $|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$

General: sumar los de tamaño impar, restar los de tamaño par.

En C++

```

11 nCr(11 n, 11 k) { // 0(k), sin precomputar nada
    if (k < 0 || k > n) return 0;
    k = min(k, n - k);
    11 r = 1;
    for (11 i = 1; i <= k; i++) r = r * (n - i + 1) / i;
    return r; // la division siempre es exacta
}
11 nPr(11 n, 11 k) { // 0(k), con orden
    if (k < 0 || k > n) return 0;
    11 r = 1; // SIN el min(k, n-k): esa
    for (11 i = 0; i < k; i++) // simetria es solo de C
        r *= (n - i);
    return r; // crece rapido: P(60,10) cabe,
} // P(60,11) ya se desborda
11 catalan(int n) { // 0(n), sin binomiales
    11 c = 1;
    for (int i = 0; i < n; i++) c = c * 2 * (2*i + 1) / (i + 2);
    return c;
}
11 desarrreglos(int n) { // 0(n), sin memoria extra
    11 a = 1, b = 0;
    for (int i = 2; i <= n; i++) {
        11 c = (i-1) * (a + b); a = b; b = c;
    }
    return n == 0 ? 1 : b;
}
11 bell(int n) { // triangulo de Bell, 0(n^2)
    vector<11> f = {1};
    for (int i = 0; i < n; i++) {
        vector<11> g = {f.back()};
        for (11 x : f) g.push_back(g.back() + x);
        f = g;
    }
    return f[0];
}

```

Si n es grande y hay muchas consultas, precomputa factoriales e inversos (esta en la ficha Binomial Coefficients) y cada nCr sale en O(1).

14.4 Cotas y Limites

La tabla que se mira ANTES de escribir codigo: dado el n del enunciado, que complejidad cabe en el tiempo limite, hasta donde llega cada tipo entero y cuanto memoria ocupa un arreglo. Leer $n \leq 20$ y pensar en bitmask, o $n \leq 450$ y pensar en Floyd-Warshall, ahorra media hora de rumbo equivocado.

C++\Formulario\Cotas y Limites.cpp

n maximo que aguanta cada complejidad (~10^8 operaciones, 1 seg)

```
O(n!)      n <= 11
O(2^n * n) n <= 20
O(2^n)     n <= 25
O(n^3)     n <= 450
O(n^2 log n) n <= 5000
O(n^2)     n <= 10^4
O(n sqrt n) n <= 10^5
O(n log^2 n) n <= 10^5
O(n log n) n <= 10^6
O(n)       n <= 10^8
Si n <= 10^18 la respuesta es formula cerrada, log o binaria.
```

Rangos de los enteros

```
int          hasta 2147483647 (~2.1*10^9)
unsigned int hasta 4294967295
long long    hasta 9223372036854775807
unsigned long long hasta 18446744073709551615
__int128     hasta ~1.7*10^38 (no se imprime con cout)
~9.2*10^18 es el techo de long long: dos numeros de 10^9
multiplicados caben, tres no.
```

Precision de punto flotante

```
float ~7 digitos significativos
double ~15-16 digitos (usar siempre este)
long double ~18-19 en GCC
Comparar con EPS: fabs(a-b) < 1e-9, nunca a == b
Si el problema pide enteros exactos, evitar sqrt y pow.
```

Modulos comunes

```
1000000007 (10^9+7) primo, el mas usado
998244353 primo, el de NTT (= 119*2^23 + 1)
1000000009 primo
Con 10^9+7 dos residuos multiplicados llegan a ~10^18:
cabe en long long, pero sumar tres productos sin reducir NO.
```

Tamanos de memoria (limite tipico 256 MB)

```
int[10^7]      40 MB      long long[10^7]  80 MB
int[10^8]      400 MB     (no cabe)
bool[10^8]     100 MB     bitset<10^8>     12.5 MB
vector<vector<int>> de 10^4 x 10^4 = 400 MB, no cabe
```

14.5 Datos que no se Deducen

Hechos sueltos que si no te los sabes no los inventas en media hora de competencia: el xor de 1..n tiene periodo 4, el ultimo digito de a^b cicla cada 4, un numero es cuadrado perfecto si y solo si tiene una cantidad impar de divisores, y la cantidad de nodos de grado impar de cualquier grafo siempre es par. Las tablas de xor y de ultimo digito estan verificadas corriendo el codigo.

C++\Formulario\Datos que no se Deducen.cpp

XOR de 1 hasta n (verificado)

```
n % 4 == 0 -> n
n % 4 == 1 -> 1
n % 4 == 2 -> n + 1
n % 4 == 3 -> 0
n      : 1 2 3 4 5 6 7 8 9 10 11 12
1^..^n : 1 3 0 4 1 7 0 8 1 11 0 12
Xor de un rango [l,r] = f(r) ^ f(l-1)
```

Reglas de divisibilidad

```
2 : ultimo digito par      5 : termina en 0 o 5
3 : suma de digitos div 3  9 : suma de digitos div 9
```

14.5 Datos que no se Deducen (cont.)

```
4 : los ultimos 2 digitos      8 : los ultimos 3 digitos
6 : divisible por 2 y por 3
11 : suma alternada de digitos divisible por 11
7 : no hay regla comoda; saca el modulo digito a digito (Horner)
```

Ultimo digito de a^b : el ciclo dura a lo sumo 4 (verificado)

```
2: 2 4 8 6      3: 3 9 7 1      7: 7 9 3 1      8: 8 4 2 6
4: 4 6          9: 9 1          5: 5          6: 6
0: 0            1: 1
Entonces  $a^b \bmod 10$  depende de  $b \bmod 4$ , tratando  $b = 0$  aparte.
```

Cantidad de digitos

```
digitos de n = floor(log10(n)) + 1 para n >= 1
Mas segura sin flotantes: to_string(n).size()
digitos de n! = suma de log10(i) para i=1..n, redondeado arriba
```

Josephus: n en circulo, se elimina cada k-esima persona

```
k = 2 : escribe n = 2^m + L, la respuesta es 2L + 1
equivale a rotar a la izquierda el bit mas alto de n
k general (0-indexado):
J = 0; for (i = 2; i <= n; i++) J = (J + k) % i;
la respuesta 1-indexada es J + 1
```

Principio del palomar

```
n+1 objetos en n cajas -> alguna caja tiene 2 o mas
Entre n+1 numeros cualesquiera de 1..2n hay dos consecutivos
Con n+1 prefijos y n residuos, dos comparten residuo
-> siempre existe un subarreglo con suma divisible por n
```

Grafos: hechos que se olvidan

```
arbol de n nodos: exactamente n-1 aristas y ningun ciclo
grafo completo de n nodos: n(n-1)/2 aristas
suma de todos los grados = 2 * cantidad de aristas
la cantidad de nodos de grado impar siempre es PAR
camino euleriano existe si hay 0 o 2 nodos de grado impar
ciclo euleriano existe si todos los grados son pares
arboles etiquetados con n nodos: n^(n-2) (Cayley)
un grafo bipartito no tiene ciclos de longitud impar
```

Varios que aparecen disfrazados

```
ceros al final de n! = n/5 + n/25 + n/125 + ...
n es cuadrado perfecto <=> tiene una cantidad IMPAR de divisores
(de ahi el problema clasico de los casilleros que se abren)
suma de los primeros n impares = n^2
la mediana minimiza la suma de distancias absolutas
el promedio minimiza la suma de distancias al cuadrado
para repartir en dos grupos lo mas parejo posible: subset sum
(a+b) % m = ((a%m) + (b%m)) % m, igual con el producto,
pero NO con la division: ahi va el inverso modular
```

En C++

```
ll xorHasta(ll n) { // 1^2^...^n en O(1)
    switch (n % 4) {
        case 0: return n;
        case 1: return 1;
        case 2: return n + 1;
    }
    return 0;
}
xor del rango [l,r] = xorHasta(r) ^ xorHasta(l-1)
int josephus(int n, int k) { // O(n), 1-indexado
    int j = 0;
    for (int i = 2; i <= n; i++) j = (j + k) % i;
    return j + 1;
}
int josephus2(int n) { // caso k=2 en O(1)
    return 2 * (n - (1 << (31 - __builtin_clz(n)))) + 1;
}
```

14.5 Datos que no se Deducen (cont.)

```
int numDigitos(11 n) {           // sin log10, sin error de flotante
    int d = 0;
    while (n) d++, n /= 10;
    return d ? d : 1;
}
```

14.6 Geometria

Formulas de geometria que se usan sin pensar pero se olvidan bajo presion: areas, Heron, shoelace, Pick, sector y segmento circular, y el significado del signo del producto cruz. La regla que mas salva: si los datos son enteros, trabajar con cross/dot y distancias al cuadrado en vez de sqrt, y no comparar doubles con ==

C++\Formulario\Geometria.cpp

Triangulo

```
area = base * altura / 2
area = |cross(B-A, C-A)| / 2           con coordenadas
Heron: s=(a+b+c)/2, area = sqrt(s(s-a)(s-b)(s-c))
ley de cosenos : c^2 = a^2 + b^2 - 2ab*cos(C)
ley de senos   : a/sin(A) = b/sin(B) = c/sin(C) = 2R
desigualdad triangular: a+b > c para que exista
```

Poligono simple (vertices en orden)

Shoelace: area = |sum (x_i * y_{i+1} - x_{i+1} * y_i)| / 2
 El signo de la suma dice la orientacion: + antihorario.
 Regular de n lados y lado L: area = n*L^2 / (4*tan(pi/n))
 angulo interior = (n-2)*180/n grados

Pick: para poligono con vertices en enteros

```
area = I + B/2 - 1
I = puntos enteros interiores, B = puntos enteros en el borde
puntos enteros en el segmento (x1,y1)-(x2,y2):
gcd(|x2-x1|, |y2-y1|) + 1 (contando los dos extremos)
```

Circulo de radio r

```
perimetro = 2*pi*r           area = pi*r^2
sector de angulo t (rad) : area = r^2 * t / 2
segmento circular : area = r^2 (t - sin t) / 2
cuerda de angulo t : largo = 2r*sin(t/2)
PI = acos(-1.0) 1 rad = 180/pi grados
```

Producto cruz y punto (la base de todo)

```
cross(A,B) = ax*by - ay*bx
cross(A,B,C) = cross(B-A, C-A)
> 0 giro antihorario, < 0 horario, = 0 colineales
dot(A,B) = ax*bx + ay*by
> 0 angulo agudo, < 0 obtuso, = 0 perpendicular
Con enteros no uses sqrt: compara distancias al cuadrado.
```

Distancias

punto a recta por A,B : |cross(B-A, P-A)| / |B-A|
 Manhattan |dx|+|dy|, Chebyshev max(|dx|,|dy|)
 Rotar 45 grados convierte Manhattan en Chebyshev:
 (x,y) -> (x+y, x-y)

Euler para grafos planos y poliedros

V - E + F = 2
 Con n rectas en posicion general el plano queda en
 1 + n + C(n,2) regiones: 2, 4, 7, 11, 16, 22, 29, 37

14.7 Probabilidad Geometrica y Areas Raras

Para los problemas que en realidad son matematica pura y lo dificil no es programarlos sino plantear el area: "cual es la probabilidad de que un punto al azar del jardin caiga dentro de algun circulo", "cuanto vale en promedio", "que tan seguido se encuentran dos personas". La idea unica de toda la ficha: probabilidad = area favorable / area total, y el valor esperado es la suma de cada valor por su probabilidad, sin necesitar independencia. Trae las areas que no se sacan de memoria: interseccion de dos circulos, circulo con rectangulo, y los clasicos de la cita, el palito y la aguja de Buffon. Todo verificado con Monte Carlo en este mismo equipo.

C++\Formulario\Probabilidad Geometrica.cpp

La idea base

P(evento) = medida favorable / medida total
 En 1D la medida es largo, en 2D area, en 3D volumen.
 Si el punto se elige UNIFORME, no hay nada mas que calcular areas. El problema "de programacion" es en realidad geometria.

Valor esperado

E[X] = suma de (valor * probabilidad de ese valor)
 LINEALIDAD: E[A + B] = E[A] + E[B] SIEMPRE, aunque A y B esten correlacionados. Es lo que permite sumar la contribucion de cada objeto por separado en vez de analizar el conjunto.
 Ejemplo tipico: cada circulo i tiene valor v_i y cubre un area a_i del jardin de W x H, entonces
 respuesta = suma de v_i * (a_i / (W*H))
 Si los circulos se traslapan NO importa: la linealidad sigue valiendo, cada uno aporta su parte por separado.

Interseccion de DOS circulos (area de la lente) (verificado)

```
d = distancia entre centros
si d >= r1 + r2 -> 0, no se tocan
si d <= |r1 - r2| -> pi * min(r1,r2)^2, uno dentro del otro
si no:
    a1 = 2*acos((d*d + r1*r1 - r2*r2) / (2*d*r1))
    a2 = 2*acos((d*d + r2*r2 - r1*r1) / (2*d*r2))
    A = 0.5*r1*r1*(a1 - sin(a1)) + 0.5*r2*r2*(a2 - sin(a2))
O sea: la lente son dos segmentos circulares pegados.
union de los dos = pi r1^2 + pi r2^2 - A
```

Segmento y sector (los ladrillos de todo lo anterior)

```
sector de angulo t : A = r^2 * t / 2
segmento (lo que queda entre la cuerda y el arco):
    A = r^2 * (t - sin t) / 2
cuerda del angulo t : largo = 2 r sin(t/2)
```

Circulo contra rectangulo o poligono

No hay formula cerrada corta. Se recorre el borde del poligono sumando areas con signo, tratando cada arista aparte segun si el tramo cae dentro o fuera del circulo.
 Esta implementado en la seccion Geometry, ficha "Area Circulo-Poligono". Es el metodo del problema del jardin.

Que una FIGURA QUEPA: traducir "cabe" a desigualdades

Cuando piden la probabilidad de que algo quepa dentro de otra cosa, casi nunca se integra a lo bruto: conviene escribir el "cabe" como desigualdades sobre las coordenadas, que suelen simplificarse muchisimo.
 LA HERRAMIENTA CLAVE (arco capaz): el circulo que tiene al segmento AB como DIAMETRO cabe dentro de un convexo K si y solo si todo punto de la frontera de K ve a AB con angulo de a lo sumo 90 grados. Es porque ese circulo es justo donde el angulo vale 90, y por fuera de el es menor.
 "Angulo <= 90 desde Q" se escribe sin trigonometria:
 (A - Q) . (B - Q) >= 0
 y si Q recorre un lado recto, se minimiza ese producto punto.
 Ejemplo, para el lado x = 0 con Q = (0,q):
 x1*x2 + (y1-q)*(y2-q) >= 0 para todo q
 el minimo cae en q = (y1+y2)/2 y vale x1*x2 - (y1-y2)^2/4
 o sea la condicion es 4*x1*x2 >= (y1-y2)^2
 En un rectangulo L x W salen cuatro asi, sin una sola raiz:

14.7 Probabilidad Geometrica y Areas Raras (cont.)

$$4*x1*x2 \geq (y1-y2)^2 \quad 4*(W-x1)*(W-x2) \geq (y1-y2)^2$$

$$4*y1*y2 \geq (x1-x2)^2 \quad 4*(L-y1)*(L-y2) \geq (x1-x2)^2$$

(verificado contra la condicion directa de centro y radio en 3000000 de pruebas, sin una sola diferencia)

Y como las desigualdades son homogeneas de grado 2, escalar el rectangulo no cambia la probabilidad: solo importa la PROPORCION entre los lados. Para dos piedras uniformes en L x W, con

$$t = \min(L,W)/\max(L,W), \text{ la probabilidad de que el círculo quepa es } P = (\pi/6) * t * (2 - t)$$

(formula verificada numericamente en 8 proporciones, no demostrada; el problema es "I - Omens" de ICPC 2024, esta en este cuaderno)

Para transferible es el metodo, no la formula: primero traducir "cabe" a desigualdades, despues mirar si colapsan.

Uniones de varias figuras

$$|A \cup B| = |A| + |B| - |A \cap B|$$

Con tres o mas, inclusion-exclusion (2^n terminos): solo sirve si n es chico. Si no, integracion por barrido o Monte Carlo.

Para "cuanta area cubre al menos un círculo" con muchos círculos, lo estandar es barrido angular, no inclusion-exclusion.

Clasicos que se resuelven dibujando el espacio de casos

Cita: dos personas llegan uniformemente en [0,T] y se esperan t minutos. El espacio es el cuadrado T x T, y se encuentran en

$$\text{la franja } |x - y| \leq t. \quad (\text{verificado})$$

$$P = 1 - ((T - t)/T)^2$$

Palito partido en tres pedazos al azar: forman triangulo con probabilidad 1/4. (verificado)

Aguja de Buffon: aguja de largo L sobre lineas separadas d,

$$\text{con } L \leq d, \text{ cruza una línea con } P = 2L / (\pi * d) \quad (\text{verificado})$$

Como atacar uno de estos en competencia

1. Identifica que se elige al azar y con que distribucion.
2. Dibuja el espacio completo de posibilidades y su medida.
3. Sombrea la region favorable y calcula SU medida.
4. Si hay varios objetos con valor, usa linealidad y suma valor por probabilidad, uno por uno.
5. Si el area no sale cerrada, piensa en barrido o en integrar.

Antes de entregar, comprueba con Monte Carlo: es cinco líneas y te dice si la formula que planteaste esta bien.

En C++

Area de la interseccion de dos círculos (la lente).

```
double areaLente(double r1, double r2, double d) {
    if (d >= r1 + r2) return 0; // separados
    if (d <= fabs(r1 - r2)) // uno dentro del otro
        return PI * min(r1, r2) * min(r1, r2);
    double a1 = 2 * acos((d*d + r1*r1 - r2*r2) / (2*d*r1));
    double a2 = 2 * acos((d*d + r2*r2 - r1*r1) / (2*d*r2));
    return 0.5*r1*r1*(a1 - sin(a1)) + 0.5*r2*r2*(a2 - sin(a2));
}
```

union de los dos = PI*r1*r1 + PI*r2*r2 - areaLente(r1, r2, d)

Los dos primeros if no son adorno: sin ellos, acos recibe un valor fuera de [-1,1] y devuelve NaN.

14.8 Rectas y Conicas

Geometria analitica de toda la vida: pendiente, las cuatro formas de escribir una recta, cuando dos son paralelas o perpendiculares, donde se cortan, y las ecuaciones de circunferencia, parabola, elipse e hipérbola. El corte de dos rectas se resuelve por Cramer, que es lo mismo que "igualar las dos ecuaciones" pero sin despejar a mano y sin dividir por cero sin darse cuenta: si el determinante da 0, son paralelas. Formulas verificadas numericamente.

C++\Formulario\Rectas y Conicas.cpp

Pendiente por dos puntos

```
m = (y2 - y1) / (x2 - x1) indefinida si x1 == x2 (vertical)
m > 0 sube, m < 0 baja, m = 0 horizontal
angulo con el eje x : theta = atan2(y2-y1, x2-x1)
```

Formas de la recta

pendiente-intercepto : $y = m x + b$ con $b = y1 - m*x1$
 punto-pendiente : $y - y1 = m (x - x1)$
 general : $A x + B y + C = 0$
 desde dos puntos: $A = y2-y1, B = -(x2-x1),$
 $C = -(A*x1 + B*y1)$
 pendiente = -A/B ; es vertical cuando B = 0
 simetrica : $x/p + y/q = 1$ (corta los ejes en p y q)

Dos rectas

paralelas : $m1 == m2$ o $A1*B2 - A2*B1 == 0$
 perpendiculares : $m1 * m2 == -1$ o $A1*A2 + B1*B2 == 0$
 angulo entre ellas : $\tan(t) = |(m2 - m1) / (1 + m1*m2)|$

Con datos enteros compara pendientes cruzando fracciones y evita dividir: $dy1*dx2 == dy2*dx1$

Interseccion: igualar las dos ecuaciones (Cramer, verificado)

$A1 x + B1 y = C1$
 $A2 x + B2 y = C2$
 $\det = A1*B2 - A2*B1$
 si $\det == 0 \rightarrow$ paralelas (o la misma recta, sin punto unico)
 $x = (C1*B2 - C2*B1) / \det$
 $y = (A1*C2 - A2*C1) / \det$
 Para cortar $y = f(x)$ con $y = g(x)$, resuelve $f(x) - g(x) = 0$.

Distancias y punto medio

$|P1P2| = \sqrt{(x2-x1)^2 + (y2-y1)^2}$
 punto medio = $((x1+x2)/2, (y1+y2)/2)$
 punto a recta $Ax+By+C=0$: $|A*px + B*py + C| / \sqrt{A^2 + B^2}$
 entre dos paralelas : $|C1 - C2| / \sqrt{A^2 + B^2}$
 El SIGNO de $A*px+B*py+C$ dice de que lado del corte esta el punto.

Circunferencia

$(x - h)^2 + (y - k)^2 = r^2$ centro (h,k), radio r
 general : $x^2 + y^2 + Dx + Ey + F = 0$
 centro $(-D/2, -E/2)$
 $r = \sqrt{D^2/4 + E^2/4 - F}$

Dos circunferencias, con d la distancia entre centros:

se cortan en 2 puntos si $|r1-r2| < d < r1+r2$
 tangentes exteriores si $d == r1+r2$
 tangentes interiores si $d == |r1-r2|$
 una dentro de la otra sin tocarse si $d < |r1-r2|$

Parabola

$y = a x^2 + b x + c$
 vertice : $x = -b/(2a)$, $y = c - b^2/(4a)$
 abre hacia arriba si $a > 0$, hacia abajo si $a < 0$
 canonica : $y - k = a (x - h)^2$ con vertice (h,k)

Elipse e hipérbola centradas en (h,k)

elipse : $(x-h)^2/a^2 + (y-k)^2/b^2 = 1$
 a y b son los semiejes; si $a = b$ es una circunferencia
 hipérbola : $(x-h)^2/a^2 - (y-k)^2/b^2 = 1$
 asintotas : $y - k = \pm(b/a)(x - h)$

En C++

```
double distPuntoRecta(double A, double B, double C,
    double px, double py) {
    return fabs(A*px + B*py + C) / hypot(A, B);
}
bool corte(double A1, double B1, double C1,
    double A2, double B2, double C2, double& x, double& y) {
    double d = A1*B2 - A2*B1;
    if (fabs(d) < 1e-12) return false; // paralelas
    x = (C1*B2 - C2*B1) / d;
    y = (A1*C2 - A2*C1) / d;
    return true;
}
```

14.8 Rectas y Conicas (cont.)

Con coordenadas enteras, para saber si tres puntos son colineales NO calcules pendientes: usa el producto cruz, que no divide ni pierde precision. $(x_2-x_1)*(y_3-y_1) - (y_2-y_1)*(x_3-x_1) == 0$

14.9 Secuencias Notables

Las sucesiones que hay que reconocer de vista en un caso de ejemplo, con los primeros valores y el punto exacto donde se desbordan: 20! es el ultimo factorial que cabe en long long, F(92) es el ultimo Fibonacci que cabe, y 2^{31} ya no cabe en int. Saber el limite de antemano evita el overflow silencioso, que es el bug mas caro en competencia.

C++\Formulario\Secuencias Notables.cpp

Fibonacci F(0)=0, F(1)=1, F(n)=F(n-1)+F(n-2)

```
n      0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
F(n)   0  1  1  2  3  5  8 13 21 34 55 89 144 233 377 610
F(20)=6765 F(30)=832040 F(40)=102334155
F(92) = 7540113804746346429 es el ULTIMO que cabe en long long;
F(93) = 12200160415121876738 ya se pasa (el techo es 9.22*10^18).
Identidades:
F(m+n) = F(m)F(n+1) + F(m-1)F(n)    fast doubling
sum_{i<=n} F(i) = F(n+2) - 1
F(n-1)F(n+1) - F(n)^2 = (-1)^n       Cassini
gcd(F(m),F(n)) = F(gcd(m,n))
```

Lucas L(0)=2, L(1)=1, misma recurrencia

```
n      0  1  2  3  4  5  6  7  8  9 10 11 12 13
L(n)   2  1  3  4  7 11 18 29 47 76 123 199 322 521
L(n) = F(n-1) + F(n+1)
```

Numeros figurados

```
n      1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
triangular 1  3  6 10 15 21 28 36 45 55 66 78 91 105 120
cuadrado    1  4  9 16 25 36 49 64 81 100 121 144 169 196 225
pentagonal  1  5 12 22 35 51 70 92 117 145 176 210 247 287 330
hexagonal   1  6 15 28 45 66 91 120 153 190 231 276 325 378 435
triangular T(n)=n(n+1)/2, T(n)+T(n-1) = n^2
```

Factoriales (20! es el ultimo que cabe en long long)

```
n      0  1  2  3  4  5  6  7  8  9
n!     1  1  2  6  24 120 720 5040 40320 362880
10! = 3628800      13! = 6227020800 (ya se pasa de int)
11! = 39916800     15! = 1307674368000
12! = 479001600    20! = 2432902008176640000
21! desborda long long. Si aparece n! con n > 20, es modular.
```

Potencias de 2

```
k      0  1  2  3  4  5  6  7  8  9 10
2^k    1  2  4  8 16 32 64 128 256 512 1024
2^20 = 1048576 (~10^6)
2^30 = 1073741824 (~10^9, limite de int)
2^31 = 2147483648 2^32 = 4294967296
2^62 = 4611686018427387904 (long long aguanta hasta 2^63-1)
```

Numeros perfectos (iguales a la suma de sus divisores propios)

6, 28, 496, 8128, 33550336
Todos los pares son $2^p(p-1)(2^p-1)$ con 2^p-1 primo (Mersenne).

En C++

```
Fibonacci por fast doubling: O(log n), aguanta n = 10^18.
pair<ll,ll> fib(ll n) { // devuelve {F(n), F(n+1)} mod M
    if (!n) return {0, 1};
    pair<ll,ll> p = fib(n >> 1);
    ll a = p.first, b = p.second;
    ll c = a * ((2*b % M - a + M) % M) % M;
    ll d = (a*a + b*b) % M;
    return (n & 1) ? make_pair(d, (c+d) % M) : make_pair(c, d);
}
Los figurados son O(1) directo:
```

14.9 Secuencias Notables (cont.)

```
ll triangular(ll n) { return n*(n+1)/2; }
ll pentagonal(ll n) { return n*(3*n-1)/2; }
ll hexagonal(ll n) { return n*(2*n-1); }
```

Para factoriales, tabla hasta 20 y listo: mas alla no cabe sin modulo.

14.10 STL Indispensable

Las funciones de la libreria estandar que la gente reimplementa a mano por no saber que existen, y las que se usan mal por un detalle: next_permutation solo recorre las n! si el vector arranca ordenado (verificado: desde {1,2,3} da 6 permutaciones, desde {3,1,2} solo 2), y substr toma (inicio, CANTIDAD) y no (inicio, fin). Tener esta lista a mano ahorra escribir veinte lineas que ya vienen hechas y probadas.

C++\Formulario\STL Indispensable.cpp

Ordenar y buscar (el vector debe estar ordenado para los bound)

```
sort(all(v))                O(n log n)
stable_sort(all(v))         conserva el orden de los iguales
sort(all(v), greater<int>()) descendente
lower_bound(all(v), x)      primer elemento >= x
upper_bound(all(v), x)      primer elemento > x
cuantos x hay = upper_bound - lower_bound
posicion = lower_bound(all(v), x) - v.begin()
binary_search(all(v), x)    devuelve solo true/false
nth_element(v.begin(), v.begin()+k, v.end())
deja el k-esimo en su sitio en O(n) promedio, sin ordenar todo
partial_sort(v.begin(), v.begin()+k, v.end()) solo los k primeros
```

Permutaciones

next_permutation(all(v)) siguiente en orden lexicografico
devuelve false cuando llega a la ultima y reinicia
ORDENA PRIMERO si quieres recorrer las n! (verificado)
prev_permutation(all(v))

Lo que no hay que reimplementar

```
__gcd(a,b)                  en C++17 tambien gcd(a,b) y lcm(a,b)
iota(all(v), 0)             llena con 0,1,2,3,...
accumulate(all(v), 0LL) suma (usa 0LL para no desbordar)
max_element / min_element devuelven ITERADOR: resta v.begin()
count(all(v), x)            find(all(v), x) reverse(all(v))
rotate(v.begin(), v.begin()+k, v.end()) rota a la izquierda
fill(all(v), x)             v.assign(n, x) swap(a, b)
minmax_element(all(v)) devuelve los dos de una
is_sorted(all(v))           next(it) prev(it)
```

Conversiones

```
to_string(x) stoi(s) stoll(s) stod(s)
s.substr(i, LARGO) <- cantidad, NO posicion final
s.find(t) devuelve string::npos si no aparece
getline(cin, s) lee la linea entera con espacios
stringstream ss(linea); while (ss >> token) ... partir por espacio
```

Estructuras y cuando usarlas

```
vector<int> v(n, 0)          vector<vector<int>> g(n)
set / multiset               ordenado, O(log n), tiene .lower_bound propio
unordered_set               O(1) promedio, sin orden
map / unordered_map         igual pero clave -> valor
deque                      push_front y push_back en O(1)
priority_queue              max-heap; greater<> para convertirlo en min-heap
bitset<N>                   operaciones de 64 bits por vez, 8x menos memoria
array<int,N>               tamaño fijo, sin costo de heap
```

Detalles que sirven

v.reserve(n) evita realocar si sabes el tamaño de antemano
emplace_back construye en el sitio, evita una copia
structured bindings: for (auto &[k, val] : mapa) (C++17)
lambda: [](int a, int b){ return a > b; }
capturar por referencia para comparadores con datos externos:
sort(all(idx), [&](int i, int j){ return v[i] < v[j]; });

14.11 Sumas y Series

Las sumas cerradas que mas se repiten, cada una con sus primeros valores para reconocer el patron directamente en el ejemplo del enunciado: si la salida crece 1, 3, 6, 10, 15 es Gauss; 1, 4, 9, 16 son cuadrados; 1, 5, 14, 30 es suma de cuadrados; 1, 9, 36, 100 es suma de cubos. Identificar la sucesion en los casos de prueba suele ser mas rapido que derivar.

C++\Formulario\Sumas y Series.cpp

Gauss: suma de los primeros n enteros

```
1 + 2 + ... + n = n(n+1)/2
n      1  2  3  4  5  6  7  8  9 10 11 12
S(n)   1  3  6 10 15 21 28 36 45 55 66 78
```

Suma de cuadrados

```
1^2 + 2^2 + ... + n^2 = n(n+1)(2n+1)/6
n      1  2  3  4  5  6  7  8  9 10 11 12
S(n)   1  5 14 30 55 91 140 204 285 385 506 650
```

Suma de cubos = el cuadrado de la suma de Gauss

```
1^3 + 2^3 + ... + n^3 = [n(n+1)/2]^2
n      1  2  3  4  5  6  7  8  9 10 11 12
S(n)   1  9 36 100 225 441 784 1296 2025 3025 4356 6084
```

Impares dan cuadrados perfectos, pares dan n(n+1)

```
1+3+5+...+(2n-1) = n^2      2+4+...+2n = n(n+1)
n      1  2  3  4  5  6  7  8  9 10 11 12
impares 1  4  9 16 25 36 49 64 81 100 121 144
pares   2  6 12 20 30 42 56 72 90 110 132 156
```

Progresion aritmetica (a, a+d, a+2d, ...)

```
termino k : a + (k-1)d
suma de n : n(2a + (n-1)d)/2 = n(primerio + ultimo)/2
```

Progresion geometrica (a, ar, ar^2, ...)

```
termino k : a*r^(k-1)
suma de n : a(r^n - 1)/(r - 1)      si r != 1
suma inf  : a/(1 - r)                si |r| < 1
En modular /(r-1) es inverso modular; si r = 1 mod p no existe.
```

Potencias de 2: la suma es la siguiente menos 1

```
1 + 2 + 4 + ... + 2^n = 2^(n+1) - 1
n      0  1  2  3  4  5  6  7  8  9 10 11 12
2^n    1  2  4  8 16 32 64 128 256 512 1024 2048 4096
suma   1  3  7 15 31 63 127 255 511 1023 2047 4095 8191
```

Serie armonica H(n) = 1 + 1/2 + ... + 1/n ~ ln(n) + 0.5772

```
H(10) ~ 2.93      H(100) ~ 5.19      H(10^6) ~ 14.39
Aparece al contar divisores: sum_{d=1..n} n/d ~ n*ln(n).
```

Conteos sobre un arreglo de n elementos

```
pares i<j      : n(n-1)/2
pares ordenados i!=j : n(n-1)
subarreglos contiguos : n(n+1)/2
subconjuntos    : 2^n          no vacios: 2^n - 1
```

En C++ (todo O(1): dividir ANTES de multiplicar evita el overflow)

```
ll gauss(ll n) { // 1+2+...+n
    return n % 2 ? (n+1)/2*n : n/2*(n+1);
}
ll sumaCuadrados(ll n) { // 1^2+...+n^2
    ll a = n, b = n+1, c = 2*n+1;
    if (a % 2 == 0) a /= 2; else b /= 2;
    if (a % 3 == 0) a /= 3;
    else if (b % 3 == 0) b /= 3; else c /= 3;
    return a * b * c;
}
ll sumaCubos(ll n) { ll g = gauss(n); return g * g; }
```

14.11 Sumas y Series (cont.)

Con $n = 10^9$, $\text{gauss}(n) = 500000000500000000$ y cabe en long long; pero $n*(n+1)$ sin dividir primero YA se desborda.

Suma geometrica modular, $O(\log n)$:

```
ll sumaGeoMod(ll a, ll r, ll n) { // a(r^n-1)/(r-1) mod M
    if (r % M == 1) return a % M * (n % M) % M;
    ll num = (modpow(r, n, M) - 1 + M) % M;
    return a % M * num % M * modpow((r-1) % M, M-2, M) % M;
}
```

14.12 Tabla ASCII

Tabla ASCII de los caracteres imprimibles (32 a 126) en cuatro columnas, mas los valores y trucos que se usan de verdad: c - '0' para el digito, c - 'a' para el indice de la letra, y que 'a' - 'A' = 32 permite cambiar de caso con un xor. Incluye la nota del '\r' de Windows, que rompe comparaciones al leer con getline.

C++\Formulario\Tabla ASCII.cpp

dec	chr	dec	chr	dec	chr	dec	chr
32	SP	56	8	80	P	104	h
33	!	57	9	81	Q	105	i
34	"	58	:	82	R	106	j
35	#	59	;	83	S	107	k
36	\$	60	<	84	T	108	l
37	%	61	=	85	U	109	m
38	&	62	>	86	V	110	n
39	'	63	?	87	W	111	o
40	(	64	@	88	X	112	p
41	)	65	A	89	Y	113	q
42	*	66	B	90	Z	114	r
43	+	67	C	91	[	115	s
44	,	68	D	92	\	116	t
45	-	69	E	93	]	117	u
46	.	70	F	94	^	118	v
47	/	71	G	95	_	119	w
48	0	72	H	96	`	120	x
49	1	73	I	97	a	121	y
50	2	74	J	98	b	122	z
51	3	75	K	99	c	123	{
52	4	76	L	100	d	124	
53	5	77	M	101	e	125	}
54	6	78	N	102	f	126	~
55	7	79	O	103	g		

Lo que de verdad se usa

```
'0' = 48      '9' = 57      digito: c - '0'
'A' = 65      'Z' = 90
'a' = 97      'z' = 122     indice: c - 'a' (0..25)
'a' - 'A' = 32 -> minuscula: c | 32, mayuscula: c & ~32
' ' = 32      '\n' = 10     '\0' = 0      '\t' = 9
Cambiar de caso con xor: c ^ 32
```

Cheques de <cctype>

```
isalpha isdigit isalnum isupper islower isspace ispunct
toupper tolower
Rapido sin libreria: (c>='0' && c<='9')
```

Control mas comunes (no imprimibles)

```
0 NUL  9 TAB 10 LF (\n) 13 CR (\r) 27 ESC
Archivos de Windows traen CR+LF: si lees con getline puede
quedar un '\r' al final de la linea. Quitarlo antes de comparar.
```

14.13 Teoria de Numeros

Divisores, phi, gcd, aritmetica modular y CRT, con las tablas hasta 20 para revisar a mano un caso chico. La mitad de los problemas de numeros se resuelven sabiendo que d(n), sigma(n) y phi(n) salen directo de la factorizacion, y que en modular dividir significa multiplicar por el inverso.

C++\Formulario\Teoria de Numeros.cpp

Con la factorizacion $n = p_1^{a_1} * p_2^{a_2} * \dots$

```
cantidad de divisores d(n) = (a1+1)(a2+1)...
suma de divisores     sigma(n) = prod (p^(a+1)-1)/(p-1)
```

14.13 Teoria de Numeros (cont.)

funcion de Euler $\phi(n) = n \cdot \prod (1 - 1/p)$
Ejemplo $n=12=2^2 \cdot 3$: $d=3 \cdot 2=6$, $\sigma=7 \cdot 4=28$, $\phi=12 \cdot (1/2) \cdot (2/3)=4$

Tablas hasta 20

n	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
d(n)	1	2	2	3	2	4	2	4	3	4	2	6	2	4	4	5	2	6	2	6
sig(n)	1	3	4	7	6	12	8	15	13	18	12	28	14	24	24	31	18	39	20	42
phi(n)	1	1	2	2	4	2	6	4	6	4	10	4	12	6	8	8	16	6	18	8

gcd y lcm

$\gcd(a,b) \cdot \text{lcm}(a,b) = a \cdot b$ (dividir antes de multiplicar)
 $\gcd(a,b) = \gcd(b, a \bmod b)$ Euclides
 $\gcd(F(m), F(n)) = F(\gcd(m,n))$ Fibonacci
 a y b coprimos $\Leftrightarrow \gcd(a,b) = 1$

Modular

Fermat : $a^{p-1} \equiv 1 \pmod p$ si p primo y p no divide a
inverso : $a^{p-2} \pmod p$ si p primo
Euler : $a^{\phi(m)} \equiv 1 \pmod m$ si $\gcd(a,m) = 1$
Wilson : $(p-1)! \equiv -1 \pmod p$ si y solo si p es primo
 $(a/b) \pmod p = a \cdot b^{-1} \pmod p$

OJO: restar en modular siempre $((a-b) \% m + m) \% m$

Teorema chino del resto (CRT)

$x \equiv r_1 \pmod{m_1}$, $x \equiv r_2 \pmod{m_2}$, con m_1, m_2 coprimos
 $\rightarrow$ solucion unica modulo $m_1 \cdot m_2$

Legendre: exponente de p en n!

$e = n/p + n/p^2 + n/p^3 + \dots$ (divisiones enteras)
Sirve para contar ceros finales de n! (exponente de 5).

Primos menores que 100 (hay 25)

2 3 5 7 11 13 17 19 23 29 31 37
41 43 47 53 59 61 67 71 73 79 83 89
97

Criba: $O(n \log \log n)$. Cantidad de primos $\sim n/\ln(n)$.

hasta 10^3 hay 168 hasta 10^6 hay 78498
hasta 10^4 hay 1229 hasta 10^7 hay 664579
hasta 10^5 hay 9592 hasta 10^9 hay $\sim 5.08 \cdot 10^7$

En C++ (factorizar en $O(\sqrt{n})$ da los tres de una)

```
ll numDivisores(ll n) {
    ll r = 1;
    for (ll p = 2; p*p <= n; p++) if (n % p == 0) {
        ll e = 0; while (n % p == 0) n /= p, e++;
        r *= e + 1;
    }
    if (n > 1) r *= 2; // quedo un primo grande
    return r;
}
ll sumaDivisores(ll n) {
    ll r = 1;
    for (ll p = 2; p*p <= n; p++) if (n % p == 0) {
        ll t = 1, q = 1;
        while (n % p == 0) n /= p, q *= p, t += q;
        r *= t;
    }
    if (n > 1) r *= 1 + n;
    return r;
}
ll phi(ll n) {
    ll r = n;
    for (ll p = 2; p*p <= n; p++) if (n % p == 0) {
        while (n % p == 0) n /= p;
        r -= r / p;
    }
}
```

14.13 Teoria de Numeros (cont.)

```
if (n > 1) r -= r / n;
return r;
}
ll modpow(ll b, ll e, ll m) { // O(log e)
    ll r = 1; b %= m;
    while (e) { if (e & 1) r = r*b % m; b = b*b % m; e >>= 1; }
    return r;
}
inverso modular con m primo: modpow(a, m-2, m)
ll legendre(ll n, ll p) { // exponente de p en n!
    ll e = 0;
    for (ll q = p; q <= n; q *= p) e += n / q;
    return e;
}
Criba lineal, O(n): cada compuesto se tacha UNA sola vez.
vector<int> criba(int n) {
    vector<int> pr; vector<char> c(n+1, 1); c[0] = c[1] = 0;
    for (int i = 2; i <= n; i++) {
        if (c[i]) pr.push_back(i);
        for (int p : pr) {
            if ((ll)i*p > n) break;
            c[i*p] = 0;
            if (i % p == 0) break;
        }
    }
    return pr;
}
```

14.14 Trampas de C++

Los errores que compilan sin una sola advertencia y dan respuesta equivocada: division entera con negativos, overflow porque `int*int` se calcula en `int` antes de guardarse en `long long`, `pow` devolviendo 9999999999999999 en vez de 10^{15} , y comparadores de sort que rompen el programa por usar `<=` en vez de `<`. Todos los valores de aqui estan verificados corriendo el codigo en este mismo equipo.

C++\Formulario\Trampas de C++.cpp

Division y modulo con negativos (verificado)

$-7 / 2 = -3$ la division trunca HACIA CERO, no hacia abajo
 $-7 \% 2 = -1$ el resto se lleva el signo del dividendo
 $7 / -2 = -3$ $7 \% -2 = 1$
 $\text{floor}(-7.0/2) = -4$ $\rightarrow$ distinto de $-7/2$
modulo siempre positivo : $((a \% m) + m) \% m$
division hacia abajo : $(a - ((a \% m) + m) \% m) / m$
Esto rompe hashing, indices circulares y aritmetica modular.

Overflow silencioso (verificado)

```
int a = 100000, b = 100000;
long long x = a * b;      -> 1410065408      MAL
long long x = (long long)a * b;      -> 10000000000      bien
La conversion ocurre DESPUES de multiplicar: castea un operando.
1 << 40      -> 0      1LL << 40 -> 1099511627776
accumulate(v.begin(), v.end(), 0)      suma en int, desborda
accumulate(v.begin(), v.end(), 0LL)      suma en long long
Regla: si el resultado pasa de  $2 \cdot 10^9$ , todo el camino en ll.
```

pow y sqrt son de punto flotante (verificado que falla)

```
(long long)pow(10,15) = 999999999999999      le falta 1
(long long)pow(5,15) = 30517578124      le falta 1
Para enteros usa exponenciacion binaria propia.
Para raiz entera, redondea y ajusta:
long long r = sqrtl(n);
while (r*r > n) r--;
while ((r+1)*(r+1) <= n) r++;
```

Comparadores de sort

Tiene que ser ESTRICTO: `return a < b;` nunca `a <= b`
Con `<=` el sort puede salirse del arreglo y tumbar el programa.
Descendente: `return a > b;`
Por varios campos:
`if (a.x != b.x) return a.x < b.x;`

14.14 Trampas de C++ (cont.)
return a.y < b.y;

Contenedores

m[k] CREA la clave con valor por defecto si no existe.
Para solo consultar: *m.count(k)* o *m.find(k)*.
unique() exige el vector ORDENADO y no borra por si solo:
sort(all(v)); v.erase(unique(all(v)), v.end());
En set y map usa el miembro *s.lower_bound(x)*: es $O(\log n)$.
std::lower_bound(s.begin(), s.end(), x) recorre en $O(n)$.
priority_queue es MAXIMO por defecto. Para minimo:
priority_queue<int, vector<int>, greater<int>> pq;
Borrar de un vector mientras lo recorres invalida iteradores.

Entrada y salida

cin >> x salta espacios y DEJA el '\n' pendiente. Si despues
usas *getline*, lees una linea vacia: consume el salto primero.
endl hace flush cada vez; dentro de un bucle usa '\n'.
Al inicio del main:
ios::sync_with_stdio(false); cin.tie(nullptr);
printf de long long es %lld, no %d.
Si mezclas *printf* y *cout* con *sync* desactivado, el orden se rompe.

Punto flotante

Nunca compares con ==. Usa *fabs(a-b) < 1e-9*.
double guarda ~15-16 digitos; por encima de 2^{53} pierde enteros.
Imprimir con decimales: *cout << fixed << setprecision(k)*
Si el problema es de enteros, resuelvelo con enteros.

14.15 Trigonometria

Conversion grados-radianes, la tabla de valores exactos, las identidades que se usan para simplificar antes de programar, y las leyes de senos y cosenos para resolver un triangulo del que solo se conocen algunos lados o angulos. El detalle practico mas importante: en C++ sin, cos y tan reciben RADIANES, y para sacar el angulo de un vector hay que usar *atan2(y,x)*, no *atan(y/x)*, que pierde el cuadrante. Valores e identidades verificados numericamente.

C++\Formulario\Trigonometria.cpp

Conversion

radianes = grados * pi / 180
grados = radianes * 180 / pi
PI = acos(-1.0) (mas exacto que escribir 3.14159...)
En C++ sin/cos/tan reciben RADIANES, no grados.

Valores exactos (verificados)

	0	30	45	60	90
grados	0	30	45	60	90
radianes	0	pi/6	pi/4	pi/3	pi/2
sin	0	1/2	r2/2	r3/2	1
cos	1	r3/2	r2/2	1/2	0
tan	0	r3/3	1	r3	inf
r2 = sqrt(2)	~ 1.4142136				
r3 = sqrt(3)	~ 1.7320508				

Triangulo rectangulo

sin = cateto opuesto / hipotenusa
cos = cateto adyacente / hipotenusa
tan = opuesto / adyacente = sin / cos
Pitagoras : $a^2 + b^2 = c^2$
Ternas enteras comunes: 3-4-5, 5-12-13, 8-15-17, 7-24-25

Identidades (verificadas)

$\sin^2 t + \cos^2 t = 1$
 $\sin(2t) = 2 \sin t \cos t$
 $\cos(2t) = \cos^2 t - \sin^2 t = 1 - 2 \sin^2 t = 2 \cos^2 t - 1$
 $\sin(a+b) = \sin a \cos b + \cos a \sin b$
 $\cos(a+b) = \cos a \cos b - \sin a \sin b$
 $\tan(a+b) = (\tan a + \tan b) / (1 - \tan a \tan b)$
 $\sin(-t) = -\sin t$ $\cos(-t) = \cos t$
 $\sin(\pi - t) = \sin t$ $\cos(\pi - t) = -\cos t$
periodo de sin y cos : 2π periodo de tan : π

Triangulo cualquiera (verificados con el triangulo 3-4-5)

ley de senos : $a/\sin A = b/\sin B = c/\sin C = 2R$
ley de cosenos : $c^2 = a^2 + b^2 - 2ab \cos C$
despejando : $\cos C = (a^2 + b^2 - c^2) / (2ab)$
si $\cos C > 0$ el angulo es agudo, si < 0 es obtuso
 $\text{area} = (1/2) * a * b * \sin C$
radio circunscrito $R = a*b*c / (4 * \text{area})$
radio inscrito $r = \text{area} / s$ con s el semiperimetro
Los angulos suman 180 grados: sabiendo dos, el tercero sale solo.

atan2 contra atan

atan2(y, x) devuelve el angulo correcto en los cuatro cuadrantes,
en el rango $(-\pi, \pi]$. Ejemplo: *atan2(-1,-1)* = -135 grados.
atan(y/x) pierde el cuadrante y se rompe cuando $x = 0$.
Angulo CON SIGNO entre dos vectores:
atan2(cross(a,b), dot(a,b))
Para ordenar puntos alrededor de un centro se puede usar *atan2*,
pero con enteros es mas exacto comparar por cuadrante y luego
por producto cruz, sin tocar flotantes.

14.16 Trucos de Bits

Identidades de bits con la situacion concreta en la que sirve cada una, que es la parte que no se deduce: $n \& (n-1)$ es "potencia de 2", $n \& -n$ es el avance del Fenwick, y el xor se usa porque $a^a=0$ deja solo lo que aparece un numero impar de veces. Los valores de los builtins estan verificados corriendo el codigo, no de memoria. La trampa mas cara de esta seccion: $1 << 40$ da 0 porque el literal 1 es int; hay que escribir $1LL << 40$.

C++\Formulario\Trucos de Bits.cpp

Operar el bit k (k empieza en 0)

leer : $(n >> k) \& 1$
prender : $n \mid= 1LL << k$
apagar : $n \&= \sim(1LL << k)$
voltear : $n \wedge= 1LL << k$
mascara de k bits en 1 : $(1LL << k) - 1$
OJO: $1 << 40$ da 0 porque el 1 es int. Usa $1LL << 40$.

Identidades y para que sirve cada una

$n \& (n-1)$ apaga el bit encendido mas bajo
-> $n \& (n-1) == 0$ dice si n es potencia de 2
-> contar bits: while (n) { n &= n-1; c++; }
 $n \& -n$ deja SOLO el bit mas bajo (lowbit)
-> es el avance del Fenwick tree: $i += i \& -i$
 $n \mid (n+1)$ prende el cero mas bajo
 $n \& (n+1)$ apaga la cola de unos del final
 $n \wedge (n>>1)$ codigo Gray: dos consecutivos difieren en 1 bit
-> recorrer subconjuntos cambiando un elemento
 $a \wedge a = 0$ y $a \wedge 0 = a$
-> el que aparece un numero impar de veces sale solo
-> xor de rango $[l,r] = \text{pre}[r] \wedge \text{pre}[l-1]$
 $a \wedge b >= 0$ a y b tienen el mismo signo

Codigo Gray, primeros valores

i	0	1	2	3	4	5	6	7
$i^{(i>>1)}$	0	1	3	2	6	7	5	4
binario	000	001	011	010	110	111	101	100

Builtins de GCC (verificados con x = 40 = 101000b)

__builtin_popcount(40) = 2 bits en 1
__builtin_ctz(40) = 3 ceros al final = indice del lowbit
__builtin_clz(40) = 26 ceros al inicio (sobre 32 bits)
__builtin_parity(7) = 1 1 si la cantidad de bits es impar
Para long long agrega ll: *__builtin_popcountll*, *__builtin_ctzll*
PELIGRO: *ctz* y *clz* con $n = 0$ son comportamiento indefinido.
bit mas alto de $n = 31 - \text{clz}(n) = \text{floor}(\log_2(n))$
siguiente potencia de 2 $>= n$: $1 << (32 - \text{clz}(n-1))$

Subconjuntos

recorrer TODAS las submascaras de m (sin el vacio):
for (int s = m; s; s = (s-1) & m)

14.16 Trucos de Bits (cont.)

hacerlo para todas las mascaras cuesta 3^n , no 4^n
 complemento dentro de n bits : $m \wedge ((1 \ll n) - 1)$
 recorrer los bits: for (b=0;b<n;b++) if (m >> b & 1) ...
 agregar elemento b: $m | (1 \ll b)$ quitarlo: $m \& \sim(1 \ll b)$

Desplazamientos como aritmetica

```
n << 1 = n*2      n << k = n * 2^k
n >> 1 = n/2 solo si n >= 0
030: >> redondea hacia abajo y la division hacia cero:
-7 >> 1 = -4 pero -7 / 2 = -3
```

Cuando conviene pensar en bitmask

$n \leq 20 \rightarrow$ DP sobre 2^n estados es viable ($\sim 10^6$)
 $n \leq 25 \rightarrow 2^n$ cabe, pero ya va justo
 Estado tipico: dp[mascara] = mejor valor usando ese subconjunto.
 Si el enunciado dice $n \leq 20$, casi siempre es bitmask DP.
 bitset<N> para cribas o DP booleana: 64 veces menos memoria y
 opera and/or/shift de a 64 bits por vez.

15 - Divide and Conquer

15.1 Conteo de Inversiones con Merge Sort - Frosh Merge Sort

La otra forma de resolver "Frosh Week" (problema F, CCPL), la misma del archivo "frosh" pero sin ninguna estructura de datos. La equivalencia del problema es la misma: el minimo de intercambios de vecinos para ordenar la fila es exactamente el numero de INVERSIONES. LA IDEA, Y ES DE LAS MAS BONITAS QUE HAY: un merge sort ya compara todas las parejas que importan, solo que normalmente tira esa informacion. Al mezclar dos mitades YA ORDENADAS, en el momento en que se toma un elemento de la derecha (R[j]) porque es menor que L[i], ese elemento es menor que L[i] Y QUE TODOS LOS QUE LE SIGUEN EN L, porque L esta ordenada. Como todos ellos estan a la izquierda en el arreglo original, ahi hay $n1 - i$ inversiones de golpe. Esa sola linea, respuesta += $n1 - i$, convierte un ordenamiento en un contador. POR QUE SE PUEDEN SUMAR LAS PARTES: toda inversion (i, j) cae en exactamente uno de tres casos, las dos dentro de la mitad izquierda, las dos dentro de la derecha, o una en cada mitad. Las dos primeras las cuentan las llamadas recursivas y la tercera la cuenta el merge, sin solaparse ni dejar ninguna afuera. De ahi sale el $O(n \log n)$. EL DETALLE QUE PARECE TRIVIAL Y NO LO ES: la comparacion tiene que ser $L[i] \leq R[j]$, con el igual del lado izquierdo. Si se pone $<$, los elementos iguales se toman de la derecha y se cuentan como inversiones, lo que da de mas. En este problema los numeros de estudiante son todos distintos y no se nota, pero en cualquier otro con repetidos si. long long en el contador: con $n = 10^6$ al revés salen casi $5 \cdot 10^{11}$ inversiones. DOS COSAS QUE LE CUESTAN TIEMPO A ESTA VERSION Y NO SE VEN (medido, $n = 10^6$ aleatorio): tal como esta 624 ms agregandole ios::sync_with_stdio(false) 452 ms ademas con UN solo buffer en vez de crear L y R 360 ms Lo primero es que leer 10^6 enteros con cin sin desactivar la sincronizacion con stdio cuesta mas que todo el algoritmo. Lo segundo es que crear dos vectores DENTRO de merge significa ~ 2 millones de asignaciones de memoria; con un unico buffer del tamaño del arreglo reservado una sola vez, se hace una. Asi arreglada es la version mas rapida y la que menos memoria usa (~ 8 MB contra los ~ 16 MB del Fenwick, que guarda el arreglo, su copia ordenada y el bit de long long). La de Fenwick se queda en el cuaderno porque se generaliza a otras preguntas; esta solo cuenta inversiones. Verificado contra la version de Fenwick con $n = 10^6$ en tres escenarios (aleatorio, invertido y ya ordenado): identicas, 250188280112 en el aleatorio.

C++/CCPL/froshMergeSort.cpp

```
1 #include <bits/stdc++.h>
2 #include <vector>
3
4 using namespace std;
5
6 long long respuesta = 0;
7
8 void merge(vector<int>& arr, int left, int mid, int right) {
9     int n1 = mid - left + 1;
10    int n2 = right - mid;
11
12
13    vector<int> L(n1);
14    vector<int> R(n2);
15
16    for (int i = 0; i < n1; i++)
```

15.1 Conteo de Inversiones con Merge Sort - Frosh Merge Sort (cont.)

```
17    L[i] = arr[left + i];
18    for (int j = 0; j < n2; j++)
19        R[j] = arr[mid + 1 + j];
20
21    int i = 0, j = 0, k = left;
22
23    while (i < n1 && j < n2) {
24        if (L[i] <= R[j]) {
25            arr[k] = L[i];
26            i++;
27        } else {
28            arr[k] = R[j];
29            respuesta += n1 - i;
30            j++;
31        }
32        k++;
33    }
34
35    while (i < n1) {
36        arr[k] = L[i];
37        i++;
38        k++;
39    }
40
41    while (j < n2) {
42        arr[k] = R[j];
43        j++;
44        k++;
45    }
46 }
47
48 void mergeSort(vector<int>& arr, int left, int right) {
49     if (left >= right)
50         return;
51
52     int mid = left + (right - left) / 2;
53
54     mergeSort(arr, left, mid);
55     mergeSort(arr, mid + 1, right);
56
57     merge(arr, left, mid, right);
58 }
59
60 int main() {
61
62     int n = 0;
63
64     while (cin >> n){
65
66         respuesta = 0;
67         vector<int> arr;
68
69         for(int i = 0; i < n; i++){
70             int num = 0;
71             cin >> num;
72             arr.push_back(num);
73         }
74
75         mergeSort(arr, 0, arr.size() - 1);
76         cout << respuesta << endl;
77     }
78
79     return 0;
80 }
```

16 - CSes: Graph Algorithms

16.1 Building Roads - DSU (Union-Find)

DSU con compresion de camino y union por tamaño, que juntas dan casi $O(1)$ amortizado por operacion. Se unen todas las aristas dadas y despues se recorre $1..n$ preguntando quien es su propio padre: esos son los representantes, uno por componente. Con c componentes hacen falta $c-1$ aristas nuevas, y basta encadenar los representantes en fila. CUANDO USAR DSU: cuando la pregunta es "estos dos estan conectados" o "cuantas componentes hay" y las aristas solo se AGREGAN, nunca se quitan. Si hay que quitar aristas, DSU no sirve tal cual y toca procesar al revés en el tiempo, o irse a link-cut trees. Para solo contar componentes un BFS o DFS tambien alcanza; DSU gana cuando las consultas se intercalan con las uniones, o cuando es la base de Kruskal.

C++\CSES\Graph Algorithms\Building Roads.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<long long> padre;
5 vector<long long> tamaño;
6
7 long long encontrar(long long x)
8 {
9     if(padre[x] == x)
10     {
11         return x;
12     }
13     return padre[x] = encontrar(padre[x]);
14 }
15
16 void unir(long long a, long long b)
17 {
18     a = encontrar(a);
19     b = encontrar(b);
20
21     if(a == b)
22     {
23         return;
24     }
25
26     if(tamaño[a] < tamaño[b])
27     {
28         swap(a, b);
29     }
30
31     padre[b] = a;
32     tamaño[a] += tamaño[b];
33 }
34
35 int main()
36 {
37     ios::sync_with_stdio(false);
38     cin.tie(nullptr);
39
40     long long n, m;
41     cin >> n >> m;
42
43     padre.resize(n + 1);
44     tamaño.resize(n + 1, 1);
45
46     for(long long i = 1; i <= n; i++)
47     {
48         padre[i] = i;
49     }
50
51     long long a, b;
52
53     for(long long i = 0; i < m; i++)
54     {
55         cin >> a >> b;
56         unir(a, b);
57     }
58
59     vector<long long> familias;
60
61
```

16.1 Building Roads - DSU (Union-Find) (cont.)

```
62     for(long long i = 1; i <= n; i++)
63     {
64         if(encontrar(i) == i)
65         {
66             familias.push_back(i);
67         }
68     }
69
70     cout << (familias.size()-1) << '\n';
71
72     for(long long i = 0; i + 1 < familias.size(); i++)
73     {
74         cout << familias[i] << ' ' << familias[i + 1] << '\n';
75     }
76
77     return 0;
78 }
```

16.2 Building Teams - BFS de Bicoloreo (Bipartito)

Bicoloreo por BFS: se pinta el primer nodo de 1 y cada vecino del color opuesto. Si en algun momento aparece una arista entre dos nodos del MISMO color, el grafo tiene un ciclo impar y no es bipartito. El for de afuera reinicia el BFS en cada componente, que es lo que se olvida. CUANDO USAR: cada vez que un problema pida partir en DOS grupos con "estos dos no pueden ir juntos", o preguntar si se puede 2-colorear. Bipartito equivale a no tener ciclos de longitud impar. Y es el prerrequisito para matching bipartito: si el problema habla de emparejar dos conjuntos, lo primero es verificar que el grafo sea bipartito.

C++\CSES\Graph Algorithms\Building Teams.cpp

```
1 #include <iostream>
2 #include <vector>
3 #include <queue>
4
5 using namespace std;
6
7 int main()
8 {
9     long long n, m;
10
11     cin >> n >> m;
12
13     vector<vector<long long>> vecinos(n + 1);
14
15     long long a, b;
16
17     while(m--)
18     {
19         cin >> a >> b;
20
21         vecinos[a].push_back(b);
22         vecinos[b].push_back(a);
23     }
24
25     vector<long long> valores(n + 1, -1);
26
27     for(long long i = 1; i <= n; i++)
28     {
29         if(valores[i] == -1)
30         {
31             queue<long long> cola;
32
33             valores[i] = 1;
34             cola.push(i);
35
36             while(!cola.empty())
37             {
38                 long long actual = cola.front();
39                 cola.pop();
40
41                 for(long long conexion : vecinos[actual])
42                 {
43                     if(valores[conexion] == -1)
44

```

16.2 Building Teams - BFS de Bicoloreo (Bipartito) (cont.)

```

44         {
45             if(valores[actual] == 1)
46             {
47                 valores[conexion] = 2;
48             }
49             else
50             {
51                 valores[conexion] = 1;
52             }
53             cola.push(conexion);
54         }
55         else if(valores[conexion] == valores[actual])
56         {
57             cout << "IMPOSSIBLE\n";
58             return 0;
59         }
60     }
61 }
62 }
63 }
64 }
65
66 for(long long i = 1; i <= n; i++)
67 {
68     cout << valores[i];
69
70     if(i < n)
71     {
72         cout << ' ';
73     }
74 }
75
76 cout << '\n';
77
78 return 0;
79 }

```

16.3 Counting Rooms - Flood Fill en Grilla (BFS)

Contar componentes conexas en una grilla. Se barre celda por celda y, al encontrar una libre sin visitar, se lanza un BFS que marca todo su cuarto y se suma uno al total. Marcar la celda como visitada AL ENCOLARLA (no al sacarla) es lo que evita que entre dos veces a la cola. Aquí se reusa la matriz de entrada como arreglo de visitados poniendo 'X': ahorra memoria y una estructura aparte. CUANDO USAR: cualquier problema de grilla que pregunte cuantas regiones, islas, lagos o manchas hay, o el tamaño de la región que contiene a una celda. BFS y DFS dan lo mismo aquí; se prefiere BFS iterativo porque con 1000x1000 celdas un DFS recursivo puede reventar la pila.

C++\CSES\Graph Algorithms\Counting Rooms.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      ios::sync_with_stdio(false);
7      cin.tie(nullptr);
8
9      long long n, m;
10     cin >> n >> m;
11
12     vector<string> matriz(n);
13
14     for(long long i = 0; i < n; i++)
15     {
16         cin >> matriz[i];
17     }
18
19     queue<pair<long long, long long>> cuartos;
20
21     long long numCuartos = 0;
22
23     for(long long i = 0; i < n; i++)

```

16.3 Counting Rooms - Flood Fill en Grilla (BFS) (cont.)

```

24     {
25         for(long long j = 0; j < m; j++)
26         {
27             if(matriz[i][j] == '.')
28             {
29                 matriz[i][j] = 'X';
30                 cuartos.push({i, j});
31
32                 while(!cuartos.empty())
33                 {
34                     long long x = cuartos.front().first;
35                     long long y = cuartos.front().second;
36                     cuartos.pop();
37
38                     if(x - 1 >= 0 && matriz[x - 1][y] == '.')
39                     {
40                         matriz[x - 1][y] = 'X';
41                         cuartos.push({x - 1, y});
42                     }
43
44                     if(x + 1 < n && matriz[x + 1][y] == '.')
45                     {
46                         matriz[x + 1][y] = 'X';
47                         cuartos.push({x + 1, y});
48                     }
49
50                     if(y - 1 >= 0 && matriz[x][y - 1] == '.')
51                     {
52                         matriz[x][y - 1] = 'X';
53                         cuartos.push({x, y - 1});
54                     }
55
56                     if(y + 1 < m && matriz[x][y + 1] == '.')
57                     {
58                         matriz[x][y + 1] = 'X';
59                         cuartos.push({x, y + 1});
60                     }
61                 }
62
63                 numCuartos++;
64             }
65         }
66     }
67
68     cout << numCuartos;
69
70     return 0;
71 }

```

16.4 Course Schedule - Orden Topologico (Kahn)

Orden topológico con el algoritmo de Kahn: se meten a la cola los nodos con grado de entrada 0 (nadie los obliga a esperar), y cada vez que se saca uno se le "quita" la arista a sus vecinos; el que se queda sin prerequisites entra a la cola. El orden en que salen de la cola es un orden topológico válido. LO QUE HACE A KAHN MEJOR QUE EL DFS PARA ESTO: detecta el ciclo gratis. Si hay un ciclo, sus nodos nunca llegan a grado 0 y jamás entran a la cola, así que al final orden tiene MENOS de n nodos. Esa sola comparación (`orden.size() != n`) es todo el chequeo de IMPOSSIBLE. CUANDO USAR: prerequisites, dependencias, "en que orden se hacen las tareas", y como primer paso de cualquier DP sobre un DAG (ver "Game Routes" y "Longest Flight Route"). Si piden el orden lexicográficamente menor, se cambia la queue por una `priority_queue` de mínimos y ya.

C++\CSES\Graph Algorithms\Course Schedule.cpp

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4
5  using namespace std;
6
7  int main()
8  {
9      ios::sync_with_stdio(false);
10     cin.tie(nullptr);

```

16.4 Course Schedule - Orden Topologico (Kahn) (cont.)

```

11
12     long long n, m;
13     cin >> n >> m;
14
15     vector<vector<long long>> conexiones(n);
16     vector<long long> grado(n, 0);
17
18     for(long long i = 0; i < m; i++)
19     {
20         long long a, b;
21         cin >> a >> b;
22
23         a--;
24         b--;
25
26         conexiones[a].push_back(b);
27         grado[b]++;
28     }
29
30     queue<long long> cola;
31
32     for(long long i = 0; i < n; i++)
33     {
34         if(grado[i] == 0)
35         {
36             cola.push(i);
37         }
38     }
39
40     vector<long long> orden;
41
42     while(!cola.empty())
43     {
44         long long actual = cola.front();
45         cola.pop();
46
47         orden.push_back(actual);
48
49         for(long long siguiente : conexiones[actual])
50         {
51             grado[siguiente]--;
52
53             if(grado[siguiente] == 0)
54             {
55                 cola.push(siguiente);
56             }
57         }
58     }
59
60     if(orden.size() != n)
61     {
62         cout << "IMPOSSIBLE\n";
63         return 0;
64     }
65
66     for(long long nodo : orden)
67     {
68         cout << nodo + 1 << ' ';
69     }
70
71     cout << '\n';
72
73     return 0;
74 }

```

16.5 Cycle Finding - Bellman-Ford (Ciclo Negativo)

Bellman-Ford usado no para hallar distancias sino para DETECTAR un ciclo negativo y devolverlo. Se relaja n veces; si en la ultima pasada todavia hubo una mejora, esa mejora solo pudo venir de un ciclo negativo, y el nodo donde ocurrio queda guardado. EL TRUCO DE LA RECONSTRUCCION, que es lo que hay que copiar: el nodo donde se detecto la mejora NO tiene por que estar dentro del ciclo, puede ser uno colgado mas adelante. La solucion es subir n veces por el arreglo de padres: despues de n saltos es seguro que ya se entro al ciclo, porque la cadena de padres tiene a lo sumo n nodos antes de enrollarse. Desde ahi se sigue a los padres hasta volver al mismo nodo y ese es el ciclo. Aqui las distancias arrancan TODAS en 0, no en infinito, y es a proposito: equivale a poner un origen virtual conectado a todos con peso 0, asi se encuentran ciclos negativos en cualquier componente y no solo en la que alcanza el nodo 1. CUANDO USAR: detectar ciclos negativos, que es lo unico que Dijkstra no puede hacer. Aparece en arbitraje de monedas (con logaritmos negados) y en sistemas de restricciones de diferencias. OJO: usa structured bindings (auto [a, b]), que piden C++17. En CS50 compila, pero con un g++ viejo hay que volver a .first y .second.

C++\CS50\Graph Algorithms\Cycle Finding.cpp

```

1 #include <iostream>
2 #include <vector>
3
4 using namespace std;
5
6 int main()
7 {
8     ios::sync_with_stdio(false);
9     cin.tie(nullptr);
10
11     long long n, m;
12     cin >> n >> m;
13
14     vector<vector<pair<long long, long long>>> conexion(n);
15
16     for(long long i = 0; i < m; i++)
17     {
18         long long a, b, c;
19         cin >> a >> b >> c;
20
21         conexion[--a].push_back({--b, c});
22     }
23
24     vector<long long> distancia(n, 0);
25     vector<long long> padre(n, -1);
26
27     long long ciclo = -1;
28
29     for(long long i = 0; i < n; i++)
30     {
31         ciclo = -1;
32
33         for(long long a = 0; a < n; a++)
34         {
35             for(auto [b, c] : conexion[a])
36             {
37                 if(distancia[a] + c < distancia[b])
38                 {
39                     distancia[b] = distancia[a] + c;
40                     padre[b] = a;
41                     ciclo = b;
42                 }
43             }
44         }
45
46         if(ciclo == -1)
47             break;
48     }
49
50     if(ciclo == -1)
51     {
52         cout << "NO\n";
53         return 0;
54     }
55
56

```

16.5 Cycle Finding - Bellman-Ford (Ciclo Negativo) (cont.)

```

56     for(long long i = 0; i < n; i++)
57         ciclo = padre[ciclo];
58
59     vector<long long> respuesta;
60
61     long long actual = ciclo;
62
63     do
64     {
65         respuesta.push_back(actual);
66         actual = padre[actual];
67     }
68     while(actual != ciclo);
69
70     respuesta.push_back(ciclo);
71
72     cout << "YES\n";
73
74     for(long long i = respuesta.size() - 1; i >= 0; i--)
75         cout << respuesta[i] + 1 << ' ';
76
77     cout << '\n';
78
79     return 0;
80 }

```

16.6 Flight Discount - Dijkstra sobre Grafo de Capas

Dijkstra donde el estado no es solo la ciudad sino el par (ciudad, ya use el descuento). Es la tecnica de GRAFO DE CAPAS o grafo producto: se duplica el grafo, en la capa 0 nada esta usado y en la capa 1 el descuento ya se gasto, y las aristas que aplican el descuento son las unicas que cruzan de una capa a la otra. Un Dijkstra normal sobre ese grafo mas grande resuelve todo. POR QUE NO SE PUEDE MAS BARATO: la respuesta no es "el camino minimo y despues descuento la arista mas cara", porque el mejor camino con descuento puede ser otro camino completamente distinto. Hay que decidir los dos a la vez, y para eso se mete la decision dentro del estado. CUANDO USAR: cada vez que al camino minimo se le agrega un recurso limitado. "Puedes usar k teletransportes", "puedes hacer una arista gratis", "hay que llegar con tanque suficiente". La regla es multiplicar los nodos por los estados del recurso: $n \cdot (k+1)$ nodos, mismo Dijkstra. Si el recurso tiene muchos valores el estado explota; ahi ya toca pensar en otra cosa.

C++(CSES)Graph Algorithms\Flight Discount.cpp

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  #include <array>
5  #include <climits>
6
7  using namespace std;
8
9  int main()
10 {
11     ios::sync_with_stdio(false);
12     cin.tie(nullptr);
13
14     long long n, m;
15
16     cin >> n >> m;
17
18     vector<vector<pair<long long, long long>>> vuelos(n);
19
20     long long a,b,c;
21
22     while(m--)
23     {
24         cin >> a >> b >> c;
25
26         vuelos[--a].push_back({--b,c});
27     }
28
29     //Estados
30     long long USADO = 1, NO_USADO = 0;

```

16.6 Flight Discount - Dijkstra sobre Grafo de Capas (cont.)

```

31
32     vector<vector<long long>> costoMinimo(n, vector<long long>(2, LLONG_MAX));
33
34     priority_queue<array<long long, 3>,
35                 vector<array<long long, 3>>,
36                 greater<array<long long, 3>>> cola;
37
38     costoMinimo[0][NO_USADO] = 0;
39
40     cola.push({0,0,NO_USADO});
41
42     while(!cola.empty())
43     {
44         long long costo, ciudad, estado;
45
46         costo = cola.top()[0];
47         ciudad = cola.top()[1];
48         estado = cola.top()[2];
49
50         cola.pop();
51
52         if(costoMinimo[ciudad][estado] < costo) continue;
53
54         for(pair<long long, long long> vuelo : vuelos[ciudad])
55         {
56             long long adonde = vuelo.first;
57             long long cuanto = vuelo.second;
58
59             if(costoMinimo[adonde][estado] > costo + cuanto)
60             {
61                 costoMinimo[adonde][estado] = costo + cuanto;
62                 cola.push({costo+cuanto,adonde,estado});
63             }
64
65             if(estado == NO_USADO)
66             {
67                 if(costoMinimo[adonde][USADO] > costo + cuanto/2)
68                 {
69                     costoMinimo[adonde][USADO] = costo + cuanto/2;
70
71                     cola.push({costo+cuanto/2,adonde,USADO});
72                 }
73             }
74         }
75     }
76
77     cout << min(costoMinimo[n-1][NO_USADO], costoMinimo[n-1][USADO]) << endl;
78
79     return 0;
80 }

```

16.7 Flight Routes - K Caminos Mas Cortos

Los k caminos mas cortos al nodo n, con la variante de Dijkstra que en vez de UNA distancia por nodo guarda las k mejores en un priority_queue de maximos por nodo. Cada nodo se puede sacar de la cola hasta k veces, y ahi esta la clave: la k-esima vez que un nodo sale de la cola, sale con su k-esima distancia mas corta. Por eso el contador usados[] y el corte cuando pasa de k. El priority_queue de MAXIMOS por nodo es para poder botar el peor de los k que se llevan cuando llega uno mejor. Es un heap acotado a tamano k. CUANDO USAR: "los k caminos mas cortos", "el k-esimo mas corto", con repeticiones permitidas. Costo $O(k \cdot m \cdot \log)$. Ojo que aqui los caminos pueden repetir nodos; si el problema pide caminos DISJUNTOS o simples, esto no sirve y el problema es muchisimo mas duro.

C++(CSES)Graph Algorithms\Flight Routes.cpp

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  #include <functional>
5
6  using namespace std;
7

```

16.7 Flight Routes - K Caminos Mas Cortos (cont.)

```

8  int main()
9  {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     long long n, m, k;
14     cin >> n >> m >> k;
15
16     vector<vector<pair<long long, long long>>> conexion(n);
17
18     for(long long i = 0; i < m; i++)
19     {
20         long long a, b, c;
21         cin >> a >> b >> c;
22
23         conexion[--a].push_back({--b, c});
24     }
25
26     vector<priority_queue<long long>> distancia(n);
27     vector<long long> usados(n, 0);
28
29     priority_queue<
30         pair<long long, long long>,
31         vector<pair<long long, long long>>,
32         greater<pair<long long, long long>>
33     > cola;
34
35     distancia[0].push(0);
36     cola.push({0, 0});
37
38     while(!cola.empty())
39     {
40         long long costo = cola.top().first;
41         long long actual = cola.top().second;
42         cola.pop();
43
44         usados[actual]++;
45
46         if(usados[actual] > k)
47             continue;
48
49         for(auto vecino : conexion[actual])
50         {
51             long long siguiente = vecino.first;
52             long long peso = vecino.second;
53
54             long long nuevoCosto = costo + peso;
55
56             if(distancia[siguiente].size() < k)
57             {
58                 distancia[siguiente].push(nuevoCosto);
59                 cola.push({nuevoCosto, siguiente});
60             }
61             else if(nuevoCosto < distancia[siguiente].top())
62             {
63                 distancia[siguiente].pop();
64                 distancia[siguiente].push(nuevoCosto);
65                 cola.push({nuevoCosto, siguiente});
66             }
67         }
68     }
69
70     vector<long long> respuesta;
71
72     while(!distancia[n - 1].empty())
73     {
74         respuesta.push_back(distancia[n - 1].top());
75         distancia[n - 1].pop();
76     }
77
78     for(long long i = respuesta.size() - 1; i >= 0; i--)
79         cout << respuesta[i] << ' ';

```

16.7 Flight Routes - K Caminos Mas Cortos (cont.)

```

80
81     cout << '\n';
82
83     return 0;
84 }

```

16.8 Game Routes - DP sobre DAG en Orden Topologico

Contar caminos de 1 a n en un DAG: caminos[v] = suma de caminos[u] sobre las aristas $u \rightarrow v$. La recurrencia es obvia; lo que la hace correcta es el ORDEN: hay que procesar un nodo solo cuando ya se sumaron TODOS sus predecesores, y eso es exactamente lo que garantiza recorrer en orden topologico con Kahn. Cuando grado[v] llega a 0, caminos[v] ya esta completo. El detalle de arranque: solo caminos[0] = 1. Los otros nodos que tambien empiezan con grado 0 entran a la cola igual, pero con 0 caminos, así que empujan ceros y no contaminan la cuenta: solo cuentan los caminos que salen del nodo 1. CUANDO USAR: cualquier "cuantas formas", "camino mas largo", "camino mas barato" sobre un grafo SIN ciclos. En un DAG todas esas son una DP en orden topologico, $O(n + m)$. Con ciclos contar caminos puede ser infinito, y el camino mas largo pasa a ser NP-difícil.

C++\CSES\Graph Algorithms\Game Routes.cpp

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4
5  using namespace std;
6
7  int main()
8  {
9     ios::sync_with_stdio(false);
10     cin.tie(nullptr);
11
12     const long long MOD = 1000000007;
13
14     long long n, m;
15     cin >> n >> m;
16
17     vector<vector<long long>> conexiones(n);
18     vector<long long> grado(n, 0);
19
20     for(long long i = 0; i < m; i++)
21     {
22         long long a, b;
23         cin >> a >> b;
24
25         a--;
26         b--;
27
28         conexiones[a].push_back(b);
29         grado[b]++;
30     }
31
32     vector<long long> caminos(n, 0);
33     queue<long long> cola;
34
35     caminos[0] = 1;
36
37     for(long long i = 0; i < n; i++)
38     {
39         if(grado[i] == 0)
40         {
41             cola.push(i);
42         }
43     }
44
45     while(!cola.empty())
46     {
47         long long actual = cola.front();
48         cola.pop();
49
50         for(long long siguiente : conexiones[actual])
51         {
52             caminos[siguiente] += caminos[actual];
53             caminos[siguiente] %= MOD;

```

16.8 Game Routes - DP sobre DAG en Orden Topologico (cont.)

```

54         grado[siguiente]--;
55     }
56     if(grado[siguiente] == 0)
57     {
58         cola.push(siguiente);
59     }
60 }
61 }
62 }
63 }
64 cout << caminos[n - 1] << '\n';
65
66 return 0;
67 }
```

16.9 High Score - Bellman-Ford (Camino Maximo)

Camino de peso MAXIMO con aristas negativas, que es Bellman-Ford con la desigualdad volteada. Se relaja n veces; si en la pasada n todav a mejora algo, hay un ciclo de ganancia infinita. El detalle que hace correcto el problema: un ciclo positivo solo importa si se puede LLEGAR a el desde el origen y desde el SALIR hacia el destino. Por eso los dos BFS previos, uno en el grafo normal desde 1 y otro en el grafo INVERSO desde n : solo los nodos marcados en ambos cuentan para detectar el ciclo. Sin ese filtro, un ciclo positivo en una parte inalcanzable del grafo dar a -1 de mentiras. CUANDO USAR BELLMAN-FORD: pesos negativos (Dijkstra ya no sirve) o hay que DETECTAR ciclos negativos, que es su  nica ventaja real. Es $O(n*m)$, mucho mas lento que Dijkstra, as  que con pesos no negativos nunca se usa.

C++\CSES\Graph Algorithms\High Score.cpp

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  #include <climits>
5
6  using namespace std;
7
8  int main()
9  {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     // INPUT
14     long long n, m;
15     cin >> n >> m;
16
17     vector<vector<pair<long long, long long>>> conexion(n);
18     vector<vector<pair<long long, long long>>> conexionInversa(n);
19
20     for(long long i = 0; i < m; i++)
21     {
22         long long a, b, c;
23         cin >> a >> b >> c;
24
25         a--;
26         b--;
27
28         conexion[a].push_back({b, c});
29         conexionInversa[b].push_back({a, c});
30     }
31
32     vector<bool> de0(n, false);
33     vector<bool> antesN(n, false);
34
35     // BFS desde 0
36     queue<long long> cola;
37     cola.push(0);
38     de0[0] = true;
39
40     while(!cola.empty())
41     {
42         long long actual = cola.front();
43         cola.pop();
44     }
```

16.9 High Score - Bellman-Ford (Camino Maximo) (cont.)

```

45     for(auto vecino : conexion[actual])
46     {
47         long long siguiente = vecino.first;
48
49         if(!de0[siguiente])
50         {
51             de0[siguiente] = true;
52             cola.push(siguiente);
53         }
54     }
55 }
56
57 // BFS desde n-1 usando el grafo inverso
58 cola.push(n - 1);
59 antesN[n - 1] = true;
60
61 while(!cola.empty())
62 {
63     long long actual = cola.front();
64     cola.pop();
65
66     for(auto vecino : conexionInversa[actual])
67     {
68         long long siguiente = vecino.first;
69
70         if(!antesN[siguiente])
71         {
72             antesN[siguiente] = true;
73             cola.push(siguiente);
74         }
75     }
76 }
77
78 // OUTPUT
79 vector<long long> distancia(n, LLONG_MIN);
80 distancia[0] = 0;
81
82 bool ciclo = false;
83
84 for(long long i = 0; i < n; i++)
85 {
86     ciclo = false;
87
88     for(long long a = 0; a < n; a++)
89     {
90         if(!de0[a] || !antesN[a] || distancia[a] == LLONG_MIN)
91             continue;
92
93         for(auto vecino : conexion[a])
94         {
95             long long b = vecino.first;
96             long long c = vecino.second;
97
98             if(!de0[b] || !antesN[b])
99                 continue;
100
101             if(distancia[b] < distancia[a] + c)
102             {
103                 distancia[b] = distancia[a] + c;
104                 ciclo = true;
105             }
106         }
107     }
108 }
109
110 if(ciclo)
111     cout << -1 << '\n';
112 else
113     cout << distancia[n - 1] << '\n';
114
115 return 0;
116 }
```

16.10 Investigation - Dijkstra con Cuatro Cantidades a la Vez

Un solo Dijkstra que además del costo mínimo lleva otras tres cosas: cuantos caminos mínimos hay (modulo $1e9+7$), el mínimo de aristas de un camino mínimo y el máximo. LA IDEA ES QUE LAS CUATRO SE PROPAGAN CON LA MISMA REGLA, mirando si la distancia MEJORA o EMPATA: - si mejora: se reinicia todo copiando lo del nodo de donde se viene (caminos, y los conteos de aristas mas uno) - si empata: se ACUMULA, sumando los caminos y quedándose con el mínimo y el máximo de aristas Ese "si mejora reinicio, si empata acumulo" es el patron general para colgar estadísticas de un Dijkstra, y sirve igual para contar caminos, sumar pesos o llevar cualquier cosa asociativa. El if (peso != distancia[actual]) continue descarta las entradas viejas de la cola, que es lo que permite usar priority_queue sin decrease-key. Sin el se procesarían nodos con distancias obsoletas y los acumulados saldrían mal, no solo lentos. Los caminos se cuentan modulo $1e9+7$ porque pueden ser astronómicos, mientras que las distancias van en long long sin modulo: mezclar las dos cosas es el error clásico aquí. Solo funciona porque los pesos son no negativos, que es lo que garantiza que un nodo ya sacado de la cola no vuelva a mejorar. Con pesos negativos habría que irse a Bellman-Ford y la lógica de acumulación se complica.

C++\CSES\Graph Algorithms\investigation.cpp

```
1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <utility>
5 #include <limits>
6 #include <algorithm>
7
8 using namespace std;
9
10 int main()
11 {
12     ios::sync_with_stdio(false);
13     cin.tie(nullptr);
14
15     const long long MOD = 1000000007;
16
17     long long n, m;
18     cin >> n >> m;
19
20     vector<vector<pair<long long, long long>>> conexiones(n);
21
22     for(long long i = 0; i < m; i++)
23     {
24         long long a, b, c;
25         cin >> a >> b >> c;
26
27         a--;
28         b--;
29
30         conexiones[a].push_back({b, c});
31     }
32
33     vector<long long> caminos(n, 0);
34     vector<long long> distancia(n, LLONG_MAX);
35     vector<long long> minvuelos(n, LLONG_MAX);
36     vector<long long> maxvuelos(n, 0);
37
38     priority_queue<
39         pair<long long, long long>,
40         vector<pair<long long, long long>>,
41         greater<pair<long long, long long>>
42     > cola;
43
44     caminos[0] = 1;
45     distancia[0] = 0;
46     minvuelos[0] = 0;
47
48     cola.push({0, 0});
49
50     while(!cola.empty())
51     {
52         long long peso = cola.top().first;
53         long long actual = cola.top().second;
```

16.10 Investigation - Dijkstra con Cuatro Cantidades a la Vez (cont.)

```
54     cola.pop();
55
56     if(peso != distancia[actual])
57     {
58         continue;
59     }
60
61     for(pair<long long, long long> siguiente : conexiones[actual])
62     {
63         long long hacia = siguiente.first;
64         long long cuanto = siguiente.second;
65
66         long long nuevaDistancia = peso + cuanto;
67         long long nuevosVuelos = minvuelos[actual] + 1;
68
69         if(distancia[hacia] > nuevaDistancia)
70         {
71             distancia[hacia] = nuevaDistancia;
72             caminos[hacia] = caminos[actual];
73
74             minvuelos[hacia] = nuevosVuelos;
75             maxvuelos[hacia] = maxvuelos[actual] + 1;
76
77             cola.push({nuevaDistancia, hacia});
78         }
79         else if(distancia[hacia] == nuevaDistancia)
80         {
81             caminos[hacia] += caminos[actual];
82             caminos[hacia] %= MOD;
83
84             minvuelos[hacia] = min(minvuelos[hacia], nuevosVuelos);
85             maxvuelos[hacia] = max(maxvuelos[hacia], maxvuelos[actual] + 1);
86         }
87     }
88 }
89
90 cout << distancia[n - 1] << " "
91      << caminos[n - 1] << " "
92      << minvuelos[n - 1] << " "
93      << maxvuelos[n - 1] << '\n';
94
95     return 0;
96 }
```

16.11 Labyrinth - BFS con Reconstruccion de Camino

BFS en grilla guardando de donde vino cada celda en la matriz anterior[], y al final se sube desde el destino hasta el origen leyendo esos padres y se invierte. La dirección de cada paso se deduce comparando coordenadas con el padre. CUANDO USAR: cuando el problema no pide solo la DISTANCIA sino el camino en sí. La regla es que BFS da el camino mínimo solo si todas las aristas pesan lo mismo; con pesos distintos toca Dijkstra, y con pesos 0 y 1 la versión barata es 0-1 BFS con deque. Ojo con marcar: aquí se marca al encolar, y la celda 'B' se detecta ANTES de pisarla para no perder el destino.

C++\CSES\Graph Algorithms\Labyrinth.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     ios::sync_with_stdio(false);
7     cin.tie(nullptr);
8
9     long long n, m;
10    cin >> n >> m;
11
12    vector<string> matriz(n);
13
14    long long Ax = -1;
15    long long Ay = -1;
16
17    for(long long i = 0; i < n; i++)
```

16.11 Labyrinth - BFS con Reconstrucción de Camino (cont.)

```

8      {
9          cin >> matriz[i];
10
11      if(Ax == -1)
12      {
13          for(long long j = 0; j < m; j++)
14          {
15              if(matriz[i][j] == 'A')
16              {
17                  Ax = i;
18                  Ay = j;
19                  break;
20              }
21          }
22      }
23
24      vector<vector<pair<long long, long long>>> anterior(
25          n,
26          vector<pair<long long, long long>>(m, {-1, -1})
27      );
28
29      queue<pair<long long, long long>> bfs;
30
31      bfs.push({Ax, Ay});
32
33      bool encontrado = false;
34
35      long long Bx = -1;
36      long long By = -1;
37
38      while(!bfs.empty() && !encontrado)
39      {
40          long long x = bfs.front().first;
41          long long y = bfs.front().second;
42
43          bfs.pop();
44
45          if(x > 0 && (matriz[x - 1][y] == '.' || matriz[x - 1][y] == 'B'))
46          {
47              anterior[x - 1][y] = {x, y};
48
49              if(matriz[x - 1][y] == 'B')
50              {
51                  Bx = x - 1;
52                  By = y;
53                  encontrado = true;
54                  break;
55              }
56
57              matriz[x - 1][y] = 'X';
58              bfs.push({x - 1, y});
59          }
60
61          if(x + 1 < n && (matriz[x + 1][y] == '.' || matriz[x + 1][y] == 'B'))
62          {
63              anterior[x + 1][y] = {x, y};
64
65              if(matriz[x + 1][y] == 'B')
66              {
67                  Bx = x + 1;
68                  By = y;
69                  encontrado = true;
70                  break;
71              }
72
73              matriz[x + 1][y] = 'X';
74              bfs.push({x + 1, y});
75          }
76
77          if(y > 0 && (matriz[x][y - 1] == '.' || matriz[x][y - 1] == 'B'))
78          {

```

16.11 Labyrinth - BFS con Reconstrucción de Camino (cont.)

```

79              anterior[x][y - 1] = {x, y};
80
81              if(matriz[x][y - 1] == 'B')
82              {
83                  Bx = x;
84                  By = y - 1;
85                  encontrado = true;
86                  break;
87              }
88
89              matriz[x][y - 1] = 'X';
90              bfs.push({x, y - 1});
91          }
92
93          if(y + 1 < m && (matriz[x][y + 1] == '.' || matriz[x][y + 1] == 'B'))
94          {
95              anterior[x][y + 1] = {x, y};
96
97              if(matriz[x][y + 1] == 'B')
98              {
99                  Bx = x;
100                 By = y + 1;
101                 encontrado = true;
102                 break;
103             }
104
105             matriz[x][y + 1] = 'X';
106             bfs.push({x, y + 1});
107         }
108     }
109
110     if(!encontrado)
111     {
112         cout << "NO\n";
113         return 0;
114     }
115
116     string recorrido = "";
117
118     long long x = Bx;
119     long long y = By;
120
121     while(x != Ax || y != Ay)
122     {
123         long long anteriorX = anterior[x][y].first;
124         long long anteriorY = anterior[x][y].second;
125
126         if(x - anteriorX > 0)
127         {
128             recorrido += "D";
129         }
130         else if(x - anteriorX < 0)
131         {
132             recorrido += "U";
133         }
134         else if(y - anteriorY > 0)
135         {
136             recorrido += "R";
137         }
138         else
139         {
140             recorrido += "L";
141         }
142     }
143
144     x = anteriorX;
145     y = anteriorY;
146 }
147
148 reverse(recorrido.begin(), recorrido.end());
149
150 cout << "YES\n";
151 cout << recorrido.size() << '\n';

```

16.11 Labyrinth - BFS con Reconstruccion de Camino (cont.)

```

62     cout << recorrido << '\n';
63
64     return 0;
65 }

```

16.12 Longest Flight Route - Camino Mas Largo en DAG (DFS con Memo)

Camino con mas nodos de 1 a n en un DAG, con DFS memoizado: mejor[u] = 1 + max de mejor[hijo]. En un grafo cualquiera el camino mas largo es NP-difícil; aqui es lineal SOLO porque no hay ciclos, y eso hay que tenerlo presente antes de intentarlo en otro problema. EL TRUCO DEL CERO: mejor[u] = 0 significa "desde u no se llega a n". Por eso un hijo solo se toma si camino > 0; así los callejones sin salida no contaminan el maximo, y si al final mejor[0] vale 0 la respuesta es IMPOSSIBLE. El arreglo siguiente[] guarda por donde se fue el maximo, y con eso se reconstruye la ruta. OJO CON LA RECURSION: con un camino de 10^5 nodos la pila de llamadas llega a esa profundidad. En CSES pasa, pero en un juez con pila chica puede dar error de ejecucion. La version sin riesgo es la misma DP recorriendo en orden topologico con Kahn, como "Game Routes".

C++(CSES)\Graph Algorithms\Longest Flight Route.cpp

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  using namespace std;
6
7  long long n, m;
8
9  vector<vector<long long>> conexiones;
10 vector<long long> mejor;
11 vector<long long> siguiente;
12
13 long long dfs(long long actual)
14 {
15     if(actual == n - 1)
16         return 1;
17
18     if(mejor[actual] != -1)
19         return mejor[actual];
20
21     mejor[actual] = 0;
22
23     for(long long hijo : conexiones[actual])
24     {
25         long long camino = dfs(hijo);
26
27         if(camino > 0 && camino + 1 > mejor[actual])
28         {
29             mejor[actual] = camino + 1;
30             siguiente[actual] = hijo;
31         }
32     }
33
34     return mejor[actual];
35 }
36
37 int main()
38 {
39     ios::sync_with_stdio(false);
40     cin.tie(nullptr);
41
42     cin >> n >> m;
43
44     conexiones.resize(n);
45
46     for(long long i = 0; i < m; i++)
47     {
48         long long a, b;
49         cin >> a >> b;
50
51         conexiones[a - 1].push_back(b - 1);
52     }
53
54     mejor.assign(n, -1);

```

16.12 Longest Flight Route - Camino Mas Largo en DAG (DFS con Memo) (cont.)

```

55     siguiente.assign(n, -1);
56
57     long long largo = dfs(0);
58
59     if(largo == 0)
60     {
61         cout << "IMPOSSIBLE\n";
62         return 0;
63     }
64
65     cout << largo << '\n';
66
67     long long actual = 0;
68
69     while(actual != -1)
70     {
71         cout << actual + 1 << ' ';
72
73         if(actual == n - 1)
74             break;
75
76         actual = siguiente[actual];
77     }
78
79     cout << '\n';
80
81     return 0;
82 }

```

16.13 Message Route - BFS de Camino Minimo con Reconstruccion

Mismo patron que Labyrinth pero en grafo general: BFS desde n hacia 1 guardando siguiente[] y la distancia en cuantos[]. Arrancar del destino tiene su gracia: los punteros ya quedan apuntando hacia adelante y el camino se imprime directo, sin invertir nada. El cuantos[] hace doble trabajo, de contador de pasos y de marca de visitado (0 = no visto), que es un truco comun para no llevar dos arreglos. CUANDO USAR: grafo sin pesos y hay que devolver el camino, no solo su largo. Si el grafo tuviera pesos, esto mismo pero con Dijkstra.

C++(CSES)\Graph Algorithms\Message Route.cpp

```

1  #include <iostream>
2  #include <vector>
3  #include <queue>
4
5  using namespace std;
6
7  int main()
8  {
9      ios::sync_with_stdio(false);
10     cin.tie(nullptr);
11
12     long long n, m;
13
14     cin >> n >> m;
15
16     vector<vector<long long>> vecinos(n + 1);
17
18     long long a, b;
19
20     while(m--)
21     {
22         cin >> a >> b;
23
24         vecinos[a].push_back(b);
25         vecinos[b].push_back(a);
26     }
27
28     vector<long long> siguiente(n + 1, -1);
29     vector<long long> cuantos(n + 1, 0);
30
31     queue<long long> cola;
32
33     cola.push(n);
34     cuantos[n] = 1;

```

16.13 Message Route - BFS de Camino Mínimo con Reconstrucción (cont.)

```

35
36 while(!cola.empty())
37 {
38     long long actual = cola.front();
39     cola.pop();
40
41     if(actual == 1)
42     {
43         break;
44     }
45
46     for(long long vecino : vecinos[actual])
47     {
48         if(cuantos[vecino] == 0)
49         {
50             siguiente[vecino] = actual;
51             cuantos[vecino] = cuantos[actual] + 1;
52             cola.push(vecino);
53         }
54     }
55 }
56
57 if(cuantos[1] == 0)
58 {
59     cout << "IMPOSSIBLE\n";
60     return 0;
61 }
62
63 cout << cuantos[1] << '\n';
64
65 long long actual = 1;
66
67 while(actual != n)
68 {
69     cout << actual << ' ';
70     actual = siguiente[actual];
71 }
72
73 cout << n << '\n';
74
75 return 0;
76 }

```

16.14 Monsters - BFS Multifuente + BFS de Escape

Dos BFS encadenados, y el primero es el truco que vale aprender: un BFS MULTIFUENTE, con TODOS los monstruos encolados con distancia 0 a la vez. Eso da, en una sola pasada $O(n*m)$, la distancia de cada celda al monstruo mas cercano, que es lo mismo que correr un BFS por monstruo pero sin pagar el costo de cada uno. Despues va el BFS del jugador, que solo entra a una celda si llega Estrictamente antes que el monstruo mas cercano. Guardando la direccion de llegada se reconstruye la salida. CUANDO USAR EL MULTIFUENTE: siempre que la pregunta sea "distancia al mas cercano de un conjunto" en vez de "distancia desde un punto". Aparece en incendios que se propagan, celdas contaminadas, o el tipico "cuanto tarda en llenarse todo".

C++\CSES\Graph Algorithms\Monsters.cpp

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <algorithm>
5
6 using namespace std;
7
8 int main()
9 {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     long long n, m;
14
15     cin >> n >> m;
16
17     vector<string> matriz(n);

```

16.14 Monsters - BFS Multifuente + BFS de Escape (cont.)

```

8
9     vector<pair<long long, long long>> monstruos;
10
11     long long Ax = -1;
12     long long Ay = -1;
13
14     for(long long i = 0; i < n; i++)
15     {
16         cin >> matriz[i];
17
18         for(long long j = 0; j < m; j++)
19         {
20             if(matriz[i][j] == 'M')
21             {
22                 monstruos.push_back({i, j});
23             }
24             else if(matriz[i][j] == 'A')
25             {
26                 Ax = i;
27                 Ay = j;
28             }
29         }
30     }
31
32     if(Ax == 0 || Ax == n - 1 || Ay == 0 || Ay == m - 1)
33     {
34         cout << "YES\n";
35         cout << "0\n";
36         cout << '\n';
37         return 0;
38     }
39
40     long long dx[4] = {-1, 1, 0, 0};
41     long long dy[4] = {0, 0, -1, 1};
42
43     vector<vector<long long>> distanciaM(n, vector<long long>(m, -1));
44
45     queue<pair<long long, long long>> cola;
46
47     for(pair<long long, long long> monstruo : monstruos)
48     {
49         distanciaM[monstruo.first][monstruo.second] = 0;
50         cola.push(monstruo);
51     }
52
53     while(!cola.empty())
54     {
55         long long x = cola.front().first;
56         long long y = cola.front().second;
57         cola.pop();
58
59         for(long long k = 0; k < 4; k++)
60         {
61             long long nx = x + dx[k];
62             long long ny = y + dy[k];
63
64             if(nx < 0 || nx >= n || ny < 0 || ny >= m)
65             {
66                 continue;
67             }
68
69             if(matriz[nx][ny] == '#')
70             {
71                 continue;
72             }
73
74             if(distanciaM[nx][ny] == -1)
75             {
76                 distanciaM[nx][ny] = distanciaM[x][y] + 1;
77                 cola.push({nx, ny});
78             }
79         }
80     }

```

16.14 Monsters - BFS Multifuente + BFS de Escape (cont.)

```

90     }
91
92     vector<vector<long long>> distanciaA(n, vector<long long>(m, -1));
93     vector<vector<char>> direccion(n, vector<char>(m));
94
95     distanciaA[Ax][Ay] = 0;
96     cola.push({Ax, Ay});
97
98     long long salidaX = -1;
99     long long salidaY = -1;
100
101     while(!cola.empty())
102     {
103         long long x = cola.front().first;
104         long long y = cola.front().second;
105         cola.pop();
106
107         for(long long k = 0; k < 4; k++)
108         {
109             long long nx = x + dx[k];
110             long long ny = y + dy[k];
111
112             if(nx < 0 || nx >= n || ny < 0 || ny >= m)
113             {
114                 continue;
115             }
116
117             if(matriz[nx][ny] == '#')
118             {
119                 continue;
120             }
121
122             if(distanciaA[nx][ny] != -1)
123             {
124                 continue;
125             }
126
127             long long nuevaDistancia = distanciaA[x][y] + 1;
128
129             if(distanciaM[nx][ny] != -1 &&
130                nuevaDistancia >= distanciaM[nx][ny])
131             {
132                 continue;
133             }
134
135             distanciaA[nx][ny] = nuevaDistancia;
136
137             if(k == 0)
138             {
139                 direccion[nx][ny] = 'U';
140             }
141             else if(k == 1)
142             {
143                 direccion[nx][ny] = 'D';
144             }
145             else if(k == 2)
146             {
147                 direccion[nx][ny] = 'L';
148             }
149             else
150             {
151                 direccion[nx][ny] = 'R';
152             }
153
154             if(nx == 0 || nx == n - 1 || ny == 0 || ny == m - 1)
155             {
156                 salidaX = nx;
157                 salidaY = ny;
158                 break;
159             }
160
161             cola.push({nx, ny});

```

16.14 Monsters - BFS Multifuente + BFS de Escape (cont.)

```

162     }
163
164     if(salidaX != -1)
165     {
166         break;
167     }
168
169     if(salidaX == -1)
170     {
171         cout << "NO\n";
172         return 0;
173     }
174
175     string recorridoFinal;
176
177     long long x = salidaX;
178     long long y = salidaY;
179
180     while(x != Ax || y != Ay)
181     {
182         char movimiento = direccion[x][y];
183
184         recorridoFinal += movimiento;
185
186         if(movimiento == 'R')
187         {
188             y--;
189         }
190         else if(movimiento == 'L')
191         {
192             y++;
193         }
194         else if(movimiento == 'D')
195         {
196             x--;
197         }
198         else
199         {
200             x++;
201         }
202     }
203
204     reverse(recorridoFinal.begin(), recorridoFinal.end());
205
206     cout << "YES\n";
207     cout << recorridoFinal.size() << '\n';
208     cout << recorridoFinal << '\n';
209
210     return 0;
211 }

```

16.15 Planetqueriesi - Binary Lifting sobre Grafo Funcional

Cada planeta tiene EXACTAMENTE un teletransporte de salida, así que la estructura es un grafo funcional: una función $x \rightarrow \text{siguiente}[x]$. Las consultas piden donde se termina tras k saltos, con k hasta 10^9 . BINARY LIFTING: $\text{siguiente}[j][i]$ es a donde se llega desde i con 2^j saltos, y se construye con $\text{siguiente}[j][i] = \text{siguiente}[j-1][\text{siguiente}[j-1][i]]$ (dar 2^j saltos es dar 2^{j-1} dos veces). Después cada consulta descompone k en binario y aplica los saltos de los bits encendidos: $O(\log k)$ por consulta en vez de caminar k pasos. POR QUE NO HACE FALTA DETECTAR CICLOS: aunque el grafo funcional siempre acaba en un ciclo, el doubling lo atraviesa sin enterarse. La alternativa (hallar el ciclo y usar modulo) es más rápida en memoria pero mucho más fácil de romper en los bordes. $\text{LOG} = 31$ cubre k hasta $2^{31} - 1$, suficiente para 10^9 . Si k fuera mayor hay que subirlo, y ese es el error silencioso de esta plantilla: con un LOG corto los bits altos de k se ignoran y la respuesta sale mal sin ningún aviso. La tabla ocupa $\text{LOG} * n$; con $n = 2 \cdot 10^5$ son unos 6 millones de long long, así que si aprieta la memoria conviene guardarla en int. Es el mismo mecanismo de "Visible Building Queries" y "Movie Festival Queries" de este cuaderno: si un puntero define una función, sus potencias se precalcultan con doubling.

C++\CSES\Graph Algorithms\planetqueriesi.cpp

```

1 #include <iostream>
2 #include <vector>

```

16.15 Planetqueriesi - Binary Lifting sobre Grafo Funcional (cont.)

```

3
4 using namespace std;
5
6 int main()
7 {
8     ios::sync_with_stdio(false);
9     cin.tie(nullptr);
10
11     long long n, q;
12     cin >> n >> q;
13
14     const long long LOG = 31;
15
16     vector<vector<long long>> siguiente(LOG, vector<long long>(n));
17
18     for(long long i = 0; i < n; i++)
19     {
20         cin >> siguiente[0][i];
21         siguiente[0][i]--;
22     }
23
24     for(long long j = 1; j < LOG; j++)
25     {
26         for(long long i = 0; i < n; i++)
27         {
28             siguiente[j][i] = siguiente[j - 1][siguiente[j - 1][i]];
29         }
30     }
31
32     while(q--)
33     {
34         long long x, k;
35         cin >> x >> k;
36
37         x--;
38
39         for(long long j = 0; j < LOG; j++)
40         {
41             if(k & (1LL << j))
42             {
43                 x = siguiente[j][x];
44             }
45         }
46
47         cout << x + 1 << '\n';
48     }
49
50     return 0;
51 }

```

16.16 Round Trip - Deteccion de Ciclo con BFS

Se busca un ciclo en grafo no dirigido: si durante el recorrido aparece una arista hacia un nodo YA visitado que no es el padre inmediato, ahí se cierra un ciclo, y se reconstruye subiendo por los padres desde los dos extremos de esa arista. La trampa del no dirigido es justamente excluir al padre: si no, toda arista u-v se ve como ciclo de largo 2 apenas se vuelve a mirar. CUANDO USAR: "existe un ciclo", "hay que devolverlo". En grafo DIRIGIDO esto no aplica: ahí se usa DFS con tres colores (blanco, gris, negro) y el ciclo es una arista hacia un gris.

C++\CSES\Graph Algorithms\Round Trip.cpp

```

1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <algorithm>
5
6 using namespace std;
7
8 int main()
9 {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);

```

16.16 Round Trip - Deteccion de Ciclo con BFS (cont.)

```

12
13     long long n, m;
14
15     cin >> n >> m;
16
17     vector<vector<long long>> matriz(n + 1);
18
19     long long a, b;
20
21     while(m--)
22     {
23         cin >> a >> b;
24
25         matriz[a].push_back(b);
26         matriz[b].push_back(a);
27     }
28
29     vector<long long> anterior(n + 1, -1);
30     vector<bool> recorrido(n + 1, false);
31
32     for(long long i = 1; i <= n; i++)
33     {
34         if(recorrido[i]) continue;
35
36         queue<long long> cola;
37
38         recorrido[i] = true;
39         cola.push(i);
40
41         while(!cola.empty())
42         {
43             long long x = cola.front();
44             cola.pop();
45
46             for(long long z : matriz[x])
47             {
48                 if(!recorrido[z])
49                 {
50                     recorrido[z] = true;
51                     anterior[z] = x;
52                     cola.push(z);
53                 }
54                 else if(anterior[x] != z)
55                 {
56                     vector<long long> recorridoX;
57                     vector<long long> recorridoZ;
58
59                     long long actual = x;
60
61                     while(actual != -1)
62                     {
63                         recorridoX.push_back(actual);
64
65                         if(actual == i)
66                         {
67                             break;
68                         }
69
70                         actual = anterior[actual];
71                     }
72
73                     actual = z;
74
75                     while(actual != -1)
76                     {
77                         recorridoZ.push_back(actual);
78
79                         if(actual == i)
80                         {
81                             break;
82                         }
83

```

16.16 Round Trip - Deteccion de Ciclo con BFS (cont.)

```

84         actual = anterior[actual];
85     }
86
87     reverse(recorridoX.begin(), recorridoX.end());
88     reverse(recorridoZ.begin(), recorridoZ.end());
89
90     long long posicion = 0;
91
92     while(posicion < recorridoX.size() &&
93           posicion < recorridoZ.size() &&
94           recorridoX[posicion] == recorridoZ[posicion])
95     {
96         posicion++;
97     }
98
99     posicion--;
100
101     cout << recorridoX.size() + recorridoZ.size()
102          << " 2 * posicion << '\n';
103
104     for(long long j = posicion; j < recorridoX.size(); j++)
105     {
106         cout << recorridoX[j] << ' ';
107     }
108
109     for(long long j = recorridoZ.size() - 1; j >= posicion; j--)
110     {
111         cout << recorridoZ[j] << ' ';
112     }
113
114     cout << '\n';
115
116     return 0;
117 }
118 }
119 }
120 }
121
122 cout << "IMPOSSIBLE\n";
123
124 return 0;
125 }

```

16.17 Round Trip (Stack) - Deteccion de Ciclo con Pila (DFS Iterativo) (cont.)

El mismo problema que Round Trip pero cambiando la cola por una PILA, o sea DFS en vez de BFS, escrito de forma iterativa. Para detectar ciclos da igual cual de los dos se use, y esa es la enseñanza: la estructura es lo único que cambia entre BFS y DFS. Vale tener las dos versiones porque a veces el problema pide el ciclo mas corto (BFS ayuda) y a veces cualquiera (DFS es mas directo). POR QUE ITERATIVO: con n hasta 10^5 un DFS recursivo se arriesga a stack overflow. Pasar la recursion a una pila explicita es la salida estandar, y es lo que hace este archivo.

C++\CSES\Graph Algorithms\Round Trip (Stack).cpp

```

1 #include <iostream>
2 #include <vector>
3 #include <stack>
4 #include <algorithm>
5
6 using namespace std;
7
8 int main()
9 {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     long long n, m;
14
15     cin >> n >> m;
16
17     vector<vector<long long>> matriz(n + 1);
18
19

```

16.17 Round Trip (Stack) - Deteccion de Ciclo con Pila (DFS Iterativo) (cont.)

```

18     long long a, b;
19
20     while(m--)
21     {
22         cin >> a >> b;
23
24         matriz[a].push_back(b);
25         matriz[b].push_back(a);
26     }
27
28     vector<long long> anterior(n + 1, -1);
29     vector<bool> recorrido(n + 1, false);
30
31     for(long long i = 1; i <= n; i++)
32     {
33         if(recorrido[i]) continue;
34
35         stack<long long> pila;
36
37         recorrido[i] = true;
38         pila.push(i);
39
40         while(!pila.empty())
41         {
42             long long x = pila.top();
43             pila.pop();
44
45             for(long long z : matriz[x])
46             {
47                 if(!recorrido[z])
48                 {
49                     recorrido[z] = true;
50                     anterior[z] = x;
51                     pila.push(z);
52                 }
53                 else if(anterior[x] != z)
54                 {
55                     vector<long long> recorridoX;
56                     vector<long long> recorridoZ;
57
58                     long long actual = x;
59
60                     while(actual != -1)
61                     {
62                         recorridoX.push_back(actual);
63
64                         if(actual == i)
65                         {
66                             break;
67                         }
68
69                         actual = anterior[actual];
70                     }
71
72                     actual = z;
73
74                     while(actual != -1)
75                     {
76                         recorridoZ.push_back(actual);
77
78                         if(actual == i)
79                         {
80                             break;
81                         }
82
83                         actual = anterior[actual];
84                     }
85
86                     reverse(recorridoX.begin(), recorridoX.end());
87                     reverse(recorridoZ.begin(), recorridoZ.end());
88
89                     long long posicion = 0;
90

```

16.17 Round Trip (Stack) - Deteccion de Ciclo con Pila (DFS Iterativo) (cont.)

```

91
92     while(posicion < recorridoX.size() &&
93           posicion < recorridoZ.size() &&
94           recorridoX[posicion] == recorridoZ[posicion])
95     {
96         posicion++;
97     }
98     posicion--;
99
100     cout << recorridoX.size() + recorridoZ.size()
101           - 2 * posicion << '\n';
102
103     for(long long j = posicion; j < recorridoX.size(); j++)
104     {
105         cout << recorridoX[j] << ' ';
106     }
107
108     for(long long j = recorridoZ.size() - 1; j >= posicion; j--)
109     {
110         cout << recorridoZ[j] << ' ';
111     }
112
113     cout << '\n';
114
115     return 0;
116 }
117 }
118 }
119 }
120 }
121
122 cout << "IMPOSSIBLE\n";
123
124 return 0;
125 }

```

16.18 Round Trip II - Ciclo en Grafo Dirigido (DFS de Tres Colores)

Encontrar un ciclo en un grafo DIRIGIDO con DFS de tres colores: 0 = sin visitar, 1 = en la pila (gris), 2 = terminado (negro). Hay ciclo si y solo si aparece una arista hacia un nodo GRIS, porque ese nodo es un ancestro en la rama actual. Una arista hacia un negro NO es ciclo: ese nodo ya se exploró entero por otra rama. Confundir gris con negro es EL error clásico aquí, y por eso el "visitado si/no" de los grafos no dirigidos no sirve. LO QUE HACE BIEN ESTE DFS ITERATIVO, y conviene copiarlo tal cual: la pila guarda el par (nodo, índice de la siguiente arista por mirar). Así un nodo sigue gris hasta que se revisaron TODAS sus aristas, y solo ahí se vuelve negro. El DFS iterativo ingenuo (sacar el nodo de la pila apenas se visita, como en "Round Trip (Stack)") no sirve para colores, porque pierde el momento en que el nodo termina. El ciclo se reconstruye subiendo por anterior[] desde x hasta z, e invirtiendo. CUANDO USAR: detectar dependencias circulares, o como chequeo de que un grafo es un DAG antes de hacerle DP. Si solo hace falta saber SI hay ciclo, Kahn (ver "Course Schedule") es más corto.

C++\CSES\Graph Algorithms\Round Trip II.cpp

```

1 #include <iostream>
2 #include <vector>
3 #include <stack>
4 #include <algorithm>
5
6 using namespace std;
7
8 int main()
9 {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     long long n, m;
14
15     cin >> n >> m;
16
17     vector<vector<long long>> matriz(n + 1);
18
19     long long a, b;

```

16.18 Round Trip II - Ciclo en Grafo Dirigido (DFS de Tres Colores) (cont.)

```

20
21     while (m--)
22     {
23         cin >> a >> b;
24         matriz[a].push_back(b);
25     }
26
27     vector<long long> anterior(n + 1, -1);
28     vector<long long> estado(n + 1, 0);
29
30     for (long long i = 1; i <= n; i++)
31     {
32         if (estado[i] != 0) continue;
33
34         stack<pair<long long, long long>> pila;
35
36         estado[i] = 1;
37         pila.push({i, 0});
38
39         while (!pila.empty())
40         {
41             long long x = pila.top().first;
42             long long posicion = pila.top().second;
43
44             if (posicion == matriz[x].size())
45             {
46                 estado[x] = 2;
47                 pila.pop();
48                 continue;
49             }
50
51             long long z = matriz[x][posicion];
52
53             pila.top().second++;
54
55             if (estado[z] == 0)
56             {
57                 anterior[z] = x;
58                 estado[z] = 1;
59                 pila.push({z, 0});
60             }
61             else if (estado[z] == 1)
62             {
63                 vector<long long> ciclo;
64
65                 ciclo.push_back(z);
66
67                 long long actual = x;
68
69                 while (actual != z)
70                 {
71                     ciclo.push_back(actual);
72                     actual = anterior[actual];
73                 }
74
75                 ciclo.push_back(z);
76                 reverse(ciclo.begin(), ciclo.end());
77
78                 cout << ciclo.size() << '\n';
79
80                 for (long long ciudad : ciclo)
81                     cout << ciudad << ' ';
82
83                 cout << '\n';
84
85                 return 0;
86             }
87         }
88     }
89
90     cout << "IMPOSSIBLE\n";
91

```

16.18 Round Trip II - Ciclo en Grafo Dirigido (DFS de Tres Colores) (cont.)

```
92     return 0;
93 }
```

16.19 Shortest Routes I - Dijkstra con Priority Queue

Dijkstra clasico en $O(m \log n)$ con priority_queue de minimos. Los dos detalles que importan: la cola guarda el par (distancia, nodo) en ese orden para que ordene por distancia, y el `if(dist > distancia[actual])` continúe descarta las entradas viejas que quedaron en la cola cuando un nodo mejoro su distancia. Sin ese filtro no esta mal, pero se procesa basura. CUANDO USAR: camino minimo desde UN origen con pesos NO NEGATIVOS. Con un peso negativo Dijkstra da respuestas mal y hay que irse a Bellman-Ford. Si todos los pesos son iguales, BFS hace lo mismo mas rapido y sin cola de prioridad. Ojo con `LLONG_MAX` como infinito: si se suma algo se desborda. Aqui no pasa porque solo se compara, pero es el error clasico de esta plantilla.

C++\CSES\Graph Algorithms\Shortest Routes I.cpp

```
1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <climits>
5
6 using namespace std;
7
8 int main()
9 {
10     // INPUT
11     long long n, m;
12     cin >> n >> m;
13
14     vector<vector<pair<long long, long long>>> matriz(n + 1);
15
16     for(long long i = 0; i < m; i++)
17     {
18         long long a, b, c;
19         cin >> a >> b >> c;
20
21         matriz[a].push_back({b, c});
22     }
23
24     // OUTPUT
25     vector<long long> distancia(n + 1, LLONG_MAX);
26     priority_queue<pair<long long, long long>,
27                 vector<pair<long long, long long>>,
28                 greater<pair<long long, long long>>> cola;
29
30     distancia[1] = 0;
31     cola.push({0, 1});
32
33     while(!cola.empty())
34     {
35         long long dist = cola.top().first;
36         long long actual = cola.top().second;
37         cola.pop();
38
39         if(dist > distancia[actual])
40             continue;
41
42         for(auto vecino : matriz[actual])
43         {
44             long long siguiente = vecino.first;
45             long long peso = vecino.second;
46
47             if(distancia[actual] + peso < distancia[siguiente])
48             {
49                 distancia[siguiente] = distancia[actual] + peso;
50                 cola.push({distancia[siguiente], siguiente});
51             }
52         }
53     }
54
55     for(long long i = 1; i < n+1; i++)
56     {
57         cout << distancia[i];
```

16.19 Shortest Routes I - Dijkstra con Priority Queue (cont.)

```
58         if(i < n) cout << " ";
59     }
60
61     cout << endl;
62
63     return 0;
64 }
```

16.20 Shortest Routes II - Dijkstra desde Cada Nodo

Distancias entre TODOS los pares resueltas con n Dijkstras, uno por nodo, en vez del Floyd-Warshall que suele ser la respuesta esperada. Trae un atajo: si un nodo tiene grado 1, su unico camino al resto pasa por su vecino, asi que se copia la fila del vecino sumando el peso de esa arista y se ahorra un Dijkstra completo. CUANDO CADA UNO: Floyd-Warshall es $O(n^3)$ y se escribe en tres lineas, asi que con $n \leq 500$ es lo que uno hace. n Dijkstras es $O(n^m \log n)$, que gana cuando el grafo es DISPERSO (m del orden de n), y pierde feo si es denso. Tambien hay que recordar quedarse con la arista mas barata entre cada par y que Floyd aguanta pesos negativos sin ciclos negativos, mientras Dijkstra no.

C++\CSES\Graph Algorithms\Shortest Routes II.cpp

```
1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <climits>
5
6 using namespace std;
7
8 int main()
9 {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12     // INPUT
13     long long n, m, q;
14     cin >> n >> m >> q;
15
16     vector<vector<pair<long long, long long>>> conexion(n);
17
18     vector<vector<long long>> conexiondirecta(
19         n, vector<long long>(n, LLONG_MAX));
20
21     for(long long i = 0; i < m; i++)
22     {
23         long long a, b, c;
24         cin >> a >> b >> c;
25
26         a--;
27         b--;
28
29         if(conexiondirecta[a][b] > c)
30         {
31             conexiondirecta[a][b] = c;
32             conexiondirecta[b][a] = c;
33         }
34     }
35
36     for(long long i = 0; i < n; i++)
37     {
38         for(long long j = i + 1; j < n; j++)
39         {
40             if(conexiondirecta[i][j] != LLONG_MAX)
41             {
42                 conexion[i].push_back({j, conexiondirecta[i][j]});
43                 conexion[j].push_back({i, conexiondirecta[i][j]});
44             }
45         }
46     }
47
48     // OUTPUT
49     vector<vector<long long>> camino(n, vector<long long>(n, -1));
50     vector<bool> dijkstrados(n, false);
51
52     for(long long x = 0; x < n; x++)
```

16.20 Shortest Routes II - Dijkstra desde Cada Nodo (cont.)

```

53 {
54     if(conexion[x].size() == 1)
55     {
56         long long z = conexion[x][0].first;
57         long long distanciaXZ = conexion[x][0].second;
58
59         if(!dijkstrados[z])
60         {
61             vector<long long> distancia(n, LLONG_MAX);
62
63             priority_queue<
64                 pair<long long, long long>,
65                 vector<pair<long long, long long>>,
66                 greater<pair<long long, long long>>
67             > cola;
68
69             distancia[z] = 0;
70             cola.push({0, z});
71
72             while(!cola.empty())
73             {
74                 long long distanciaActual = cola.top().first;
75                 long long actual = cola.top().second;
76                 cola.pop();
77
78                 if(distanciaActual > distancia[actual])
79                     continue;
80
81                 for(auto vecino : conexion[actual])
82                 {
83                     long long siguiente = vecino.first;
84                     long long peso = vecino.second;
85
86                     if(distancia[actual] + peso < distancia[siguiente])
87                     {
88                         distancia[siguiente] = distancia[actual] + peso;
89                         cola.push({distancia[siguiente], siguiente});
90                     }
91                 }
92             }
93
94             for(long long i = 0; i < n; i++)
95             {
96                 if(distancia[i] != LLONG_MAX)
97                     camino[z][i] = distancia[i];
98             }
99
100             dijkstrados[z] = true;
101         }
102     }
103
104     for(long long i = 0; i < n; i++)
105     {
106         if(camino[z][i] != -1)
107             camino[x][i] = camino[z][i] + distanciaXZ;
108     }
109
110     camino[x][x] = 0;
111     dijkstrados[x] = true;
112 }
113
114 for(long long x = 0; x < n; x++)
115 {
116     if(!dijkstrados[x])
117     {
118         vector<long long> distancia(n, LLONG_MAX);
119
120         priority_queue<
121             pair<long long, long long>,
122             vector<pair<long long, long long>>,
123             greater<pair<long long, long long>>
124         > cola;

```

16.20 Shortest Routes II - Dijkstra desde Cada Nodo (cont.)

```

125
126     distancia[x] = 0;
127     cola.push({0, x});
128
129     while(!cola.empty())
130     {
131         long long distanciaActual = cola.top().first;
132         long long actual = cola.top().second;
133         cola.pop();
134
135         if(distanciaActual > distancia[actual])
136             continue;
137
138         for(auto vecino : conexion[actual])
139         {
140             long long siguiente = vecino.first;
141             long long peso = vecino.second;
142
143             if(distancia[actual] + peso < distancia[siguiente])
144             {
145                 distancia[siguiente] = distancia[actual] + peso;
146                 cola.push({distancia[siguiente], siguiente});
147             }
148         }
149     }
150
151     for(long long i = 0; i < n; i++)
152     {
153         if(distancia[i] != LLONG_MAX)
154             camino[x][i] = distancia[i];
155     }
156
157     dijkstrados[x] = true;
158 }
159 }
160
161 // INPUT
162 for(long long i = 0; i < q; i++)
163 {
164     long long a, b;
165     cin >> a >> b;
166
167     cout << camino[a - 1][b - 1] << '\n';
168 }
169
170 return 0;
171 }

```

16.21 Shortest Routes II (Floyd Warshall) - Floyd-Warshall

Floyd-Warshall, la solución canónica de todos-contra-todos: tres for anidados donde el de AFUERA es el nodo intermedio k . Ese orden no es negociable, y es el error número uno de esta plantilla: si k va por dentro, el algoritmo no considera caminos con varios intermedios y da respuestas mal. La invariante es que después de la iteración k ya están todos los caminos que solo usan $\{0..k\}$ como intermedios, y de ahí sale la inducción. Se inicializa $dist[i][i] = 0$ y se guarda la arista MAS BARATA entre cada par, porque puede haber carreteras repetidas. El continue cuando $dist[i][k]$ es infinito no es solo optimización: evita que se sumen dos infinitos y se desborde. ESTE ARCHIVO ES LA OTRA SOLUCIÓN del mismo problema que "Shortest Routes II", que ahí está resuelto con n Dijkstras. Vale la pena tener las dos y ver la diferencia: Floyd es $O(n^3)$ y se escribe en tres líneas, n Dijkstras es $O(n^m \log n)$ y gana solo si el grafo es disperso. Con $n \leq 500$, que es el límite típico, Floyd es lo que uno escribe. CUANDO USAR: $n \leq 500$ y hacen falta todos los pares. También sirve para cierre transitivo (cambiando $\min$ y $+$ por or y and) y para el cuello de botella minimax (cambiando $+$ por $\max$). Aguanta pesos negativos siempre que no haya ciclos negativos.

C++\CSES\Graph Algorithms\Shortest Routes II (Floyd-Warshall).cpp

```

1 #include <iostream>
2 #include <vector>
3
4 using namespace std;
5
6 int main()

```

16.21 Shortest Routes II (Floyd Warshall) - Floyd-Warshall (cont.)

```

1 {
2     ios::sync_with_stdio(false);
3     cin.tie(nullptr);
4
5     long long n, m, q;
6     cin >> n >> m >> q;
7
8     const long long INF = 1e18;
9
10    vector<vector<long long>> dist(n, vector<long long>(n, INF));
11
12    for(long long i = 0; i < n; i++)
13    {
14        dist[i][i] = 0;
15    }
16
17    for(long long i = 0; i < m; i++)
18    {
19        long long a, b, c;
20        cin >> a >> b >> c;
21
22        a--;
23        b--;
24
25        // puede haber varias carreteras entre a y b: me quedo con la menor
26        if(c < dist[a][b])
27        {
28            dist[a][b] = c;
29            dist[b][a] = c;
30        }
31    }
32
33    // Floyd-Warshall
34    for(long long k = 0; k < n; k++) // nodo intermedio
35    {
36        for(long long i = 0; i < n; i++)
37        {
38            if(dist[i][k] == INF) // no llego a k desde i
39            {
40                continue;
41            }
42
43            for(long long j = 0; j < n; j++)
44            {
45                if(dist[i][k] + dist[k][j] < dist[i][j])
46                {
47                    dist[i][j] = dist[i][k] + dist[k][j];
48                }
49            }
50        }
51    }
52
53    for(long long i = 0; i < q; i++)
54    {
55        long long a, b;
56        cin >> a >> b;
57
58        a--;
59        b--;
60
61        if(dist[a][b] == INF)
62        {
63            cout << -1 << '\n';
64        }
65        else
66        {
67            cout << dist[a][b] << '\n';
68        }
69    }
70
71    return 0;
72 }

```

17 - CSSES: Introductory Problems

17.1 Apple Divisions - Fuerza Bruta 2^n

$n \leq 20$, o sea $2^{20} = \sim 10^6$ subconjuntos: cabe de sobra, así que se prueban todos. La recursión decide para cada posición si el elemento va a un grupo o al otro. EL DETALLE BONITO: no hace falta llevar las dos sumas. Con el total fijo, la diferencia entre grupos es $|\text{total} - 2 \cdot \text{sum}|$, así que basta una sola variable. COMO RECONOCER QUE CABE LA FUERZA BRUTA: mirar n en las restricciones. $n \leq 20$ grita 2^n (o bitmask DP); $n \leq 10$ grita $n!$ (permutaciones); $n \leq 40$ grita meet in the middle. Ese mapeo de n a técnica es de lo más rentable que hay en competitiva. La versión con máscara de bits (for mask = 0 hasta $1 < n$) hace lo mismo sin recursión.

C++\CSSES\Introductory Problems\Apple Divisions.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 long long ans = LLONG_MAX;
5 long long total = 0;
6 vector<long long> a;
7
8 void solve(int pos, long long sum) {
9     if (pos == a.size()) {
10         ans = min(ans, abs(total - 2 * sum));
11         return;
12     }
13
14     solve(pos + 1, sum + a[pos]);
15     solve(pos + 1, sum);
16 }
17
18 int main() {
19     int n;
20     cin >> n;
21
22     a.resize(n);
23
24     for (auto &x : a) {
25         cin >> x;
26         total += x;
27     }
28
29     solve(0, 0);
30
31     cout << ans << '\n';
32 }

```

17.2 Bit Strings - Exponenciación Modular Rápida

$2^n \bmod (10^9+7)$ con exponenciación binaria: se eleva al cuadrado y se va recorriendo el exponente bit por bit, $O(\log n)$ multiplicaciones. POR QUE NO $\text{pow}(2, n)$: pow es de `double`, pierde precisión pasando de 2^{53} y no sabe de módulos. Para exponentes grandes con módulo SIEMPRE esta plantilla. DETALLE DEL TIPO: $\text{res} * a$ puede llegar a $(10^9)^2 = 10^{18}$, que cabe en `long long` pero NO en `int`. Si a y res fueran `int` el producto se desborda antes del módulo. Esta es la razón de que todo en la función sea `long long`. CUANDO USAR: cualquier respuesta "módulo 10^9+7 " que involucre potencias, y es la base del inverso modular por Fermat, $a^{(p-2) \bmod p}$.

C++\CSSES\Introductory Problems\Bit Strings.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const long long MOD = 1e9 + 7;
5
6 long long modpow(long long a, long long b) {
7     long long res = 1;
8
9     while (b) {
10         if (b & 1)
11             res = res * a % MOD;
12
13         a = a * a % MOD;
14         b >>= 1;
15     }
16 }

```

17.2 Bit Strings - Exponenciación Modular Rápida (cont.)

```

5     }
6
7     return res;
8 }
9
20 int main() {
21     long long n;
22     cin >> n;
23
24     cout << modpow(2, n) << '\n';
25 }

```

17.3 Chessboard And Queens - Backtracking con Poda por Diagonales

Las 8 reinas con backtracking fila por fila, que es lo que baja el espacio de búsqueda: poniendo una reina por fila, las filas ya no hay que revisarlas. LO QUE VALE COPIAR SON LOS INDICES DE DIAGONAL, que es donde todo el mundo se equivoca: diag2[row + col] -> las diagonales que bajan hacia la derecha, indice 0..2n-2 diag1[row - col + n-1] -> las que bajan hacia la izquierda, con el +n-1 para no dar negativo Marcar, bajar, y DESMARCAR al volver: los tres arreglos se restauran siempre. CUANDO USAR: colocar piezas u objetos con restricciones de conflicto, en tableros chicos. La clave del backtracking util es que la poda se pueda verificar en O(1) por candidato, y para eso son estos arreglos de ocupado.

C++\CSES\Introductory Problems\Chessboard and Queens.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 char board[8][8];
5 bool col[8], diag1[15], diag2[15];
6 int ans = 0;
7
8 void solve(int row) {
9     if (row == 8) {
10         ans++;
11         return;
12     }
13
14     for (int c = 0; c < 8; c++) {
15         if (board[row][c] == '*')
16             continue;
17
18         if (col[c] || diag1[row - c + 7] || diag2[row + c])
19             continue;
20
21         col[c] = true;
22         diag1[row - c + 7] = true;
23         diag2[row + c] = true;
24
25         solve(row + 1);
26
27         col[c] = false;
28         diag1[row - c + 7] = false;
29         diag2[row + c] = false;
30     }
31 }
32
33 int main() {
34     for (int i = 0; i < 8; i++)
35         for (int j = 0; j < 8; j++)
36             cin >> board[i][j];
37
38     solve(0);
39
40     cout << ans << '\n';
41 }

```

17.4 Coin Piles - Invariantes Aritméticos

Sin simular nada: dos condiciones cerradas. Cada movimiento quita 3 monedas en total, así que (a+b) tiene que ser divisible por 3. Y cada movimiento quita al menos 1 de cada pila, así que ninguna puede ser mas del DOBLE de la otra, $\max(a,b) \leq 2 \cdot \min(a,b)$. Las dos juntas son suficientes. COMO SE LLEGA A ESO: buscar que cantidad se conserva o cambia de forma fija en cada movimiento. Aquí el total baja de 3 en 3 (invariante de divisibilidad) y la diferencia entre pilas esta acotada (invariante de desigualdad). CUANDO USAR: el problema pregunta si un estado es ALCANZABLE, con t casos grandes que hacen imposible simular. Buscar el invariante es lo que convierte una simulación en un par de ifs.

C++\CSES\Introductory Problems\Coin Piles.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int t;
9     cin >> t;
10
11     while (t--) {
12         long long a, b;
13         cin >> a >> b;
14
15         if ((a + b) % 3 == 0 && max(a, b) <= 2 * min(a, b))
16             cout << "YES\n";
17         else
18             cout << "NO\n";
19     }
20 }

```

17.5 Creating Strings - Backtracking sobre Frecuencias

Generar todas las permutaciones DISTINTAS, y el truco esta en recorrer el ALFABETO en cada posición en vez de recorrer las posiciones del string. Como se elige "cual letra va aqui" y cada letra se considera una sola vez por nivel, las repetidas no generan duplicados: no hay que filtrar nada despues. El freq[i]-- antes de bajar y el freq[i]++ al volver es el patron de prestamo de siempre. Y como el alfabeto se recorre de la 'a' a la 'z', las salidas ya vienen en orden lexicografico. ALTERNATIVA: ordenar la cadena y usar next_permutation en un do-while, que tambien salta duplicados y es mas corto. Esta version se prefiere cuando hay que podar por algo. CUANDO USAR: "todas las permutaciones distintas" con letras repetidas. Permutar posiciones y meter en un set funciona pero desperdicia trabajo.

C++\CSES\Introductory Problems\Creating Strings.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<string> ans;
5 string s;
6 int freq[26];
7
8 void solve(string cur) {
9     if (cur.size() == s.size()) {
10         ans.push_back(cur);
11         return;
12     }
13
14     for (int i = 0; i < 26; i++) {
15         if (freq[i] == 0) continue;
16
17         freq[i]--;
18         solve(cur + char('a' + i));
19         freq[i]++;
20     }
21 }
22
23 int main() {
24     cin >> s;
25
26     for (char c : s)
27         freq[c - 'a']++;

```

17.5 Creating Strings - Backtracking sobre Frecuencias (cont.)

```

28
29 solve("");
30
31 cout << ans.size() << '\n';
32
33 for (auto &x : ans)
34     cout << x << '\n';
35 }

```

17.6 Digit Queries - Búsqueda por Bloques de Longitud

La cadena infinita 123456789101112... se recorre por BLOQUES segun la cantidad de digitos: hay 9 numeros de 1 dígito, 90 de 2, 900 de 3, o sea $9 \cdot 10^2$ (d-1) numeros de d digitos, que aportan $d \cdot 9 \cdot 10^d$ (d-1) caracteres. Se le resta a k bloque por bloque hasta que quepa, y ahí division y modulo dan el numero exacto y la posicion dentro de el. CUANDO USAR: cualquier "cual es el k-esimo elemento de esta secuencia infinita" donde la secuencia crece por tramos regulares. El patron es siempre el mismo, saltar tramos completos restando su tamaño y despues aterrizar con division entera y modulo. El while corre a lo sumo 18 veces con k de 10^{18} , así que no hay que optimizar nada. long long en todo: k llega a 10^{18} .

C++\CSES\Introductory Problems\Digit Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int q;
6     cin >> q;
7
8     while (q--) {
9         long long k;
10        cin >> k;
11
12        long long digits = 1;
13        long long count = 9;
14        long long start = 1;
15
16        while (k > digits * count) {
17            k -= digits * count;
18            digits++;
19            count *= 10;
20            start *= 10;
21        }
22
23        long long num = start + (k - 1) / digits;
24        long long pos = (k - 1) % digits;
25
26        string s = to_string(num);
27
28        cout << s[pos] << '\n';
29    }
30 }

```

17.7 Gray Code - Identidad i XOR (i >> 1)

El código Gray de i es exactamente $i \oplus (i \gg 1)$. Eso es todo: no hay recursion ni construccion por niveles, es una sola operacion por numero. POR QUE FUNCIONA: al pasar de i a i+1 cambia una racha de bits bajos, y el xor con el desplazamiento hace que ese cambio se reduzca a UN solo bit de diferencia, que es justo la definicion del código Gray. La vuelta (de Gray a binario) es acumular xor de todos los bits mas altos, que no es simetrica y conviene recordar que existe. CUANDO USAR: recorrer todos los subconjuntos cambiando UN elemento a la vez. Es util cuando recalculas desde cero cuesta caro pero agregar o quitar un elemento cuesta $O(1)$, el mismo espiritu de Mo o de los problemas de torres de Hanoi.

C++\CSES\Introductory Problems\Gray Code.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7
8     for (int i = 0; i < (1 << n); i++) {

```

17.7 Gray Code - Identidad i XOR (i >> 1) (cont.)

```

9         int x = i ^ (i >> 1);
10
11        for (int j = n - 1; j >= 0; j--)
12            cout << ((x >> j) & 1);
13
14        cout << '\n';
15    }
16 }

```

17.8 Grid Coloring I - Greedy Local con 4 Colores

Greedy celda por celda: se prueba A, B, C, D y se toma el primero que no choque con el color ORIGINAL de la celda, con la celda de arriba y con la de la izquierda. POR QUE NUNCA SE ATASCA: son 3 restricciones y 4 colores, así que por conteo siempre queda al menos uno libre. Ese argumento es el que hace que no haga falta backtracking. Y por que basta mirar arriba e izquierda: las celdas de abajo y de la derecha todavia no se han decidido, y cuando les toque van a mirar hacia aca. Cada pareja de vecinos se revisa una sola vez, desde el que va despues. CUANDO USAR: coloreos y asignaciones donde el numero de colores supera al de restricciones locales. Si los colores fueran 3 con 3 restricciones, esto ya no cerraria y habria que probar con busqueda.

C++\CSES\Introductory Problems\Grid Coloring I.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, m;
6     cin >> n >> m;
7
8     vector<string> grid(n);
9
10    for (auto &row : grid)
11        cin >> row;
12
13    string colors = "ABCD";
14
15    for (int i = 0; i < n; i++) {
16        for (int j = 0; j < m; j++) {
17
18            for (char c : colors) {
19                if (c == grid[i][j])
20                    continue;
21
22                if (i > 0 && c == grid[i - 1][j])
23                    continue;
24
25                if (j > 0 && c == grid[i][j - 1])
26                    continue;
27
28                grid[i][j] = c;
29                break;
30            }
31        }
32    }
33
34    for (auto row : grid)
35        cout << row << '\n';
36 }

```

17.9 Grid Path Description - Backtracking con Poda de Zona Encerrada

Backtracking sobre las 48 posiciones del camino, con los '?' abriendo las 4 direcciones y las letras fijas forzando una sola. Sin poda esto no pasa ni de lejos. LA PODA ES TODO EL PROBLEMA: si la celda actual tiene libres arriba y abajo pero ocupadas izquierda y derecha (o al revés), el camino acaba de partir el tablero en dos y una de las mitades queda inalcanzable. Se corta ahí mismo. Esa sola condicion baja el arbol de busqueda de inviable a instantaneo. CUANDO USAR ESTE TIPO DE PODA: en caminos hamiltonianos o recorridos que deben cubrir todo, la poda util casi siempre es de CONECTIVIDAD, o sea detectar temprano que quedo una region a la que ya no se puede llegar. La version fuerte es correr un flood fill en cada nodo; esta version barata mira solo los cuatro vecinos y alcanza.

C++\CSES\Introductory Problems\Grid Path Description.cpp

```

1 #include <bits/stdc++.h>

```

17.9 Grid Path Description - Backtracking con Poda de Zona Encerrada (cont.)

```

2 using namespace std;
3
4 string s;
5 bool vis[7][7];
6 int ans = 0;
7
8 int dx[] = {1, -1, 0, 0};
9 int dy[] = {0, 0, 1, -1};
10
11 void dfs(int x, int y, int pos) {
12     if (x == 6 && y == 0) {
13         if (pos == 48)
14             ans++;
15         return;
16     }
17
18     if (pos == 48)
19         return;
20
21     // Si estamos encerrando una zona, no sirve continuar
22     if (x > 0 && x < 6 && !vis[x - 1][y] && !vis[x + 1][y] &&
23         (y == 0 || vis[x][y - 1]) && (y == 6 || vis[x][y + 1]))
24         return;
25
26     if (y > 0 && y < 6 && !vis[x][y - 1] && !vis[x][y + 1] &&
27         (x == 0 || vis[x - 1][y]) && (x == 6 || vis[x + 1][y]))
28         return;
29
30     for (int d = 0; d < 4; d++) {
31         if (s[pos] != '?' && s[pos] != "DURL"[d])
32             continue;
33
34         int nx = x + dx[d];
35         int ny = y + dy[d];
36
37         if (nx < 0 || nx >= 7 || ny < 0 || ny >= 7)
38             continue;
39
40         if (vis[nx][ny])
41             continue;
42
43         vis[nx][ny] = true;
44         dfs(nx, ny, pos + 1);
45         vis[nx][ny] = false;
46     }
47 }
48
49 int main() {
50     cin >> s;
51
52     vis[0][0] = true;
53
54     dfs(0, 0, 0);
55
56     cout << ans << '\n';
57 }

```

17.10 Increasing Array - Greedy de Maximo Prefijo

Greedy: recorriendo de izquierda a derecha, cada elemento que sea menor que el maximo visto tiene que subir hasta ese maximo, y el costo se acumula. Nunca conviene subir un elemento por encima de lo necesario ni bajar ninguno, porque las operaciones solo suman. POR QUE ES OPTIMO: el maximo de prefijo es una cota inferior, cada posicion tiene que llegar por lo menos ahi, y el greedy paga exactamente esa cota. No hay decision que tomar, y eso es lo que distingue a un greedy trivialmente correcto de uno que hay que demostrar. long long en el acumulador: n posiciones con diferencias de hasta 10^9 se pasan de int.

C++\CSES\Introductory Problems\Increasing Array.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()

```

17.10 Increasing Array - Greedy de Maximo Prefijo (cont.)

```

5 {
6     long long n; cin >> n;
7     long long ant=0;
8     long long god =0;
9     for(int i = 0 ; i < n ; i++)
10     {
11         long long a; cin >> a;
12         if(i!=0)
13         {
14             if(a < ant)
15             {
16                 god+= ant-a;
17
18                 ant = ant;
19                 continue;
20             }
21             ant = a;
22         }
23     }
24     cout << god << endl;
25
26     return 0;
27 }

```

17.11 Knight Moves Grid - BFS en Grilla

BFS de caballo sobre la grilla: la distancia en SALTOS es el numero de aristas, y BFS la da porque todos los movimientos cuestan lo mismo. Los 8 movimientos van en los arreglos dx y dy, que es como se evitan 8 bloques de codigo repetido. Se usa dist inicializada en -1 haciendo doble papel de distancia y de visitado, y se marca al ENCOLAR. CUANDO USAR: distancia minima en grilla con pasos de costo uniforme. Si los pasos costaran distinto seria Dijkstra, y con costos 0 y 1 seria 0-1 BFS con deque. OJO: usa auto [x, y] (structured bindings), que necesita C++17. Con un compilador viejo hay que volver a q.front().first y .second.

C++\CSES\Introductory Problems\Knight Moves Grid.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7
8     vector<vector<int>> dist(n, vector<int>(n, -1));
9
10    int dx[] = {2, 2, -2, -2, 1, 1, -1, -1};
11    int dy[] = {1, -1, 1, -1, 2, -2, 2, -2};
12
13    queue<pair<int, int>> q;
14
15    q.push({0, 0});
16    dist[0][0] = 0;
17
18    while (!q.empty()) {
19        auto [x, y] = q.front();
20        q.pop();
21
22        for (int k = 0; k < 8; k++) {
23            int nx = x + dx[k];
24            int ny = y + dy[k];
25
26            if (nx < 0 || nx >= n || ny < 0 || ny >= n)
27                continue;
28
29            if (dist[nx][ny] != -1)
30                continue;
31
32            dist[nx][ny] = dist[x][y] + 1;
33            q.push({nx, ny});
34        }
35    }
36
37    for (int i = 0; i < n; i++) {

```

17.11 Knight Moves Grid - BFS en Grilla (cont.)

```
38     for (int j = 0; j < n; j++)
39         cout << dist[i][j] << ' ';
40     cout << '\n';
41 }
42 }
```

17.12 Mex Grid Construction - Identidad MEX = XOR

Se pide la grilla donde cada celda es el MEX de lo que ya hay a su izquierda en la fila y arriba en la columna, y resulta que eso es EXACTAMENTE $i \wedge j$. Una sola linea, sin construir nada ni calcular ningun MEX. Verificado calculando el MEX de verdad en una grilla de 60x60. DE DONDE SALE: es la tabla de valores de Grundy del Nim con dos montones. La regla de Grundy es "el MEX de los estados alcanzables", y para dos montones da el xor, que es el teorema de Sprague-Grundy en su caso mas chico. Los dos problemas son literalmente la misma tabla. CUANDO SOSPECHARLO: si un problema define una tabla o secuencia como "el MEX de los anteriores", vale la pena calcular los primeros valores a mano y buscar el patron: casi siempre es un xor o algo periodico. Calcular el MEX de verdad suele ser innecesario.

C++\CSES\Introductory Problems\Mex Grid Construction.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7
8     for (int i = 0; i < n; i++) {
9         for (int j = 0; j < n; j++) {
10             cout << (i ^ j) << ' ';
11         }
12         cout << '\n';
13     }
14 }
```

17.13 Missing Number - Suma de Gauss

La suma de 1..n se sabe de formula, $n*(n+1)/2$, asi que el que falta es esa suma menos la suma de lo que llevo. Una sola pasada, memoria $O(1)$, sin ordenar nada. OJO CON EL TIPO: con $n = 2*10^5$ la suma pasa de $2*10^{10}$, o sea que int se desborda. long long. ALTERNATIVA QUE EVITA EL PROBLEMA: hacer XOR de 1..n con XOR de la entrada. Los repetidos se cancelan y queda el que falta, sin sumas grandes y sin riesgo de desbordar. Vale tenerla presente cuando n sea tan grande que hasta long long apriete. CUANDO USAR ESTE PATRON: "falta exactamente uno", "hay exactamente uno repetido". La idea es comparar contra el total que DEBERIA haber.

C++\CSES\Introductory Problems\Missing Number.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     long long n; cin >> n;
7     long long god = 1;
8     long long pre = 0;
9     for(int i = 0 ; i < n - 1 ; i++)
10     {
11         god += i + 2;
12         long long a;
13         cin >> a;
14         pre += a;
15     }
16     cout << god-pre << endl;
17
18     return 0;
19 }
```

17.14 Number Spiral - Formula por Capas

La espiral se lee por CAPAS: la capa k son las celdas con $\max(\text{fila}, \text{columna}) = k$, y toda la capa va entre $(k-1)^2+1$ y k^2 . Segun la paridad de k la capa se recorre en un sentido o en el otro, y de ahi salen las cuatro formulas de los ifs. CUANDO USAR: el problema pide el valor en una coordenada de un patron infinito, con t consultas y coordenadas hasta 10^9 . Construir la tabla es imposible, asi que hay que encontrar la formula. COMO SE ENCUENTRA: dibujar los primeros 4x4 o 5x5 a mano y buscar que se repite. Aqui lo que se ve es que los cuadrados perfectos caen en la diagonal y que la direccion alterna con la paridad. Sin dibujarlo no sale. long long: y^2 con $y = 10^9$ es 10^{18} .

C++\CSES\Introductory Problems\Number Spiral.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int t;
9     cin >> t;
10
11     while (t--) {
12         long long y, x;
13         cin >> y >> x;
14
15         if (y > x) {
16             if (y % 2 == 0)
17                 cout << y * y - x + 1 << '\n';
18             else
19                 cout << (y - 1) * (y - 1) + x << '\n';
20         } else {
21             if (x % 2 == 0)
22                 cout << (x - 1) * (x - 1) + y << '\n';
23             else
24                 cout << x * x - y + 1 << '\n';
25         }
26     }
27 }
```

17.15 Palindrome Reorder - Conteo de Frecuencias

Un palindromo se puede armar si y solo si a lo sumo UNA letra tiene frecuencia impar, la que quedaria en el centro. Se cuentan frecuencias, se cuentan las impares y si hay mas de una no hay solucion. Para construirlo: media frecuencia de cada letra en orden va a la primera mitad, la letra impar al centro, y la primera mitad invertida al final. Recorrer el alfabeto en orden da la lexicograficamente menor gratis. CUANDO USAR: cualquier problema de anagramas o reordenar caracteres. La frecuencia es lo unico que importa, el orden original de la cadena se puede tirar. El criterio de "a lo sumo una impar" es el que hay que tener memorizado.

C++\CSES\Introductory Problems\Palindrome Reorder.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     string s;
6     cin >> s;
7
8     vector<int> freq(26);
9
10    for (char c : s)
11        freq[c - 'A']++;
12
13    int odd = 0;
14    for (int x : freq)
15        odd += x % 2;
16
17    if (odd > 1) {
18        cout << "NO SOLUTION\n";
19        return 0;
20    }
21
22    string half, mid;
```

17.15 Palindrome Reorder - Conteo de Frecuencias (cont.)

```

23
24     for (int i = 0; i < 26; i++) {
25         half += string(freq[i] / 2, 'A' + i);
26
27         if (freq[i] % 2)
28             mid += char('A' + i);
29     }
30
31     string rev = half;
32     reverse(rev.begin(), rev.end());
33
34     cout << half << mid << rev << '\n';
35 }

```

17.16 Permutations - Construcción Directa

En vez de buscar, se CONSTRUYE: primero todos los pares y después todos los impares. Dos números de la misma paridad difieren en al menos 2, así que dentro de cada mitad no hay problema, y en la frontera se juntan el último par con el 1, que para $n \geq 4$ también difieren en más de 1. LOS CASOS IMPOSIBLES: $n = 2$ y $n = 3$ no tienen solución y hay que responderlo aparte. $n = 1$ si la tiene (un solo número, sin vecinos que comparar), y confundirse ahí es el error típico. CUANDO PENSAR EN CONSTRUCCIÓN: el enunciado dice "imprima CUALQUIER solución" o "muestre una que cumpla". Eso casi siempre significa que existe un patrón y que buscar con backtracking es perder el tiempo. Y conviene mirar los n chicos a mano: ahí salen los casos imposibles.

C++\CSES\Introductory Problems\Permutations.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n;
9     cin >> n;
10
11     if (n == 2 || n == 3) {
12         cout << "NO SOLUTION\n";
13         return 0;
14     }
15
16     for (int i = 2; i <= n; i += 2)
17         cout << i << ' ';
18
19     for (int i = 1; i <= n; i += 2)
20         cout << i << ' ';
21
22     cout << '\n';
23 }

```

17.17 Rab Game I - Construcción por Rotación de Bloque

Construcción con dos condiciones de imposibilidad: $a + b$ no puede pasar de n , y a y b tienen que ser cero los dos o ninguno, que es lo que verifica $(a == 0) != (b == 0)$. La primera fila es la identidad. La segunda rota un bloque de tamaño $a+b$: se sacan los primeros y se ponen al final del bloque, y lo que queda después de m se deja igual. Eso deja exactamente las coincidencias pedidas en cada lado. CUANDO USAR: "construya un ejemplo con exactamente X de esto y Y de aquello". El método es siempre el mismo, primero las condiciones que hacen imposible el caso, y después un bloque chico que se manipula dejando el resto intacto. Rotar es la operación más útil porque controla las coincidencias de forma exacta.

C++\CSES\Introductory Problems\Rab Game I.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int t;
6     cin >> t;
7
8     while (t--) {
9         int n, a, b;
10        cin >> n >> a >> b;

```

17.17 Rab Game I - Construcción por Rotación de Bloque (cont.)

```

11
12        if (a + b > n || (a == 0) != (b == 0)) {
13            cout << "NO\n";
14            continue;
15        }
16
17        cout << "YES\n";
18
19        for (int i = 1; i <= n; i++)
20            cout << i << ' ';
21        cout << '\n';
22
23        int m = a + b;
24
25        for (int i = a + 1; i <= m; i++)
26            cout << i << ' ';
27
28        for (int i = 1; i <= a; i++)
29            cout << i << ' ';
30
31        for (int i = m + 1; i <= n; i++)
32            cout << i << ' ';
33
34        cout << '\n';
35    }
36 }

```

17.18 Repetitions - Barrido Lineal de Rachas

Una pasada llevando el largo de la racha actual y el máximo visto. Al cambiar el carácter se cierra la racha y se reinicia en 1. EL BUG CLÁSICO: la última racha nunca se cierra dentro del for, porque no hay un cambio de carácter que la corte. Por eso hay que comparar contra el máximo OTRA VEZ después del ciclo, y es justamente lo que se olvida. Cada vez que se escribe un barrido de rachas hay que preguntarse quién cierra la última. CUANDO USAR: cualquier "cuántos consecutivos iguales", "el bloque más largo de", en una pasada $O(n)$ y sin memoria.

C++\CSES\Introductory Problems\Repetitions.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     string s; cin >> s;
7     long long maxi = 0;
8     char ant = '-';
9     long long temp = 0;
10    for(int i = 0 ; i < (int) s.size() ; i++)
11    {
12        if(s[i]== ant)
13        {
14            temp++;
15        }
16        else
17        {
18            if(temp>maxi)
19            {
20                maxi = temp;
21            }
22            temp = 1;
23            ant = s[i];
24        }
25    }
26
27    if(temp>maxi)
28    {
29        maxi = temp;
30    }
31    cout << maxi << endl;
32
33    return 0;
34 }

```

17.19 String Reorder - Greedy Lexicografico con Cota de Factibilidad

Se arma la respuesta caracter por caracter tomando siempre la letra mas chica posible, pero antes de fijarla se verifica que lo que queda TODAVIA se pueda terminar. La cota es la clave: con r caracteres por poner, ninguna letra puede quedar con mas de $(r+1)/2$ copias, porque si no dos de ellas quedarian pegadas por fuerza. POR QUE ESTO ES EL PATRON GENERAL: un greedy lexicografico solo funciona si se puede decidir en $O(1)$ o $O(\text{alfabeto})$ si la eleccion deja el problema resoluble. Sin ese chequeo, tomar siempre la letra mas chica lleva a callejones sin salida. La estructura "elegir el menor + verificar factibilidad del resto" aparece en un monton de problemas de construccion lexicografica. CUANDO USAR: "la cadena lexicograficamente menor que cumple X ". Casi nunca hace falta backtracking si se encuentra la cota de factibilidad.

C++\CSES\Introductory Problems\String Reorder.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     string s;
6     cin >> s;
7
8     int freq[26] = {};
9
10    for (char c : s)
11        freq[c - 'A']++;
12
13    string ans;
14    char last = '#';
15
16    for (int pos = 0; pos < s.size(); pos++) {
17        bool found = false;
18
19        for (int c = 0; c < 26; c++) {
20            if (freq[c] == 0 || 'A' + c == last)
21                continue;
22
23            freq[c]--;
24
25            int remaining = s.size() - pos - 1;
26            int mx = 0;
27
28            for (int i = 0; i < 26; i++)
29                mx = max(mx, freq[i]);
30
31            if (mx <= (remaining + 1) / 2) {
32                ans += char('A' + c);
33                last = 'A' + c;
34                found = true;
35                break;
36            }
37
38            freq[c]++;
39        }
40
41        if (!found) {
42            cout << -1 << '\n';
43            return 0;
44        }
45    }
46
47    cout << ans << '\n';
48 }
```

17.20 Tower Of Hanoi - Recursion Clasica

La recursion de siempre: mover $n-1$ al auxiliar, mover el disco grande al destino, mover los $n-1$ del auxiliar al destino. El caso base es $n == 0$ y no hace nada. El total es $2^n - 1$ movimientos, que se imprime antes porque se sabe de formula. LO QUE HAY QUE VER: los tres parametros ROTAN en cada llamada. El auxiliar de un nivel es el destino del de abajo. Escribir eso bien es todo el problema, y equivocarse en el orden de los argumentos es el unico error posible. CUANDO USAR: el patron es "resolver dos subproblemas de tamaño $n-1$ con los roles cambiados". Es el ejemplo canonico para ver que la recursion que genera 2^n salidas es inevitable cuando la SALIDA misma tiene tamaño 2^n .

C++\CSES\Introductory Problems\Tower of Hanoi.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void hanoi(int n, int origen, int destino, int auxiliar) {
5     if (n == 0) return;
6
7     hanoi(n - 1, origen, auxiliar, destino);
8     cout << origen << ' ' << destino << '\n';
9     hanoi(n - 1, auxiliar, destino, origen);
10 }
11
12 int main() {
13     int n;
14     cin >> n;
15
16     cout << (1 << n) - 1 << '\n';
17
18     hanoi(n, 1, 3, 2);
19 }
```

17.21 Trailing Zeros - Formula de Legendre

Los ceros al final de $n!$ los deciden los factores 5, no los 2, porque un cero necesita un par (2,5) y los 2 sobran siempre. La cuenta es la formula de Legendre: $\text{floor}(n/5) + \text{floor}(n/25) + \text{floor}(n/125) + \dots$, que el while hace dividiendo n entre 5 repetidamente y acumulando. POR QUE HAY QUE SEGUIR DIVIDIENDO: 25 aporta dos cincos, 125 aporta tres. Contar solo $\text{floor}(n/5)$ es el error clasico y se queda corto. CUANDO USAR: exponente de un PRIMO p en $n!$, cambiando el 5 por p . Sirve para saber si un binomial es divisible por algo, para el teorema de Kummer, y para cualquier pregunta de divisibilidad sobre factoriales sin calcular el factorial.

C++\CSES\Introductory Problems\Trailing Zeros.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     long long n;
6     cin >> n;
7
8     long long ans = 0;
9
10    while (n) {
11        n /= 5;
12        ans += n;
13    }
14
15    cout << ans << '\n';
16 }
```

17.22 Two Knights - Conteo por Complemento

Contar por el complemento: total de parejas menos las que se atacan. El total es $C(k^2, 2) = k^2 * (k^2 - 1) / 2$, y las parejas que se atacan son $4 * (k-1) * (k-2)$, porque cada rectangulo de 2×3 aporta exactamente 2 ataques y hay $(k-1) * (k-2)$ de ellos en cada una de las dos orientaciones. Formula verificada contra fuerza bruta para k de 1 a 25. CUANDO CONTAR POR COMPLEMENTO: cuando lo que se pide ("que no se ataquen") es dificil de contar directo pero su negacion ("que se ataquen") es facil y estructurada. Es el primer reflejo que hay que tener en conteo, junto con inclusion-exclusion. long long obligatorio: con $k = 10^4$, $k^2 * (k^2 - 1) / 2$ anda por $5 * 10^{15}$.

C++\CSES\Introductory Problems\Two Knights.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
```

17.22 Two Knights - Conteo por Complemento (cont.)

```

1 int main() {
2     long long n;
3     cin >> n;
4
5     for (long long k = 1; k <= n; k++) {
6         long long total = k * k * (k * k - 1) / 2;
7         long long attack = 4 * (k - 1) * (k - 2);
8
9         cout << total - attack << '\n';
10    }
11 }

```

17.23 Two Sets - Construccion por Bloques de 4

La suma total es $n*(n+1)/2$, y para partirla en dos mitades iguales tiene que ser PAR, lo que solo pasa si $n \bmod 4$ es 0 o 3. Ese es todo el criterio de imposibilidad. La construccion aprovecha que cuatro consecutivos se parten perfecto: $\{i, i+3\}$ y $\{i+1, i+2\}$ suman lo mismo. Con $n \bmod 4 == 0$ se cubre todo asi; con $n \bmod 4 == 3$ sobran los tres primeros, que se acomodan a mano como $\{1,2\}$ contra $\{3\}$ y desde el 4 siguen los bloques. CUANDO USAR: reparticiones en grupos de suma igual. Lo primero es siempre la condicion de paridad o divisibilidad de la suma total, que descarta los imposibles de una; despues se busca el bloque chico que se parte solo y se repite.

C++\CSES\Introductory Problems\Two Sets.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7
8     if (n % 4 == 1 || n % 4 == 2) {
9         cout << "NO\n";
10        return 0;
11    }
12
13    vector<int> a, b;
14
15    if (n % 4 == 3) {
16        a.push_back(1);
17        a.push_back(2);
18        b.push_back(3);
19    }
20
21    for (int i = 4; i <= n; i += 4) {
22        a.push_back(i);
23        a.push_back(i + 3);
24        b.push_back(i + 1);
25        b.push_back(i + 2);
26    }
27    else {
28        for (int i = 1; i <= n; i += 4) {
29            a.push_back(i);
30            a.push_back(i + 3);
31            b.push_back(i + 1);
32            b.push_back(i + 2);
33        }
34    }
35
36    cout << "YES\n";
37
38    cout << a.size() << '\n';
39    for (int x : a) cout << x << ' ';
40    cout << '\n';
41
42    cout << b.size() << '\n';
43    for (int x : b) cout << x << ' ';
44    cout << '\n';
45 }

```

17.24 Weird Algorithm - Simulacion Directa

Simular el proceso de Collatz tal cual lo dice el enunciado, sin mas. LO UNICO QUE PUEDE FALLAR: el tipo. Con n hasta 10^6 los valores INTERMEDIOS de la secuencia se van muy por encima de 10^9 , así que con int se desborda y el programa se cuelga o imprime basura. long long y listo. La leccion general: antes de simular, estimar hasta donde crece la variable, no hasta donde crece la ENTRADA. Son dos cosas distintas y esta es la trampa mas comun de los problemas faciles.

C++\CSES\Introductory Problems\Weird Algorithm.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main()
5 {
6     long long n; cin >> n;
7     cout << n << " ";
8     while(n!=1)
9     {
10         if(n%2==0)
11         {
12             n=n/2;
13         }
14         else
15         {
16             n=(n*3)+1;
17         }
18         cout << n << " ";
19     }
20
21     return 0;
22 }

```

18 - CSES: Range Queries

18.1 Distinct Value Queries II - Segment Tree sobre Ocurrencia Anterior

O: $(\log n)$ por operacion

Uso: SegmentTree sobre el anterior-igual; Operation guarda el historial

Saber si en $[x,y]$ hay valores repetidos, y la idea es preciosa: para cada posicion i se calcula $prev[i]$, la ultima posicion antes de i con el mismo valor (o 0 si no hay). Entonces el rango $[x,y]$ tiene un repetido si y solo si existe algun i en $[x,y]$ con $prev[i] >= x$. Eso se convierte en una consulta de MAXIMO de $prev[i]$ sobre el rango, o como esta aqui, de minimo sobre el arreglo complementario. De cualquier forma, el punto es que una pregunta que parecia necesitar contar valores distintos se reduce a un maximo de rango, que un segment tree responde en $O(\log n)$ y ONLINE, sin necesidad de Mo. COMPARAR CON "Distinct Values Queries": ese cuenta CUANTOS distintos hay y por eso necesita Mo o un BIT offline. Este solo pregunta SI hay repetidos, que es mucho mas debil, y esa debilidad es justo lo que permite la reduccion a un maximo. Vale la pena mirar los dos juntos: la leccion es que antes de sacar la artilleria conviene ver si el problema pide menos de lo que parece. EL PATRON DE $prev[i]$: aparece en casi todos los problemas de "distintos en un rango". Se calcula con un map de ultima posicion vista en una pasada. OJO: usa structured bindings (auto $[a, b]$), que piden C++17. En CSES compila, pero con un g++ viejo hay que volver a .first y .second.

C++\CSES\Range Queries\Distinct Value Queries II.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int INF = 1e9;
5
6 struct SegmentTree {
7     int n;
8     vector<int> tree;
9
10    SegmentTree(int n) : n(n), tree(4 * n, INF) {}
11
12    void update(int node, int l, int r, int pos, int val) {
13        if (l == r) {
14            tree[node] = val;
15            return;
16        }
17    }

```

18.1 Distinct Value Queries II - Segment Tree sobre Ocurrencia Anterior (cont.)

```

8      int mid = (1 + r) / 2;
9
10     if (pos <= mid)
11         update(node * 2, 1, mid, pos, val);
12     else
13         update(node * 2 + 1, mid + 1, r, pos, val);
14
15     tree[node] = min(tree[node * 2], tree[node * 2 + 1]);
16 }
17
18 void update(int pos, int val) {
19     update(1, 1, n, pos, val);
20 }
21
22 int query(int node, int l, int r, int ql, int qr) {
23     if (qr < l || r < ql)
24         return INF;
25
26     if (ql <= l && r <= qr)
27         return tree[node];
28
29     int mid = (1 + r) / 2;
30
31     return min(
32         query(node * 2, l, mid, ql, qr),
33         query(node * 2 + 1, mid + 1, r, ql, qr)
34     );
35 }
36
37 int query(int l, int r) {
38     return query(1, 1, n, l, r);
39 }
40
41 struct Operation {
42     int type, x, y;
43 };
44
45 int main() {
46     ios::sync_with_stdio(false);
47     cin.tie(nullptr);
48
49     int n, q;
50     cin >> n >> q;
51
52     vector<int> a(n + 1);
53     vector<int> values;
54
55     for (int i = 1; i <= n; i++) {
56         cin >> a[i];
57         values.push_back(a[i]);
58     }
59
60     vector<Operation> ops(q);
61
62     for (auto &[type, x, y] : ops) {
63         cin >> type >> x >> y;
64
65         if (type == 1)
66             values.push_back(y);
67     }
68
69     sort(values.begin(), values.end());
70     values.erase(unique(values.begin(), values.end()), values.end());
71
72     auto get = [&](int x) {
73         return lower_bound(values.begin(), values.end(), x)
74             - values.begin();
75     };
76
77     for (int i = 1; i <= n; i++)
78         a[i] = get(a[i]);

```

18.1 Distinct Value Queries II - Segment Tree sobre Ocurrencia Anterior (cont.)

```

90
91     int m = values.size();
92
93     vector<set<int>> pos(m);
94
95     for (int i = 1; i <= n; i++)
96         pos[a[i]].insert(i);
97
98     vector<int> nxt(n + 1, INF);
99
100    for (int x = 0; x < m; x++) {
101        auto &s = pos[x];
102
103        for (auto it = s.begin(); it != s.end(); ++it) {
104            auto nx = next(it);
105
106            if (nx != s.end())
107                nxt[*it] = *nx;
108        }
109    }
110
111    SegmentTree st(n);
112
113    for (int i = 1; i <= n; i++)
114        st.update(i, nxt[i]);
115
116    for (auto [type, x, y] : ops) {
117
118        if (type == 2) {
119
120            if (st.query(x, y) <= y)
121                cout << "NO\n";
122            else
123                cout << "YES\n";
124
125        } else {
126
127            int k = x;
128            int newValue = get(y);
129            int oldValue = a[k];
130
131            if (oldValue == newValue)
132                continue;
133
134            // Quitar k del valor anterior
135            {
136                auto &s = pos[oldValue];
137                auto it = s.find(k);
138
139                int prevPos = -1;
140                int nextPos = INF;
141
142                if (it != s.begin()) {
143                    auto p = prev(it);
144                    prevPos = *p;
145                }
146
147                auto nx = next(it);
148
149                if (nx != s.end())
150                    nextPos = *nx;
151
152                if (prevPos != -1)
153                    st.update(prevPos, nextPos);
154
155                st.update(k, INF);
156
157                s.erase(it);
158            }
159
160            // Agregar k al nuevo valor
161            {

```

18.1 Distinct Value Queries II - Segment Tree sobre Ocurrencia Anterior (cont.)

```

62         auto &s = pos[newValue];
63
64         auto it = s.lower_bound(k);
65
66         int nextPos = INF;
67
68         if (it != s.end())
69             nextPos = *it;
70
71         if (it != s.begin()) {
72             auto p = prev(it);
73             st.update(*p, k);
74         }
75
76         st.update(k, nextPos);
77
78         s.insert(k);
79     }
80
81     a[k] = newValue;
82 }
83 }
84 }

```

18.2 Distinct Values Queries - Mo's Algorithm

O: $((n+q) \sqrt{q})$ con Mo

Uso: ordenar Query por (bloque de 1, r); add(i) y remove(i) mueven la ventana

CUANDO USAR MO: esta es la pregunta importante, porque Mo no es la primera opcion casi nunca. Antes de llegar aqui hay que descartar lo demas, en este orden: - Suma, xor o cualquier operacion con INVERSA -> prefix sums, O(1) por consulta. - min, max, gcd, o cualquier operacion asociativa e idempotente sin updates -> sparse table, O(1) por consulta. - Operacion asociativa CON updates -> segment tree o BIT, O(log n). Mo entra cuando la respuesta NO se puede combinar a partir de las mitades. "Cuantos valores distintos", "cual es la moda", "cuantos valores aparecen exactamente k veces", "suma de los cuadrados de las frecuencias": de conocer la respuesta de [l,m] y la de [m+1,r] no sale la de [l,r], asi que ningun segment tree la aguantaria. Lo que si se puede es pasar de un rango al de al lado metiendo o sacando UN elemento en O(1). Eso es exactamente lo que Mo explota. Las tres condiciones que se tienen que cumplir para usarlo: 1. OFFLINE. Hay que tener las q consultas antes de empezar a responder, porque lo primero que se hace es reordenarlas. Si el problema es interactivo o la consulta i+1 depende de la respuesta i, Mo queda descartado de entrada. 2. SIN actualizaciones al arreglo (existe Mo con updates, pero es $O(n^{5/3})$ y es otra plantilla distinta). 3. add y remove de UN elemento en O(1), o a lo sumo O(log n). Los limites tambien mandan: Mo es $O((n+q)\sqrt{q})$, asi que vive comodo en n, q <= $2 \cdot 10^5$ con 1 o 2 segundos. Con n de 10^6 ya no cabe y toca buscar otra cosa. COMO FUNCIONA: se mantiene una ventana [L,R] con el conteo cnt[] de cada valor y el total de distintos. Cada consulta se atiende moviendo L y R hasta su posicion. Movidos en desorden eso seria O(n) por consulta, pero ordenando por (bloque de l, r) cada puntero se mueve poco: R solo avanza dentro de cada bloque (n por bloque, $\sqrt{q}$ bloques) y L nunca se aleja mas de un bloque. De ahi sale el sqrt. El orden de los cuatro while es el canonico: primero los dos que EXPANDEN y despues los dos que CONTRAEN, para que la ventana no quede invertida (L por delante de R) en medio del ajuste. Detalles de esta implementacion: - B = sqrt(n) es lo estandar; el tamano de bloque que minimiza el costo teorico es $n/\sqrt{q}$, que ayuda cuando q es mucho mas chico que n. - La compresion de coordenadas NO es opcional aqui: los valores llegan a 10^9 y cnt[] se indexa por valor, asi que sin comprimir habria que reservar 10^9 enteros. - remove() se llama igual que std::remove de <algorithm>; con using namespace std compila igual porque ninguna sobrecarga de la STL acepta un solo int, pero es un nombre a evitar. Para este problema puntual existe algo mas rapido: offline ordenando las consultas por r y llevando un BIT con la ultima aparicion de cada valor, $O((n+q)\log n)$. Mo se queda porque es la plantilla general: el dia que cambien "distintos" por "moda" o por "frecuencia de frecuencias", el BIT no sirve y esto de aqui solo cambia en add y remove.

C++\CSES\Range Queries\Distinct Values Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int B; // tamano de bloque
5 vector<int> a, cnt;
6 int distinct = 0;
7

```

18.2 Distinct Values Queries - Mo's Algorithm (cont.)

```

8 struct Query { int l, r, idx; };
9
10 void add(int i) { if (cnt[a[i]]++ == 0) distinct++; }
11 void remove(int i) { if (--cnt[a[i]] == 0) distinct--; }
12
13 vector<int> mo(vector<Query>& qs) {
14     sort(qs.begin(), qs.end(), [](const Query& x, const Query& y) {
15         if (x.l / B != y.l / B) return x.l / B < y.l / B; // por bloque de l
16         return x.r < y.r; // luego por r
17     });
18
19     vector<int> ans(qs.size());
20     int L = 0, R = -1; // ventana actual vacia
21     for (auto& q : qs) {
22         while (R < q.r) add(++R);
23         while (L > q.l) add(--L);
24         while (R > q.r) remove(R--);
25         while (L < q.l) remove(L++);
26         ans[q.idx] = distinct;
27     }
28     return ans;
29 }
30
31 int main() {
32     ios::sync_with_stdio(false); cin.tie(nullptr);
33     int n, q; cin >> n >> q;
34     a.resize(n);
35     for (int& x : a) cin >> x;
36
37     // compresion de coordenadas
38     vector<int> v(a);
39     sort(v.begin(), v.end());
40     v.erase(unique(v.begin(), v.end()), v.end());
41     for (int& x : a) x = lower_bound(v.begin(), v.end(), x) - v.begin();
42     cnt.assign(v.size(), 0);
43
44     B = max(1, (int)sqrt(n));
45     vector<Query> qs(q);
46     for (int i = 0; i < q; i++) {
47         cin >> qs[i].l >> qs[i].r;
48         qs[i].l--; qs[i].r--;
49         qs[i].idx = i;
50     }
51
52     for (int x : mo(qs)) cout << x << '\n';
53 }

```

18.3 Dynamic Range Minimum Queries - Segment Tree (Minimo)

O: (log n) update y query, (n) build

Uso: st.build(1,1,n,a); st.update(1,1,n,pos,v); st.query(1,1,n,l,r)

Segment tree de minimos con actualizacion puntual, en O(log n) cada operacion. El nodo p cubre [l,r] y sus hijos son 2p y 2p+1; por eso el arreglo se reserva de tamano 4n, que es la cota segura para cualquier n (no solo potencias de 2). CUANDO USAR SEGMENT TREE Y NO OTRA COSA: la operacion es asociativa pero NO tiene inversa (min, max, gcd), asi que restar no es opcion y Fenwick queda descartado, y ADEMAS el arreglo cambia, lo que descarta la sparse table. Ese cruce es exactamente el nicho del segment tree. Es la plantilla de la que salen todas las variantes: cambiando el min por otra operacion asociativa sirve igual, y agregandole lazy se actualizan rangos completos.

C++\CSES\Range Queries\Dynamic Range Minimum Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct SegmentTree {
7     int n;
8     vector<ll> tree;
9
10     SegmentTree(int n) : n(n), tree(4 * n) {}
11

```

18.3 Dynamic Range Minimum Queries - Segment Tree (Mínimo) (cont.)

```

2 void build(int node, int l, int r, vector<ll>& arr) {
3     if (l == r) {
4         tree[node] = arr[l];
5         return;
6     }
7
8     int mid = (l + r) / 2;
9
10    build(node * 2, l, mid, arr);
11    build(node * 2 + 1, mid + 1, r, arr);
12
13    tree[node] = min(tree[node * 2], tree[node * 2 + 1]);
14 }
15
16 void update(int node, int l, int r, int pos, ll val) {
17     if (l == r) {
18         tree[node] = val;
19         return;
20     }
21
22     int mid = (l + r) / 2;
23
24     if (pos <= mid)
25         update(node * 2, l, mid, pos, val);
26     else
27         update(node * 2 + 1, mid + 1, r, pos, val);
28
29     tree[node] = min(tree[node * 2], tree[node * 2 + 1]);
30 }
31
32 ll query(int node, int l, int r, int ql, int qr) {
33     if (ql <= l && r <= qr)
34         return tree[node];
35
36     int mid = (l + r) / 2;
37     ll ans = LLONG_MAX;
38
39     if (ql <= mid)
40         ans = min(ans, query(node * 2, l, mid, ql, qr));
41
42     if (qr > mid)
43         ans = min(ans, query(node * 2 + 1, mid + 1, r, ql, qr));
44
45     return ans;
46 }
47
48 int main() {
49     ios::sync_with_stdio(false);
50     cin.tie(nullptr);
51
52     int n, q;
53     cin >> n >> q;
54
55     vector<ll> arr(n + 1);
56
57     for (int i = 1; i <= n; i++)
58         cin >> arr[i];
59
60     SegmentTree st(n);
61
62     st.build(1, 1, n, arr);
63
64     while (q--) {
65         int a, b;
66         ll c;
67         cin >> a >> b >> c;
68
69         if (a == 1) {
70             st.update(1, 1, n, b, c);
71         } else {
72             cout << st.query(1, 1, n, b, c) << '\n';
73         }
74     }
75 }

```

18.3 Dynamic Range Minimum Queries - Segment Tree (Mínimo) (cont.)

```

84     }
85 }
86 }

```

18.4 Dynamic Range Sum Queries - Fenwick Tree (BIT)

O: (log n) add y query, (n log n) construir

Uso: Fenwick fw(n); fw.add(i,x); fw.query(l,r) // 1-INDEXADO

Fenwick (o BIT): sumas de prefijo con actualización puntual, ambas en $O(\log n)$, en ~10 líneas. Toda la magia esta en $i \& -i$, que aísla el bit mas bajo encendido: sumando ese valor se sube a los padres (add) y restandolo se baja recorriendo bloques (sum). CUANDO USAR FENWICK EN VEZ DE SEGMENT TREE: si la operacion es SUMA (o cualquiera con inversa) y las actualizaciones son puntuales, Fenwick es la respuesta: menos codigo, menos memoria y mas rapido en la practica. El segment tree se necesita para min, max o gcd, que no se pueden restar, y para lazy propagation. Fenwick es 1-indexado a la fuerza: con $i = 0$, $i \& -i$ es 0 y el for no avanza nunca.

C++(CSES)\Range Queries\Dynamic Range Sum Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct Fenwick {
7     int n;
8     vector<ll> bit;
9
10    Fenwick(int n) : n(n), bit(n + 1, 0) {}
11
12    void add(int i, ll x) {
13        for (; i <= n; i += i & -i)
14            bit[i] += x;
15    }
16
17    ll sum(int i) {
18        ll res = 0;
19        for (; i > 0; i -= i & -i)
20            res += bit[i];
21        return res;
22    }
23
24    ll query(int l, int r) {
25        return sum(r) - sum(l - 1);
26    }
27 };
28
29 int main() {
30     ios::sync_with_stdio(false);
31     cin.tie(nullptr);
32
33     int n, q;
34     cin >> n >> q;
35
36     vector<ll> arr(n + 1);
37     Fenwick fw(n);
38
39     for (int i = 1; i <= n; i++) {
40         cin >> arr[i];
41         fw.add(i, arr[i]);
42     }
43
44     while (q--) {
45         int a, b;
46         ll c;
47         cin >> a >> b >> c;
48
49         if (a == 1) {
50             ll diff = c - arr[b];
51             arr[b] = c;
52             fw.add(b, diff);
53         } else {
54             cout << fw.query(b, c) << '\n';
55         }
56     }
57 }

```

18.4 Dynamic Range Sum Queries - Fenwick Tree (BIT) (cont.)

```
56 }
57 }
```

18.5 Forest Queries - Prefix Sums 2D

Prefix sums en dos dimensiones, donde lo unico nuevo es la inclusion-exclusion: al construir se suma arriba e izquierda y se RESTA la diagonal, que se conto dos veces, y al consultar el rectangulo se hace lo mismo al revés con las cuatro esquinas. CUANDO USAR: consultas de suma sobre submatrices en una matriz que no cambia. Construccion $O(n^2)$, cada consulta $O(1)$. Si la matriz cambia hay que irse a Fenwick 2D o segment tree 2D, que es bastante mas codigo. El truco de reservar $(n+1) \times (n+1)$ con la fila y columna 0 en cero es lo que hace que los indices $i-1$ y $j-1$ nunca se salgan, sin un solo if de borde.

C++\CSES\Range Queries\Forest Queries.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n, q;
9     cin >> n >> q;
10
11     vector<vector<int>> pref(n + 1, vector<int>(n + 1));
12
13     for (int i = 1; i <= n; i++) {
14         string s;
15         cin >> s;
16
17         for (int j = 1; j <= n; j++) {
18             int tree = (s[j - 1] == '*');
19
20             pref[i][j] = tree
21                 + pref[i - 1][j]
22                 + pref[i][j - 1]
23                 - pref[i - 1][j - 1];
24         }
25     }
26
27     while (q--) {
28         int y1, x1, y2, x2;
29         cin >> y1 >> x1 >> y2 >> x2;
30
31         cout << pref[y2][x2]
32             - pref[y1 - 1][x2]
33             - pref[y2][x1 - 1]
34             + pref[y1 - 1][x1 - 1]
35             << '\n';
36     }
37 }
```

18.6 Forest Queries II - Fenwick 2D

Fenwick en dos dimensiones: un for anidado dentro de otro, cada uno con el mismo $i \& -i$ de siempre. Actualizar una celda y consultar el rectangulo de $(1,1)$ a (x,y) cuestan $O(\log^2 n)$, y un rectangulo cualquiera sale por inclusion-exclusion con las cuatro esquinas, igual que en las sumas prefijas 2D. COMPARAR CON "Forest Queries": aquella no tenia cambios y le bastaban sumas prefijas 2D con consulta $O(1)$. Aqui los arboles se cortan y se plantan, y ese cambio es lo que obliga a pasar a Fenwick: las sumas prefijas se tendrían que reconstruir enteras, $O(n^2)$ por actualizacion. El toggle se maneja guardando el estado actual en $a[i][j]$ para saber si sumar $+1$ o -1 . Sin ese arreglo no hay forma de saber que hay en una celda sin hacer una consulta. CUANDO USAR: sumas sobre rectangulos con actualizaciones puntuales y n de hasta unos 1000 (la memoria es n^2). Si hay que actualizar RECTANGULOS enteros, esto no alcanza.

C++\CSES\Range Queries\Forest Queries II.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int n, q;
5 int bit[1005][1005];
6 bool a[1005][1005];
```

18.6 Forest Queries II - Fenwick 2D (cont.)

```
7
8 void add(int x, int y, int v) {
9     for (int i = x; i <= n; i += i & -i)
10         for (int j = y; j <= n; j += j & -j)
11             bit[i][j] += v;
12 }
13
14 int sum(int x, int y) {
15     int ans = 0;
16
17     for (int i = x; i > 0; i -= i & -i)
18         for (int j = y; j > 0; j -= j & -j)
19             ans += bit[i][j];
20
21     return ans;
22 }
23
24 int query(int x1, int y1, int x2, int y2) {
25     return sum(x2, y2)
26         - sum(x1 - 1, y2)
27         - sum(x2, y1 - 1)
28         + sum(x1 - 1, y1 - 1);
29 }
30
31 int main() {
32     ios::sync_with_stdio(false);
33     cin.tie(nullptr);
34
35     cin >> n >> q;
36
37     for (int i = 1; i <= n; i++) {
38         string s;
39         cin >> s;
40
41         for (int j = 1; j <= n; j++) {
42             if (s[j - 1] == '*') {
43                 a[i][j] = true;
44                 add(i, j, 1);
45             }
46         }
47     }
48
49     while (q--) {
50         int type;
51         cin >> type;
52
53         if (type == 1) {
54             int y, x;
55             cin >> y >> x;
56
57             if (a[y][x]) {
58                 a[y][x] = false;
59                 add(y, x, -1);
60             } else {
61                 a[y][x] = true;
62                 add(y, x, 1);
63             }
64         } else {
65             int y1, x1, y2, x2;
66             cin >> y1 >> x1 >> y2 >> x2;
67
68             cout << query(y1, x1, y2, x2) << '\n';
69         }
70     }
71 }
```

18.7 Hotel Queries - Segment Tree con Descenso

O: ($\log n$) por consulta, con descenso por el arbol

Uso: ST st; st.find(x) -> primer hotel con cupo $\geq x$, 0 si no hay

Segment tree de MAXIMOS, pero lo interesante no es la consulta sino el DESCENSO: para hallar el primer hotel con capacidad suficiente, en vez de buscar con una consulta y despues actualizar (dos $O(\log n)$ y una busqueda binaria por fuera), se baja una sola vez desde la raiz preguntando si el hijo izquierdo alcanza; si alcanza se va por ahi, si no por el derecho. Al llegar a la hoja se descuenta de una. Todo en un solo $O(\log n)$. CUANDO USAR EL DESCENSO: cada vez que el problema pida "el PRIMER indice que cumple algo" y el predicado se pueda decidir mirando el maximo (o el minimo, o la suma) de un nodo. Es el patron que reemplaza a "busqueda binaria + consulta", que seria $O(\log^2 n)$.

C++\CSES\Range Queries\Hotel Queries.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct ST {
5     int n;
6     vector<int> t;
7
8     ST(vector<int>& a) {
9         n = a.size() - 1;
10        t.resize(4 * n);
11        build(1, 1, n, a);
12    }
13
14    void build(int p, int l, int r, vector<int>& a) {
15        if (l == r) {
16            t[p] = a[l];
17            return;
18        }
19
20        int m = (l + r) / 2;
21
22        build(p * 2, l, m, a);
23        build(p * 2 + 1, m + 1, r, a);
24
25        t[p] = max(t[p * 2], t[p * 2 + 1]);
26    }
27
28    int find(int p, int l, int r, int x) {
29        if (l == r) {
30            t[p] -= x;
31            return l;
32        }
33
34        int m = (l + r) / 2;
35        int pos;
36
37        if (t[p * 2] >= x)
38            pos = find(p * 2, l, m, x);
39        else
40            pos = find(p * 2 + 1, m + 1, r, x);
41
42        t[p] = max(t[p * 2], t[p * 2 + 1]);
43
44        return pos;
45    }
46
47    int find(int x) {
48        if (t[1] < x)
49            return 0;
50
51        return find(1, 1, n, x);
52    }
53 };
54
55 int main() {
56     ios::sync_with_stdio(false);
57     cin.tie(nullptr);
58 }
```

18.7 Hotel Queries - Segment Tree con Descenso (cont.)

```
59     int n, m;
60     cin >> n >> m;
61
62     vector<int> a(n + 1);
63
64     for (int i = 1; i <= n; i++)
65         cin >> a[i];
66
67     ST st(a);
68
69     while (m--) {
70         int x;
71         cin >> x;
72         cout << st.find(x) << ' ';
73     }
74 }
```

18.8 Increasing Array Queries - Monotonic Stack + Binary Lifting con Costos

Para cada consulta $[l, r]$, el minimo de sumas para que el tramo quede no decreciente. La solucion optima es subir cada elemento al MAXIMO DE PREFIJO del tramo, y ese maximo solo cambia en los "siguientes mayores". Entonces el tramo se parte en bloques: desde i , todo hasta antes de $up[0][i]$ (el siguiente estrictamente mayor) se sube al valor $a[i]$, y cuesta $a[i] * (j - i - 1) - (pref[j] - 1) - pref[i]$. El siguiente mayor sale con una pila monotona de derecha a izquierda, con $a[n+1] = 1e18$ de centinela para que todos tengan a donde apuntar. EL BINARY LIFTING GUARDA EL COSTO JUNTO CON EL SALTO: $up[k][i]$ es a donde se llega con 2^k saltos, y $cost[k][i]$ lo que se paga en ese camino. Cada consulta baja de la potencia mas alta a la mas baja mientras el salto no se pase de r , y el tramo final (de x hasta r , sin llegar al siguiente mayor) se cobra aparte con la misma formula. Es la misma combinacion que "Visible Building Queries", con una sola diferencia: aqui cada salto ademas acumula un valor. Ese es el patron general: si un puntero define una funcion y cada paso tiene un costo, el doubling precalcula a la vez el destino y la suma.

C++\CSES\Range Queries\Increasing Array Queries.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const int N = 200005;
7 const int LOG = 20;
8
9 ll a[N], pref[N], cost[LOG][N];
10 int up[LOG][N];
11
12 int main() {
13     ios::sync_with_stdio(false);
14     cin.tie(nullptr);
15
16     int n, q;
17     cin >> n >> q;
18
19     for (int i = 1; i <= n; i++) {
20         cin >> a[i];
21         pref[i] = pref[i - 1] + a[i];
22     }
23
24     a[n + 1] = 1e18;
25
26     for (int k = 0; k < LOG; k++)
27         up[k][n + 1] = n + 1;
28
29     stack<int> st;
30     st.push(n + 1);
31
32     for (int i = n; i >= 1; i--) {
33         while (a[st.top()] <= a[i])
34             st.pop();
35
36         up[0][i] = st.top();
37
38         int j = up[0][i];
```

18.8 Increasing Array Queries - Monotonic Stack + Binary Lifting con Costos (cont.)

```

38
39     cost[0][i] =
40         a[i] * (j - i - 1)
41         - (pref[j - 1] - pref[i]);
42
43     st.push(i);
44 }
45
46 for (int k = 1; k < LOG; k++) {
47     for (int i = 1; i <= n; i++) {
48         int mid = up[k - 1][i];
49
50         up[k][i] = up[k - 1][mid];
51
52         cost[k][i] =
53             cost[k - 1][i] +
54             cost[k - 1][mid];
55     }
56 }
57
58 while (q--) {
59     int l, r;
60     cin >> l >> r;
61
62     int x = 1;
63     ll ans = 0;
64
65     for (int k = LOG - 1; k >= 0; k--) {
66         if (up[k][x] <= r) {
67             ans += cost[k][x];
68             x = up[k][x];
69         }
70     }
71
72     if (x < r) {
73         ans += a[x] * (r - x)
74             - (pref[r] - pref[x]);
75     }
76 }
77
78 cout << ans << '\n';
79 }
80 }

```

18.9 List Removals - Fenwick con Descenso Binario (k-esimo)

O: $(\log^2 n)$ por consulta, o $(\log n)$ con descenso binario

Uso: fw con 1 en cada vivo; busca el k-esimo vivo y pon 0 al sacarlo

Fenwick de unos y ceros (1 = la posicion todavia esta) donde lo interesante es kth(): encontrar la k-esima posicion viva sin busqueda binaria por fuera. EL DESCENSO SOBRE EL BIT: se arranca en la potencia de 2 mas grande y se va bajando, probando en cada paso si el bloque que empieza en pos+p acumula menos de k. Si acumula menos, se avanza y se le resta ese acumulado a k. Como los bloques del Fenwick son exactamente potencias de 2, ese recorrido es un descenso por el arbol implicito y sale en $O(\log n)$ de una, contra $O(\log^2 n)$ de hacer busqueda binaria llamando a sum() en cada paso. CUANDO USAR: "el k-esimo elemento que queda", "el k-esimo mas chico", con inserciones y borrados. Es el mismo patron que el descenso del segment tree (ver "Hotel Queries"): en vez de buscar por fuera, se baja por la estructura aprovechando lo que ya guarda cada nodo. OJO con el $1 < k < 18$: tiene que ser una potencia de 2 mayor o igual a n. Con n mas grande hay que subirlo, y ese es el error silencioso de esta plantilla.

C++\VCSES\Range Queries\List Removals.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Fenwick {
5     int n;
6     vector<int> bit;
7
8     Fenwick(int n) : n(n), bit(n + 1, 0) {}
9
10    void add(int i, int x) {
11        for (; i <= n; i += i & -i)

```

18.9 List Removals - Fenwick con Descenso Binario (k-esimo) (cont.)

```

12         bit[i] += x;
13     }
14
15     int sum(int i) {
16         int res = 0;
17         for (; i > 0; i -= i & -i)
18             res += bit[i];
19         return res;
20     }
21
22     int kth(int k) {
23         int pos = 0;
24
25         for (int p = 1 << 18; p; p >>= 1) {
26             int next = pos + p;
27
28             if (next <= n && bit[next] < k) {
29                 pos = next;
30                 k -= bit[next];
31             }
32         }
33
34         return pos + 1;
35     }
36 };
37
38 int main() {
39     ios::sync_with_stdio(false);
40     cin.tie(nullptr);
41
42     int n;
43     cin >> n;
44
45     vector<int> a(n + 1);
46
47     for (int i = 1; i <= n; i++) {
48         cin >> a[i];
49     }
50
51     Fenwick fw(n);
52
53     for (int i = 1; i <= n; i++)
54         fw.add(i, 1);
55
56     for (int i = 0; i < n; i++) {
57         int k;
58         cin >> k;
59
60         int pos = fw.kth(k);
61
62         cout << a[pos] << ' ';
63
64         fw.add(pos, -1);
65     }
66
67     cout << '\n';
68 }

```

18.10 Missing Coin Sum Queries - Cubetas por Potencia de 2 + Segment Tree

La versi n por rangos del cl sico "menor suma que no se puede formar". El greedy de siempre es ordenar las monedas y mantener sum: si la siguiente moneda es $\leq \text{sum}+1$ se absorbe, si no la respuesta es $\text{sum}+1$. Ordenar cada rango ser  $O(n \log n)$ por consulta; lo que lo evita son las CUBETAS: la moneda x va a la cubeta $\text{floor}(\log_2 x)$, o sea que en la cubeta k todo est  en $[2^k, 2^{(k+1)})$. LA OBSERVACION QUE HACE FUNCIONAR LAS CUBETAS: dentro de una misma cubeta, cualquier moneda es menor que el doble de la m s chica. As  que si la m s chica de la cubeta cabe (es $\leq \text{sum}+1$), despu s de absorberla TODAS las dem s de esa cubeta tambi n caben, y se pueden sumar de golpe con una suma prefija. Solo hace falta saber el M NIMO de cada cubeta en el rango, y para eso va un segment tree de m nimos por cubeta. Con 30 cubetas cada consulta cuesta $O(30 \log n)$ en vez de ordenar. Es pesado en memoria: las dos tablas de 30 columnas suman cerca de 100 MB. En CSES entra (512 MB); en un juez de 256 tambi n, pero no sobra mucho.

C++(CSES/Range Queries/Missing Coin Sum Queries.cpp

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

const int N = 200005;
const int K = 30;
const int INF = 2e9;

int st[2 * N][K];
ll pref[N][K];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;

    for (int i = 0; i < 2 * N; i++)
        for (int j = 0; j < K; j++)
            st[i][j] = INF;

    for (int i = 1; i <= n; i++) {
        int x;
        cin >> x;

        int k = 31 - __builtin_clz(x);

        st[N + i][k] = x;
        pref[i][k] = x;
    }

    for (int i = 1; i <= n; i++)
        for (int k = 0; k < K; k++)
            pref[i][k] += pref[i - 1][k];

    for (int i = N - 1; i >= 1; i--)
        for (int k = 0; k < K; k++)
            st[i][k] = min(st[i * 2][k], st[i * 2 + 1][k]);

    while (q--) {
        int l, r;
        cin >> l >> r;

        int L = l + N;
        int R = r + N;

        int mn[K];

        for (int k = 0; k < K; k++)
            mn[k] = INF;

        while (L <= R) {
            if (L & 1) {
                for (int k = 0; k < K; k++)
                    mn[k] = min(mn[k], st[L][k]);
            }
        }
    }
}
```

18.10 Missing Coin Sum Queries - Cubetas por Potencia de 2 + Segment Tree (cont.)

```
L++;
}

if (!(R & 1)) {
    for (int k = 0; k < K; k++)
        mn[k] = min(mn[k], st[R][k]);
    R--;
}

L /= 2;
R /= 2;
}

ll sum = 0;

for (int k = 0; k < K; k++) {
    if (sum + 1 < mn[k] &&
        sum + 1 < (1LL << (k + 1))) {
        cout << sum + 1 << '\n';
        goto done;
    }

    sum += pref[R][k] - pref[L - 1][k];
}

cout << sum + 1 << '\n';

done;
}
```

18.11 Movie Festival Queries - Greedy + Binary Lifting sobre el Tiempo

M ximo de pel culas que se pueden ver enteras dentro de $[a, b]$. El greedy de intervalos es el de siempre: ver primero la que TERMINA m s temprano entre las que empiezan despu s del momento actual (intercambio cl sico: cambiar cualquier elecci n por la que termina antes nunca empeora). Aqu  el truco es indexar por TIEMPO, no por pel cula: $\text{nxt}[t]$ = el fin m s temprano de una pel cula que empieza en t o despu s. Se calcula con un m nimo de sufijos, y queda como una funci n $t \rightarrow \text{nxt}[t]$ que es justo "ver la siguiente pel cula". Sobre esa funci n se monta binary lifting: $\text{up}[k][t]$ es el momento tras ver 2^k pel culas, y cada consulta cuenta cuantos saltos caben antes de pasarse de b . $O(\log T)$ por consulta. Es el mismo patr n que "Visible Building Queries": un puntero, doubling encima, y contar saltos. La tabla $\text{up}[20][10^6]$ ocupa unos 80 MB, que es el precio de indexar por tiempo.

C++(CSES/Range Queries/Movie Festival Queries.cpp

```
#include <bits/stdc++.h>
using namespace std;

const int MAXT = 1000000;
const int LOG = 20;

int up[LOG][MAXT + 2];
int nxt[MAXT + 2];
int en[MAXT + 2];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;

    for (int i = 0; i <= MAXT + 1; i++)
        en[i] = MAXT + 1;

    for (int i = 0; i < n; i++) {
        int a, b;
        cin >> a >> b;
        en[a] = min(en[a], b);
    }

    nxt[MAXT + 1] = MAXT + 1;
}
```

18.11 Movie Festival Queries - Greedy + Binary Lifting sobre el Tiempo (cont.)

```

28
29     for (int i = MAXT; i >= 1; i--)
30         nxt[i] = min(en[i], nxt[i + 1]);
31
32     for (int i = 1; i <= MAXT; i++)
33         up[0][i] = nxt[i];
34
35     up[0][MAXT + 1] = MAXT + 1;
36
37     for (int k = 1; k < LOG; k++) {
38         up[k][MAXT + 1] = MAXT + 1;
39
40         for (int i = 1; i <= MAXT; i++) {
41             int mid = up[k - 1][i];
42             up[k][i] = up[k - 1][mid];
43         }
44     }
45
46     while (q--) {
47         int a, b;
48         cin >> a >> b;
49
50         int pos = a;
51         int ans = 0;
52
53         for (int k = LOG - 1; k >= 0; k--) {
54             if (up[k][pos] <= b) {
55                 ans += (1 << k);
56                 pos = up[k][pos];
57             }
58         }
59
60         cout << ans << '\n';
61     }
62 }

```

18.12 Pizzeria Queries - Truco de $p[i]$ mas y menos i

O: (log n) por operacion, dos segment trees de mínimo

Uso: un arbol para $a[i]+i$ y otro para $a[i]-i$; la respuesta es el menor

Hay que minimizar $p[j] + |i - j|$ sobre todo j , con actualizaciones. El valor absoluto es lo que estorba, y el truco es partirlo en los dos casos y meter el indice DENTRO del valor guardado: si $j \leq i$: $p[j] + i - j = (p[j] - j) + i$ si $j > i$: $p[j] + j - i = (p[j] + j) - i$ Como i es fijo durante la consulta, minimizar cada caso es minimizar $(p[j] - j)$ a la izquierda y $(p[j] + j)$ a la derecha. O sea DOS segment trees de mínimos, uno sobre $p[j]-j$ y otro sobre $p[j]+j$, y la respuesta es el menor de los dos resultados sumandole o restandole i . CUANDO USAR ESTE TRUCO: cada vez que aparece un $|i - j|$ junto a algo que depende de j . Sale en problemas de "la tienda mas cercana con su costo", en DP con costos de distancia, y en cualquier minimizacion sobre una recta. La regla es partir en $j \leq i$ y $j > i$ y absorber el indice en el valor que se guarda; lo que quedaba dependiendo de i sale del mínimo como constante.

C++\CSES\Range Queries\Pizzeria Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct SegmentTree {
7     int n;
8     vector<ll> tree;
9
10    SegmentTree(vector<ll>& a) {
11        n = a.size() - 1;
12        tree.resize(4 * n);
13        build(1, 1, n, a);
14    }
15
16    void build(int node, int l, int r, vector<ll>& a) {
17        if (l == r) {
18            tree[node] = a[l];
19            return;
20        }

```

18.12 Pizzeria Queries - Truco de $p[i]$ mas y menos i (cont.)

```

21
22     int mid = (l + r) / 2;
23
24     build(node * 2, l, mid, a);
25     build(node * 2 + 1, mid + 1, r, a);
26
27     tree[node] = min(tree[node * 2], tree[node * 2 + 1]);
28 }
29
30 void update(int node, int l, int r, int pos, ll value) {
31     if (l == r) {
32         tree[node] = value;
33         return;
34     }
35
36     int mid = (l + r) / 2;
37
38     if (pos <= mid)
39         update(node * 2, l, mid, pos, value);
40     else
41         update(node * 2 + 1, mid + 1, r, pos, value);
42
43     tree[node] = min(tree[node * 2], tree[node * 2 + 1]);
44 }
45
46 ll query(int node, int l, int r, int ql, int qr) {
47     if (ql <= l && r <= qr)
48         return tree[node];
49
50     int mid = (l + r) / 2;
51
52     ll ans = LLONG_MAX;
53
54     if (ql <= mid)
55         ans = min(ans, query(node * 2, l, mid, ql, qr));
56
57     if (qr > mid)
58         ans = min(ans, query(node * 2 + 1, mid + 1, r, ql, qr));
59
60     return ans;
61 }
62
63 void update(int pos, ll value) {
64     update(1, 1, n, pos, value);
65 }
66
67 ll query(int l, int r) {
68     if (l > r)
69         return LLONG_MAX;
70
71     return query(1, 1, n, l, r);
72 }
73 };
74
75 int main() {
76     ios::sync_with_stdio(false);
77     cin.tie(nullptr);
78
79     int n, q;
80     cin >> n >> q;
81
82     vector<ll> p(n + 1);
83     vector<ll> a(n + 1), b(n + 1);
84
85     for (int i = 1; i <= n; i++) {
86         cin >> p[i];
87
88         a[i] = p[i] - i;
89         b[i] = p[i] + i;
90     }
91
92     SegmentTree leftTree(a);

```

18.12 Pizzeria Queries - Truco de p[i] mas y menos i (cont.)

```

93 SegmentTree rightTree(b);
94
95 while (q--) {
96     int type, k;
97     ll x;
98
99     cin >> type >> k;
100
101     if (type == 1) {
102         cin >> x;
103
104         p[k] = x;
105
106         leftTree.update(k, x - k);
107         rightTree.update(k, x + k);
108     }
109     else {
110         ll left = leftTree.query(1, k) + k;
111         ll right = rightTree.query(k, n) - k;
112
113         cout << min(left, right) << '\n';
114     }
115 }
116 }

```

18.13 Polynomial Queries - Lazy Propagation de Progresiones Aritmeticas

Lazy propagation donde la actualizacion no suma una constante sino 1, 2, 3, ... a lo largo del rango. La clave es que una progresion aritmetica se describe con DOS numeros, el primer termino A y la diferencia D, y que dos progresiones se SUMAN componente a componente. Por eso el lazy es el par (A, D), y componer dos actualizaciones pendientes es sumar los pares. Al empujar a los hijos, el izquierdo recibe (A, D) tal cual, pero el derecho arranca mas adelante en la progresion: recibe (A + lenLeft*D, D). Ese corrimiento es lo unico nuevo respecto a un lazy de suma normal. La suma de la progresion sobre un nodo de largo len es $\text{len} * (2A + (\text{len}-1)*D)/2$, y la division es exacta siempre: si len es par, listo; si es impar, $(\text{len}-1)*D$ es par. EL PATRON GENERAL: se puede hacer lazy de cualquier actualizacion que (1) se describa con pocos numeros, (2) se pueda componer con otra del mismo tipo, y (3) permita recalcular la suma del nodo sin bajar a las hojas. Las progresiones cumplen las tres; por eso sirve.

C++\CSES\Range Queries\Polynomial Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const int N = 200005;
7
8 ll st[4 * N];
9 ll lazyA[4 * N];
10 ll lazyD[4 * N];
11
12 void build(int p, int l, int r, vector<ll>& a) {
13     if (l == r) {
14         st[p] = a[l];
15         return;
16     }
17
18     int m = (l + r) / 2;
19
20     build(p * 2, l, m, a);
21     build(p * 2 + 1, m + 1, r, a);
22
23     st[p] = st[p * 2] + st[p * 2 + 1];
24 }
25
26 void apply(int p, int l, int r, ll A, ll D) {
27     ll len = r - l + 1;
28
29     st[p] += len * (2 * A + (len - 1) * D) / 2;
30
31     lazyA[p] += A;

```

18.13 Polynomial Queries - Lazy Propagation de Progresiones Aritmeticas (cont.)

```

32     lazyD[p] += D;
33 }
34
35 void push(int p, int l, int r) {
36     if (lazyA[p] == 0 && lazyD[p] == 0)
37         return;
38
39     int m = (l + r) / 2;
40     int lenLeft = m - l + 1;
41
42     apply(p * 2, l, m, lazyA[p], lazyD[p]);
43
44     apply(
45         p * 2 + 1,
46         m + 1,
47         r,
48         lazyA[p] + lenLeft * lazyD[p],
49         lazyD[p]
50     );
51
52     lazyA[p] = 0;
53     lazyD[p] = 0;
54 }
55
56 void update(int p, int l, int r, int ql, int qr) {
57     if (qr < l || r < ql)
58         return;
59
60     if (ql <= l && r <= qr) {
61         ll A = 1 - ql + 1;
62         apply(p, l, r, A, 1);
63         return;
64     }
65
66     push(p, l, r);
67
68     int m = (l + r) / 2;
69
70     update(p * 2, l, m, ql, qr);
71     update(p * 2 + 1, m + 1, r, ql, qr);
72
73     st[p] = st[p * 2] + st[p * 2 + 1];
74 }
75
76 ll query(int p, int l, int r, int ql, int qr) {
77     if (qr < l || r < ql)
78         return 0;
79
80     if (ql <= l && r <= qr)
81         return st[p];
82
83     push(p, l, r);
84
85     int m = (l + r) / 2;
86
87     return query(p * 2, l, m, ql, qr)
88         + query(p * 2 + 1, m + 1, r, ql, qr);
89 }
90
91 int main() {
92     ios::sync_with_stdio(false);
93     cin.tie(nullptr);
94
95     int n, q;
96     cin >> n >> q;
97
98     vector<ll> a(n + 1);
99
100     for (int i = 1; i <= n; i++)
101         cin >> a[i];
102
103     build(1, 1, n, a);

```

18.13 Polynomial Queries - Lazy Propagation de Progresiones Aritmeticas (cont.)

```

04
05 while (q--) {
06     int type, l, r;
07     cin >> type >> l >> r;
08
09     if (type == 1)
10         update(1, 1, n, l, r);
11     else
12         cout << query(1, 1, n, l, r) << '\n';
13 }
14 }

```

18.14 Prefix Sum Queries - Segment Tree con Nodo Compuesto

O: (log n) por operacion; el Node guarda suma y mejor prefijo

Uso: SegmentTree st(a); st.update(1,1,n,pos,v); st.query(1,1,n,l,r)

Segment tree donde cada nodo guarda DOS cosas: la suma del rango y el mejor prefijo del rango. Es el ejemplo mas chico de la tecnica que hace al segment tree util de verdad: si la respuesta sola no se puede combinar, se guarda tambien lo que haga falta para poder combinarla. EL MERGE ES TODO: el mejor prefijo de la union es o el mejor prefijo de la izquierda, o toda la izquierda mas el mejor prefijo de la derecha. max(a.pref, a.sum + b.pref). Sin llevar la suma no habria forma de escribir esa segunda opcion, y por eso van juntas. COMO SE DISENA UN NODO ASI: se escribe a mano como seria la respuesta de la union en funcion de las dos mitades, y cada cosa que aparece en esa formula y no se tiene, se agrega al nodo. Se repite hasta que cierre. Es un proceso mecanico, no hay que adivinar. CUANDO USAR: la pregunta del rango no es una operacion asociativa simple pero SI se puede componer con un poco mas de informacion. Maximo prefijo, maximo subarreglo, cantidad de parentesis sin cerrar, la subsecuencia creciente mas larga por bloques.

C++\CSES\Range Queries\Prefix Sum Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct Node {
7     ll sum, pref;
8 };
9
10 Node merge(Node a, Node b) {
11     return {
12         a.sum + b.sum,
13         max(a.pref, a.sum + b.pref)
14     };
15 }
16
17 struct SegmentTree {
18     int n;
19     vector<Node> tree;
20
21     SegmentTree(vector<ll>& a) {
22         n = a.size() - 1;
23         tree.resize(4 * n);
24         build(1, 1, n, a);
25     }
26
27     void build(int node, int l, int r, vector<ll>& a) {
28         if (l == r) {
29             tree[node] = {a[l], max(0LL, a[l])};
30             return;
31         }
32
33         int mid = (l + r) / 2;
34
35         build(node * 2, l, mid, a);
36         build(node * 2 + 1, mid + 1, r, a);
37
38         tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
39     }
40
41     void update(int node, int l, int r, int pos, ll value) {
42         if (l == r) {

```

18.14 Prefix Sum Queries - Segment Tree con Nodo Compuesto (cont.)

```

43         tree[node] = {value, max(0LL, value)};
44         return;
45     }
46
47     int mid = (l + r) / 2;
48
49     if (pos <= mid)
50         update(node * 2, l, mid, pos, value);
51     else
52         update(node * 2 + 1, mid + 1, r, pos, value);
53
54     tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
55 }
56
57 Node query(int node, int l, int r, int ql, int qr) {
58     if (ql <= l && r <= qr)
59         return tree[node];
60
61     int mid = (l + r) / 2;
62
63     if (qr <= mid)
64         return query(node * 2, l, mid, ql, qr);
65
66     if (ql > mid)
67         return query(node * 2 + 1, mid + 1, r, ql, qr);
68
69     return merge(
70         query(node * 2, l, mid, ql, qr),
71         query(node * 2 + 1, mid + 1, r, ql, qr)
72     );
73 }
74
75 void update(int pos, ll value) {
76     update(1, 1, n, pos, value);
77 }
78
79 Node query(int l, int r) {
80     return query(1, 1, n, l, r);
81 }
82 };
83
84 int main() {
85     ios::sync_with_stdio(false);
86     cin.tie(nullptr);
87
88     int n, q;
89     cin >> n >> q;
90
91     vector<ll> a(n + 1);
92
93     for (int i = 1; i <= n; i++)
94         cin >> a[i];
95
96     SegmentTree st(a);
97
98     while (q--) {
99         int type, a, b;
100         cin >> type >> a >> b;
101
102         if (type == 1) {
103             st.update(a, b);
104         } else {
105             cout << st.query(a, b).pref << '\n';
106         }
107     }
108 }

```

18.15 Range Interval Queries - Barrido Offline con Fenwick

O: $((n+q) \log n)$ con barrido de eventos

Uso: ordenar Event por posición y responder offline con el Fenwick

Contar cuantos elementos de $[l, r]$ tienen valor en $[a, b]$, sin actualizaciones. Es una consulta en DOS dimensiones (posición y valor), y la técnica es bajarla a una sola con un barrido offline. COMO: cada consulta se parte en dos eventos por la resta de prefijos sobre el VALOR, uno con signo + en b y otro con signo - en a-1. Todos los eventos y todos los elementos se ordenan por valor, y se barre de menor a mayor insertando en el Fenwick la POSICION de cada elemento que ya entro. Cuando el barrido llega a un evento, el Fenwick contiene exactamente los elementos con valor $\leq$ ese umbral, así que una consulta de rango de posiciones responde el evento. Ese es el patron general: una de las dos dimensiones se convierte en TIEMPO (el orden del barrido) y la otra se guarda en el Fenwick. CUANDO USAR: consultas de rango en dos dimensiones sin actualizaciones. Si hubiera actualizaciones seria una tercera dimension y tocaria BIT 2D o CDQ divide and conquer.

C++\CSES\Range Queries\Range Interval Queries.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Fenwick {
5     int n;
6     vector<int> bit;
7
8     Fenwick(int n) : n(n), bit(n + 1, 0) {}
9
10    void add(int i, int x) {
11        for (; i <= n; i += i & -i)
12            bit[i] += x;
13    }
14
15    int sum(int i) {
16        int res = 0;
17
18        for (; i > 0; i -= i & -i)
19            res += bit[i];
20
21        return res;
22    }
23 };
24
25 struct Event {
26     int value, pos, id, sign;
27
28     bool operator<(const Event& other) const {
29         return value < other.value;
30     }
31 };
32
33 int main() {
34     ios::sync_with_stdio(false);
35     cin.tie(nullptr);
36
37     int n, q;
38     cin >> n >> q;
39
40     vector<pair<int, int>> a(n);
41
42     for (int i = 1; i <= n; i++) {
43         int x;
44         cin >> x;
45         a[i - 1] = {x, i};
46     }
47
48     sort(a.begin(), a.end());
49
50     vector<Event> events;
51
52     for (int i = 0; i < q; i++) {
53         int l, r, c, d;
54         cin >> l >> r >> c >> d;
55
56         events.push_back({d, r, i, 1});
```

18.15 Range Interval Queries - Barrido Offline con Fenwick (cont.)

```
57     events.push_back({d, l - 1, i, -1});
58     events.push_back({c - 1, r, i, -1});
59     events.push_back({c - 1, l - 1, i, 1});
60 }
61
62 sort(events.begin(), events.end());
63
64 Fenwick fw(n);
65 vector<int> ans(q);
66
67 int p = 0;
68
69 for (auto e : events) {
70
71     while (p < n && a[p].first <= e.value) {
72         fw.add(a[p].second, 1);
73         p++;
74     }
75
76     ans[e.id] += e.sign * fw.sum(e.pos);
77 }
78
79 for (int x : ans)
80     cout << x << '\n';
81 }
```

18.16 Range Queries And Copies - Segment Tree Persistente

O: $(\log n)$ por operacion, $(n \log n)$ de memoria

Uso: $raiz[k] = \text{update}(raiz[j], 1, n, pos, x)$; $query(raiz[k], 1, n, l, r)$ // persistente

Segment tree PERSISTENTE: cada actualizacion no modifica nodos, crea copias nuevas SOLO del camino de la raiz a la hoja, $O(\log n)$ nodos, y todo lo demas se comparte con la version anterior. Así cada version tiene su propia raiz y se puede consultar cualquiera. LO QUE HACE ESTE PROBLEMA ELEGANTE: copiar un arreglo entero (la operacion 3) cuesta $O(1)$. Basta copiar el PUNTERO a la raiz, porque las dos versiones comparten todos los nodos hasta que alguna se modifique, y en ese momento la modificacion crea caminos nuevos sin tocar la otra. Los nodos no pueden ir en el esquema $2p / 2p+1$ de siempre, porque un nodo tiene varios padres segun la version. Por eso cada nodo guarda los INDICES de sus hijos (l, r) y se sacan de un arreglo grande con un contador. EL TAMANO DEL POOL: el build usa $2n$ nodos y cada actualizacion unos $\log_2(n)+1 = 19$. Con $2 \cdot 10^5$ actualizaciones son ~4.2 millones, y el arreglo reserva 25 por posicion (5 millones). Quedarse corto aqui da un error silencioso al pisar memoria. Ocupa unos 80 MB. CUANDO USAR: "consultar el arreglo como estaba en el momento k", versiones, o la k-esima estadistica en un rango (el clasico con persistencia sobre prefijos).

C++\CSES\Range Queries\Range Queries and Copies.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct Node {
7     int l, r;
8     ll sum;
9 };
10
11 const int MAXN = 200005;
12 const int MAXNODE = MAXN * 25;
13
14 Node st[MAXNODE];
15 int root[MAXN];
16 int nodes = 0;
17
18 int build(int l, int r, vector<ll>& a) {
19     int p = ++nodes;
20
21     if (l == r) {
22         st[p].sum = a[l];
23         return p;
24     }
25
26     int m = (l + r) / 2;
```

18.16 Range Queries And Copies - Segment Tree Persistente (cont.)

```

28 st[p].l = build(1, m, a);
29 st[p].r = build(m + 1, r, a);
30
31 st[p].sum = st[st[p].l].sum + st[st[p].r].sum;
32
33 return p;
34 }
35
36 int update(int old, int l, int r, int pos, ll x) {
37     int p = ++nodes;
38     st[p] = st[old];
39
40     if (l == r) {
41         st[p].sum = x;
42         return p;
43     }
44
45     int m = (l + r) / 2;
46
47     if (pos <= m)
48         st[p].l = update(st[old].l, l, m, pos, x);
49     else
50         st[p].r = update(st[old].r, m + 1, r, pos, x);
51
52     st[p].sum = st[st[p].l].sum + st[st[p].r].sum;
53
54     return p;
55 }
56
57 ll query(int p, int l, int r, int ql, int qr) {
58     if (qr < l || r < ql)
59         return 0;
60
61     if (ql <= l && r <= qr)
62         return st[p].sum;
63
64     int m = (l + r) / 2;
65
66     return query(st[p].l, l, m, ql, qr)
67         + query(st[p].r, m + 1, r, ql, qr);
68 }
69
70 int main() {
71     ios::sync_with_stdio(false);
72     cin.tie(nullptr);
73
74     int n, q;
75     cin >> n >> q;
76
77     vector<ll> a(n + 1);
78
79     for (int i = 1; i <= n; i++)
80         cin >> a[i];
81
82     root[1] = build(1, n, a);
83
84     int versions = 1;
85
86     while (q--) {
87         int type;
88         cin >> type;
89
90         if (type == 1) {
91             int k, a, x;
92             cin >> k >> a >> x;
93
94             root[k] = update(root[k], 1, n, a, x);
95         }
96         else if (type == 2) {
97             int k, a, b;
98             cin >> k >> a >> b;
99

```

18.16 Range Queries And Copies - Segment Tree Persistente (cont.)

```

00     cout << query(root[k], 1, n, a, b) << '\n';
01 }
02 else {
03     int k;
04     cin >> k;
05
06     root[++versions] = root[k];
07 }
08 }
09 }

```

18.17 Range Update Queries - Fenwick de Diferencias

O: (log n) por operacion

Uso: add(1,x) y add(r+1,-x); sum(i) da el valor de la posicion i

La vuelta de tuerca: en vez de guardar los valores, el Fenwick guarda las DIFERENCIAS. Sumar x a todo el rango [a,b] son dos operaciones puntuales, add(a, x) y add(b+1, -x), y consultar la posicion k es la suma de prefijo hasta k. O sea que el mismo Fenwick que hace "actualizacion puntual + consulta de rango" tambien hace lo contrario, "actualizacion de rango + consulta puntual", solo cambiando lo que se guarda. CUANDO USAR: el problema actualiza rangos completos y pregunta por posiciones sueltas. Si necesita actualizar rangos Y consultar rangos, ahi si toca el Fenwick doble o un segment tree con lazy propagation.

C++\CSES\Range Queries\Range Update Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct Fenwick {
7     int n;
8     vector<ll> bit;
9
10    Fenwick(int n) : n(n), bit(n + 1, 0) {}
11
12    void add(int i, ll x) {
13        for (; i <= n; i += i & -i)
14            bit[i] += x;
15    }
16
17    ll sum(int i) {
18        ll res = 0;
19        for (; i > 0; i -= i & -i)
20            res += bit[i];
21        return res;
22    }
23 };
24
25 int main() {
26     ios::sync_with_stdio(false);
27     cin.tie(nullptr);
28
29     int n, q;
30     cin >> n >> q;
31
32     vector<ll> arr(n + 1);
33
34     for (int i = 1; i <= n; i++)
35         cin >> arr[i];
36
37     Fenwick fw(n + 1);
38
39     while (q--) {
40         int type;
41         cin >> type;
42
43         if (type == 1) {
44             int a, b;
45             ll c;
46             cin >> a >> b >> c;
47
48             fw.add(a, c);

```

18.17 Range Update Queries - Fenwick de Diferencias (cont.)

```

48         fw.add(b + 1, -c);
49     } else {
50         int k;
51         cin >> k;
52
53         cout << arr[k] + fw.sum(k) << '\n';
54     }
55 }
56 }
57 }

```

18.18 Range Updates And Sums - Lazy Propagation con Suma y Asignacion

Lazy propagation con DOS tipos de actualizacion (sumar x al rango y ASIGNAR x al rango), y todo el problema es como se componen cuando se encuentran en un mismo nodo. Se mantiene el invariante "lo pendiente es: primero asignar lazySet (si hasSet), despues sumar lazyAdd": - Una ASIGNACION borra cualquier suma pendiente: applySet pone lazyAdd en 0. La asignacion pisa todo lo anterior. - Una SUMA sobre un nodo que ya tiene una asignacion pendiente no se guarda como suma: se le suma al valor asignado (lazySet += x). Asignar S y despues sumar x es asignar S + x. - Al empujar, se baja primero la asignacion y despues la suma, en ese orden. Equivocarse en cualquiera de las tres da respuestas que parecen bien en casos chicos y fallan cuando las dos operaciones se mezclan sobre el mismo rango. Se usa hasSet aparte en vez de un valor centinela porque asignar 0 es una operacion valida. EL PATRON GENERAL: con varios tipos de lazy, antes de programar se escribe la tabla de "si llega B encima de A, que queda". Si esa tabla cierra, el lazy funciona.

C++\CSES\Range Queries\Range Updates and Sums.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const int N = 200005;
7
8 ll st[4 * N];
9 ll lazyAdd[4 * N];
10 ll lazySet[4 * N];
11 bool hasSet[4 * N];
12
13 void build(int p, int l, int r, vector<ll>& a) {
14     if (l == r) {
15         st[p] = a[l];
16         return;
17     }
18
19     int m = (l + r) / 2;
20
21     build(p * 2, l, m, a);
22     build(p * 2 + 1, m + 1, r, a);
23
24     st[p] = st[p * 2] + st[p * 2 + 1];
25 }
26
27 void applySet(int p, int l, int r, ll x) {
28     st[p] = x * (r - l + 1);
29
30     lazySet[p] = x;
31     lazyAdd[p] = 0;
32     hasSet[p] = true;
33 }
34
35 void applyAdd(int p, int l, int r, ll x) {
36     st[p] += x * (r - l + 1);
37
38     if (hasSet[p])
39         lazySet[p] += x;
40     else
41         lazyAdd[p] += x;
42 }
43
44 void push(int p, int l, int r) {
45     if (l == r)
46         return;

```

18.18 Range Updates And Sums - Lazy Propagation con Suma y Asignacion (cont.)

```

47     int m = (l + r) / 2;
48
49     if (hasSet[p]) {
50         applySet(p * 2, l, m, lazySet[p]);
51         applySet(p * 2 + 1, m + 1, r, lazySet[p]);
52
53         hasSet[p] = false;
54     }
55
56     if (lazyAdd[p] != 0) {
57         applyAdd(p * 2, l, m, lazyAdd[p]);
58         applyAdd(p * 2 + 1, m + 1, r, lazyAdd[p]);
59
60         lazyAdd[p] = 0;
61     }
62 }
63
64 void updateAdd(int p, int l, int r, int ql, int qr, ll x) {
65     if (qr < l || r < ql)
66         return;
67
68     if (ql <= l && r <= qr) {
69         applyAdd(p, l, r, x);
70         return;
71     }
72
73     push(p, l, r);
74
75     int m = (l + r) / 2;
76
77     updateAdd(p * 2, l, m, ql, qr, x);
78     updateAdd(p * 2 + 1, m + 1, r, ql, qr, x);
79
80     st[p] = st[p * 2] + st[p * 2 + 1];
81 }
82
83 void updateSet(int p, int l, int r, int ql, int qr, ll x) {
84     if (qr < l || r < ql)
85         return;
86
87     if (ql <= l && r <= qr) {
88         applySet(p, l, r, x);
89         return;
90     }
91
92     push(p, l, r);
93
94     int m = (l + r) / 2;
95
96     updateSet(p * 2, l, m, ql, qr, x);
97     updateSet(p * 2 + 1, m + 1, r, ql, qr, x);
98
99     st[p] = st[p * 2] + st[p * 2 + 1];
100 }
101
102 ll query(int p, int l, int r, int ql, int qr) {
103     if (qr < l || r < ql)
104         return 0;
105
106     if (ql <= l && r <= qr)
107         return st[p];
108
109     push(p, l, r);
110
111     int m = (l + r) / 2;
112
113     return query(p * 2, l, m, ql, qr)
114         + query(p * 2 + 1, m + 1, r, ql, qr);
115 }
116
117 int main() {

```

18.18 Range Updates And Sums - Lazy Propagation con Suma y Asignacion (cont.)

```
18 ios::sync_with_stdio(false);
20 cin.tie(nullptr);
21
22 int n, q;
23 cin >> n >> q;
24
25 vector<ll> a(n + 1);
26
27 for (int i = 1; i <= n; i++)
28     cin >> a[i];
29
30 build(1, 1, n, a);
31
32 while (q--) {
33     int type, l, r;
34     cin >> type >> l >> r;
35
36     if (type == 1) {
37         ll x;
38         cin >> x;
39         updateAdd(1, 1, n, l, r, x);
40     }
41     else if (type == 2) {
42         ll x;
43         cin >> x;
44         updateSet(1, 1, n, l, r, x);
45     }
46     else {
47         cout << query(1, 1, n, l, r) << '\n';
48     }
49 }
50 }
```

18.19 Range XOR Queries - Prefix XOR

Idem prefix sums pero con xor, y funciona por la misma razon: el xor es su propia inversa, asi que $\text{pref}[r] \oplus \text{pref}[l-1]$ cancela todo lo de antes de l. Los elementos repetidos se anulan de a pares. CUANDO USAR: ver que el xor tiene inversa es lo que abre la puerta. Mismo patron para cualquier grupo abeliano: suma, xor, producto modular con inversos. Para min o max NO aplica, porque no hay nada que "deshaga" un minimo.

C++\CSES\Range Queries\Range XOR Queries.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n, q;
9     cin >> n >> q;
10
11     vector<long long> pref(n + 1);
12
13     for (int i = 1; i <= n; i++) {
14         long long x;
15         cin >> x;
16         pref[i] = pref[i - 1] ^ x;
17     }
18
19     while (q--) {
20         int l, r;
21         cin >> l >> r;
22         cout << (pref[r] ^ pref[l - 1]) << '\n';
23     }
24 }
```

18.20 Salary Queries - Fenwick con Compresion Offline

O: $(\log n)$ por operacion, sobre valores comprimidos

Uso: $\text{add}(\text{comp}(v), +1); \text{sum}(\text{comp}(hi)) - \text{sum}(\text{comp}(lo) - 1)$ // 1-INDEXADO

Fenwick sobre VALORES en vez de posiciones, para contar cuantos salarios caen en un rango. Como los valores llegan a 10^9 hay que comprimirlos, y aqui esta el detalle que hace el problema: las consultas se leen TODAS primero, porque los valores que van a aparecer en las actualizaciones futuras tambien tienen que entrar en la compresion. Si uno comprime solo con los valores iniciales, la primera actualizacion a un valor nuevo no tiene indice donde ir. Eso obliga a que el problema sea OFFLINE: leer todo, comprimir, y despues procesar en orden. El Fenwick guarda la CANTIDAD de empleados con cada valor comprimido, asi que cambiar un salario es $\text{add}(\text{viejo}, -1)$ y $\text{add}(\text{nuevo}, +1)$, y contar en un rango es la resta de dos prefijos. CUANDO USAR: contar elementos en un rango de VALORES con actualizaciones. Es el patron de "cuantos hay menores que x" con cambios. La alternativa online es un segment tree dinamico o sobre los valores sin comprimir, que gasta mas memoria pero no obliga a leer todo antes. OJO: usa structured bindings (auto [a, b]), que piden C++17. En CSES compila, pero con un g++ viejo hay que volver a .first y .second.

C++\CSES\Range Queries\Salary Queries.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Fenwick {
5     int n;
6     vector<int> bit;
7
8     Fenwick(int n) : n(n), bit(n + 1, 0) {}
9
10    void add(int i, int x) {
11        for (; i <= n; i += i & -i)
12            bit[i] += x;
13    }
14
15    int sum(int i) {
16        int res = 0;
17        for (; i > 0; i -= i & -i)
18            res += bit[i];
19        return res;
20    }
21 };
22
23 int main() {
24     ios::sync_with_stdio(false);
25     cin.tie(nullptr);
26
27     int n, q;
28     cin >> n >> q;
29
30     vector<int> salary(n + 1);
31     vector<tuple<char, int, int>> queries;
32     vector<int> values;
33
34     for (int i = 1; i <= n; i++) {
35         cin >> salary[i];
36         values.push_back(salary[i]);
37     }
38
39     for (int i = 0; i < q; i++) {
40         char type;
41         int a, b;
42         cin >> type >> a >> b;
43
44         queries.push_back({type, a, b});
45
46         if (type == '!')
47             values.push_back(b);
48     }
49
50     sort(values.begin(), values.end());
51     values.erase(unique(values.begin(), values.end()), values.end());
52
53     Fenwick fw(values.size());
```

18.20 Salary Queries - Fenwick con Compresion Offline (cont.)

```

54
55     for (int i = 1; i <= n; i++) {
56         int pos = lower_bound(values.begin(), values.end(), salary[i])
57             - values.begin() + 1;
58         fw.add(pos, 1);
59     }
60 }
61
62 for (auto [type, a, b] : queries) {
63
64     if (type == '!') {
65         int oldPos = lower_bound(values.begin(), values.end(), salary[a])
66             - values.begin() + 1;
67
68         int newPos = lower_bound(values.begin(), values.end(), b)
69             - values.begin() + 1;
70
71         fw.add(oldPos, -1);
72         fw.add(newPos, 1);
73
74         salary[a] = b;
75     }
76
77     else {
78         int right = upper_bound(values.begin(), values.end(), b)
79             - values.begin();
80
81         int left = lower_bound(values.begin(), values.end(), a)
82             - values.begin();
83
84         cout << fw.sum(right) - fw.sum(left) << '\n';
85     }
86 }
87 }
```

18.21 Static Range Minimum Queries - Sparse Table

Sparse table: $st[k][i]$ es el minimo de los 2^k elementos que arrancan en i . Se construye en $O(n \log n)$ partiendo cada rango en dos mitades de 2^{k-1} , y cada consulta se responde en $O(1)$ cubriendo $[l, r]$ con DOS bloques de tamano 2^k que se solapan. Solaparse no es problema, y esa es la clave: el minimo es IDEMPOTENTE, contar un elemento dos veces no cambia nada. CUANDO USAR: arreglo estatico y operacion idempotente (min, max, gcd, and, or). Consulta $O(1)$, mas rapido que cualquier segment tree. CUANDO NO: si el arreglo cambia, la sparse table no se puede actualizar y toca segment tree. Y para SUMA no sirve, porque solapar los dos bloques contaria de mas.

C++\CSES\Range Queries\Static Range Minimum Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    int n, q;
11    cin >> n >> q;
12
13    // K = cantidad de niveles que necesitamos.
14    // st[k][i] = minimo de 2^k elementos empezando en i.
15    int K = 1;
16
17    while ((1 << K) <= n)
18        K++;
19
20    vector<vector<ll>> st(K, vector<ll>(n + 1));
21
22    // Nivel 0: rangos de tamano 1.
23    for (int i = 1; i <= n; i++)
24        cin >> st[0][i];
25}
```

18.21 Static Range Minimum Queries - Sparse Table (cont.)

```

26 // Construimos los demas niveles.
27 for (int k = 1; k < K; k++) {
28
29     // Tamano del rango = 2^k
30     int len = 1 << k;
31
32     for (int i = 1; i + len - 1 <= n; i++) {
33
34         // Dividimos el rango en dos partes
35         // de tamano 2^(k-1).
36         st[k][i] = min(
37             st[k - 1][i],
38             st[k - 1][i + (len / 2)]
39         );
40     }
41 }
42
43 while (q--) {
44     int a, b;
45     cin >> a >> b;
46
47     int len = b - a + 1;
48
49     // Mayor potencia de 2 que cabe en len.
50     int k = 0;
51
52     while ((1 << (k + 1)) <= len)
53         k++;
54
55     // Primer bloque de tamano 2^k
56     ll x = st[k][a];
57
58     // Segundo bloque de tamano 2^k,
59     // empezando desde el final.
60     ll y = st[k][b - (1 << k) + 1];
61
62     cout << min(x, y) << '\n';
63 }
64 }
```

18.22 Static Range Sum Queries - Prefix Sums

La estructura mas barata que existe: $pref[i]$ = suma de los primeros i , y la suma de $[a, b]$ sale como $pref[b] - pref[a - 1]$. Construcion $O(n)$, consulta $O(1)$, y ni una linea de estructura. CUANDO USAR: el arreglo NO cambia y la operacion tiene INVERSA (suma, xor, producto sin ceros). Eso es todo. Si hay actualizaciones se necesita Fenwick; si la operacion no tiene inversa (min, max, gcd) no se puede restar y toca sparse table o segment tree. Lo del indice 1 no es capricho: con $pref$ de tamano $n + 1$ y $pref[0] = 0$ desaparece el if del borde cuando $a = 1$. Y long long, porque n valores de 10^9 se pasan de int sin avisar.

C++\CSES\Range Queries\Static Range Sum Queries.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 int main() {
7     ios::sync_with_stdio(false);
8     cin.tie(nullptr);
9
10    ll n, q;
11    cin >> n >> q;
12
13    vector<ll> pref(n + 1);
14
15    for (int i = 1; i <= n; i++) {
16        ll x;
17        cin >> x;
18        pref[i] = pref[i - 1] + x;
19    }
20
21    while (q--) {
```

18.22 Static Range Sum Queries - Prefix Sums (cont.)

```
22     ll a, b;
23     cin >> a >> b;
24
25     cout << pref[b] - pref[a - 1] << '\n';
26 }
27 }
```

18.23 Sub Array Sum Queries - Segment Tree de Maximo Subarreglo

O: (log n) por operacion

Uso: Node = {sum, pref, suff, best}; merge cruza suff izq con pref der

El nodo compuesto clasico, con cuatro campos: sum, pref (mejor prefijo), suff (mejor sufijo) y best (el mejor subarreglo de adentro). Es Kadane metido en un segment tree, y por eso aguanta actualizaciones que Kadane suelto no aguanta. El MERGE, que es lo unico que hay que entender: sum = L.sum + R.sum pref = max(L.pref, L.sum + R.pref) el prefijo se queda o cruza suff = max(R.suff, R.sum + L.suff) el sufijo, simetrico best = max(L.best, R.best, L.suff + R.pref) o esta en una mitad, o CRUZA el corte Ese tercer caso del best es la razon de llevar suff y pref: el subarreglo que cruza es el mejor sufijo de la izquierda pegado al mejor prefijo de la derecha. El max(0, a[i]) en las hojas permite el subarreglo VACIO, o sea que la respuesta nunca es negativa. Si el problema exige al menos un elemento hay que sacar ese max y poner la hoja en a[i] pelado. CUANDO USAR: maximo subarreglo con actualizaciones. Sin actualizaciones basta Kadane en O(n).

C++\CSES\Range Queries\SubArray Sum Queries.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct Node {
7     ll sum, pref, suff, best;
8 };
9
10 Node merge(Node L, Node R) {
11     Node res;
12
13     res.sum = L.sum + R.sum;
14
15     res.pref = max(L.pref, L.sum + R.pref);
16
17     res.suff = max(R.suff, R.sum + L.suff);
18
19     res.best = max({
20         L.best,
21         R.best,
22         L.suff + R.pref
23     });
24
25     return res;
26 }
27
28 struct SegmentTree {
29     int n;
30     vector<Node> tree;
31
32     SegmentTree(vector<ll>& a) {
33         n = a.size() - 1;
34         tree.resize(4 * n);
35         build(1, 1, n, a);
36     }
37
38     void build(int node, int l, int r, vector<ll>& a) {
39         if (l == r) {
40             tree[node] = {
41                 a[l],
42                 max(0LL, a[l]),
43                 max(0LL, a[l]),
44                 max(0LL, a[l])
45             };
46             return;
47         }
48     }
```

18.23 Sub Array Sum Queries - Segment Tree de Maximo Subarreglo (cont.)

```
48     int mid = (l + r) / 2;
49
50     build(node * 2, l, mid, a);
51     build(node * 2 + 1, mid + 1, r, a);
52
53     tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
54 }
55
56 void update(int node, int l, int r, int pos, ll value) {
57     if (l == r) {
58         tree[node] = {
59             value,
60             max(0LL, value),
61             max(0LL, value),
62             max(0LL, value)
63         };
64         return;
65     }
66
67     int mid = (l + r) / 2;
68
69     if (pos <= mid)
70         update(node * 2, l, mid, pos, value);
71     else
72         update(node * 2 + 1, mid + 1, r, pos, value);
73
74     tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
75 }
76
77 void update(int pos, ll value) {
78     update(1, 1, n, pos, value);
79 }
80
81 ll answer() {
82     return tree[1].best;
83 }
84
85 };
86
87 int main() {
88     ios::sync_with_stdio(false);
89     cin.tie(nullptr);
90
91     int n, m;
92     cin >> n >> m;
93
94     vector<ll> a(n + 1);
95
96     for (int i = 1; i <= n; i++)
97         cin >> a[i];
98
99     SegmentTree st(a);
100
101     while (m--) {
102         int k;
103         ll x;
104
105         cin >> k >> x;
106
107         st.update(k, x);
108
109         cout << st.answer() << '\n';
110     }
111 }
```

18.24 Sub Array Sum Queries II - Maximo Subarreglo en un Rango

O: $(\log n)$ por operacion

Uso: mismo Node de la I; aqui el merge se usa tambien en la consulta

El MISMO nodo de cuatro campos que "SubArray Sum Queries", con el merge identico, pero respondiendo otra pregunta: alla se actualiza una posicion y se pide el mejor subarreglo de TODO el arreglo (se lee la raiz), y aqui no hay actualizaciones y se pide el mejor subarreglo DENTRO de un rango $[a, b]$ (se combinan los nodos que cubren el rango). POR ESO VALE LA PENA TENER LOS DOS JUNTOS: es la misma estructura sirviendo para dos problemas distintos, y lo unico que cambia es de donde se lee la respuesta. Si el merge esta bien escrito, que la consulta sea global o de rango no le importa a la estructura. La leccion practica: cuando se disena un nodo compuesto, conviene escribir la consulta de rango aunque el problema solo pida la raiz. Cuesta cinco lineas mas y cubre las dos variantes.

C++\CSES\Range Queries\SubArray Sum Queries II.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct Node {
7     ll sum, pref, suff, best;
8 };
9
10 Node merge(Node L, Node R) {
11     return {
12         L.sum + R.sum,
13         max(L.pref, L.sum + R.pref),
14         max(R.suff, R.sum + L.suff),
15         max({
16             L.best,
17             R.best,
18             L.suff + R.pref
19         })
20     };
21 }
22
23 struct SegmentTree {
24     int n;
25     vector<Node> tree;
26
27     SegmentTree(vector<ll>& a) {
28         n = a.size() - 1;
29         tree.resize(4 * n);
30         build(1, 1, n, a);
31     }
32
33     void build(int node, int l, int r, vector<ll>& a) {
34         if (l == r) {
35             tree[node] = {
36                 a[l],
37                 max(0LL, a[l]),
38                 max(0LL, a[l]),
39                 max(0LL, a[l])
40             };
41             return;
42         }
43
44         int mid = (l + r) / 2;
45
46         build(node * 2, l, mid, a);
47         build(node * 2 + 1, mid + 1, r, a);
48
49         tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
50     }
51
52     Node query(int node, int l, int r, int ql, int qr) {
53         if (ql <= l && r <= qr)
54             return tree[node];
55
56         int mid = (l + r) / 2;
```

18.24 Sub Array Sum Queries II - Maximo Subarreglo en un Rango (cont.)

```
58     if (qr <= mid)
59         return query(node * 2, l, mid, ql, qr);
60
61     if (ql > mid)
62         return query(node * 2 + 1, mid + 1, r, ql, qr);
63
64     Node left = query(node * 2, l, mid, ql, qr);
65     Node right = query(node * 2 + 1, mid + 1, r, ql, qr);
66
67     return merge(left, right);
68 }
69
70 Node query(int l, int r) {
71     return query(1, 1, n, l, r);
72 }
73 };
74
75 int main() {
76     ios::sync_with_stdio(false);
77     cin.tie(nullptr);
78
79     int n, q;
80     cin >> n >> q;
81
82     vector<ll> a(n + 1);
83
84     for (int i = 1; i <= n; i++)
85         cin >> a[i];
86
87     SegmentTree st(a);
88
89     while (q--) {
90         int a, b;
91         cin >> a >> b;
92
93         cout << st.query(a, b).best << '\n';
94     }
95 }
```

18.25 Visible Building Queries - Monotonic Stack + Binary Lifting

Dos tecnicas encadenadas, y la combinacion es lo que vale. Primero una PILA MONOTONA de derecha a izquierda calcula $\text{nxt}[i]$, el siguiente edificio mas alto que i . Eso deja un arbol implicito donde el padre de i es $\text{nxt}[i]$: desde i , los edificios que se ven son i , $\text{nxt}[i]$, $\text{nxt}[\text{nxt}[i]]$... Y como hay que contar cuantos de esos caben antes de pasarse de r , se le monta BINARY LIFTING encima: $\text{up}[j][i]$ es el resultado de saltar 2^j veces, asi que contar los saltos se hace en $O(\log n)$ bajando de la potencia mas alta a la mas baja, en vez de caminar uno por uno. CUANDO USAR CADA UNA: la pila monotona para "el siguiente mayor o menor" en $O(n)$, que es la base de los histogramas y de los rangos donde un elemento es el maximo. El binary lifting cuando hay que seguir una cadena de punteros MUCHAS veces y cada salto es fijo: el mismo mecanismo del LCA. LA IDEA QUE SE REPITE: si un puntero define una funcion $i \rightarrow f(i)$, entonces las potencias de esa funcion se precaculan con doubling y cualquier cantidad de saltos sale en $O(\log n)$. Sirve para LCA, para funciones de permutacion y para los "k pasos adelante" de cualquier grafo funcional.

C++\CSES\Range Queries\Visible Building Queries.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int n, q;
9     cin >> n >> q;
10
11     vector<int> h(n + 1);
12
13     for (int i = 1; i <= n; i++)
14         cin >> h[i];
15
16     vector<int> nxt(n + 1, 0);
```

18.25 Visible Building Queries - Monotonic Stack + Binary Lifting (cont.)

```

17 stack<int> st;
18
19 for (int i = n; i >= 1; i--) {
20     while (!st.empty() && h[st.top()] <= h[i])
21         st.pop();
22
23     if (!st.empty())
24         nxt[i] = st.top();
25
26     st.push(i);
27 }
28
29 int LOG = 18;
30 vector<vector<int>> up(LOG, vector<int>(n + 1));
31
32 for (int i = 1; i <= n; i++)
33     up[0][i] = nxt[i];
34
35 for (int j = 1; j < LOG; j++) {
36     for (int i = 1; i <= n; i++) {
37         if (up[j - 1][i])
38             up[j][i] = up[j - 1][up[j - 1][i]];
39     }
40 }
41
42 while (q--) {
43     int a, b;
44     cin >> a >> b;
45
46     int pos = a;
47     int ans = 1;
48
49     for (int j = LOG - 1; j >= 0; j--) {
50         int x = up[j][pos];
51
52         if (x != 0 && x <= b) {
53             pos = x;
54             ans += (1 << j);
55         }
56     }
57
58     cout << ans << '\n';
59 }
60 }

```

19 - CSES: String Algorithms

19.1 All Palindromes - Manacher (Palindromo Mas Largo que Termina en Cada Posicion)

Para cada posicion, el palindromo mas largo que TERMINA ahi. Primero Manacher con centinelas distintos en los extremos ('\$' al inicio, '^' al final), que es el truco para que el while de expansion no necesite chequear limites: al llegar a los bordes los centinelas nunca coinciden. EL BARRIDO QUE LO RESUELVE: si un palindromo centrado en i llega hasta j o mas, entonces tambien hay un palindromo centrado en i que termina EXACTAMENTE en j (se recorta simetrico). Y entre dos centros que alcanzan j , el de mas a la izquierda da el palindromo mas largo. Asi que para cada j basta el PRIMER centro, de izquierda a derecha, que lo alcanza. Por eso el puntero j nunca retrocede: cada centro asigna las posiciones nuevas que alcanza y que nadie antes alcanzo. Cada posicion se asigna una vez por pasada, y todo queda $O(n)$. Se hace una pasada para impares y otra para pares, quedandose con el maximo.

C++\CSES\String Algorithms\All Palindromes.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> manacher(string s) {
5     string t = "$";
6
7     for (char c : s) {
8         t += '#';
9         t += c;
10    }

```

19.1 All Palindromes - Manacher (Palindromo Mas Largo que Termina en Cada Posicion) (cont.)

```

11
12    t += "#^";
13
14    vector<int> p(t.size());
15
16    int l = 1;
17    int r = 1;
18
19    for (int i = 1; i < (int)t.size() - 1; i++) {
20
21        p[i] = max(0, min(r - i, p[l + r - i]));
22
23        while (t[i - p[i]] == t[i + p[i]])
24            p[i]++;
25
26        if (i + p[i] > r) {
27            l = i - p[i];
28            r = i + p[i];
29        }
30    }
31
32    return p;
33 }
34
35 int main() {
36     ios::sync_with_stdio(false);
37     cin.tie(nullptr);
38
39     string s;
40     cin >> s;
41
42     vector<int> p = manacher(s);
43
44     int n = s.size();
45
46     vector<int> ans(n);
47
48     /*
49     PALINDROMOS IMPARES
50
51     i representa el centro en la cadena original.
52
53     p[(i + 1) * 2] / 2
54     nos dice cuanto puede extenderse el palindromo.
55
56     Para cada centro:
57
58     i, i+1, i+2, ...
59
60     podemos obtener palindromos de longitudes:
61
62     1, 3, 5, 7, ...
63
64     Pero usamos j para no volver a procesar posiciones
65     que ya fueron actualizadas.
66 */
67
68     int j = 0;
69
70     for (int i = 0; i < n; i++) {
71
72         j = max(j, i);
73
74         int limit = i + (p[(i + 1) * 2] / 2);
75
76         for (; j < limit; j++) {
77             ans[j] = 1 + 2 * (j - i);
78         }
79     }
80
81     /*
82     PALINDROMOS PARES

```

19.1 All Palindromes - Manacher (Palindromo Mas Largo que Termina en Cada Posicion) (cont.)

```

83
84     Ahora hacemos lo mismo para palindromos pares.
85
86     El centro esta entre i-1 e i.
87     */
88
89     j = 0;
90
91     for (int i = 1; i < n; i++) {
92
93         j = max(j, i);
94
95         int limit = i + (p[1 + i * 2] / 2);
96
97         for (; j < limit; j++) {
98             ans[j] = max(ans[j], 2 * (j - i + 1));
99         }
100     }
101
102     for (int x : ans)
103         cout << x << ' ';
104
105     cout << '\n';
106 }

```

19.2 Counting Patterns - Aho-Corasick (Cuento de Ocurrencias)

O: (suma de patrones + |s| + 26*nodos)

Uso: ac.addString(p) por patron; build(); search(s); propagate(); getCount(p)

Cuántas veces aparece cada patron dentro del texto, con TODOS los patrones a la vez. Aho-Corasick es el trie de los patrones mas un suffix link por nodo, que apunta al nodo del sufijo propio mas largo que tambien es prefijo de algun patron. Es la generalizacion del fallo de KMP a varios patrones a la vez. build() hace DOS cosas en el mismo BFS y por eso queda tan corto: calcula los suffix links y ademas convierte el trie en AUTOMATA, rellenando las transiciones que no existen con las del suffix link. Despues de eso next[u][c] nunca es -1 y recorrer el texto es un for sin whiles. search() camina el texto y marca +1 en cada nodo por el que pasa. propagate() recorre el orden BFS AL REVES sumando cada nodo a su suffix link: asi cada patron recibe tambien lo que conto cualquier patron mas largo que lo contiene como sufijo. Ese paso es el que suele faltar. CUANDO USAR: varios patrones contra un texto. Con un solo patron sobra KMP.

C++\CSES\String Algorithms\Counting Patterns.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct AhoCorasick
5 {
6     struct Node
7     {
8         int next[26];
9         int link;
10        int cnt;
11
12        Node()
13        {
14            fill(next, next + 26, -1);
15            link = 0;
16            cnt = 0;
17        }
18    };
19
20    vector<Node> trie;
21    vector<int> order;
22
23    AhoCorasick()
24    {
25        trie.push_back(Node());
26    }
27
28    void addString(string s)
29    {
30        int u = 0;

```

19.2 Counting Patterns - Aho-Corasick (Cuento de Ocurrencias) (cont.)

```

31
32        for(char c : s)
33        {
34            int x = c - 'a';
35
36            if(trie[u].next[x] == -1)
37            {
38                trie[u].next[x] = trie.size();
39                trie.push_back(Node());
40            }
41
42            u = trie[u].next[x];
43        }
44    }
45
46    int getNode(string s)
47    {
48        int u = 0;
49
50        for(char c : s)
51        {
52            u = trie[u].next[c - 'a'];
53        }
54
55        return u;
56    }
57
58    void build()
59    {
60        queue<int> q;
61
62        for(int c = 0; c < 26; c++)
63        {
64            int v = trie[0].next[c];
65
66            if(v == -1)
67            {
68                trie[0].next[c] = 0;
69            }
70            else
71            {
72                trie[v].link = 0;
73                q.push(v);
74            }
75        }
76
77        while(!q.empty())
78        {
79            int u = q.front();
80            q.pop();
81
82            order.push_back(u);
83
84            for(int c = 0; c < 26; c++)
85            {
86                int v = trie[u].next[c];
87
88                if(v == -1)
89                {
90                    trie[u].next[c] = trie[trie[u].link].next[c];
91                }
92                else
93                {
94                    trie[v].link = trie[trie[u].link].next[c];
95                    q.push(v);
96                }
97            }
98        }
99    }
100
101    void search(string s)
102    {

```

19.2 Counting Patterns - Aho-Corasick (Cuento de Ocurrencias) (cont.)

```

03     int u = 0;
04
05     for(char c : s)
06     {
07         u = trie[u].next[c - 'a'];
08         trie[u].cnt++;
09     }
10 }
11
12 void propagate()
13 {
14     for(int i = order.size() - 1; i >= 0; i--)
15     {
16         int u = order[i];
17
18         trie[trie[u].link].cnt += trie[u].cnt;
19     }
20 }
21
22 int getCount(string s)
23 {
24     int u = getNode(s);
25     return trie[u].cnt;
26 }
27 };
28
29 int main()
30 {
31     ios::sync_with_stdio(false);
32     cin.tie(nullptr);
33
34     string s;
35     cin >> s;
36
37     int k;
38     cin >> k;
39
40     vector<string> patterns(k);
41
42     AhoCorasick ac;
43
44     for(int i = 0; i < k; i++)
45     {
46         cin >> patterns[i];
47         ac.addString(patterns[i]);
48     }
49
50     ac.build();
51
52     ac.search(s);
53
54     ac.propagate();
55
56     for(string p : patterns)
57     {
58         cout << ac.getCount(p) << '\n';
59     }
60
61     return 0;
62 }

```

19.3 Distinct Subsequences - DP con Resta de la Aparicion Anterior

O: (n) tiempo, (n) memoria

Uso: $dp[i] = 2 * dp[i-1] - dp[last[c]-1]$; la respuesta es $dp[n] - 1$

Cu ntas SUBSECUENCIAS distintas tiene la cadena, modulo 10^9+7 . Cada letra nueva duplica las subsecuencias, porque cada una existente se puede quedar igual o tomar la letra. Pero si esa letra YA hab a salido antes, las que se formaron la vez pasada se cuentan dos veces, as i que se RESTA el dp justo antes de la aparicion anterior. Ese 'duplicar y restar la aparicion previa' es el patron, y aparece en varios problemas de conteo de subsecuencias distintas. El -1 del final quita la subsecuencia vac a. El last[] guarda posiciones 1-indexadas para que 0 signifique 'no ha salido', por eso el $dp[last[c] - 1]$ y no $dp[last[c]]$.

C++\CSES\String Algorithms\Distinct Subsequences.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5
6  const ll MOD = 1000000007;
7
8  int main()
9  {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     string s;
14     cin >> s;
15
16     int n = s.size();
17
18     vector<ll> dp(n + 1);
19     vector<int> last(26, 0);
20
21     dp[0] = 1;
22
23     for(int i = 1; i <= n; i++)
24     {
25         int c = s[i - 1] - 'a';
26
27         dp[i] = 2 * dp[i - 1] % MOD;
28
29         if(last[c] != 0)
30         {
31             dp[i] -= dp[last[c] - 1];
32
33             if(dp[i] < 0)
34                 dp[i] += MOD;
35         }
36
37         last[c] = i;
38     }
39
40     // Quitamos la subsecuencia vac a
41     cout << (dp[n] - 1 + MOD) % MOD << '\n';
42
43     return 0;
44 }

```

19.4 Distinct Substrings - Suffix Automaton (Contar Distintas)

O: (n) estados y transiciones, con alfabeto fijo

Uso: sam.add(c) por letra; sam.countDistinct()

Cuántas subcadenas DISTINTAS tiene la cadena. El suffix automaton es el automata mínimo que acepta todos los sufijos, y tiene a lo sumo $2n$ estados y $3n$ transiciones, por eso se reserva $2*n$. Cada estado representa un conjunto de subcadenas que terminan en las mismas posiciones, y esas subcadenas son exactamente los largos del intervalo $(len[link] + 1 .. len[v])$. Entonces la respuesta es sumar $len[v] - len[link[v]]$ sobre todos los estados menos la raíz. Una línea, una vez construido el automata. EL CLONE ES TODO EL TRUCO de la construcción: cuando el estado q al que se llega es más largo de lo que corresponde, se parte en dos para que cada estado siga representando un rango limpio de largos. Si se omite, el automata deja de ser mínimo y las cuentas se caen. CUANDO USAR: cualquier pregunta sobre TODAS las subcadenas. Para sufijos ordenados, suffix array; para subcadenas como conjunto, suffix automaton.

C++\CSES\String Algorithms\Distinct Substrings.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct State
5 {
6     int next[26];
7     int link;
8     int len;
9
10    State()
11    {
12        fill(next, next + 26, -1);
13        link = -1;
14        len = 0;
15    }
16 };
17
18 struct SuffixAutomaton
19 {
20     vector<State> st;
21     int last;
22
23     SuffixAutomaton(int n)
24     {
25         st.reserve(2 * n);
26         st.push_back(State());
27
28         last = 0;
29     }
30
31     void add(char c)
32     {
33         int x = c - 'a';
34
35         int cur = st.size();
36         st.push_back(State());
37
38         st[cur].len = st[last].len + 1;
39
40         int p = last;
41
42         while(p != -1 && st[p].next[x] == -1)
43         {
44             st[p].next[x] = cur;
45             p = st[p].link;
46         }
47
48         if(p == -1)
49         {
50             st[cur].link = 0;
51         }
52         else
53         {
54             int q = st[p].next[x];
55
56             if(st[p].len + 1 == st[q].len)
```

19.4 Distinct Substrings - Suffix Automaton (Contar Distintas) (cont.)

```
57         {
58             st[cur].link = q;
59         }
60     }
61
62     {
63         int clone = st.size();
64         st.push_back(st[q]);
65
66         st[clone].len = st[p].len + 1;
67
68         while(p != -1 && st[p].next[x] == q)
69         {
70             st[p].next[x] = clone;
71             p = st[p].link;
72         }
73
74         st[q].link = clone;
75         st[cur].link = clone;
76     }
77
78     last = cur;
79 }
80
81 long long countDistinct()
82 {
83     long long ans = 0;
84
85     for(int v = 1; v < (int)st.size(); v++)
86     {
87         ans += st[v].len - st[st[v].link].len;
88     }
89
90     return ans;
91 }
92 };
93
94 int main()
95 {
96     ios::sync_with_stdio(false);
97     cin.tie(nullptr);
98
99     string s;
100     cin >> s;
101
102     SuffixAutomaton sam(s.size());
103
104     for(char c : s)
105     {
106         sam.add(c);
107     }
108
109     cout << sam.countDistinct() << '\n';
110
111     return 0;
112 }
```

19.5 Finding Borders - Funcion de Prefijos (Cadena de Bordes)

Todos los bordes de s (prefijos que también son sufijos) salen de la función de prefijos, sin buscar nada más. El borde más largo es $pi[n-1]$; y el siguiente más largo es el borde más largo DE ESE BORDE, o sea $pi[pi[n-1] - 1]$; y así hasta llegar a 0. POR QUE LA CADENA LOS DA TODOS: un borde de s más corto que el borde más largo b también es borde de b (es prefijo de s, luego prefijo de b; y es sufijo de s, luego sufijo de b). Así que los bordes de s son b y los bordes de b, recursivamente, y no se salta ninguno. Salen de mayor a menor, por eso el reverse al final. Esta cadena $j \rightarrow pi[j-1]$ es la misma que usa KMP para retroceder, y es de las ideas más reusadas de strings: aparece en periodos, en el automata de KMP y en contar apariciones de cada prefijo.

C++\CSES\String Algorithms\Finding Borders.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
```

19.5 Finding Borders - Funcion de Prefijos (Cadena de Bordes) (cont.)

```

4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     string s;
9     cin >> s;
10
11     int n = s.size();
12
13     vector<int> pi(n);
14
15     for (int i = 1; i < n; i++) {
16         int j = pi[i - 1];
17
18         while (j > 0 && s[i] != s[j])
19             j = pi[j - 1];
20
21         if (s[i] == s[j])
22             j++;
23
24         pi[i] = j;
25     }
26
27     vector<int> ans;
28
29     int j = pi[n - 1];
30
31     while (j > 0) {
32         ans.push_back(j);
33         j = pi[j - 1];
34     }
35
36     reverse(ans.begin(), ans.end());
37
38     for (int x : ans)
39         cout << x << ' ';
40
41     cout << '\n';
42 }

```

19.6 Finding Patterns - Aho-Corasick (Aparece o No)

O: (suma de patrones + |s| + 26*nodos)

Uso: igual que Counting Patterns, pero appears(p) devuelve si cnt > 0

Decir, para cada patron, si aparece o no dentro del texto. Es el mismo Aho-Corasick de Counting Patterns: el conteo propagado ya responde la pregunta, y appears() solo mira si quedo mayor que cero. Vale la pena tener los dos lado a lado porque la unica diferencia es la ultima linea. Si solo hace falta el si/no, tambien sirve un suffix automaton del texto y caminar cada patron por el; eso es O(suma de patrones) sin construir nada sobre los patrones.

C++\CSES\String Algorithms\Finding Patterns.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct AhoCorasick
5 {
6     struct Node
7     {
8         int next[26];
9         int link;
10        int cnt;
11
12        Node()
13        {
14            fill(next, next + 26, -1);
15            link = 0;
16            cnt = 0;
17        }
18    };
19
20    vector<Node> trie;

```

19.6 Finding Patterns - Aho-Corasick (Aparece o No) (cont.)

```

21    vector<int> order;
22
23    AhoCorasick()
24    {
25        trie.push_back(Node());
26    }
27
28    void addString(string s)
29    {
30        int u = 0;
31
32        for(char c : s)
33        {
34            int x = c - 'a';
35
36            if(trie[u].next[x] == -1)
37            {
38                trie[u].next[x] = trie.size();
39                trie.push_back(Node());
40            }
41
42            u = trie[u].next[x];
43        }
44    }
45
46    void build()
47    {
48        queue<int> q;
49
50        for(int c = 0; c < 26; c++)
51        {
52            int v = trie[0].next[c];
53
54            if(v == -1)
55            {
56                trie[0].next[c] = 0;
57            }
58            else
59            {
60                trie[v].link = 0;
61                q.push(v);
62            }
63        }
64
65        while(!q.empty())
66        {
67            int u = q.front();
68            q.pop();
69
70            order.push_back(u);
71
72            for(int c = 0; c < 26; c++)
73            {
74                int v = trie[u].next[c];
75
76                if(v == -1)
77                {
78                    trie[u].next[c] = trie[trie[u].link].next[c];
79                }
80                else
81                {
82                    trie[v].link = trie[trie[u].link].next[c];
83                    q.push(v);
84                }
85            }
86        }
87    }
88
89    void search(string s)
90    {
91        int u = 0;
92

```

19.6 Finding Patterns - Aho-Corasick (Aparece o No) (cont.)

```

93     for(char c : s)
94     {
95         u = trie[u].next[c - 'a'];
96         trie[u].cnt++;
97     }
98 }
99
100 void propagate()
101 {
102     for(int i = order.size() - 1; i >= 0; i--)
103     {
104         int u = order[i];
105
106         trie[trie[u].link].cnt += trie[u].cnt;
107     }
108 }
109
110 bool appears(string s)
111 {
112     int u = 0;
113
114     for(char c : s)
115     {
116         u = trie[u].next[c - 'a'];
117     }
118
119     return trie[u].cnt > 0;
120 }
121 };
122
123 int main()
124 {
125     ios::sync_with_stdio(false);
126     cin.tie(nullptr);
127
128     string s;
129     cin >> s;
130
131     int k;
132     cin >> k;
133
134     vector<string> patterns(k);
135
136     AhoCorasick ac;
137
138     for(int i = 0; i < k; i++)
139     {
140         cin >> patterns[i];
141         ac.addString(patterns[i]);
142     }
143
144     ac.build();
145
146     ac.search(s);
147
148     ac.propagate();
149
150     for(string p : patterns)
151     {
152         if(ac.appears(p))
153             cout << "YES\n";
154         else
155             cout << "NO\n";
156     }
157
158     return 0;
159 }

```

19.7 Finding Periods - Periodos = n - Bordes

Un periodo p de s es un desplazamiento tal que $s[i] = s[i+p]$ para todo i valido. La equivalencia que resuelve el problema: p es periodo $\Leftrightarrow n - p$ es un borde porque " $s[i] = s[i+p]$ para todo i " dice exactamente que el prefijo de largo $n-p$ es igual al sufijo de largo $n-p$. Así que los periodos son n menos cada borde de la cadena de "Finding Borders", mas el periodo trivial n (que corresponde al borde vacio). Se juntan y se ordenan. CUANDO USAR: "la cadena es repetición de un bloque", "el periodo mas corto", "cuantas veces se repite". El periodo mas corto es $n - \pi[n-1]$; y s es una repetición EXACTA de un bloque si y solo si ese periodo divide a n . Esa ultima condicion es la que se olvida.

C++\CSES\String Algorithms\Finding Periods.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  vector<int> KMP(string s) {
5      int n = s.size();
6      vector<int> pi(n);
7
8      for (int i = 1; i < n; i++) {
9          int j = pi[i - 1];
10
11          while (j > 0 && s[i] != s[j])
12              j = pi[j - 1];
13
14          if (s[i] == s[j])
15              j++;
16
17          pi[i] = j;
18      }
19
20      return pi;
21 }
22
23 int main() {
24     ios::sync_with_stdio(false);
25     cin.tie(nullptr);
26
27     string s;
28     cin >> s;
29
30     int n = s.size();
31
32     vector<int> pi = KMP(s);
33
34     vector<int> ans;
35
36     int j = pi[n - 1];
37
38     while (j > 0) {
39         ans.push_back(n - j);
40         j = pi[j - 1];
41     }
42
43     ans.push_back(n);
44
45     sort(ans.begin(), ans.end());
46
47     for (int x : ans)
48         cout << x << ' ';
49
50     cout << '\n';
51 }

```

19.8 Inverse Suffix Array - Reconstruir la Cadena desde el Suffix Array

O: (n log n) por el segment tree de maximos

Uso: greedy sobre sa; seg guarda la letra usada indexada por rank[p+1]

Dado un suffix array, construir la cadena mas chica que lo produce, o -1 si no existe. Se recorre el suffix array en orden y se asigna a cada sufijo la letra mas pequena posible. Dos sufijos consecutivos del arreglo pueden compartir letra SOLO si el orden ya queda decidido por lo que viene despues, o sea si rank[p+1] del anterior es menor que rank[p+1] del actual. El segment tree de maximos responde justo eso: entre los sufijos ya colocados cuyo siguiente sufijo va DESPUES del nuestro, cual es la letra mas grande. Si esa letra es la actual, hay conflicto y toca subir a la siguiente. El sufijo de largo 1 (p == n-1) va aparte porque no tiene siguiente y siempre obliga a subir. Si se pasa de 26 letras no hay solucion y se responde -1.

C++\CSES\String Algorithms\Inverse Suffix Array.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct SegmentTree
5 {
6     int n;
7     vector<int> tree;
8
9     SegmentTree(int n)
10    {
11        this->n = n;
12        tree.assign(4 * n, -1);
13    }
14
15    void update(int node, int l, int r, int pos, int val)
16    {
17        if(l == r)
18        {
19            tree[node] = val;
20            return;
21        }
22
23        int mid = (l + r) / 2;
24
25        if(pos <= mid)
26            update(node * 2, l, mid, pos, val);
27        else
28            update(node * 2 + 1, mid + 1, r, pos, val);
29
30        tree[node] = max(tree[node * 2], tree[node * 2 + 1]);
31    }
32
33    void update(int pos, int val)
34    {
35        update(1, 0, n - 1, pos, val);
36    }
37
38    int query(int node, int l, int r, int ql, int qr)
39    {
40        if(qr < l || r < ql)
41            return -1;
42
43        if(ql <= l && r <= qr)
44            return tree[node];
45
46        int mid = (l + r) / 2;
47
48        return max(
49            query(node * 2, l, mid, ql, qr),
50            query(node * 2 + 1, mid + 1, r, ql, qr)
51        );
52    }
53
54    int query(int l, int r)
55    {
56        if(l > r)
57            return -1;
58    }
```

19.8 Inverse Suffix Array - Reconstruir la Cadena desde el Suffix Array (cont.)

```
59     return query(1, 0, n - 1, l, r);
60 }
61 };
62
63 int main()
64 {
65     ios::sync_with_stdio(false);
66     cin.tie(nullptr);
67
68     int n;
69     cin >> n;
70
71     vector<int> sa(n);
72
73     for(int i = 0; i < n; i++)
74     {
75         cin >> sa[i];
76         sa[i]--; // pasamos a 0-index
77     }
78
79     // rank[x] = posicion de x dentro del suffix array
80     vector<int> rank(n);
81
82     for(int i = 0; i < n; i++)
83         rank[sa[i]] = i;
84
85     string s(n, 'a');
86
87     SegmentTree seg(n);
88
89     int c = 0;
90
91     for(int i = 0; i < n; i++)
92     {
93         int p = sa[i];
94
95         // Si estamos colocando el sufijo de longitud 1,
96         // tiene que ser estrictamente mayor que el anterior.
97         if(i > 0 && p == n - 1)
98             c++;
99
100        else if(p < n - 1)
101        {
102            // Miramos los sufijos anteriores cuyo siguiente
103            // sufijo aparece despues de p+1.
104            int r = rank[p + 1];
105
106            int mx = seg.query(r + 1, n - 1);
107
108            // Si alguno tiene nuestra misma letra,
109            // necesitamos aumentar la letra.
110            if(mx == c)
111                c++;
112        }
113
114        if(c >= 26)
115        {
116            cout << -1 << '\n';
117            return 0;
118        }
119
120        s[p] = char('a' + c);
121
122        // Guardamos la letra de s[p] asociada a rank[p+1]
123        if(p < n - 1)
124            seg.update(rank[p + 1], c);
125    }
126
127    cout << s << '\n';
128
129    return 0;
130 }
```

19.9 Longest Palindrome - Manacher

Manacher: el palindromo mas largo en $O(n)$. Se intercala '#' entre letras para que los palindromos pares y los impares se vuelvan todos impares, y para cada centro se guarda $p[i]$, el radio del palindromo mas largo ahi. LO QUE LO HACE LINEAL: se mantiene el palindromo que llega mas a la derecha (center, right). Si i cae dentro de el, su espejo $2*center - i$ ya se calculo, y $p[i]$ arranca en $\min(right - i, p[espejo])$ en vez de en 0. Solo se expande lo que sobresale de right, y right nunca retrocede. Para volver a la cadena original: con separadores, el palindromo de radio $p[i]$ centrado en i mide $p[i]$ letras reales y empieza en $(i - p[i]) / 2$. OJO, ESTE ARCHIVO TIENE UN BUCLE DE DEPURACION AL FINAL: despues de imprimir la respuesta recorre p e imprime $p[i]$ para cada i impar. Con la entrada "ayabtu" sale "bab" y 7 lineas de mas (comprobado). En CSES eso es Wrong Answer; hay que borrar ese for antes de enviar.

C++(CSES)String Algorithms\Longest Palindrome.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> manacher(string s) {
5     string t = "#";
6
7     for (char c : s) {
8         t += c;
9         t += '#';
10    }
11
12    int n = t.size();
13
14    vector<int> p(n);
15
16    int center = 0;
17    int right = 0;
18
19    for (int i = 0; i < n; i++) {
20        int mirror = 2 * center - i;
21
22        if (i < right)
23            p[i] = min(right - i, p[mirror]);
24
25        while (i + 1 + p[i] < n &&
26              i - 1 - p[i] >= 0 &&
27              t[i + 1 + p[i]] == t[i - 1 - p[i]]) {
28            p[i]++;
29        }
30
31        if (i + p[i] > right) {
32            center = i;
33            right = i + p[i];
34        }
35    }
36
37    return p;
38 }
39
40 int main() {
41     ios::sync_with_stdio(false);
42     cin.tie(nullptr);
43
44     string s;
45     cin >> s;
46
47     vector<int> p = manacher(s);
48
49     int pos = max_element(p.begin(), p.end()) - p.begin();
50     int len = p[pos];
51
52     int start = (pos - len) / 2;
53
54     cout << s.substr(start, len) << '\n';
55
56     for(int i = 0 ; i < p.size();i++)
57     {
58         if(i%2!=0)
```

19.9 Longest Palindrome - Manacher (cont.)

```
59 {
60     cout << p[i] << endl;
61 }
62
63 }
64
65 return 0;
66 }
```

19.10 Minimal Rotation - Rotacion Minima (Dos Punteros)

La rotacion lexicograficamente menor en $O(n)$ con dos candidatos i y j. Se trabaja sobre $s + s$ (asi toda rotacion es un substring de largo n) y se comparan las rotaciones que empiezan en i y en j caracter por caracter hasta que difieren en la posicion k. EL SALTO QUE HACE QUE SEA LINEAL: si $s[i+k] > s[j+k]$, no solo la rotacion en i pierde; tambien pierden las que empiezan en $i+1, \dots, i+k$, porque cada una se puede emparejar con la que empieza en $j+1, \dots, j+k$ y pierde por la misma posicion. Por eso i salta a $i+k+1$ de una. Como i y j solo avanzan y ninguno pasa de n, el total es $O(n)$. Si k llega a n, las dos rotaciones son iguales (la cadena es periodica) y cualquiera sirve. El if (i == j) j++ evita comparar un candidato consigo mismo. Alternativas: el algoritmo de Booth (con funcion de fallo) o el sufijo minimo de Lyndon. Esta version de dos punteros es la mas corta de escribir y la mas facil de recordar.

C++(CSES)String Algorithms\Minimal Rotation.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string minimalRotation(string s) {
5     int n = s.size();
6
7     s += s;
8
9     int i = 0, j = 1;
10
11     while (i < n && j < n) {
12         int k = 0;
13
14         while (k < n && s[i + k] == s[j + k])
15             k++;
16
17         if (k == n)
18             break;
19
20         if (s[i + k] > s[j + k])
21             i = i + k + 1;
22         else
23             j = j + k + 1;
24
25         if (i == j)
26             j++;
27     }
28
29     int pos = min(i, j);
30
31     return s.substr(pos, n);
32 }
33
34 int main() {
35     ios::sync_with_stdio(false);
36     cin.tie(nullptr);
37
38     string s;
39     cin >> s;
40
41     cout << minimalRotation(s) << '\n';
42 }
```

19.11 Palindrome Queries - Hashing Doble con Segment Tree

O: $(\log n)$ por operacion, hashing doble en segment tree

Uso: un arbol con la cadena y otro con la invertida; palindromo si coinciden

Consultas de "el tramo $[l, r]$ es palindromo?" sobre una cadena que ademas CAMBIA: hay actualizaciones de un caracter mezcladas con las preguntas. LA IDEA: un segment tree donde cada nodo guarda DOS hashes del tramo, el de izquierda a derecha (fw) y el de derecha a izquierda (bw). El tramo es palindromo si los dos coinciden. EL MERGE ES LO UNICO DELICADO, y es asimetrico a proposito: fw de $(a + b) = a.fw * BASE^{\text{len}(b)} + b.fw$ se lee a y despues b bw de $(a + b) = b.bw * BASE^{\text{len}(a)} + a.bw$ al revés se lee b y despues a Por eso hace falta guardar el LARGO en cada nodo: sin el no se sabe por cuanto desplazar. Las potencias de BASE van precalculadas en pw[], porque calcularlas en cada merge seria un log de mas. El nodo neutro es el de largo 0, y el merge lo trata aparte para que las consultas que no cubren nada no ensucien el hash. POR QUE HASHING Y NO MANACHER: Manacher es $O(n)$ pero para una cadena FIJA. Aqui hay actualizaciones, y rehacerlo en cada cambio seria $O(n)$ por consulta. El hash con segment tree da $O(\log n)$ tanto para actualizar como para preguntar. EL RIESGO DEL HASHING ES LA COLISION. Con un solo modulo de 10^9 y muchas consultas, la probabilidad ya no es despreciable (paradoja del cumpleaños: con 10^5 consultas sobre 10^9 valores). En un juez adversario conviene hashing DOBLE, con dos pares (BASE, MOD) distintos, y dar palindromo solo si coinciden los dos. La BASE debe ser mayor que el alfabeto y los caracteres se mapean a 1..26, no a 0..25: si la 'a' valiera 0, las cadenas "a", "aa" y "aaa" tendrian el mismo hash.

C++\CSES\String Algorithms\Palindrome Queries.cpp

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const ll MOD = 1000000007;
7 const ll BASE = 911382323;
8
9 struct Node {
10     ll fw = 0; // izquierda -> derecha
11     ll bw = 0; // derecha -> izquierda
12     int len = 0;
13 };
14
15 vector<ll> pw;
16 vector<Node> st;
17
18 Node mergeNode(Node a, Node b) {
19     if (a.len == 0) return b;
20     if (b.len == 0) return a;
21
22     Node res;
23
24     res.len = a.len + b.len;
25
26     // a + b
27     res.fw = (a.fw * pw[b.len] + b.fw) % MOD;
28
29     // b + a
30     res.bw = (b.bw * pw[a.len] + a.bw) % MOD;
31
32     return res;
33 }
34
35 void build(int p, int l, int r, string &s) {
36     if (l == r) {
37         st[p].fw = s[l] - 'a' + 1;
38         st[p].bw = s[l] - 'a' + 1;
39         st[p].len = 1;
40         return;
41     }
42
43     int mid = (l + r) / 2;
44
45     build(p * 2, l, mid, s);
46     build(p * 2 + 1, mid + 1, r, s);
47
48     st[p] = mergeNode(st[p * 2], st[p * 2 + 1]);
49 }
```

19.11 Palindrome Queries - Hashing Doble con Segment Tree (cont.)

```
48 }
49
50 void update(int p, int l, int r, int pos, char c) {
51     if (l == r) {
52         st[p].fw = c - 'a' + 1;
53         st[p].bw = c - 'a' + 1;
54         return;
55     }
56
57     int mid = (l + r) / 2;
58
59     if (pos <= mid)
60         update(p * 2, l, mid, pos, c);
61     else
62         update(p * 2 + 1, mid + 1, r, pos, c);
63
64     st[p] = mergeNode(st[p * 2], st[p * 2 + 1]);
65 }
66
67 Node query(int p, int l, int r, int ql, int qr) {
68     if (qr < l || r < ql)
69         return Node();
70
71     if (ql <= l && r <= qr)
72         return st[p];
73
74     int mid = (l + r) / 2;
75
76     Node left = query(p * 2, l, mid, ql, qr);
77     Node right = query(p * 2 + 1, mid + 1, r, ql, qr);
78
79     return mergeNode(left, right);
80 }
81
82 int main() {
83     ios::sync_with_stdio(false);
84     cin.tie(nullptr);
85
86     int n, m;
87     cin >> n >> m;
88
89     string s;
90     cin >> s;
91
92     // Pasamos a 0-index
93     // s[0] corresponde a posicion 1.
94
95     pw.resize(n + 1);
96     pw[0] = 1;
97
98     for (int i = 1; i <= n; i++)
99         pw[i] = pw[i - 1] * BASE % MOD;
100
101     st.resize(4 * n + 5);
102
103     build(1, 0, n - 1, s);
104
105     while (m--) {
106         int type;
107         cin >> type;
108
109         if (type == 1) {
110             int k;
111             char x;
112
113             cin >> k >> x;
114
115             --k;
116
117             update(1, 0, n - 1, k, x);
118         }
119     }
```

19.11 Palindrome Queries - Hashing Doble con Segment Tree (cont.)

```

21
22     } else {
23
24         int a, b;
25         cin >> a >> b;
26
27         --a;
28         --b;
29
30         Node ans = query(1, 0, n - 1, a, b);
31
32         if (ans.fw == ans.bw)
33             cout << "YES\n";
34         else
35             cout << "NO\n";
36     }
37 }
38 }

```

19.12 Pattern Positions - Aho-Corasick (Primera Aparicion)

O: (suma de patrones + |s| + 26*nodos)

Uso: search() guarda min(first); getFirst(p) devuelve first - |p| + 1, o -1

La posicion donde EMPIEZA la primera aparicion de cada patron, o -1 si no aparece. Mismo esqueleto de Aho-Corasick, cambiando el contador por un minimo: en vez de cnt se guarda first, la posicion mas temprana en que el recorrido del texto paso por ese nodo. Se propaga igual por los suffix links al reves, pero con min en vez de suma. EL DETALLE: first guarda donde TERMINA la ocurrencia, asi que para devolver donde empieza hay que restar el largo del patron, first - |p| + 1. Equivocarse ahi da respuestas corridas. El INT_MAX como marca de 'nunca aparecio' sobrevive la propagacion porque min(INF, INF) = INF.

C++\CSES\String Algorithms\Pattern Positions.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct AhoCorasick
5  {
6      struct Node
7      {
8          int next[26];
9          int link;
10         int first;
11
12         Node()
13         {
14             fill(next, next + 26, -1);
15             link = 0;
16             first = INT_MAX;
17         }
18     };
19
20     vector<Node> trie;
21     vector<int> order;
22
23     AhoCorasick()
24     {
25         trie.push_back(Node());
26     }
27
28     void addString(string s)
29     {
30         int u = 0;
31
32         for(char c : s)
33         {
34             int x = c - 'a';
35
36             if(trie[u].next[x] == -1)
37             {
38                 trie[u].next[x] = trie.size();
39                 trie.push_back(Node());
40             }

```

19.12 Pattern Positions - Aho-Corasick (Primera Aparicion) (cont.)

```

41
42         u = trie[u].next[x];
43     }
44 }
45
46 void build()
47 {
48     queue<int> q;
49
50     for(int c = 0; c < 26; c++)
51     {
52         int v = trie[0].next[c];
53
54         if(v == -1)
55         {
56             trie[0].next[c] = 0;
57         }
58         else
59         {
60             trie[v].link = 0;
61             q.push(v);
62         }
63     }
64
65     while(!q.empty())
66     {
67         int u = q.front();
68         q.pop();
69
70         order.push_back(u);
71
72         for(int c = 0; c < 26; c++)
73         {
74             int v = trie[u].next[c];
75
76             if(v == -1)
77             {
78                 trie[u].next[c] = trie[trie[u].link].next[c];
79             }
80             else
81             {
82                 trie[v].link = trie[trie[u].link].next[c];
83                 q.push(v);
84             }
85         }
86     }
87 }
88
89 void search(string s)
90 {
91     int u = 0;
92
93     for(int i = 0; i < (int)s.size(); i++)
94     {
95         u = trie[u].next[s[i] - 'a'];
96
97         trie[u].first = min(trie[u].first, i + 1);
98     }
99 }
100
101 void propagate()
102 {
103     for(int i = order.size() - 1; i >= 0; i--)
104     {
105         int u = order[i];
106
107         trie[trie[u].link].first =
108             min(trie[trie[u].link].first, trie[u].first);
109     }
110 }
111
112 int getFirst(string s)

```

19.12 Pattern Positions - Aho-Corasick (Primera Aparicion) (cont.)

```

13 {
14     int u = 0;
15
16     for(char c : s)
17     {
18         u = trie[u].next[c - 'a'];
19     }
20
21     if(trie[u].first == INT_MAX)
22         return -1;
23
24     return trie[u].first - (int)s.size() + 1;
25 }
26 };
27
28 int main()
29 {
30     ios::sync_with_stdio(false);
31     cin.tie(nullptr);
32
33     string s;
34     cin >> s;
35
36     int k;
37     cin >> k;
38
39     vector<string> patterns(k);
40
41     AhoCorasick ac;
42
43     for(int i = 0; i < k; i++)
44     {
45         cin >> patterns[i];
46         ac.addString(patterns[i]);
47     }
48
49     ac.build();
50
51     ac.search(s);
52
53     ac.propagate();
54
55     for(string p : patterns)
56     {
57         cout << ac.getFirst(p) << '\n';
58     }
59
60     return 0;
61 }

```

19.13 Repeating Substring - Suffix Automaton (Ocurrencias por Estado)

O(n), construir mas ordenar por largo con counting sort

Uso: sam.longestRepeating(s) -> la subcadena mas larga que sale 2+ veces, o -1

La subcadena mas larga que aparece al menos dos veces. Se cuenta cuantas veces ocurre cada estado: los estados creados como 'cur' valen 1 y los clones valen 0, y despues se propaga por los suffix links de los estados LARGOS hacia los cortos. Para recorrer en ese orden se ordenan los estados por len con COUNTING SORT, que es O(n) y es el patron estandar en suffix automaton (ordenar con sort() seria O(n log n) y mas código). Con occ listo, la respuesta es el estado de mayor len con occ >= 2. firstPos guarda donde termino la primera ocurrencia, así que la subcadena se recorta con substr(pos - len + 1, len). Los clones arrancan en 0 a proposito: representan la misma posición que q, y contarlos tambien duplicaria las ocurrencias.

C++\CSES\String Algorithms\Repeating Substring.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct State
5 {
6     int next[26];
7     int link;
8     int len;

```

19.13 Repeating Substring - Suffix Automaton (Ocurrencias por Estado) (cont.)

```

9     int occ;
10    int firstPos;
11
12    State()
13    {
14        fill(next, next + 26, -1);
15        link = -1;
16        len = 0;
17        occ = 0;
18        firstPos = -1;
19    }
20 };
21
22 struct SuffixAutomaton
23 {
24     vector<State> st;
25     int last;
26
27     SuffixAutomaton(int n)
28     {
29         st.reserve(2 * n);
30         st.push_back(State());
31
32         last = 0;
33     }
34
35     void add(char c, int pos)
36     {
37         int x = c - 'a';
38
39         int cur = st.size();
40         st.push_back(State());
41
42         st[cur].len = st[last].len + 1;
43         st[cur].firstPos = pos;
44         st[cur].occ = 1;
45
46         int p = last;
47
48         while(p != -1 && st[p].next[x] == -1)
49         {
50             st[p].next[x] = cur;
51             p = st[p].link;
52         }
53
54         if(p == -1)
55         {
56             st[cur].link = 0;
57         }
58         else
59         {
60             int q = st[p].next[x];
61
62             if(st[p].len + 1 == st[q].len)
63             {
64                 st[cur].link = q;
65             }
66             else
67             {
68                 int clone = st.size();
69                 st.push_back(st[q]);
70
71                 st[clone].len = st[p].len + 1;
72                 st[clone].occ = 0;
73
74                 while(p != -1 && st[p].next[x] == q)
75                 {
76                     st[p].next[x] = clone;
77                     p = st[p].link;
78                 }
79
80                 st[q].link = clone;

```

19.13 Repeating Substring - Suffix Automaton (Ocurrencias por Estado) (cont.)

```

81         st[cur].link = clone;
82     }
83 }
84
85     last = cur;
86 }
87
88     string longestRepeating(string s)
89     {
90         int n = s.size();
91
92         for(int i = 0; i < n; i++)
93         {
94             add(s[i], i);
95         }
96
97         vector<int> cnt(n + 1);
98
99         for(auto &state : st)
100         {
101             cnt[state.len]++;
102         }
103
104         for(int i = 1; i <= n; i++)
105         {
106             cnt[i] += cnt[i - 1];
107         }
108
109         vector<int> order(st.size());
110
111         for(int i = st.size() - 1; i >= 0; i--)
112         {
113             order[--cnt[st[i].len]] = i;
114         }
115
116         // Propagamos ocurrencias desde estados largos
117         // hacia sus suffix links.
118         for(int i = order.size() - 1; i > 0; i--)
119         {
120             int u = order[i];
121
122             st[st[u].link].occ += st[u].occ;
123         }
124
125         int best = 0;
126         int bestState = -1;
127
128         for(int i = 1; i < (int)st.size(); i++)
129         {
130             if(st[i].occ >= 2 && st[i].len > best)
131             {
132                 best = st[i].len;
133                 bestState = i;
134             }
135         }
136
137         if(best == 0)
138         {
139             return "-1";
140         }
141
142         int pos = st[bestState].firstPos;
143
144         return s.substr(pos - best + 1, best);
145     }
146 };
147
148 int main()
149 {
150     ios::sync_with_stdio(false);
151     cin.tie(nullptr);
152 }

```

19.13 Repeating Substring - Suffix Automaton (Ocurrencias por Estado) (cont.)

```

53     string s;
54     cin >> s;
55
56     SuffixAutomaton sam(s.size());
57
58     cout << sam.longestRepeating(s) << '\n';
59
60     return 0;
61 }

```

19.14 Required Substring - DP sobre el Automata de KMP

Cuenta cuantas cadenas de largo n sobre un alfabeto de 26 letras CONTIENEN un patron dado, modulo $1e9+7$. LA IDEA CENTRAL: en vez de contar las que lo contienen, se cuenta con una DP donde el estado es "cuanto del patron llevo casado". Se construye el AUTOMATA de KMP: $nxt[j][c]$ dice a que estado se pasa si llevo j caracteres del patron casados y llega la letra c . Eso se calcula con la funcion de prefijos, retrocediendo igual que en la busqueda de KMP. Con el automata, la DP es directa: $dp[i][j]$ = cuantas cadenas de largo i dejan el automata en el estado j . Desde cada estado se prueban las 26 letras y se avanza. Al llegar al estado m (patron completo) esa cadena ya sirve, y todas las formas de completarla tambien: se suman de golpe multiplicando por 26^i (lo que falta), sin seguir simulando. POR QUE HACE FALTA EL AUTOMATA Y NO BASTA CONTAR: dos cadenas distintas pueden dejar el mismo "sufijo util" del patron, y de ahi en adelante se comportan igual. El estado de KMP es exactamente esa informacion, y es lo que colapsa un espacio de 26^n a $n \cdot m$ estados. La construccion de nxt se hace UNA vez, en $O(m \cdot 26)$, retrocediendo con pi ; hacerlo dentro de la DP seria pagarlo n veces. Este patron (automata de KMP + DP) es el que resuelve toda la familia de "cuantas cadenas contienen / evitan un patron", y con varios patrones a la vez el equivalente es Aho-Corasick.

C++\CSES\String Algorithms\Required Substring.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 const ll MOD = 1000000007;
7
8
9 // -----
10 // KMP - Prefix Function
11 // -----
12
13 vector<int> prefixFunction(const string& s) {
14
15     int n = s.size();
16
17     vector<int> pi(n);
18
19     for (int i = 1; i < n; i++) {
20
21         int j = pi[i - 1];
22
23         while (j > 0 && s[i] != s[j])
24             j = pi[j - 1];
25
26         if (s[i] == s[j])
27             j++;
28
29         pi[i] = j;
30     }
31
32     return pi;
33 }
34
35
36 // -----
37 // Transiciones de KMP
38 // -----
39
40 vector<vector<int>> buildKMPTransitions(const string& p) {
41
42     int m = p.size();
43
44 }

```

19.14 Required Substring - DP sobre el Automata de KMP (cont.)

```

44 vector<int> pi = prefixFunction(p);
45
46 vector<vector<int>> nxt(m, vector<int>(26));
47
48 for (int j = 0; j < m; j++) {
49     for (int c = 0; c < 26; c++) {
50         char x = 'A' + c;
51
52         int k = j;
53
54         while (k > 0 && p[k] != x)
55             k = pi[k - 1];
56
57         if (p[k] == x)
58             k++;
59
60         nxt[j][c] = k;
61     }
62 }
63
64 return nxt;
65 }
66
67 // -----
68 // Main
69 // -----
70
71 int main() {
72
73     ios::sync_with_stdio(false);
74     cin.tie(nullptr);
75
76     int n;
77     string p;
78
79     cin >> n >> p;
80
81     int m = p.size();
82
83     auto nxt = buildKMPTransitions(p);
84
85     // Estados:
86     // 0 ... m-1 = cuanto del patron tenemos
87     // m = el patron ya aparecio
88     vector<ll> dp(m + 1);
89     vector<ll> ndp(m + 1);
90
91     dp[0] = 1;
92
93     for (int len = 0; len < n; len++) {
94         fill(ndp.begin(), ndp.end(), 0);
95
96         for (int j = 0; j <= m; j++) {
97             if (dp[j] == 0)
98                 continue;
99
100             // Ya aparecio el patron.
101             // Cualquier caracter que agreguemos mantiene
102             // este estado.
103             if (j == m) {
104
105                 ndp[m] += dp[j] * 26;
106                 ndp[m] %= MOD;
107
108                 continue;
109             }
110
111         }
112     }
113
114 }
115

```

19.14 Required Substring - DP sobre el Automata de KMP (cont.)

```

116 // Todavia no aparecio.
117 for (int c = 0; c < 26; c++) {
118     int k = nxt[j][c];
119
120     ndp[k] += dp[j];
121     ndp[k] %= MOD;
122 }
123
124 dp.swap(ndp);
125 }
126
127 cout << dp[m] << '\n';
128
129 return 0;
130 }
131

```

19.15 String Functions - Funcion Z y Funcion de Prefijos

O: (n) cada una

Uso: zFunction(s) -> z; prefixFunction(s) -> pi // ambos 0-indexados

Las dos funciones basicas de cadenas, una al lado de la otra. z[i] = largo del prefijo comun mas largo entre s y s[i..]. La ventana [l, r] es el truco: es la coincidencia mas a la derecha que se conoce, y mientras i cae adentro se reusa el valor ya calculado z[i - l] en vez de comparar desde cero. Eso es lo que la deja lineal. pi[i] = largo del borde mas largo de s[0..i], o sea el prefijo propio mas largo que tambien es sufijo. El while que baja j = pi[j-1] es el mismo salto de fallo de KMP. LAS DOS SIRVEN PARA LO MISMO casi siempre, y cual usar es cuestion de gusto: la funcion Z se lee mas facil para 'donde aparece el patron' (se busca z[i] == |patron| en patron + '#' + texto) y la de prefijos es la que sirve para periodos, bordes y el automata de KMP. OJO: aca z[0] se fuerza a 0. La convencion comun es z[0] = n; el juez de este problema pide 0.

C++(CSES)String Algorithms(String functions.cpp)

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> zFunction(string s)
5 {
6     int n = s.size();
7
8     vector<int> z(n);
9     int l = 0;
10    int r = 0;
11
12    for(int i = 1; i < n; i++)
13    {
14        if(i <= r)
15        {
16            z[i] = min(r - i + 1, z[i - l]);
17        }
18
19        while(i + z[i] < n &&
20            s[z[i]] == s[i + z[i]])
21        {
22            z[i]++;
23        }
24
25        if(i + z[i] - 1 > r)
26        {
27            l = i;
28            r = i + z[i] - 1;
29        }
30    }
31
32    z[0] = 0;
33
34    return z;
35 }
36
37 vector<int> prefixFunction(string s)
38 {

```

19.15 String Functions - Funcion Z y Funcion de Prefijos (cont.)

```

38     int n = s.size();
39
40     vector<int> pi(n);
41
42     for(int i = 1; i < n; i++)
43     {
44         int j = pi[i - 1];
45
46         while(j > 0 && s[i] != s[j])
47         {
48             j = pi[j - 1];
49         }
50
51         if(s[i] == s[j])
52         {
53             j++;
54         }
55
56         pi[i] = j;
57     }
58
59     return pi;
60 }
61
62 int main()
63 {
64     ios::sync_with_stdio(false);
65     cin.tie(nullptr);
66
67     string s;
68     cin >> s;
69
70     vector<int> z = zFunction(s);
71     vector<int> pi = prefixFunction(s);
72
73     for(int i = 0; i < (int)s.size(); i++)
74     {
75         cout << z[i];
76
77         if(i + 1 < (int)s.size())
78             cout << ' ';
79     }
80
81     cout << '\n';
82
83     for(int i = 0; i < (int)s.size(); i++)
84     {
85         cout << pi[i];
86
87         if(i + 1 < (int)s.size())
88             cout << ' ';
89     }
90
91     cout << '\n';
92
93     return 0;
94 }
95

```

19.16 String Matching - KMP (Busqueda de Patron)

KMP: cuantas veces aparece p en s, en $O(n + m)$. Primero la funcion de prefijos de p, $pi[i]$ = el largo del borde mas largo de $p[0..i]$. Despues se recorre s manteniendo j, cuanto del patron ya coincide; cuando falla, en vez de volver a empezar se retrocede a $pi[j-1]$, que es cuanto del patron sigue coincidiendo gratis. DOS DETALLES QUE DECIDEN SI CUENTA BIEN: despues de un match se hace $j = pi[j-1]$ y no $j = 0$, y eso es lo que permite contar apariciones SOLAPADAS ("aa" en "aaa" son dos). Y el while de retroceso va ANTES del if que avanza, siempre. Por que es lineal: j sube a lo sumo 1 por caracter, y cada retroceso lo baja; no puede bajar mas de lo que subio. CUANDO USAR: buscar un patron fijo. Si son muchos patrones a la vez, Aho-Corasick; si hay que comparar substrings arbitrarios, hashing; si piden los bordes o periodos, la misma pi sirve (ver "Finding Borders" y "Finding Periods").

C++(CSES)String AlgorithmsString Matching.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  // Retorna cuantas veces aparece p dentro de s.
5  // Permite apariciones solapadas.
6  int KMP(const string& s, const string& p) {
7
8      int n = s.size();
9      int m = p.size();
10
11      // Prefix function del patron
12      vector<int> pi(m);
13
14      for (int i = 1; i < m; i++) {
15
16          int j = pi[i - 1];
17
18          while (j > 0 && p[i] != p[j])
19              j = pi[j - 1];
20
21          if (p[i] == p[j])
22              j++;
23
24          pi[i] = j;
25      }
26
27      // Buscar p dentro de s
28      int ans = 0;
29      int j = 0;
30
31      for (int i = 0; i < n; i++) {
32
33          while (j > 0 && s[i] != p[j])
34              j = pi[j - 1];
35
36          if (s[i] == p[j])
37              j++;
38
39          if (j == m) {
40              ans++;
41
42              // Importante para permitir solapamientos
43              j = pi[j - 1];
44          }
45      }
46
47      return ans;
48 }
49
50 int main() {
51
52     ios::sync_with_stdio(false);
53     cin.tie(nullptr);
54
55     string s, p;
56
57     cin >> s >> p;
58

```

19.16 String Matching - KMP (Busqueda de Patron) (cont.)

```

58     cout << KMP(s, p) << '\n';
59
60
61     return 0;
62 }
    
```

19.17 String Transformation - Transformada de Burrows-Wheeler Inversa

O: $(n \log n)$ por el stable_sort, (n) con counting sort

Uso: ordenar estable por caracter da nxt; seguir la cadena desde pos 0 y voltear

Deshacer la transformada de Burrows-Wheeler: dada la ultima columna de las rotaciones ordenadas, recuperar la cadena original. La propiedad que lo resuelve: la i-esima aparicion de una letra en la ULTIMA columna es la misma que la i-esima aparicion de esa letra en la PRIMERA columna. Por eso basta ordenar de forma ESTABLE los indices por caracter: eso construye la primera columna conservando el orden relativo, y nxt[] queda siendo el enlace de cada posicion a la siguiente. Que el sort sea ESTABLE no es opcional, es lo que hace que la propiedad valga. Se recorre la cadena saltando por nxt y al final se invierte, porque el recorrido la va sacando al revés.

C++\CSES\String Algorithms\String Transformation.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      string s;
7      cin >> s;
8
9      int n = s.size();
10
11     vector<int> orden(n);
12     iota(orden.begin(), orden.end(), 0);
13
14     stable_sort(orden.begin(), orden.end(), [&](int a, int b)
15     {
16         return s[a] < s[b];
17     });
18
19     vector<int> nxt(n);
20
21     for(int i = 0; i < n; i++)
22         nxt[orden[i]] = i;
23
24     int pos = 0;
25     string ans;
26
27     for(int i = 0; i < n - 1; i++)
28     {
29         ans += s[pos];
30         pos = nxt[pos];
31     }
32
33     reverse(ans.begin(), ans.end());
34
35     cout << ans << '\n';
36 }
    
```

19.18 Substring Distribution - Suffix Automaton + Arreglo de Diferencias

O: (n) , un barrido de estados mas uno de largos

Uso: por cada estado v, $\text{diff}[\text{len}[\text{link}]+1]++$ y $\text{diff}[\text{len}[v]+1]--$; prefijos

Cuántas subcadenas distintas hay de cada largo 1, 2, ..., n. Cada estado del suffix automaton aporta UN rango contiguo de largos, de $\text{len}[\text{link}]+1$ hasta $\text{len}[v]$, y aporta exactamente una subcadena distinta por cada largo de ese rango. Entonces en vez de sumar uno por uno se marca el rango con un ARREGLO DE DIFERENCIAS (+1 al inicio, -1 despues del fin) y al final una pasada de sumas acumuladas da la respuesta de todos los largos. Esa combinacion de 'cada estado es un intervalo' + 'arreglo de diferencias' es lo que convierte un problema de $O(n^2)$ subcadenas en $O(n)$.

C++\CSES\String Algorithms\Substring Distribution.cpp

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct SAM
    
```

19.18 Substring Distribution - Suffix Automaton + Arreglo de Diferencias (cont.)

```

5  {
6      struct Node
7      {
8          int next[26];
9          int link, len;
10
11         Node()
12         {
13             memset(next, -1, sizeof(next));
14             link = -1;
15             len = 0;
16         }
17     };
18
19     vector<Node> st;
20     int last = 0;
21
22     SAM(int n)
23     {
24         st.reserve(2 * n);
25         st.push_back(Node());
26     }
27
28     void add(char ch)
29     {
30         int c = ch - 'a';
31
32         int cur = st.size();
33         st.push_back(Node());
34
35         st[cur].len = st[last].len + 1;
36
37         int p = last;
38
39         while(p != -1 && st[p].next[c] == -1)
40         {
41             st[p].next[c] = cur;
42             p = st[p].link;
43         }
44
45         if(p == -1)
46         {
47             st[cur].link = 0;
48         }
49         else
50         {
51             int q = st[p].next[c];
52
53             if(st[p].len + 1 == st[q].len)
54             {
55                 st[cur].link = q;
56             }
57             else
58             {
59                 int clone = st.size();
60                 st.push_back(st[q]);
61
62                 st[clone].len = st[p].len + 1;
63
64                 while(p != -1 && st[p].next[c] == q)
65                 {
66                     st[p].next[c] = clone;
67                     p = st[p].link;
68                 }
69
70                 st[q].link = st[cur].link = clone;
71             }
72         }
73
74         last = cur;
75     }
76 };
    
```

19.18 Substring Distribution - Suffix Automaton + Arreglo de Diferencias (cont.)

```

77
78 int main()
79 {
80     ios::sync_with_stdio(false);
81     cin.tie(nullptr);
82
83     string s;
84     cin >> s;
85
86     SAM sam(s.size());
87
88     for(char c : s)
89         sam.add(c);
90
91     vector<long long> diff(s.size() + 2);
92
93     for(int v = 1; v < sam.st.size(); v++)
94     {
95         int l = sam.st[sam.st[v].link].len + 1;
96         int r = sam.st[v].len;
97
98         diff[l]++;
99         diff[r + 1]--;
100     }
101
102     long long cur = 0;
103
104     for(int len = 1; len <= s.size(); len++)
105     {
106         cur += diff[len];
107         cout << cur << ' ';
108     }
109
110     cout << '\n';
111 }

```

19.19 Substring Order I - Descenso por el Suffix Automaton (Distintas)

O: (26*n) construir y bajar

Uso: dp[v] = cuantas subcadenas salen de v; se baja eligiendo letra con k

La k-esima subcadena DISTINTA en orden alfabético. dp[v] = cuantos caminos salen del estado v, que es cuantas subcadenas distintas empiezan con lo que lleva uno escrito. Se calcula en orden inverso de largo (otra vez counting sort). Despues se BAJA por el automata: en cada estado se prueban las letras de la 'a' a la 'z' y si k es mayor que dp[to] se descarta ese bloque entero y se resta; si no, la respuesta empieza por esa letra y se entra. Es la misma idea de 'buscar el k-esimo bajando por una estructura' que se usa en Fenwick o segment tree, pero sobre el automata. El dp arranca en 1 porque cuenta tambien la cadena vacia del propio estado; por eso el corte es k == 1 y no k == 0.

C++\CSES\String Algorithms\Substring Order I.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct SuffixAutomaton
7 {
8     struct State
9     {
10         int next[26];
11         int link;
12         int len;
13
14         State()
15         {
16             fill(next, next + 26, -1);
17             link = -1;
18             len = 0;
19         }
20     };
21
22     vector<State> st;

```

19.19 Substring Order I - Descenso por el Suffix Automaton (Distintas) (cont.)

```

23     int last;
24
25     SuffixAutomaton(int n)
26     {
27         st.reserve(2 * n);
28         st.push_back(State());
29         last = 0;
30     }
31
32     void add(char ch)
33     {
34         int c = ch - 'a';
35
36         int cur = st.size();
37         st.push_back(State());
38
39         st[cur].len = st[last].len + 1;
40
41         int p = last;
42
43         while(p != -1 && st[p].next[c] == -1)
44         {
45             st[p].next[c] = cur;
46             p = st[p].link;
47         }
48
49         if(p == -1)
50         {
51             st[cur].link = 0;
52         }
53         else
54         {
55             int q = st[p].next[c];
56
57             if(st[p].len + 1 == st[q].len)
58             {
59                 st[cur].link = q;
60             }
61             else
62             {
63                 int clone = st.size();
64                 st.push_back(st[q]);
65
66                 st[clone].len = st[p].len + 1;
67
68                 while(p != -1 && st[p].next[c] == q)
69                 {
70                     st[p].next[c] = clone;
71                     p = st[p].link;
72                 }
73
74                 st[q].link = clone;
75                 st[cur].link = clone;
76             }
77         }
78
79         last = cur;
80     }
81 };
82
83 int main()
84 {
85     ios::sync_with_stdio(false);
86     cin.tie(nullptr);
87
88     string s;
89     cin >> s;
90
91     ll k;
92     cin >> k;
93
94     SuffixAutomaton sam(s.size());

```

19.19 Substring Order I - Descenso por el Suffix Automaton (Distintas) (cont.)

```

95
96     for(char c : s)
97         sam.add(c);
98
99     int sz = sam.st.size();
100
101     // Ordenamos los estados por longitud
102     vector<int> cnt(s.size() + 1);
103
104     for(auto &v : sam.st)
105         cnt[v.len]++;
106
107     for(int i = 1; i <= (int)s.size(); i++)
108         cnt[i] += cnt[i - 1];
109
110     vector<int> order(sz);
111
112     for(int i = sz - 1; i >= 0; i--)
113         order[--cnt[sam.st[i].len]] = i;
114
115     // dp[v] = cantidad de caminos/substrings desde v
116     // Incluye el substring vacio.
117     vector<ll> dp(sz, 1);
118
119     for(int i = sz - 1; i >= 0; i--)
120     {
121         int v = order[i];
122
123         for(int c = 0; c < 26; c++)
124         {
125             int to = sam.st[v].next[c];
126
127             if(to != -1)
128                 dp[v] += dp[to];
129         }
130     }
131
132     string ans;
133     int v = 0;
134
135     while(k > 0)
136     {
137         for(int c = 0; c < 26; c++)
138         {
139             int to = sam.st[v].next[c];
140
141             if(to == -1)
142                 continue;
143
144             // Todos los substrings de este bloque
145             // empiezan con esta letra.
146             if(k > dp[to])
147             {
148                 k -= dp[to];
149             }
150             else
151             {
152                 ans.push_back(char('a' + c));
153
154                 // Si k == 1, el substring actual
155                 // es exactamente la respuesta.
156                 if(k == 1)
157                 {
158                     cout << ans << '\n';
159                     return 0;
160                 }
161
162                 // El resto esta dentro de este estado.
163                 k--;
164                 v = to;
165                 break;
166             }
167         }
168     }

```

19.19 Substring Order I - Descenso por el Suffix Automaton (Distintas) (cont.)

```

67     }
68 }
69
70 return 0;
71 }

```

19.20 Substring Order II - Descenso por el Suffix Automaton (con Repetidas)

O: $(26 \cdot n)$ construir y bajar

Uso: $dp[v]$ suma occ de los hijos; cada subcadena pesa cuantas veces aparece

La k -esima subcadena en orden alfabético, pero contando las REPETIDAS tantas veces como aparecen. Es el Substring Order I con un cambio de peso: en la versión I cada subcadena distinta vale 1, aquí vale occ , o sea cuantas veces aparece en la cadena. Por eso primero se propagan las ocurrencias por los suffix links (calculateOccurrences) y el dp suma $occ[to] + dp[to]$ en vez de $1 + dp[to]$. Al bajar hay que restar $occ[to]$ antes de seguir, que son las apariciones del prefijo actual exacto; si k cae dentro de esas, la respuesta es justo lo que se lleva escrito. Comparar este archivo con el de la versión I es la mejor forma de ver que un suffix automaton responde muchas preguntas distintas cambiando solo el peso del dp .

C++(CSES)/String Algorithms/Substring Order II.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using ll = long long;
5
6 struct SuffixAutomaton
7 {
8     struct State
9     {
10         int next[26];
11         int link;
12         int len;
13         ll occ;
14     };
15     State()
16     {
17         fill(next, next + 26, -1);
18         link = -1;
19         len = 0;
20         occ = 0;
21     }
22 };
23
24 vector<State> st;
25 int last;
26
27 SuffixAutomaton(int n)
28 {
29     st.reserve(2 * n);
30     st.push_back(State());
31     last = 0;
32 }
33
34 void add(char ch)
35 {
36     int c = ch - 'a';
37
38     int cur = st.size();
39     st.push_back(State());
40
41     st[cur].len = st[last].len + 1;
42     st[cur].occ = 1;
43
44     int p = last;
45
46     while(p != -1 && st[p].next[c] == -1)
47     {
48         st[p].next[c] = cur;
49         p = st[p].link;
50     }
51
52     if(p == -1)

```

19.20 Substring Order II - Descenso por el Suffix Automaton (con Repetidas) (cont.)

```

53     {
54         st[cur].link = 0;
55     }
56     else
57     {
58         int q = st[p].next[c];
59
60         if(st[p].len + 1 == st[q].len)
61         {
62             st[cur].link = q;
63         }
64         else
65         {
66             int clone = st.size();
67             st.push_back(st[q]);
68
69             st[clone].len = st[p].len + 1;
70             st[clone].occ = 0;
71
72             while(p != -1 && st[p].next[c] == q)
73             {
74                 st[p].next[c] = clone;
75                 p = st[p].link;
76             }
77
78             st[q].link = clone;
79             st[cur].link = clone;
80         }
81     }
82
83     last = cur;
84 }
85 };
86
87 vector<int> buildOrder(SuffixAutomaton &sam, int n)
88 {
89     int sz = sam.st.size();
90
91     vector<int> cnt(n + 1);
92
93     for(auto &v : sam.st)
94         cnt[v.len]++;
95
96     for(int i = 1; i <= n; i++)
97         cnt[i] += cnt[i - 1];
98
99     vector<int> order(sz);
100
101     for(int i = sz - 1; i >= 0; i--)
102         order[--cnt[sam.st[i].len]] = i;
103
104     return order;
105 }
106
107 void calculateOccurrences(SuffixAutomaton &sam, vector<int> &order)
108 {
109     for(int i = (int)order.size() - 1; i > 0; i--)
110     {
111         int v = order[i];
112
113         if(sam.st[v].link != -1)
114         {
115             sam.st[sam.st[v].link].occ += sam.st[v].occ;
116         }
117     }
118 }
119
120 vector<ll> calculateDP(SuffixAutomaton &sam, vector<int> &order)
121 {
122     int sz = sam.st.size();
123
124     vector<ll> dp(sz, 0);

```

19.20 Substring Order II - Descenso por el Suffix Automaton (con Repetidas) (cont.)

```

125
126     for(int i = sz - 1; i >= 0; i--)
127     {
128         int v = order[i];
129
130         for(int c = 0; c < 26; c++)
131         {
132             int to = sam.st[v].next[c];
133
134             if(to == -1)
135                 continue;
136
137             dp[v] += sam.st[to].occ;
138             dp[v] += dp[to];
139         }
140     }
141
142     return dp;
143 }
144
145 string kthSubstring(SuffixAutomaton &sam, vector<ll> &dp, ll k)
146 {
147     string ans;
148     int v = 0;
149
150     while(true)
151     {
152         for(int c = 0; c < 26; c++)
153         {
154             int to = sam.st[v].next[c];
155
156             if(to == -1)
157                 continue;
158
159             // Todo el bloque que empieza con esta letra
160             ll block = sam.st[to].occ + dp[to];
161
162             if(k > block)
163             {
164                 k -= block;
165                 continue;
166             }
167
168             // La respuesta puede ser exactamente ans + c
169             if(k <= sam.st[to].occ)
170             {
171                 ans.push_back(char('a' + c));
172                 return ans;
173             }
174
175             // Saltamos las ocurrencias del propio prefijo
176             k -= sam.st[to].occ;
177
178             ans.push_back(char('a' + c));
179             v = to;
180
181             break;
182         }
183     }
184 }
185
186 int main()
187 {
188     ios::sync_with_stdio(false);
189     cin.tie(nullptr);
190
191     string s;
192     cin >> s;
193
194     ll k;
195     cin >> k;
196

```


20.2 Beautiful Array - Construccion Divide y Conquista

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 932 "Beautiful Array": una permutacion de 1 a n donde nunca haya $i < k < j$ con $arr[k]*2 == arr[i]+arr[j]$. Tecnica: construccion recursiva. Si un arreglo sirve para $n/2$, entonces poniendo todos los IMPARES ($2x-1$) antes de todos los PARES ($2x$) tambien sirve, porque impar + par nunca da un numero par al doble, asi que ningun trio a caballo entre los dos bloques puede fallar. Es el tipo de problema donde no hay busqueda, solo una construccion que se demuestra.

leetcode\arrays\0932_beautiful_array.py

```
1 class Solution:
2     def beautifulArray(self, n: int) -> list[int]:
3
4         if n == 1:
5             return [1]
6
7         arr = self.beautifulArray((n + 1) // 2)
8
9         impares = []
10        pares = []
11
12        for x in arr:
13            if 2 * x - 1 <= n:
14                impares.append(2 * x - 1)
15
16            if 2 * x <= n:
17                pares.append(2 * x)
18
19        return impares + pares
```

20.3 Buy Two Chocolates - Los Dos mas Baratos

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 2706 "Buy Two Chocolates": comprar dos chocolates sin quedar en rojo, devolviendo lo que sobra. Tecnica: genera todas las parejas y toma la suma minima, $O(n^2)$. Basta con los dos precios mas pequenos, que salen en una sola pasada. La pareja mas barata siempre son los dos minimos.

leetcode\arrays\2706_buy_two_chocolates.py

```
1 class Solution:
2     def buyChoco(self, prices: List[int], money: int) -> int:
3         val = []
4         for i in range(0, len(prices)):
5             for j in range(i+1, len(prices)):
6                 val.append(prices[i]+prices[j])
7         minimo = min(val)
8         if(minimo > money):
9             return money
10        else:
11            return money-minimo
```

20.4 Check If Array Is Sorted And Rotated - Ordenado y Rotado

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1752 "Check if Array Is Sorted and Rotated": decir si el arreglo es una rotacion de su version ordenada. Tecnica del codigo: busca donde arranca el minimo y compara dando la vuelta contra el arreglo ordenado. La parte enredada es escoger cual de las apariciones repetidas del minimo es el punto de rotacion. El truco de una linea: contar cuantas veces $nums[i] > nums[i+1]$ dando la vuelta; si es 0 o 1, es ordenado y rotado.

leetcode\arrays\1752_check_if_array_is_sorted_and_rotated.py

```
1 class Solution:
2     def check(self, nums: List[int]) -> bool:
3         l = sorted(nums)
4         po = 0
5         for k in range(len(nums)-1, -1, -1):
6             #print(nums[k], l[0])
7             if(nums[k]==l[0] and nums[k-1]!=l[0]):
8                 po = k
9                 break
10        #print(po)
11        po2 = nums.index(l[0])
12        cant = 0
13        f = False
```

20.4 Check If Array Is Sorted And Rotated - Ordenado y Rotado (cont.)

```
14        for h in range(po2, po+1):
15            if(nums[h]== l[0]):
16                continue
17            else:
18                f = True
19                break
20        if(f==False):
21            po = po2
22        else:
23            po = po
24            i = po
25            j = 0
26            flag = 0
27            while (j < len(nums) and flag == 0):
28                if(i==len(nums)):
29                    i=0
30                    if(nums[i] == l[j]):
31                        i+=1
32                        j+=1
33                        continue
34                else:
35                    flag = 1
36                    break
37            if(flag == 1):
38                return False
39        else:
40            return True
```

20.5 Cinema Seat Allocation - Grupos de Sillas por Fila

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1386 "Cinema Seat Allocation": maximo de familias de 4 que se pueden sentar juntas, con algunas sillas reservadas. Tecnica: en cada fila solo hay tres bloques posibles de 4 sillas contiguas sin cruzar el pasillo, 2-5, 4-7 y 6-9, y los bloques 1 y 3 son los unicos que caben a la vez. Se revisa cada fila con reservas y las n - f filas vacias aportan 2 cada una. La clave es no recorrer las n filas (n llega a 10^9), solo las que aparecen en la lista.

leetcode\arrays\1386_cinema_seat_allocation.py

```
1 class Solution:
2     def maxNumberOfFamilies(self, n: int, reservedSeats: List[List[int]]) -> int:
3         filas = {}
4         for i in range(len(reservedSeats)):
5             if(reservedSeats[i][0] in filas):
6                 filas[reservedSeats[i][0]].append(reservedSeats[i][1])
7             else:
8                 filas[reservedSeats[i][0]] = [reservedSeats[i][1]]
9         set1 = {2,3,4,5}
10        set2 = {4,5,6,7}
11        set3 = {6,7,8,9}
12        c = 0
13        f = 0
14        for clave, valor in filas.items():
15            a = [1,1,1]
16            for i in valor:
17                if(i in set1):
18                    a[0] = 0
19                if(i in set2):
20                    a[1] = 0
21                if(i in set3):
22                    a[2] = 0
23            if(a[0]==1 and a[2]==1):
24                c+=2
25            else:
26                c += min(1, sum(a))
27            f+=1
28
29        faltantes = n-f
30        c+= 2*faltantes
31
32        return c
```

20.6 Concatenation Of Array - Concatenacion

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1929

"Concatenation of Array": devolver el arreglo pegado consigo mismo. Tecnica: nums + nums. Una linea.

leetcode\arrays\1929_concatenation_of_array.py

```
1 class Solution:
2     def getConcatenation(self, nums: List[int]) -> List[int]:
3         return nums+nums
```

20.7 Construct Uniform Parity Array I - Paridad por Fuerza Bruta

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3875 "Construct Uniform Parity Array I": decir si se puede dejar todo el arreglo con la misma paridad usando restas entre elementos. Tecnica: por cada elemento busca otro con el que la resta cambie su paridad, y cuenta cuantos pueden quedar pares y cuantos impares. Es $O(n^2)$. Lo que importa es que par - impar es impar y par - par es par: la paridad del resultado sale de la paridad de los dos operandos, nada mas. La version II usa eso para bajar a $O(n)$.

leetcode\arrays\3875_construct_uniform_parity_array_i.py

```
1 class Solution:
2     def uniformArray(self, nums1: list[int]) -> bool:
3         lPar = []
4         lImpar = []
5         for i in range(len(nums1)):
6             if(nums1[i]%2==0):
7                 lPar.append(nums1[i])
8                 for j in range(len(nums1)):
9                     if(i!=j and (nums1[i]-nums1[j])%2==1):
10                        lImpar.append(nums1[i]-nums1[j])
11                        break
12             else:
13                 lImpar.append(nums1[i])
14                 for j in range(len(nums1)):
15                     if(i!=j and (nums1[i]-nums1[j])%2==0):
16                        lPar.append(nums1[i]-nums1[j])
17                        break
18
19         if(len(lPar)== len(nums1) or len(lImpar)==len(nums1)):
20             return True
21         else:
22             return False
```

20.8 Construct Uniform Parity Array II - Paridad con el Menor Impar

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3876 "Construct Uniform Parity Array II": lo mismo que el 3875 pero con arreglos grandes. Tecnica: solo hacen falta el menor par y el menor IMPAR del arreglo. Para volver impar un numero par basta restarle el menor impar, y eso solo se puede si el resultado sigue siendo positivo, de ahi el $i - \text{menor_impar} \geq 1$. Dos pasadas lineales en vez de los pares del 3875. Quedarse con el minimo de cada paridad es lo que convierte el $O(n^2)$ en $O(n)$.

leetcode\arrays\3876_construct_uniform_parity_array_ii.py

```
1 class Solution:
2     def uniformArray(self, nums1: list[int]) -> bool:
3         menor_par = float('inf')
4         posP = 0
5         menor_impar = float('inf')
6         posI = 0
7         pos = 0
8         for x in nums1:
9             if x % 2 == 0:
10                 if(menor_par > x):
11                     menor_par = x
12                 posP = pos
13             else:
14                 if(menor_impar > x):
15                     menor_impar = x
16                 posI = pos
17             pos+=1
18         #if(menor_par == float('inf') or menor_impar == float('inf')):
19         #    return False
20
```

20.8 Construct Uniform Parity Array II - Paridad con el Menor Impar (cont.)

```
21         cp = 0
22         ci = 0
23         pos = 0
24         for i in nums1:
25             if(i % 2 == 0):
26                 cp+=1
27             elif(pos != posI and i-menor_impar >= 1):
28                 cp+=1
29
30             if(i % 2 == 1):
31                 ci+=1
32             elif(pos != posI and i-menor_impar >= 1):
33                 ci+=1
34             pos+=1
35
36         if(ci == len(nums1) or cp == len(nums1)):
37             return True
38
39         return False
```

20.9 Contains Duplicate - Buscar Repetidos

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 217 "Contains Duplicate": decir si hay algun valor repetido. Tecnica: diccionario como set, y se corta en el primer repetido. Cortar temprano es la unica gracia; tambien sale con $\text{len}(\text{set}(\text{nums})) \neq \text{len}(\text{nums})$.

leetcode\arrays\0217_contains_duplicate.py

```
1 class Solution:
2     def containsDuplicate(self, nums: List[int]) -> bool:
3         l = {}
4         for i in range(len(nums)):
5             if nums[i] in l:
6                 return True
7             else:
8                 l[nums[i]] = 1
9         return False
```

20.10 Count Subarrays With Majority Element I - Contar por Bloques Iguales

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3737 "Count Subarrays With Majority Element I": cuantos subarreglos tienen a target como elemento mayoritario. Tecnica del codigo: cuenta los bloques consecutivos de target y suma las combinaciones internas de cada bloque. OJO: eso cuenta los subarreglos formados SOLO por target, no todos los que lo tienen por mayoria. Pasa la version I porque los limites son chicos y los casos de prueba lo permiten, pero el metodo no es el del enunciado. Lo correcto es mapear target a +1 y el resto a -1 y contar subarreglos con suma positiva, que es contar inversiones sobre las sumas de prefijos.

leetcode\arrays\3737_count_subarrays_with_majority_element_i.py

```
1 import math
2
3 class Solution:
4     def countMajoritySubarrays(self, nums: List[int], target: int) -> int:
5         conteo = nums.count(target)
6         valF = 0
7         valF += conteo
8         j = 0
9         while j < len(nums):
10             c = 0
11             k = j
12             if(nums[j]==target):
13                 while k < len(nums):
14                     if(nums[k]==target):
15                         c+=1
16                     else:
17                         k+=1
18                         break
19                     k+=1
20             j=k
21             for i in range(1,c+1):
22                 valF += c-i
23             j+=1
```


20.19 Find Right Interval - Buscar el Intervalo de la Derecha (cont.)

```

6         mejor = float("inf")
7         indice = -1
8
9         for inicio in ic:
10             if inicio >= fin and inicio < mejor:
11                 mejor = inicio
12                 indice = ic[inicio][0]
13
14         res.append(indice)
15
16     return res

```

20.20 Find The Highest Altitude - Suma Acumulada

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1732 "Find the Highest Altitude": la altitud maxima alcanzada, dados los desniveles. Técnica: suma de prefijos arrancando en 0 y max() al final. El 0 inicial importa porque la respuesta puede ser el punto de partida si todos los tramos son de bajada.

leetcode\arrays\1732_find_the_highest_altitude.py

```

1 class Solution:
2     def largestAltitude(self, gain: List[int]) -> int:
3         g = [0]
4         for i in range(len(gain)):
5             g.append(gain[i]+g[-1])
6
7
8     return max(g)

```

20.21 Find The Prefix Common Array Of Two Arrays - Interseccion de Prefijos

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 2657 "Find the Prefix Common Array of Two Arrays": para cada i, cuantos valores estan en los primeros i+1 de las dos permutaciones. Técnica: reconstruye los dos sets en cada paso y los cruza, o sea $O(n^2)$. Incremental es $O(n)$: se mantienen dos sets que van creciendo y en cada paso solo se revisa si el valor nuevo de A ya estaba en B (y al contrario). Reconstruir en vez de actualizar es el error tipico de este tipo de problema.

leetcode\arrays\2657_find_the_prefix_common_array_of_two_arrays.py

```

1 class Solution:
2     def findThePrefixCommonArray(self, A: List[int], B: List[int]) -> List[int]:
3         i = 0
4         c = []
5         cont = 0
6         while i < len(A):
7             set1 = set(A[0:i+1])
8             set2 = set(B[0:i+1])
9             cont = 0
10            for j in set1:
11                if(j in set2):
12                    cont+= 1
13            c.append(cont)
14            i+=1
15        return c

```

20.22 First Missing Positive - Primer Positivo Faltante

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 41 "First Missing Positive": el menor entero positivo que no esta en el arreglo. Técnica: set con todos los valores y probar 1, 2, 3, ... hasta que falte uno. OJO: el problema pide $O(1)$ de memoria extra y esto usa $O(n)$. La version buena usa el ARREGLO como marca: se manda cada valor v que este entre 1 y n a la posición v-1, y al final el primer índice i donde $\text{nums}[i] \neq i+1$ es la respuesta. La respuesta siempre esta entre 1 y n+1, y eso es lo que hace que el truco funcione.

leetcode\arrays\0041_first_missing_positive.py

```

1 class Solution:
2     def firstMissingPositive(self, nums: List[int]) -> int:
3         n = set(nums)
4         ma = max(nums)
5         if(ma<1):
6             return 1
7         for i in range(1, ma):

```

20.22 First Missing Positive - Primer Positivo Faltante (cont.)

```

8             if(i in n):
9                 continue
10            else:
11                return i
12        return ma+1

```

20.23 Hoy Many Numbers Are Smaller Than The Current Number - Fuerza Bruta $O(n^2)$

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1365 "How Many Numbers Are Smaller Than the Current Number": para cada numero, cuantos son menores. Técnica: dos ciclos anidados. Con n hasta 500 alcanza. Con un arreglo de frecuencias y sumas de prefijos sale en $O(n + \text{rango})$, que es lo que se haria si n fuera grande.

leetcode\arrays\1365_hoy_many_numbers_are_smaller_than_the_current_number.py

```

1 class Solution:
2     def smallerNumbersThanCurrent(self, nums: List[int]) -> List[int]:
3         c = []
4         for i in range(len(nums)):
5             can = 0
6             for j in range(len(nums)):
7                 if(nums[i]>nums[j]):
8                     can += 1
9
10            c.append(can)
11        return c

```

20.24 Keep Multiplying Found Values By Two - Duplicar mientras Exista

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 2154 "Keep Multiplying Found Values by Two": mientras original este en el arreglo, se duplica; devolver el valor final. Técnica: simulacion directa. El in sobre una lista es $O(n)$; con set(nums) queda $O(1)$ por vuelta.

leetcode\arrays\2154_keep_multiplying_found_values_by_two.py

```

1 class Solution:
2     def findFinalValue(self, nums: List[int], original: int) -> int:
3         if original in nums:
4             while True:
5                 original*=2
6                 if original in nums:
7                     continue
8             else:
9                 break
10        return original

```

20.25 Kth Largest Element In An Array - Counting Sort Parcial

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 215 "Kth Largest Element in an Array": el k-esimo mas grande. Técnica: frecuencias en un diccionario y barrido del maximo hacia abajo acumulando hasta pasar de k. Es $O(n + \text{rango})$. OJO: ma arranca en 0, asi que con todos los valores negativos el barrido empieza mal. Con los datos del problema no falla, pero el ma deberia inicializarse en -infinito. Con un heap de tamaño k es $O(n \log k)$, y con quickselect $O(n)$ promedio.

leetcode\arrays\0215_kth_largest_element_in_an_array.py

```

1 class Solution:
2     def findKthLargest(self, nums: list[int], k: int) -> int:
3         mi = float("inf")
4         ma = 0
5         dic = {}
6         for i in nums:
7             if(i>ma):
8                 ma = i
9             if(i<mi):
10                mi = i
11            if(i in dic):
12                dic[i]+=1
13            else:
14                dic[i]=1
15
16        c = 0
17        for i in range(ma,mi-1,-1):
18            try:
19                c += dic[i]
20                if(c>=k):

```

20.25 Kth Largest Element In An Array - Counting Sort Partial (cont.)

```
20         break
21     except:
22         continue
23     return i
```

20.26 Last Stone Weight - Simulacion con los Dos Mayores

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1046 "Last Stone Weight": choque de las dos piedras mas pesadas hasta que quede una. Tecnica: max() y remove() en un while. Es $O(n^2)$ porque cada max recorre todo. Es el ejercicio de HEAP MAXIMO: con heapq y los valores negados, cada choque cuesta $O(\log n)$ y el total queda $O(n \log n)$. Ver heap/0373 en esta misma carpeta para ese truco del signo.

leetcode\arrays\1046_last_stone_weight.py

```
1 class Solution:
2     def lastStoneWeight(self, stones: List[int]) -> int:
3         while (len(stones)>1):
4             ma1 = max(stones)
5             stones.remove(ma1)
6             ma2 = max(stones)
7             stones.remove(ma2)
8             if(ma1==ma2):
9                 continue
10            else:
11                stones.append(abs(ma1-ma2))
12        if(len(stones)==0):
13            return 0
14        else:
15            return stones[0]
```

20.27 Left And Rigth Sum Differences - Sumas a Izquierda y Derecha

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 2574 "Left and Right Sum Differences": para cada posicion, el valor absoluto de la suma de lo que hay a su izquierda menos lo de su derecha. Tecnica: sum() de los dos pedazos en cada posicion, o sea $O(n^2)$. Con sumas de prefijos es $O(n)$: la izquierda es pref[i] y la derecha es total - pref[i+1]. Es el ejemplo mas simple de para que sirven los prefijos.

leetcode\arrays\2574_left_and_rigth_sum_differences.py

```
1 class Solution:
2     def leftRightDifference(self, nums: List[int]) -> List[int]:
3         f = []
4         for i in range(len(nums)):
5             ll = sum(nums[0:i])
6             rr = sum(nums[i+1:len(nums)])
7             f.append(abs(ll-rr))
8         return f
```

20.28 Magical String - Secuencia Autodescriptiva

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 481 "Magical String": la cadena de 1 y 2 donde los grupos de iguales tienen las longitudes que la cadena misma describe; contar los 1 entre los primeros n. Tecnica: se construye hacia adelante leyendo con un puntero chi la cantidad que toca escribir, y una bandera alterna si el bloque es de 1 o de 2. La cadena se va alimentando de si misma, y ese puntero que persigue al final es toda la idea. El if de i <= n es para no contar los 1 que se pasan del limite.

leetcode\arrays\0481_magical_string.py

```
1 class Solution:
2     def magicalString(self, n: int) -> int:
3         chi = 2
4         cant = 1
5         s = "122"
6         F = True
7         i = 3
8         while i < n:
9             if(F):
10                can = int(s[chi])
11                chi+=1
12                so = "1"*can
13                s += so
14                i += can
15            if(i<=n):
```

20.28 Magical String - Secuencia Autodescriptiva (cont.)

```
6                 cant += can
7             else:
8                 cant += 1
9             F= False
10            else:
11                can = int(s[chi])
12                chi+=1
13                so = "2"*can
14                s += so
15                i += can
16                F= True
17
18        return cant
```

20.29 Majority Element - Elemento Mayoritario

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 169 "Majority Element": el que aparece mas de n/2 veces. Tecnica: diccionario de frecuencias y se toma el maximo. $O(n)$ tiempo, $O(n)$ memoria. El algoritmo de Boyer-Moore lo hace con $O(1)$ de memoria: se lleva un candidato y un contador, se suma si coincide y se resta si no; cuando llega a 0 se cambia de candidato. Vale la pena conocerlo, aparece en maraton.

leetcode\arrays\0169_majority_element.py

```
1 class Solution:
2     def majorityElement(self, nums: List[int]) -> int:
3         l = {}
4         for i in range(len(nums)):
5             if (nums[i] in l):
6                 l[nums[i]]+=1
7             else:
8                 l[nums[i]]=1
9         mas = 0
10        v = 0
11        for i,j in l.items():
12            if(j > mas):
13                mas = j
14                v = i
15
16        return v
```

20.30 Max Consecutive Ones - Racha Maxima

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 485 "Max Consecutive Ones": la racha mas larga de unos. Tecnica: un contador que se reinicia en cada cero. El if del final es porque si el arreglo termina en 1, la ultima racha nunca se comparo adentro del ciclo. Ese caso del final es el error clasico de todos los problemas de rachas.

leetcode\arrays\0485_max_consecutive_ones.py

```
1 class Solution:
2     def findMaxConsecutiveOnes(self, nums: List[int]) -> int:
3         ma = 0
4         co = 0
5         for i in range(len(nums)):
6             if(nums[i]==1):
7                 co += 1
8             else:
9                 if(co>ma):
10                    ma = co
11                co = 0
12            if(nums[-1]== 1 and co>ma):
13                ma = co
14        return ma
```

20.31 Maximum Ice Cream Bars - Greedy del mas Barato Primero

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1833 "Maximum Ice Cream Bars": cuantos helados se pueden comprar con las monedas. Técnica: ordenar por precio y comprar del mas barato hasta que no alcance. Es el greedy correcto porque todos los helados valen lo mismo para la respuesta (solo cuenta la cantidad).

leetcode\arrays\1833_maximum_ice_cream_bars.py

```
1 class Solution:
2     def maxIceCream(self, costs: List[int], coins: int) -> int:
3         s = sorted(costs)
4         c = 0
5         cant = 0
6         for i in s:
7             c+=i
8             if(c > coins):
9                 break
10
11
12         cant+=1
13         return cant
```

20.32 Maximum Product Of Two Elements In An Array - Los Dos Mayores

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1464 "Maximum Product of Two Elements in an Array": maximizar (a-1)*(b-1). Técnica: ordenar y tomar los dos ultimos. Como los valores son positivos, los dos mayores dan el producto mayor sin tener que probar nada. Con una pasada guardando los dos maximos sale en O(n) sin ordenar.

leetcode\arrays\1464_maximum_product_of_two_elements_in_an_array.py

```
1 class Solution:
2     def maxProduct(self, nums: List[int]) -> int:
3         t = sorted(nums)
4         return (t[len(t)-1]-1)*(t[len(t)-2]-1)
```

20.33 Maximum Total Subarray Value I - Observacion que Colapsa el Problema

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3689 "Maximum Total Subarray Value I": escoger k subarreglos y sumar (max - min) de cada uno; maximizar. Técnica: tres lineas. El mejor subarreglo posible es el arreglo COMPLETO, que da max - min, y como los subarreglos se pueden repetir, la respuesta es (max - min) * k. Otra vez: leer bien antes de programar. Aquí no hay que escoger nada.

leetcode\arrays\3689_maximum_total_subarray_value_i.py

```
1 class Solution:
2     def maxTotalValue(self, nums: List[int], k: int) -> int:
3         ma = max(nums)
4         mi = min(nums)
5         p = ma - mi
6         return p*k
```

20.34 Median Of Two Sorted Arrays - Avanzar hasta la Mitad

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 4 "Median of Two Sorted Arrays": la mediana de dos arreglos ordenados. Técnica: dos punteros que avanzan como en un merge, pero sin construir el arreglo fusionado, solo contando hasta llegar a la posición del medio y guardando el anterior para el caso par. Es O(n+m). El problema en realidad pide O(log(n+m)), que se logra con binaria sobre el punto de corte de uno de los dos arreglos; esa versión es bastante mas difícil de escribir bien.

leetcode\arrays\0004_median_of_two_sorted_arrays.py

```
1 class Solution:
2     def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
3
4         i = 0
5         j = 0
6         val = 0
7         if (len(nums1) == 0 and len(nums2)==0):
8             actual = 0
9         elif(len(nums1)== 0):
10             actual = nums2[j]
11             j+=1
12         elif(len(nums2)==0):
13             actual = nums1[i]
```

20.34 Median Of Two Sorted Arrays - Avanzar hasta la Mitad (cont.)

```
14         i+=1
15
16         elif nums1[i] < nums2[j]:
17             actual = nums1[i]
18             i += 1
19         else:
20             actual = nums2[j]
21             j += 1
22
23         p = (len(nums1) + len(nums2)) // 2
24
25         while True:
26             ant = actual
27
28             if i < len(nums1) and j < len(nums2):
29
30                 if nums1[i] < nums2[j]:
31
32                     if actual <= nums1[i]:
33                         actual = nums1[i]
34                         i += 1
35
36                 else:
37
38                     if actual <= nums2[j]:
39                         actual = nums2[j]
40                         j += 1
41
42                 elif i < len(nums1):
43
44                     if actual <= nums1[i]:
45                         actual = nums1[i]
46                         i += 1
47
48                 elif j < len(nums2):
49
50                     if actual <= nums2[j]:
51                         actual = nums2[j]
52                         j += 1
53
54                 val += 1
55
56             if val >= p:
57                 break
58
59             if (len(nums1) + len(nums2)) % 2 == 1:
60                 return actual
61
62         else:
63
64
65
66         return (actual+ant)/2
```

20.35 Merge Intervals - Fusionar Intervalos

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 56 "Merge Intervals": unir los intervalos que se traslapan. Técnica: ordenar por inicio y barrer con un intervalo abierto; si el siguiente empieza antes de que el actual termine, se estira el final al maximo, y si no, se cierra y se abre uno nuevo. Ordenar por inicio es lo que garantiza que solo hay que mirar el intervalo anterior. Es el molde de casi todos los problemas de intervalos.

leetcode\arrays\0056_merge_intervals.py

```
1 class Solution:
2     def merge(self, intervals: List[List[int]]) -> List[List[int]]:
3         a= sorted(intervals)
4         final = []
5         ini = a[0][0]
6         fin = a[0][1]
7         for i in a:
8             if(ini == i[0] and fin == i[1]):
9                 continue
```

20.35 Merge Intervals - Fusionar Intervalos (cont.)

```

1         else:
2             if(i[0]>= ini and i[0]<= fin):
3                 if(i[1]>fin):
4                     fin = i[1]
5             else:
6                 final.append([ini,fin])
7                 ini = i[0]
8                 fin = i[1]
9         final.append([ini,fin])
10        return final

```

20.36 Minimum Absolute Difference - Diferencia Minima en Ordenado

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1200 "Minimum Absolute Difference": todos los pares con la diferencia absoluta minima. Tecnica: ordenar. La diferencia minima SIEMPRE esta entre dos vecinos del arreglo ordenado, así que basta una pasada para hallarla y otra para recolectar los pares que la alcanzan. Esa observacion es la que convierte un problema de pares ($O(n^2)$) en uno lineal.

leetcode/arrays/1200_minimum_absolute_difference.py

```

1 class Solution:
2     def minimumAbsDifference(self, arr: List[int]) -> List[List[int]]:
3         a = sorted(arr)
4         mi = float("inf")
5         for i in range(1,len(arr)):
6             if(a[i]-a[i-1] < mi):
7                 mi = a[i]-a[i-1]
8
9         fin = []
10        for i in range(1,len(arr)):
11            if(a[i]-a[i-1] == mi):
12                fin.append([a[i-1], a[i]])
13        return fin

```

20.37 Minimum Cost Of Buying Candies With Discount - Greedy de Tercias

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 2144 "Minimum Cost of Buying Candies With Discount": por cada dos dulces comprados, el tercero (mas barato de los tres) es gratis. Tecnica: ordenar y recorrer DE MAYOR A MENOR pagando dos y saltando el tercero. Así el regalo siempre cae en el dulce mas caro posible, que es lo que minimiza el total. El contador t es el que lleva la cuenta de dos pagados, uno gratis.

leetcode/arrays/2144_minimum_cost_of_buying_candies_with_discount.py

```

1 class Solution:
2     def minimumCost(self, cost: List[int]) -> int:
3         cost = sorted(cost)
4         c = 0
5         i = len(cost)-1
6         t = 0
7         while i>=0:
8             if t==2:
9                 t=0
10                i-=1
11                continue
12            c+=cost[i]
13            t += 1
14            i-=1
15        return(c)

```

20.38 Minimum Element After Replacement With Digit Sum - Suma de Digitos

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3300 "Minimum Element After Replacement With Digit Sum": el minimo despues de cambiar cada numero por la suma de sus digitos. Tecnica: pasar a string, sumar los digitos y quedarse con el minimo. Una pasada.

leetcode/arrays/3300_minimum_element_after_replacement_with_digit_sum.py

```

1 class Solution:
2     def minElement(self, nums: List[int]) -> int:
3         me = 9999999999999
4         for i in nums:
5             h = str(i)
6             ca = 0

```

20.38 Minimum Element After Replacement With Digit Sum - Suma de Digitos (cont.)

```

7         for j in h:
8             ca += int(j)
9         if(ca<me):
10            me = ca
11        return me

```

20.39 Missing Number - Numero Faltante

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 268 "Missing Number": el numero de 0 a n que falta. Tecnica: ordenar y buscar la primera posicion donde $nums[i] \neq i$. Es $O(n \log n)$ por el sort. Sale en $O(n)$ sin ordenar de dos formas: la suma de Gauss $n*(n+1)/2$ menos la suma real, o el XOR de todos los indices contra todos los valores.

leetcode/arrays/0268_missing_number.py

```

1 class Solution:
2     def missingNumber(self, nums: List[int]) -> int:
3         nu = len(nums)
4         nums.sort()
5         for i in range(0,nu):
6             if(i != nums[i]):
7                 continue
8         else:
9             return i
10        return nu

```

20.40 Pascal's Triangle - Triangulo de Pascal

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 118 "Pascal's Triangle": las primeras numRows filas del triangulo. Tecnica: cada fila sale de la anterior sumando vecinos, con un 1 en cada punta. Es la recurrencia $C(n,k) = C(n-1,k-1) + C(n-1,k)$ construida hacia abajo, sin factoriales y sin division. Así se construye la tabla de binomiales cuando hay que consultarla muchas veces.

leetcode/arrays/0118_pascal's_triangle.py

```

1 class Solution:
2     def generate(self, numRows: int) -> List[List[int]]:
3         numFilas = numRows
4
5         matriz = []
6
7         fila = [1]
8
9         cont = 0
10
11        while cont < numFilas:
12
13            matriz.append(fila)
14
15            nueva = [1]
16
17            for i in range(len(fila) - 1):
18
19                nueva.append(fila[i] + fila[i + 1])
20
21            nueva.append(1)
22
23            fila = nueva
24            cont+=1
25
26
27
28        return(matriz)

```

20.41 Plus One - Sumar Uno a un Numero por Digitos

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 66 "Plus One": sumar 1 a un numero dado como arreglo de digitos. Tecnica: aca arma el entero completo multiplicando por potencias de 10, le suma 1 y lo vuelve a partir en digitos. En Python funciona porque los enteros son de tamaño ilimitado. En C++ o Java esto se desborda y hay que hacerlo con acarreo de atras hacia adelante; ese es el metodo que vale la pena tener presente.

leetcode\arrays\0066_plus_one.py

```
1 class Solution:
2     def plusOne(self, digits: List[int]) -> List[int]:
3         c = len(digits)-1
4         v = 0
5         for i in digits:
6             v+= (i*10**c)
7             c-=1
8         v+=1
9         va = str(v)
10        l = []
11        for i in va:
12            l.append(int(i))
13        return l
```

20.42 Rank Transform Of An Array - Compresion de Coordenadas

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1331 "Rank Transform of an Array": reemplazar cada valor por su puesto en el orden. Tecnica: sorted(set(arr)) y un diccionario valor -> puesto. Eso es exactamente COMPRESION DE COORDENADAS, que es la herramienta para meter valores gigantes en un arreglo o un Fenwick. El notebook tiene la version en C++ (sort + unique + lower_bound).

leetcode\arrays\1331_rank_transform_of_an_array.py

```
1 class Solution:
2     def arrayRankTransform(self, arr: List[int]) -> List[int]:
3         k = sorted(list(set(arr)))
4         dic = {}
5
6         for i in range(len(k)):
7             dic[k[i]] = i+1
8         f = []
9         for i in range(len(arr)):
10            f.append(dic[arr[i]])
11        return f
```

20.43 Reduction Operations To Make The Array Elements Equal - Contar por Escalones

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1887 "Reduction Operations to Make the Array Elements Equal": operaciones para igualar todo al minimo, bajando cada vez el maximo al siguiente valor distinto. Tecnica: ordenar y en cada cambio de valor sumar cuantos elementos quedan a la derecha. Cada uno de esos va a tener que bajar ese escalon. Contar por escalones en vez de simular es lo que lo vuelve O(n log n).

leetcode\arrays\1887_reduction_operations_to_make_the_array_elements_equal.py

```
1 class Solution:
2     def reductionOperations(self, nums: List[int]) -> int:
3         a = sorted(nums)
4         c = 0
5         k = a[0]
6         for i in range(1,len(a)):
7             if(a[i]!=a[i-1]):
8                 c += len(a)-i
9         return c
```

20.44 Remove Covered Intervals - Intervalos Cubiertos

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1288 "Remove Covered Intervals": cuantos intervalos quedan despues de borrar los que estan contenidos en otro. Tecnica: comparar todos los pares, O(n^2). La version buena: ordenar por inicio creciente y, en empate, por final DECRECIENTE. Asi basta recordar el mayor final visto, y todo intervalo con final menor o igual esta cubierto. O(n log n) y sin comparar pares. Ese criterio de desempate es el detalle que la gente olvida.

leetcode\arrays\1288_remove_covered_intervals.py

```
1 class Solution:
2     def removeCoveredIntervals(self, intervals: List[List[int]]) -> int:
3         n = len(intervals)
4         deleted = set()
5         for i in range(len(intervals)):
6             for j in range(len(intervals)):
7                 if i != j:
8                     if intervals[i][0] <= intervals[j][0] and intervals[i][1] >= intervals[j][1] and
9                       d (intervals[j][0], intervals[j][1]) not in deleted:
10                        n -= 1
11                        deleted.add((intervals[j][0], intervals[j][1]))
12        return n
```

20.45 Remove Duplicates From Sorted Array - Quitar Repetidos en Ordenado

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 26 "Remove Duplicates from Sorted Array": dejar cada valor una vez, en el sitio. Tecnica: recorrer DE ATRAS hacia adelante y hacer pop cuando el actual es igual al anterior. Ir al reves es lo que permite borrar sin danar los indices que faltan. pop(i) es O(n), asi que en total es O(n^2). Con dos punteros (uno de escritura y uno de lectura) queda en O(n) sin borrar nada.

leetcode\arrays\0026_remove_duplicates_from_sorted_array.py

```
1 class Solution:
2     def removeDuplicates(self, nums: List[int]) -> int:
3         cont = 0
4         for i in range(len(nums)-1,0,-1):
5             if(nums[i]==nums[i-1]):
6                 nums.pop(i)
7         return len(nums)
```

20.46 Remove Element - Filtrar en el Sitio

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 27 "Remove Element": quitar todas las apariciones de val y devolver cuantos quedaron. Tecnica: remove() en un while hasta que no quede ninguno, y despues rellena el final. OJO: rellena con el string "_" en un arreglo de enteros. Funciona porque el juez solo revisa los primeros k elementos, pero es un tipo mezclado. Lo limpio es el puntero de escritura: se recorre el arreglo y se copia hacia adelante solo lo que no es val.

leetcode\arrays\0027_remove_element.py

```
1 class Solution:
2     def removeElement(self, nums: List[int], val: int) -> int:
3         cont2 = 0
4         cont = 0
5         i = 0
6         while i< len(nums):
7             if(nums[i]==val):
8                 nums.remove(val)
9                 cont+=1
10                continue
11            cont2+=1
12            i+=1
13        i=0
14        while i<cont:
15            nums.append("_")
16            i+=1
17
18        return cont2
```

20.47 Search Insert Position - Búsqueda Lineal

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 35 "Search Insert Position": donde esta el target, o donde habría que insertarlo. Técnica: recorrer hasta el primer valor mayor o igual. Es $O(n)$. El problema pide $O(\log n)$: es binaria, y este es justo el lower_bound de C++. Vale la pena hacerlo con binaria porque es el esqueleto que despues se usa para todo.

leetcode/arrays/0035_search_insert_position.py

```
1 class Solution:
2     def searchInsert(self, nums: List[int], target: int) -> int:
3         for i in range(len(nums)):
4             if(nums[i]==target or nums[i]>target):
5                 return i
6         return len(nums)
```

20.48 Set Mismatch - Repetido y Faltante

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 645 "Set Mismatch": en un arreglo de 1 a n, cual valor se duplico y cual falta. Técnica: ordenar y mirar los vecinos. Si dos son iguales, ese es el repetido; si la diferencia es mayor que 1, el faltante es el de la mitad. Los if del final cubren cuando falta el 1 o el n, que no tienen vecino de un lado. Sin ordenar tambien sale con las sumas: la suma real menos la de Gauss da repetido - faltante.

leetcode/arrays/0645_set_mismatch.py

```
1 class Solution:
2     def findErrorNums(self, nums: List[int]) -> List[int]:
3         nums = sorted(nums)
4         f = []
5         r = 0
6         fa = 0
7         for i in range(1,len(nums)):
8             if(nums[i]== nums[i-1]):
9                 r = nums[i]
10            elif(nums[i]-1!= nums[i-1]):
11                fa = nums[i]-1
12
13        if(fa == 0):
14            if(nums[0]== 1):
15                fa = nums[0]+1
16            else:
17                fa = 1
18
19        f = [r,fa]
20        return f
```

20.49 Shuffle The Array - Intercalar Dos Mitades

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1470 "Shuffle the Array": intercalar la primera mitad con la segunda. Técnica: cortar por la mitad y meter alternando. Directo.

leetcode/arrays/1470_shuffle_the_array.py

```
1 class Solution:
2     def shuffle(self, nums: List[int], n: int) -> List[int]:
3         l = []
4         nums2 = nums[len(nums)//2:len(nums)]
5
6         for i in range(len(nums)//2):
7             l.append(nums[i])
8             l.append(nums2[i])
9         return l
```

20.50 Single Number - El que Aparece Una Vez

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 136 "Single Number": el unico elemento que no esta repetido. Técnica: diccionario de frecuencias y se busca el que da 1. El truco elegante es el XOR de todo el arreglo: $a \oplus a = 0$, así que los pares se cancelan y queda el impar. Es $O(1)$ de memoria y una sola línea; ese es el que piden.

leetcode/arrays/0136_single_number.py

```
1 class Solution:
2     def singleNumber(self, nums: List[int]) -> int:
3         dic = {}
4         for i in nums:
```

20.50 Single Number - El que Aparece Una Vez (cont.)

```
5         if(i in dic):
6             dic[i]+=1
7         else:
8             dic[i] = 1
9         for clave,val in dic.items():
10            if(val == 1):
11                return clave
```

20.51 Smallest Index With Digit Sum Equal To Index - Suma de Dígitos

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3550 "Smallest Index With Digit Sum Equal to Index": el primer indice donde la suma de dígitos del valor es igual al indice. Técnica: `sum(map(int, str(x)))` por elemento, que es la forma corta de sumar dígitos en Python.

leetcode/arrays/3550_smallest_index_with_digit_sum_equal_to_index.py

```
1 class Solution:
2     def smallestIndex(self, nums: List[int]) -> int:
3         for i in range(len(nums)):
4             val = sum(list(map(int,str(nums[i]))))
5             if(val == i):
6                 return i
7         return -1
```

20.52 Smallest Missing Integer Greater Than Sequential Prefix Sum - Prefijo Secuencial

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 2996 "Smallest Missing Integer Greater Than Sequential Prefix Sum": se suma el prefijo mas largo donde cada numero es el anterior mas 1, y se busca el menor entero ausente que sea al menos esa suma. Técnica: una pasada para medir el prefijo secuencial y sumar, y despues subir de uno en uno hasta encontrar un valor que no este en el set. OJO: deja un print(c).

leetcode/arrays/2996_smallest_missing_integer_greater_than_sequential_prefix_sum.py

```
1 class Solution:
2     def missingInteger(self, nums: List[int]) -> int:
3         c = [nums[0]]
4         v = nums[0]
5         i = 1
6         while i<len(nums):
7             if(nums[i]-1 == nums[i-1]):
8                 v += nums[i]
9             else:
10                c.append(v)
11                break
12            i+=1
13        c.append(v)
14        print(c)
15        mas = max(c)
16        k = set(nums)
17        while True:
18            if(mas not in k):
19                return mas
20            mas += 1
```

20.53 Smallest Missing Multiple Of K - Buscar el Primer Multiplo Ausente

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3718 "Smallest Missing Multiple of K": el menor multiplo de k que no esta en el arreglo. Técnica: set con los valores y probar k, 2k, 3k, ... hasta que falte uno. El ciclo prueba todos los enteros y filtra con $m \% k \neq 0$; saltando de k en k daría lo mismo y k veces mas rapido.

leetcode/arrays/3718_smallest_missing_multiple_of_k.py

```
1 class Solution:
2     def missingMultiple(self, nums: List[int], k: int) -> int:
3         t = set(nums)
4         m = 1
5         while True:
6             if(m%k==0 and m not in t):
7                 return m
8             m+=1
```

20.54 Smallest Stable Index I - Maximo por Prefijo y Minimo por Sufijo

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3909 "Smallest Stable Index I": la version chica del 3904. Tecnica: la misma, pero guardando los dos arreglos completos, maximos por prefijo y minimos por sufijo, y comparandolos posición por posición. Se lee mas facil que la version II. OJO: usa insert(0, ...) para llenar el arreglo de minimos, que es O(n) cada vez y deja el total en O(n^2). Con append y luego [: -1] queda lineal.

leetcode\arrays\3909_smallest_stable_index_i.py

```
1 class Solution:
2     def firstStableIndex(self, nums: list[int], k: int) -> int:
3         mayor = 0
4         mayores = []
5         for i in nums:
6             if i > mayor:
7                 mayor = i
8             mayores.append(mayor)
9         nums2 = nums[::-1]
10        menor = float('inf')
11        menores = []
12        for i in nums2:
13            if i < menor:
14                menor = i
15            menores.insert(0, menor)
16
17        for i in range(len(nums)):
18            if(mayores[i]-menores[i] <= k):
19                return i
20        return -1
```

20.55 Smallest Stable Index II - Maximo por Prefijo y Minimo por Sufijo

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3904 "Smallest Stable Index II": el primer indice donde (maximo del prefijo) menos (minimo del sufijo) no pasa de k. Tecnica: se precalcu el minimo de cada SUFIJO recorriendo de derecha a izquierda, y despues se barre de izquierda a derecha llevando el maximo del prefijo en una variable. Con eso cada indice se responde en O(1). Ese par prefijo-maximo / sufijo-minimo es una herramienta que aparece muchisimo. OJO: menores queda al revés (se lleno con append recorriendo hacia atras), de ahi el indice len(menores)-1-i, que es facil de equivocar.

leetcode\arrays\3904_smallest_stable_index_ii.py

```
1 class Solution:
2     def firstStableIndex(self, nums: list[int], k: int) -> int:
3         menor = float('inf')
4         menores = []
5         for i in range(len(nums)-1,-1,-1):
6             if nums[i] < menor:
7                 menor = nums[i]
8             menores.append(menor)
9
10        mayor = 0
11        for i in range(len(nums)):
12            if nums[i] > mayor:
13                mayor = nums[i]
14            if(mayor-menores[len(menores)-1-i] <= k):
15                return i
16        return -1
```

20.56 Summary Ranges - Agrupar Rangos Consecutivos

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 228 "Summary Ranges": resumir un arreglo ordenado en rangos como "0->2". Tecnica: barrer y cortar cada vez que el siguiente no es el actual mas 1. El try/except es lo que maneja el final del arreglo, donde nums[i+1] se sale. Depender del except para cerrar el ultimo rango es fragil; queda mas claro con un for hasta n-1 y el cierre afuera del ciclo.

leetcode\arrays\0228_summary_ranges.py

```
1 class Solution:
2     def summaryRanges(self, nums: List[int]) -> List[str]:
3
4
5
6         resultado = []
7
```

20.56 Summary Ranges - Agrupar Rangos Consecutivos (cont.)

```
8         if len(nums) == 0:
9
10            return ([])
11
12        else:
13
14            a = nums[0]
15
16            try:
17
18                for i in range(len(nums)):
19
20                    b = nums[i + 1]
21
22                    if b == nums[i] + 1:
23
24                        continue
25
26                    else:
27
28                        if a == nums[i]:
29
30                            resultado.append(str(a))
31
32                        else:
33
34                            resultado.append(f"{a}->{nums[i]}")
35
36                        a = b
37
38            except:
39
40                if a == nums[-1]:
41
42                    resultado.append(str(a))
43
44                else:
45
46                    resultado.append(f"{a}->{nums[-1]}")
47
48            return (resultado)
```

20.57 Two Sum - Fuerza Bruta O(n^2)

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1 "Two Sum": los indices de los dos numeros que suman target. Tecnica: dos ciclos anidados probando todas las parejas. La version buena es un diccionario valor -> indice en una sola pasada: para cada numero se pregunta si target - numero ya se vio. Eso es O(n) y es el patron que hay que tener de memoria.

leetcode\arrays\0001_two_sum.py

```
1 class Solution:
2     def twoSum(self, nums: List[int], target: int) -> List[int]:
3         for i in range(len(nums)):
4             for j in range(len(nums)):
5                 if((nums[i]+nums[j])==target and i!=j):
6                     return [i, j]
```

20.58 Valid Mountain Array - Verificar Sube y Baja

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 941 "Valid Mountain Array": decir si el arreglo sube estrictamente y luego baja estrictamente. Tecnica: una bandera subiendo que solo puede cambiar una vez. Si vuelve a subir despues de haber bajado, no es montana; si alguna diferencia es 0, tampoco (tiene que ser estricto). Los casos que hay que cuidar son los que el codigo pone al inicio y al final: menos de 3 elementos, y que nunca haya bajado.

leetcode\arrays\0941_valid_mountain_array.py

```
1 class Solution:
2     def validMountainArray(self, arr: List[int]) -> bool:
3         subiendo = True
4         if len(arr) < 3:
5             return False
```

20.58 Valid Mountain Array - Verificar Sube y Baja (cont.)

```
6         if arr[0] > arr[1]:
7             return False
8         for i in range(1, len(arr)):
9             diff = arr[i] - arr[i-1]
10            if diff == 0:
11                return False
12            elif subiendo and diff < 0:
13                subiendo = False
14            elif not subiendo and diff > 0:
15                return False
16        if subiendo:
17            return False
18        return True
```

21 - Hub LeetCode: Binary Search

21.1 Binary Search - Búsqueda Binaria

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 704 "Binary Search": buscar un valor en un arreglo ordenado. Técnica: la binaria de libro con $ini \leq fin$ y los saltos $mitad+1$ / $mitad-1$. Es la plantilla mas corta y la que hay que tener en la cabeza; los dos archivos anteriores son la version complicada del mismo esqueleto.

leetcode\binary_search\0704_binary_search.py

```
1 class Solution:
2     def search(self, nums: List[int], target: int) -> int:
3         ini = 0
4         fin = len(nums)-1
5         while ini <= fin:
6             mitad = (ini+fin)//2
7             if(nums[mitad]==target):
8                 return mitad
9             elif(nums[mitad]<target):
10                ini = mitad+1
11            else:
12                fin = mitad-1
13        return -1
```

21.2 Count Elements With At Least K Greater Values - Ordenar y Buscar

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3759 "Count Elements With At Least K Greater Values": cuantos elementos tienen al menos k elementos estrictamente mayores. Técnica: ordenar y para cada posición buscar con binaria el primer valor mayor que el actual; si desde ahí quedan k o mas, cuenta. El break cuando faltan k o menos elementos es la poda. Ordenado, tambien sale de un tirón: cuenta cuantos hay estrictamente mayores por frecuencias.

leetcode\binary_search\3759_count_elements_with_at_least_k_greater_values.py

```
1 class Solution:
2     def countElements(self, nums: List[int], k: int) -> int:
3         if(k==0):
4             return len(nums)
5         nums.sort()
6         contF = 0
7         for i in range(len(nums)):
8             cont=0
9             if(len(nums)-i <= k):
10                break
11            else:
12                j = i
13                fi = len(nums)-1
14                while j <= fi:
15                    mitad = (j+fi)//2
16                    if(nums[i]<nums[mitad]):
17                        if(len(nums)-mitad >= k):
18                            contF+=1
19                            break
20                else:
21                    fi = mitad -1
22            else:
23                j = mitad + 1
24        return contF
```

21.3 Find Minimum In Rotated Sorted Array - Búsqueda en Arreglo Rotado

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 153 "Find Minimum in Rotated Sorted Array": el mínimo de un ordenado y rotado. Técnica: binaria comparando el medio contra los extremos. Si $nums[ini] > nums[m]$ el punto de rotación está a la izquierda; si $nums[fin] < nums[m]$ está a la derecha; si no hay desorden el trozo ya está ordenado y el mínimo es $nums[ini]$. La versión canónica es más corta: while $ini < fin$, si $nums[m] > nums[fin]$ entonces $ini = m+1$, si no $fin = m$; al salir $nums[ini]$ es el mínimo.

leetcode\binary_search\0153_find_minimum_in_rotated_sorted_array.py

```
1 class Solution:
2     def findMin(self, nums: List[int]) -> int:
3         n = len(nums)
4         ini = 0
5         fin = n-1
6         while (ini<fin):
7             m = (fin+ini)//2
8             if(nums[ini]>nums[m]):
9                 fin = m
10            elif(nums[fin]<nums[m]):
11                ini = m
12            else:
13                break
14
15            if(fin-ini==1):
16                break
17            if(nums[ini] > nums[fin]):
18                return (nums[fin])
19        else:
20            return(nums[ini])
```

21.4 Peak Index In A Mountain Array - Binaria sobre Monotonía

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 852 "Peak Index in a Mountain Array": la posición del pico de un arreglo que sube y luego baja. Técnica: binaria comparando con el vecino. Si $arr[m] > arr[m+1]$ el pico está en m o antes; si $arr[m] > arr[m-1]$ está en m o después. No hace falta que el arreglo esté ordenado, basta que la condición sea monótona, y esa es la idea que se lleva uno. OJO: los dos if no son excluyentes y $arr[mitad-1]$ con $mitad = 0$ lee el último elemento (los índices negativos de Python no dan error). Con el arreglo del problema no falla, pero el invariante es frágil.

leetcode\binary_search\0852_peak_index_in_a_mountain_array.py

```
1 class Solution:
2     def peakIndexInMountainArray(self, arr: List[int]) -> int:
3         ini = 0
4         fin = len(arr)-1
5         while ini < fin:
6             mitad = (ini+fin)//2
7             if(arr[mitad]>arr[mitad+1]):
8                 fin = mitad
9             if(arr[mitad]>arr[mitad-1]):
10                ini = mitad
11        return mitad
```

21.5 Search In Rotated Sorted Array - Búsqueda en Arreglo Rotado

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 33 "Search in Rotated Sorted Array": buscar un valor en un arreglo ordenado que fue rotado. Técnica del código: binaria con casos, y cuando no puede decidir de qué lado está el objetivo encoge los dos extremos de uno en uno. OJO: ese $ini += 1$ / $fin -= 1$ hace que en el peor caso sea $O(n)$, no $O(\log n)$. Además deja un print adentro del ciclo. La forma limpia: mirar si $nums[ini] \leq nums[m]$ para saber cuál mitad está ordenada, y ahí decidir con un solo if.

leetcode\binary_search\0033_search_in_rotated_sorted_array.py

```
1 class Solution:
2     def search(self, nums: List[int], target: int) -> int:
3         ini = 0
4         fin = len(nums)-1
5         if(nums[ini]==target):
6             return ini
7         elif(nums[fin]==target):
8             return fin
9
10        ant = 0
```

21.5 Search In Rotated Sorted Array - Búsqueda en Arreglo Rotado (cont.)

```

11     ant2 = ini
12     ant3 = fin
13     m=0
14     while ini < fin:
15         m = (fin+ini)//2
16         print(ini,fin,m)
17         if(nums[m]==target or nums[ini]== target or nums[fin]==target):
18             break
19         if(target>nums[ini] and target<nums[m]):
20             fin = m
21         elif(target>nums[m] and target<nums[fin]):
22             ini = m
23         else:
24             ini += 1
25             fin -= 1
26
27     if(ant2 == ini and ant3 == fin):
28         break
29
30     ant = m
31     ant2 = ini
32     ant3 = fin
33     if(nums[m]==target):
34         return m
35     elif(nums[ini]==target):
36         return ini
37     elif(nums[fin]==target):
38         return fin
39     else:
40         return -1

```

22 - Hub LeetCode: Binary Tree

22.1 Construct Binary Tree From Inorder And Postorder Traversal - Divide y Conquista sobre Rec...

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 106 "Construct Binary Tree from Inorder and Postorder Traversal": reconstruir el árbol a partir de sus recorridos inorden y postorden. Técnica: el ÚLTIMO de postorden es la raíz. Se busca en inorden y eso parte los dos recorridos en izquierda y derecha, y se repite. La cantidad de nodos de la izquierda es lo que dice donde cortar el postorden. OJO: inorder.index() es O(n), así que en el peor caso queda O(n^2). Con un diccionario valor -> posición baja a O(n).

leetcode\binary_tree\0106_construct_binary_tree_from_inorder_and_postorder_traversal.py

```

1 # Definition for a binary tree node.
2 # class TreeNode:
3 #     def __init__(self, val=0, left=None, right=None):
4 #         self.val = val
5 #         self.left = left
6 #         self.right = right
7
8 class Solution:
9
10     def construir(self, inorder, postorder):
11         if len(inorder) == 0:
12             return None
13         padre = postorder[-1]
14         nodo = TreeNode(padre)
15
16         pos = inorder.index(padre)
17
18         inorder_izq = inorder[:pos]
19         inorder_der = inorder[pos + 1:]
20         cantidad_izq = len(inorder_izq)
21
22         postorder_izq = postorder[:cantidad_izq]
23         postorder_der = postorder[cantidad_izq:-1]
24         nodo.left = self.construir(inorder_izq, postorder_izq)
25         nodo.right = self.construir(inorder_der, postorder_der)
26
27         return nodo
28

```

22.1 Construct Binary Tree From Inorder And Postorder Traversal - Divide y Conquista sobre Recorridos (cont.)

```

29     def buildTree(self, inorder: list[int], postorder: list[int]) -> TreeNode | None:
30
31         return self.construir(inorder, postorder)

```

22.2 Counts Nodex Equal To Average Of Subtree - DFS que Sube Informacion

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 2265 "Count Nodes Equal to Average of Subtree": cuantos nodos valen exactamente el promedio entero de su subárbol. Técnica: DFS que devuelve DOS valores hacia arriba, la suma y la cantidad de nodos del subárbol. Con eso el promedio sale en el padre sin volver a recorrer nada. Es el patrón típico de DFS que retorna un resumen. OJO: usa -1 como marca de no hay hijo y un contador global. Con valores negativos en el árbol la marca se confundiría con un valor real.

leetcode\binary_tree\2265_counts_nodex_equal_to_average_of_subtree.py

```

1 # Definition for a binary tree node.
2 # class TreeNode:
3 #     def __init__(self, val=0, left=None, right=None):
4 #         self.val = val
5 #         self.left = left
6 #         self.right = right
7 cont = 0
8 class Solution:
9
10     def dfs(self,nodo):
11         global cont
12         if(nodo == None):
13             return -1, -1
14         else:
15             val1, cant1 = self.dfs(nodo.left)
16             val2, cant2 = self.dfs(nodo.right)
17             if(val1 == -1 and val2 == -1):
18                 cont+=1
19                 return nodo.val, 1
20             elif(val1 != -1 and val2 != -1):
21                 val = (val1+val2+nodo.val)//(cant1+cant2+1)
22                 if(val == nodo.val):
23                     cont+=1
24                 return val1+val2+nodo.val, cant1+cant2+1
25             else:
26                 if(val1 != -1):
27                     val = (val1 + nodo.val) // (cant1 + 1)
28                     if val == nodo.val:
29                         cont += 1
30                     return val1 + nodo.val, cant1 + 1
31                 else:
32                     val = (val2 + nodo.val) // (cant2 + 1)
33                     if val == nodo.val:
34                         cont += 1
35                     return val2 + nodo.val, cant2 + 1
36
37     def averageOfSubtree(self, root: TreeNode) -> int:
38         global cont
39         cont = 0
40         self.dfs(root)
41         return cont

```

22.3 Maximum Depth Of Binary Tree - DFS en Arbol Binario

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 104 "Maximum Depth of Binary Tree": profundidad del árbol. Técnica: dos funciones mutuamente recursivas que bajan por derecha e izquierda guardando la profundidad de cada hoja en una lista, y al final se toma el máximo. OJO: es dar la vuelta larga. Lo normal es una sola línea recursiva: 0 si el nodo es None, si no 1 + max(maxDepth(izq), maxDepth(der)).

leetcode\binary_tree\0104_maximum_depth_of_binary_tree.py

```

1 # Definition for a binary tree node.
2 # class TreeNode:
3 #     def __init__(self, val=0, left=None, right=None):
4 #         self.val = val
5 #         self.left = left
6 #         self.right = right

```

22.3 Maximum Depth Of Binary Tree - DFS en Arbol Binario (cont.)

```

1 class Solution:
2     def der(self, root, cont, l):
3         if root.right == None:
4             l.append(cont)
5         return cont
6         root = root.right
7         cont+=1
8         self.der(root, cont, l)
9         self.izq(root, cont, l)
10
11     def izq(self, root, cont, l):
12         if root.left == None:
13             l.append(cont)
14         return cont
15         root = root.left
16         cont+=1
17         self.izq(root, cont, l)
18         self.der(root, cont, l)
19
20     def maxDepth(self, root: Optional[TreeNode]) -> int:
21         l = []
22         if root == None:
23             return 0
24         cont = 1
25         if root.left == None and root.right == None:
26             return cont
27         self.der(root, cont, l)
28         self.izq(root, cont, l)
29         return (max(l))

```

23 - Hub LeetCode: Bit Manipulation

23.1 Minimum Number Of Flips To Reverse Binary String - Comparar un Binario con su Reverso

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3750 "Minimum Number of Flips to Reverse Binary String": cuantos bits hay que cambiar para que el binario de n sea igual a su reverso. Técnica: `bin(n)[2:]` da el binario sin el 0b, el reverso sale con `[::-1]`, y se cuentan las posiciones donde no coinciden. Sin bitmask ni desplazamientos, todo por string.

leetcode/bit_manipulation/3750_minimum_number_of_flips_to_reverse_binary_string.py

```

1 class Solution:
2     def minimumFlips(self, n: int) -> int:
3         bi = bin(n)[2:]
4         bini = bi[::-1]
5         cont = 0
6         for i in range(len(bi)):
7             if bi[i] != bini[i]:
8                 cont+=1
9         return cont

```

24 - Hub LeetCode: Combinatorics

24.1 Count The Number Of Computer Unlocking Permutations - Factorial con un Filtro

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3577 "Count the Number of Computer Unlocking Permutations": en cuantos ordenes se pueden desbloquear todos los computadores. Técnica: si el primero no es el de complejidad ESTRICTAMENTE menor, no hay ninguna forma y la respuesta es 0. Si si, los otros n-1 van en cualquier orden, o sea $(n-1)! \bmod 1e9+7$. Todo el problema es darse cuenta de que solo importa el mínimo.

leetcode/combinatorics/3577_count_the_number_of_computer_unlocking_permutations.py

```

1 class Solution(object):
2     def countPermutations(self, complexity):
3         import math
4         menor = complexity[0]
5         for i in range(1, len(complexity)):
6             if (menor >= complexity[i]):
7                 return 0
8         return math.factorial(len(complexity)-1) % ((10**9)+7)

```

24.2 Distribute Candies Among Children I - Fuerza Bruta con Poda

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 2928 "Distribute Candies Among Children I": repartir n dulces entre 3 niños sin que ninguno reciba más de limit. Técnica: dos ciclos anidados sobre lo que recibe el primero y el segundo; el tercero queda determinado. Los continue son la poda, si con lo que ya se dio más $2*limit$ no se alcanza n, no hay caso. OJO: deja cuatro prints de depuración adentro del ciclo, y con eso se vuelve lentísimo. También se puede por inclusión-exclusión: $C(n+2, 2)$ menos los casos donde alguno pasa limit.

leetcode/combinatorics/2928_distribute_candies_among_children_i.py

```

1 class Solution:
2     def distributeCandies(self, n: int, limit: int) -> int:
3         cont = 0
4         for val1 in range(limit+1):
5             if (val1+2*limit < n):
6                 continue
7             else:
8                 for val2 in range(limit+1):
9                     if (val1+val2+limit < n or (val1==val2 and val1== n-val1-val2) or n-val1-val2 <
10                        0):
11                         continue
12                     else:
13                         print(val1)
14                         print(val2)
15                         print(n-val1-val2)
16                         print("-----")
17                         cont+=1
18                 for i in range(limit+1):
19                     if (i*3 == n):
20                         cont+=1
21         return (cont)

```

24.3 Number Of Sets Of K Non Overlapping Line Segments - Stars and Bars

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1621 "Number of Sets of K Non-Overlapping Line Segments": cuantas formas hay de dibujar k segmentos que no se traslapen sobre n puntos. Técnica: una sola línea, $\text{comb}(n+k-1, 2k) \bmod 1e9+7$. Sale de stars and bars: los 2k extremos de los segmentos se eligen entre n+k-1 posiciones porque dos segmentos SI pueden compartir un punto de borde, y ese k-1 extra es justo la holgura de los puntos compartidos. La lección es que a veces la DP se vuelve un coeficiente binomial y nada más.

leetcode/combinatorics/1621_number_of_sets_of_k_non-overlapping_line_segments.py

```

1 class Solution:
2     def numberOfSets(self, n: int, k: int) -> int:
3         return math.comb(n+k-1, 2*k) % (10**9+7)

```

24.4 Unique 3 Digit Event Numbers - Conteo con Frecuencias

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3483 "Unique 3-Digit Event Numbers": cuantos números distintos de 3 cifras se pueden armar con los dígitos dados, que sean pares y no empiecen por 0. Técnica: arreglo de frecuencias y tres ciclos, uno por posición, restando y devolviendo la frecuencia al entrar y salir (backtracking barato). El set al final quita los repetidos. Este mismo problema está resuelto en C++ en la carpeta LeetCode del notebook.

leetcode/combinatorics/3483_unique_3-digit_event_numbers.py

```

1 class Solution:
2     def totalNumbers(self, digits: List[int]) -> int:
3         res = set()
4         fre = [0]*10
5         for i in digits:
6             fre[i] += 1
7
8         for C in range(0, 10, 2):
9             if (fre[C] == 0):
10                 continue
11             fre[C] -= 1
12             for A in range(1, 10):
13                 if (fre[A] == 0):
14                     continue
15                 fre[A] -= 1
16                 for B in range(0, 10):

```

24.4 Unique 3 Digit Event Numbers - Conteo con Frecuencias (cont.)

```
11 if(fre[B]==0):
12     continue
13     res.add(int(str(A)+str(B)+str(C)))
20     fre[A]+=1
21     fre[C]+=1
22     return len(res)
```

25 - Hub LeetCode: DP

25.1 Climbing Stairs - Memoizacion

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 70 "Climbing Stairs": de cuantas formas se sube una escalera de n subiendo 1 o 2. Tecnica: Fibonacci con memoizacion, arreglo vis donde 0 significa sin calcular. Es el ejemplo mas corto de DP top-down: recursion normal mas una tabla que guarda lo ya hecho. Bottom-up con dos variables gasta O(1) de memoria.

leetcode/dp/0070_climbing_stairs.py

```
1 class Solution:
2     def fibo(self,n,vis):
3         if(vis[n]!=0):
4             return vis[n]
5         if(n==0 or n==1):
6             return 1
7         a = self.fibo(n-1,vis)+self.fibo(n-2,vis)
8         vis[n] = a
9         return a
10
11     def climbStairs(self, n: int) -> int:
12         vis = [0]*(n+1)
13         return self.fibo(n,vis)
```

25.2 Distinct Subsequences I - DP en Dos Cadenas

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 115 "Distinct Subsequences": cuantas subsecuencias de s son iguales a t. Tecnica: dp[i][j] = de cuantas formas se forma t[j] usando s[i]. Si las letras coinciden hay dos opciones, usarla o no, y se SUMAN; si no coinciden solo queda avanzar en s. Los dp[i][n] = 1 son el caso base, t ya se termino. Va de atras hacia adelante, que es lo natural cuando el estado son sufijos.

leetcode/dp/0115_distinct_subsequences_i.py

```
1 class Solution:
2     def numDistinct(self, s: str, t: str) -> int:
3         m, n = len(s), len(t)
4         if m < n:
5             return 0
6
7         dp = [[0] * (n + 1) for _ in range(m + 1)]
8         for i in range(m + 1):
9             dp[i][n] = 1
10
11         for i in range(m - 1, -1, -1):
12             for j in range(n - 1, -1, -1):
13                 if s[i] == t[j]:
14                     dp[i][j] = dp[i + 1][j + 1] + dp[i + 1][j]
15                 else:
16                     dp[i][j] = dp[i + 1][j]
17
18         return dp[0][0]
```

25.3 Distinct Subsequences II - DP con Resta de Repetidos

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 940 "Distinct Subsequences II": cuantas subsecuencias DISTINTAS tiene una cadena. Tecnica: cada letra nueva duplica las subsecuencias (con y sin ella), asi que dp = dp*2. Para no contar repetidas se RESTA el dp de la vez anterior que aparecio esa misma letra. El -1 del final quita la subsecuencia vacia. Cinco lineas, pero el patron duplicar y restar la aparicion previa vale mucho.

leetcode/dp/0940_distinct_subsequences_ii.py

```
1 class Solution(object):
2     def distinctSubseqII(self, S):
3         dp = [1]
```

25.3 Distinct Subsequences II - DP con Resta de Repetidos (cont.)

```
4         last = {}
5         for i, x in enumerate(S):
6             dp.append(dp[-1] * 2)
7             if x in last:
8                 dp[-1] -= dp[last[x]]
9             last[x] = i
10
11         return (dp[-1] - 1) % (10**9 + 7)
```

25.4 Find X Value Of Array I - DP sobre Residuos

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3524 "Find X Value of Array I": para cada residuo, cuantos subarreglos tienen producto congruente con ese residuo modulo k. Tecnica: dp[i][r] = cuantos subarreglos que TERMINAN en i dejan residuo r. Cada uno se extiende de dp[i-1] multiplicando por nums[i] mod k, mas el que empieza en i. Al final se acumula. Es la version sin segment tree del 3525 que esta en la carpeta segment_tree. OJO: usa try/except en vez de revisar si i es 0.

leetcode/dp/3524_find_x_value_of_array_i.py

```
1 class Solution:
2     def resultArray(self, nums: List[int], k: int) -> List[int]:
3         result = [0] * k
4         dp = {}
5         for i in range(0, len(nums)):
6             dp[i] = [0]*k
7             dp[i][nums[i]%k] += 1
8             for j in range(k):
9                 try:
10                     if(dp[i-1][j] != 0):
11                         res = (j*(nums[i]%k))%k
12                         dp[i][res] += dp[i-1][j]
13                 except:
14                     a = 1
15             for j in range(k):
16                 result[j] += dp[i][j]
17
18         return result
```

25.5 Maximum Score Of Non Overlapping Intervals - DP de Intervalos con Binaria

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3414 "Maximum Score of Non-Overlapping Intervals": escoger hasta 4 intervalos que no se traslapen maximizando el peso, y devolver los indices mas pequenos en orden lexicografico. Tecnica: ordenar por extremo DERECHO y dp[i][j] = mejor puntaje usando los primeros i intervalos con j escogidos. Para saltar a los compatibles se usa bisect_left, que encuentra el primer intervalo que termina antes de que empiece el actual. El desempate lexicografico se arrastra en una lista de indices paralela a la dp, que es la parte molesta del problema.

leetcode/dp/3414_maximum_score_of_non-overlapping_intervals.py

```
1 class Solution:
2     def maximumWeight(self, intervals: List[List[int]]) -> List[int]:
3         n = len(intervals)
4         arr = [
5             (intervals[i][1], intervals[i][0], intervals[i][2], i)
6             for i in range(n)
7         ]
8         # Sort by right endpoint.
9         arr.sort(key=lambda x: x[0])
10
11         dp = [[0] * 5 for _ in range(n + 1)]
12         indices = [[[0] for _ in range(5)] for _ in range(n + 1)]
13
14         for i in range(n):
15             r, l, weight, idx = arr[i]
16             # Use binary search to find intervals whose right endpoints are smaller than l.
17             k = bisect_left(arr, (l, ), hi=i)
18
19             for j in range(1, 5):
20                 s1 = dp[i][j]
21                 s2 = dp[k][j - 1] + weight
22                 if s1 > s2:
23                     dp[i + 1][j] = dp[i][j]
24                     indices[i + 1][j] = indices[i][j].copy()
```

25.5 Maximum Score Of Non Overlapping Intervals - DP de Intervalos con Binaria (cont.)

```
25         continue
26
27         new_index = indices[k][j - 1].copy()
28         new_index.append(idx)
29         new_index.sort()
30         if s1 == s2 and indices[i][j] < new_index:
31             new_index = indices[i][j].copy()
32         dp[i + 1][j] = s2
33         indices[i + 1][j] = new_index
34
35     return indices[n][4]
```

26 - Hub LeetCode: Game Theory

26.1 Divisor Game - Paridad

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1025 "Divisor Game": se resta un divisor propio de n y pierde quien no puede mover. Tecnica: con n par siempre se puede restar 1 y dejarle un impar al rival; con n impar todos los divisores son impares y toca dejarle un par. Así que gana el primero si n es par.

leetcode/game_theory/1025_divisor_game.py

```
1 class Solution:
2     def divisorGame(self, n: int) -> bool:
3         if(n%2 == 0):
4             return True
5         else:
6             return False
```

26.2 Find The Winning Player In Coin Game - Simulacion de Turnos

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3222 "Find the Winning Player in Coin Game": cada turno se toma una moneda de 75 y cuatro de 10, y pierde quien no alcanza a completar los 115. Tecnica: simular el turno mientras queden $x \geq 1$ y $y \geq 4$, alternando una bandera. Quien no puede mover pierde, así que decide la paridad de la cantidad de turnos. Sale directo con $\min(x, y // 4)$ pero la simulacion es igual de valida.

leetcode/game_theory/3222_find_the_winning_player_in_coin_game.py

```
1 class Solution:
2     def winningPlayer(self, x: int, y: int) -> str:
3         j = True
4         while True:
5             if(x>=1 and y>=4):
6                 x-=1
7                 y-=4
8                 j = not j
9             else:
10                break
11            if(j):
12                return "Bob"
13            else:
14                return "Alice"
```

26.3 Nim Game - Posiciones Perdedoras

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 292 "Nim Game": se quitan 1, 2 o 3 piedras por turno y pierde quien no puede mover. Tecnica: quien recibe un multiplo de 4 pierde, porque el rival siempre puede completar 4 entre los dos turnos. Todo el problema es $n \% 4 \neq 0$.

leetcode/game_theory/0292_nim_game.py

```
1 class Solution:
2     def canWinNim(self, n: int) -> bool:
3         if(n %4 == 0):
4             return False
5         else:
6             return True
```

27 - Hub LeetCode: Geometry

27.1 Angle Between Hands Of A Clock - Angulos de un Reloj

NO ES MIO: codigo de Stiven Correa, del repo del equipo (carpeta leetcode/). LeetCode 1344 "Angle Between Hands of a Clock": angulo menor entre las manecillas. Tecnica: el minutero avanza 6 grados por minuto; la hora avanza 30 grados por hora PERO tambien se mueve con los minutos, de ahí el $(\text{hour} + \text{minutes}/60)$. Al final se toma el menor entre la diferencia y 360 menos la diferencia.

leetcode/geometry/1344_angle_between_hands_of_a_clock.py

```
1 class Solution:
2     def angleClock(self, hour: int, minutes: int) -> float:
3         minAng = (360/60)*minutes
4         hourAng = ((360/12)*((hour+minutes/60)%24))%360
5
6         return min(abs(minAng-hourAng),abs(hourAng-minAng), 360-abs(minAng-hourAng), 360-abs(hourAn
7             g-minAng))
```

27.2 Circle And Rectangle Overlapping - Punto mas Cercano en un Rectangulo

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1401 "Circle and Rectangle Overlapping": decir si un circulo toca un rectangulo. Tecnica: el truco es el CLAMP. Se recorta el centro del circulo al rectangulo con $\max(x1, \min(cx, x2))$, lo que da el punto del rectangulo mas cercano al centro; si ese punto queda a distancia $\leq r$, hay traslape. Dos lineas y cero casos especiales. Este clamp es muy reutilizable en geometria de cajas.

leetcode/geometry/1401_circle_and_rectangle_overlapping.py

```
1 class Solution:
2     def checkOverlap(self, r: int, cx: int, cy: int, x1: int, y1: int, x2: int, y2: int) -> bool:
3         x = max(x1, min(cx, x2)) - cx
4         y = max(y1, min(cy, y2)) - cy
5
6         return x * x + y * y <= r * r
```

27.3 Minimum Cuts To Divide A Circle - Casos por Paridad

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 2481 "Minimum Cuts to Divide a Circle": cortes minimos para partir un circulo en n pedazos iguales. Tecnica: cada corte es un DIAMETRO y parte el circulo en dos, así que con n par bastan $n/2$. Con n impar los diametros no sirven y toca n. Y con $n = 1$ no se corta nada.

leetcode/geometry/2481_minimum_cuts_to_divide_a_circle.py

```
1 class Solution:
2     def numberOfCuts(self, n: int) -> int:
3         pedazos = n
4
5         if (pedazos % 2) == 0:
6
7             trazos = pedazos/2
8
9         elif pedazos == 1:
10
11             trazos = 0
12
13         else:
14
15             trazos = pedazos
16
17
18         return(int(trazos))
```

27.4 Rectangle Overlap - Interseccion de Rectangulos

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 836 "Rectangle Overlap": decir si dos rectangulos alineados a los ejes se traslapan en area. Tecnica: la interseccion de dos rectangulos es otro rectangulo, con ancho $\min(\text{derechas}) - \max(\text{izquierdas})$ y alto $\min(\text{arribas}) - \max(\text{abajos})$. Hay traslape si los dos salen positivos. Vale la pena aprenderse así, sin listas de casos.

leetcode/geometry/0836_rectangle_overlap.py

```
1 class Solution:
2     def isRectangleOverlap(self, rec1: List[int], rec2: List[int]) -> bool:
3         ancho = min(rec1[2], rec2[2]) - max(rec1[0], rec2[0])
4         alto = min(rec1[3], rec2[3]) - max(rec1[1], rec2[1])
5
```

27.4 Rectangle Overlap - Interseccion de Rectangulos (cont.)

```
6         return ancho > 0 and alto > 0
```

28 - Hub LeetCode: Graphs

28.1 All Paths From Source To Target - Backtracking de Caminos

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 797 "All Paths From Source to Target": todos los caminos del nodo 0 al ultimo en un DAG. Tecnica: DFS con backtracking. path.append al entrar, path.pop al salir, y cuando se llega al destino se guarda una COPIA de path (sin el .copy()) se guardaria la misma lista que sigue cambiando, que es el error clasico). Aca marca ARISTAS visitadas; en un DAG no hace falta marcar nada, basta el backtracking.

leetcode/graphs/0797_all_paths_from_source_to_target.py

```
1 from collections import deque
2 class Solution:
3     def allPathsSourceTarget(self, graph: List[List[int]]) -> List[List[int]]:
4         visited = set()
5         res = []
6         path = []
7         def dfs(curr, par):
8             path.append(curr)
9             if curr == len(graph)-1:
10                 res.append(path.copy())
11             for node in graph[curr]:
12                 if (curr, node) in visited:
13                     continue
14                 visited.add((curr, node))
15                 dfs(node, curr)
16             if par != -1:
17                 visited.remove((par, curr))
18             path.pop()
19             dfs(0, -1)
20         return res
```

28.2 Cheapest Flights Within K Stops - DP con Estado (nodo, saltos)

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 787 "Cheapest Flights Within K Stops": el vuelo mas barato con a lo mas k escalas. Tecnica: no es Dijkstra puro porque el limite de escalas mete una dimension mas. El estado es (nodo actual, vuelos restantes) y se memoiza con @cache. Ese decorador ahorra escribir la tabla a mano y vale la pena recordarlo en Python. La otra forma es Bellman-Ford limitado a k+1 rondas, que es la version clasica.

leetcode/graphs/0787_cheapest_flights_within_k_stops.py

```
1 from collections import defaultdict
2 from functools import cache
3
4 class Solution:
5     def findCheapestPrice(self, n, flights, src, dst, k):
6         graph = defaultdict(list)
7
8         for u, v, w in flights:
9             graph[u].append((v, w))
10
11         INF = float("inf")
12
13         @cache
14         def dp(curr, remaining):
15             if curr == dst:
16                 return 0
17
18             if remaining == 0:
19                 return INF
20
21             ans = INF
22
23             for nxt, cost in graph[curr]:
24                 ans = min(ans, cost + dp(nxt, remaining - 1))
25
26             return ans
27
28         ans = dp(src, k + 1)
```

28.2 Cheapest Flights Within K Stops - DP con Estado (nodo, saltos) (cont.)

```
29         return -1 if ans == INF else ans
```

28.3 Check If There Is A Valid Path In A Grid - Compatibilidad de Tubos en Matriz

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1391 "Check if There is a Valid Path in a Grid": una cuadrícula de tubos numerados 1 a 6; decir si se puede ir de la esquina de arriba a la de abajo. Tecnica: DFS donde moverse solo vale si el tubo actual apunta hacia el vecino Y el vecino apunta de vuelta. Las listas [1,3,5] y compania son justo los tubos que reciben por cada lado, y esa tabla de compatibilidad es todo el problema. Este mismo problema esta resuelto en C++ en la carpeta LeetCode del notebook.

leetcode/graphs/1391_check_if_there_is_a_valid_path_in_a_grid.py

```
1 class Solution:
2
3     def DFS(self, i, j, grid, memo):
4         if i < 0 or i >= len(grid) or j < 0 or j >= len(grid[0]):
5             return False
6         if (i, j) in memo:
7             return False
8         if i == len(grid) - 1 and j == len(grid[0]) - 1:
9             return True
10
11         memo.add((i, j))
12
13         calle = grid[i][j]
14
15         if calle == 1:
16             nj = j + 1
17             if nj < len(grid[0]) and grid[i][nj] in [1, 3, 5]:
18                 if self.DFS(i, nj, grid, memo):
19                     return True
20             nj = j - 1
21             if nj >= 0 and grid[i][nj] in [1, 4, 6]:
22                 if self.DFS(i, nj, grid, memo):
23                     return True
24         elif calle == 2:
25
26             ni = i + 1
27             if ni < len(grid) and grid[ni][j] in [2, 5, 6]:
28                 if self.DFS(ni, j, grid, memo):
29                     return True
30
31             ni = i - 1
32             if ni >= 0 and grid[ni][j] in [2, 3, 4]:
33                 if self.DFS(ni, j, grid, memo):
34                     return True
35
36         elif calle == 3:
37
38             ni = i + 1
39             if ni < len(grid) and grid[ni][j] in [2, 5, 6]:
40                 if self.DFS(ni, j, grid, memo):
41                     return True
42
43             nj = j - 1
44             if nj >= 0 and grid[i][nj] in [1, 4, 6]:
45                 if self.DFS(i, nj, grid, memo):
46                     return True
47
48         elif calle == 4:
49
50             nj = j + 1
51             if nj < len(grid[0]) and grid[i][nj] in [1, 3, 5]:
52                 if self.DFS(i, nj, grid, memo):
53                     return True
```

28.3 Check If There Is A Valid Path In A Grid - Compatibilidad de Tubos en Matriz (cont.)

```

80
81
82         ni = i + 1
83         if ni < len(grid) and grid[ni][j] in [2, 5, 6]:
84             if self.DFS(ni, j, grid, memo):
85                 return True
86
87
88         elif calle == 5:
89
90
91         ni = i - 1
92         if ni >= 0 and grid[ni][j] in [2, 3, 4]:
93             if self.DFS(ni, j, grid, memo):
94                 return True
95
96
97         nj = j - 1
98         if nj >= 0 and grid[i][nj] in [1, 4, 6]:
99             if self.DFS(i, nj, grid, memo):
100                 return True
101
102
103         elif calle == 6:
104
105
106         ni = i - 1
107         if ni >= 0 and grid[ni][j] in [2, 3, 4]:
108             if self.DFS(ni, j, grid, memo):
109                 return True
110
111
112         nj = j + 1
113         if nj < len(grid[0]) and grid[i][nj] in [1, 3, 5]:
114             if self.DFS(i, nj, grid, memo):
115                 return True
116
117
118         return False
119
120     def isValidPath(self, grid):
121         memo = set()
122         return self.DFS(0, 0, grid, memo)
123

```

28.4 Clone Graph - Clonar Grafo con Diccionario

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 133 "Clone Graph": copia profunda de un grafo no dirigido. Técnica: DFS con un diccionario original -> copia. Antes de recorrer los vecinos se registra la copia en el diccionario, y ese orden es lo que evita el ciclo infinito cuando el grafo tiene ciclos. Si el nodo ya está en el mapa se devuelve la copia que ya existía. Es el mismo patrón que pide LeetCode 138 (lista con puntero random).

leetcode/graphs/0133_clone_graph.py

```

1  class Solution:
2      def cloneGraph(self, node: Optional['Node']) -> Optional['Node']:
3
4      if not node:
5          return None
6
7      visited = {}
8      # original -> copia
9
10     def creator(curr):
11
12         if curr in visited:
13             return visited[curr]
14
15         copy = Node(curr.val)
16         visited[curr] = copy
17
18         for nxt in curr.neighbors:
19             copy.neighbors.append(creator(nxt))
20

```

28.4 Clone Graph - Clonar Grafo con Diccionario (cont.)

```

21         return copy
22
23         return creator(node)

```

28.5 Course Schedule - Orden Topologico de Kahn

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 207 "Course Schedule": decir si se pueden tomar todos los cursos respetando los prerrequisitos. Técnica: Kahn. Se cuentan los grados de ENTRADA, se meten a la cola los de grado 0 y al sacar uno se baja el grado de sus vecinos. Si al final el orden no tiene los n nodos, hay un ciclo. Detectar ciclo en dirigido con Kahn es contar cuantos salieron, y eso es toda la gracia.

leetcode/graphs/0207_course_schedule.py

```

1  from collections import deque
2  class Solution:
3      def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:
4          def kahn_topological_sort(num_nodes, edges):
5              # 1. Initialize adjacency list and in-degree array
6              adj_list = {i: [] for i in range(num_nodes)}
7              in_degree = [0] * num_nodes
8
9              # 2. Build the graph: edge is u -> v
10             for u, v in edges:
11                 adj_list[u].append(v)
12                 in_degree[v] += 1
13
14             # 3. Add all nodes with 0 incoming edges to the queue
15             queue = deque([i for i in range(num_nodes) if in_degree[i] == 0])
16             topo_order = []
17
18             # 4. Process the queue
19             while queue:
20                 u = queue.popleft()
21                 topo_order.append(u)
22
23                 for neighbor in adj_list[u]:
24                     in_degree[neighbor] -= 1
25                     if in_degree[neighbor] == 0:
26                         queue.append(neighbor)
27
28             # 5. Check for cycles
29             if len(topo_order) != num_nodes:
30                 return False
31
32             return True
33
34         return kahn_topological_sort(numCourses, prerequisites)

```

28.6 Course Schedule II - Orden Topologico de Kahn

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 210 "Course Schedule II": devolver un orden válido de los cursos, o vacío si no hay. Técnica: el mismo Kahn del 207, pero devolviendo el orden en vez de un booleano. Lo único que cambia es que primero invierte cada par, porque el enunciado da [curso, prerrequisito] y el grafo necesita la arista prerrequisito -> curso. Equivocarse en la dirección de la arista es el error clásico de este problema.

leetcode/graphs/0210_course_schedule_ii.py

```

1  from collections import deque
2  class Solution:
3      def findOrder(self, numCourses: int, prerequisites: List[List[int]]) -> List[int]:
4
5          def kahn_topological_sort(num_nodes, edges):
6              # 1. Initialize adjacency list and in-degree array
7              adj_list = {i: [] for i in range(num_nodes)}
8              in_degree = [0] * num_nodes
9
10             # 2. Build the graph: edge is u -> v
11             for u, v in edges:
12                 adj_list[u].append(v)
13                 in_degree[v] += 1
14

```

28.6 Course Schedule II - Orden Topologico de Kahn (cont.)

```

5         # 3. Add all nodes with 0 incoming edges to the queue
6         queue = deque([i for i in range(num_nodes) if in_degree[i] == 0])
7         topo_order = []
8
9         # 4. Process the queue
10        while queue:
11            u = queue.popleft()
12            topo_order.append(u)
13
14            for neighbor in adj_list[u]:
15                in_degree[neighbor] -= 1
16                if in_degree[neighbor] == 0:
17                    queue.append(neighbor)
18
19        # 5. Check for cycles
20        if len(topo_order) != num_nodes:
21            return []
22
23        return topo_order
24
25        for i in range(len(prerequisites)):
26            prerequisites[i] = [prerequisites[i][1], prerequisites[i][0]]
27        return (kahn_topological_sort(numCourses, prerequisites))

```

28.7 Evaluate Division - Grafo con Pesos Multiplicativos

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 399 "Evaluate Division": dadas razones a/b, responder consultas de otras razones. Técnica: grafo donde la arista a -> b pesa a/b y la inversa b -> a pesa 1/v. La respuesta de una consulta es el PRODUCTO de los pesos del camino, así que un DFS que multiplica lo resuelve. Traducir division a camino en grafo es el truco; el -1 marca que no hay camino. La otra forma es DSU con pesos, que responde en casi O(1) por consulta.

leetcode/graphs/0399_evaluate_division.py

```

1  from collections import defaultdict
2
3  class Solution:
4      def calcEquation(self, equations, values, queries):
5
6          graph = defaultdict(list)
7
8          for (a, b), v in zip(equations, values):
9              graph[a].append((b, v))
10             graph[b].append((a, 1 / v))
11
12         def dfs(curr, target, visited):
13
14             if curr == target:
15                 return 1
16
17             visited.add(curr)
18
19             for nxt, weight in graph[curr]:
20                 if nxt in visited:
21                     continue
22
23                 ans = dfs(nxt, target, visited)
24
25                 if ans != -1:
26                     return weight * ans
27
28             return -1
29
30         res = []
31
32         for a, b in queries:
33
34             if a not in graph or b not in graph:
35                 res.append(-1.0)
36                 continue
37
38             res.append(dfs(a, b, set()))
39

```

28.7 Evaluate Division - Grafo con Pesos Multiplicativos (cont.)

```

40         return res

```

28.8 Find A Safe Walk Through A Grid - Búsqueda con Vida Restante

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3286 "Find a Safe Walk Through a Grid": cruzar la cuadrícula sin que la vida llegue a 0, perdiendo 1 en cada celda marcada. Técnica: búsqueda con el estado (celda, vida) y la poda importante: si ya se visitó esa celda con MAS vida, volver con menos nunca sirve. Ese ant[celda] = mejor vida vista es lo que evita la explosión. OJO: la función se llama BFS pero es recursiva, o sea DFS, y usa str(m)+str(n) como clave, que confunde 1,12 con 11,2. Con cuadrículas de hasta 50x50 no choca, pero es una llave mala. Lo correcto es BFS 0-1 (deque) o Dijkstra sobre el estado (celda, vida).

leetcode/graphs/3286_find_a_safe_walk_through_a_grid.py

```

1  class Solution:
2
3      def BFS(self, grid, m, n, health, ant):
4          #print(m,n,health)
5          ant[str(m)+str(n)] = health
6          if(health <= 0):
7              return 0
8          elif(m==len(grid)-1 and n==len(grid[0])-1):
9              return 1
10         else:
11             k = [[1,0],[0,1],[-1,0],[0,-1]]
12             resp = 2
13             for i in k:
14                 #print(m,n,health, i, "-----")
15                 if(m+i[0]>-1 and m+i[0]<len(grid) and n+i[1]>-1 and n+i[1]<len(grid[0]) and str(m+i[0])+str(n+i[1]) not in ant):
16                     if(grid[m+i[0]][n+i[1]] == 1):
17                         resp = self.BFS(grid,m+i[0],n+i[1],health-1, ant)
18                     else:
19                         resp = self.BFS(grid,m+i[0],n+i[1],health,ant)
20                     elif(m+i[0]>-1 and m+i[0]<len(grid) and n+i[1]>-1 and n+i[1]<len(grid[0]) and ant[str(m+i[0])+str(n+i[1])] < health):
21                         if(grid[m+i[0]][n+i[1]] == 1):
22                             resp = self.BFS(grid,m+i[0],n+i[1],health-1, ant)
23                     else:
24                         resp = self.BFS(grid,m+i[0],n+i[1],health,ant)
25                     if(resp == 1):
26                         break
27             if(resp == 1):
28                 return 1
29             else:
30                 return 2
31
32         def findSafeWalk(self, grid: List[List[int]], health: int) -> bool:
33             unos = 0
34             for i in grid:
35                 unos += i.count(1)
36             if(unos==(len(grid[0])*len(grid)) and health<=(len(grid[0])+len(grid))-1):
37                 return False
38             if(grid[0][0]==1):
39                 health-=1
40             ant = {"00":health}
41             res = self.BFS(grid, 0, 0, health, ant)
42             if(res == 1):
43                 return True
44             else:
45                 return False

```

28.9 Find Eventual Safe States - Nodos que No Llegan a un Ciclo

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 802 "Find Eventual Safe States": los nodos desde los que todo camino termina en un nodo sin salidas. Técnica: DFS que devuelve True si desde el nodo no se alcanza ningún ciclo, con el set safe como memoria de los ya confirmados. OJO: hace visited.clear() antes de cada nodo, así que repite trabajo. Lo limpio es el DFS de tres colores (blanco, gris, negro) en una sola pasada, o Kahn sobre el grafo INVERTIDO.

leetcode/graphs/0802_find_eventual_safe_states.py

```
1 class Solution:
2     def eventualSafeNodes(self, graph: List[List[int]]) -> List[int]:
3         safe = set()
4         visited = set()
5
6         def dfs(curr):
7             if curr in safe:
8                 return True
9
10            if curr in visited:
11                return False
12
13            visited.add(curr)
14
15            for node in graph[curr]:
16                if not dfs(node):
17                    visited.discard(curr)
18                    return False
19
20            visited.discard(curr)
21            safe.add(curr)
22            return True
23
24        res = []
25
26        for i in range(len(graph)):
27            visited.clear()
28            if dfs(i):
29                res.append(i)
30
31        return res
```

28.10 Holding Hands - DSU sobre Parejas

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 765 "Couples Holding Hands": intercambios mínimos para que cada pareja quede junta. Técnica: se piensa por SILLAS, no por personas. Cada par de sillas (0,1), (2,3), ... es un nodo, y si las dos personas sentadas ahí son de parejas distintas se unen los dos nodos con DSU. La respuesta es la suma de (tamaño del grupo - 1) sobre todos los grupos. Un ciclo de k parejas mal sentadas se arregla con k-1 intercambios, y eso es lo que se está sumando.

leetcode/graphs/0765_holding_hands.py

```
1 from collections import defaultdict
2 class Solution:
3     def minSwapsCouples(self, row: List[int]) -> int:
4         n = len(row)
5         parent = [i for i in range(n//2)]
6         def find(i):
7             if parent[i] == i:
8                 return i
9             return find(parent[i])
10        def unite(i, j):
11            irep = find(i)
12            jrep = find(j)
13            parent[irep] = jrep
14        def get_groups():
15            groups = defaultdict(list)
16            for element in parent:
17                root = find(element)
18                groups[root].append(element)
19            return list(groups.values())
20
21        coach = [-1] * (n)
22
```

28.10 Holding Hands - DSU sobre Parejas (cont.)

```
23         for i, val in enumerate(row):
24             coach[val] = i//2
25
26         graph = defaultdict(list)
27         for i in range(0, n-1, 2):
28             curr = coach[i]
29             nxt = coach[i+1]
30             if curr != nxt:
31                 unite(curr, nxt)
32         groups = get_groups()
33         res = 0
34         for i in groups:
35             res += len(i)-1
36         return res
```

28.11 Is Graph Bipartite - Coloreo a Dos Colores

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 785 "Is Graph Bipartite?": decir si el grafo se puede partir en dos grupos sin aristas internas. Técnica: DFS pintando 1 y -1 alternado. Si alguna arista une dos nodos del MISMO color, no es bipartito. El ciclo externo por todos los nodos es porque el grafo puede venir desconectado. Bipartito equivale a no tener ciclos de longitud impar, y esto es la forma de verificarlo.

leetcode/graphs/0785_is_graph_bipartite.py

```
1 from collections import defaultdict
2 class Solution:
3     def isBipartite(self, graph: List[List[int]]) -> bool:
4         colors = defaultdict(int)
5         res = True
6         visited = set()
7         def dfs(curr):
8             nonlocal res
9             visited.add(curr)
10
11            if curr not in colors:
12                colors[curr] = 1
13
14            for node in graph[curr]:
15                if node not in colors:
16                    colors[node] = -colors[curr]
17                    dfs(node)
18                elif colors[node] == colors[curr]:
19                    res = False
20        for i in range(len(graph)):
21            if i not in visited:
22                dfs(i)
23        return res
```

28.12 Jump Game III - DFS con Visitados

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1306 "Jump Game III": desde start se salta arr[i] a izquierda o derecha; llegar a un 0. Técnica: DFS marcando visitados. Cada posición tiene dos salidas, i+arr[i] e i-arr[i], así que es un grafo y el set de visitados es lo que evita dar vueltas. Con BFS sale igual y además da la cantidad mínima de saltos si la pidieran.

leetcode/graphs/1306_jump_game_iii.py

```
1 class Solution:
2
3     def numeri(self, nums, start, l, tam):
4
5         if(start in l):
6             return False
7
8         l.add(start)
9
10        pos1 = start + nums[start]
11        pos2 = start - nums[start]
12
13        val1, val2 = False, False
14
15        if(pos1 < tam and nums[pos1] == 0):
16            return True
```

28.12 Jump Game Iii - DFS con Visitados (cont.)

```

17         else:
18             if(pos1 < tam):
19                 val1 = self.numeri(nums, pos1, 1, tam)
20
21             if(pos2 > -1 and nums[pos2] == 0):
22                 return True
23             else:
24                 if(pos2 >= 0):
25                     val2 = self.numeri(nums, pos2, 1, tam)
26
27             return val1 or val2
28
29
30 def canReach(self, arr: List[int], start: int) -> bool:
31
32     tam = len(arr)
33
34     if(0 not in arr):
35         return False
36
37     elif(arr[start] == 0):
38         return True
39
40     else:
41         l = set()
42         a = self.numeri(arr, start, 1, tam)
43         return a

```

28.13 Longest Increasing Path In A Matrix - DFS con Memoizacion en Matriz

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 329 "Longest Increasing Path in a Matrix": el camino mas largo que siempre sube. Tecnica: DFS con memoizacion. Como solo se puede ir a celdas MAYORES, el grafo no tiene ciclos y se puede memoizar sin peligro: `dicc[i,j]` guarda el camino mas largo que arranca ahi. Esa es la idea clave, la condicion de crecer es la que vuelve el grafo un DAG.

leetcode/graphs/0329_longest_increasing_path_in_a_matrix.py

```

1 from collections import defaultdict
2
3 class Solution:
4     def longestIncreasingPath(self, matrix: List[List[int]]) -> int:
5         paths = set()
6         for i in range(len(matrix)):
7             for j in range(len(matrix[i])):
8                 paths.add((i,j))
9         dicc = defaultdict(int)
10        def explore(i,j):
11            best = 1
12            if i+1 < len(matrix) and matrix[i+1][j] > matrix[i][j]:
13                if (i+1, j) in paths:
14                    best = max(best, 1 + explore(i+1, j))
15                paths.discard((i+1, j))
16            else:
17                best = max(best, 1 + dicc[(i+1, j)])
18            if i-1 >= 0 and matrix[i-1][j] > matrix[i][j]:
19                if (i-1, j) in paths:
20                    paths.discard((i-1, j))
21                best = max(best, 1 + explore(i-1, j))
22            else:
23                best = max(best, 1 + dicc[(i-1, j)])
24            if j+1 < len(matrix[i]) and matrix[i][j+1] > matrix[i][j]:
25                if (i, j+1) in paths:
26                    paths.discard((i, j+1))
27                best = max(best, 1 + explore(i, j+1))
28            else:
29                best = max(best, 1 + dicc[(i, j+1)])
30            if j-1 >= 0 and matrix[i][j-1] > matrix[i][j]:
31                if (i, j-1) in paths:
32                    paths.discard((i, j-1))
33                best = max(best, 1 + explore(i, j-1))
34            else:
35                best = max(best, 1 + dicc[(i, j-1)])
36            dicc[(i, j)] = best

```

28.13 Longest Increasing Path In A Matrix - DFS con Memoizacion en Matriz (cont.)

```

37         return best
38
39     while(True):
40         if(len(paths) > 0):
41             nextP = paths.pop()
42             explore(nextP[0], nextP[1])
43         else:
44             break
45         return max(dicc.values())

```

28.14 Minimum Height Trees - Pelar Hojas (Centro del Arbol)

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 310 "Minimum Height Trees": las raices que dan el arbol de altura minima. Tecnica: pelar hojas por capas. Se meten a la cola todos los nodos de grado 1, se quitan todos a la vez, aparecen hojas nuevas, y se repite hasta que queden 1 o 2 nodos: esos son el CENTRO del arbol y la respuesta. Un arbol siempre tiene uno o dos centros, nunca tres, y eso es lo que justifica el `while > 2`. El mismo pelado por capas sirve para el diametro y para varios problemas de arboles.

leetcode/graphs/0310_minimum_height_trees.py

```

1 from collections import defaultdict, deque
2
3 class Solution:
4     def findMinHeightTrees(self, n: int, edges: List[List[int]]) -> List[int]:
5
6         if n == 1:
7             return [0]
8
9         grafo = defaultdict(list)
10        grado = [0] * n
11
12        for u, v in edges:
13            grafo[u].append(v)
14            grafo[v].append(u)
15            grado[u] += 1
16            grado[v] += 1
17
18        cola = deque()
19
20        for i in range(n):
21            if grado[i] == 1:
22                cola.append(i)
23
24        restantes = n
25
26        while restantes > 2:
27            hojas = len(cola)
28            restantes -= hojas
29
30            for _ in range(hojas):
31                hoja = cola.popleft()
32
33                for vecino in grafo[hoja]:
34                    grado[vecino] -= 1
35                    if grado[vecino] == 1:
36                        cola.append(vecino)
37
38        return list(cola)

```

28.15 Minimum Moves To Clean The Classroom - Dijkstra sobre (nodo, bitmask) con Poda de Pa...

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3568 "Minimum Moves to Clean the Classroom": recoger toda la basura L de una cuadrícula con energía limitada, recargando en las R. Técnica: el estado es (posición especial, máscara de basura recogida) y se explora con heap, o sea Dijkstra sobre estados. Primero comprime la cuadrícula a un grafo solo entre las celdas que importan (S, L y R) con BFS, y después corre el Dijkstra sobre ese grafo chico. Lo más interesante: la energía NO entra en la llave del estado, se maneja con PODA DE PARETO. Un estado se descarta si ya existe otro con menos o iguales pasos Y más o igual energía. Eso es lo que hace que el espacio de estados no explote. Es el archivo más avanzado de esta carpeta y el patrón (comprimir a nodos clave + bitmask + frente de Pareto) es directamente útil en problemas de maratón.

leetcode\graphs\3568_minimum_moves_to_clean_the_classroom.py

```
1 from collections import deque
2 from heapq import heappush, heappop
3 from typing import List
4
5 class Solution:
6     def minMoves(self, classroom: List[str], energy: int) -> int:
7         filas = len(classroom)
8         columnas = len(classroom[0])
9
10        # -----
11        # 1. Encontrar S, L y R
12        # -----
13
14        especiales = []
15        inicio = -1
16        posiciones_L = {}
17
18        for i in range(filas):
19            for j in range(columnas):
20
21                tipo = classroom[i][j]
22
23                if tipo == 'S':
24                    inicio = len(especiales)
25                    especiales.append((i, j, 'S'))
26
27                elif tipo == 'L':
28                    indice = len(posiciones_L)
29                    posiciones_L[(i, j)] = indice
30                    especiales.append((i, j, 'L'))
31
32                elif tipo == 'R':
33                    especiales.append((i, j, 'R'))
34
35        cantidad_L = len(posiciones_L)
36
37        if cantidad_L == 0:
38            return 0
39
40        # -----
41        # 2. Posicion -> nodo
42        # -----
43
44        indice_especial = {}
45
46        for i, (x, y, tipo) in enumerate(especiales):
47            indice_especial[(x, y)] = i
48
49        cantidad_nodos = len(especiales)
50
51        # -----
52        # 3. Construir grafo
53        # -----
54
55        grafo = [[] for _ in range(cantidad_nodos)]
56
57        movimientos = [
58            (1, 0),
```

28.15 Minimum Moves To Clean The Classroom - Dijkstra sobre (nodo, bitmask) con Poda de Pareto (cont.)

```
59            (-1, 0),
60            (0, 1),
61            (0, -1)
62        ]
63
64        for nodo, (sx, sy, _) in enumerate(especiales):
65
66            cola = deque()
67            cola.append((sx, sy, 0))
68
69            visitados = {(sx, sy)}
70
71            while cola:
72
73                x, y, distancia = cola.popleft()
74
75                if distancia > 0 and (x, y) in indice_especial:
76
77                    destino = indice_especial[(x, y)]
78
79                    grafo[nodo].append(
80                        (destino, distancia)
81                    )
82
83                # IMPORTANTE:
84                #
85                # No seguimos atravesando el nodo especial.
86                #
87                # Si es R, Dijkstra debe encargarse de la
88                # recarga antes de continuar.
89                continue
90
91            for dx, dy in movimientos:
92
93                nx = x + dx
94                ny = y + dy
95
96                if 0 <= nx < filas and 0 <= ny < columnas:
97
98                    if (nx, ny) not in visitados:
99
100                        if classroom[nx][ny] != 'X':
101
102                            visitados.add((nx, ny))
103
104                            cola.append(
105                                (nx, ny, distancia + 1)
106                            )
107
108                # -----
109                # 4. Bitmask
110                # -----
111
112            objetivo = (1 << cantidad_L) - 1
113
114            # -----
115            # 5. Dijkstra + estados dominados
116            # -----
117
118            #
119            # Estado:
120            #
121            # (pasos, nodo, energía, mask)
122            #
123
124            heap = [
125                (0, inicio, energy, 0)
126            ]
127
128            #
129            # Para cada (nodo, mask) guardamos estados:
130            #
```

28.15 Minimum Moves To Clean The Classroom - Dijkstra sobre (nodo, bitmask) con Poda de Pareto (cont.)

```

81 # (pasos, energia)
82 #
83 # Solo mantenemos estados NO dominados.
84 #
85 # Un estado A domina a B si:
86 #
87 # A tiene <= pasos
88 # A tiene >= energia
89 #
90
91 frontera = {}
92
93 frontera[(inicio, 0)] = [
94     (0, energia)
95 ]
96
97 while heap:
98
99     pasos, nodo, energia_actual, mask = heappop(heap)
100
101     clave = (nodo, mask)
102
103     # -----
104     # Comprobar si este estado fue dominado
105     # -----
106
107     dominado = False
108
109     for pasos_viejos, energia_vieja in frontera.get(
110         clave, []
111     ):
112
113         if pasos_viejos <= pasos and energia_vieja >= energia_actual:
114
115             if pasos_viejos < pasos or energia_vieja > energia_actual:
116                 dominado = True
117                 break
118
119     if dominado:
120         continue
121
122     # -----
123     # Todas las L
124     # -----
125
126     if mask == objetivo:
127         return pasos
128
129     # -----
130     # Explorar vecinos
131     # -----
132
133     for siguiente, costo in grafo[nodo]:
134
135         if costo > energia_actual:
136             continue
137
138         nueva_energia = energia_actual - costo
139         nuevo_mask = mask
140
141         x, y, tipo = especiales[siguiente]
142
143         # -----
144         # L
145         # -----
146
147         if tipo == 'L':
148
149             indice_L = posiciones_L[(x, y)]
150
151             nuevo_mask |= 1 << indice_L
152

```

28.15 Minimum Moves To Clean The Classroom - Dijkstra sobre (nodo, bitmask) con Poda de Pareto (cont.)

```

203 # -----
204 # R
205 # -----
206
207 elif tipo == 'R':
208
209     nueva_energia = energia
210
211     nuevos_pasos = pasos + costo
212
213     nueva_clave = (siguiente, nuevo_mask)
214
215     # -----
216     # El nuevo estado esta dominado?
217     # -----
218
219     estados = frontera.get(nueva_clave, [])
220
221     dominado = False
222
223     for pasos_viejos, energia_vieja in estados:
224
225         if (
226             pasos_viejos <= nuevos_pasos
227             and energia_vieja >= nueva_energia
228         ):
229             dominado = True
230             break
231
232     if dominado:
233         continue
234
235     # -----
236     # El nuevo estado elimina estados que el domina
237     # -----
238
239     nuevos_estados = []
240
241     for pasos_viejos, energia_vieja in estados:
242
243         if not (
244             nuevos_pasos <= pasos_viejos
245             and nueva_energia >= energia_vieja
246         ):
247             nuevos_estados.append(
248                 (pasos_viejos, energia_vieja)
249             )
250
251     nuevos_estados.append(
252         (nuevos_pasos, nueva_energia)
253     )
254
255     frontera[nueva_clave] = nuevos_estados
256
257     heappush(
258         heap,
259         (
260             nuevos_pasos,
261             siguiente,
262             nueva_energia,
263             nuevo_mask
264         )
265     )
266
267 return -1

```

28.16 Network Delay Time - Dijkstra

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 743 "Network Delay Time": cuanto tarda una señal en llegar a todos los nodos. Técnica: Dijkstra con heap. La respuesta es la MAYOR de las distancias mínimas; si alguna queda en infinito, hay nodos inalcanzables y se responde -1. El if dist_actual > distancias[nodo] es la poda que evita reprocesar entradas viejas del heap, y es lo que hace que Dijkstra con heap sea $O(m \log n)$ sin necesitar decrease-key.

leetcode\graphs\0743_network_delay_time.py

```
1 import heapq
2 from collections import defaultdict
3
4 class Solution:
5     def networkDelayTime(self, times: List[List[int]], n: int, k: int) -> int:
6         def dijkstra(grafo, inicio):
7             distancias = {nodo: float('inf') for nodo in range(1, n+1)}
8             distancias[inicio] = 0
9             cola = [(0, inicio)]
10
11             while cola:
12                 dist_actual, nodo_actual = heapq.heappop(cola)
13
14                 if dist_actual > distancias[nodo_actual]:
15                     continue
16
17                 for vecino, peso in grafo[nodo_actual].items():
18                     distancia = dist_actual + peso
19                     if distancia < distancias[vecino]:
20                         distancias[vecino] = distancia
21                         heapq.heappush(cola, (distancia, vecino))
22
23             return distancias
24
25         graph = defaultdict(dict)
26
27         for u,v,w in times:
28             graph[u][v] = w
29         distances = dijkstra(graph, k).values()
30         res = 0
31         for i in list(distances):
32             if i > res:
33                 res = i
34         if res == float('inf'):
35             return -1
36         else:
37             return res
```

28.17 Network Recovery Pathways - Binaria sobre la Respuesta en un DAG

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3620 "Network Recovery Pathways": maximizar el peso MINIMO de las aristas del camino, con el costo total del camino acotado por k. Técnica: binaria sobre la respuesta. Se fija un umbral mid, se borran todas las aristas de peso menor, y con el orden topológico se calcula el camino mas BARATO que queda; si cabe en k, el umbral es alcanzable y se busca mas arriba. Maximizar un mínimo casi siempre sale así: binaria sobre el valor y una verificación fácil. El orden topológico es lo que permite el camino mínimo en $O(m)$ sin Dijkstra, porque es un DAG.

leetcode\graphs\3620_network_recovery_pathways.py

```
1 from collections import deque
2
3 class Solution:
4     def findMaxPathScore(self, edges: List[List[int]], online: List[bool], k: int) -> int:
5         n = len(online)
6
7         graph = [[] for _ in range(n)]
8         indeg = [0] * n
9         pesos = []
10
11         for u, v, w in edges:
12             graph[u].append((v, w))
13             indeg[v] += 1
14         pesos.append(w)
15
```

28.17 Network Recovery Pathways - Binaria sobre la Respuesta en un DAG (cont.)

```
16 # Orden topológico
17 topo = []
18 q = deque()
19
20 for i in range(n):
21     if indeg[i] == 0:
22         q.append(i)
23
24 while q:
25     u = q.popleft()
26     topo.append(u)
27     for v, _ in graph[u]:
28         indeg[v] -= 1
29         if indeg[v] == 0:
30             q.append(v)
31
32 def check(mid):
33     INF = 10**30
34     dp = [INF] * n
35     dp[0] = 0
36
37     for u in topo:
38         if dp[u] == INF:
39             continue
40
41         if u != 0 and u != n - 1 and not online[u]:
42             continue
43
44         for v, w in graph[u]:
45             if w < mid:
46                 continue
47
48             if v != n - 1 and not online[v]:
49                 continue
50
51             if dp[u] + w < dp[v]:
52                 dp[v] = dp[u] + w
53
54     return dp[n - 1] <= k
55
56 if not pesos:
57     return -1
58
59 l = 0
60 r = max(pesos)
61 ans = -1
62
63 while l <= r:
64     mid = (l + r) // 2
65
66     if check(mid):
67         ans = mid
68         l = mid + 1
69     else:
70         r = mid - 1
71
72 return ans
```

28.18 Number Of Provinces - Componentes Conexas

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 547 "Number of Provinces": cuantos grupos de ciudades conectadas hay. Técnica: contar componentes conexas con DFS y un set de visitados. Cada vez que se arranca un DFS nuevo se suma una componente. OJO: recorre los nodos del diccionario graph, no el rango 0..n-1. Si una ciudad no tiene ninguna arista no queda en el diccionario; aca no falla porque la matriz siempre marca la diagonal, pero con lista de adyacencia habría que recorrer range(n).

leetcode\graphs\0547_number_of_provinces.py

```
1 from collections import defaultdict
2 class Solution:
3     def findCircleNum(self, isConnected: List[List[int]]) -> int:
4         graph = defaultdict(list)
5         for i in range(len(isConnected)):
```

28.18 Number Of Provinces - Componentes Conexas (cont.)

```

6         for j in range(len(isConnected)):
7             if isConnected[i][j] == 1:
8                 graph[i].append(j)
9                 graph[j].append(i)
10        visited = set()
11        def dfs(curr):
12            visited.add(curr)
13            for node in graph[curr]:
14                if node not in visited:
15                    dfs(node)
16
17        res = 0
18        for node in graph:
19            if node not in visited:
20                dfs(node)
21                res+=1
22        return res

```

28.19 Reconstruct Itinerary - Camino Euleriano (Hierholzer)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 332 "Reconstruct Itinerary": usar todos los tiquetes una sola vez, empezando en JFK y escogiendo el itinerario lexicograficamente menor. Técnica: Hierholzer iterativo. Se avanza consumiendo aristas hasta quedarse atascado; el nodo donde uno se atasca se fija al camino y se retrocede. Al final se INVIERTE el camino. Ordenar los destinos y sacarlos en orden es lo que garantiza el menor lexicografico. OJO: deja un diccionario itinerarios cableado que no se usa para nada.

leetcode/graphs/0332_reconstruct_itinerary.py

```

1 from collections import defaultdict
2
3 class Solution:
4     def findItinerary(self, tickets: List[List[str]]) -> List[str]:
5         def encontrar_camino_euleriano(grafo, inicio="JFK"):
6             grafo_copia = {k: sorted(list(v), reverse=True) for k, v in grafo.items()}
7
8             pila = [inicio]
9             camino = []
10
11             while pila:
12                 nodo_actual = pila[-1]
13                 if nodo_actual in grafo_copia and grafo_copia[nodo_actual]:
14                     siguiente_nodo = grafo_copia[nodo_actual].pop()
15                     pila.append(siguiente_nodo)
16                 else:
17                     # Si no hay mas aristas salientes, fijamos el nodo en el camino
18                     camino.append(pila.pop())
19
20             camino.reverse()
21             return camino
22
23             itinerarios = {
24                 "JFK": ["SFO", "ATL"],
25                 "SFO": ["ATL"],
26                 "ATL": ["JFK", "SFO"]
27             }
28
29             graph = defaultdict(list)
30             for a, b in tickets:
31                 graph[a].append(b)
32             for node in graph:
33                 graph[node].sort()
34
35             sol = encontrar_camino_euleriano(graph)
36
37             return sol

```

28.20 Redundant Connection - DSU (Union-Find)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 684 "Redundant Connection": la arista que sobra en un arbol al que le agregaron una. Técnica: DSU. Se van uniendo las aristas en orden y la PRIMERA que encuentre los dos extremos ya en el mismo grupo es la que cierra el ciclo, o sea la respuesta. OJO: el find no tiene compresion de caminos ni el union por tamaño, así que en el peor caso es $O(n)$ por consulta y en Python puede reventar el límite de recursión. Con compresión queda casi $O(1)$: if parents[x] != x: parents[x] = find(parents[x]).

leetcode/graphs/0684_redundant_connection.py

```

1 class Solution:
2     def findRedundantConnection(self, edges: List[List[int]]) -> List[int]:
3         parents = {}
4         for a,b in edges:
5             parents[a] = a
6             parents[b] = b
7         def find(node):
8             if parents[node] == node:
9                 return node
10            return find(parents[node])
11        def union(a,b):
12            fatherA = find(a)
13            fatherB = find(b)
14            if fatherA == fatherB:
15                return False
16            parents[fatherA] = fatherB
17            return True
18
19        for a,b in edges:
20            if not union(a,b):
21                return [a,b]

```

28.21 Redundant Connection II - Ciclo en Grafo Dirigido por Casos

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 685 "Redundant Connection II": la arista que sobra, pero en un arbol DIRIGIDO. Técnica: hay dos síntomas posibles y se tratan por separado. Un nodo con grado de entrada 2 (dos padres) y/o un ciclo. Un DFS con el set visiting detecta el ciclo y guarda sus aristas; después se mira si hay nodo con dos padres. Si solo hay ciclo, se devuelve la última arista del ciclo en el orden de entrada; si hay nodo con dos padres, la que haya que quitar es la que además está en el ciclo. Es bastante más delicado que la versión I: aquí lo que importa son los CASOS, no la estructura.

leetcode/graphs/0685_redundant_connection_ii.py

```

1 from collections import defaultdict
2
3 class Solution:
4     def findRedundantDirectedConnection(self, edges: List[List[int]]) -> List[int]:
5         graph = defaultdict(list)
6         indegree = defaultdict(int)
7         nodes = set()
8
9         for a, b in edges:
10             graph[a].append(b)
11             indegree[b] += 1
12             nodes.add(a)
13             nodes.add(b)
14
15         visiting = set()
16         visited = set()
17         stack = []
18         edgeStack = []
19         ciclo = set()
20
21         def dfs(node):
22             visiting.add(node)
23             stack.append(node)
24
25             for nxt in graph[node]:
26                 if nxt in visiting:
27                     idx = stack.index(nxt)
28                     for i in range(idx, len(edgeStack)):
29                         ciclo.add(edgeStack[i])

```

28.21 Redundant Connection II - Ciclo en Grafo Dirigido por Casos (cont.)

```

30         ciclo.add((node, nxt))
31         return True
32
33     if nxt not in visited:
34         edgeStack.append((node, nxt))
35         if dfs(nxt):
36             return True
37         edgeStack.pop()
38
39     stack.pop()
40     visiting.remove(node)
41     visited.add(node)
42     return False
43
44     for node in nodes:
45         if node not in visited:
46             if dfs(node):
47                 break
48
49     # Buscar si existe un nodo con dos padres
50     doble = -1
51     for node in nodes:
52         if indegree[node] == 2:
53             doble = node
54             break
55
56     # Caso 1: solo ciclo
57     if doble == -1:
58         for a, b in reversed(edges):
59             if (a, b) in ciclo:
60                 return [a, b]
61
62     # Caso 2 o 3: nodo con dos padres
63     candidatos = []
64     for a, b in edges:
65         if b == doble:
66             candidatos.append([a, b])
67
68     # Si una de las dos aristas pertenece al ciclo
69     for a, b in candidatos:
70         if (a, b) in ciclo:
71             return [a, b]
72
73     # Si ninguna pertenece al ciclo
74     return candidatos[-1]

```

28.22 Remove Methods From Project - Alcanzabilidad y Contaminacion

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3310 "Remove Methods From Project": borrar el metodo k y todo lo que solo el invoca, salvo que algo de afuera dependa de eso. Tecnica: DFS desde k para marcar todo lo alcanzable (el grupo a borrar). Despues se revisa si alguna arista entra al grupo desde AFUERA; si si, no se puede borrar nada y se devuelve todo. Ese chequeo de arista que cruza el borde es la parte que la gente olvida.

leetcode/graphs/3310_remove_methods_from_project.py

```

1 class Solution:
2     def dfs(self, nodo, dic, vis):
3         if nodo not in vis and nodo != -1:
4             vis.add(nodo)
5             for i in dic[nodo]:
6                 self.dfs(i, dic, vis)
7
8     def remainingMethods(self, n: int, k: int, invocations: List[List[int]]) -> List[int]:
9         dic = {}
10        sol1 = []
11        for i in range(n):
12            dic[i] = [-1]
13            sol1.append(i)
14        for i in invocations:
15            dic[i[0]].append(i[1])
16        vis = set()
17        self.dfs(k, dic, vis)

```

28.22 Remove Methods From Project - Alcanzabilidad y Contaminacion (cont.)

```

8
8        sol = set()
20        for i in range(n):
21            for j in dic[i]:
22                if (i not in vis and j not in vis):
23                    sol.add(i)
24                elif (i not in vis and j in vis):
25                    return sol1
26        return list(sol)

```

28.23 Sum Of Distances In Tree - Rerooting en Arbol

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 834 "Sum of Distances in Tree": para cada nodo, la suma de distancias a todos los demas. Tecnica: tres DFS. El primero calcula el TAMANO de cada subarbol, el segundo la respuesta de la raiz, y el tercero la propaga a los hijos con la formula clave: $res[hijo] = res[padre] - tamaño(hijo) + (n - tamaño(hijo))$ que dice que al mover la raiz un paso, los nodos del subarbol quedan uno mas cerca y el resto uno mas lejos. Esa es la tecnica de REROOTING y aparece mucho en arboles. OJO: dfs3 usa $res[curr] \neq 0$ como marca de ya calculado, que es fragil si la respuesta real fuera 0.

leetcode/graphs/0834_sum_of_distances_in_tree.py

```

1 from collections import defaultdict
2 class Solution:
3     def sumOfDistancesInTree(self, n: int, edges: List[List[int]]) -> List[int]:
4         if len(edges) == 0:
5             return [0]
6         tree = defaultdict(list)
7         for a, b in edges:
8             tree[a].append(b)
9             tree[b].append(a)
10
11        sizes = defaultdict(int)
12        def dfs(curr, par):
13            total = 1
14            for node in tree[curr]:
15                if node != par:
16                    total += dfs(node, curr)
17            sizes[curr] = total
18            return total
19        dfs(0, None)
20
21        res = [0]*n
22        def dfs2(curr, depth, par):
23            res[0] += depth
24            for node in tree[curr]:
25                if node != par:
26                    dfs2(node, depth+1, curr)
27        dfs2(0, 0, None)
28
29        def dfs3(curr, prev, par):
30            currSum = 0
31            if res[curr] != 0:
32                currSum = res[curr]
33            else:
34                currSum = prev - sizes[curr] + (n-sizes[curr])
35            res[curr] = currSum
36            for node in tree[curr]:
37                if node != par:
38                    dfs3(node, currSum, curr)
39        dfs3(0, 0, None)
40        return res

```

29 - Hub LeetCode: Greedy

29.1 Maximum Number Of Coins You Can Get - Greedy sobre Ordenado

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1561 "Maximum Number of Coins You Can Get": se reparten los montones en tercias y de cada tercia uno se queda con el del medio; maximizar lo propio. Técnica: ordenar y de cada tercia botar el mas grande (se lo lleva Alice), tomar el segundo mas grande y botar el mas pequeño (se lo lleva Bob). Los pop() desde los dos extremos son exactamente eso.

leetcode/greedy/1561_maximum_number_of_coins_you_can_get.py

```
1 class Solution:
2     def maxCoins(self, piles: List[int]) -> int:
3         k = sorted(piles)
4         c = 0
5         while len(k)>0:
6             k.pop()
7             c += k[len(k)-1]
8             k.pop(0)
9             k.pop()
10        return c
```

29.2 Maximum Number Of Non Overlapping Palindrome Substrings - Greedy de Izquierda a Der...

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 2472 "Maximum Number of Non-overlapping Palindrome Substrings": maximo de palindromos de largo al menos k que no se traslapen. Técnica: solo hace falta probar longitudes k y k+1. Cualquier palindromo mas largo contiene uno de esas dos longitudes en el centro, y tomar el mas corto siempre deja mas espacio a la derecha. Con eso el greedy de tomar el primero que aparezca es optimo.

leetcode/greedy/2472_maximum_number_of_non-overlapping_palindrome_substrings.py

```
1 class Solution:
2     def maxPalindromes(self, s: str, k: int) -> int:
3         n = len(s)
4         if k == 1: return n
5
6         res = i = 0
7
8         while i <= n - k:
9             for d in (k, k + 1):
10                if i + d <= n and s[i : i + d] == s[i : i + d][::-1]:
11                    res += 1
12                    i += d
13                    break
14            else:
15                i += 1
16
17        return res
```

29.3 Stone Game - Greedy que Acierta por Casualidad

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 877 "Stone Game": dos jugadores toman de los extremos de una fila de montones y gana quien junte mas piedras. Técnica del código: ordena los montones y los reparte alternando de mayor a menor. OJO: eso NO es el juego, en el juego solo se puede tomar de los EXTREMOS. Pasa porque con una cantidad par de montones el primero siempre gana y la respuesta es True siempre, como dice el comentario del final. Sirve de ejemplo de que pasar no es lo mismo que estar bien.

leetcode/greedy/0877_stone_game.py

```
1 class Solution:
2     def stoneGame(self, piles: List[int]) -> bool:
3         piles = sorted(piles)
4         A = 0
5         B = 0
6         va = True
7         i = len(piles)-1
8         while i > -1:
9             if(va):
10                A += piles[i]
11                va = not va
12            else:
13                B += piles[i]
14                va = not va
```

29.3 Stone Game - Greedy que Acierta por Casualidad (cont.)

```
5         i -= 1
6         if(A > B):
7             return True
8         else:
9             return False
10
11
12
13
14
15
16
17
18
19
20
21
22        #return True always
```

30 - Hub LeetCode: Heap

30.1 Construct Target Array With Multiple Sums - Deshacer el Proceso al Revés

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1354 "Construct Target Array With Multiple Sums": desde un arreglo de unos, cada paso reemplaza un elemento por la suma total; decir si se llega al objetivo. Técnica: ir AL REVÉS. El mayor del arreglo tuvo que ser el ultimo escrito, así que antes valia mayor - (suma del resto); con un heap maximo se devuelve ese paso y se repite. El modulo evita hacer las restas una por una cuando el resto es pequeño. OJO: deja un print de depuracion adentro del ciclo y varios cortes ad hoc.

leetcode/heap/1354_construct_target_array_with_multiple_sums.py

```
1 import heapq
2 class Solution:
3     def isPossible(self, target: List[int]) -> bool:
4         tam = []
5         for i in target:
6             heapq.heappush(tam, i*-1)
7
8         if(len(target) == 1 and 1 in target):
9             return True
10        elif(len(target) == 1 and 1 not in target):
11            return False
12
13        cantidad = 0
14        suma = sum(tam)
15        ant = tam.copy()
16        while cantidad != len(target):
17            mayor = heapq.heappop(tam)
18            suma -= mayor
19            if(mayor == suma):
20                break
21            me = (-mayor) % (-suma)
22            if(me == 0):
23                me = -suma
24            elif(me < 0):
25                break
26            heapq.heappush(tam, me*-1)
27            cantidad = tam.count(-1)
28            print(mayor, suma, me, tam, ant)
29            suma -= me
30            if(tam == ant):
31                break
32            ant = tam.copy()
33
34        if(cantidad != len(target)):
35            return False
36        else:
37            return True
```

30.2 Find K Pairs With Smallest Sums - Heap Maximo de Tamano k

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 373 "Find K Pairs with Smallest Sums": los k pares (a, b) de menor suma. Técnica: heap de tamaño k con los signos invertidos (Python solo trae heap mínimo, así que se guarda -valor para simular un maximo) y un diccionario suma -> lista de pares para poder reconstruir cuales eran. OJO: arranca con un caso especial cableado, if nums1 == nums2 and 3 not in nums1 responde [[1,1]]*k. Eso es un parche para un caso de prueba, no parte de la solución.

leetcode/heap/0373_find_k_pairs_with_smallest_sums.py

```
1 import heapq
2 class Solution:
```

30.2 Find K Pairs With Smallest Sums - Heap Maximo de Tamano k (cont.)

```

3 def kSmallestPairs(self, nums1: List[int], nums2: List[int], k: int) -> List[List[int]]:
4
5     if(nums1 == nums2 and 3 not in nums1):
6         return [[1,1]]*k
7
8     tam = []
9     h = []
10    dic = {}
11    may = 0
12    for i in nums1:
13        for j in nums2:
14            l = i+j
15            if(len(tam)<k or l <= may):
16                if(l==may):
17                    may = l
18                elif(len(tam)>k and may >= l):
19                    may = -heapq.heappop(tam)
20                #tam.append(l)
21                heapq.heappush(tam, l*-1)
22                if(l not in dic):
23                    dic[l] = [[i,j]]
24                else:
25                    dic[l].append([i,j])
26            else:
27                break
28
29    for i in range(len(tam)):
30        tam[i] *= -1
31    #print(tam)
32    tam = sorted(tam)
33    fi = []
34    i = 0
35    while i < len(tam):
36        q = dic[tam[i]]
37        if(len(q) == 1):
38            fi.append(q[0])
39            i+=1
40        else:
41            for j in range(len(q)):
42                fi.append(q[j])
43            i+=1
44
45    return fi[:k]

```

30.3 Maximum Number Of Eaten Apples - Heap por Fecha de Vencimiento

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1705 "Maximum Number of Eaten Apples": cada dia aparecen manzanas que se danan en cierta fecha y se puede comer una por dia; maximizar cuantas se comen. Tecnica: heap minimo con la pareja (dia en que se dana, cuantas quedan). Cada dia se come de las que vencen PRIMERO, que es el greedy correcto: guardar una que vence lejos nunca es peor que guardar una que vence ya. OJO: usa try/except para detectar el heap vacio en vez de revisar el tamano.

leetcode/heap/1705_maximum_number_of_eaten_apples.py

```

1 import heapq
2
3 class Solution:
4     def eatenApples(self, apples: List[int], days: List[int]) -> int:
5         heap = []
6         d = 0
7         c = 0
8         while True:
9             if(d<len(days)):
10                 heapq.heappush(heap, (days[d]+d, apples[d]))
11             if (not heap):
12                 break
13             try:
14                 menor,quedan = heapq.heappop(heap)
15                 while menor<=d or quedan < 1:
16                     menor,quedan = heapq.heappop(heap)
17                 quedan -= 1
18                 c+= 1
19                 heapq.heappush(heap, (menor,quedan))

```

30.3 Maximum Number Of Eaten Apples - Heap por Fecha de Vencimiento (cont.)

```

20 except:
21     d+=1
22     continue
23     d+=1
24     return c

```

31 - Hub LeetCode: Linked List

31.1 Add Two Numbers - Suma con Acarreo en Lista Ligada

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 2 "Add Two Numbers": sumar dos numeros guardados en listas ligadas, con el digito menos significativo primero. Tecnica: recorrer las dos listas a la vez sumando digito por digito, y en una segunda pasada propagar el acarreo (si pasa de 9 se le resta 10 y se suma 1 al siguiente). Si el acarreo se sale por el final, se agrega un digito mas. Lo normal es hacerlo en UNA pasada con una variable carry, sin la lista intermedia.

leetcode/linked_list/0002_add_two_numbers.py

```

1 # Definition for singly-linked list.
2 # class ListNode:
3 #     def __init__(self, val=0, next=None):
4 #         self.val = val
5 #         self.next = next
6 class Solution:
7     def addTwoNumbers(self, l1: Optional[ListNode], l2: Optional[ListNode]) -> Optional[ListNode]:
8         f = []
9         cabeza1 = l1
10        cabeza2 = l2
11        while True:
12            if(cabeza1 and cabeza2):
13                v = cabeza1.val + cabeza2.val
14                f.append(v)
15                cabeza1 = cabeza1.next
16                cabeza2 = cabeza2.next
17            elif(cabeza1):
18                v = cabeza1.val
19                f.append(v)
20                cabeza1 = cabeza1.next
21            elif(cabeza2):
22                v = cabeza2.val
23                f.append(v)
24                cabeza2 = cabeza2.next
25            else:
26                break
27
28        k = []
29        i = 0
30        while (i < len(f)):
31            if(f[i]<=9):
32                k.append(f[i])
33            else:
34                k.append(f[i]-10)
35                if(i==len(f)-1):
36                    f.append(1)
37                else:
38                    f[i+1] += 1
39                i+=1
40        final = ListNode()
41        final.val = k[0]
42        cabeza = final
43        for i in range(1, len(k)):
44            nuevo = ListNode()
45            nuevo.val = k[i]
46            cabeza.next = nuevo
47            cabeza = cabeza.next
48
49        return final

```

31.2 Copy List With Random Pointer - Atajo de Libreria

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 138 "Copy List with Random Pointer": copia profunda de una lista con un puntero extra que apunta a cualquier nodo. Técnica del código: copy.deepcopy(head). Una línea y pasa. OJO: se salta el ejercicio. Lo que se buscaba es la técnica del diccionario nodo viejo -> nodo nuevo en una primera pasada, y en la segunda conectar next y random usando ese mapa. Ese patron si sirve para clonar grafos (LeetCode 133 es el mismo).

leetcode\linked_list\0138_copy_list_with_random_pointer.py

```
1 import copy
2 class Solution:
3     def copyRandomList(self, head: 'Optional[Node]') -> 'Optional[Node]':
4         return copy.deepcopy(head)
```

31.3 Insertion Sort List - Insertion Sort en Lista Ligada

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 147 "Insertion Sort List": ordenar una lista ligada con insercion. Técnica: se va armando una lista ordenada aparte y cada nodo se inserta buscando su lugar desde el inicio. Guardar siguiente ANTES de mover actual.next es lo unico delicado, si no se pierde el resto de la lista. Es $O(n^2)$. En lista ligada el que si sirve en serio es merge sort, que baja a $O(n \log n)$ sin necesitar acceso por indice.

leetcode\linked_list\0147_insertion_sort_list.py

```
1 class Solution:
2     def insertionSortList(self, head: ListNode | None) -> ListNode | None:
3
4         if head is None:
5             return None
6
7         ordenada = None
8         actual = head
9
10        while actual:
11            siguiente = actual.next
12            if ordenada is None or actual.val < ordenada.val:
13                actual.next = ordenada
14                ordenada = actual
15            else:
16                p = ordenada
17
18                while p.next and p.next.val < actual.val:
19                    p = p.next
20
21                actual.next = p.next
22                p.next = actual
23
24                actual = siguiente
25
26        return ordenada
```

31.4 Linked List Cycle - Liebre y Tortuga

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 141 "Linked List Cycle": decir si la lista tiene un ciclo. Técnica: dos punteros, uno avanza de a uno y el otro de a dos. Si hay ciclo se encuentran; si no, el rapido llega al final. Es el algoritmo de Floyd y gasta $O(1)$ de memoria, que es la ventaja sobre el set de visitados (como dice el comentario del autor). El mismo truco encuentra el INICIO del ciclo: al encontrarse, se manda uno a la cabeza y se avanzan los dos de a uno hasta que coincidan.

leetcode\linked_list\0141_linked_list_cycle.py

```
1 # Definition for singly-linked list.
2 # class ListNode:
3 #     def __init__(self, x):
4 #         self.val = x
5 #         self.next = None
6
7 class Solution:
8     def hasCycle(self, head: Optional[ListNode]) -> bool:
9         slow = head
10        fast = head
11
12        #Puedo hacer el de visitados y sale, pero para practicar eso
13        #uno mas rapido que otro
```

31.4 Linked List Cycle - Liebre y Tortuga (cont.)

```
14
15        while fast != None and fast.next != None:
16            slow = slow.next
17            fast = fast.next.next
18
19            if slow == fast:
20                return True
21
22        return False
```

31.5 Merge Two Sorted Lists - Fusion de Listas Ligadas

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 21 "Merge Two Sorted Lists": mezclar dos listas ligadas ordenadas. Técnica del código: cirugía de punteros en el sitio, insertando los nodos de una lista dentro de la otra segun cual cabeza sea menor, y al final se pega la cola que sobro. OJO: es difícil de seguir y tiene casos simetricos duplicados. El truco que lo vuelve corto es el NODO CENTINELA: se crea un ListNode(0) falso, se van colgando nodos de la lista menor y al final se devuelve centinela.next. Asi desaparecen todos los if de bordes.

leetcode\linked_list\0021_merge_two_sorted_lists.py

```
1 # Definition for singly-linked list.
2 # class ListNode:
3 #     def __init__(self, val=0, next=None):
4 #         self.val = val
5 #         self.next = next
6 class Solution:
7     def mergeTwoLists(self, list1: Optional[ListNode], list2: Optional[ListNode]) -> Optional[ListNode]:
8         if(list1 == None and list2 == None):
9             return list1
10        elif(list1 == None):
11            return list2
12        elif(list2 == None):
13            return list1
14        h1 = list1
15        h2 = list2
16        cabeza1 = list1
17        cabeza2 = list2
18        if(cabeza1.val <= cabeza2.val):
19            ant = cabeza1
20            ant2 = cabeza2
21            while (cabeza1 != None):
22                if(cabeza2 == None):
23                    return list1
24                if(cabeza1.val >= cabeza2.val):
25                    cabeza1 = ant
26                    cabeza2 = cabeza2.next
27                    ant2.next = cabeza1.next
28                    cabeza1.next = ant2
29                    ant2 = cabeza2
30            ant = cabeza1
31            cabeza1 = cabeza1.next
32
33        cabeza1 = h1
34        while (cabeza1.next != None):
35            cabeza1 = cabeza1.next
36        cabeza1.next = cabeza2
37        return list1
38    else:
39        ant = cabeza1
40        ant2 = cabeza2
41        while (cabeza2 != None):
42            if(cabeza1 == None):
43                return list2
44            if(cabeza2.val >= cabeza1.val):
45                cabeza2 = ant2
46                cabeza1 = cabeza1.next
47                ant.next = cabeza2.next
48                cabeza2.next = ant
49                ant = cabeza1
50            ant2 = cabeza2
```

31.5 Merge Two Sorted Lists - Fusion de Listas Ligadas (cont.)

```

51         cabeza2 = cabeza2.next
52     cabeza2 = h2
53     while (cabeza2.next != None):
54         cabeza2 = cabeza2.next
55     cabeza2.next = cabeza1
56     return list2

```

31.6 Odd Even Linked List - Separar en Dos Listas

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 328 "Odd Even Linked List": reordenar para que queden primero las posiciones impares y desp es las pares, conservando el orden relativo. T cnica: se arman dos listas mientras se recorre, una con las posiciones impares y otra con las pares, y al final se pega la de pares al final de la de impares. La variable voy lleva la posici n, no el valor, que es lo que puede confundir al leerlo. El caso de un solo nodo va aparte porque ah  la lista de pares nunca se crea.

leetcode/linked_list/0328_odd_even_linked_list.py

```

1 # Definition for singly-linked list.
2 # class ListNode:
3 #     def __init__(self, val=0, next=None):
4 #         self.val = val
5 #         self.next = next
6 class Solution:
7     def oddEvenList(self, head: Optional[ListNode]) -> Optional[ListNode]:
8         if(head is None):
9             return head
10        voy = 1
11        impar = ListNode(head.val)
12        p = impar
13        while head != None:
14            head = head.next
15            if(head is None):
16                break
17            voy += 1
18            if(voy % 2 == 0 and voy == 2):
19                par = ListNode(head.val)
20                q = par
21            elif(voy % 2 == 0):
22                new2 = ListNode(head.val)
23                par.next = new2
24                par = par.next
25            else:
26                new2 = ListNode(head.val)
27                impar.next = new2
28                impar = impar.next
29        if(voy != 1):
30            impar.next = q
31            return p
32        else:
33            return impar

```

31.7 Palindrome Linked List - Palindromo en Lista Ligada

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 234 "Palindrome Linked List": decir si la lista se lee igual al reves. T cnica: se mide el largo, se parte a la mitad y se guardan la primera mitad en orden y la segunda al reves (con insert(0, ...)), y se comparan. La bandera flag maneja el caso de largo impar, donde el elemento del centro se ignora. OJO: deja dos prints, y el insert(0, ...) es O(n) cada vez, as  que esa parte es O(n^2). La version O(1) de memoria: liebre y tortuga para hallar el medio, invertir la segunda mitad en el sitio y comparar.

leetcode/linked_list/0234_palindrome_linked_list.py

```

1 # Definition for singly-linked list.
2 # class ListNode:
3 #     def __init__(self, val=0, next=None):
4 #         self.val = val
5 #         self.next = next
6 class Solution:
7     def isPalindrome(self, head: Optional[ListNode]) -> bool:
8         num2 = 0
9         punt = head
10        while(punt != None):
11            punt = punt.next

```

31.7 Palindrome Linked List - Palindromo en Lista Ligada (cont.)

```

12        num2+=1
13        if(num2%2==0):
14            num=num2/2
15            flag = False
16        else:
17            num=int(num2/2)
18            flag = True
19        punt1 = head
20        p1 = []
21        p2 = []
22        cont = 0
23        while(cont < num2):
24            if(flag == False):
25                if(cont >= num):
26                    p2.insert(0,punt1.val)
27            else:
28                p1.append(punt1.val)
29            else:
30                if(cont > num):
31                    p2.insert(0,punt1.val)
32                elif(cont<num):
33                    p1.append(punt1.val)
34            punt1 = punt1.next
35            cont+=1
36        print(p1)
37        print(p2)
38        if(p1 == p2 or num2==1):
39            return True
40        else:
41            return False

```

31.8 Remove Duplicates From Sorted List - Recorrido de Lista Ligada

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 83 "Remove Duplicates from Sorted List": quitar los repetidos de una lista ordenada. T cnica: como esta ordenada, los repetidos estan pegados, as  que basta recordar el  ltimo valor visto y solo copiar cuando cambia. Aca construye una lista NUEVA en vez de saltarse nodos. En el sitio seria mas barato: si nodo.val == nodo.next.val entonces nodo.next = nodo.next.next.

leetcode/linked_list/0083_remove_duplicates_from_sorted_list.py

```

1 # Definition for singly-linked list.
2 # class ListNode:
3 #     def __init__(self, val=0, next=None):
4 #         self.val = val
5 #         self.next = next
6 class Solution:
7     def deleteDuplicates(self, head: Optional[ListNode]) -> Optional[ListNode]:
8         if(head is None):
9             return head
10        voy = head.val
11        new = ListNode(head.val)
12        p = new
13        while head != None:
14            if(head.val != voy):
15                voy = head.val
16                new2 = ListNode(head.val)
17                new.next = new2
18                new = new.next
19            head = head.next
20        return p

```

31.9 Reverse Linked List - Invertir Lista Ligada

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 206 "Reverse Linked List": invertir la lista. T cnica del codigo: pasar los valores a un arreglo y reconstruir la lista al reves. Funciona pero gasta O(n) de memoria. La forma cl sica, que vale la pena tener de memoria, son tres punteros: prev = None y en cada paso guardar nxt = act.next, apuntar act.next = prev, y correr prev = act, act = nxt. Al final prev es la nueva cabeza. Es O(1) de memoria y cinco lineas.

leetcode/linked_list/0206_reverse_linked_list.py

```

1 # Definition for singly-linked list.

```

31.9 Reverse Linked List - Invertir Lista Ligada (cont.)

```
1 # class ListNode:
2 #     def __init__(self, val=0, next=None):
3 #         self.val = val
4 #         self.next = next
5
6 class Solution:
7     def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
8         if(head is None):
9             return head
10
11         nodos = []
12         while head != None:
13             nodos.append(head.val)
14             head = head.next
15
16         prev = ListNode(nodos[len(nodos)-1])
17         q = prev
18         for i in range(len(nodos)-2,-1,-1):
19             act = ListNode(nodos[i])
20             prev.next = act
21             prev = prev.next
22
23         return q
```

32 - Hub LeetCode: Math

32.1 Counts Commas In Range I - Conteo por Magnitud

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3870 "Count Commas in Range I": cuantas comas de separador de miles se escriben en total al listar los numeros hasta n. Técnica: la cantidad de comas de un numero depende solo de cuantos digitos tiene (una cada tres digitos). El ciclo cuenta cuantos grupos de tres tiene n y multiplica por cuantos numeros hay desde 1000. OJO: es la version I, donde todos los numeros del rango caben en la misma banda de magnitud. Cuando el rango cruza varias bandas hay que sumar banda por banda, y eso es el 3871.

leetcode/math/3870_counts_commas_in_range_i.py

```
1 class Solution:
2     def countCommas(self, n: int) -> int:
3         if(len(str(n))<=3):
4             return 0
5         else:
6             k = 1000
7             c = 1
8             ja = 0
9             while True:
10                 if(n//k == 0):
11                     break
12                 if(ja==3):
13                     c+=1
14                     ja = 0
15                 k*=10
16                 ja+=1
17             k=1000
18             return int((n-k+1)*c)
```

32.2 Counts Commas In Range II - Conteo por Bandas de Magnitud

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3871 "Count Commas in Range II": la misma cuenta de comas, pero con rangos grandes. Técnica: bajar por bandas. Se toma la potencia de 10 mas cercana por debajo de n, se cuenta cuantos numeros hay de ahi a n (todos con la misma cantidad de comas, (digitos-1)//3), se suma y se repite con n = potencia - 1 hasta llegar por debajo de 1000. Ese partir el rango en tramos donde la respuesta es constante es la idea reutilizable.

leetcode/math/3871_counts_commas_in_range_ii.py

```
1 class Solution:
2     def countCommas(self, n: int) -> int:
3         cont = 0
4         num = int("1"+"0"*(len(str(n))-1))
5         while num > 999:
6             div = (len(str(num))-1)//3
7             cont+=(n-num+1)*div
```

32.2 Counts Commas In Range II - Conteo por Bandas de Magnitud (cont.)

```
8         n=num-1
9         num = int("1"+"0"*(len(str(num))-2))
10        return cont
```

32.3 Palindrome Number - Palindromo con Dos Punteros

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 9 "Palindrome Number": decir si un entero se lee igual al revés. Técnica: pasarlo a string y comparar el caracter i con el de la otra punta hasta la mitad. OJO: los negativos nunca son palindromos por el signo, y aca el '-' se compara como un caracter mas, así que -121 da False por casualidad y no por la regla.

leetcode/math/0009_palindrome_number.py

```
1 class Solution:
2     def isPalindrome(self, x: int) -> bool:
3         b = str(x)
4         i = 0
5         while i < len(b)/2:
6             if(b[i]==b[len(b)-1-i]):
7                 i+=1
8             continue
9         else:
10            return False
11        return True
```

32.4 Roman To Integer - Numeros Romanos

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 13 "Roman to Integer": convertir un numero romano a entero. Técnica: dos diccionarios, uno de simbolos sueltos y otro de los seis pares restadores (IV, IX, XL, XC, CD, CM). Primero se consumen los pares y despues se suma lo que sobra. La forma clasica es un solo barrido: si el valor actual es menor que el siguiente, se resta. Sale mas corto y no hace falta el segundo diccionario.

leetcode/math/0013_roman_to_integer.py

```
1 class Solution:
2     def romanToInt(self, s: str) -> int:
3         dic = {"I":1, "V":5, "X":10, "L":50, "C":100, "D":500, "M":1000}
4         dic2 = {"IV":4, "IX":9, "XL":40, "XC":90, "CD":400, "CM":900}
5         s = list(s)
6         i = 0
7         cont = 0
8         while i < len(s):
9             try:
10                 su = str(s[i])+str(s[i+1])
11                 cont +=dic2[su]
12                 s.pop(i+1)
13                 s.pop(i)
14                 continue
15             except:
16                 a=0
17                 i+=1
18             for i in s:
19                 cont+=dic[i]
20        return cont
```

32.5 Smallest Divisible Digit Product I - Fuerza Bruta

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3345 "Smallest Divisible Digit Product I": el menor numero mayor o igual que n cuyo producto de digitos sea multiplo de t. Técnica: probar n, n+1, n+2, ... calculando math.prod de los digitos. Con los limites del problema (t hasta 10) la respuesta esta muy cerca, así que la fuerza bruta alcanza. Sirve de recordatorio de math.prod, que ahorra escribir el ciclo del producto.

leetcode/math/3345_smallest_divisible_digit_product_i.py

```
1 import math
2 class Solution:
3     def smallestNumber(self, n: int, t: int) -> int:
4         while True:
5             s = str(n)
6             u = math.prod(list(map(int,s)))
7             if(u%t == 0):
8                 break
```

32.5 Smallest Divisible Digit Product I - Fuerza Bruta (cont.)

```

9         n += 1
10    return n

```

32.6 Sqrt(x) - Raiz Entera

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 69 "Sqrt(x)": la parte entera de la raiz cuadrada, sin usar sqrt. Tecnica: probar $i = 0, 1, 2, \dots$ hasta que i^2 pase de x . Es $O(\sqrt{x})$, suficiente aqui. Con busqueda binaria entre 0 y x queda en $O(\log x)$, que es la version que uno quiere si el limite sube. Cuidado con sqrt() de punto flotante, que en los bordes se equivoca en 1.

leetcode/math/0069_sqrt(x).py

```

1 class Solution:
2     def mySqrt(self, x: int) -> int:
3         for i in range(x//2+2):
4             k = i*i
5             if(k>x):
6                 return i-1
7             elif(k==x):
8                 return i
9             else:
10                continue
11    return 1

```

32.7 Sum Of Square Numbers - Dos Punteros sobre Cuadrados

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 633 "Sum of Square Numbers": decir si c se puede escribir como $a^2 + b^2$. Tecnica: se avanza a^2 desde 0 y se baja b^2 desde floor(sqrt(c)) al mismo tiempo, probando en cada paso si el complemento es un cuadrado perfecto. Es la idea de dos punteros desde los extremos, aunque escrita de forma poco obvia. La version clara: mientras $a \leq b$, si $a^2 + b^2 == c$ listo, si es menor sube a , si no baja b .

leetcode/math/0633_sum_of_square_numbers.py

```

1 import math
2 class Solution:
3     def judgeSquareSum(self, c: int) -> bool:
4         if(c == 0):
5             return True
6         b1 = math.floor(math.sqrt(c))
7         for a2 in range(b1):
8             a1 = math.floor(math.sqrt(c-(b1*b1)))
9             b2 = math.floor(math.sqrt(c-(a2*a2)))
10
11            if(((a1*a1)+(b1*b1)) == c):
12                return True
13            elif(((a2*a2)+(b2*b2)) == c):
14                return True
15            b1 -= 1
16    return False

```

32.8 Total Waviness Of Numbers In Range I - Fuerza Bruta sobre Digitos

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3751 "Total Waviness of Numbers in Range I": contar los picos y valles de los digitos de todos los numeros del rango. Tecnica: recorrer el rango entero y en cada numero mirar las ternas de digitos consecutivos: es pico si el del medio es mayor que sus dos vecinos, valle si es menor. Es la version I, con rango chico. La II pediria digit DP, o sea contar por posicion en vez de recorrer numero por numero.

leetcode/math/3751_total_waviness_of_numbers_in_range_i.py

```

1 class Solution:
2     def totalWaviness(self, num1: int, num2: int) -> int:
3         cont=0
4         for i in range(num1,num2+1,1):
5             a = str(i)
6             for j in range(1,len(a)-1,1):
7                 if(a[j-1]<a[j] and a[j+1] < a[j]):
8                     cont+=1
9                 elif(a[j-1]>a[j] and a[j+1] > a[j]):
10                    cont+=1
11    return cont

```

33 - Hub LeetCode: Matrix

33.1 Image Overlap - Conteo de Desplazamientos

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 835 "Image Overlap": deslizando una imagen binaria sobre otra, maximo de unos que se superponen. Tecnica: en vez de probar cada traslacion, se saca la lista de posiciones con 1 de cada imagen y para cada par se anota el desplazamiento (dr, dc) en un diccionario. El desplazamiento que mas veces aparece ES la respuesta. $O(u_1 \cdot u_2)$ con u = cantidad de unos.

leetcode/matrix/0835_image_overlap.py

```

1 class Solution:
2     def largestOverlap(self, img1, img2):
3         n = len(img1)
4
5         ones1 = [(r, c) for r in range(n) for c in range(n) if img1[r][c] == 1]
6         ones2 = [(r, c) for r in range(n) for c in range(n) if img2[r][c] == 1]
7
8         frequency = {}
9         maxOverlap = 0
10
11        for r1, c1 in ones1:
12            for r2, c2 in ones2:
13                dr = r1 - r2
14                dc = c1 - c2
15
16                key = (dr, dc)
17                frequency[key] = frequency.get(key, 0) + 1
18                maxOverlap = max(maxOverlap, frequency[key])
19
20    return maxOverlap

```

33.2 Island Perimeter - Recorrido de Matriz

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 463 "Island Perimeter": perimetro de la isla dibujada con 1 en una matriz. Tecnica: por cada celda de tierra se cuentan sus vecinos de tierra y se aporta 4 - vecinos. No hace falta DFS ni marcar visitados, es un solo barrido.

leetcode/matrix/0463_island_perimeter.py

```

1 class Solution(object):
2     def islandPerimeter(self, grid):
3         n = len(grid)
4         m = len(grid[0])
5         p = 0
6         for i in range(len(grid)):
7             for j in range(len(grid[0])):
8                 if(grid[i][j]==1):
9                     l = 0
10                    if(i != 0 and grid[i-1][j] == 1):
11                        l+=1
12                    if(i != n-1 and grid[i+1][j] == 1):
13                        l+=1
14                    if(j != 0 and grid[i][j-1] == 1):
15                        l+=1
16                    if(j != m-1 and grid[i][j+1] == 1):
17                        l+=1
18                    if(l==0):
19                        p+=4
20                    elif(l==1):
21                        p+=3
22                    elif(l==2):
23                        p+=2
24                    elif(l==3):
25                        p+=1
26    return p

```

34 - Hub LeetCode: Prefix Sum

34.1 Continuous Subarray Sum - Residuos de Prefijos

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 523 "Continuous Subarray Sum": decir si hay un subarreglo de largo al menos 2 con suma múltiplo de k. Técnica: la suma de $[i+1..j]$ es múltiplo de k si los prefijos i y j dejan el MISMO residuo. Se guarda en un diccionario el residuo -> primera posición donde apareció, y se exige que la distancia sea al menos 2. El dic arranca con {0: -1} para cubrir el prefijo vacío. Guardar solo la PRIMERA aparición es lo que maximiza la distancia.

leetcode/prefix_sum/0523_continuous_subarray_sum.py

```
1 class Solution:
2     def checkSubarraySum(self, nums: List[int], k: int) -> bool:
3         if(len(nums)==1):
4             return False
5         dic = {0: -1}
6         res = 0
7         ini = 0
8         for i in range(len(nums)):
9             ini += nums[i]
10            res = ini%k
11            if(res not in dic):
12                dic[res] = i
13            else:
14                if i - dic[res] >= 2:
15                    return True
16        return False
```

34.2 Make Sum Divisible By P - Residuos de Prefijos

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1590 "Make Sum Divisible by P": quitar el subarreglo mas corto para que lo que queda sea múltiplo de p. Técnica: si la suma total deja residuo f, el subarreglo que se quita tiene que dejar residuo f también. Con prefijos: se busca un residuo anterior igual a (suma_actual - f) mod p en un diccionario residuo -> última posición. El (suma - f) % p en Python ya sale positivo. La condición longitud < len(nums) es porque no se puede quitar todo el arreglo.

leetcode/prefix_sum/1590_make_sum_divisible_by_p.py

```
1 class Solution:
2     def minSubarray(self, nums: List[int], p: int) -> int:
3         j = sum(nums)
4         f = j % p
5         if(j%p == 0):
6             return 0
7         dic = {0: -1} #esto es que ya encontro
8         suma = 0
9         g = []
10        for pos in range(len(nums)):
11            actual = nums[pos]
12            suma += actual%p
13            suma %= p #voy mirando cuanto me aporta cada numero
14
15            n = (suma - f)%p #Esta ecuacion me dice cuanto hace falta para poder eliminarlo
16            if(n in dic):
17                longitud = pos - dic[n]
18                if longitud < len(nums): #El ejercicio dice que no puedo quitar todos los numeros
19                    g.append(longitud)
20            dic[suma] = pos
21
22        if(len(g) == 0):
23            return -1
24        return min(g)
```

34.3 Minimum Operations To Reduce X To Zero - Prefijos y Sufijos

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1658 "Minimum Operations to Reduce X to Zero": se quitan elementos solo de los extremos hasta sumar exactamente x; mínimo de operaciones. Técnica: lo que se quita es un PREFIJO mas un SUFIJO. Se arman las dos sumas acumuladas y dos diccionarios suma -> primera posición, y para cada prefijo se busca el sufijo que complete x. La condición $i + pos + 2 \leq n$ evita que se traslapen. La otra forma es sliding window sobre el subarreglo del medio, que debe sumar total - x.

leetcode/prefix_sum/1658_minimum_operations_to_reduce_x_to_zero.py

```
1 class Solution:
```

34.3 Minimum Operations To Reduce X To Zero - Prefijos y Sufijos (cont.)

```
2     def minOperations(self, nums: List[int], x: int) -> int:
3         suma1 = []
4         suma2 = []
5         posiciones1 = {}
6         posiciones2 = {}
7
8         s1 = 0
9         s2 = 0
10
11        for i in range(len(nums)):
12            s1 += nums[i]
13            s2 += nums[len(nums)-i-1]
14
15            suma1.append(s1)
16            suma2.append(s2)
17
18            if s1 not in posiciones1:
19                posiciones1[s1] = i
20
21            if s2 not in posiciones2:
22                posiciones2[s2] = i
23
24        mejor = float("inf")
25
26        for i in range(len(suma1)):
27
28            if x-suma1[i] in posiciones2:
29                pos = posiciones2[x-suma1[i]]
30
31                if i + pos + 2 <= len(nums):
32                    best = i + pos + 2
33                    if best < mejor:
34                        mejor = best
35
36            if x-suma2[i] in posiciones1:
37                pos = posiciones1[x-suma2[i]]
38
39                if i + pos + 2 <= len(nums):
40                    best = i + pos + 2
41                    if best < mejor:
42                        mejor = best
43
44            if x-suma1[i] == 0:
45                best = i + 1
46                if best < mejor:
47                    mejor = best
48
49            if x-suma2[i] == 0:
50                best = i + 1
51                if best < mejor:
52                    mejor = best
53
54        if mejor == float("inf"):
55            return -1
56
57        return mejor
```

34.4 Ways To Make A Fair Array - Prefijos Pares e Impares por Separado

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1664 "Ways to Make a Fair Array": cuantos índices se pueden borrar para que la suma de posiciones pares quede igual a la de impares. Técnica: DOS arreglos de prefijos, uno de las posiciones pares y otro de las impares. Al borrar un índice, todo lo que está a la derecha CAMBIA de paridad, así que la suma par nueva es (par hasta i) + (impar desde i+1 hasta el final). Ese cruce es todo el problema.

leetcode/prefix_sum/1664_ways_to_make_a_fair_array.py

```
1 class Solution:
2     def waysToMakeFair(self, nums: List[int]) -> int:
3         par = [0]
4         sumap = 0
5         impar = [0]
6         sumai = 0
```

34.4 Ways To Make A Fair Array - Prefijos Pares e Impares por Separado (cont.)

```

1   for i in range(len(nums)):
2       if(i%2==0):
3           sumap += nums[i]
4       else:
5           sumai += nums[i]
6       par.append(sumap)
7       impar.append(sumai)
8   van = 0
9   for i in range(len(nums)):
10      sumap = par[i] + impar[-1] - impar[i+1]
11      sumai = impar[i] + par[-1] - par[i+1]
12      if(sumap == sumai):
13          van += 1
14
15  return van

```

35 - Hub LeetCode: Queue

35.1 Design Circular Queue - Cola Circular con Arreglo Fijo

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 622 "Design Circular Queue": cola de capacidad fija con enqueue, dequeue, front y rear. Técnica: arreglo de tamaño k con dos índices, ini y rea, que dan la vuelta al llegar a k. El tamaño se guarda aparte, y eso es lo que permite distinguir vacía de llena sin desperdiciar una casilla. Rear() usa rea-1 porque rea apunta a la siguiente libre.

leetcode/queue/0622_design_circular_queue.py

```

1   class MyCircularQueue:
2
3       def __init__(self, k: int):
4           self.lista = [0]*k
5           self.tam = 0
6           self.k = k
7           self.ini = 0
8           self.rea = 0
9
10      def enqueue(self, value: int) -> bool:
11          if(self.tam < self.k):
12              self.lista[self.rea] = value
13              self.tam += 1
14              self.rea += 1
15              if(self.rea == self.k):
16                  self.rea = 0
17              return True
18          else:
19              return False
20
21      def dequeue(self) -> bool:
22          if(self.tam > 0):
23              self.tam -= 1
24              self.ini += 1
25              if(self.ini == self.k):
26                  self.ini = 0
27              return True
28          else:
29              return False
30
31      def front(self) -> int:
32          if(self.tam > 0):
33              return self.lista[self.ini]
34          else:
35              return -1
36
37      def rear(self) -> int:
38          if(self.tam > 0):
39              return self.lista[self.rea-1]
40          else:
41              return -1
42
43      def isEmpty(self) -> bool:
44          if(self.tam > 0):
45              return False
46          else:

```

35.1 Design Circular Queue - Cola Circular con Arreglo Fijo (cont.)

```

47      return True
48
49      def isFull(self) -> bool:
50          if(self.tam == self.k):
51              return True
52          else:
53              return False
54
55
56
57  # Your MyCircularQueue object will be instantiated and called as such:
58  # obj = MyCircularQueue(k)
59  # param_1 = obj.enqueue(value)
60  # param_2 = obj.dequeue()
61  # param_3 = obj.front()
62  # param_4 = obj.rear()
63  # param_5 = obj.isEmpty()
64  # param_6 = obj.isFull()

```

35.2 Implement Queue Using Stacks - Cola

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 232 "Implement Queue using Stacks": cola con push, pop, peek y empty. Técnica del código: una sola lista con append al final y pop(0) al inicio. OJO: el problema pide hacerlo con DOS PILAS y aca usa una lista, o sea se salta el ejercicio. pop(0) en Python es O(n). Lo que se buscaba: una pila de entrada y una de salida, y cuando la de salida esta vacía se voltea la de entrada encima; eso da O(1) amortizado.

leetcode/queue/0232_implement_queue_using_stacks.py

```

1   class MyQueue:
2
3       def __init__(self):
4           self.stack = []
5
6       def push(self, x: int) -> None:
7           self.stack.append(x)
8
9       def pop(self) -> int:
10          k = self.stack[0]
11          self.stack.pop(0)
12          return k
13
14      def peek(self) -> int:
15          return self.stack[0]
16
17
18      def empty(self) -> bool:
19          return len(self.stack)==0
20
21
22  # Your MyQueue object will be instantiated and called as such:
23  # obj = MyQueue()
24  # obj.push(x)
25  # param_2 = obj.pop()
26  # param_3 = obj.peek()
27  # param_4 = obj.empty()

```

35.3 Number Of Students Unable To Eat Lunch - Simulación de Cola

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1700 "Number of Students Unable to Eat Lunch": los que no comen porque el sandwich de arriba no les gusta y se van al final de la fila. Técnica: simular la fila con pop(0) y append. El contador c cuenta rechazos seguidos, y cuando llega al tamaño de la fila significa que ya nadie más puede comer. También sale contando cuantos quieren cada tipo, sin simular.

leetcode/queue/1700_number_of_students_unable_to_eat_lunch.py

```

1   class Solution:
2       def countStudents(self, students: List[int], sandwiches: List[int]) -> int:
3           c = -1
4           while c != len(students):
5               if(students[0]==sandwiches[0]):
6                   students.pop(0)

```

35.3 Number Of Students Unable To Eat Lunch - Simulacion de Cola (cont.)

```

1         sandwiches.pop(0)
2         c = 0
3     else:
4         student = students[0]
5         students.pop(0)
6         students.append(student)
7         c += 1
8     return c

```

35.4 Time Needed To Buy Tickets - Simulacion de Cola

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 2073 "Time Needed to Buy Tickets": segundos hasta que la persona en la posicion k termine de comprar, con la fila rotando. Tecnica: rotar la lista con pop(0) y append restando uno, y mover k con ella. Sale directo sin simular sumando por persona min(tickets[i], tickets[k]) y un +1 para los que van despues de k, pero la simulacion es facil de escribir bajo presion.

leetcode/queue/2073_time_needed_to_buy_tickets.py

```

1 class Solution:
2     def timeRequiredToBuy(self, tickets: List[int], k: int) -> int:
3         c = 0
4         while tickets[k] != 0:
5             j = tickets[0]
6             if(j>0):
7                 c+=1
8                 tickets.pop(0)
9                 tickets.append(j-1)
10                k -= 1
11                if(k < 0):
12                    k = len(tickets)-1
13        return c

```

36 - Hub LeetCode: Segment Tree

36.1 Find X Value Of Array li - Segment Tree con Nodo Compuesto

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3525 "Find X Value of Array li": con actualizaciones de una posicion, contar cuantos prefijos de un sufijo del arreglo tienen producto congruente con x modulo k. Tecnica: segment tree donde cada nodo guarda DOS cosas, prod (producto del segmento mod k) y freq[r] (cuantos prefijos del segmento dan residuo r). El merge multiplica el prod del hijo izquierdo por cada residuo del derecho, y ese es el truco que hace la informacion combinable. Cuesta O(k) por merge, así que el total queda O(n*k*log n). El mas interesante de esta carpeta: es la plantilla de segment tree con nodo que no es un solo numero, justo lo que hace falta cuando la consulta no es una suma.

leetcode/segment_tree/3525_find_x_value_of_array_li.py

```

1 class Solution:
2     def resultArray(self, nums: List[int], k: int, queries: List[List[int]]) -> List[int]:
3         class Node:
4             __slots__ = ('prod', 'freq')
5             def __init__(self):
6                 self.prod = 1
7                 self.freq = [0] * k
8
9         n = len(nums)
10        tree = [Node() for _ in range(4 * n)]
11
12        def merge(L: Node, R: Node) -> Node:
13            res = Node()
14            res.prod = (L.prod * R.prod) % k
15            res.freq = L.freq[:]
16            for r in range(k):
17                if R.freq[r]:
18                    nr = (L.prod * r) % k
19                    res.freq[nr] += R.freq[r]
20            return res
21
22        def build(v: int, tl: int, tr: int) -> None:
23            if tl == tr:
24                tree[v].prod = nums[tl] % k
25                tree[v].freq[tree[v].prod] = 1

```

36.1 Find X Value Of Array li - Segment Tree con Nodo Compuesto (cont.)

```

26            return
27            tm = (tl + tr) // 2
28            build(v * 2, tl, tm)
29            build(v * 2 + 1, tm + 1, tr)
30            tree[v] = merge(tree[v * 2], tree[v * 2 + 1])
31
32        def update(v: int, tl: int, tr: int, pos: int, val: int) -> None:
33            if tl == tr:
34                tree[v].prod = val % k
35                tree[v].freq = [0] * k
36                tree[v].freq[tree[v].prod] = 1
37                return
38            tm = (tl + tr) // 2
39            if pos <= tm:
40                update(v * 2, tl, tm, pos, val)
41            else:
42                update(v * 2 + 1, tm + 1, tr, pos, val)
43            tree[v] = merge(tree[v * 2], tree[v * 2 + 1])
44
45        def query(v: int, tl: int, tr: int, l: int, r: int) -> Node:
46            if l > r:
47                return Node()
48            if l == tl and r == tr:
49                return tree[v]
50            tm = (tl + tr) // 2
51            return merge(query(v * 2, tl, tm, l, min(r, tm)),
52                        query(v * 2 + 1, tm + 1, tr, max(l, tm + 1), r))
53
54        build(1, 0, n - 1)
55        ans = []
56        for idx, val, start, x in queries:
57            update(1, 0, n - 1, idx, val)
58            res = query(1, 0, n - 1, start, n - 1)
59            ans.append(res.freq[x])
60        return ans

```

37 - Hub LeetCode: Sliding Window

37.1 Find The Largest Almost Missing Integer - Contar en Cuantas Ventanas Aparece

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3471 "Find the Largest Almost Missing Integer": el mayor valor que aparece en exactamente UNA de las ventanas de tamaño k. Tecnica: para cada ventana se saca el set de sus valores y se cuenta en cuantas ventanas aparece cada uno; despues se toma el maximo de los que dan 1. El set por ventana es lo que evita contar dos veces un valor repetido dentro de la misma ventana. Es O(n*k) por rehacer el set en cada paso; con un diccionario incremental seria O(n).

leetcode/sliding_window/3471_find_the_largest_almost_missing_integer.py

```

1 class Solution:
2     def largestInteger(self, nums: List[int], k: int) -> int:
3         lista = {}
4         i = 0
5         while i < (len(nums)-k+1):
6             ka = list(set(nums[i:i+k]))
7             for j in ka:
8                 if(j in lista):
9                     lista[j] += 1
10            else:
11                lista[j] = 1
12            i += 1
13        unicos = []
14        for clave, valor in lista.items():
15            if(valor == 1):
16                unicos.append(clave)
17        if(len(unicos) == 0):
18            return -1
19        return max(unicos)

```

37.2 Find Two Non Overlapping Subarrays Each With Target Sum - Dos Ventanas sin Traslape

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1477 "Find Two Non-overlapping Sub-arrays Each With Target Sum": dos subarreglos que sumen target cada uno y que juntos sean lo mas cortos posible. Técnica: primero una ventana deslizante saca TODOS los subarreglos con suma target (los valores son positivos, por eso la ventana funciona). Despues se ordenan por donde terminan y con un puntero se va guardando el mas corto que termina ANTES de que empiece el actual. Ese mejor-hasta-ahora es lo que evita comparar todos los pares.

leetcode/sliding_window/1477_find_two_non-overlapping_subarrays_each_with_target_sum.py

```
1 class Solution:
2     def minSumOfLengths(self, arr: list[int], target: int) -> int:
3         rango = {}
4         c = []
5         can = 0
6         van = 0
7         i = 0
8         while i < len(arr):
9             can += arr[i]
10            van += 1
11            if can <= target:
12                if can == target:
13                    if can in rango:
14                        rango[can].append([van, i-van+1,i])
15                        c.append(van)
16                    else:
17                        rango[can] = [[van,i-van+1,i]]
18                        c.append(van)
19            else:
20                voy = van
21                for j in range(i-voy+1,i+1):
22                    can -= arr[j]
23                    van -= 1
24                    if can <= target:
25                        break
26                if can == target:
27                    if can in rango:
28                        rango[can].append([van,i-van+1,i])
29                        c.append(van)
30                    else:
31                        rango[can] = [[van,i-van+1,i]]
32                        c.append(van)
33                i+=1
34            if len(c)<2:
35                return -1
36            l = sorted(rango[target], key=lambda x: x[2])
37
38            mini = float("inf")
39            mejor = float("inf")
40            p = 0
41
42            for actual in l:
43
44                longitud, inicio, fin = actual
45
46                while p < len(l) and l[p][2] < inicio:
47                    mejor = min(mejor, l[p][0])
48                    p += 1
49
50                if mejor != float("inf"):
51                    mini = min(mini, longitud + mejor)
52            if(mini==float("inf")):
53                return -1
54            return mini
```

37.3 Length Of Longest Subarray With At Most K Frequency - Ventana con Frecuencias

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 2958 "Length of Longest Subarray With at Most K Frequency": el subarreglo mas largo donde ningun valor aparece mas de k veces. Técnica: ventana deslizante con diccionario de frecuencias. Cuando un valor llega a k+1 se encoge por la izquierda hasta que deje de pasarse. La bandera f es para no volver a contar el elemento de la derecha mientras la ventana se encoge; con el patron normal (un while adentro del for) no hace falta.

leetcode/sliding_window/2958_length_of_longest_subarray_with_at_most_k_frequency.py

```
1 class Solution:
2     def maxSubarrayLength(self, nums: List[int], k: int) -> int:
3         ini = 0
4         fin = 1
5         dic = {nums[ini]: 1}
6         cap = []
7         f = True
8         while fin < len(nums):
9             if(nums[fin] in dic and f):
10                dic[nums[fin]] += 1
11            elif(f):
12                dic[nums[fin]] = 1
13            if dic[nums[fin]] == (k+1):
14                cap.append(fin-ini)
15                dic[nums[ini]] -= 1
16                ini += 1
17                f = False
18            else:
19                fin+=1
20                f = True
21            cap.append(fin-ini)
22            return max(cap)
```

37.4 Longest Substring Without Repeating Characters - Ventana Deslizante con Set

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3 "Longest Substring Without Repeating Characters": la subcadena mas larga sin letras repetidas. Técnica: dos punteros y un set con las letras de la ventana. Si la letra de la derecha ya esta, se encoge por la izquierda quitando del set; si no, se agrega y se estira. OJO: deja dos prints, y el clamp fin = len(s)-1 al final del ciclo es fragil; la version limpia recorre fin de 0 a n-1 y mueve ini con un while adentro.

leetcode/sliding_window/0003_longest_substring_without_repeating_characters.py

```
1 class Solution:
2     def lengthOfLongestSubstring(self, s: str) -> int:
3         ini = 0
4         fin = 1
5         resp = 0
6         l = set()
7         if(len(s)==0):
8             return 0
9         elif(len(s)==1):
10            return 1
11        l.add(s[ini])
12        while(ini<len(s)):
13            print(ini, fin)
14            if(s[fin] in l):
15                l.remove(s[ini])
16                ini += 1
17                continue
18            l.add(s[fin])
19            print(l)
20            if(fin-ini > resp):
21                resp = (fin-ini)
22            fin += 1
23            if(fin >= len(s)):
24                fin = len(s)-1
25            return resp+1
```

37.5 Maximum Length Substring With Two Occurrences - Ventana con Frecuencias

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3090 "Maximum Length Substring With Two Occurrences": la subcadena mas larga donde ninguna letra aparece mas de dos veces. Técnica: el mismo molde del 2958 pero con k fijo en 2. Diccionario de frecuencias, y cuando una letra llega a 3 se encoge por la izquierda hasta que vuelva a 2.

leetcode/sliding_window/3090_maximum_length_substring_with_two_occurrences.py

```
1 class Solution:
2     def maximumLengthSubstring(self, s: str) -> int:
3         dic = {}
4         inicio = 0
5         fin = 0
6         tam = []
7         while fin < len(s):
8             if(s[fin] not in dic):
9                 dic[s[fin]] = 1
10            else:
11                dic[s[fin]]+=1
12            if(dic[s[fin]] > 2):
13                tam.append(fin-inicio)
14                while dic[s[fin]] > 2:
15                    dic[s[inicio]]-=1
16                    inicio += 1
17            fin += 1
18            if(len(tam)==0):
19                return fin
20            tam.append(fin-inicio)
21            return max(tam)
```

37.6 Shortest And Lexicographically Smallest Beautiful String - Ventana con Conteo Exacto

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 2904 "Shortest and Lexicographically Smallest Beautiful String": la subcadena mas corta con exactamente k unos, y entre las de ese largo la menor alfabeticamente. Técnica: ventana deslizante que crece por la derecha y en cuanto tiene k unos se encoge por la izquierda guardando todas las de largo mínimo. Al final un min() sobre las candidatas resuelve el desempate lexicográfico. Guardar los rangos y no las cadenas evita cortar strings dentro del ciclo.

leetcode/sliding_window/2904_shortest_and_lexicographically_smallest_beautiful_string.py

```
1 class Solution:
2     def shortestBeautifulSubstring(self, s: str, k: int) -> str:
3         if len(s) == 0:
4             return ""
5
6         inicio = 0
7         fin = 0
8         cont = 0
9         resp = []
10        menor = 999999999999999
11
12        while fin < len(s):
13
14            if s[fin] == "1":
15                cont += 1
16
17            while cont == k:
18
19                tam = fin - inicio + 1
20
21                if tam < menor:
22                    menor = tam
23                    resp = [[inicio, fin]]
24
25                elif tam == menor:
26                    resp.append([inicio, fin])
27
28                if s[inicio] == "1":
29                    cont -= 1
30
31                inicio += 1
32
33        return min(resp)
```

37.6 Shortest And Lexicographically Smallest Beautiful String - Ventana con Conteo Exacto (cont.)

```
34        fin += 1
35
36        if len(resp) == 0:
37            return ""
38
39        r = []
40
41        for i in resp:
42            r.append(s[i[0]:i[1] + 1])
43
44        return min(r)
```

38 - Hub LeetCode: Sorting

38.1 Merge Sorted Array - Merge Sort

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 88 "Merge Sorted Array": mezclar dos arreglos ya ordenados dentro del primero. Técnica: aca hace merge sort completo de la concatenación. Lo que vale del archivo es la plantilla limpia de merge sort. OJO: los arreglos YA venian ordenados, así que basta la función combinar() sola en O(n+m) en vez de volver a ordenar todo.

leetcode/sorting/0088_merge_sorted_array.py

```
1 class Solution:
2
3     def combinar(self, izquierda, derecha):
4
5         resultado = []
6
7         i = 0
8         j = 0
9
10        while i < len(izquierda) and j < len(derecha):
11
12            if izquierda[i] <= derecha[j]:
13                resultado.append(izquierda[i])
14                i += 1
15            else:
16                resultado.append(derecha[j])
17                j += 1
18
19        while i < len(izquierda):
20            resultado.append(izquierda[i])
21            i += 1
22
23        while j < len(derecha):
24            resultado.append(derecha[j])
25            j += 1
26
27        return resultado
28
29        def merge_sort(self, arr):
30
31            if len(arr) <= 1:
32                return arr
33
34            mitad = len(arr) // 2
35
36            izquierda = self.merge_sort(arr[:mitad])
37            derecha = self.merge_sort(arr[mitad:])
38
39            return self.combinar(izquierda, derecha)
40
41        def merge(self, nums1: list[int], m: int, nums2: list[int], n: int) -> None:
42            resultado = self.merge_sort(nums1[:m] + nums2[:n])
43
44            for i in range(m + n):
45                nums1[i] = resultado[i]
```

38.2 Reverse Pairs - Contar en la Fusión del Merge Sort

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 493 "Reverse Pairs": contar pares $i < j$ con $\text{nums}[i] > 2 * \text{nums}[j]$. Técnica: la de contar inversiones, pero con el 2. Durante el merge sort, al fusionar dos mitades YA ordenadas, un puntero j avanza mientras $\text{izquierda}[i] > 2 * \text{derecha}[j]$ y suma j pares de golpe. Es $O(n \log n)$ y el patrón sirve para cualquier condición monótona. El puntero j NO se reinicia en cada i , y eso es lo que lo deja lineal por fusión.

leetcode/sorting/0493_reverse_pairs.py

```
1 class Solution:
2     def reversePairs(self, nums: List[int]) -> int:
3
4         def merge_sort(arr):
5             if len(arr) <= 1:
6                 return 0
7
8             mitad = len(arr) // 2
9
10            izquierda = arr[:mitad]
11            derecha = arr[mitad:]
12
13            cantidad = merge_sort(izquierda)
14            cantidad += merge_sort(derecha)
15
16            # Contar reverse pairs
17            j = 0
18
19            for i in range(len(izquierda)):
20                while j < len(derecha) and izquierda[i] > 2 * derecha[j]:
21                    j += 1
22
23            cantidad += j
24
25            # Merge
26            i = 0
27            j = 0
28            k = 0
29
30            while i < len(izquierda) and j < len(derecha):
31
32                if izquierda[i] <= derecha[j]:
33                    arr[k] = izquierda[i]
34                    i += 1
35                else:
36                    arr[k] = derecha[j]
37                    j += 1
38
39                k += 1
40
41            while i < len(izquierda):
42                arr[k] = izquierda[i]
43                i += 1
44                k += 1
45
46            while j < len(derecha):
47                arr[k] = derecha[j]
48                j += 1
49                k += 1
50
51            return cantidad
52
53            return merge_sort(nums)
```

38.3 Sort An Array - Counting Sort

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 912 "Sort an Array": ordenar, sin usar el sort de la librería. Técnica: counting sort con diccionario de frecuencias y un barrido del mínimo al máximo. Es $O(n + \text{rango})$, así que solo conviene cuando el rango de valores es chico; acá los valores llegan a $5 * 10^4$ y cabe. Con valores grandes y dispersos esto se vuelve lentísimo.

leetcode/sorting/0912_sort_an_array.py

```
1 class Solution:
```

38.3 Sort An Array - Counting Sort (cont.)

```
2     def sortArray(self, nums: List[int]) -> List[int]:
3         mi = float("inf")
4         ma = 0
5         dic = {}
6         for i in nums:
7             if i > ma:
8                 ma = i
9             if i < mi:
10                mi = i
11            if i in dic:
12                dic[i] += 1
13            else:
14                dic[i] = 1
15
16        fin = [0] * len(nums)
17        va = 0
18        for i in range(mi, ma + 1):
19            try:
20                c = dic[i]
21                for j in range(c):
22                    fin[va] = i
23                    va += 1
24            except:
25                continue
26        return fin
```

39 - Hub LeetCode: Stack

39.1 Build An Array With Stack Operations - Simulación de Pila

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1441 "Build an Array With Stack Operations": la lista de Push y Pop para construir el arreglo objetivo leyendo 1, 2, 3, ..., n . Técnica: recorrer de 1 a n haciendo Push siempre, y si el número no está en el objetivo, inmediatamente un Pop. El set es solo para consultar en $O(1)$. OJO: deja un print(g) al inicio.

leetcode/stack/1441_build_an_array_with_stack_operations.py

```
1 class Solution:
2     def buildArray(self, target: List[int], n: int) -> List[str]:
3         g = set(target)
4         print(g)
5         res = []
6         num = []
7         for i in range(1, n + 1):
8             if target == num:
9                 return res
10            res.append("Push")
11            num.append(i)
12            if i not in g:
13                res.append("Pop")
14                num = num[:len(num) - 1]
15        return res
```

39.2 Evaluate Reverse Polish Notation - Evaluación en Notación Polaca

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 150 "Evaluate Reverse Polish Notation": evaluar una expresión en notación postfix. Técnica: pila de números. Si el token es número se empuja; si es operador se sacan los DOS de arriba, se opera y se devuelve el resultado. Al final queda uno solo. El orden importa, $\text{lit}[-2]$ es el izquierdo y $\text{lit}[-1]$ el derecho. La división usa $\text{int}(a/b)$ y no a/b porque el problema pide trunca hacia cero y // de Python redondea hacia abajo.

leetcode/stack/0150_evaluate_reverse_polish_notation.py

```
1 class Solution:
2     def evalRPN(self, tokens: List[str]) -> int:
3         lit = []
4         ope = set(["+", "-", "*", "/"])
5         for i in range(len(tokens)):
6             if tokens[i] not in ope:
7                 lit.append(int(tokens[i]))
8             else:
9                 if tokens[i] == "+":
10                    v = lit[-2] + lit[-1]
```

39.2 Evaluate Reverse Polish Notation - Evaluación en Notación Polaca (cont.)

```

11         lit = lit[0:len(lit)-2]
12         lit.append(v)
13     elif(tokens[i]=="-"):
14         v = lit[-2]-lit[-1]
15         lit = lit[0:len(lit)-2]
16         lit.append(v)
17     elif(tokens[i]=="*"):
18         v = lit[-2]*lit[-1]
19         lit = lit[0:len(lit)-2]
20         lit.append(v)
21     elif(tokens[i]=="/"):
22         v = int(lit[-2]/lit[-1])
23         lit = lit[0:len(lit)-2]
24         lit.append(v)
25     return lit[0]

```

39.3 Exclusive Time Of Functions - Pila de Llamadas

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 636 "Exclusive Time of Functions": tiempo propio de cada función, sin contar lo que gasta en las que llama. Técnica: pila con los ids de las funciones abiertas y una variable val con el instante del último evento. En cada start se le cobra a la de la cima el tiempo transcurrido, y en cada end se le cobra el tramo final y se saca. El +1 y el val = fin+1 son porque los timestamps son unidades completas y no instantes. OJO: arma la respuesta recorriendo el diccionario en orden de inserción, no por id de función. Si la primera función que arranca no es la 0, el resultado sale en el orden equivocado.

leetcode\stack\0636_exclusive_time_of_functions.py

```

1 class Solution:
2     def exclusiveTime(self, n: int, logs: List[str]) -> List[int]:
3         dic = {}
4         val = 0
5         stack = []
6         for i in range(len(logs)):
7             partes = logs[i].split(":")
8             if(partes[1]== "start"):
9                 if(partes[0] not in dic):
10                     dic[partes[0]]=0
11                 if(int(partes[2])!=val):
12                     dic[stack[-1]] += int(partes[2])-val
13                 stack.append(partes[0])
14                 val = int(partes[2])
15             else:
16                 time = int(partes[2]) - val + 1
17                 dic[stack[-1]] += time
18                 stack.pop()
19                 val = int(partes[2])+1
20
21         f = []
22         for clave,val in dic.items():
23             f.append(val)
24
25         return f

```

39.4 Largest Rectangle In Histogram - Pila Monotona (version enredada)

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 84 "Largest Rectangle in Histogram": el rectángulo de área máxima bajo el histograma. Técnica del código: dos pasadas con pila guardando para cada altura los rangos donde sobrevive, y después cruza los rangos en diccionarios. OJO, ESTE ARCHIVO NO SIRVE DE PLANTILLA: tiene un caso cableado, if heights[0] == 7303, que responde otra cosa para pasar un caso de prueba específico, y deja prints adentro del ciclo. Lo correcto es la pila monotona de una sola pasada: se mantiene la pila con alturas crecientes y al sacar una se calcula su área con el ancho hasta el índice actual. Son 12 líneas.

leetcode\stack\0084_largest_rectangle_in_histogram.py

```

1 from collections import Counter
2 class Solution:
3     def largestRectangleArea(self, heights: List[int]) -> int:
4         mi = min(heights)
5         fic = Counter(heights)
6         valores = set(list(fic.values()))

```

39.4 Largest Rectangle In Histogram - Pila Monotona (version enredada) (cont.)

```

7         if(len(fic) == 1):
8             return heights[0]*len(heights)
9
10        if(heights[0]==7303):
11            may = 0
12            es = 0
13            print(fic)
14            for i,j in fic.items():
15                if(j>may):
16                    may = j
17                es = i
18            elif(j == may and es<i):
19                may = j
20                es = i
21            return es*(may)
22
23        dic = {}
24        stack = []
25        pos = []
26        for i in range(len(heights)):
27            if(len(stack) == 0):
28                stack.append(heights[i])
29                pos.append(i)
30            else:
31                j = len(stack)-1
32                while j>-1:
33                    if(stack[j]>heights[i]):
34                        if(stack[j] not in dic):
35                            dic[stack[j]] = [[pos[j],i-1]]
36                        else:
37                            dic[stack[j]].append([pos[j],i-1])
38                stack.pop()
39                pos.pop()
40            else:
41                break
42            j-=1
43            stack.append(heights[i])
44            pos.append(i)
45
46        j = len(stack)-1
47        while j>-1:
48            if(stack[j] not in dic):
49                dic[stack[j]] = [[pos[j],len(heights)-1]]
50            else:
51                dic[stack[j]].append([pos[j],len(heights)-1])
52            stack.pop()
53            pos.pop()
54            j-=1
55
56        stack = []
57        pos = []
58        for i in range(len(heights)-1,-1,-1):
59            if(len(stack) == 0):
60                stack.append(heights[i])
61                pos.append(i)
62            else:
63                j = len(stack)-1
64                while j>-1:
65                    if(stack[j]>heights[i]):
66                        if(stack[j] not in dic):
67                            dic[stack[j]] = [[pos[j],i+1]]
68                        else:
69                            dic[stack[j]].append([pos[j],i+1])
70                stack.pop()
71                pos.pop()
72            else:
73                break
74            j-=1

```

39.4 Largest Rectangle In Histogram - Pila Monotona (version enredada) (cont.)

```

78         stack.append(heights[i])
79         pos.append(i)
80
81
82     #print(stack)
83     j = len(stack)-1
84     while j>-1:
85         if(stack[j] not in dic):
86             dic[stack[j]] = [[pos[j],0]]
87         else:
88             dic[stack[j]].append([pos[j],0])
89         stack.pop()
90         pos.pop()
91         j-=1
92
93
94
95
96
97
98     #dic[mi] = [[0,len(heights)-1]]
99
100     dicF = {}
101     for clave,val in dic.items():
102         for i in range(len(val)):
103             resp = [val[i][0],val[i][1]]
104             for j in range(i,len(val)):
105                 if(val[i][0]==val[j][0]):
106                     resp = [val[i][1], val[j][1]]
107             if(clave not in dicF):
108                 dicF[clave] = [resp]
109             else:
110                 dicF[clave].append(resp)
111
112
113
114
115     may = 0
116     for clave,val in dicF.items():
117         for cubo in val:
118             rest = (abs(cubo[1]-cubo[0])+1) * clave
119             print(clave, cubo, rest)
120             if(rest>may):
121                 may = rest
122
123
124     return may

```

39.5 Next Greater Node In Linked List - Siguiente Mayor con Pila

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1019 "Next Greater Node In Linked List": para cada nodo, el primer valor mayor que aparece despues. Tecnica: pila monotona decreciente que guarda valores Y posiciones. Cuando llega uno mas grande, se le resuelve la respuesta a todos los de la pila que sean menores. Cada elemento entra y sale una vez, así que es O(n). El patron next greater element se usa en muchísimos problemas; este es la version de lista ligada, pero con arreglo es identico.

leetcode/stack/1019_next_greater_node_in_linked_list.py

```

1 # Definition for singly-linked list.
2 # class ListNode:
3 #     def __init__(self, val=0, next=None):
4 #         self.val = val
5 #         self.next = next
6 class Solution:
7     def nextLargerNodes(self, head: Optional[ListNode]) -> List[int]:
8         lista = []
9         stack = []
10        stackpos = []
11        p = 0
12        c = head
13        while c != None:
14            lista.append(0)
15            while True:
16                if(len(stack) != 0 and stack[-1] < c.val):
17                    lista[stackpos[-1]] = c.val

```

39.5 Next Greater Node In Linked List - Siguiente Mayor con Pila (cont.)

```

8         stack.pop()
9         stackpos.pop()
10        else:
11            break
12        stack.append(c.val)
13        stackpos.append(p)
14        p+= 1
15        c = c.next
16
17    return lista

```

39.6 Remove Duplicate Letters - Pila Monotona Lexicografica

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 316 "Remove Duplicate Letters": dejar una sola copia de cada letra y que el resultado sea el lexicograficamente menor. Tecnica: pila monotona con dos apoyos, un Counter de lo que FALTA por ver y un set de lo que ya esta en la pila. Se saca de la pila mientras la cima sea mayor que la letra actual Y todavia quede otra copia de esa cima mas adelante. Ese conteo de lo que falta es la clave. Es el molde de todos los problemas de subsecuencia lexicograficamente minima.

leetcode/stack/0316_remove_duplicate_letters.py

```

1 from collections import Counter
2
3 class Solution:
4     def removeDuplicateLetters(self, s: str) -> str:
5
6         count = Counter(s)
7         stack = []
8         seen = set()
9
10        for c in s:
11
12            count[c] -= 1
13
14            if c in seen:
15                continue
16
17            while (
18                stack and
19                c < stack[-1] and
20                count[stack[-1]] > 0
21            ):
22                seen.remove(stack.pop())
23
24            stack.append(c)
25            seen.add(c)
26
27    return "".join(stack)

```

39.7 Valid Parentheses - Pila de Parentesis

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 20 "Valid Parentheses": decir si una cadena de (), [] y {} esta bien balanceada. Tecnica: pila. Al abrir se empuja un código (1, 2, 3) y al cerrar se exige que la cima sea el que corresponde. Al final la pila tiene que quedar vacia. El try/except cubre el caso de cerrar con la pila vacia, que es un cierre sin apertura. En el notebook esta la version en C++ con las 13 variantes de este problema.

leetcode/stack/0020_valid_parentheses.py

```

1 class Solution:
2     def isValid(self, s: str) -> bool:
3         cola = []
4
5         for i in range(len(s)):
6             try:
7                 if(s[i]== "("):
8                     cola.append(1)
9                 elif(s[i]== "["):
10                    cola.append(2)
11                elif(s[i]== "{"):
12                    cola.append(3)
13                elif(s[i]== ")" and cola[len(cola)-1]==1):

```

39.7 Valid Parentheses - Pila de Parentesis (cont.)

```

14         cola.pop(len(cola)-1)
15         elif(s[i]== "]" and cola[len(cola)-1]==2):
16             cola.pop(len(cola)-1)
17         elif(s[i]== "]" and cola[len(cola)-1]==3):
18             cola.pop(len(cola)-1)
19         elif(s[i]== ")"):
20             break
21         elif(s[i]== "]"):
22             break
23         elif(s[i]== "}"):
24             break
25     except:
26         return False
27 if(len(cola)==0):
28     return True
29 else:
30     return False

```

40 - Hub LeetCode: String

40.1 Brace Expansion II - Expansion Recursiva de Llaves

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1096 "Brace Expansion II": expandir una expresión con llaves y comas anidadas. Técnica: se busca la PRIMERA llave que cierra y su llave que abre más cercana hacia atrás (rfind). Eso aísla el grupo más interno, se reemplaza por cada opción separada por comas y se vuelve a llamar. El set junta y quita duplicados y el sorted ordena al final. Atacar siempre el grupo más interno es lo que hace que la recursión sea tan corta.

leetcode/string/1096_brace_expansion_ii.py

```

1 class Solution:
2     def braceExpansionII(self, expression):
3         ans = set()
4
5         def dfs(s):
6             r = s.find('}')
7
8             # No braces left
9             if r == -1:
10                 ans.add(s)
11                 return
12
13             # Find matching '{'
14             l = s.rfind('{', 0, r)
15
16             left = s[:l]
17             right = s[r + 1:]
18
19             # Content inside { }
20             inside = s[l + 1:r]
21
22             for part in inside.split(','):
23                 dfs(left + part + right)
24
25         dfs(expression)
26         return sorted(ans)

```

40.2 Concatenate Non Zero Digits And Multiply By Sum I - Manejo de Dígitos

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3754 "Concatenate Non-Zero Digits and Multiply by Sum I": pegar los dígitos que no son 0 y multiplicar ese número por la suma de sus dígitos. Técnica: armar el string sin ceros, sumar los dígitos y multiplicar. El try/except cubre el caso de que no quede ningún dígito (n era todo ceros), donde int("") falla.

leetcode/string/3754_concatenate_non_zero_digits_and_multiply_by_sum_i.py

```

1 class Solution:
2     def sumAndMultiply(self, n: int) -> int:
3         h = ""
4         j = str(n)
5         for i in j:
6             if(i!="0"):
7                 h+=i

```

40.2 Concatenate Non Zero Digits And Multiply By Sum I - Manejo de Dígitos (cont.)

```

8         sums = 0
9         for i in h:
10             sums+=int(i)
11         try:
12             return int(h)*sums
13         except:
14             return 0

```

40.3 Counts The Number Of Special Characters I - Sets de Letras

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3120 "Count the Number of Special Characters I": cuantas letras aparecen en minúscula y también en mayúscula. Técnica: un set con todos los caracteres y para cada minúscula se pregunta si su upper() también está. Dos líneas.

leetcode/string/3120_counts_the_number_of_special_characters_i.py

```

1 class Solution:
2     def numberOfSpecialChars(self, word: str) -> int:
3
4         w = set(word)
5         c = 0
6         for i in w:
7             if(i.islower() and i.upper() in w ):
8                 c+=1
9         return c

```

40.4 Counts The Number Of Special Characters II - Orden de Primera y Última Aparición

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3121 "Count the Number of Special Characters II": igual que la I, pero exigiendo que la ÚLTIMA minúscula aparezca antes de la PRIMERA mayúscula. Técnica: un barrido con varios sets (l, u, esta, ya) que va sumando cuando encuentra la mayúscula después de la minúscula y RESTANDO si después vuelve a aparecer la minúscula, que rompería el orden. Ese descuento es el corazón del problema. Sale más claro guardando última posición de la minúscula y primera de la mayúscula y comparando.

leetcode/string/3121_counts_the_number_of_special_characters_ii.py

```

1 class Solution:
2     def numberOfSpecialChars(self, word: str) -> int:
3         l = set()
4         u = set()
5         c = 0
6         esta = set()
7         ya = set()
8         for i in word:
9             if(i.islower() and i not in l):
10                 l.add(i)
11             elif(i in esta and i not in ya):
12                 c-=1
13                 ya.add(i)
14             elif(i.isupper() and i.lower() in l and i not in u):
15                 c+=1
16                 esta.add(i.lower())
17             if(i.isupper()):
18                 u.add(i)
19         return c

```

40.5 Detect Capital - Casos de Mayúsculas

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 520 "Detect Capital": decir si el uso de mayúsculas es válido (todas, ninguna, o solo la primera). Técnica: comparar la palabra contra su upper() y su lower(); si coincide con alguno, listo. El tercer caso es primera en mayúscula y ninguna otra. En Python los tres casos son word.isupper() or word.islower() or word.istitle().

leetcode/string/0520_detect_capital.py

```

1 class Solution:
2     def detectCapitalUse(self, word: str) -> bool:
3         gr = word.upper()
4         pe = word.lower()
5         if(word == gr):
6             return True
7         elif(word == pe):
8             return True
9         else:

```

40.5 Detect Capital - Casos de Mayusculas (cont.)

```

1  if(word[0] == word[0].upper()):
2      c = 0
3      for i in range(1,len(word)):
4          if(word[i] == word[i].upper()):
5              c+=1
6          if(c == 0):
7              return True
8          return False

```

40.6 Evaluate The Bracket Pairs Of A String - Sustitucion de Plantilla

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1807 "Evaluate the Bracket Pairs of a String": reemplazar cada (clave) por su valor, o por ? si no se conoce. Tecnica: diccionario con el conocimiento y un barrido que al ver un parentesis lee hasta el cierre y sustituye. Un solo recorrido, O(n).

leetcode/string/1807_evaluate_the_bracket_pairs_of_a_string.py

```

1  class Solution:
2      def evaluate(self, s: str, knowledge: list[list[str]]) -> str:
3          dic = {}
4          for i in knowledge:
5              dic[i[0]] = i[1]
6          res = ""
7          i=0
8          while i < len(s):
9              if(s[i]=="("):
10                 ini = i+1
11                 fin = i+2
12                 pal = s[ini]
13                 while (s[fin] != ")"):
14                     pal += s[fin]
15                     fin += 1
16                 if(pal in dic):
17                     res += dic[pal]
18                 else:
19                     res += "?"
20                 i = fin
21             else:
22                 res+= s[i]
23                 i+=1
24             return res

```

40.7 Find The Index Of The First Occurrence In A String - Búsqueda de Subcadena

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 28 "Find the Index of the First Occurrence in a String": la posicion de la primera aparicion de needle en haystack. Tecnica: haystack.index(needle) y listo. Sirve como recordatorio de que en Python la busqueda de subcadena viene de fabrica; en C++ es s.find(t). Si pidieran hacerlo a mano, es KMP, que esta en el notebook.

leetcode/string/0028_find_the_index_of_the_first_occurrence_in_a_string.py

```

1  class Solution:
2      def strStr(self, haystack: str, needle: str) -> int:
3          try:
4              p = haystack.index(needle)
5              return p
6          except:
7              return -1

```

40.8 Find The Length Of The Longest Common Prefix - Prefijos en un Set

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3043 "Find the Length of the Longest Common Prefix": el prefijo comun mas largo entre cualquier numero del primer arreglo y cualquiera del segundo. Tecnica: se meten TODOS los prefijos de los numeros del primer arreglo en un set y se busca cual aparece tambien en el set del segundo. Eso evita comparar todos los pares. OJO: compara con int(i) para quedarse con el mayor, pero lo que se busca es el mas LARGO. Con numeros de la misma cantidad de digitos coincide, pero el criterio correcto es len(). Con un trie esto es el ejercicio natural, y sale mas claro.

leetcode/string/3043_find_the_length_of_the_longest_common_prefix.py

```

1  class Solution:
2      def longestCommonPrefix(self, arr1: List[int], arr2: List[int]) -> int:
3
4          v1 = set()
5          for i in arr1:

```

40.8 Find The Length Of The Longest Common Prefix - Prefijos en un Set (cont.)

```

6          k = str(i)
7          l = ""
8          for j in k:
9              l += j
10             v1.add(l)
11         v2 = set()
12         for i in arr2:
13             k = str(i)
14             l = ""
15             for j in k:
16                 l += j
17                 v2.add(l)
18         v1 = sorted(v1)
19         c = 0
20         for i in v1:
21             if(i in v2 and c<int(i)):
22                 c = int(i)
23
24         if(c==0):
25             return 0
26         else:
27             return len(str(c))

```

40.9 Length Of Last Word - Split

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 58 "Length of Last Word": el largo de la ultima palabra. Tecnica: split() sin argumentos ya ignora los espacios de sobra, asi que basta len(l[-1]).

leetcode/string/0058_length_of_last_word.py

```

1  class Solution:
2      def lengthOfLastWord(self, s: str) -> int:
3          l = s.split()
4          return len(l[-1])

```

40.10 License Key Formatting - Agrupar de Atras hacia Adelante

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 482 "License Key Formatting": reagrupar la llave en bloques de k separados por guion, donde solo el PRIMER grupo puede ser mas corto. Tecnica: como el grupo corto queda al inicio, primero se cuenta cuantos caracteres utiles hay y se saca el residuo modulo k; ese residuo es el tamaño del primer bloque. Despues los demas van de k en k. Sale mas limpio construyendo el resultado al REVES de a k y volteandolo al final.

leetcode/string/0482_license_key_formatting.py

```

1  class Solution:
2      def licenseKeyFormatting(self, s: str, k: int) -> str:
3          c = 0
4          for i in s:
5              if(i != "-"):
6                  c+=1
7          cant = c%k
8          fin = ""
9          if(cant!= 0):
10             c = 0
11             voy = 0
12             for i in range(len(s)):
13                 if(s[i] != "-"):
14                     try:
15                         fin += s[i].upper()
16                     except:
17                         fin += s[i]
18                 c+=1
19             voy = i
20             if(c==cant):
21                 fin+="-"
22                 break
23         else:
24             c = 0
25             voy = 0
26             for i in range(len(s)):
27                 if(s[i] != "-"):
28                     try:

```

40.10 License Key Formatting - Agrupar de Atras hacia Adelante (cont.)

```

29         fin += s[i].upper()
30     except:
31         fin += s[i]
32     c+=1
33     voy = i
34     if(c==k):
35         fin+="-"
36         break
37     c = 0
38     for i in range(voy+1, len(s)):
39         if(s[i] != "-"):
40             try:
41                 fin += s[i].upper()
42             except:
43                 fin += s[i]
44         c+=1
45         if(c==k):
46             fin += "-"
47             c = 0
48     try:
49         if(fin[-1]=="-"):
50             return fin[0:-1]
51     else:
52         return fin
53 except:
54     return fin

```

40.11 Longest Common Prefix - Prefijo Común por Columnas

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 14 "Longest Common Prefix": el prefijo común más largo de un arreglo de cadenas. Técnica: comparar por COLUMNAS. Se mira la letra i de todas las palabras y en el primer desacuerdo se corta. El try/except cubre la palabra que se acaba antes.

leetcode(string)\0014_longest_common_prefix.py

```

1 class Solution(object):
2     def longestCommonPrefix(self, strs):
3
4         num = len(strs)
5         pr = ""
6         for i in range(len(strs[0])):
7             l = strs[0][i]
8             try:
9                 cont = 0
10                for j in range(num):
11                    if(strs[j][i] == l):
12                        cont+=1
13                if(cont==num):
14                    pr+=l
15            else:
16                break
17        except:
18            break
19        return (pr)
20
21
22 """
23 :type strs: List[str]
24 :rtype: str
25 """

```

40.12 Masking Personal Information - Enmascarar por Formato

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 831 "Masking Personal Information": tapar un correo o un teléfono según reglas fijas. Técnica: primero decide si es teléfono o correo intentando convertir el último carácter a entero; si falla, es correo. Para el teléfono se sacan solo los dígitos y el prefijo de asteriscos depende de cuántos queden (10, 11, 12 o 13). Para el correo se deja la primera letra, cinco asteriscos, la última letra del nombre y el dominio en minúsculas. Es puro manejo de casos; lo que se lleva uno es la idea de detectar el formato por el último carácter en vez de buscar la arroba.

leetcode(string)\0831_masking_personal_information.py

```

1 class Solution:
2     def maskPII(self, s: str) -> str:
3         #NUMERO
4         try:
5             for i in range(len(s)-1,-1,-1):
6                 if(s[i]!="-" and s[i]!=" "):
7                     break
8             f = int(s[i])
9             nums = ""
10            for i in s:
11                try:
12                    a = int(i)
13                    nums += i
14                except:
15                    continue
16            tam = len(nums)
17            if(tam == 10):
18                return "****-***-" + nums[len(nums)-4:len(nums)]
19            elif(tam == 11):
20                return "+*-***-***-" + nums[len(nums)-4:len(nums)]
21            elif(tam == 12):
22                return "+*-***-***-" + nums[len(nums)-4:len(nums)]
23            else:
24                return "+*-***-***-***-" + nums[len(nums)-4:len(nums)]
25
26        #CORREO
27        except:
28            ini = s[0].lower()
29            f = ""
30            ya = False
31            correo = ""
32            for i in range(len(s)):
33                if(s[i] == "@"):
34                    ya = True
35                if(s[i] != "@" and ya == False):
36                    try:
37                        f = s[i].lower()
38                    except:
39                        f = s[i]
40                elif(s[i] == "." and f != ""):
41                    correo = s[i:len(s)]
42                    break
43            g = ""
44            for i in correo:
45                try:
46                    g += i.lower()
47                except:
48                    g += i
49            c = ini + "*****" + f + g
50            return c

```

40.13 Maximum Number Of Ballons - Frecuencias con Letras Repetidas

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1189 "Maximum Number of Ballons": cuántas veces se puede armar la palabra balloon. Técnica: contar b, a, l, o, n y dividir entre lo que pide la palabra. La trampa es que la l y la o van DOS veces, así que esas se dividen entre 2, y la respuesta es el mínimo. Con Counter sale en dos líneas: `min(cnt[c] // necesita[c] sobre las cinco letras`.

leetcode(string)\1189_maximum_number_of_ballons.py

```

1 class Solution:
2     def maxNumberOfBalloons(self, text: str) -> int:
3         dic = {"b":0,"a":0,"l":0,"o":0,"n":0}

```

40.13 Maximum Number Of Ballons - Frecuencias con Letras Repetidas (cont.)

```

4   for i in text:
5       if(i in dic):
6           dic[i] += 1
7       lt = dic["l"]
8       ot = dic["o"]
9       mi = dic["b"]
10      for cla, val in dic.items():
11          if(val < mi):
12              mi = val
13      if (lt < 2 or ot < 2):
14          return 0
15      else:
16          h = min(int(lt/2), int(ot/2))
17          if(h == mi):
18              return mi
19          elif(h>mi):
20              return mi
21      else:
22          return h

```

40.14 Maximum Number Of Non Overlapping Substrings - Intervalos Cerrados por Letra

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1520 "Maximum Number of Non-Overlapping Substrings": la mayor cantidad de subcadenas que no se traslapen y donde cada letra que aparece, aparezca COMPLETA. Tecnica: para cada letra se toma el intervalo de su primera a su ultima aparicion y se va extendiendo mientras alguna letra de adentro se salga; cuando ya cierra, es un candidato. Despues se quedan los candidatos que no contienen a otro (los mas cortos), que es el greedy. OJO: tiene un caso cableado, if s == 'cadaaedec' devuelve una respuesta fija, y el filtrado final usa 'esta contenido como subcadena' en vez de comparar los rangos, que no es lo mismo.

leetcode(string)\1520_maximum_number_of_non-overlapping_substrings.py

```

1   class Solution(object):
2       def maxNumOfSubstrings(self, s):
3           le = {}
4           for i in range(len(s)):
5               if(s[i] in le):
6                   le[s[i]].append(i)
7               else:
8                   le[s[i]] = [i]
9
10          posibles = []
11          for clave1,valor1 in le.items():
12              m11 = valor1[0]
13              ma1 = valor1[-1]
14
15              for clave2,valor2 in le.items():
16                  if(clave1==clave2):
17                      continue
18
19                  m12 = valor2[0]
20                  ma2 = valor2[-1]
21
22                  if(ma1<m12 or ma2<m11):
23                      continue
24                  elif(ma1<ma2 and m11>m12):
25                      continue
26                  elif(m11<m12 and ma2>ma1):
27                      ma1 = ma2
28                  elif(m11>m12 and ma2<ma1):
29                      m11 = m12
30
31          valido = True
32
33          for i in range(m11, ma1 + 1):
34              letra = s[i]
35
36              if(le[letra][0] < m11 or le[letra][-1] > ma1):
37                  valido = False
38                  break
39
40          if(valido and m11 == valor1[0]):

```

40.14 Maximum Number Of Non Overlapping Substrings - Intervalos Cerrados por Letra (cont.)

```

41      posibles.append(s[m11:ma1+1])
42
43      if(len(posibles) == 0):
44          return [s]
45      elif(s== "cadaaedec"):
46          return ["adaaede"]
47
48      pos = []
49
50      for i in range(len(posibles)):
51          c = False
52
53          for j in range(len(posibles)):
54              if(i==j):
55                  continue
56
57              if(posibles[j] in posibles[i]):
58                  c = True
59                  break
60
61          if not c:
62              pos.append(posibles[i])
63
64      return pos

```

40.15 Maximum Repeating Substring - Repeticiones Consecutivas

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1668 "Maximum Repeating Substring": el mayor k tal que word repetido k veces esta dentro de sequence. Tecnica: desde cada posicion se prueba word*1, word*2, ... hasta que no calce. OJO: compara con hash() en vez de comparar las cadenas directamente. hash() de Python puede coincidir para cadenas distintas, asi que es una comparacion con riesgo y no aporta velocidad porque igual construye el string. Con == es mas corto y mas seguro.

leetcode(string)\1668_maximum_repeating_substring.py

```

1   class Solution:
2       def maxRepeating(self, sequence: str, word: str) -> int:
3           ca = []
4           i = 0
5           while i<len(sequence):
6               try:
7                   j = 1
8                   c=0
9                   while True:
10                      k = word*j
11                      val = hash(k)
12                      s = hash(sequence[i:i+len(k)])
13                      if(s == val):
14                          c+=1
15                      else:
16                          break
17                      j+=1
18                      ca.append(c)
19          except:
20              break
21          i+=1
22          return max(ca)

```

40.16 Maximum Substrings With Distinct Start - Observacion que Colapsa el Problema

NO ES MIO: codigo de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3760 "Maximum Substrings With Distinct Start": la mayor cantidad de subcadenas que se pueden escoger empezando todas con letras distintas. Tecnica: dos lineas, len(set(s)). Cada letra distinta puede ser el inicio de una subcadena y ninguna mas, así que la respuesta es cuantas letras distintas hay. Buen recordatorio de leer bien antes de programar: la respuesta a veces es una sola expresion.

leetcode(string)\3760_maximum_substrings_with_distinct_start.py

```

1   class Solution:
2       def maxDistinct(self, s: str) -> int:
3           h = set(s)
4           return (len(h))

```

40.17 Minimum Number Of Pushes To Type Word I - Asignacion por Bloques de 8

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3014 "Minimum Number of Pushes to Type Word I": costo minimo de teclear la palabra, con 8 teclas disponibles y sin letras repetidas. Tecnica: las primeras 8 letras distintas cuestan 1 pulsacion, las siguientes 8 cuestan 2, y así. Se va asignando el costo en orden de aparicion porque en la version I cada letra aparece una sola vez y el orden no cambia nada.

leetcode/string/3014_minimum_number_of_pushes_to_type_word_i.py

```
1 class Solution:
2     def minimumPushes(self, word: str) -> int:
3         dic = {}
4         c = 0
5         t = 0
6         va = 1
7         for i in word:
8             if(i in dic):
9                 c+=dic[i]
10            else:
11                dic[i] = va
12                c += va
13                t+=1
14                if(t==8):
15                    t = 0
16                    va +=1
17        return c
```

40.18 Minimum Number Of Pushes To Type Word II - Greedy por Frecuencia

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3016 "Minimum Number of Pushes to Type Word II": lo mismo pero con letras repetidas. Tecnica: ahora SI importa el orden. Se cuentan frecuencias, se ordena de mayor a menor y se les va dando el costo 1 a las 8 mas frecuentes, 2 a las 8 siguientes, etc. Es el greedy de darle la posicion barata a lo que mas se usa, la misma idea que Huffman. La diferencia con la version I es exactamente ese sorted por frecuencia.

leetcode/string/3016_minimum_number_of_pushes_to_type_word_ii.py

```
1 class Solution:
2     def minimumPushes(self, word: str) -> int:
3         dic = {}
4         for i in word:
5             if(i in dic):
6                 dic[i]+= 1
7             else:
8                 dic[i] = 1
9
10        va = 1
11        c = 0
12        t = 0
13        dic = dict(sorted(dic.items(), key=lambda item: item[1], reverse=True))
14        for clave,valor in dic.items():
15            t += 1
16            if(t > 8):
17                t=1
18                va +=1
19            c += (valor*va)
20
21        return c
```

40.19 Number Of Strings That Appear As Substrings In Word - Contar Subcadenas

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1967 "Number of Strings That Appear as Substrings in Word": cuantos patrones aparecen en la palabra. Tecnica: el operador in de Python sobre cada patron. Dos lineas.

leetcode/string/1967_number_of_strings_that_appear_as_substrings_in_word.py

```
1 class Solution:
2     def numOfStrings(self, patterns: List[str], word: str) -> int:
3         c=0
4         for i in patterns:
5             if(i in word):
6                 c+=1
7        return c
```

40.20 Number Of Substrings Containing All Three Characters - Contar Subcadenas con las Tres L...

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1358 "Number of Substrings Containing All Three Characters": cuantas subcadenas tienen al menos una a, una b y una c. Tecnica: para cada inicio i se busca el primer final j donde ya estan las tres, y desde ahi TODAS las mas largas tambien valen, o sea se suman n - j de golpe. Ese salto es la idea buena. OJO: tiene un caso cableado, if len(dic)==3 and pos[2]==len(s)-1 and len(s)==50000 responde len(s)-2, que es un parche para el caso de prueba grande porque el metodo es O(n^2). Con ventana deslizante sale en O(n) y no hace falta el parche.

leetcode/string/1358_number_of_substrings_containing_all_three_characters.py

```
1 class Solution:
2     def numberOfSubstrings(self, s: str) -> int:
3         cant = 0
4         dic = {}
5         pos = []
6         for i in range(len(s)):
7             if(s[i] not in dic):
8                 dic[s[i]] = i
9                 pos.append(i)
10
11        if(len(dic)<3):
12            return 0
13
14        if(len(dic)==3 and pos[2]==len(s)-1 and len(s)==50000):
15            return len(s) -2
16
17        for i in range(len(s)):
18            ora = s[i:i+2]
19            a = set(ora)
20            for j in range(i+2,len(s)):
21                a.add(s[j])
22                if(len(a) >= 3):
23                    tam = len(s)-j
24                    cant+=tam
25                    break
26        return cant
```

40.21 Process String With Special Operations I - Simulacion de Operaciones

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3612 "Process String with Special Operations I": simular una cadena donde * borra el ultimo, # duplica todo y % invierte. Tecnica: simulacion directa sobre el string. Es O(n^2) porque duplicar e invertir copian todo. La version II no se puede simular (la cadena se vuelve gigantesca) y hay que ir AL REVES, rastreando de donde viene el caracter que se pide.

leetcode/string/3612_process_string_with_special_operations_i.py

```
1 class Solution:
2     def processStr(self, s: str) -> str:
3         res = ""
4         for i in s:
5             if(i not in "##%"):
6                 res += i
7             elif(i == "*" and len(res) > 0):
8                 res = res[:-1]
9             elif(i == "#"):
10                res += res
11            elif(i == "%"):
12                res = res[::-1]
13        return res
```

40.22 Reformat Date - Formateo de Fecha

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1507 "Reformat Date": pasar "20th Oct 2052" a "2052-10-20". Tecnica: split, diccionario de mes a numero, y quitar las dos ultimas letras del dia (st, nd, rd, th) con d[: -2]. El if de len(d) == 1 es para el cero de relleno.

leetcode/string/1507_reformat_date.py

```
1 class Solution:
2     def reformatDate(self, date: str) -> str:
3
4         dic2 = {'Jan': '01',
5                 'Feb': '02',
```

40.22 Reformat Date - Formateo de Fecha (cont.)

```

6         'Mar': '03',
7         'Apr': '04',
8         'May': '05',
9         'Jun': '06',
10        'Jul': '07',
11        'Aug': '08',
12        'Sep': '09',
13        'Oct': '10',
14        'Nov': '11',
15        'Dec': '12'}
16
17        d,m,a = list(date.split())
18        d = str(d[:-2])
19        if len(d) == 1:
20            d = '0'+d
21        return str(a)+'-'+dic2[m]+'-'+d

```

40.23 Repeated String Match - Repetir hasta Contener

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 686 "Repeated String Match": cuantas veces hay que repetir a para que b quede adentro. Técnica: repetir a mientras no contenga b. El límite $\text{len}(b)/\text{len}(a) + 5$ es la parte importante: si con esa cantidad no aparecio, ya nunca va a aparecer. En realidad bastan +2 copias, una para cubrir el desfase inicial y otra el final. Sin ese límite el ciclo no termina cuando la respuesta es -1.

leetcode/string/0686_repeated_string_match.py

```

1 class Solution:
2     def repeatedStringMatch(self, a: str, b: str) -> int:
3         tam1 = len(a)
4         tam2 = len(b)
5         ne = tam2//tam1 + 5
6         j = a
7         n = 1
8         while n<ne:
9             if(b in j):
10                 return n
11             n+=1
12             j+=a
13         return -1

```

40.24 Repeated Substring Pattern - Periodo de una Cadena

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 459 "Repeated Substring Pattern": decir si la cadena es un bloque repetido. Técnica: probar cada largo de bloque de 1 hasta $n/2$ y verificar que la cadena entera sea ese bloque repetido. Es $O(n^2)$ en el peor caso. El truco corto: la cadena es periódica si esta dentro de $(s+s)[1:-1]$. Y con KMP el periodo sale directo de la funcion de prefijo, $n - \text{pi}[n-1]$, que es lo que esta en el notebook.

leetcode/string/0459_repeated_substring_pattern.py

```

1 class Solution:
2     def repeatedSubstringPattern(self, s: str) -> bool:
3         if(len(s)==1):
4             return False
5         i = 1
6         f = False
7         while i<len(s)//2+1:
8             pal = s[0:i]
9             k = 0
10            while k <len(s):
11                try:
12                    if(pal == s[k:k+i]):
13                        k+=i
14                        continue
15                    else:
16                        break
17                except:
18                    break
19                if(k == len(s)):
20                    return True
21            i+=1
22        return False

```

40.25 Reverse Degree Of A String - Suma Ponderada

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3498 "Reverse Degree of a String": sumar, por cada letra, su posición invertida en el alfabeto (a vale 26, z vale 1) por su posición en la cadena. Técnica: diccionario con los 26 valores y un solo ciclo. El valor invertido también sale con $27 - (\text{ord}(c) - \text{ord}('a') + 1)$, sin escribir la tabla.

leetcode/string/3498_reverse_degree_of_a_string.py

```

1 class Solution:
2     def reverseDegree(self, s: str) -> int:
3         le = {
4             "a": 26, "b": 25, "c": 24, "d": 23, "e": 22, "f": 21,
5             "g": 20, "h": 19, "i": 18, "j": 17, "k": 16, "l": 15,
6             "m": 14, "n": 13, "o": 12, "p": 11, "q": 10, "r": 9,
7             "s": 8, "t": 7, "u": 6, "v": 5, "w": 4, "x": 3, "y": 2,
8             "z": 1
9         }
10
11        #s = s[::-1]
12        suma = 0
13        for i in range(len(s)):
14            suma += (le[s[i]]*(i+1))
15
16        return suma

```

40.26 Reverse Substrings Between Each Pair Of Parentheses - Pila de Parentesis con Reverso

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 1190 "Reverse Substrings Between Each Pair Of Parentheses": invertir lo que hay dentro de cada par de parentesis, respetando el anidamiento. Técnica: pila con las posiciones de los parentesis que abren. Al encontrar uno que cierra se invierte el pedazo de adentro y se reescribe la cadena sin los parentesis. El $i - 2$ es para reacomodar el índice después de que la cadena se acorto en dos caracteres. Modificar la cadena adentro del ciclo es lo delicado; con una pila de STRINGS acumulados no hay que tocar índices.

leetcode/string/1190_reverse_substrings_between_each_pair_of_parentheses.py

```

1 class Solution:
2     def reverseParentheses(self, s: str) -> str:
3         stack = []
4         i = 0
5         while i< len(s):
6             if(s[i]=="("):
7                 stack.append(i)
8             elif(s[i]==")"):
9                 pal = s[stack[-1]+1:i]
10                pal = pal[::-1]
11                ini = s[:stack[-1]]
12                fin = s[i+1:]
13                s = ini+pal+fin
14                stack.pop()
15                i-=2
16            i+=1
17        return s

```

40.27 Rotate String - Rotacion de Cadena

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 796 "Rotate String": decir si goal es una rotación de s. Técnica: para cada posición donde aparece la primera letra de goal, se compara dando la vuelta con el módulo (el $k = 0$ cuando se pasa del final). El truco de una línea: goal esta en $s+s$ si y solo si es una rotación. Y si hay que hacerlo rápido para cadenas grandes, es KMP sobre $s+s$.

leetcode/string/0796_rotate_string.py

```

1 class Solution:
2     def rotateString(self, s: str, goal: str) -> bool:
3         if(len(s)!=len(goal)):
4             return False
5
6         inicio = goal[0]
7
8         for i in range(len(s)):
9             if(s[i] == inicio):
10                j = 1
11                k = i+1

```

40.27 Rotate String - Rotacion de Cadena (cont.)

```

1 f = True
2 while j<len(goal):
3     if(k>len(s)):
4         k=0
5         if(goal[j]!=s[k]):
6             f = False
7             break
8         j+=1
9         k+=1
10    if(f):
11        return True
12    return False

```

40.28 Smallest Palindromic Rearrangement I - Palindromo Lexicograficamente Menor

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3517 "Smallest Palindromic Rearrangement I": reordenar las letras para formar el palindromo alfabeticamente menor. Tecnica: se cuentan las letras, se parte cada cuenta a la mitad y la primera mitad se escribe en orden alfabético; el centro es la única letra que quedo impar y la segunda mitad es el reverso. Poner las letras chicas primero es lo que minimiza, y como el palindromo obliga a que la segunda mitad sea el espejo, no hay nada mas que decidir.

leetcode(string)3517_smallest_palindromic_rearrangement_i.py

```

1 class Solution:
2     def smallestPalindrome(self, s: str) -> str:
3         t = [
4             "a","b","c","d","e","f","g",
5             "h","i","j","k","l","m","n",
6             "o","p","q","r","s","t","u",
7             "v","w","x","y","z"
8         ]
9         dic = {}
10        for i in s:
11            if(i not in dic):
12                dic[i] = 1
13            else:
14                dic[i]+=1
15
16        ini = ""
17        for i in t:
18            if(i in dic and dic[i] > 1):
19                cant = dic[i]//2
20                dic[i] -= cant*2
21                agre = i*cant
22                ini+=agre
23
24        mitad = ""
25        for i in t:
26            if(i in dic and dic[i] == 1):
27                mitad = i
28
29        fin = ini[::-1]
30        resp = ini + mitad + fin
31        return resp

```

40.29 Valid Anagram - Anagrama por Orden

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 242 "Valid Anagram": decir si dos cadenas son anagramas. Tecnica: ordenar las dos y comparar. Es $O(n \log n)$; con un Counter o un arreglo de 26 frecuencias queda en $O(n)$, que es lo que uno usaria si n fuera grande.

leetcode(string)0242_valid_anagram.py

```

1 class Solution:
2     def isAnagram(self, s: str, t: str) -> bool:
3         s = list(s)
4         t = list(t)
5         s.sort()
6         t.sort()
7         if(s == t):
8             return True
9         else:
10            return False

```

40.30 Valid Palindrome - Palindromo Filtrando Caracteres

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 125 "Valid Palindrome": palindromo ignorando mayusculas, espacios y signos. Tecnica: primero se limpia la cadena dejando solo letras y digitos, y despues se compara con dos punteros desde las puntas. OJO: deja un print(guar). En Python el filtro sale mas corto con i.isalnum().

leetcode(string)0125_valid_palindrome.py

```

1 class Solution:
2     def isPalindrome(self, s: str) -> bool:
3         s = s.lower()
4         lt = ["a","b","c","d","e","f","g","h","i","j",
5             "k","l","m","n","o","p","q","r","s","t",
6             "u","v","w","x","y","z"]
7
8         n = ["0","1","2","3","4","5","6","7","8","9"]
9
10        guar = ""
11        for i in range(0,len(s)):
12            if ((s[i] in lt) or (s[i] in n)):
13                guar += s[i]
14
15        print(guar)
16
17        total = len(guar)
18
19        for i in range(0,total):
20            if guar[i] != guar[total-1-i]:
21                return False
22            total = total - 1
23
24        return True

```

41 - Hub LeetCode: Trie

41.1 Counts Prefix And Suffix Pairs I - Trie hacia Adelante y al Reves

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 3042 "Count Prefix and Suffix Pairs I": contar pares donde una palabra es a la vez prefijo y sufijo de la otra. Tecnica: dos tries por palabra, uno con la palabra derecha y otro con la palabra invertida. Ser sufijo es ser prefijo del reverso, y ese es el unico truco. Igual compara todos los pares, asi que es $O(n^2 * \text{largo})$. OJO: deja un print(j, i) de depuracion adentro del ciclo.

leetcode(trie)3042_counts_prefix_and_suffix_pairs_i.py

```

1 class Node:
2     def __init__(self):
3         self.dictionary = {}
4         self.isEndLetter = False
5
6
7 class Trie:
8     def __init__(self):
9         self.root = Node()
10
11    def insert(self, word):
12        curr = self.root
13
14        for ch in word:
15            if ch not in curr.dictionary:
16                curr.dictionary[ch] = Node()
17
18        curr = curr.dictionary[ch]
19
20        curr.isEndLetter = True
21
22    def search(self, word):
23        curr = self.root
24
25        for ch in word:
26            if ch not in curr.dictionary:

```

41.1 Counts Prefix And Suffix Pairs I - Trie hacia Adelante y al Reves (cont.)

```

27         return False
28
29         curr = curr.dictionary[ch]
30
31     return True
32
33 class Solution:
34     def countPrefixSuffixPairs(self, words: List[str]) -> int:
35         derecho = []
36         reves = []
37
38         for i in words:
39             trieD = Trie()
40             trieR = Trie()
41             a = reversed(i)
42             trieD.insert(i)
43             trieR.insert(a)
44             derecho.append(trieD)
45             reves.append(trieR)
46
47         cont = 0
48         for i in range(len(derecho)):
49             for j in range(len(words)):
50                 if(i <= j):
51                     continue
52                 a = reversed(words[j])
53                 if(derecho[i].search(words[j]) and reves[i].search(a)):
54                     print(j,i)
55                     cont+=1
56
57         return cont

```

41.2 Implement Trie (prefix Tree) - Trie

NO ES MIO: código de Juan Jose Lozano, del repo del equipo (carpeta leetcode/). LeetCode 208 "Implement Trie (Prefix Tree)": insertar, buscar palabra exacta y buscar prefijo. Técnica: trie clásico con un diccionario de hijos por nodo y una bandera isEndLetter para distinguir 'la palabra termina aquí' de 'solo es un prefijo'. Esa bandera es toda la diferencia entre search y startsWith.

leetcode(trie)/0208_implement_trie_(prefix_tree).py

```

1 class Node:
2     def __init__(self):
3         self.dictionary = {}
4         self.isEndLetter = False
5
6 class Trie:
7
8     def __init__(self):
9         self.root = Node()
10
11     def insert(self, word):
12         curr = self.root
13
14         for ch in word:
15             if ch not in curr.dictionary:
16                 curr.dictionary[ch] = Node()
17
18             curr = curr.dictionary[ch]
19
20         curr.isEndLetter = True
21
22     def search(self, word):
23         curr = self.root
24
25         for ch in word:
26             if ch not in curr.dictionary:
27                 return False
28
29             curr = curr.dictionary[ch]
30
31         return curr.isEndLetter
32
33

```

41.2 Implement Trie (prefix Tree) - Trie (cont.)

```

34     def startsWith(self, prefix: str) -> bool:
35         curr = self.root
36
37         for ch in prefix:
38             if ch not in curr.dictionary:
39                 return False
40
41             curr = curr.dictionary[ch]
42
43         return True
44
45
46
47 # Your Trie object will be instantiated and called as such:
48 # obj = Trie()
49 # obj.insert(word)
50 # param_2 = obj.search(word)
51 # param_3 = obj.startsWith(prefix)

```

S			Buses	Day 2	5.15	Counts Prefix And Suffix Pairs I	Hub	41.1	
3sum Closest	Hub	20.1	Buy Low Sell High	Notebook	10.4	Counts The Number Of Special Chara	Hub	40.3	
A			Buy Two Chocolates	Hub	20.3	Counts The Number Of Special Chara	Hub	40.4	
	A Healthy Menu	Day 6	5.12	Bytelandia's Stones	Day 9	7.15	Couple Of Bip Bop	Day 3	9.14
	A Mazing!	Day 11	3.20				Course Schedule	CSES	16.4
A Story About Tea	Day 7	13.13	Cahokia Ruins	Day 5	5.24	Course Schedule	Hub	28.5	
A String Problem	Codeforces	9.17	Cake Hater	Day 10	13.8	Course Schedule Ii	Hub	28.6	
Accounting Qualifying	ICPC 2025	13.18	Call	RPC	5.18	Coven Complications	Day 11	10.21	
Accounting Qualifying Con Prefix	ICPC 2025	1.15	Celestial Veins	ICPC 2025	4.8	Creating Multiples	Day 4	6.14	
Add Two Numbers	Hub	31.1	Cerros	Codeforces	10.16	Creating Strings	CSES	17.5	
Aesthetics In Poetry	Day 7	6.8	Cheapest Flights Within K Stops	Hub	28.2	Criba De Eratostenes (Solo Impares	Notebook	6.4	
Algebra Y Ecuaciones	Formulario	14.1	Check If Array Is Sorted And Rotat	Hub	20.4	Crochet	RPC	13.16	
All Palindromes	CSES	19.1	Check If There Is A Valid Path In	Hub	28.3	Cycle Finding	CSES	16.5	
All Paths From Source To Target	Hub	28.1	Chequeo De Palindromo	Notebook	9.2	D			
Almost Shortest Path	Notebook	3.32	Chessboard And Queens	CSES	17.3		D Query	Day 5	1.5
Angle Between Hands Of A Clock	Hub	27.1	Chinese Innovation	Day 10	3.22		Daily Temperatures	Hub	20.11
Antiarithmetic	Day 11	5.5	Cinema Seat Allocation	Hub	20.5	Dangerous Dive	Day 2	1.9	
Apple Divisions	CSES	17.1	Circle And Rectangle Overlapping	Hub	27.2	Datos Que No Se Deducen	Formulario	14.5	
Apple Pie	ICPC	3.24	Clean Streets	ICPC	11.1	Denouncing Mafia	Day 1	3.19	
Aquarium Bubble Lights	Codeforces	1.10	Climbing Stairs	Hub	25.1	Design Circular Queue	Hub	35.1	
Archmage's Crystals	Day 9	6.7	Clone Graph	Hub	28.4	Destroying Asteroids	Hub	20.12	
Area Circulo Poligono	Notebook	4.2	Coach	Codeforces	5.19	Detect Capital	Hub	40.5	
Areas Y Volumenes	Formulario	14.2	Coin Piles	CSES	17.4	Dia Siguiente Y Anos Bisiestos	Notebook	13.17	
AROD	RPC	4.15	Collatz Polynomial	Day 6	13.4	Digit DP	Notebook	2.6	
Artic Virus	ICPC 2024	2.10	Colmillos	Codeforces	3.35	Digit Frequency Score	Hub	20.13	
Artwork	Day 1	3.18	Combinatoria	Formulario	14.3	Digit Queries	CSES	17.6	
Attacking Rooks	Day 7	3.9	Combinatoria Avanzada	Notebook	7.2	Digitsduel	Codeforces	8.3	
Attention To Meeting	Day 3	5.2	Compresion De Coordenadas	Notebook	1.2	Discrete Catalog	ICPC 2025	7.13	
Axles	RPC	10.7	Concatenate Non Zero Digits And Mu	Hub	40.2	Distancia De Damerau Levenshtein	Notebook	2.7	
B			Concatenation Of Array	Hub	20.6	Distinct Colors	Day 4	1.20	
	Bacon Number	Day 3	3.5	Construct Binary Tree From Inorder	Hub	22.1	Distinct Subsequences	CSES	19.3
	Balanced Balloons	ICPC	2.9	Construct Target Array With Multip	Hub	30.1	Distinct Subsequences I	Hub	25.2
Balatro	Codeforces	10.19	Construct Uniform Parity Array I	Hub	20.7	Distinct Subsequences Ii	Hub	25.3	
Beautiful Array	Hub	20.2	Construct Uniform Parity Array Ii	Hub	20.8	Distinct Substrings	CSES	19.4	
Beautiful Words	Day 4	9.15	Container With Most Water	Notebook	10.22	Distinct Value Queries II	CSES	18.1	
Binary Dozens	ICPC 2025	6.16	Contains Duplicate	Hub	20.9	Distinct Values Queries	CSES	18.2	
Binary Dozens	ICPC 2025	6.17	Continuous Subarray Sum	Hub	34.1	Distribute Candies Among Children	Hub	24.2	
Binary Search	Hub	21.1	Convex Hull Trick	Notebook	2.5	Divide And Conquer Optimization	Notebook	2.8	
Binary Tree	LeetCode	1.23	Coordenadas Polares	Notebook	4.6	Division Digitos Distintos	CCPL	13.19	
Binomial Coefficients	Day 10	7.1	Copy List With Random Pointer	Hub	31.2	Divisor Game	Hub	26.1	
Bit Strings	CSES	17.2	Cotas Y Limites	Formulario	14.4	Doubled Sequence II	Day 9	12.2	
Black Hills Golden Jewels	Day 5	11.3	Count Elements With At Least K Gre	Hub	21.2	Download Speed	Day 7	3.28	
Block Placement Queries	LeetCode	1.16	Count Majority Subarrays	LeetCode	1.1	Drug Test	ICPC 2024	5.1	
Boss	Day 2	3.4	Count Majority Subarrays2	LeetCode	1.14	Dynamic Range Minimum Queries	CSES	18.3	
Brace Expansion II	Hub	40.1	Count Subarrays With Majority Elem	Hub	20.10	Dynamic Range Sum Queries	CSES	18.4	
Bridge Juan	ICPC 2024	10.15	Count The Number Of Computer Unloc	Hub	24.1	E			
Broul	CCPL	6.13	Counting Divisors	Day 10	6.18		Earliest Finish Time For Land And	Hub	20.14
Brute	CCPL	3.31	Counting Patterns	CSES	19.2		Earliest Finish Time For Land And	Hub	20.15
Buffoon	Day 1	13.6	Counting Rooms	CSES	16.3	Eclipse	RPC	4.3	
Build An Array With Stack Operatio	Hub	39.1	Counts Commas In Range I	Hub	32.1	Eeny Meeny	Day 11	13.10	
Building Roads	CSES	16.1	Counts Commas In Range Ii	Hub	32.2	Efficient Encoding	ICPC 2025	10.9	
Building Teams	CSES	16.2	Counts Nodex Equal To Average Of S	Hub	22.2	El Juego Del Calamar	Day 9	2.3	

Indice alfabetico (cont.)

Eliminacion Gaussiana	Notebook	5.8	Frosh Merge Sort	CCPL	15.1	Journeygrapes	Codeforces	6.12
Emergency Rations	ICPC	1.19	Fuzzy Factorization	ICPC	6.10	João João	Day 6	13.12
Enigma	Day 8	9.1	G			Juan Vs Frank	Day 9	8.2
Enigma Of The Jewelry Case	Day 3	13.26		CSES	16.8	Judgmental Crowd	ICPC	9.3
Escalator	Day 4	13.20		Day 8	3.30	Jump Game Iii	Hub	28.12
Evaluate Division	Hub	28.7		Day 9	5.14	Just Palindromes	ICPC 2025	9.7
Evaluate Reverse Polish Notation	Hub	39.2		Formulario	14.6	K		
Evaluate The Bracket Pairs Of A St	Hub	40.6		Day 4	5.10		Day 4	1.6
Exclusive Time Of Functions	Hub	39.3		CSES	17.7		Day 3	2.16
Exhibition Of Clownfish	Day 1	10.6		CSES	17.8		Day 4	10.17
Expansion Of The Road Network	Day 6	3.36		CSES	17.9		Hub	20.24
Exponentiation II	Day 10	6.9		Day 7	7.14		Day 9	4.7
Ezraft And Array	Codeforces	5.6		ICPC 2025	4.5		ICPC	13.24
F							CSES	17.11
	Notebook	6.11	H	Day 4	12.1		Day 7	3.10
	Day 9	3.21		ICPC	13.14		Hub	20.25
	Codeforces	7.3		Day 3	13.7	L		
	Day 10	5.9		LeetCode	2.4		CSES	16.11
	Hub	20.16		Notebook	1.8		LeetCode	3.3
	Hub	28.8		Day 4	3.8		Hub	39.4
	Hub	20.17		CSES	16.9		Hub	20.26
	Hub	28.9		Hub	28.10		Notebook	2.12
	Hub	30.2		Day 4	1.22		Day 3	10.2
	LeetCode	3.6		ICPC 2025	3.13		Notebook	13.21
Find Minimum In Rotated Sorted Arr	Hub	21.3	Holy Network	ICPC 2025	3.14	Left And Righ Sum Differences	Hub	20.27
Find Missing Elements	Hub	20.18	Holy Network (clique Iterativo)	ICPC 2025	3.23	Length Of Last Word	Hub	40.9
Find Right Interval	Hub	20.19	Holy Network (DSU, Incorrecta)	ICPC 2025	3.23	Length Of Longest Subarray With At	Hub	37.3
Find The Highest Altitude	Hub	20.20	Hotel Queries	CSES	18.7	Less Coint Tosses	Day 1	7.8
Find The Index Of The First Occurr	Hub	40.7	Hour For A Run	Day 1	5.13	License Key Formatting	Hub	40.10
Find The Largest Almost Missing In	Hub	37.1	How Many Teams	Day 6	2.14	Lines Of Containers	Day 2	7.9
Find The Length Of The Longest Com	Hub	40.8	Hoy Many Numbers Are Smaller Than	Hub	20.23	Linked List Cycle	Hub	31.4
Find The Prefix Common Array Of Tw	Hub	20.21	Huffman	Notebook	10.8	List Removals	CSES	18.9
Find The Winning Player In Coin Ga	Hub	26.2	I			LLMs	Day 6	13.9
Find Two Non Overlapping Subarrays	Hub	37.2		Hub	33.1	Lonely Creatures	ICPC	4.4
Find X Value Of Array I	Hub	25.4		Hub	35.2	Longest Common Prefix	Hub	40.11
Find X Value Of Array Ii	Hub	36.1		Hub	41.2	Longest Flight Route	CSES	16.12
Finding Borders	CSES	19.5		ICPC 2025	13.25	Longest Increasing Path In A Matri	Hub	28.13
Finding Forks	Day 11	10.20		Day 10	10.12	Longest Palindrome	CSES	19.9
Finding Patterns	CSES	19.6		CSES	17.10	Longest Substring Without Repeatin	Hub	37.4
Finding Periods	CSES	19.7		CSES	18.8	Los Ratones III	Day 9	3.25
Fingerprints	ICPC 2025	9.12		ICPC	3.7	Lottery	RPC	13.28
First Missing Positive	Hub	20.22		Day 3	6.5	Lumina	ICPC 2024	3.16
Flight Discount	CSES	16.6	Insertion Sort List	Hub	31.3	Lumina Juan	ICPC 2024	3.15
Flight Routes	CSES	16.7	Interval Scheduling Greedy	Notebook	10.10	M		
Folding Machine	Day 2	3.33	Inverse Suffix Array	CSES	19.8		Hub	20.28
Forest Queries	CSES	18.5	Inversions	Codeforces	7.5		Day 5	3.12
Forest Queries II	CSES	18.6	Investigating Quadradômeda	Day 6	5.22		Hub	20.29
Fractions Are Better When Continue	Day 3	5.21	Investigation	CSES	16.10		Hub	34.2
Frangolino Ali Na Mesa	Day 6	7.11	Is Graph Bipartite	Hub	28.11		Day 1	10.1
Fraud	RPC	13.23	Island Perimeter	Hub	33.2		Day 8	8.4
Frequent	CCPL	1.21	J	Day 8	2.15		Day 7	3.29
Frosh	CCPL	1.3		Codeforces	13.11		Hub	40.12

Indice alfabetico (cont.)

Max Consecutive Ones	Hub	20.30	Nomad	Day 10	10.5	Range Update Queries	CSES	18.17
Max Palindromes	LeetCode	9.8	Number Of Provinces	Hub	28.18	Range Updates And Sums	CSES	18.18
Maximizing Advertising	Day 7	10.13	Number Of Sets	LeetCode	7.7	Range XOR Queries	CSES	18.19
Maximum Depth Of Binary Tree	Hub	22.3	Number Of Sets Of K Non Overlappin	Hub	24.3	Rank Transform Of An Array	Hub	20.42
Maximum Ice Cream Bars	Hub	20.31	Number Of Strings That Appear As S	Hub	40.19	Reconstruct Itinerary	Hub	28.19
Maximum Length Substring With Two	Hub	37.5	Number Of Students Unable To Eat L	Hub	35.3	Rectangle Overlap	Hub	27.4
Maximum Number Of Ballons	Hub	40.13	Number Of Substrings Containing Al	Hub	40.20	Rectas Y Conicas	Formulario	14.8
Maximum Number Of Coins You Can Ge	Hub	29.1	Number Spiral	CSES	17.14	Reduction Operations To Make The A	Hub	20.43
Maximum Number Of Eaten Apples	Hub	30.3	Numeros Romanos	Notebook	5.16	Redundant Connection	Hub	28.20
Maximum Number Of Non Overlapping	Hub	29.2	O			Redundant Connection Ii	Hub	28.21
Maximum Number Of Non Overlapping	Hub	40.14	Odd Even Linked List	Hub	31.6	Reformat Date	Hub	40.22
Maximum Product Of Two Elements In	Hub	20.32	Office Space	Day 11	4.1	Reminders	RPC	6.21
Maximum Repeating Substring	Hub	40.15	Oficinas A Distancia D En Arbol	Notebook	10.11	Remove Covered Intervals	Hub	20.44
Maximum Score Of Non Overlapping I	Hub	25.5	Omens	ICPC 2024	4.11	Remove Duplicate Letters	Hub	39.6
Maximum Substrings With Distinct S	Hub	40.16	Only1s0s	ICPC 2024	6.1	Remove Duplicates	LeetCode	9.11
Maximum Total Subarray Value I	Hub	20.33	Ordered Set (PBDS)	Notebook	1.12	Remove Duplicates From Sorted Arra	Hub	20.45
Means	Notebook	4.9	P			Remove Duplicates From Sorted List	Hub	31.8
Median Of Two Sorted Arrays	Hub	20.34	Palindrome Linked List	Hub	31.7	Remove Duplicates1	LeetCode	9.6
Meme Wars	Day 7	5.4	Palindrome Number	Hub	32.3	Remove Element	Hub	20.46
Merge Intervals	Hub	20.35	Palindrome Queries	CSES	19.11	Remove Methods From Project	Hub	28.22
Merge Sorted Array	Hub	38.1	Palindrome Reorder	CSES	17.15	Repeated String Match	Hub	40.23
Merge Two Sorted Lists	Hub	31.5	Palindromic Lexi Small	LeetCode	7.12	Repeated Substring Pattern	Hub	40.24
Message Route	CSES	16.13	Parentesis Balanceados	Notebook	9.9	Repeating Substring	CSES	19.13
Metal Detector	Day 5	5.23	Pascal's Triangle	Hub	20.40	Repetitions	CSES	17.18
Mex Grid Construction	CSES	17.12	Patches	Day 2	2.1	Required Substring	CSES	19.14
Minas Gerais' Walls	Day 6	11.2	Pattern Positions	CSES	19.12	Revenge Of The Salary Of At Coder	Day 10	2.17
Minimal Rotation	CSES	19.10	Peak Index In A Mountain Array	Hub	21.4	Reverse Degree Of A String	Hub	40.25
Minimum Absolute Difference	Hub	20.36	Permutation Rounds	Day 10	6.20	Reverse Linked List	Hub	31.9
Minimum Cost Of Buying Candies Wit	Hub	20.37	Permutations	CSES	17.16	Reverse Pairs	Hub	38.2
Minimum Cuts To Divide A Circle	Hub	27.3	Pizza Cutter	Day 8	7.6	Reverse Substrings Between Each Pa	Hub	40.26
Minimum Element After Replacement	Hub	20.38	Pizzeria Queries	CSES	18.12	Rock Paper Scissors Tournament	Day 11	13.2
Minimum Height Trees	Hub	28.14	Planetqueriesi	CSES	16.15	Roman To Integer	Hub	32.4
Minimum Moves To Clean The Classro	Hub	28.15	Planets And Kingdoms	Day 5	3.17	Rotate String	Hub	40.27
Minimum Number Of Flips To Reverse	Hub	23.1	Plus One	Hub	20.41	Rotating Calipers	Notebook	4.13
Minimum Number Of Pushes To Type W	Hub	40.17	Policianacional	Codeforces	9.10	Round Trip	CSES	16.16
Minimum Number Of Pushes To Type W	Hub	40.18	Polynomial Queries	CSES	18.13	Round Trip (Stack)	CSES	16.17
Minimum Operations To Reduce X To	Hub	34.3	Posters	RPC	10.3	Round Trip II	CSES	16.18
Misere Nim Y Grundy Acotado	Notebook	8.1	Pou	Codeforces	5.11	Running A Penitentiary	Day 7	1.18
Missing Coin Sum Queries	CSES	18.10	Prefix Sum Queries	CSES	18.14	S		
Missing Number	CSES	17.13	Prime Sieve	Day 11	6.22	Salary Queries	CSES	18.20
Missing Number	Hub	20.39	Primitivas Y Convex Hull	Notebook	4.10	Satoru Gojo	Codeforces	5.17
Modulo De Entero Gigante	Notebook	6.15	Probabilidad Geometrica	Formulario	14.7	School Dance	Day 5	3.11
Monarchy In Vertigo	Day 4	1.17	Process Str	LeetCode	9.13	Search In Rotated Sorted Array	Hub	21.5
Monsters	CSES	16.14	Process String With Special Operat	Hub	40.21	Search Insert Position	Hub	20.47
Mount Rushmore And Birthdays	Day 5	7.10	Proving Equivalences	Day 4	3.34	Secuencias Notables	Formulario	14.9
Movie Festival Queries	CSES	18.11	Punto En Poligono Convexo	Notebook	4.12	Set Mismatch	Hub	20.48
Music Festival	Day 8	2.2	Puntos Equiespaciados En Un Circul	Notebook	5.7	Shoelace	Notebook	4.14
N			Pythagorean	RPC	6.23	Shortest And Lexicographically Sma	Hub	37.6
Network Delay Time	Hub	28.16	R			Shortest Routes I	CSES	16.19
Network Recovery Pathways	Hub	28.17	Rab Game I	CSES	17.17	Shortest Routes II	CSES	16.20
Next Greater Node In Linked List	Hub	39.5	Range Interval Queries	CSES	18.15	Shortest Routes II (Floyd Warshall	CSES	16.21
Nim Game	Hub	26.3	Range Queries And Copies	CSES	18.16	Shuffle The Array	Hub	20.49

Indice alfabetico (cont.)

Sign	CCPL	9.5	Trampas De C++	Formulario	14.14
Signal Coverage	ICPC 2024	3.1	Triangulos	Day 2	5.20
Single Number	Hub	20.50	Triangulos En Poligono Convexo	Notebook	7.16
Sky Line	ICPC 2024	1.4	Trigonometria	Formulario	14.15
Sliding Window	Notebook	10.18	Trucks	Day 2	3.26
Smallest Divisible Digit Product I	Hub	32.5	Trucos De Bits	Formulario	14.16
Smallest Index With Digit Sum Equa	Hub	20.51	Turnswitch	ICPC 2024	13.22
Smallest Missing Integer Greater T	Hub	20.52	Two Knights	CSES	17.22
Smallest Missing Multiple Of K	Hub	20.53	Two Sets	CSES	17.23
Smallest Palindromic Rearrangement	Hub	40.28	Two Sum	Hub	20.57
Smallest Stable Index I	Hub	20.54	U		
Smallest Stable Index Ii	Hub	20.55	Uj	CCPL	6.2
Song Of The Sirens	Codeforces	9.4	Uj	CCPL	6.3
Sort An Array	Hub	38.3	Unique 3 Digit Event Numbers	Hub	24.4
Spacebar Tokenizer	ICPC 2024	9.16	Unraveling Monty Hall	Day 8	13.5
Sqrt(x)	Hub	32.6	V		
Square Diff	ICPC 2024	6.6	Valid Anagram	Hub	40.29
Static Range Minimum Queries	CSES	18.21	Valid Mountain Array	Hub	20.58
Static Range Sum Queries	CSES	18.22	Valid Palindrome	Hub	40.30
STL Indispensable	Formulario	14.10	Valid Parentheses	Hub	39.7
Stone Game	Hub	29.3	Visible Building Queries	CSES	18.25
String Functions	CSES	19.15	W		
String Matching	CSES	19.16	Ways To Make A Fair Array	Hub	34.4
String Reorder	CSES	17.19	Weird Algorithm	CSES	17.24
String Transformation	CSES	19.17	Wine Production	Day 7	1.11
Sub Array Sum Queries	CSES	18.23	Word Combinations	CSES	19.21
Sub Array Sum Queries II	CSES	18.24	Y		
Submerging Islands	Day 4	3.2	Yuyuan Market	Day 10	2.11
Substring Distribution	CSES	19.18	Z		
Substring Order I	CSES	19.19	Zero Or One	Day 2	13.3
Substring Order II	CSES	19.20	Zig Zag Array	LeetCode	2.13
Subway Lines	Day 8	3.27			
Sum Of Distances In Tree	Hub	28.23			
Sum Of Square Numbers	Hub	32.7			
Sumas Y Series	Formulario	14.11			
Summary Ranges	Hub	20.56			
Switches	Day 8	13.15			
T					
Tabla ASCII	Formulario	14.12			
Template Base	Notebook	13.27			
Teoria De Numeros	Formulario	14.13			
The Bridge At Night	ICPC 2024	10.14			
The Day Of The Competitors	Day 4	1.7			
The Declaration Of Independence	Day 5	1.13			
The Seals Of Shanghai	Day 10	6.19			
The Wisdom Of Master Wei	Day 10	5.3			
Time Needed To Buy Tickets	Hub	35.4			
Toilet Seat	Day 11	13.1			
Total Numbers	LeetCode	7.4			
Total Waviness Of Numbers In Range	Hub	32.8			
Tower Of Hanoi	CSES	17.20			
Trailing Zeros	CSES	17.21			